ZYNQ MPSoC 开发平台基础教程

2021-11-10 18:07:01

ALINX

版权声明

Copyright ©2012-2021 芯驿电子科技（上海）有限公司

公司网址:
Http://www.alinx.com

官方旗舰店:
http://alinx.jd.com

邮箱:
avic@alinx.com

电话:
021-67676997

传真:
021-37737073

ALINX 微信公众号:



文档修订记录:

版本	时间	描述
1.01	2021/10/19	初始版本

我们承诺本教程并非一劳永逸，固守不变的文档。我们会根据大家的反馈意见，以及实际的开发实践经验积累不断的修正和优化教程。

前言

首先感谢大家购买芯驿电子科技（上海）有限公司出品的 Ultrascale+ MPSoC 的开发板！您对我们和我们产品的支持和信任,给我们增添了勇往直前的信心和勇气。

“播下一粒种子，收获一片森林”，更是芯驿电子科技（上海）有限公司的美好愿望，同时我们会在黑金动力社区 *http://www.heijin.org* 和大家一起讨论，一起学习，一起进步，一起成长。

通过此教程我们了解传统 FPGA 开发的基本流程，并学习 Vitis 下 Ultrascale+ MPSoC 程序的开发，芯驿电子科技（上海）有限公司只能抛砖引玉，把最常用的知识点收集分享给大家。

目录

版权声明 2
前言 4
目录 5
准备工作及注意事项 11
软件环境 11
硬件环境 11
批处理下载 QSPI Flash 11
批处理建立 Vitis 工程 12
FPGA 篇 14
第一章 Ultrascale+ MPSoC 介绍 15
1.1 PS 和 PL 互联技术 16
1.2 ZYNQ 芯片开发流程的简介 21
1.3 学习 ZYNQ 要具备哪些技能 22
1.3.1 软件开发人员 22
1.3.2 逻辑开发人员 23
第二章 PL 的"Hello World"LED 实验 23
2.1 LED 硬件介绍 23
2.2 创建 Vivado 工程 24
2.3 创建 Verilog HDL 文件点亮 LED 29
2.4 添加管脚约束 34
2.5 添加时序约束 37
2.6 生成 BIT 文件 41
2.7 Vivado 仿真 42
2.8 下载 47
2.9 在线调试 50
2.9.1 添加 ILA IP 核 50
2.9.2 MARK DEBUG 54
2.10 实验总结 57
第三章 PL 端 DDR4 读写测试实验 58
3.1 硬件介绍 58
3.2 Vivado 工程建立 58
3.2.1 创建一个 PL 端 ddr4 测试工程并配置 ddr4 IP 58
3.2.2 添加其他测试代码 61
3.3 下载调试 62
3.4 实验总结 63
第四章 GTY 收发器误码率测试 IBERT 实验 64

4.1 硬件介绍 64
4.2 Vivado 工程建立 64
4.3 编译下载调试 67
4.4 实验总结 70
PS 端外设篇 71
第五章 体验 ARM，裸机输出 “Hello World” 72
5.1 硬件介绍 72
FPGA 工程师工作内容 72
5.2 Vivado 工程建立 72
1.1.1 Low Speed 配置 75
1.1.2 High Speed 配置 77
1.1.3 时钟配置 79
5.2.1 DDR 配置 81
软件工程师工作内容 87
5.3 Vitis 调试 87
5.3.1 创建 Application 工程 87
5.4 固化程序 100
5.4.1 生成 FSBL 101
5.4.2 SD 卡启动测试 105
5.4.3 QSPI 启动测试 106
5.4.4 Vivado 下烧写 QSPI 108
5.4.5 使用批处理文件快速烧写 QSPI 110
5.5 常见问题 111
5.5.1 仅有 PL 端逻辑的固化 111
5.6 使用技巧分享 113
5.7 本章小结 114
第六章 PS 端 RTC 中断实验 115
6.1 RTC 介绍 115
6.2 中断介绍 116
软件工程师工作内容 122
6.3 Vitis 程序编写 122
6.3.1 创建 Platform 工程 122
6.4 下载调试 129
6.5 实验总结 130
第七章 PS 端 MIO 的使用 131
软件工程师工作内容 131
7.1 原理介绍 131
7.2 Vivado 工程 132

7.3 Vitis 程序开发 133
7.3.1 MIO 点亮 PS 端 LED 灯 133
7.3.2 MIO 按键中断 140
7.4 知识点分享 143
7.5 本章小结 146
第八章 PS 端 UART 读写控制 147
软件工程师工作内容 147
8.1 UART 模块介绍 147
8.2 Vitis 程序开发 148
8.3 板上验证 151
8.4 总结 152
第九章 PS 端 CAN 的使用 153
软件工程师工作内容 153
9.1 Vitis 程序开发 153
9.2 下载测试 154
第十章 PS 端 I2C 的使用 159
软件工程师工作内容 159
10.1 Vitis 程序开发 159
10.1.1 温度传感器测试 159
10.1.2 EEPROM 读写 161
第十一章 PS 端 DiplayPort 的使用 164
软件工程师工作内容 164
11.1 接口介绍 164
11.2 Example 工程介绍 165
11.3 板上验证 166
第十二章 PS 端 SD 卡读写 169
12.1 FatFs 简介 169
12.2 Vitis 程序开发 170
12.3 板上验证 173
第十三章 PS 端以太网使用之 lwip 175
软件工程师工作内容 175
13.1 Vitis 程序开发 175
13.1.1 LWIP 库修改 175
13.1.2 创建 APP 工程时基于 LWIP 模板 180
13.2 下载调试 181
13.2.1 以太网测试 181
13.3 实验总结 183
第十四章 PS 端之网络远程更新 QSPI 184

软件工程师工作内容 184
14.1 Vitis 程序开发 184
14.1.1 UDP 传输方式 184
14.1.2 TCP 传输方式 187
14.1.3 QSPI Flash 读写控制 187
14.2 板上验证 188
14.2.1 UDP 方式 189
14.2.2 TCP 方式 191
第十五章 System Monitor 的使用 193
FPGA 工程师工作内容 193
15.1 Hardware 读取 System Monitor 193
软件工程师工作内容 196
15.2 PS 读取 System Monitor 信息 196
PS 与 PL 互联篇 198
第十六章 PS 端 EMIO 的使用 199
16.1 原理介绍 199
FPGA 工程师工作内容 199
16.2 Vivado 工程建立 200
16.3 XDC 文件约束 PL 管脚 201
软件工程师工作内容 203
16.4 Vitis 程序编写 203
16.4.1 EMIO 点亮 PL 端 LED 灯 203
16.4.2 EMIO 实现 PL 端按键中断 204
16.5 固化程序 205
16.6 UART 串口的 EMIO 用法 206
16.7 引脚绑定常见错误 208
16.8 本章小节 209
第十七章 PL 端 AXI GPIO 的使用 210
17.1 原理介绍 210
FPGA 工程师工作内容 210
17.2 Vivado 工程建立 211
17.2.1 添加 AXI GPIO 211
17.3 XDC 文件约束 PL 管脚 218
软件工程师工作内容 220
17.4 Vitis 程序编写 220
17.4.1 AXI GPIO 点亮 PL 端 LED 灯 220
17.4.2 下载调试 223
17.4.3 寄存器方式实现 223

17.4.4 AXI GPIO 之 PL 端按键中断 224
17.4.5 下载调试 226
17.5 实验总结 227
17.6 知识点分享 227
第十八章 PL 端 RS485 测试 230
FPGA 工程师工作内容 230
18.1 硬件环境搭建 230
软件工程师工作内容 235
18.2 Vitis 程序开发 235
18.3 板上验证 237
18.4 本章小结 238
第十九章 PL 端以太网实验 239
FPGA 工程师工作内容 239
19.1 硬件环境搭建 239
软件工程师工作内容 245
19.2 Vitis 程序开发 246
19.2.1 PL 端以太网测试 246
19.2.2 PS 端以太网测试 246
第二十章 自定义 IP 实验 248
FPGA 工程师工作内容 248
20.1 PWM 介绍 248
20.2 Vivado 工程建立 249
20.2.1 创建自定义 IP 249
20.2.2 添加自定义 IP 到工程 257
软件工程师工作内容 259
20.3 Vitis 软件编写调试 259
20.4 实验总结 263
第二十一章 双核 AMP 的使用 264
软件工程师工作内容 264
21.1 Vitis 程序开发 264
21.1.1 CPU0 Vitis 工程建立 264
21.1.2 CPU1 Vitis 工程建立 266
21.1.3 CPU0 程序介绍 267
21.1.4 CPU1 程序介绍 268
21.2 板上验证 268
21.3 QSPI Flash 启动 269
21.4 本章小结 271
第二十二章 ZYNQ 下使用 Free RTOS 272

软件工程师工作内容 272
22.1 Vitis 程序开发 272
22.2 板上验证 273
22.3 本章小结 274
第二十三章 PL 读写 PS 端 DDR 数据 275
FPGA 工程师工作内容 275
23.1 ZYNQ 的 HP 端口使用 275
23.2 硬件环境搭建 275
23.3 PL 端 AXI Master 280
23.4 ddr 读写数据的检验 282
23.5 Vivado 软件的调试技巧 282
23.6 Vitis 工程开发 283
23.7 本章小结 284
第二十四章 通过 BRAM 实现 PS 与 PL 数据交互 285
FPGA 工程师工作内容 285
24.1 硬件环境搭建 285
24.1.1 Block Design 添加逻辑分析仪方法 289
软件工程师工作内容 290
24.2 Vitis 程序开发 290
24.3 实验现象 291
1.1 本章小结 294
第二十五章 基于 AN5641 模块的 MIPI 采集 DP 显示 295
25.1 原理介绍 295
25.1.1 MIPI 物理层(D-PHY) 295
25.1.2 MIPI 协议层(CSI-2) 296
FPGA 工程师工作内容 297
25.2 硬件环境搭建 297
软件工程师工作内容 305
25.3 Vitis 程序开发 305
25.4 板上验证 306
第二十六章 PCIe ROOT 309
26.1 硬件环境搭建 309

准备工作及注意事项

软件环境

软件开发环境基于 Vivado 2020.1

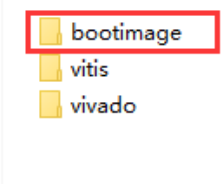


硬件环境

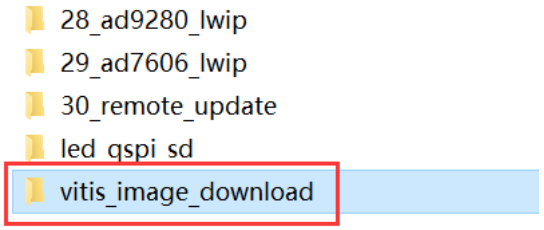
开发板型号	芯片型号
Z19	Xczu19eg-ffvc1760-2-i

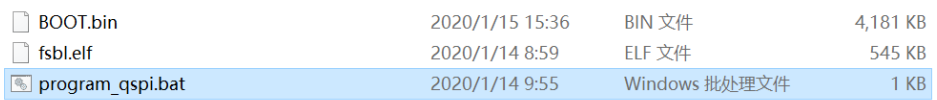
批处理下载 QSPI Flash

所有的工程目录下都有个 bootimage 文件夹，存放了对应的 BOOT.bin 文件，可将此文件拷贝到 Vitis_image_download 文件夹，覆盖原有的 BOOT.bin。也可以把 BOOT.bin 放到 SD 卡启动验证功能



vitis_image_download 文件夹在 course_s2 目录下面，进入文件夹，右键点击 program_qspi.bat，打开编辑

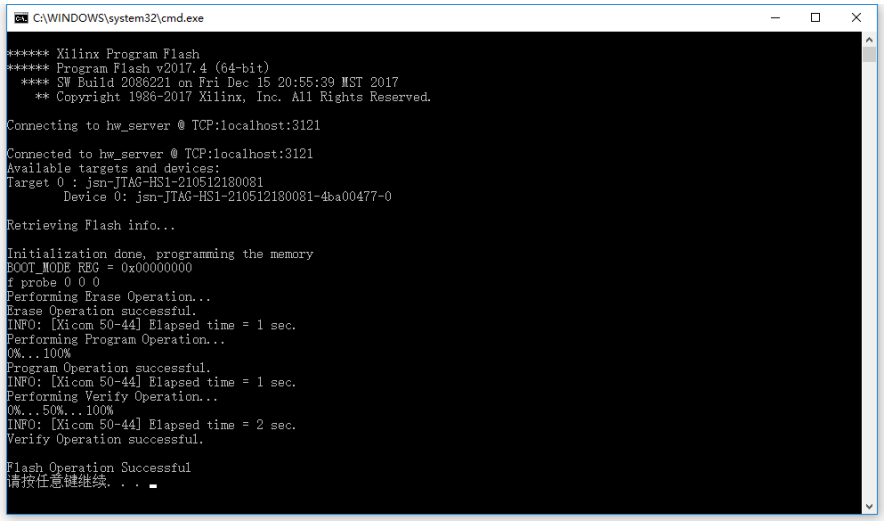




将 program_flash 路径改成自己的软件安装路径，保存并关闭。

```
call E:\XilinxVitis\Vitis\2020.1\bin\program_flash -f BOOT.bin  -offset 0 -flash_type qspi-x4-single  -fsbl fsbl.elf -verify
pause
```

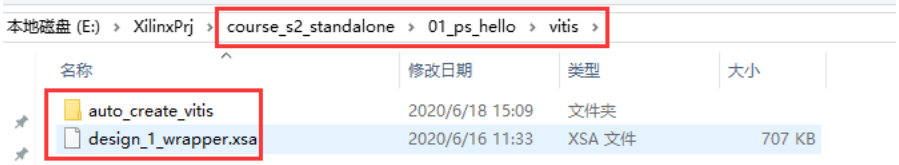
双击 program_qspi.bat，即可下载 BOOT.BIN 到 QSPI FLASH，建议用 JTAG 模式下载。



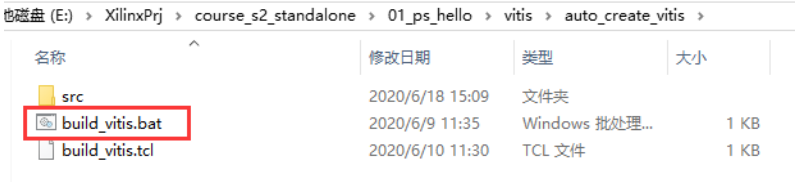
也可以采用 SD 卡启动方法，把 BOOT.bin 文件拷贝到 SD 内启动。

批处理建立 Vitis 工程

由于 Vitis 工程编译后占用空间较大，因此为了节省大家宝贵的时间，我们提供了 Vitis 工程的批处理 tcl 脚本，在每个工程下都有个 vitis 文件夹，里面包含硬件描述文件 xx.xsa，以及自动创建工程的脚本



大家需要做的是编辑 auto_create_vitis 文件夹中的 build_vitis.bat 文件



将黄色框中的 xsct.bat 路径换成自己安装的路径，路径为 xx\Vitis\2020.1\bin\xsct.bat

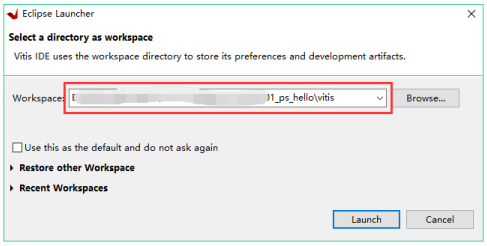
```
call E:\XilinxVitis\Vitis\2020.1\bin\xsct.bat build_vitis.tcl
pause
```

保存之后，再双击 build_vitis.bat 就可以创建工程了

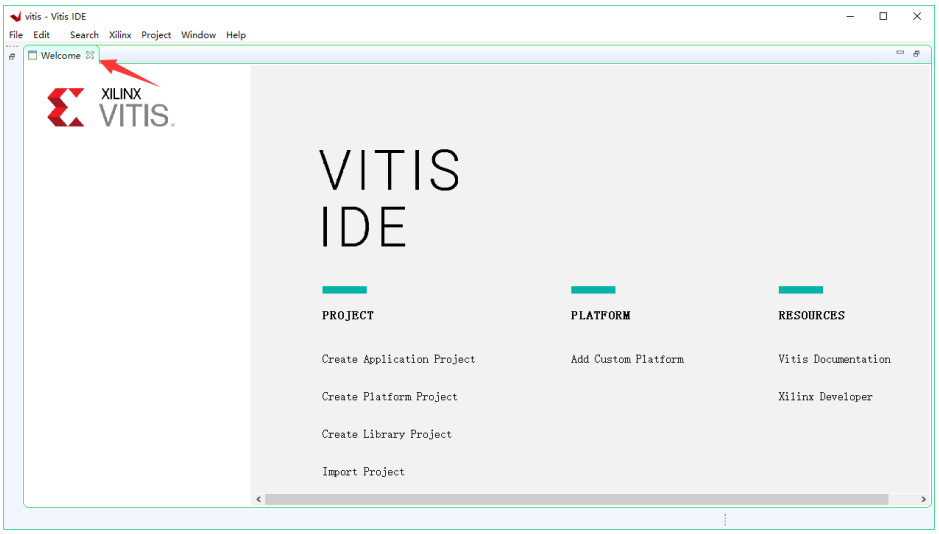
编译结束，按任意键退出



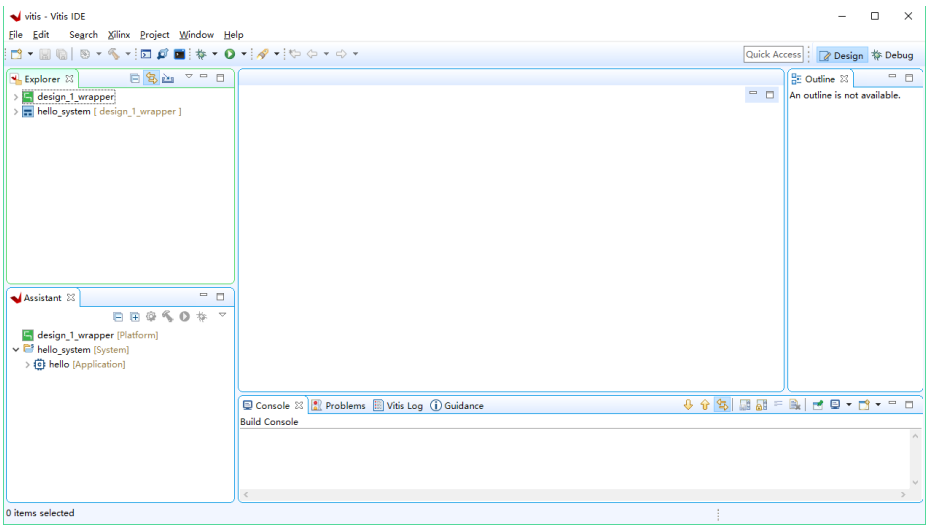
打开 Vitis 软件，选择工程路径，Launch



打开后，关闭 Welcom 界面



工程即可使用



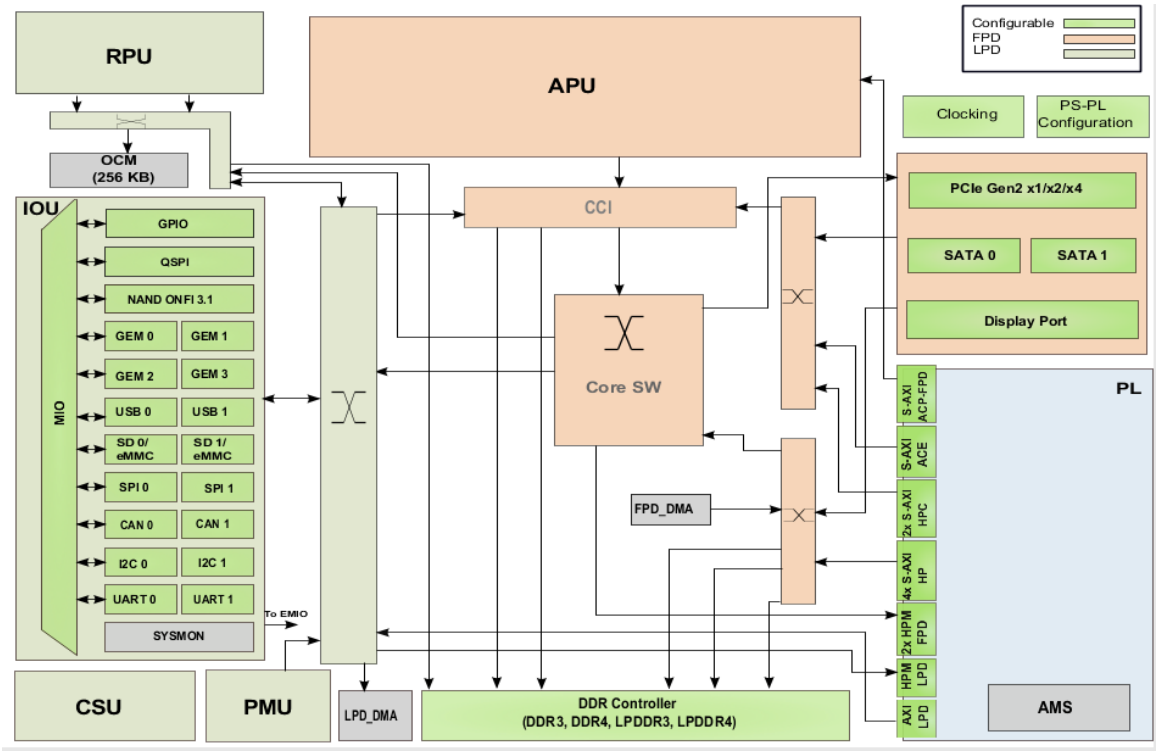
FPGA 篇

FPGA 篇主要介绍 ZYNQ 架构，纯逻辑的工程创建、调试等基础实验，熟悉 vivado 使用方法。

第一章 Ultrascale+ MPSoC 介绍

Zynq UltraScale+ MPSoC 系列是 Xilinx 第二代 Zynq 平台。其亮点在于 FPGA 里包含了完整的 ARM 处理子系统（PS），包含了四核 Cortex-A53 处理器或双核 Cortex-A53 加双核 Cortex-R5 处理器，整个处理器的搭建都以处理器为中心， 而且处理器子系统中集成了内存控制器和大量的外设， 使处理器核在 Zynq 中完全独立于可编程逻辑单元，也就是说如果暂时没有用到可编程逻辑单元部分（PL）， ARM 处理器的子系统也可以独立工作，这与以前的 FPGA 有本质区别，其是以处理器为中心的。

Zynq 就是两大功能块，PS 部分和 PL 部分， 说白了，就是 ARM 的 SOC 部分，和 FPGA 部分。其中，PS 集成了 APU ARM Cortex™-A53 处理器，RPU Cortex-R5 处理器，AMBA®互连，内部存储器（OCM），外部存储器接口（DDR Controller）和外设（IOU）。这些外设（IOU）主要包括 USB 总线接口，以太网接口，SD/eMMC 接口，I2C 总线接口，CAN 总线接口，UART 接口，GPIO 等。高速接口如 PCIe，SATA，Display Port。



ZYNQ MPSoC 芯片的总体框图

PS: 处理系统 （Processing System)， 就是与 FPGA 无关的 ARM 的 SoC 的部分。

PL: 可编程逻辑（Progarmmable Logic)，就是 FPGA 部分。

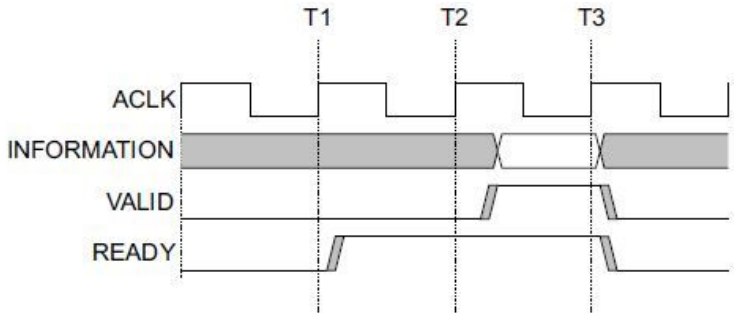
1.1 PS 和 PL 互联技术

ZYNQ 作为将高性能 ARM Cortex-A53 系列处理器与高性能 FPGA 在单芯片内紧密结合的产品，为了实现 ARM 处理器和 FPGA 之间的高速通信和数据交互，发挥 ARM 处理器和 FPGA 的性能优势，需要设计高效的片内高性能处理器与 FPGA 之间的互联通路。因此，如何设计高效的 PL 和 PS 数据交互通路是 ZYNQ 芯片设计的重中之重，也是产品设计的成败关键之一。本节，我们就将主要介绍 PS 和 PL 的连接，让用户了解 PS 和 PL 之间连接的技术。

其实，在具体设计中我们往往不需要在连接这个地方做太多工作，我们加入 IP 核以后，系统会自动使用 AXI 接口将我们的 IP 核与处理器连接起来，我们只需要再做一点补充就可以了。

AXI 全称 Advanced eXtensible Interface，是 Xilinx 从 6 系列的 FPGA 开始引入的一个接口协议，主要描述了主设备和从设备之间的数据传输方式。在 ZYNQ 中继续使用，版本是 AXI4，所以我们经常会看到 AXI4.0，ZYNQ 内部设备都有 AXI 接口。其实 AXI 就是 ARM 公司提出的 AMBA（Advanced Microcontroller Bus Architecture）的一个部分，是一种高性能、高带宽、低延迟的片内总线，也用来替代以前的 AHB 和 APB 总线。第一个版本的 AXI（AXI3）包含在 2003 年发布的 AMBA3.0 中，AXI 的第二个版本 AXI（AXI4）包含在 2010 年发布的 AMBA 4.0 之中。

AXI 协议主要描述了主设备和从设备之间的数据传输方式，主设备和从设备之间通过握手信号建立连接。当从设备准备好接收数据时，会发出 READY 信号。当主设备的数据准备好时，会发出和维持 VALID 信号，表示数据有效。数据只有在 VALID 和 READY 信号都有效的时候才开始传输。当这两个信号持续保持有效，主设备会继续传输下一个数据。主设备可以撤销 VALID 信号，或者从设备撤销 READY 信号终止传输。AXI 的协议如图，T2 时，从设备的 READY 信号有效，T3 时主设备的 VILID 信号有效，数据传输开始。



AXI 握手时序图

在 ZYNQ 中，支持 AXI-Lite，AXI4 和 AXI-Stream 三种总线，通过表 5-1,我们可以看到这三中 AXI 接口的特性。

接口协议	特性	应用场合
AXI4-Lite	地址/单数据传输	低速外设或控制
AXI4	地址/突发数据传输	地址的批量传输
AXI4-Stream	仅传输数据，突发传输	数据流和媒体流传输

AXI4-Lite：

具有轻量级，结构简单的特点，适合小批量数据、简单控制场合。不支持批量传输，读写时一次只能读写一个字（32bit）。主要用于访问一些低速外设和外设的控制。

AXI4：

接口和 AXI-Lite 差不多，只是增加了一项功能就是批量传输，可以连续对一片地址进行一次性读写。也就是说具有数据读写的 burst 功能。

上面两种均采用内存映射控制方式，即 ARM 将用户自定义 IP 编入某一地址进行访问，读写时就像在读写自己的片内 RAM，编程也很方便，开发难度较低。代价就是资源占用过多，需要额外的读地址线、写地址线、读数据线、写数据线、写应答线这些信号线。

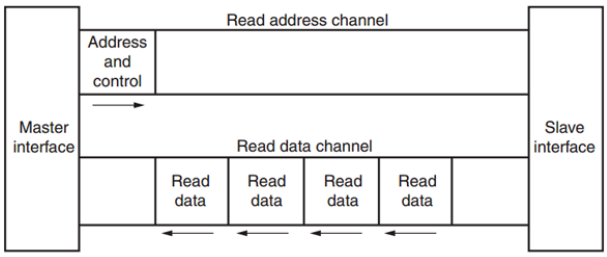
AXI4-Stream：

这是一种连续流接口，不需要地址线（很像 FIFO，一直读或一直写就行）。对于这类 IP，ARM 不能通过上面的内存映射方式控制（FIFO 根本没有地址的概念），必须有一个转换装置，例如 AXI-DMA 模块来实现内存映射到流式接口的转换。AXI-Stream 适用的场合有很多：视频流处理；通信协议转换；数字信号处理；无线通信等。其本质都是针对数值流构建的数据通路，从信源（例如 ARM 内存、DMA、无线接收前端等）到信宿（例如 HDMI 显示器、高速 AD 音频输出，等）构建起连续的数据流。这种接口适合做实时信号处理。

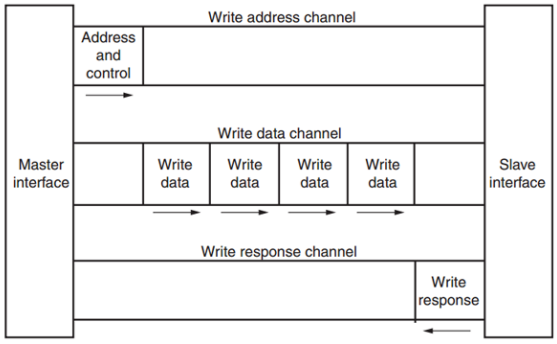
AXI4 和 AXI4-Lite 接口包含 5 个不同的通道：

- Read Address Channel
- Write Address Channel
- Read Data Channel
- Write Data Channel
- Write Response Channel

其中每个通道都是一个独立的 AXI 握手协议。下面两个图分别显示了读和写的模型：



AXI 读数据通道



AXI 写数据通道

在 ZYNQ 芯片内部用硬件实现了 AXI 总线协议，包括 12 个物理接口，分别为 S_AXI_HP{0:3}_FPD，S_AXI_LPD，S_AXI_ACE_FPD，S_AXI_ACP_FPD，S_AXI_HPC{0,1}_FPD，M_AXI_HPM{0,1}_FPD，M_AXI_HPM0_LPD 接口。

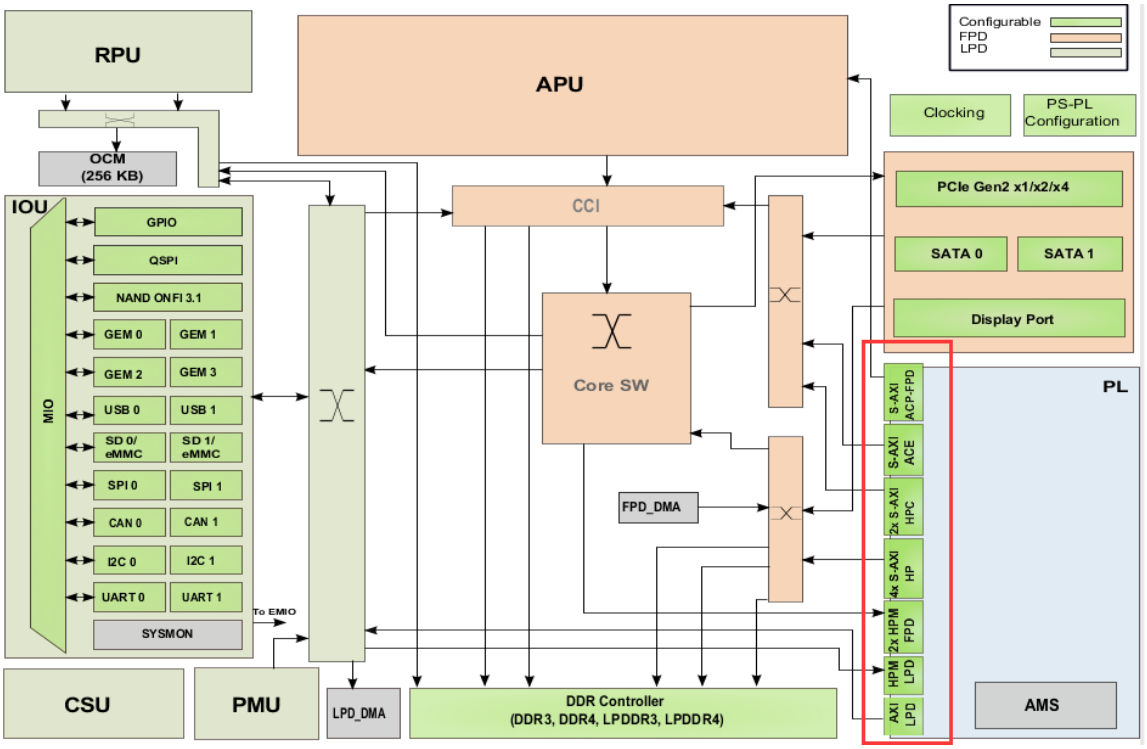
S_AXI_HP{0:3}_FPD 接口，是高性能/带宽的 AXI4 标准的接口，总共有四个，PL 模块作为主设备连接。主要用于 PL 访问 PS 上的存储器（DDR 和 FPD Main Switch）

S_AXI_LPD 接口，高性能端口，连接 PL 到 LPD。低延迟访问 OCM 和 TCM，访问 PS 端 DDR。

S_AXI_HPC{0,1}_FPD 接口，连接 PL 到 FPD，可连接到 CCI，访问 L1 和 L2 Cache，由于通过 CCI，访问 DDR 控制器会有较大延迟。

M_AXI_HPM{0,1}_FPD 接口，高性能总线，PS 为 master，连接 FPD 到 PL，可用于 CPU, DMA, PCIe 等从 PS 推送大量数据到 PL。

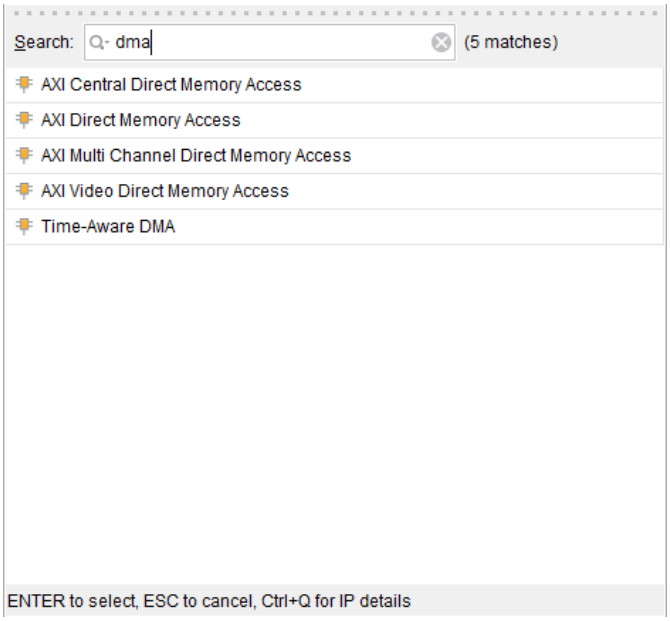
M_AXI_HPM0_LPD 接口，低延迟接口总线，PS 为 master，连接 LPD 到 PL，可直接访问 PL 端的 BRAM，DDR 等，也经常用于配置 PL 端的寄存器。



只有 M_AXI_HPM{0,1}_FPD 和 M_AXI_HPM0_LPD 是 Master Port，即主机接口，其余都是 Slave Port（从机接口）。主机接口具有发起读写的权限，ARM 可以利用两个主机接口主动访问 PL 逻辑，其实就是把 PL 映射到某个地址，读写 PL 寄存器如同在读写自己的存储器。其余从机接口就属于被动接口，接受来自 PL 的读写，逆来顺受。在 PS 与 PL 互联应用，用的最多的接口为 S_AXI_HP{0:3}_FPD、M_AXI_HPM{0,1}_FPD 和 M_AXI_HPM0_LPD。

位于 PS 端的 ARM 直接有硬件支持 AXI 接口，而 PL 则需要使用逻辑实现相应的 AXI 协议。Xilinx 在 Vivado 开发环境里提供现成 IP 如 AXI-DMA，AXI-GPIO，AXI-Dataover,

AXI-Stream 都实现了相应的接口，使用时直接从 Vivado 的 IP 列表中添加即可实现相应的功能。下图为 Vivado 下的各种 DMA IP：



下面为几个常用的 AXI 接口 IP 的功能介绍：

AXI-DMA：实现从 PS 内存到 PL 高速传输高速通道 AXI-HP<---->AXI-Stream 的转换

AXI-FIFO-MM2S：实现从 PS 内存到 PL 通用传输通道 AXI-HPM<----->AXI-Stream 的转换

AXI-Datamover：实现从 PS 内存到 PL 高速传输高速通道 AXI-HP<---->AXI-Stream 的转换，只不过这次是完全由 PL 控制的，PS 是完全被动的。

AXI-VDMA：实现从 PS 内存到 PL 高速传输高速通道 AXI-HP<---->AXI-Stream 的转换，只不过是专门针对视频、图像等二维数据的。

AXI-CDMA：这个是由 PL 完成的将数据从内存的一个位置搬移到另一个位置，无需 CPU 来插手。

关于如何使用这些 IP，我们会在后面的章节中举例讲到。有时，用户需要开发自己定义的 IP 同 PS 进行通信，这时可以利用向导生成对应的 IP。用户自定义 IP 核可以拥有 AXI4-Lite，AXI4，AXI-Stream，PLB 和 FSL 这些接口。后两种由于 ARM 这一端不支持，所以不用。

有了上面的这些官方 IP 和向导生成的自定义 IP，用户其实不需要对 AXI 时序了解太多（除非确实遇到问题），因为 Xilinx 已经将和 AXI 时序有关的细节都封装起来，用户只需要关注自己的逻辑实现即可。

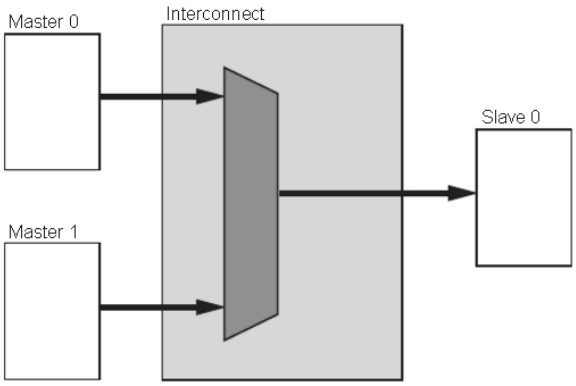
AXI 协议严格的讲是一个点对点的主从接口协议，当多个外设需要互相交互数据时，我们需要加入一个 AXI Interconnect 模块，也就是 AXI 互联矩阵，作用是提供将一个或多个 AXI 主设备连接到一个或多个 AXI 从设备的一种交换机制（有点类似于交换机里面的交换矩阵）。

这个 AXI Interconnect IP 核最多可以支持 16 个主设备、16 个从设备，如果需要更多的接口，可以多加入几个 IP 核。

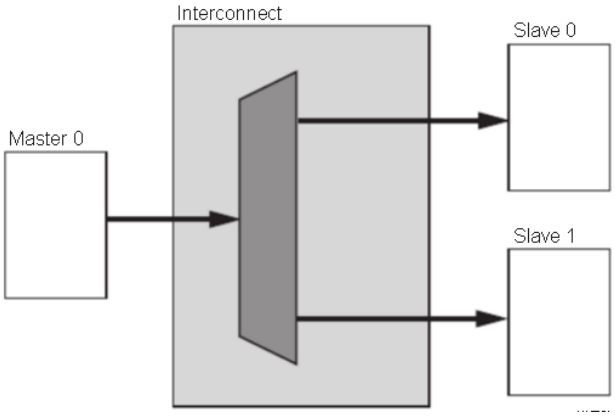
AXI Interconnect 基本连接模式有以下几种：

- N-to-1 Interconnect
- to-N Interconnect

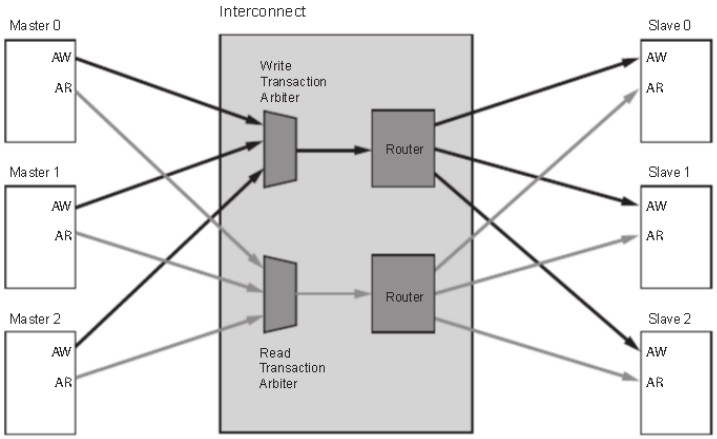
- N-to-M Interconnect (Crossbar Mode)
- N-to-M Interconnect (Shared Access Mode)



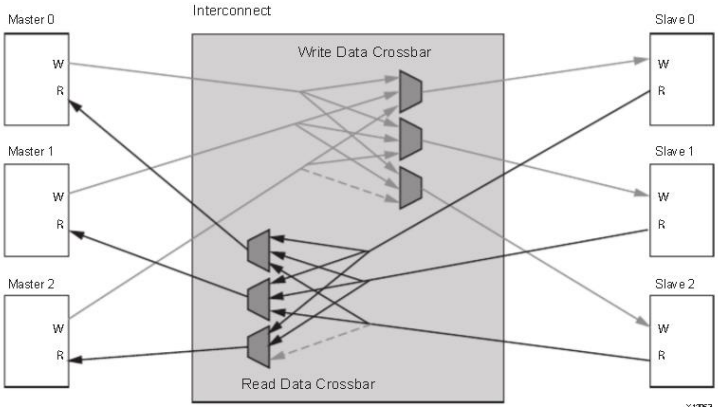
多对一的情况



一对多的情况

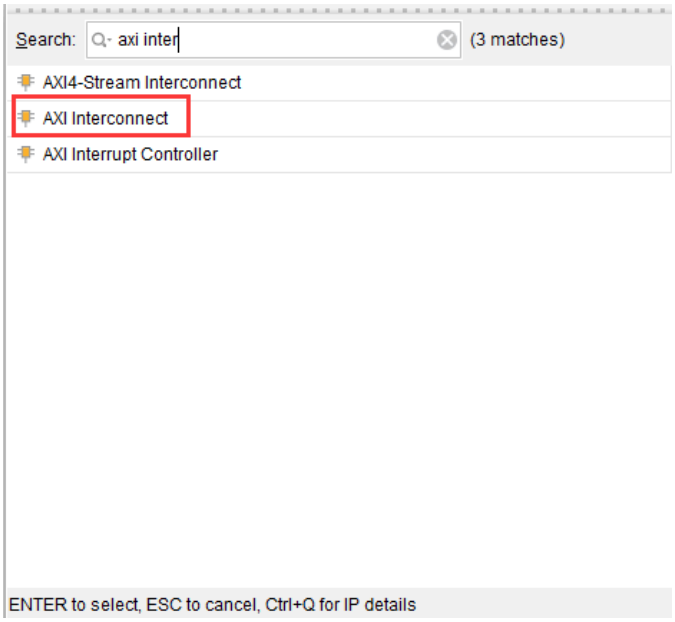


多对多读写地址通道



多对多读写数据通道

ZYNQ 内部的 AXI 接口设备就是通过互联矩阵的的方式互联起来的，既保证了传输数据的高效性，又保证了连接的灵活性。Xilinx 在 Vivado 里我们提供了实现这种互联矩阵的 IP 核 axi_interconnect，我们只要调用就可以。



AXI Interconnect IP

1.2 ZYNQ 芯片开发流程的简介

由于 ZYNQ 将 CPU 与 FPGA 集成在了一起，开发人员既需要设计 ARM 的操作系统应用程序和设备的驱动程序，又需要设计 FPGA 部分的硬件逻辑设计。开发中既要了解 Linux 操作系统，系统的构架，也需要搭建一个 FPGA 和 ARM 系统之间的硬件设计平台。所以 ZYNQ 的开发是需要软件人员和硬件硬件人员协同设计并开发的。这既是 ZYNQ 开发中所谓的"软硬件协同设计”。

ZYNQ 系统的硬件系统和软件系统的设计和开发需要用到一下的开发环境和调试工具：

Xilinx Vivado。

Vivado 设计套件实现 FPGA 部分的设计和开发，管脚和时序的约束，编译和仿真，实现

RTL 到比特流的设计流程。Vivado 并不是 ISE 设计套件的简单升级，而是一个全新的设计套件。它替代了 ISE 设计套件的所有重要工具，比如 Project Navigator、Xilinx Synthesis Technology、Implementation、CORE Generator、Constraint、Simulator、Chipscope Analyzer、FPGA Editor 等设计工具。

Xilinx SDK（Software Development Kit），SDK 是 Xilinx 软件开发套件(SDK),在 Vivado 硬件系统的基础上，系统会自动配置一些重要参数，其中包括工具和库路径、编译器选项、JTAG 和闪存设置，调试器连接已经裸机板支持包(BSP)。SDK 也为所有支持的 Xilinx IP 硬核提供了驱动程序。SDK 支持 IP 硬核（FPGA 上）和处理器软件协同调试，我们可以使用高级 C 或 C++语言来开发和调试 ARM 和 FPGA 系统，测试硬件系统是否工作正常。SDK 软件也是 Vivado 软件自带的，无需单独安装。

ZYNQ 的开发也是先硬件后软件的方法。具体流程如下：

1) 在 Vivado 上新建工程，增加一个嵌入式的源文件。
2) 在 Vivado 里添加和配置 PS 和 PL 部分基本的外设，或需要添加自定义的外设。
3) 在 Vivado 里生成顶层 HDL 文件，并添加约束文件。再编译生成比特流文件（*.bit）。
4) 导出硬件信息到 SDK 软件开发环境，在 SDK 环境里可以编写一些调试软件验证硬件和软件，结合比特流文件单独调试 ZYNQ 系统。
5) 在 SDK 里生成 FSBL 文件。
6) 在 VMware 虚拟机里生成 u-boot.elf、bootloader 镜像。
7) 在 SDK 里通过 FSBL 文件，比特流文件 system.bit 和 u-boot.elf 文件生成一个 BOOT.bin 文件。
8) 在 VMware 里生成 Ubuntu 的内核镜像文件 Zimage 和 Ubuntu 的根文件系统。另外还需要要对 FPGA 自定义的 IP 编写驱动。
9) 把 BOOT、内核、设备树、根文件系统文件放入到 SD 卡中，启动开发板电源，Linux 操作系统会从 SD 卡里启动。

以上是典型的 ZYNQ 开发流程，但是 ZYNQ 也可以单独做为 ARM 来使用，这样就不需要关系 PL 端资源，和传统的 ARM 开发没有太大区别。ZYNQ 也可以只使用 PL 部分，但是 PL 的配置还是要 PS 来完成的，就是无法通过传统的固化 Flash 方式把只要 PL 的固件固化起来。

1.3 学习 ZYNQ 要具备哪些技能

学习 ZYNQ 比学习 FPGA、MCU、ARM 等传统工具开发要求更高，想学好 ZYNQ 也不是一蹴而就的事情。

1.3.1 软件开发人员

- ✓ 计算机组成原理
- ✓ C、C++语言

- ✓ 计算机操作系统
- ✓ tcl 脚本
- ✓ 良好的英语阅读基础

1.3.2 逻辑开发人员

- ✓ 计算机组成原理
- ✓ C 语言
- ✓ 数字电路基础

第二章 PL 的"Hello World"LED 实验

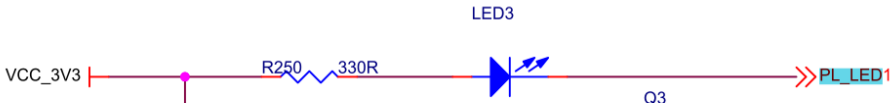
实验 Vivado 工程为“led”。

对于 ZYNQ 来说 PL（FPGA）开发是至关重要的，这也是 ZYNQ 比其他 ARM 的有优势的地方，可以定制化很多 ARM 端的外设，在定制 ARM 端的外设之前先让我们通过一个 LED 例程来熟悉 PL（FPGA）的开发流程，熟悉 Vivado 软件的基本操作，这个开发流程和不带 ARM 的 FPGA 芯片完全一致。

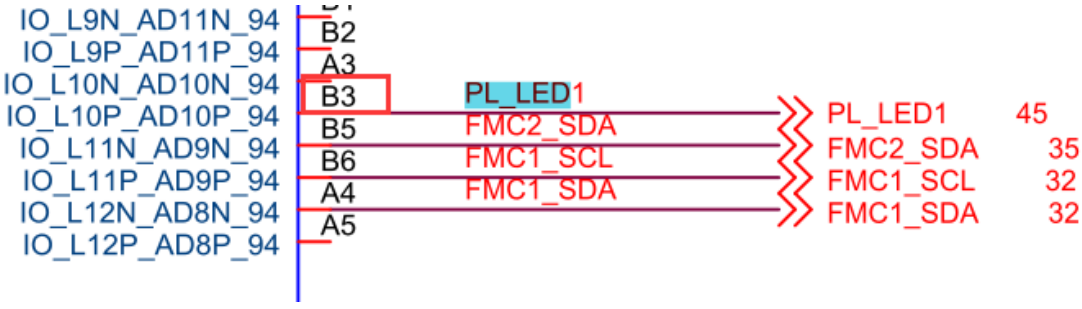
在本例程中，我们要做的是 LED 灯控制实验，每秒钟控制开发板上的 LED 灯翻转一次，实现亮、灭、亮、灭的控制。会控制 LED 灯，其它外设也慢慢就会了。

2.1 LED 硬件介绍

1) 开发板的 PL 部分连接了 1 个用户 LED 灯。这 1 个灯完全由 PL 控制。如果 PL_LED1 为高电平，灯会灭，否则会亮。



2) 我们可以根据原理图的连线关系确定 LED 和 PL 管脚的绑定关系。



3) 原理图中以 PS_MIO 开头的 IO 都是 PS 端 IO，不需要绑定，也不能绑定

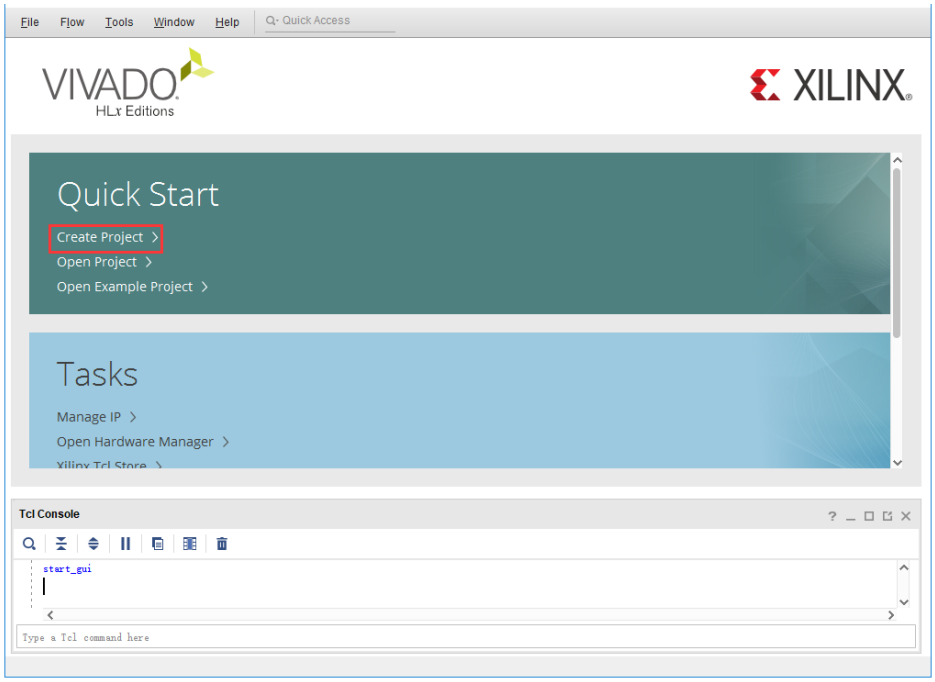
PS_MIO0_500	AF16
PS_MIO1_500	AJ16
PS_MIO2_500	AD16
PS_MIO3_500	AG16
PS_MIO4_500	AH16
PS_MIO5_500	AM15
PS_MIO6_500	AL15
PS_MIO7_500	AD17
PS_MIO8_500	AE17
PS_MIO9_500	AP15
PS_MIO10_500	AH17
PS_MIO11_500	AF17
PS_MIO12_500	AJ17
PS_MIO13_500	AK17
PS_MIO14_500	AL16
PS_MIO15_500	AN16
PS_MIO16_500	AM16
PS_MIO17_500	AP16
PS_MIO18_500	AE18
PS_MIO19_500	AL17
PS_MIO20_500	AD18
PS_MIO21_500	AF18
PS_MIO22_500	AD20
PS_MIO23_500	AD19
PS_MIO24_500	AE20
PS_MIO25_500	AE19

2.2 创建 Vivado 工程

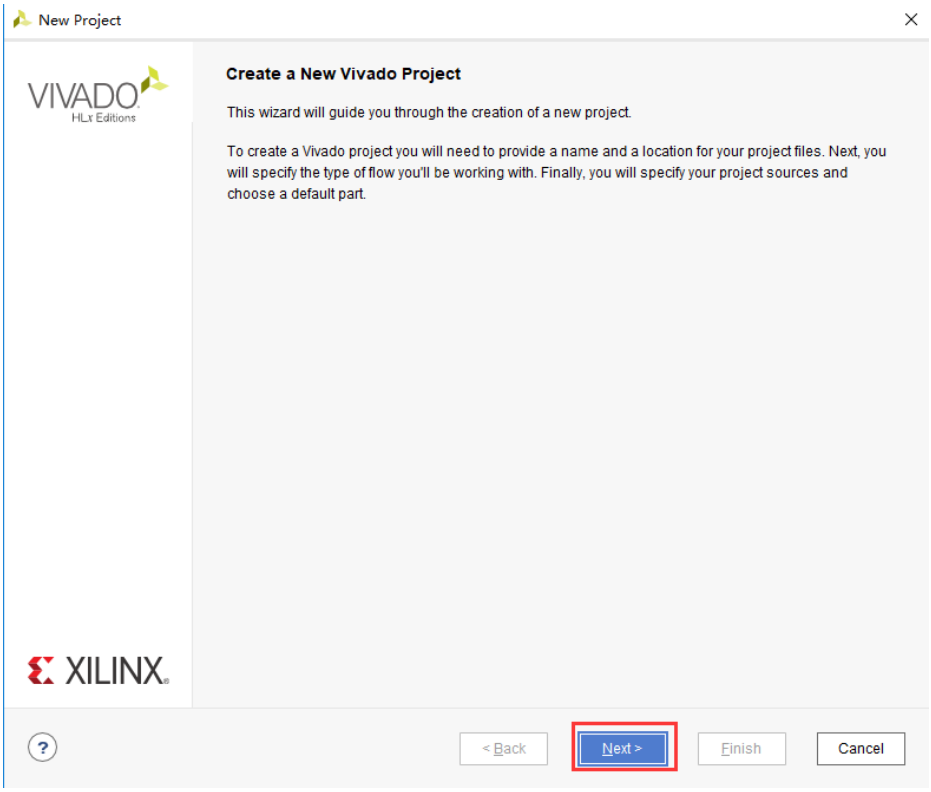
1) 启动 Vivado，在 Windows 中可以通过双击 Vivado 快捷方式启动



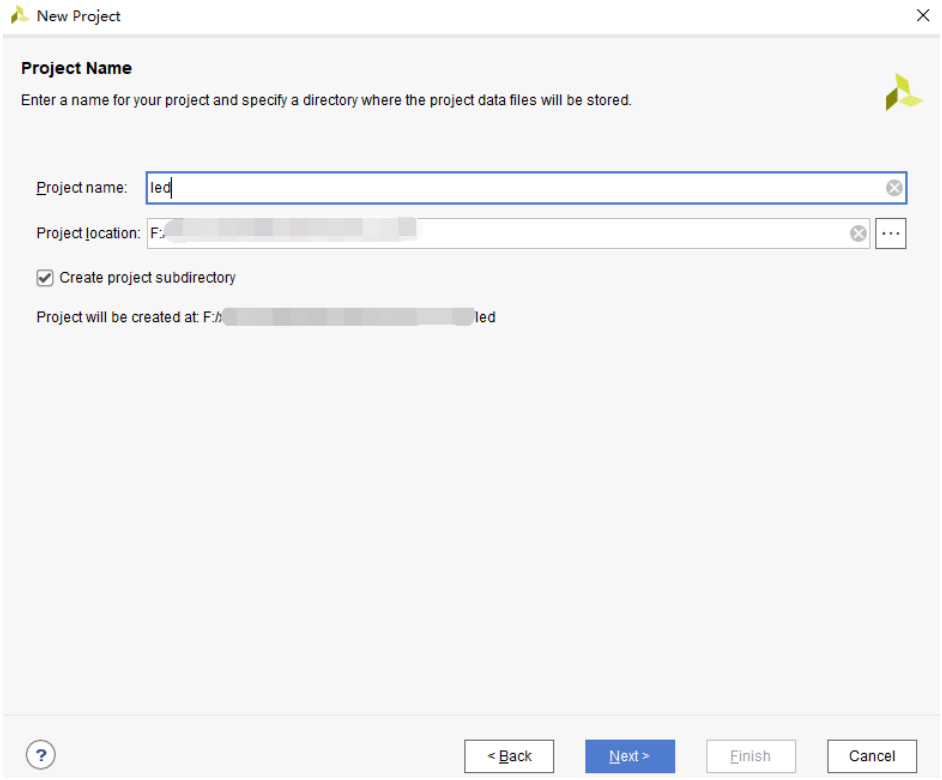
2) 在 Vivado 开发环境里点击“Create New Project”，创建一个新的工程。



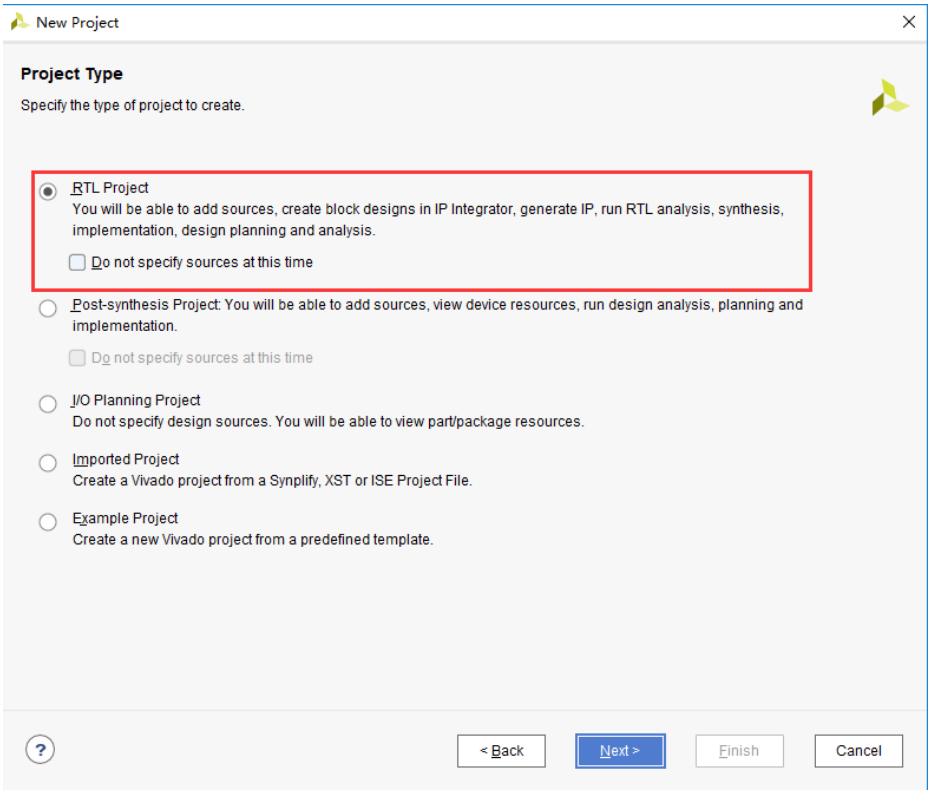
3) 弹出一个建立新工程的向导，点击“Next”



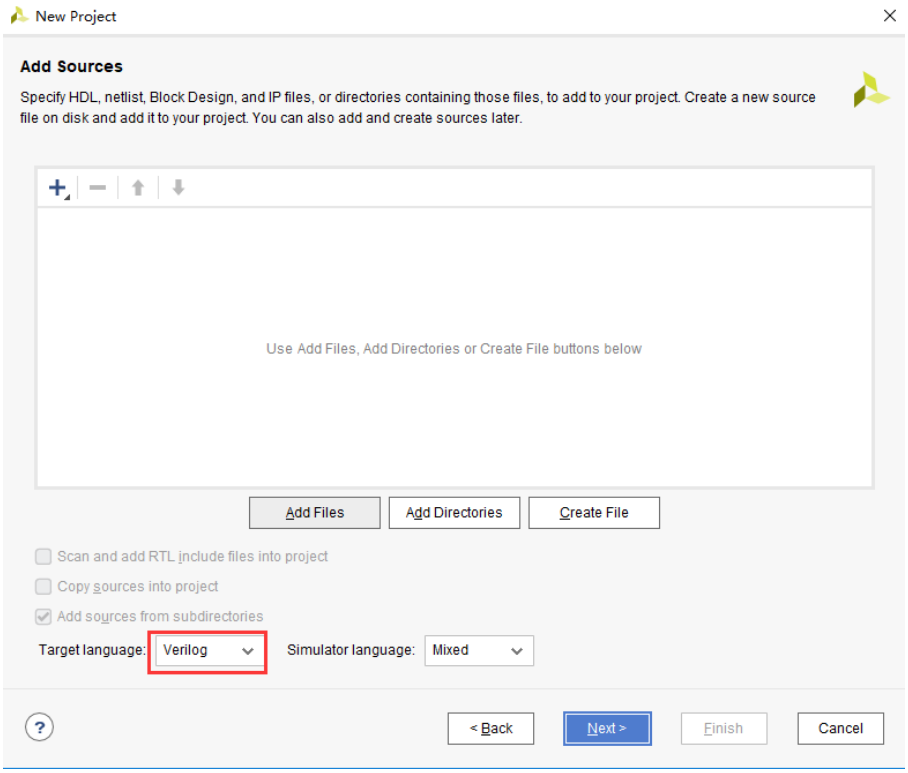
4) 在弹出的对话框中输入工程名和工程存放的目录，我们这里取一个 led 的工程名。需要注意工程路径“Project location”不能有中文空格，路径名称也不能太长。



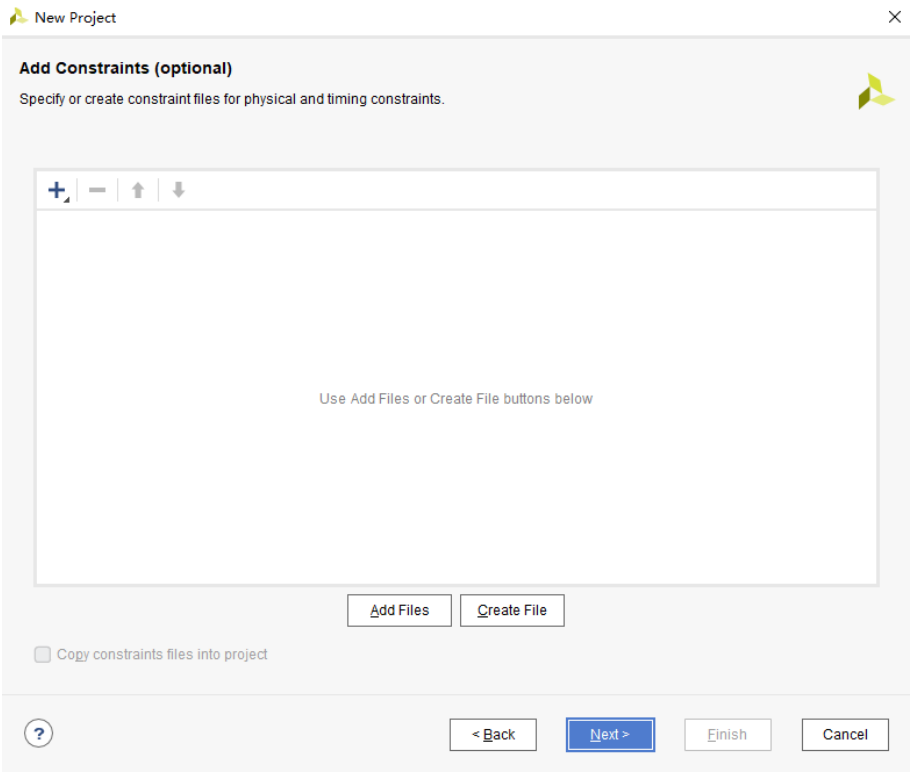
5) 在工程类型中选择“RTL Project”



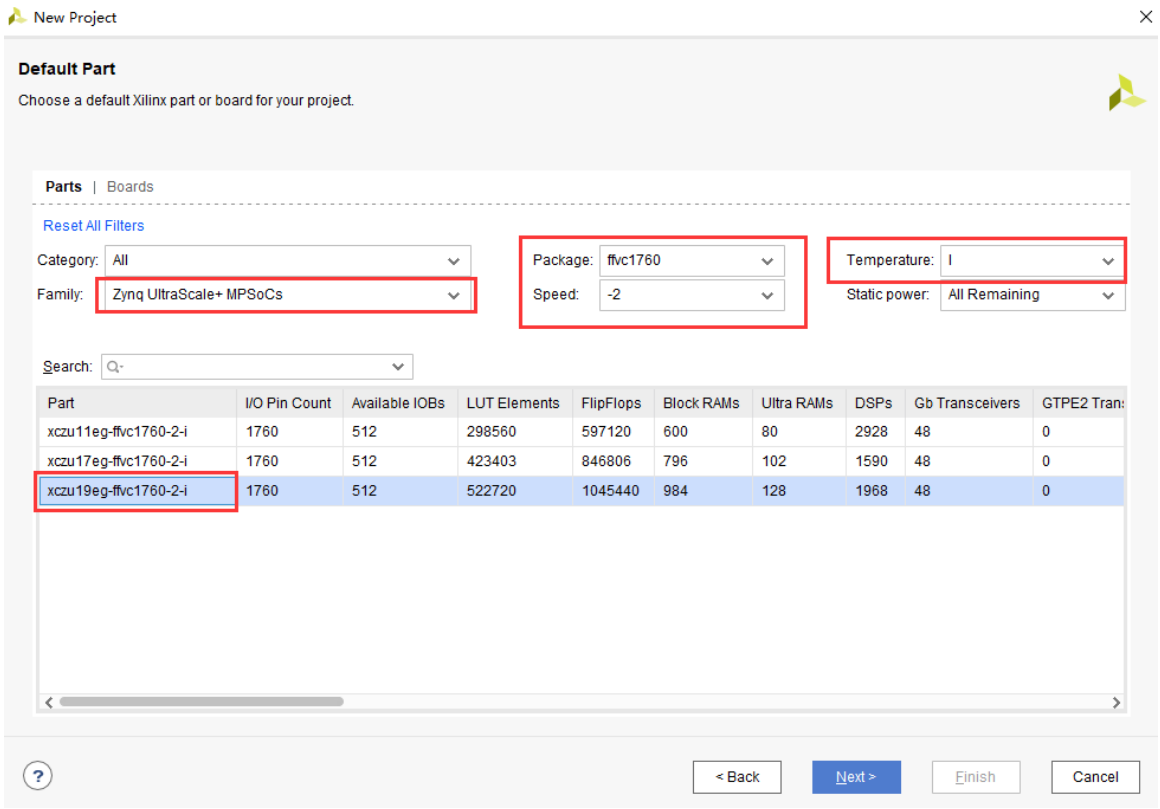
6) 目标语言“Target language”选择“Verilog”，虽然选择 Verilog，但 VHDL 也可以使用，支持多语言混合编程。



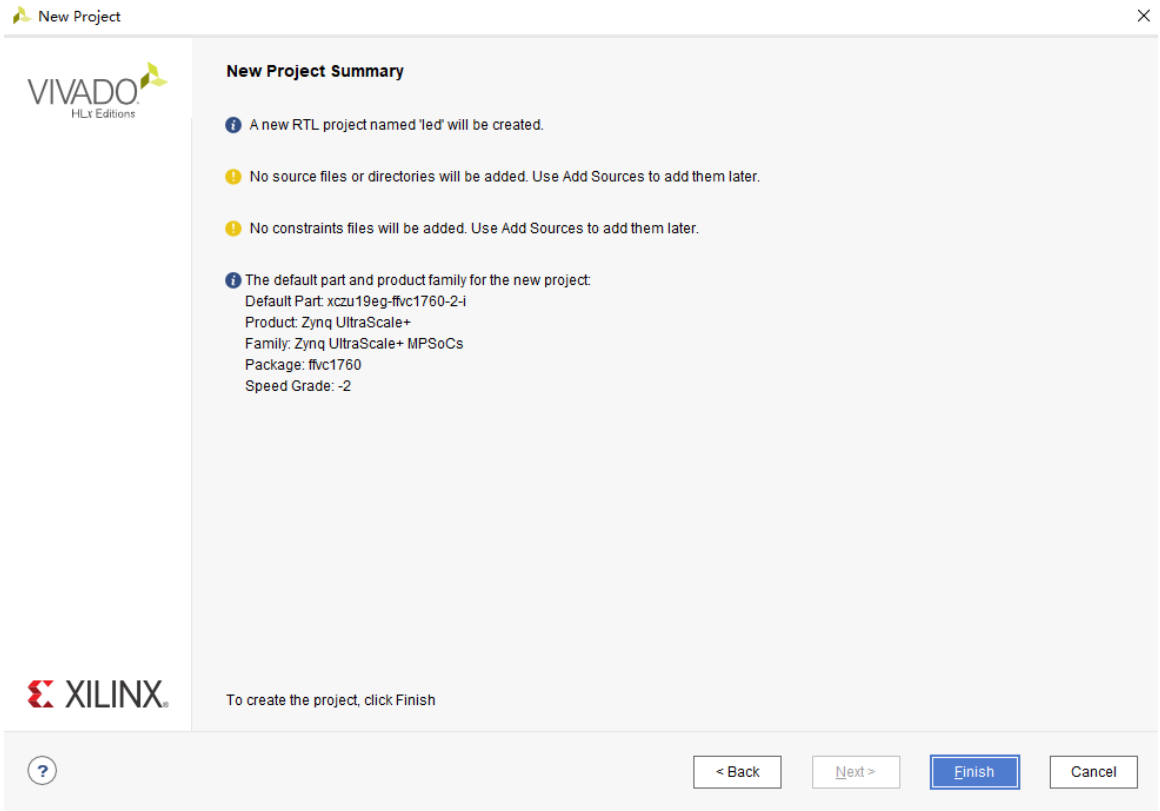
7) 点击“Next”，不添加任何文件



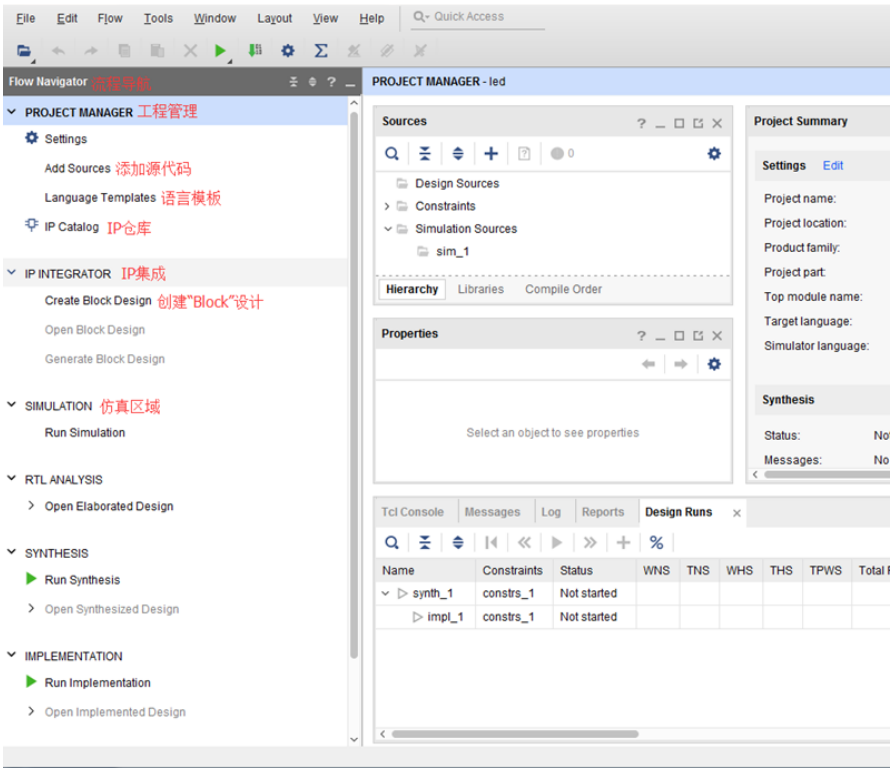
8) 在“Part”选项中，器件家族“Family”选择“Zynq UltraScale+ MPSoCs”，封装类型“Package”选择“ffvc1760”,Speed 选择”-2”,Temperature 选择“I”减少选择范围。在下拉列表中选择“xczu19eg-ffvc1760-2-i”，“-2”表示速率等级，数字越大，性能越好，速率高的芯片向下兼容速率低的芯片。



9) 点击“Finish”就可以完成以后名为“led”工程的创建。

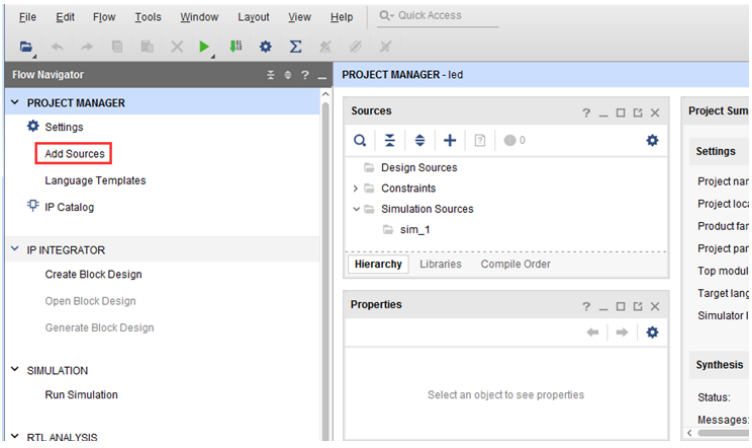


10) Vivado 软件界面

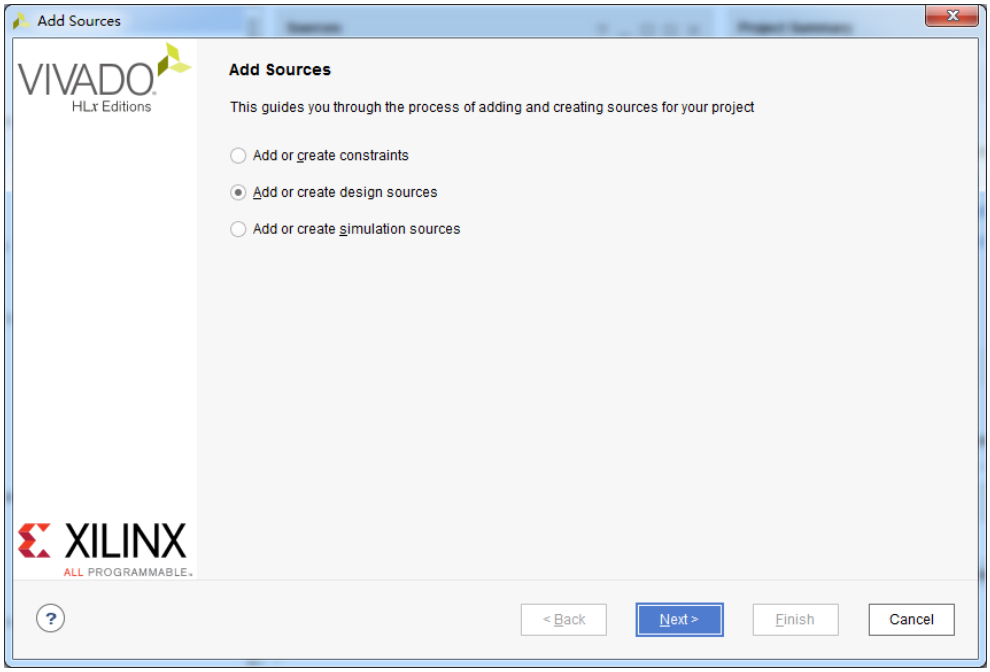


2.3 创建 Verilog HDL 文件点亮 LED

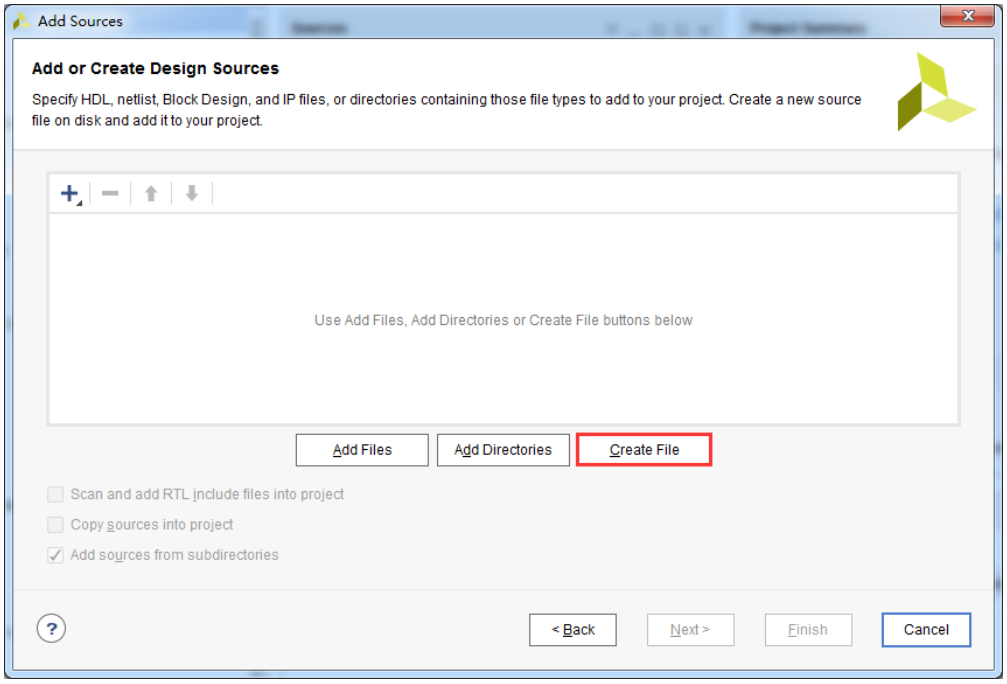
1) 点击 Project Manager 下的 Add Sources 图标（或者使用快捷键 Alt+A）



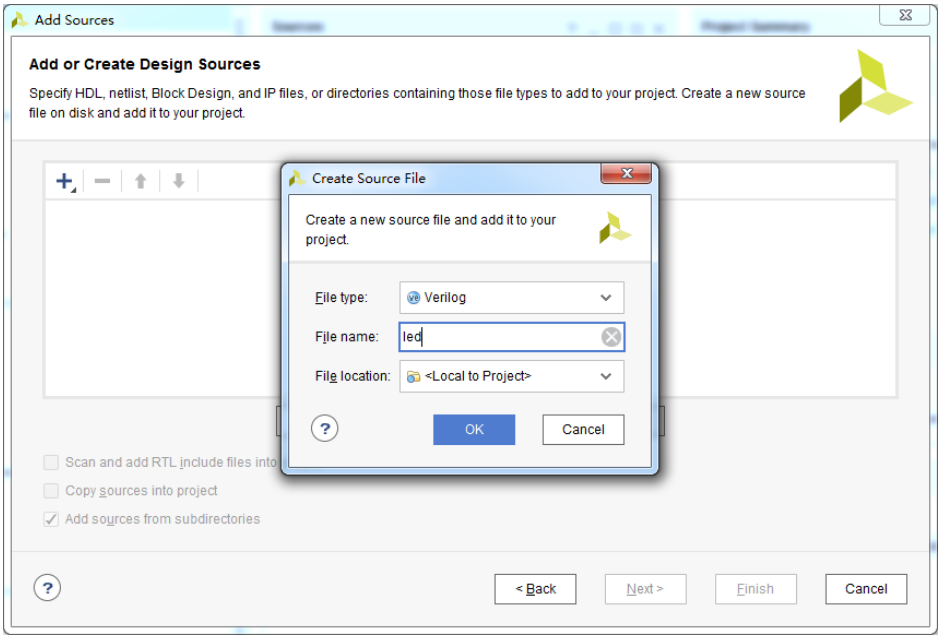
2) 选择添加或创建设计源文件“Add or create design sources”,点击“Next”



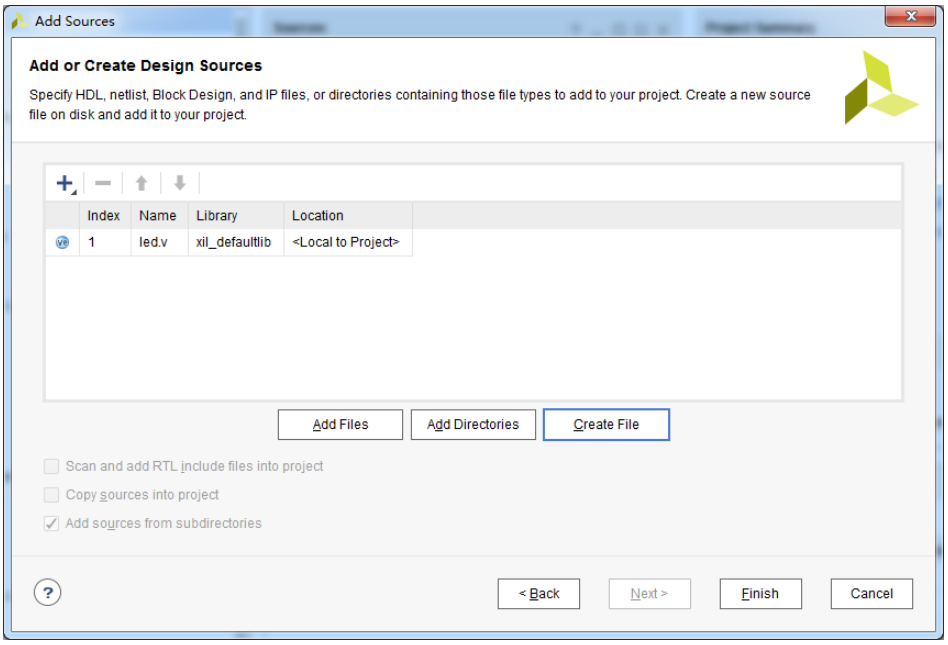
3) 选择创建文件“Create File”



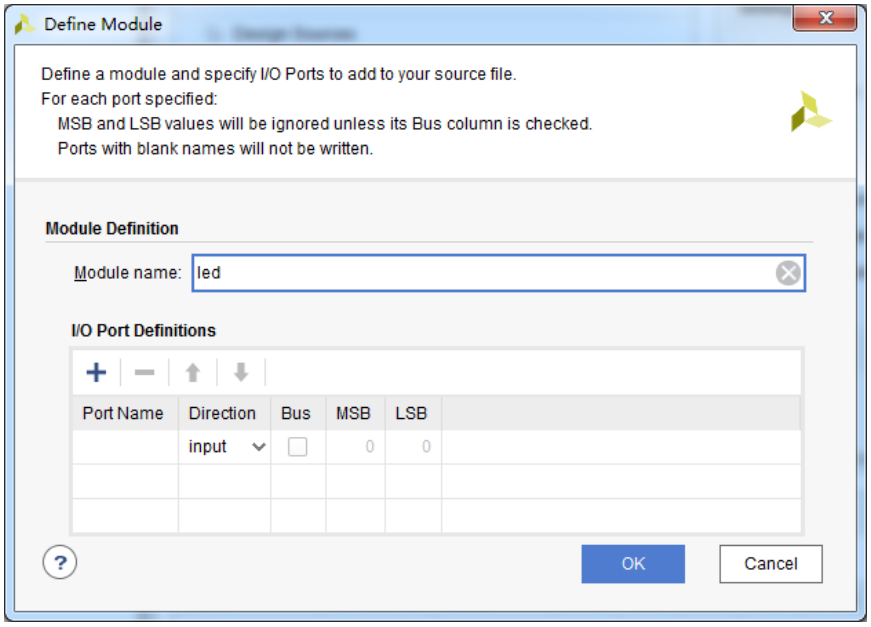
4) 文件名“File name”设置为“led”，点击“OK”



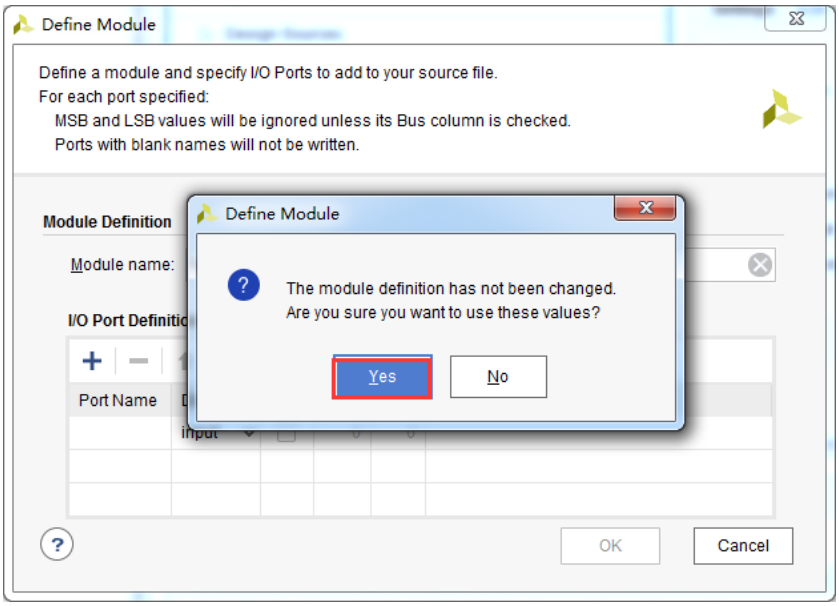
5) 点击“Finish”,完成“led.v”文件添加



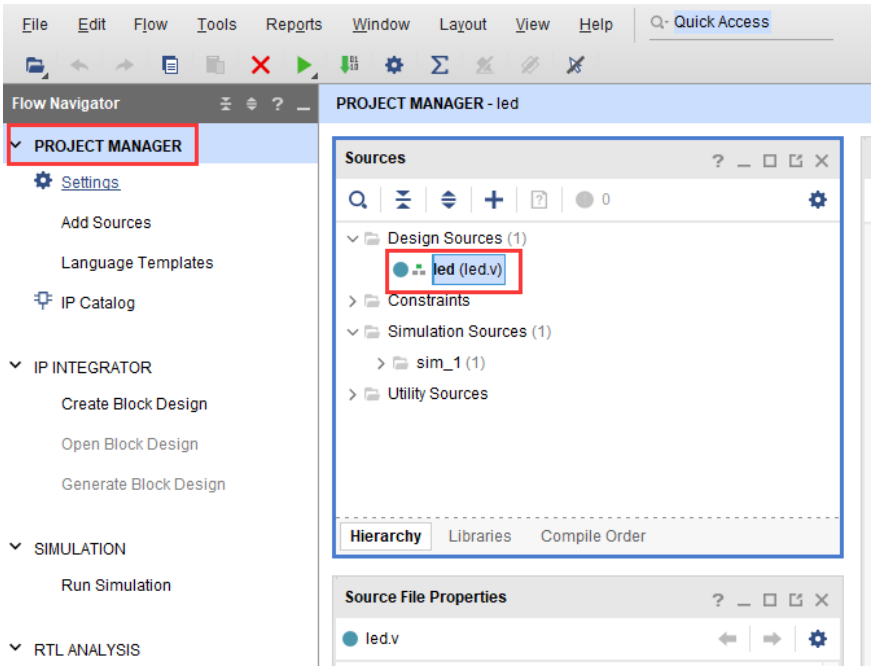
6) 在弹出的模块定义“Define Module”,中可以指定“led.v”文件的模块名称“Module name”,这里默认不变为“led”，还可以指定一些端口，这里暂时不指定，点击“OK”。



7) 在弹出的对话框中选择“Yes”



8) 双击“led.v”可以打开文件，然后编辑



9) 编写“led.v”,这里定义了一个 32 位的寄存器 timer，用于循环计数 0~199999999(1 秒钟)，计数到 199999999(1 秒)的时候，寄存器 timer 变为 0，并翻转四个 LED。这样原来 LED 是灭的话，就会点亮，如果原来 LED 为亮的话，就会熄灭。由于输入时钟为 200MHz 的差分时钟，因此需要添加 IBUFDS 原语连接差分信号，编写好后的代码如下：

```
`timescale 1ns / 1ps
module led(
//Differential system clock
    input sys_clk_p,
    input sys_clk_n,
    input rst_n,
    output reg  led
    );
reg[31:0] timer_cnt;
wire sys_clk ;

   IBUFDS IBUFDS_inst (
      .O(sys_clk),  // Buffer output
      .I(sys_clk_p),  // Diff_p buffer input (connect directly
to top-level port)
      .IB(sys_clk_n) // Diff_n buffer input (connect directly to
top-level port)
   );

always@(posedge sys_clk)
begin
    if (!rst_n)
    begin
      led <= 1'b0 ;
      timer_cnt <= 32'd0 ;
    end
    else if(timer_cnt >= 32'd199_999_999)   //1 second counter,
200M-1=199999999
    begin
        led <= ~led;
        timer_cnt <= 32'd0;
```

```
            end
            else
            begin
                led <= led;
                timer_cnt <= timer_cnt + 32'd1;
            end

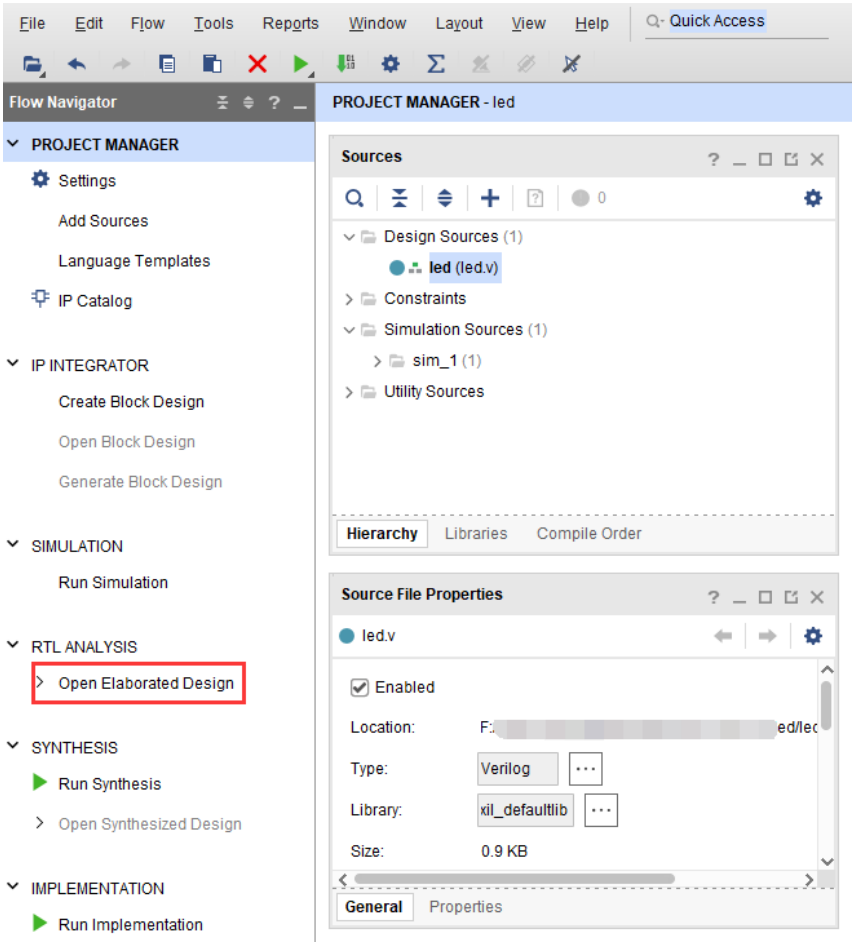
        end
        endmodule
```

10) 编写好代码后保存

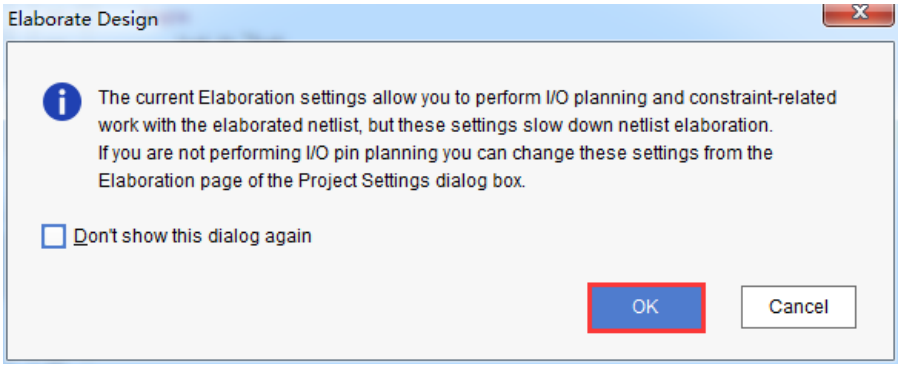
2.4 添加管脚约束

Vivado 使用的约束文件格式为 xdc 文件。xdc 文件里主要是完成管脚的约束,时钟的约束,以及组的约束。这里我们需要对 led.v 程序中的输入输出端口分配到 FPGA 的真实管脚上。

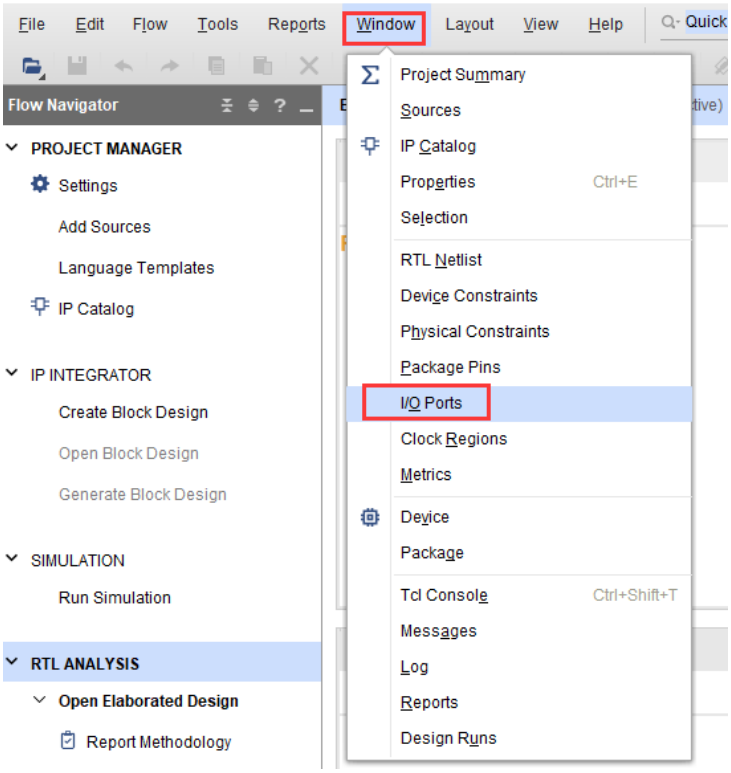
1) 点击“Open Elaborated Design”



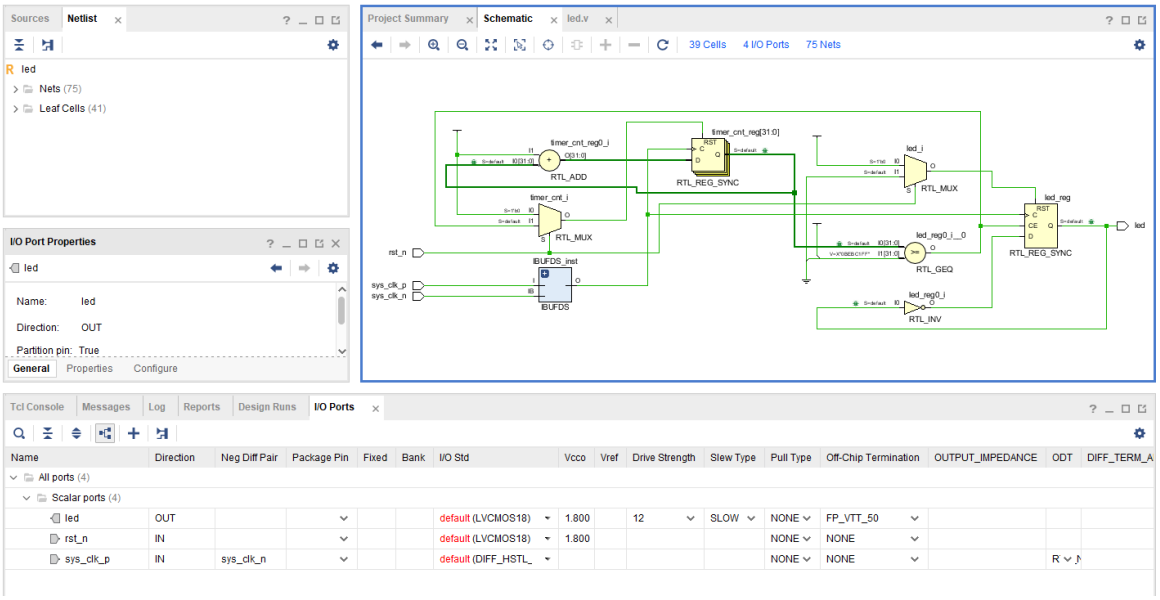
2) 在弹出的窗口中点击“OK”按钮



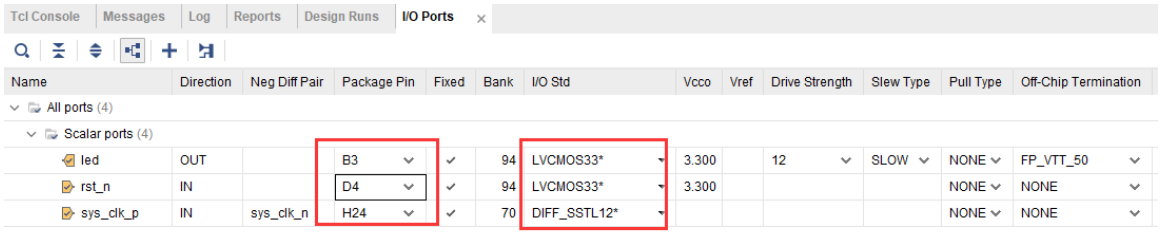
3) 在菜单中选择“Window -> I/O Ports”



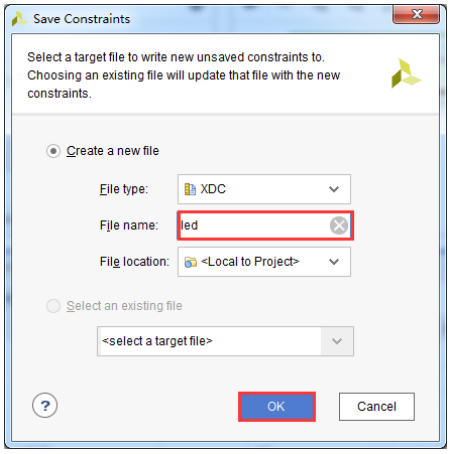
4) 在弹出的 I/O Ports 中可以看到管脚分配情况



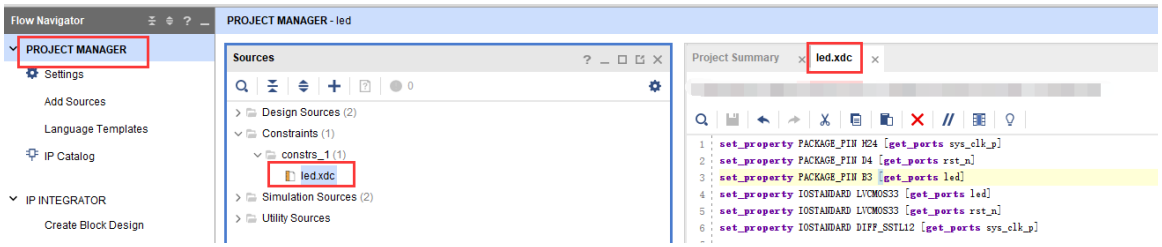
5) 将复位信号 rst_n 绑定到 PL 端的按键，给 LED 和时钟分配管脚、电平标准，完成后点击保存图标



6) 弹出窗口，要求保存约束文件，文件名我们填写“led”，文件类型默认“XDC”，点击“OK”



7) 打开刚才生成的“led.xdc”文件，我们可以看到是一个 TCL 脚本，如果我们了解这些语法，完全可以通过自己编写 led.xdc 文件的方式来约束管脚



下面来介绍一下最基本的 XDC 编写的语法，普通 IO 口只需约束引脚号和电压，管脚约束如下：

set_property PACKAGE_PIN "引脚编号" [get_ports "端口名称"]

电平信号的约束如下：

set_property IOSTANDARD "电平标准" [get_ports "端口名称"]

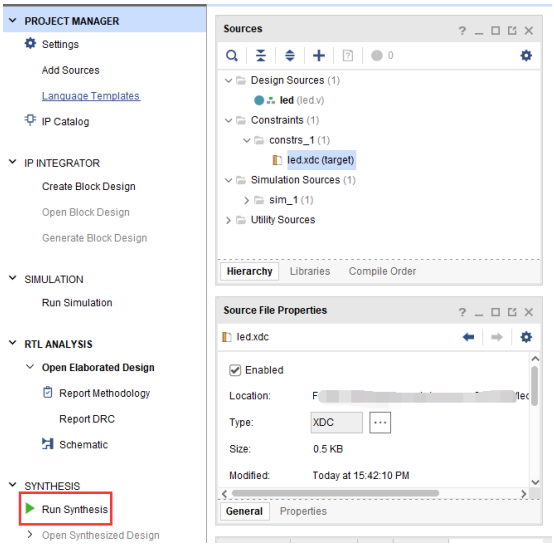
这里需要注意文字的大小写，端口名称是数组的话用{ }括起来，端口名称必须和源代码中的名字一致，且端口名字不能和关键字一样。

电平标准中“LVCMOS33”后面的数字指 FPGA 的 BANK 电压，LED 所在 BANK 电压为 3.3 伏，所以电平标准为“LVCMOS33”。*Vivado 默认要求为所有 IO 分配正确的电平标准和管脚编号*。

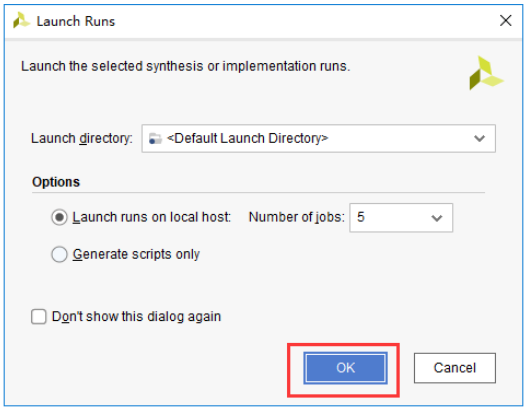
2.5 添加时序约束

一个 FPGA 设计除了管脚分配以外，还有一个重要的约束，那就是时序约束，这里通过向导方式演示如果进行一个时序约束。

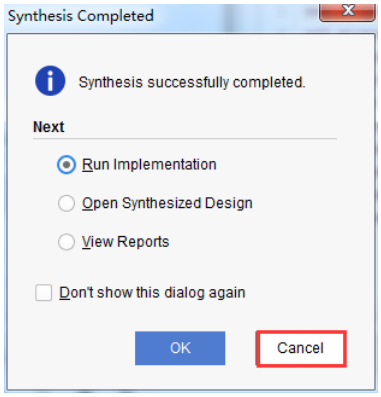
1) 点击“Run Synthesis”开始综合



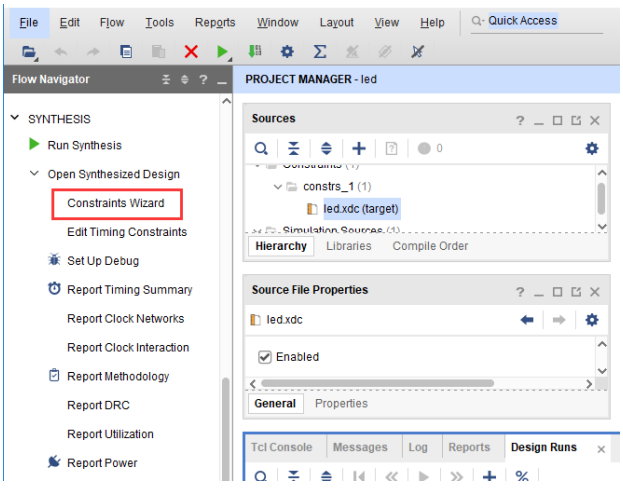
2) 弹出对话框点击“OK”



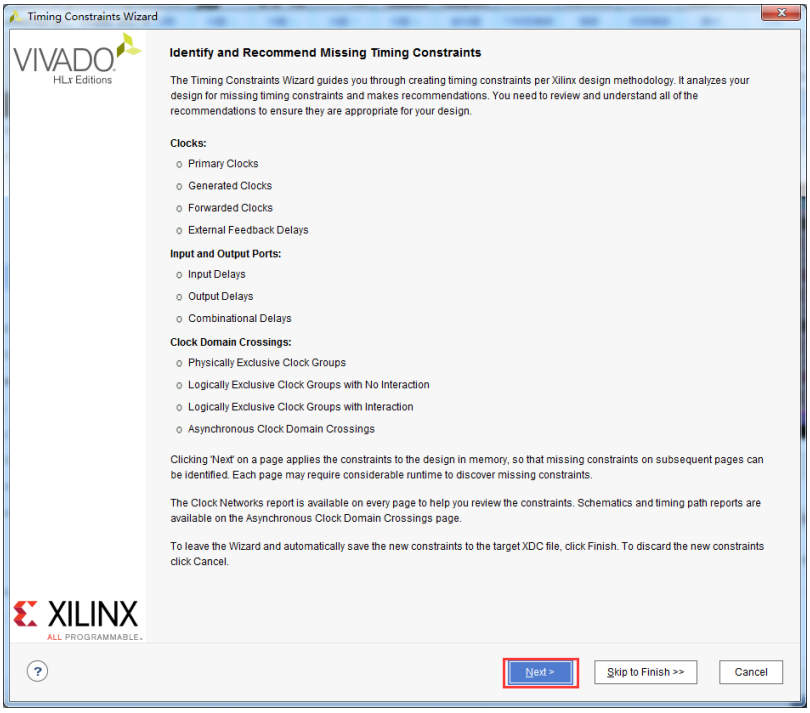
3) 综合完成以后点击“Cancel”



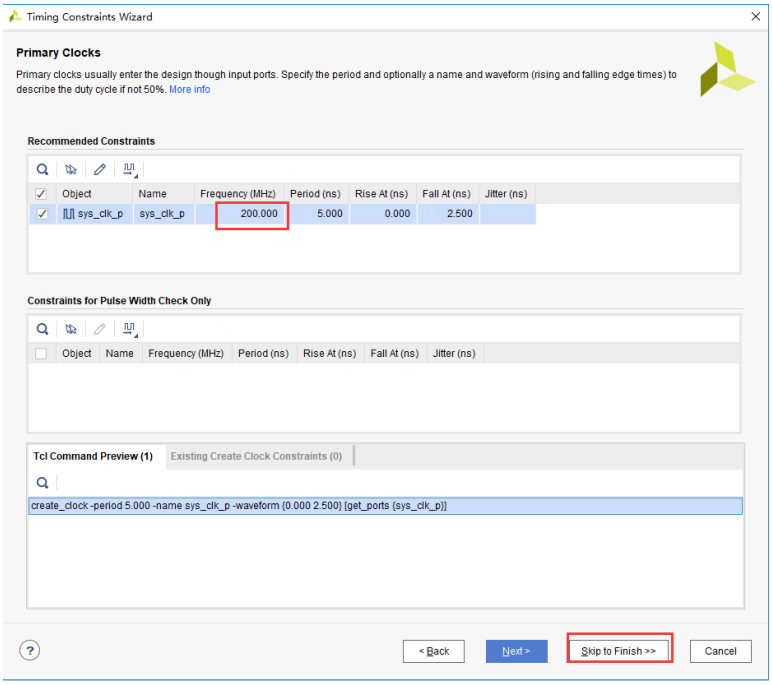
4) 点击“Constraints Wizard”



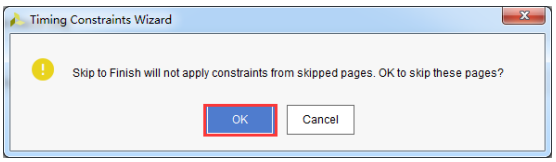
5) 在弹出的窗口中点击“Next”



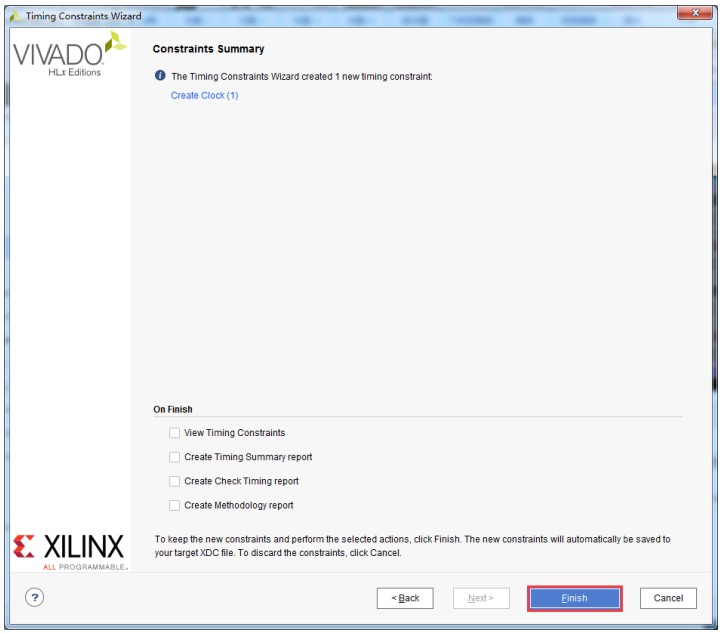
6) 时序约束向导分析出设计中的时钟，这里把“sys_clk_p”频率设置为 200Mhz，然后点击“Skip to Finish”结束时序约束向导。



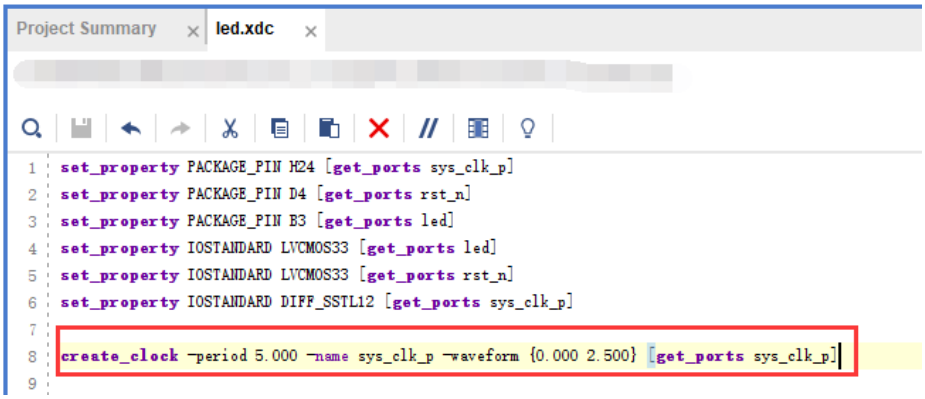
7) 弹出的窗口中点击“OK”



8) 点击“Finish”

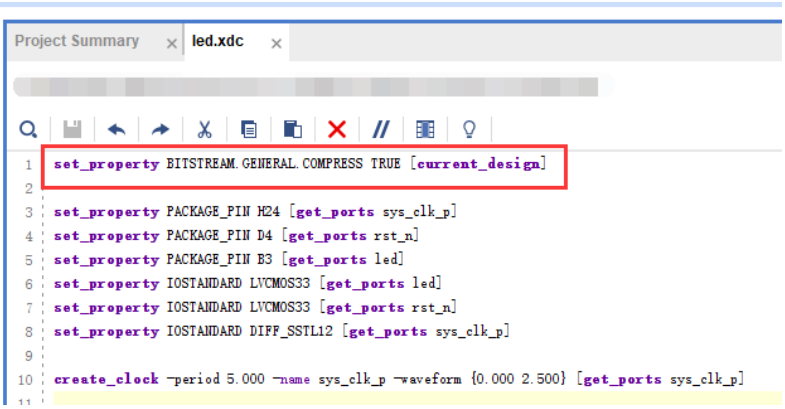


9) 这个时候 led.xdc 文件已经更新，点击“Reload”重新加载文件，并保存文件



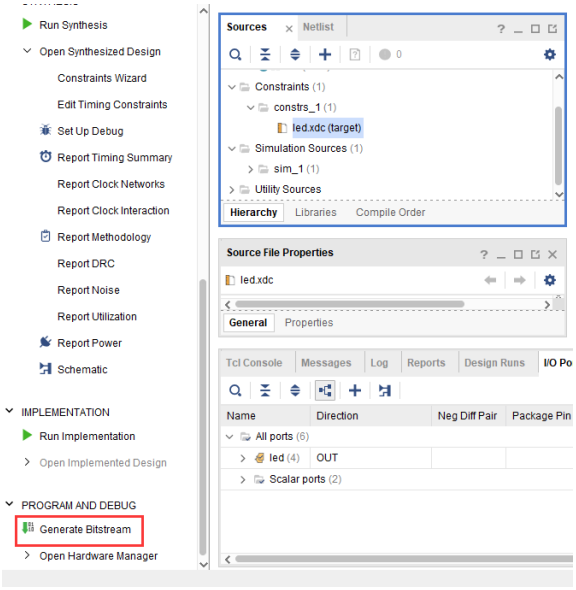
10) 由于生成的 bit 会很大，为了减小 bit 文件大小，在 XDC 文件中添加以下语句，压缩 bit 文件。

set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]

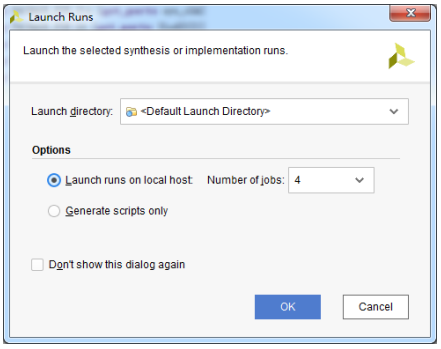


2.6 生成 BIT 文件

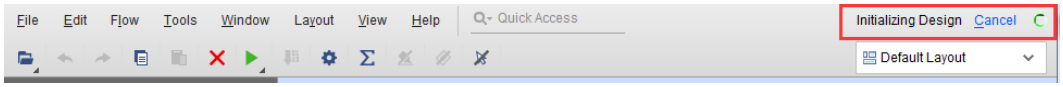
1) 编译的过程可以细分为综合、布局布线、生成 bit 文件等，这里我们直接点击“Generate Bitstream”，直接生成 bit 文件。



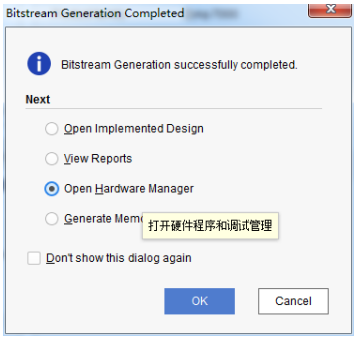
2) 在弹出的对话框中可以选择任务数量，这里和 CPU 核心数有关，一般数字越大，编译越快，点击“OK”



3) 这个时候开始编译，可以看到右上角有个状态信息，在编译过程中可能会被杀毒软件、电脑管家拦截运行，导致无法编译或很长时间没有编译成功。



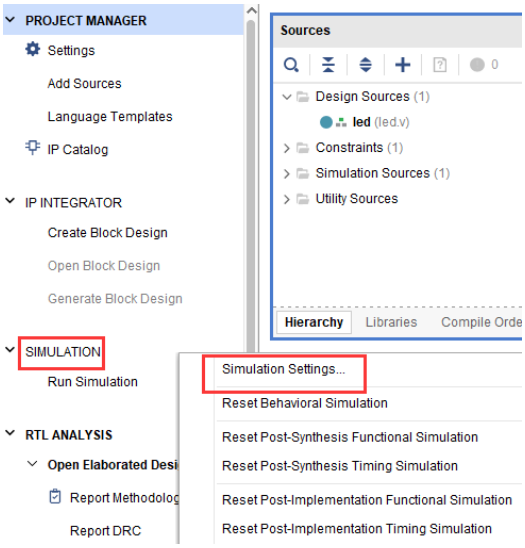
4) 编译中没有任何错误，编译完成，弹出一个对话框让我们选择后续操作，可以选择“Open Hardware Manger”，当然，也可以选择“Cancel”，我们这里选择“Cancel”，先不下载。



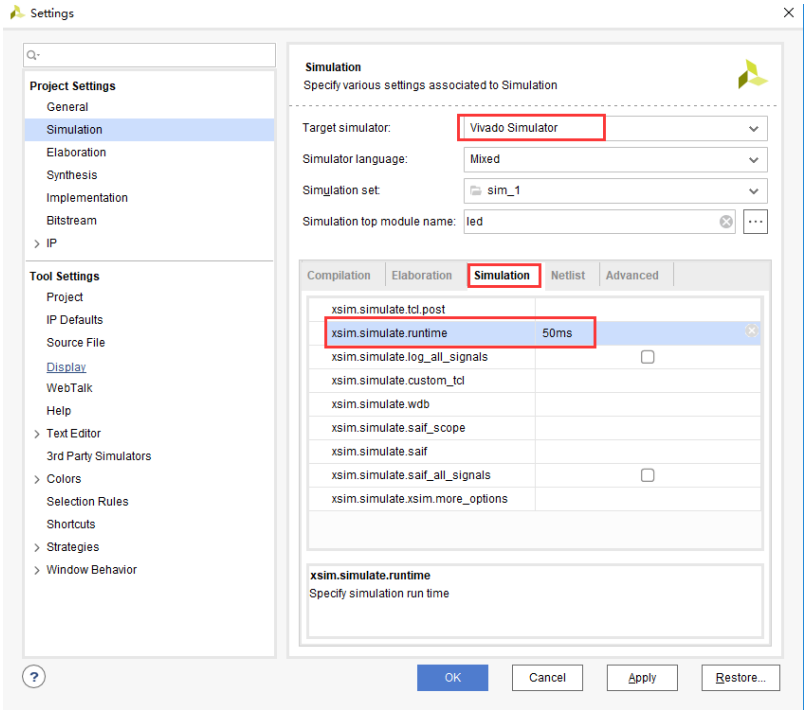
2.7 Vivado 仿真

接下来我们不妨小试牛刀，利用 Vivado 自带的仿真工具来输出波形验证流水灯程序设计结果和我们的预想是否一致（注意：在生成 bit 文件之前也可以仿真）。具体步骤如下：

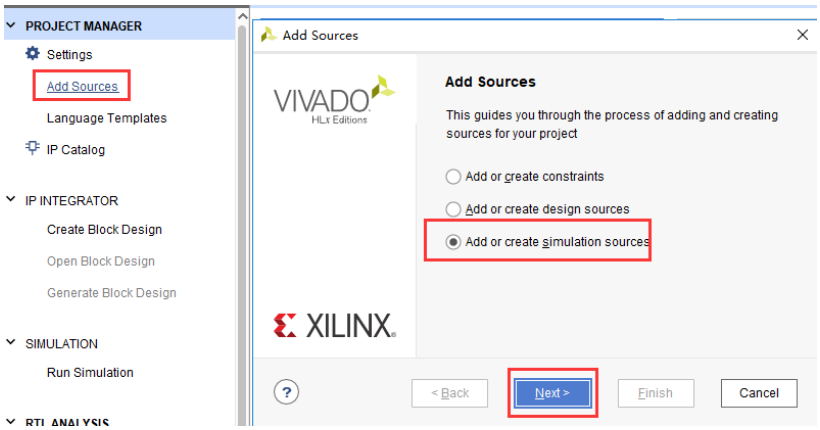
1） 设置 Vivado 的仿真配置，右击 SIMULATION 中 Simulation Settings。



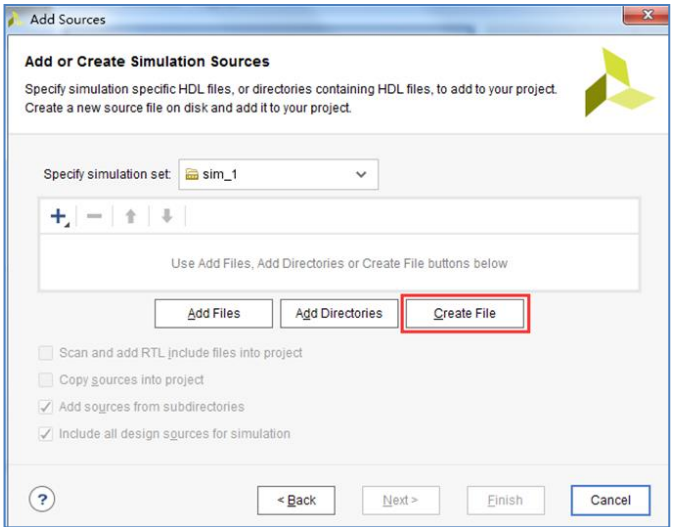
2） 在 Simulation Settings 窗口中进行如下图来配置，这里设置成 50ms（根据需要自行设定），其它按默认设置，单击 OK 完成。



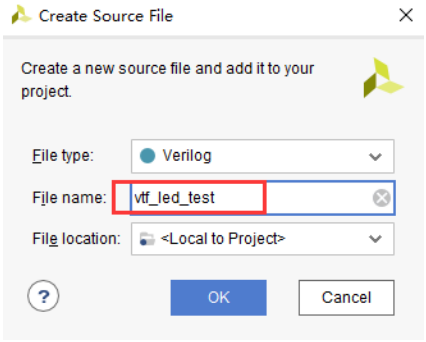
3） 添加激励测试文件，点击 Project Manager 下的 Add Sources 图标,按下图设置后单击 Next。



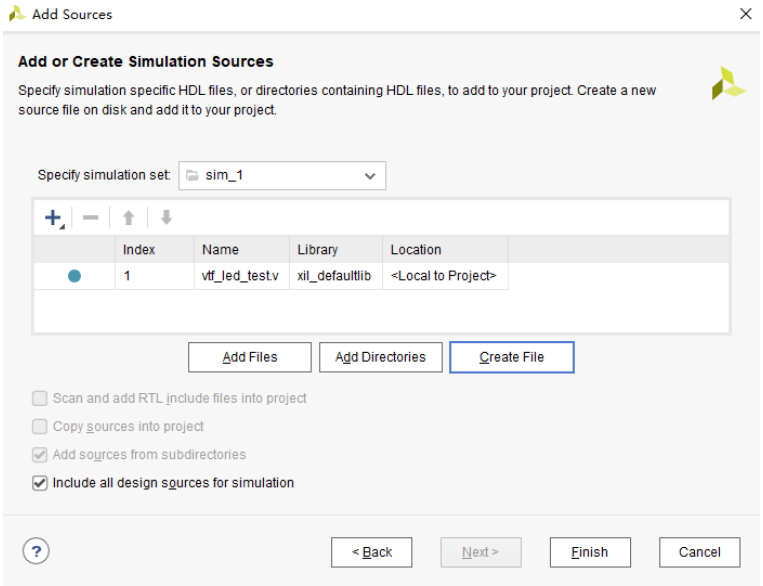
4） 点击 Create File 生成仿真激励文件。



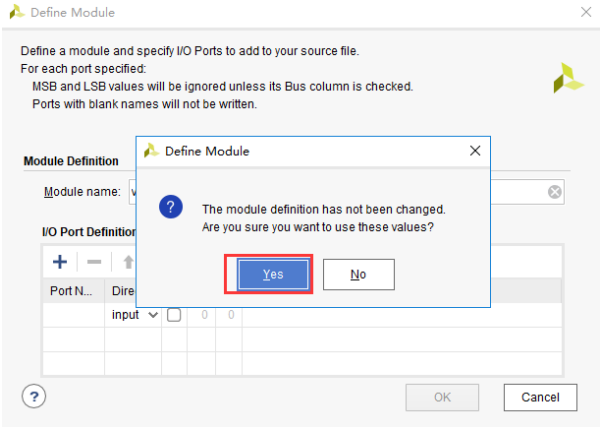
在弹出的对话框中输入激励文件的名字，这里我们输入名为 vtf_led_test。



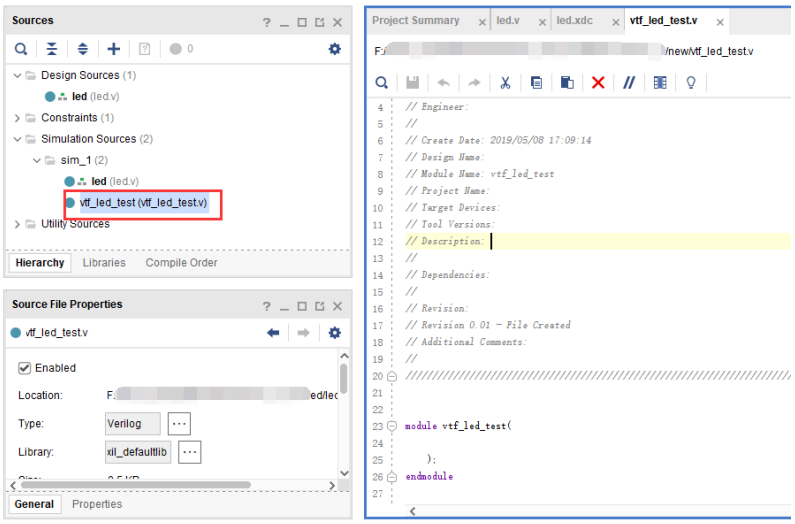
5） 点击 Finish 按钮返回。



这里我们先不添加 IO Ports，点击 OK。



在 Simulation Sources 目录下多了一个刚才添加的 vtf_led_test 文件。双击打开这个文件，可以看到里面只有 module 名的定义，其它都没有。



6) 接下去我们需要编写这个 vtf_led_test.v 文件的内容。首先定义输入和输出信号，然后需要实例化 led_test 模块，让 led_test 程序作为本测试程序的一部分。再添加复位和时钟的激励。完成后的 vtf_led_test.v 文件如下：

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Module Name: vtf_led_test
//////////////////////////////////////////////////////////////////////////////////

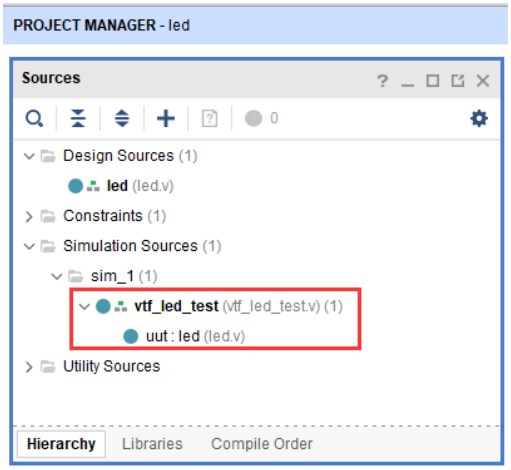
module vtf_led_test;
// Inputs
reg sys_clk_p;
reg rst_n ;
wire sys_clk_n;
// Outputs
wire led;

// Instantiate the Unit Under Test (UUT)
led uut (
    .sys_clk_p(sys_clk_p),
    .sys_clk_n(sys_clk_n),
    .rst_n(rst_n),
    .led(led)
 );

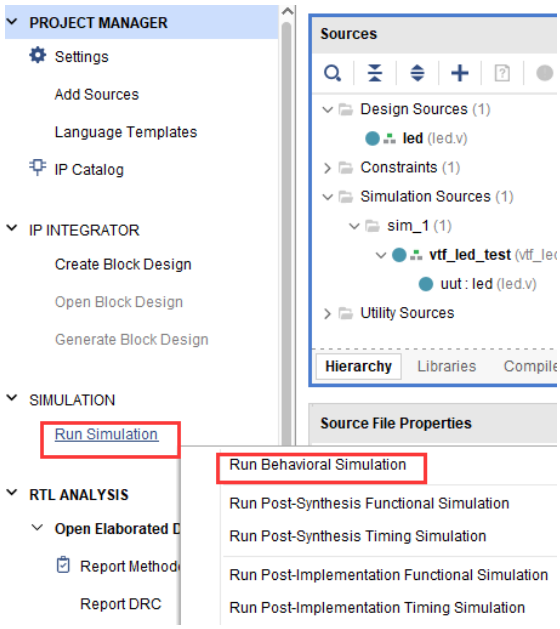
initial
begin
// Initialize Inputs
    sys_clk_p = 0;
    rst_n = 0;
// Wait for global reset to finish
    #1000;
    rst_n = 1;
end
//Create clock
always #2.5 sys_clk_p = ~ sys_clk_p;
assign  sys_clk_n = ~sys_clk_p ;

endmodule
```

7) 编写好后保存，vtf_led_test.v 自动成了这个仿真 Hierarchy 的顶层了，它下面是设计文件 led_test.v。

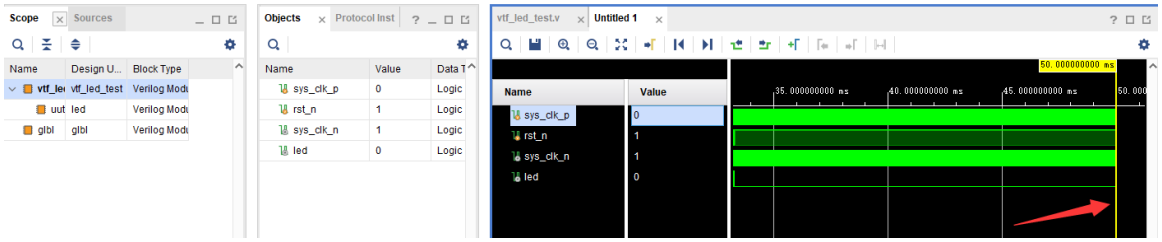


8) 点击 Run Simulation 按钮，再选择 Run Behavioral Simulation。这里我们做一下行为级的仿真就可以了。

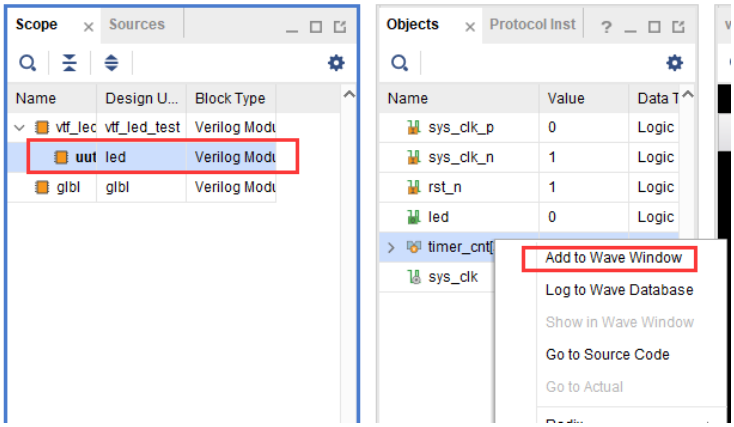


如果没有错误，Vivado 中的仿真软件开始工作了。

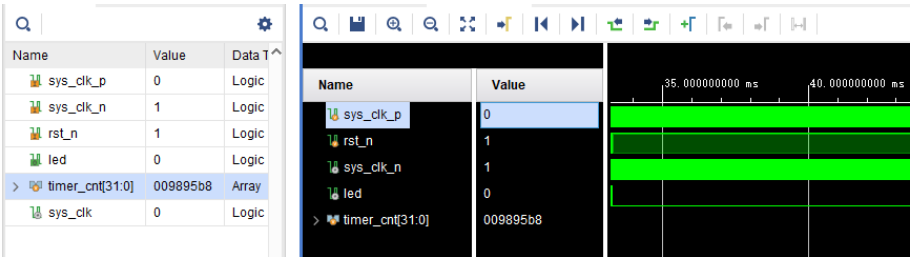
10. 在弹出仿真界面后如下图，界面是仿真软件自动运行到仿真设置的 50ms 的波形。



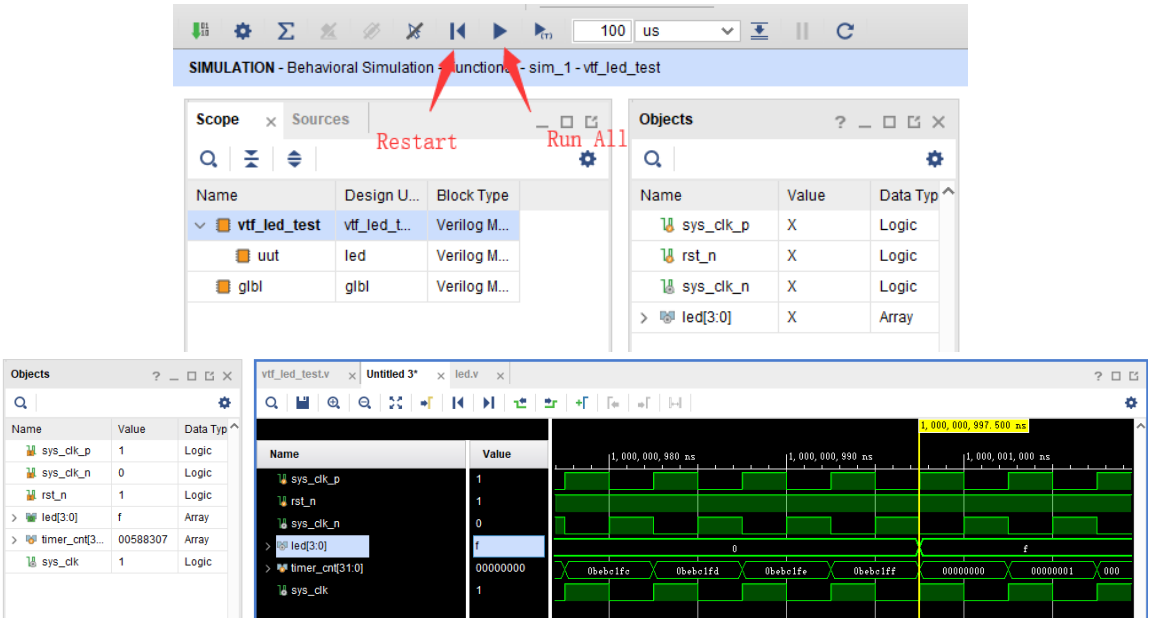
由于 LED[3：0]在程序中设计的状态变化时间长，而仿真又比较耗时，在这里观测 timer[31:0]计数器变化。把它放到 Wave 中观察(点击 Scope 界面下的 uut，再右键选择 Objects 界面下的 timer，在弹出的下拉菜单里选择 Add Wave Window)。



添加后 timer 显示在 Wave 的波形界面上，如下图所示。



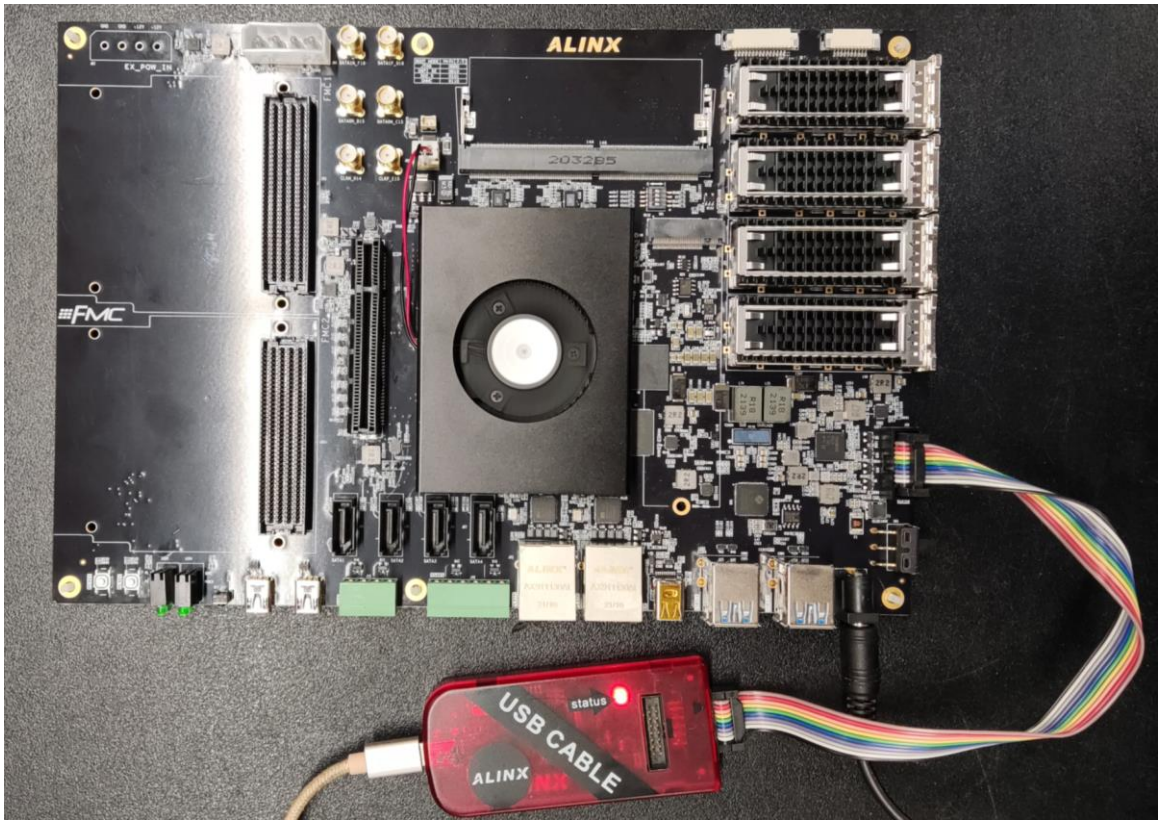
11. 点击如下标注的 Restart 按钮复位一下，再点击 Run All 按钮。(需要耐心!!!)，可以看到仿真波形与设计相符。(注意：仿真的时间越长，仿真的波形文件占用的磁盘空间越大，波形文件在工程目录的 xx.sim 文件夹)



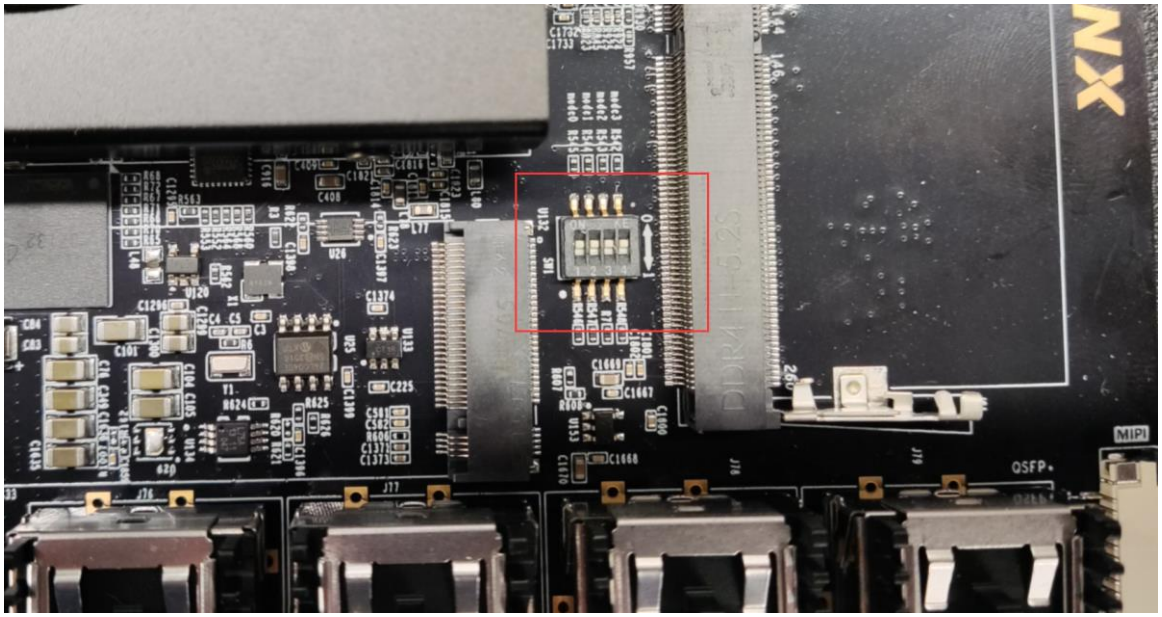
我们可以看到 led 的信号会变成 1，说明 LED 灯会变亮。

2.8 下载

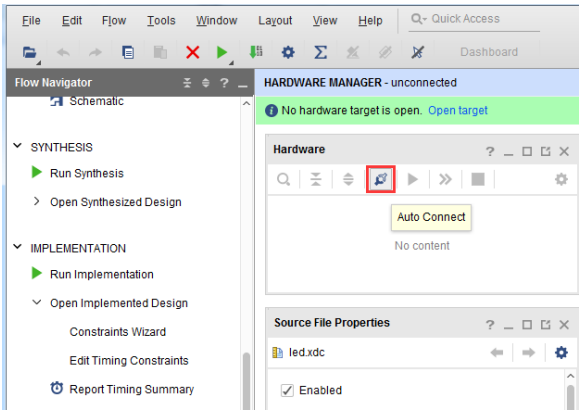
1) 连接好开发板的 JTAG 接口，给开发板上电



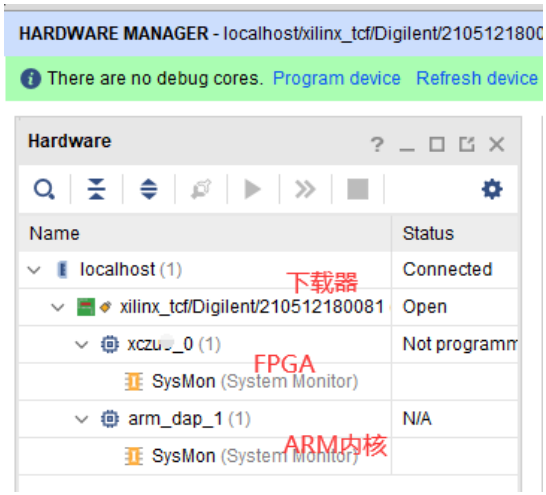
注意拨码开关要选择 JTAG 模式，也就是全部拨到”ON”，“ON”代表的值是 0，不用 JTAG 模式，下载会报错。



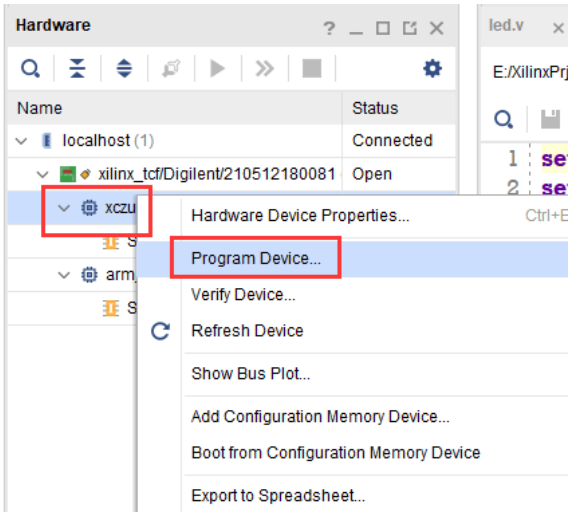
2) 在“HARDWARE MANAGER”界面点击“Auto Connect”，自动连接设备



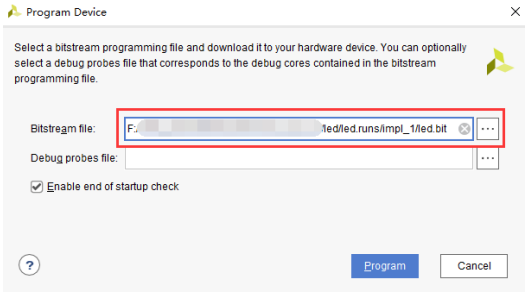
3) 可以看到 JTAG 扫描到 arm 和 FPGA 内核



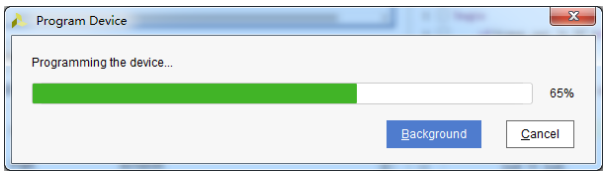
4) 选择芯片，右键“Program Device...”



5) 在弹出窗口中点击“Program”



6) 等待下载



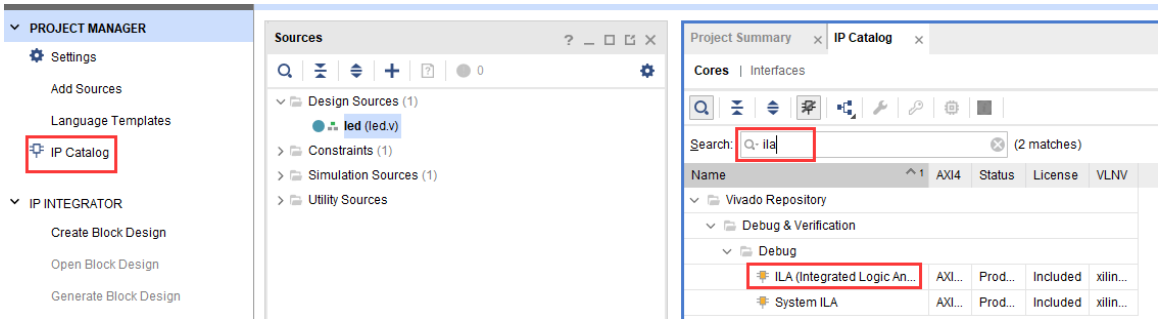
7) 下载完成以后，我们可以看到 PL LED 开始每秒变化一次。到此为止 Vivado 简单流程体验完成。后面的章节会介绍如果把程序烧录到 Flash，需要 PS 系统的配合才能完成，只有 PL 的工程不能直接烧写 Flash。在”体验 ARM，裸机输出”Hello World”一章的常见问题中有介绍。

2.9 在线调试

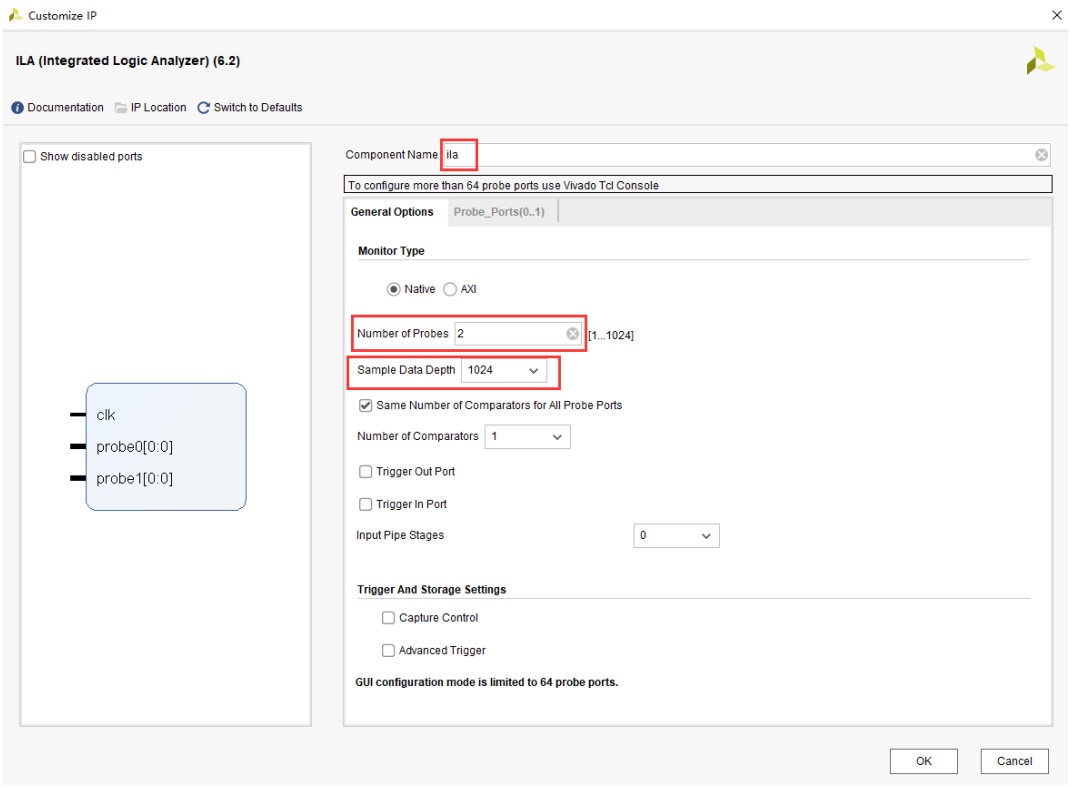
前面介绍了仿真和下载，但仿真并不需要程序烧写到板子，是比较理想化的结果，下面介绍 Vivado 在线调试方法，观察内部信号的变化。Vivado 有内嵌的逻辑分析仪，叫做 ILA，可以用于在线观察内部信号的变化，对于调试有很大帮助。在本实验中我们观察 timer_cnt 和 led 的信号变化。

2.9.1 添加 ILA IP 核

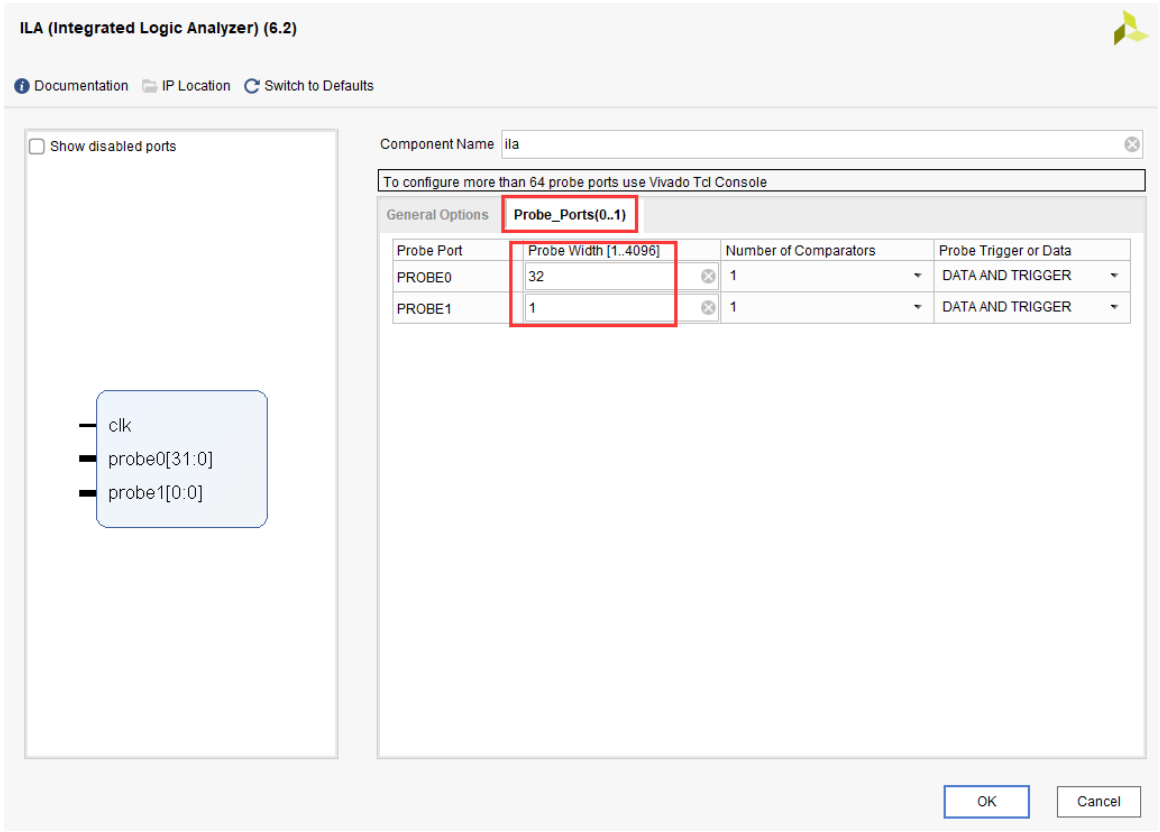
1) 点击 IP Catalog，在搜索框中搜索 ila，双击 ILA 的 IP



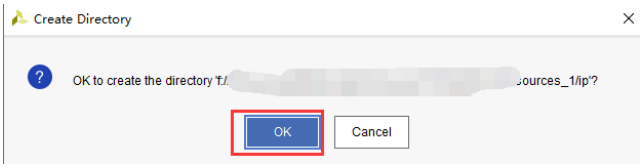
2) 修改名称为 ila，由于要采样两个信号，Probes 的数量设置为 2，Sample Data Depth 指的是采样深度，设置的越高，采集的信号越多，同样消耗的资源也会越多。



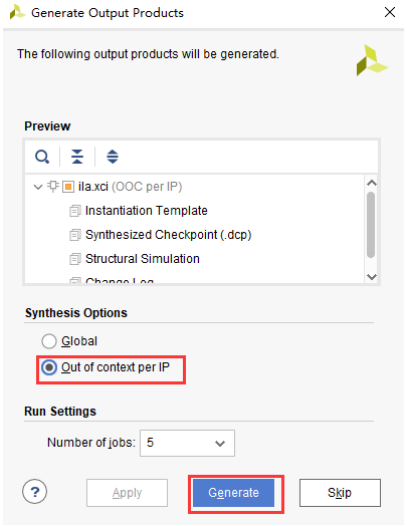
3）在 Probe_Ports 页面，设置 Probe 的宽度，设置 PROBE0 位宽为 32，用于采样 timer_cnt，设置 PROBE1 位宽为 1，用于采样 led。点击 OK



弹出界面，选择 OK



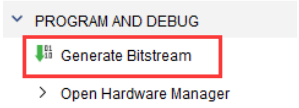
再如下设置，点击 Generate



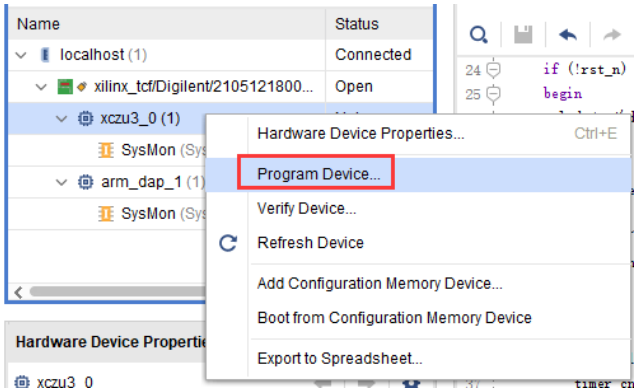
4） 在 led.v 中例化 ila，并保存

```
36      else if(timer_cnt >= 32'd49_999_999)
37      begin
38          led <= ~led;
39          timer_cnt <= 32'd0;
40      end
41      else
42      begin
43          led <= led;
44          timer_cnt <= timer_cnt + 32'd1;
45      end
46
47  end
48
49  //Instantiate ila in source file
50  ila ila_inst(
51    .clk(sys_clk),
52    .probe0(timer_cnt),
53    .probe1(led)
54    );
55
56  endmodule
57
```

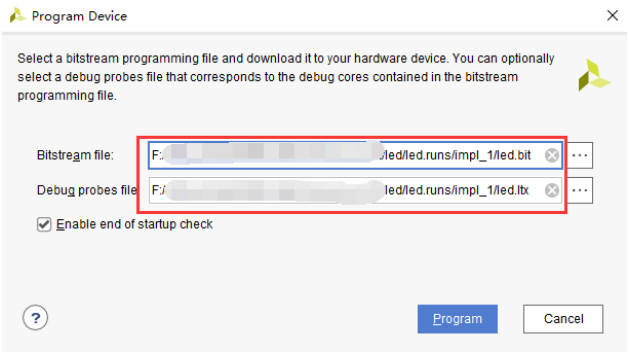
5） 重新生成 Bitstream



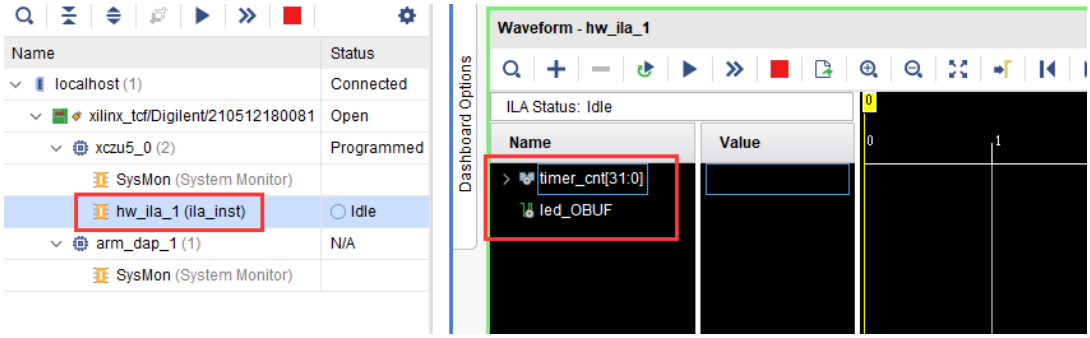
6） 下载程序



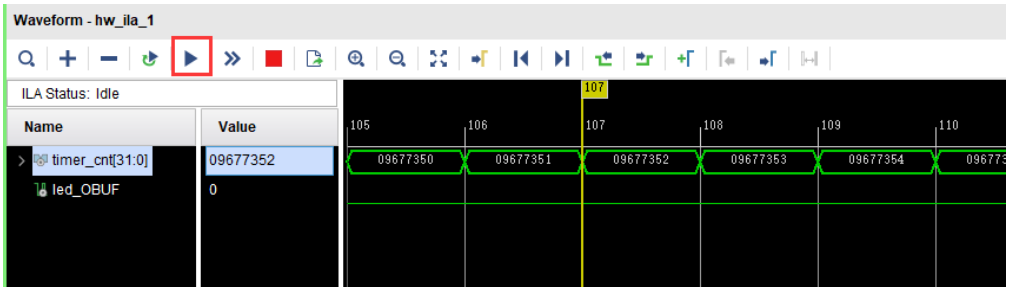
7） 这时候看到有 bit 和 ltx 文件，点击 program



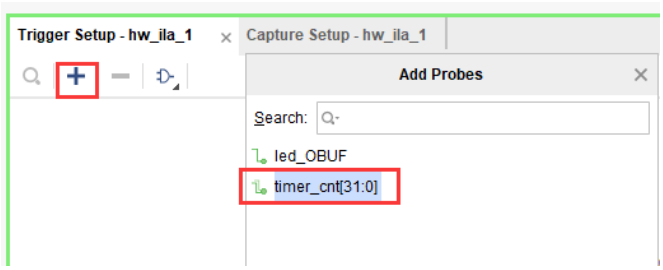
8） 此时弹出在线调试窗口，出现了我们添加的信号



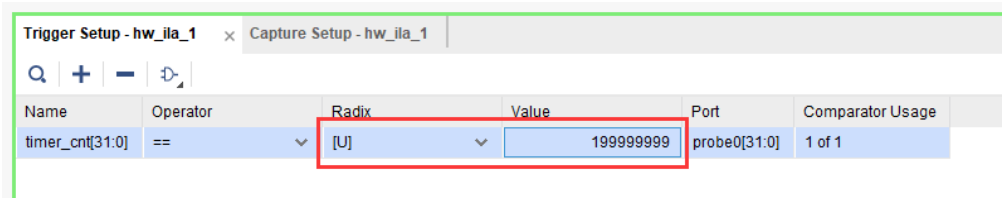
点击运行按钮，出现信号的数据



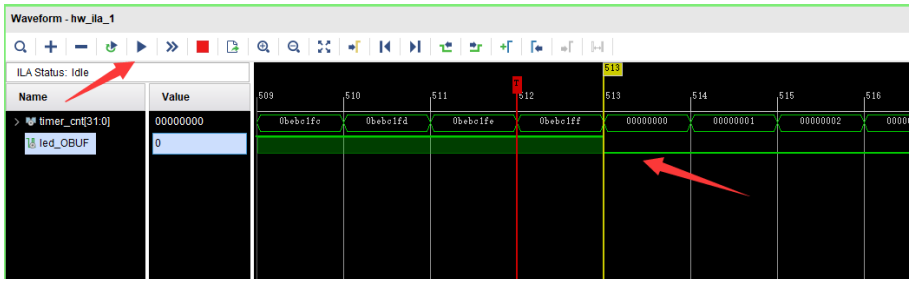
也可以触发采集，在 Trigger Setup 窗口点击“+”，深度选择 timer_cnt 信号



将 Radix 改为 U，也就是十进制，在 Value 中设置为 199999999，也就是 timer_cnt 计数的最大值



再次点击运行，即可以看到触发成功，此时 timer_cnt 显示为十六进制，而 led 也在此时翻转。



2.9.2 MARK DEBUG

上面介绍了添加 ILA IP 的方式在线调试，下面介绍在代码中添加综合属性，实现在线调试。

1） 首先打开 led.v，将 ila 的例化部分注释掉

Project Summary | led.v

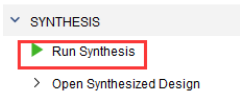
E:/.../led.srcs/sources_1/new/led.v

```
20      if (!rst_n)
21      begin
22        led <= 1'b0 ;
23        timer_cnt <= 32'd0 ;
24      end
25      else if(timer_cnt >= 32'd199_999_999)   //1 second counter, 200M-1=199999999
26      begin
27          led <= ~led;
28          timer_cnt <= 32'd0;
29      end
30      else
31      begin
32          led <= led;
33          timer_cnt <= timer_cnt + 32'd1;
34      end
35
36  end
37  //Instantiate ila in source file
38  //ila ila_inst(
39  //  .clk(sys_clk),
40  //  .probe0(timer_cnt),
41  //  .probe1(led)
42  //  );
43
44
45  endmodule
46
```

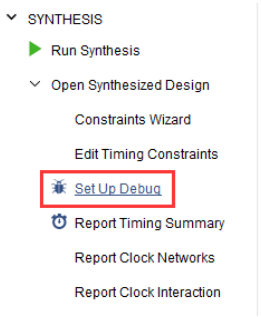
2） 在 led 和 timer_cnt 的定义前面添加(* MARK_DEBUG=”true” *)，保存文件。

```
`timescale 1ns / 1ps
module led(
//Differential system clock
    input sys_clk_p,
    input sys_clk_n,
    input rst_n,
(* MARK_DEBUG="true" *)    output reg  led
    );
(* MARK_DEBUG="true" *)reg[31:0] timer_cnt;
wire sys_clk ;
```

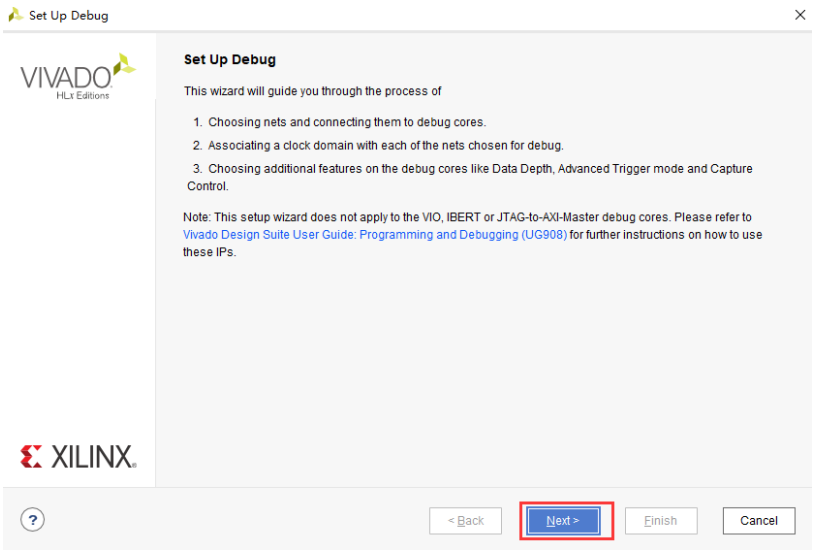
3） 点击综合



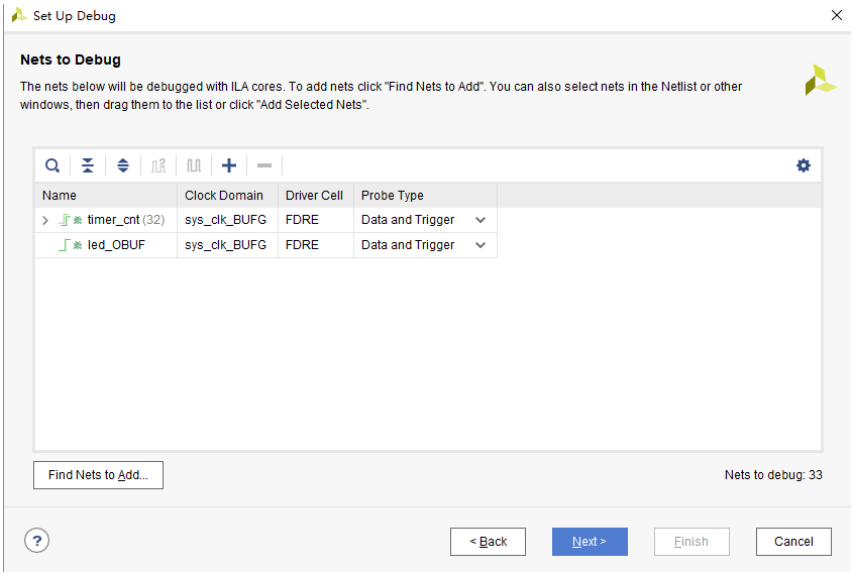
4） 综合结束后，点击 Set Up Debug



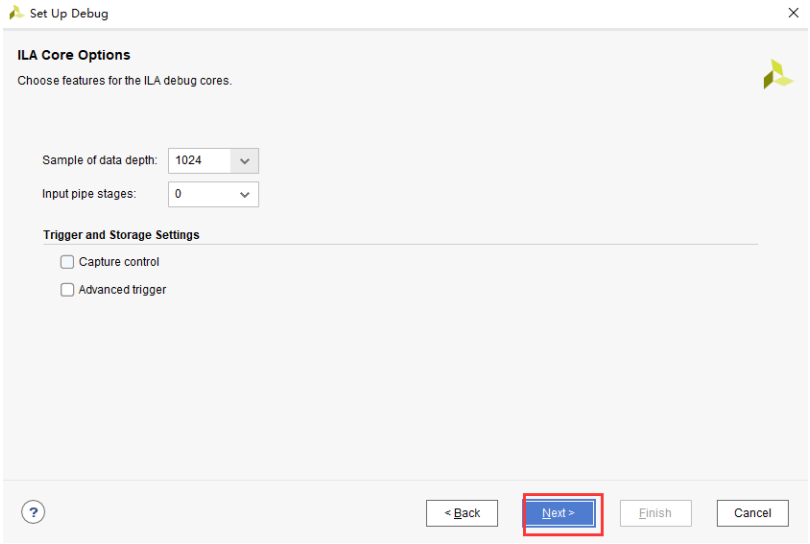
5） 弹出的窗口点击 Next



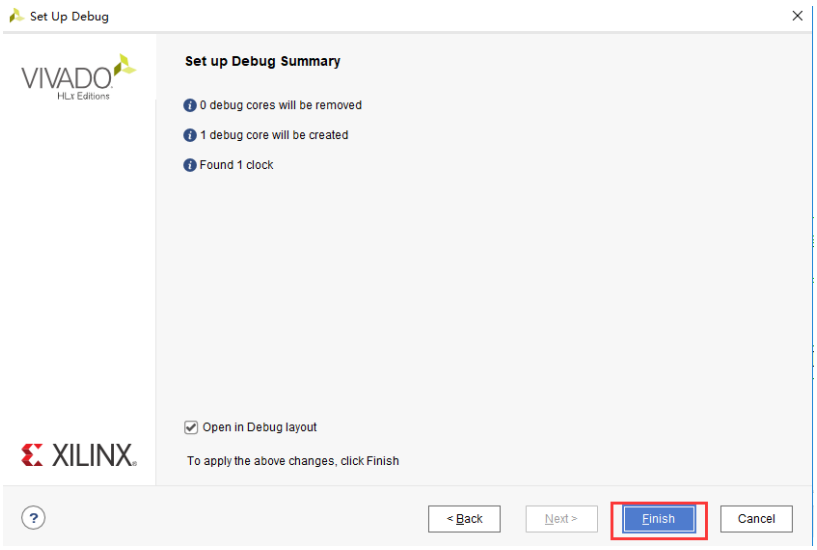
按照默认点击 Next



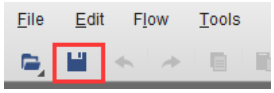
采样深度窗口，选择 Next



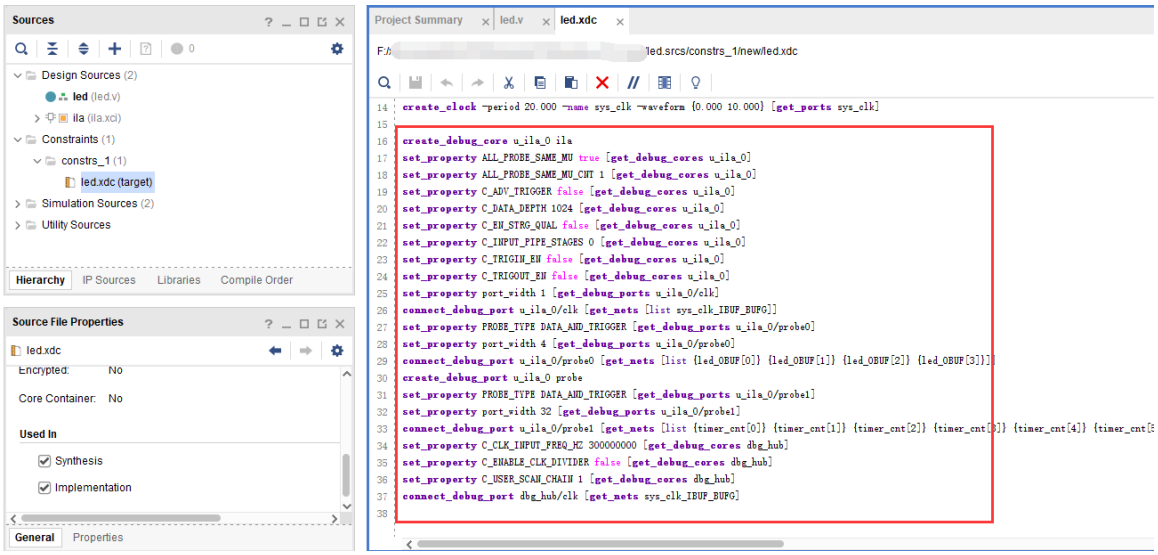
点击 Finish



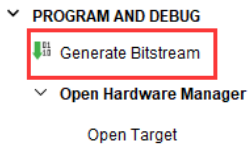
点击保存



在 xdc 文件中即可看到添加的 ila 核约束



5） 重新生成 bitstream



6） 调试方法与前面一样，不再赘述。

2.10 实验总结

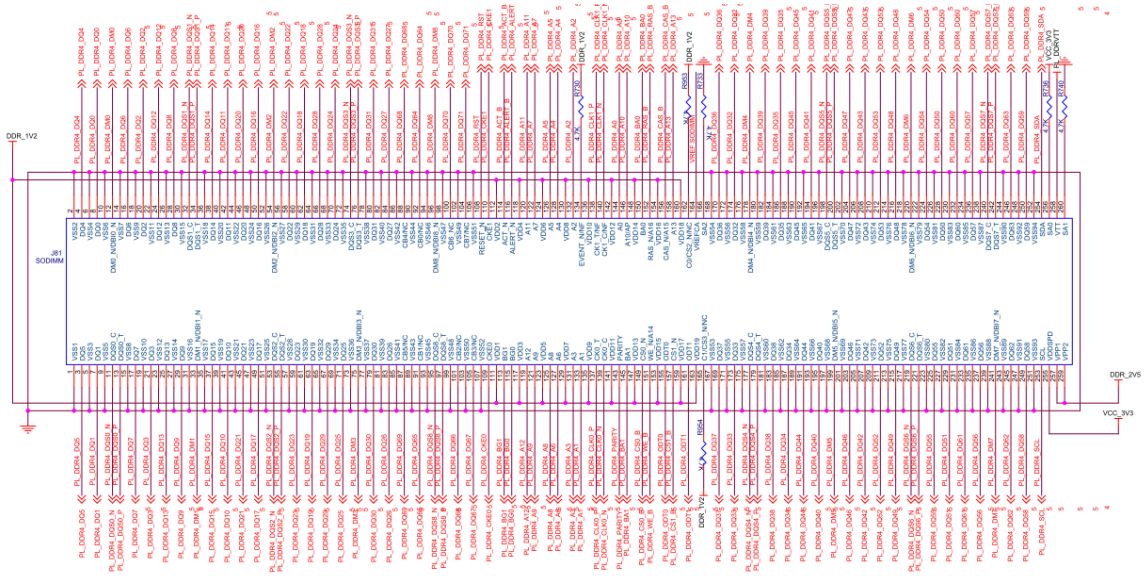
本章节介绍了如何在 PL 端开发程序，包括工程建立，约束，仿真，在线调试等方法，在后续的代码开发方式中皆可参考此方法。

第三章 PL 端 DDR4 读写测试实验

实验 Vivado 工程为“pl_ddr4_test”。

3.1 硬件介绍

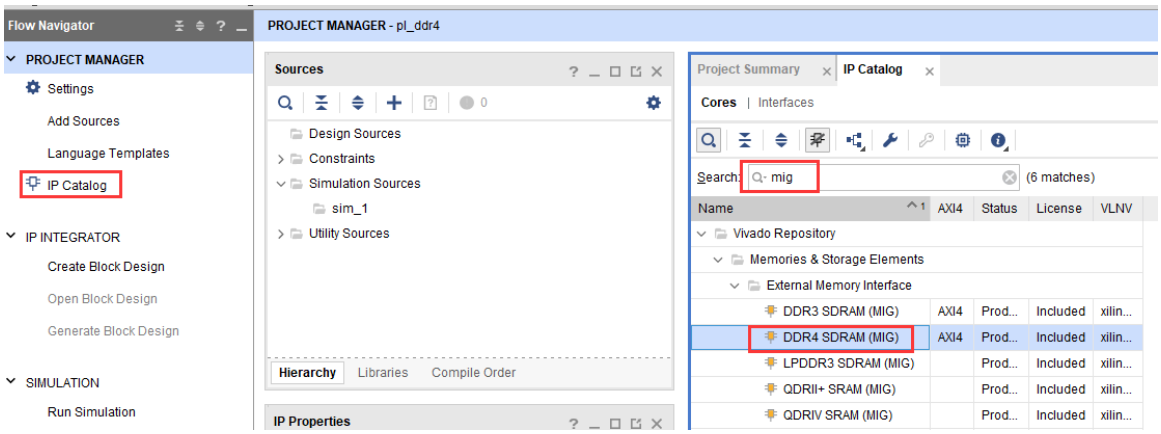
开发板的 PL 端有 SODIMM 插槽，这很大程度方便我们移植以前的 FPGA 工程到 ZYNQ 系统中，同时也提供了更大的带宽。



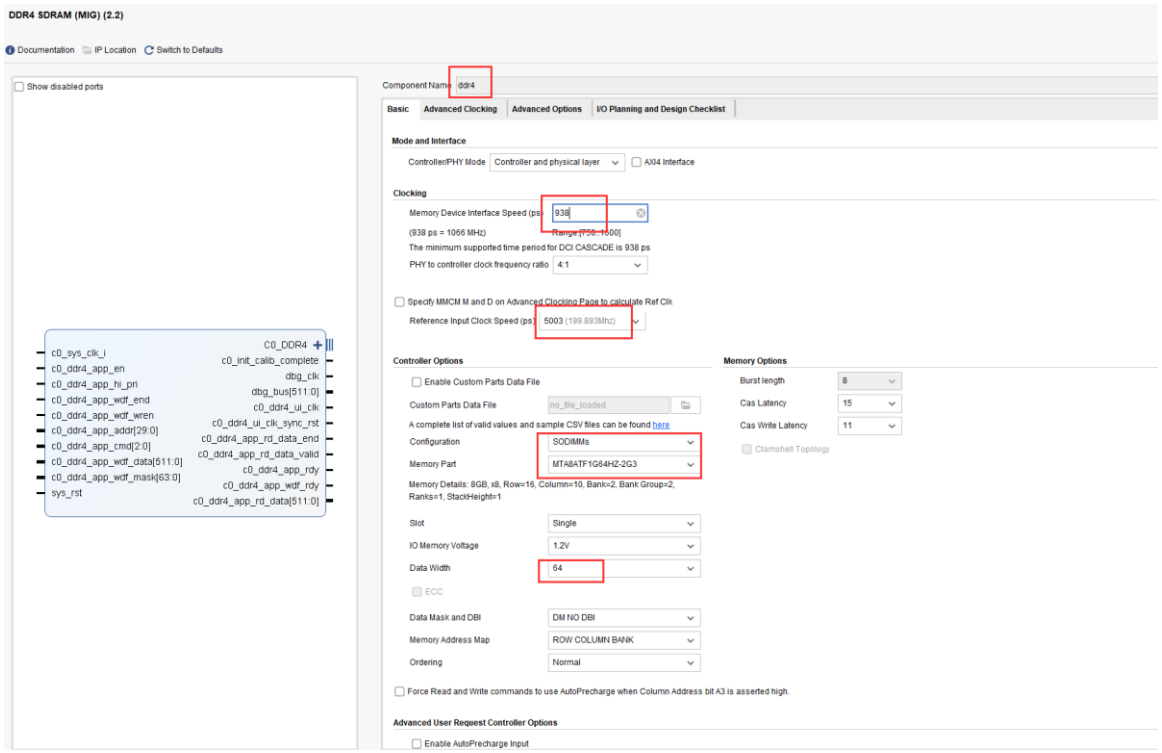
3.2 Vivado 工程建立

3.2.1 创建一个 PL 端 ddr4 测试工程并配置 ddr4 IP

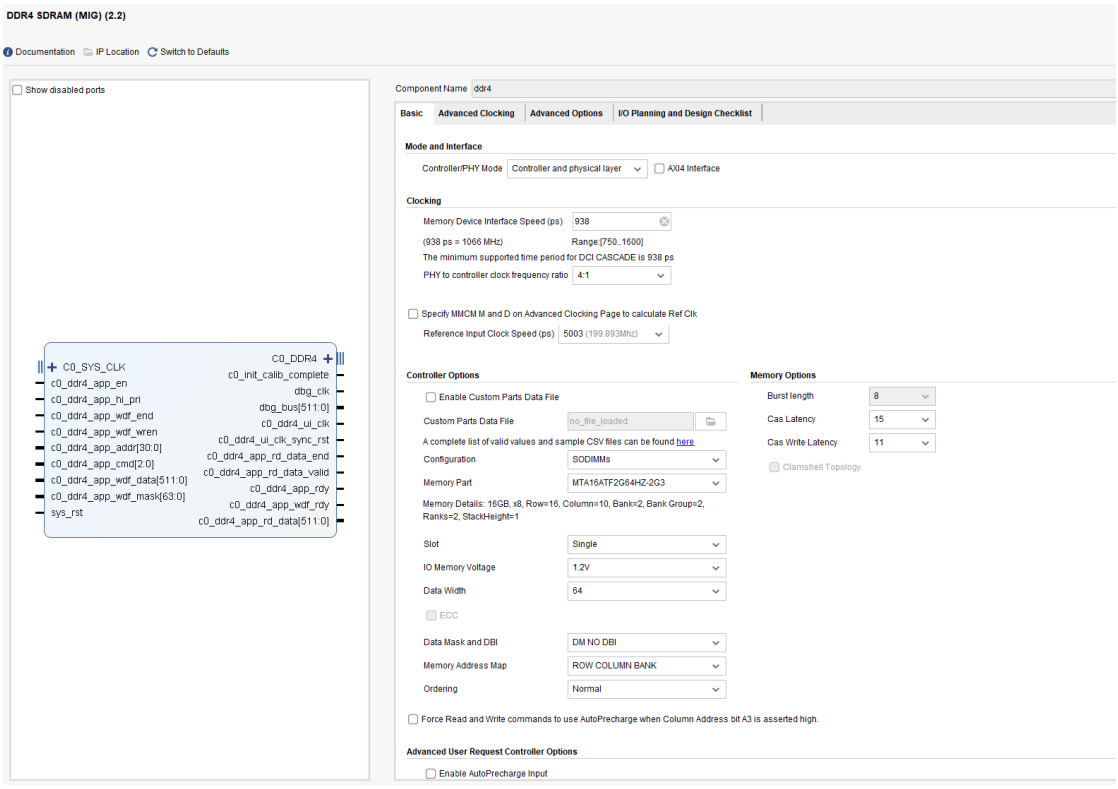
1) 在“IP Catalog”的搜索框搜索“mig”，快速找到“Memory Interface Generator”，双击



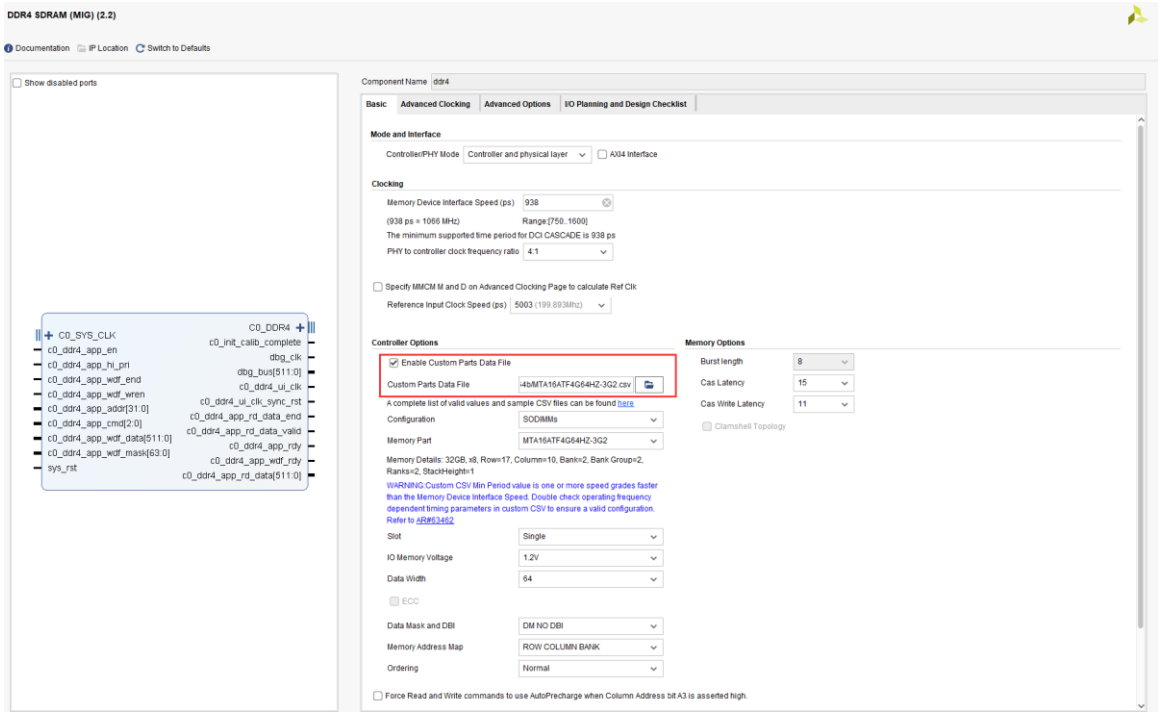
2) 我们选用了 MTA8ATF1G64HZ-2G3 型号的 SODIMM 条，Component Name 可以修改，Controller/PHY Mode 选择“Controller and physical layer”，先选择 Configration 为 SODIMMs，Momory Part 选择 ”MTA8ATF1G64HZ-2G3”，Memory Device Interface Speed(ps)选择 938,也就是 1066MHz。参考时钟选择 200MHz，即 5003ps，Data Width 选择 64，其他设置保持默认，点击 OK



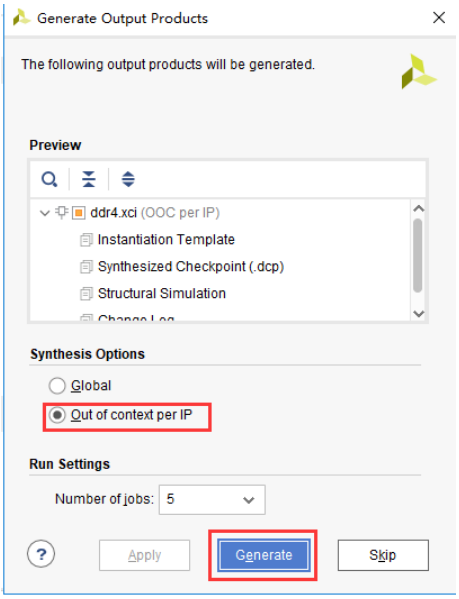
MTA16ATF2G64HZ-2G3 的配置如下



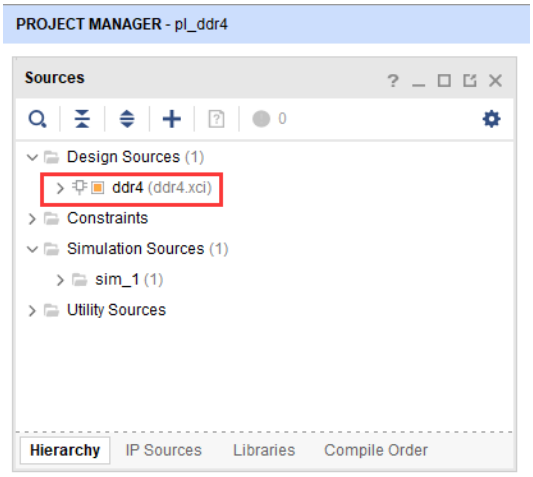
MTA16ATF4G64HZ-3G2 的配置如下，在工程下提供了 CSV 文件



3) Generate

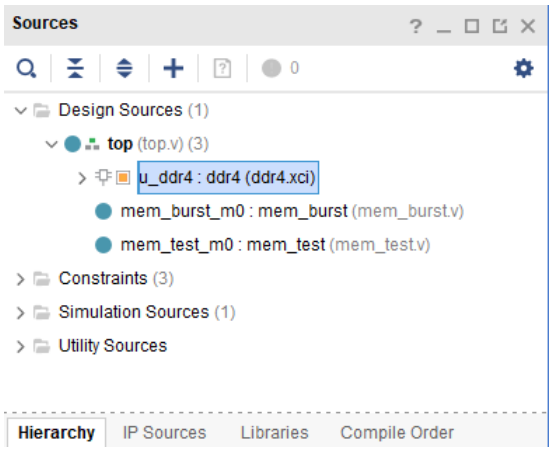


4) 生成结果如下

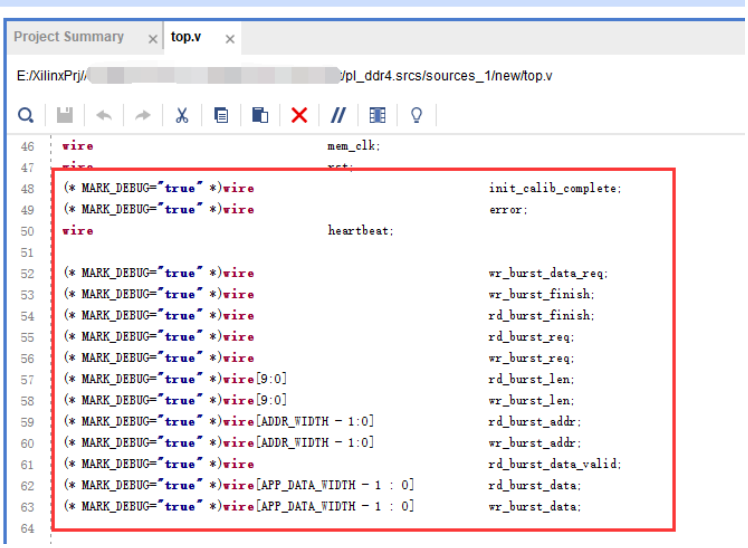


3.2.2 添加其他测试代码

其他代码主要功能是读写 ddr3 并比较数据是否一致，这里不做详细介绍，可参考工程代码。



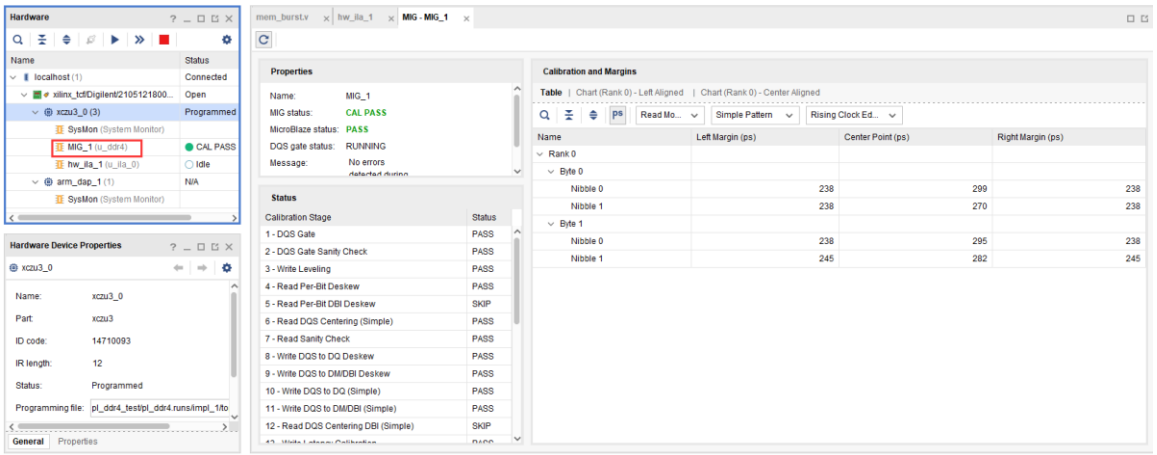
在 mem_test.v 中添加 mark_debug 调试，具体操作流程请参考 PL 的”Hello World”LED 实验



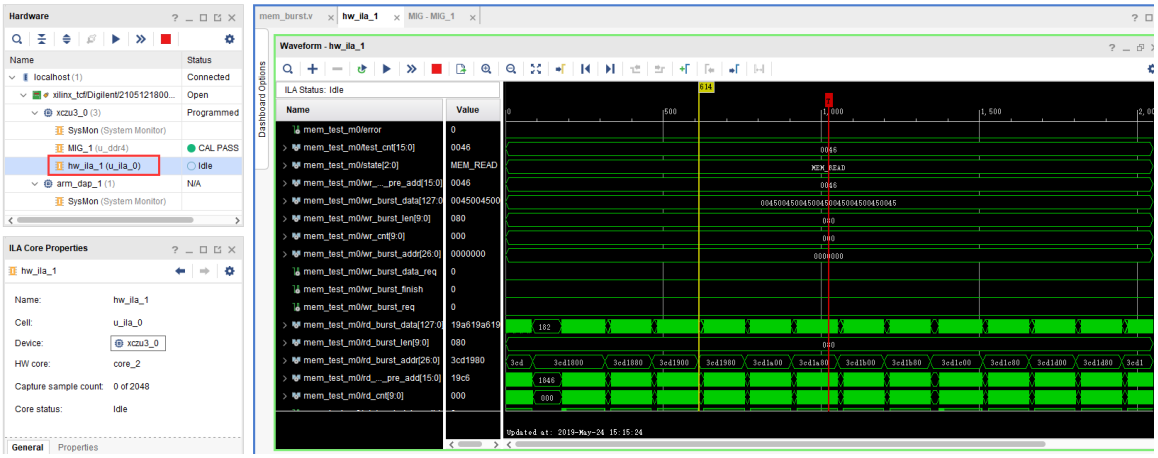
3.3 下载调试



生成 bit 文件以后，使用 JTAG 下载到开发板，在 MIG_1 窗口会显示 DDR4 校准等信息



在 hw_ila_1 中可以查看调试信号



3.4 实验总结

本实验通过 PL 端 Verilog 代码直接读写 ddr4，我们也可以把 ddr4 配置成 AXI 接口，这样方便和 ARM 系统完成数据交互。

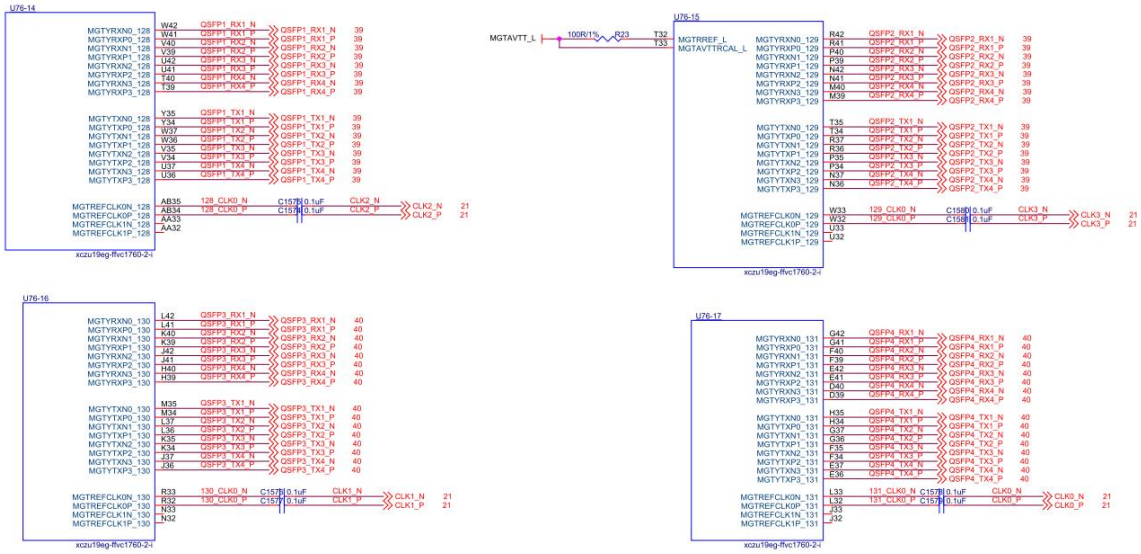
第四章 GTY 收发器误码率测试 IBERT 实验

实验 Vlvado 工程为“ibert_test”,目录中还有一个“ibert_ex”，是生成的测试工程。

Vidado 软件为我们提供了强大的误码率测试器 IBERT，不但可以测试误码率还能测试眼图，给我们使用高速收发器带来很大的便利，本实验做个抛砖引玉，简单介绍 IBERT 的使用。

4.1 硬件介绍

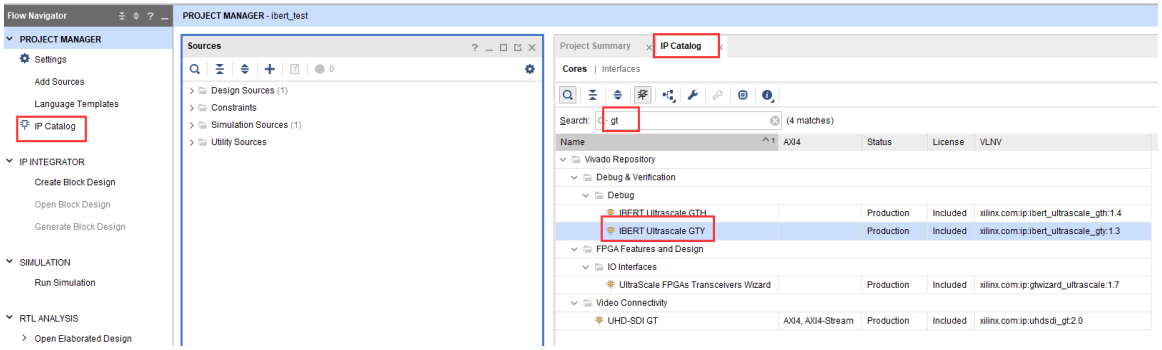
使用 IBERT 测试误码率和眼图必须有个收发环通的硬件，开发板上有 4 个 QSFP 光纤接口，也就是 4 个 100G 光口。本实验把 4 个光接口收发两两连接，形成 4 个收发环通链路。



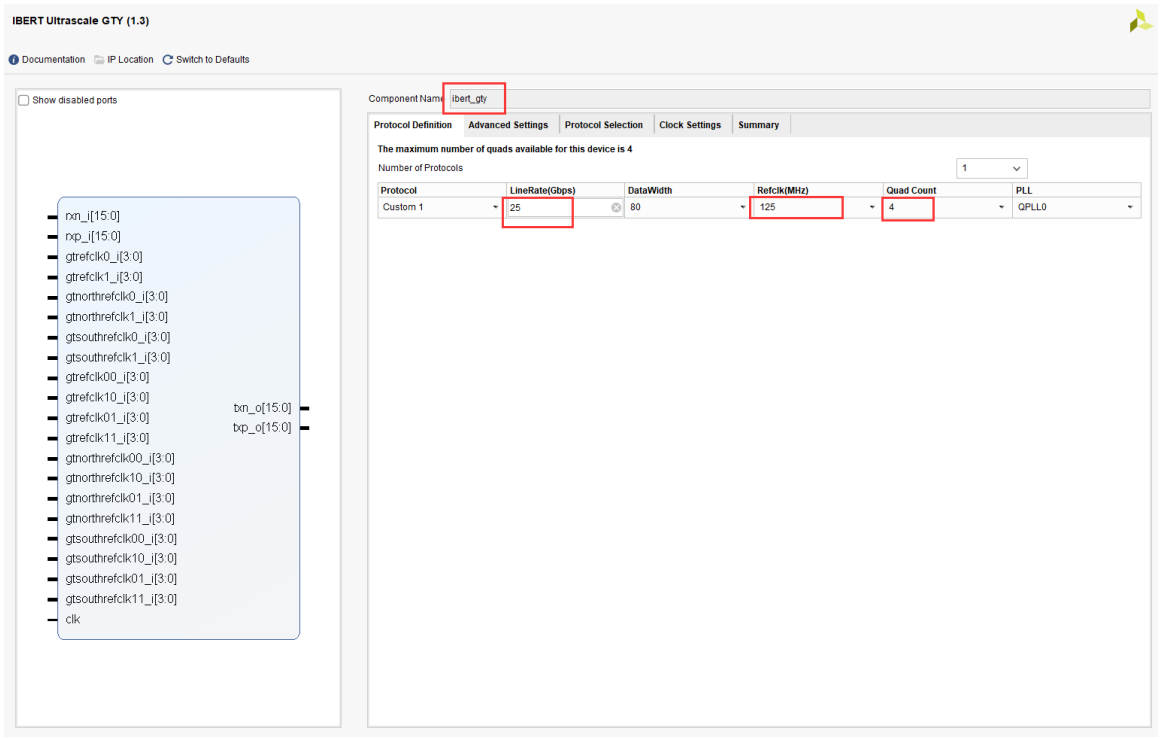
4.2 Vivado 工程建立

1) 新建一个工程名为“ibert_test”

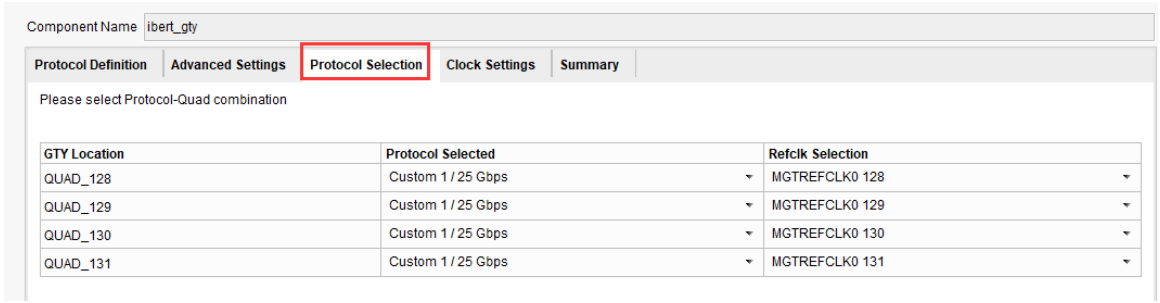
2) 在“IP Catalog”中搜索“ibert”快速找到“IBERT Ultrascale GTY”,双击



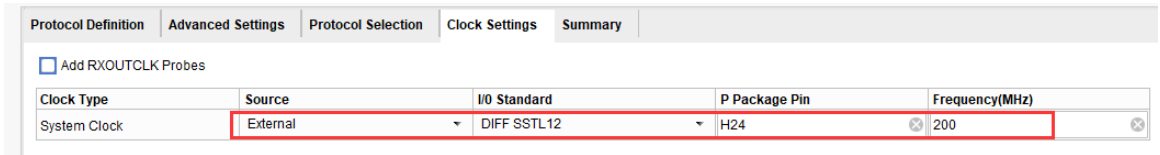
3) “Component Name”改为“ibert”，LineRate 填写 25G，Refclk（MHZ）选择 125.000，Quad Count 选择 4，因为有 4 个 Quad

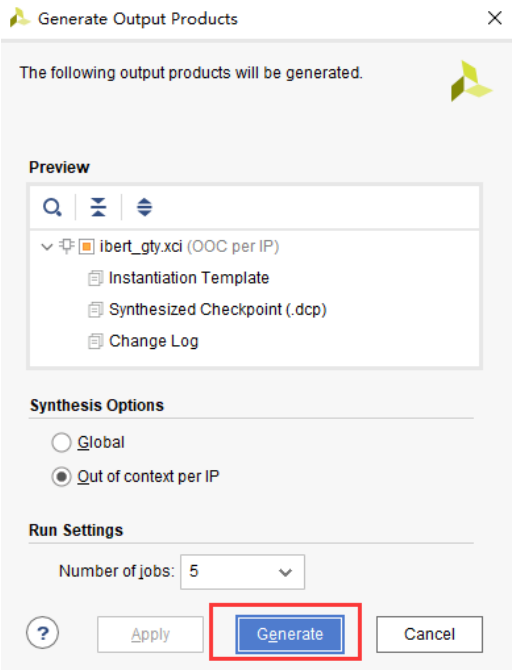


4) 在“Protocol Selected”中选择如下

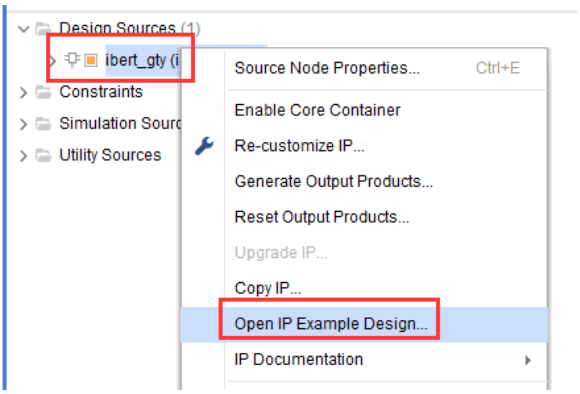


5) 在“Clock Settings”页选择“Source”选择“External”,用板子上的 200MHz 时钟，“I/O Standard”选择“DIFF SSTL12”，“P Package Pin”选择填入“H24”，“Frequency(MHz)”填入 200，点击“OK”，生成 IP 核

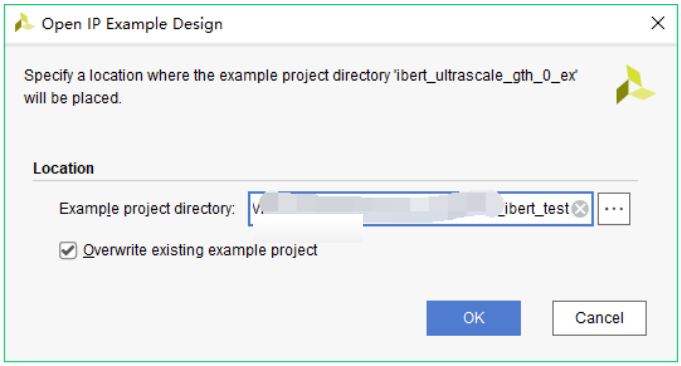




6) 生成 IP core 后等待一段时间后选择 IP，右键“Open IP Example Design...”,为 IP 生成一个测试例程，很多 IP 都带有测试例程。

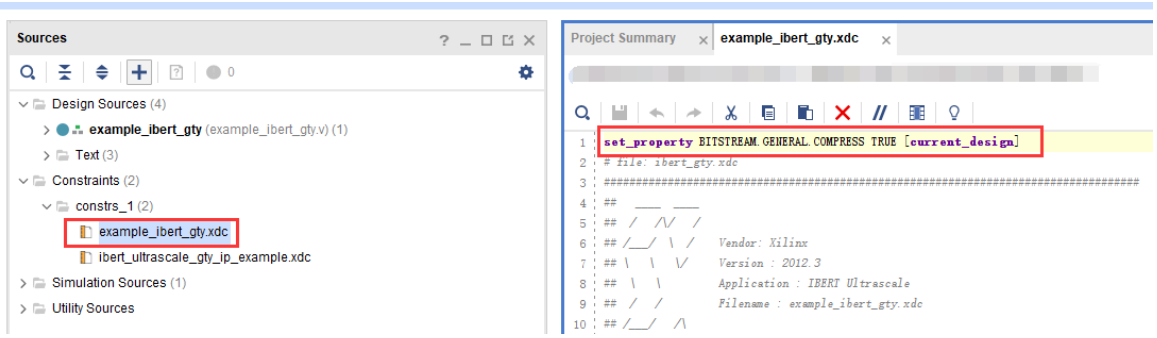


7) 选择测试工程的路径，点击“OK”



8) 在 XDC 中添加压缩 bit 的命令

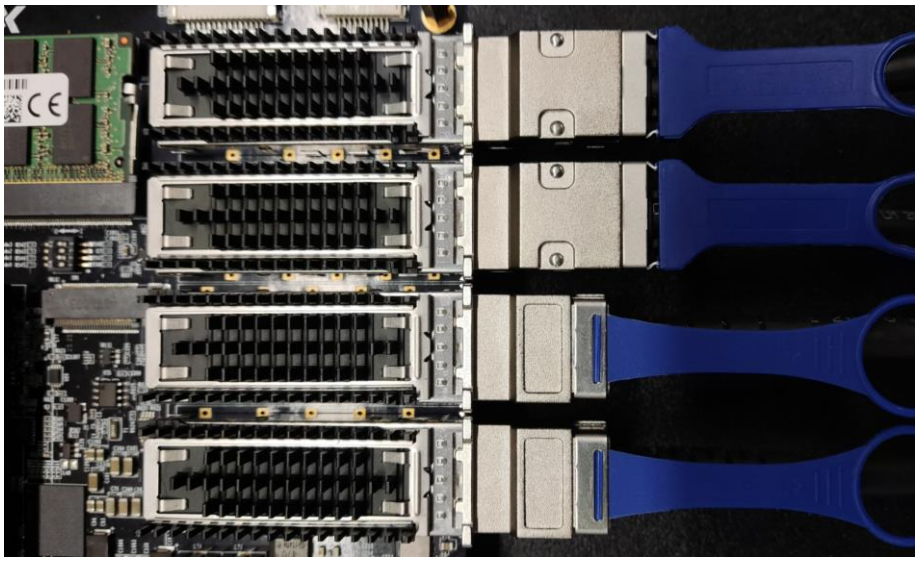
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]



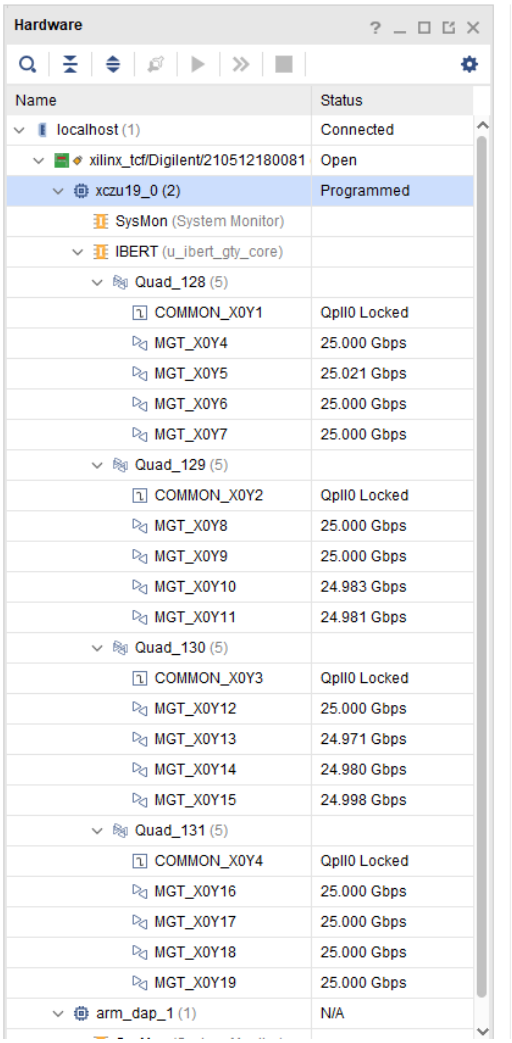
4.3 编译下载调试

1) 将 ibert 测试例程编译生成 BIT 文件

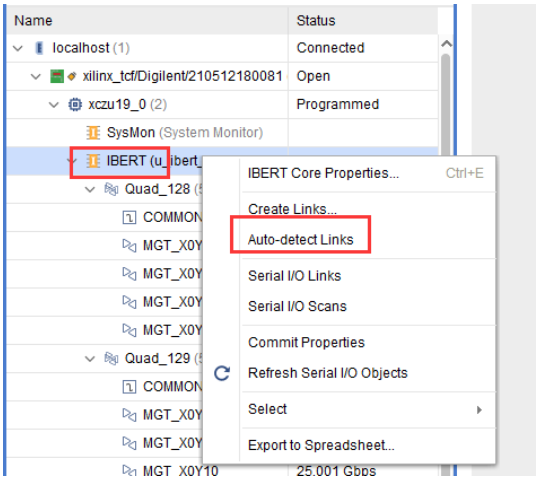
2) 插入光模块，然后使用光纤将 2 个光口对接，连接好 JTAG 下载线，给开发板上电



3) 使用 JTAG 下载 BIT 文件到开发板，可以看到有 4 个 Quad 链路锁定，速度 25Gbps。



4) 选择 IBERT，右键，选择“Auto-detect Links”

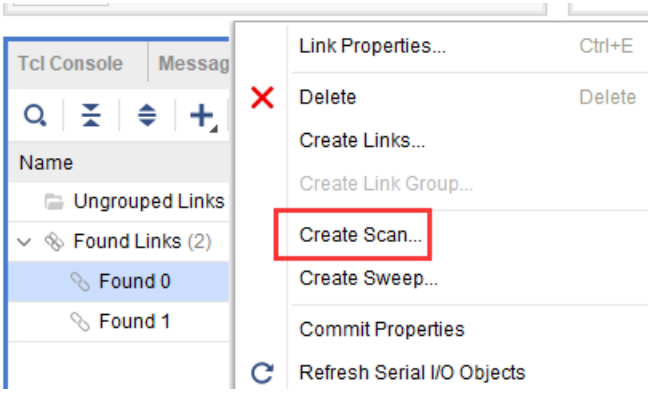


5) 点击“Serial I/O Links”,可以看到 Errors 都是 0，如果不是 0，可以点击“Reset”，重新开始测试。

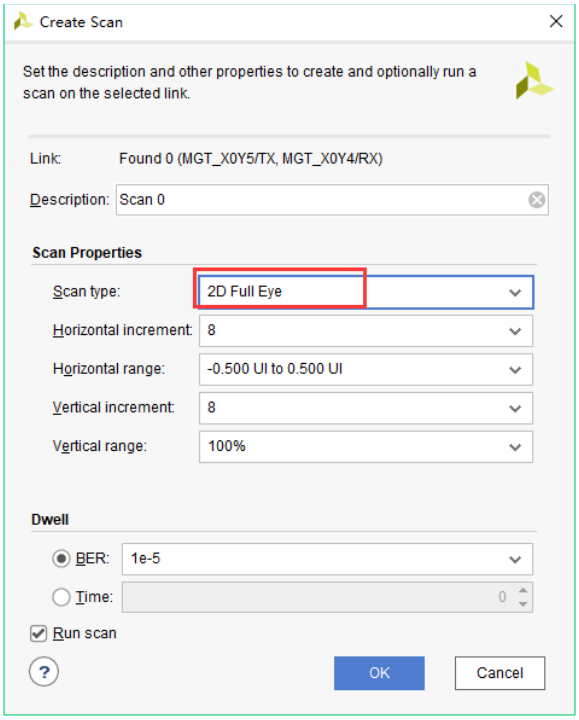
Tcl Console | Messages | Serial I/O Links | Serial I/O Scans

Name	TX	RX	Status	Bits	Errors	BER	BERT Reset	TX Pattern	RX Pattern
Ungrouped Links (0)									
Found Links (16)							Reset	PRBS 7-bit	PRBS 7-bit
Found 0	Quad_129/MGT_X0Y8/TX (xczu19_0)	Quad_128/MGT_X0Y4/RX (xczu19_0)	25.000 Gbps	5.143E11	0E0	1.944E-12	Reset	PRBS 7-bit	PRBS 7-bit
Found 1	Quad_129/MGT_X0Y9/TX (xczu19_0)	Quad_128/MGT_X0Y5/RX (xczu19_0)	25.030 Gbps	5.143E11	0E0	1.944E-12	Reset	PRBS 7-bit	PRBS 7-bit
Found 2	Quad_129/MGT_X0Y10/TX (xczu19_0)	Quad_128/MGT_X0Y6/RX (xczu19_0)	25.000 Gbps	5.144E11	0E0	1.944E-12	Reset	PRBS 7-bit	PRBS 7-bit
Found 3	Quad_129/MGT_X0Y11/TX (xczu19_0)	Quad_128/MGT_X0Y7/RX (xczu19_0)	25.000 Gbps	5.145E11	0E0	1.944E-12	Reset	PRBS 7-bit	PRBS 7-bit
Found 4	Quad_128/MGT_X0Y4/TX (xczu19_0)	Quad_129/MGT_X0Y8/RX (xczu19_0)	25.048 Gbps	5.142E11	0E0	1.945E-12	Reset	PRBS 7-bit	PRBS 7-bit
Found 5	Quad_128/MGT_X0Y5/TX (xczu19_0)	Quad_129/MGT_X0Y9/RX (xczu19_0)	25.000 Gbps	5.143E11	0E0	1.944E-12	Reset	PRBS 7-bit	PRBS 7-bit
Found 6	Quad_128/MGT_X0Y6/TX (xczu19_0)	Quad_129/MGT_X0Y10/RX (xczu19_0)	25.000 Gbps	5.144E11	0E0	1.944E-12	Reset	PRBS 7-bit	PRBS 7-bit
Found 7	Quad_128/MGT_X0Y7/TX (xczu19_0)	Quad_129/MGT_X0Y11/RX (xczu19_0)	25.000 Gbps	5.144E11	0E0	1.944E-12	Reset	PRBS 7-bit	PRBS 7-bit
Found 8	Quad_131/MGT_X0Y16/TX (xczu19_0)	Quad_130/MGT_X0Y12/RX (xczu19_0)	24.991 Gbps	5.144E11	0E0	1.944E-12	Reset	PRBS 7-bit	PRBS 7-bit
Found 9	Quad_131/MGT_X0Y17/TX (xczu19_0)	Quad_130/MGT_X0Y13/RX (xczu19_0)	25.000 Gbps	5.145E11	0E0	1.944E-12	Reset	PRBS 7-bit	PRBS 7-bit
Found 10	Quad_131/MGT_X0Y18/TX (xczu19_0)	Quad_130/MGT_X0Y14/RX (xczu19_0)	25.021 Gbps	5.146E11	0E0	1.943E-12	Reset	PRBS 7-bit	PRBS 7-bit
Found 11	Quad_131/MGT_X0Y19/TX (xczu19_0)	Quad_130/MGT_X0Y15/RX (xczu19_0)	25.000 Gbps	5.146E11	0E0	1.943E-12	Reset	PRBS 7-bit	PRBS 7-bit
Found 12	Quad_130/MGT_X0Y12/TX (xczu19_0)	Quad_131/MGT_X0Y16/RX (xczu19_0)	25.000 Gbps	5.144E11	0E0	1.944E-12	Reset	PRBS 7-bit	PRBS 7-bit
Found 13	Quad_130/MGT_X0Y13/TX (xczu19_0)	Quad_131/MGT_X0Y17/RX (xczu19_0)	25.000 Gbps	5.145E11	0E0	1.944E-12	Reset	PRBS 7-bit	PRBS 7-bit
Found 14	Quad_130/MGT_X0Y14/TX (xczu19_0)	Quad_131/MGT_X0Y18/RX (xczu19_0)	25.000 Gbps	5.145E11	0E0	1.944E-12	Reset	PRBS 7-bit	PRBS 7-bit
Found 15	Quad_130/MGT_X0Y15/TX (xczu19_0)	Quad_131/MGT_X0Y19/RX (xczu19_0)	24.998 Gbps	5.146E11	0E0	1.943E-12	Reset	PRBS 7-bit	PRBS 7-bit

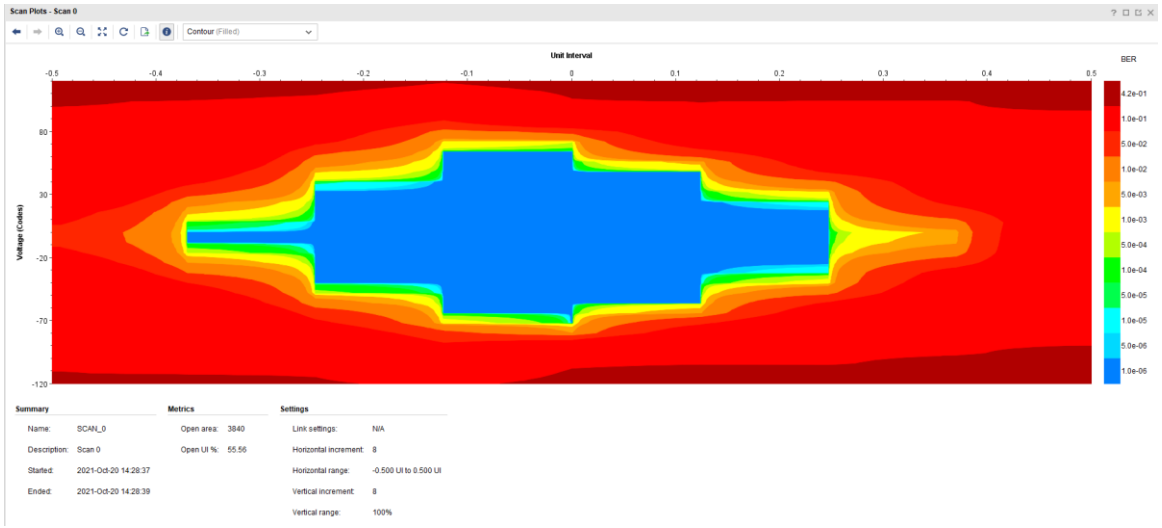
6) 选择一个链路，右键“Create Scan...”



7) 选择 2D Full Eye



8) 默认配置出来的眼图，注意：使用不同的软件版本，测量眼图可能会有差异。



4.4 实验总结

IBERT 提供了强大的调试功能，本实验仅仅展示了部分功能，更详细的使用参考 xilinx 文档 pg173。

PS 端外设篇

基础篇主要介绍 ZYNQ 核的配置，PS 端的应用，如 MIO，以太网，RTC 等的使用基础实验，为后面的应用打下基础。

第五章 体验 ARM，裸机输出“Hello World”

vivado 工程目录为“ps_hello/vivado”
vitis 工程目录为“ps_hello/vitis”

从本章开始由 FPGA 工程师与软件开发工程师协同实现。

前面的实验都是在 PL 端进行的，可以看到和普通 FPGA 开发流程没有任何区别，ZYNQ 的主要优势就是 FPGA 和 ARM 的合理结合，这对开发人员提出了更高的要求。从本章开始，我们开始使用 ARM，也就是我们说的 PS，本章我们使用一个简单的串口打印来体验一下 Vivado Vitis 和 PS 端的特性。

前面的实验都是 FPGA 工程师应该做的事情，从本章节开始就有了分工，FPGA 工程师负责把 Vivado 工程搭建好，提供好硬件给软件开发人员，软件开发人员便能在这个基础上开发应用程序。做好分工，也有利于项目的推进。如果是软件开发人员想把所有的事情都做了，可能需要花费很多时间和精力去学习 FPGA 的知识，由软件思维转成硬件思维是个比较痛苦的过程，如果纯粹的学习，又有时间，就另当别论了。专业的人做专业的事，是个很好的选择。

5.1 硬件介绍

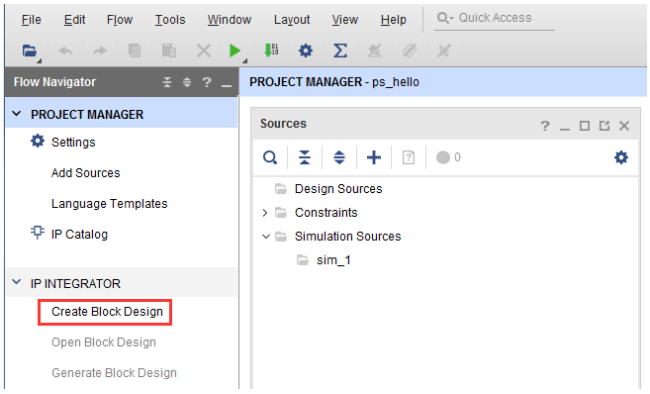
我们从原理图中可以看到 ZYNQ 芯片分为 PL 和 PS，PS 端的 IO 分配相对是固定的，不能任意分配，而且不需要在 Vivado 软件里分配管脚，虽然本实验仅仅使用了 PS，但是还要建立一个 Vivado 工程，用来配置 PS 管脚。虽然 PS 端的 ARM 是硬核，但是在 ZYNQ 当中也要将 ARM 硬核添加到工程当中才能使用。前面章节介绍的是代码形式的工程，本章开始介绍 ZYNQ 的图形化方式建立工程。

FPGA 工程师工作内容

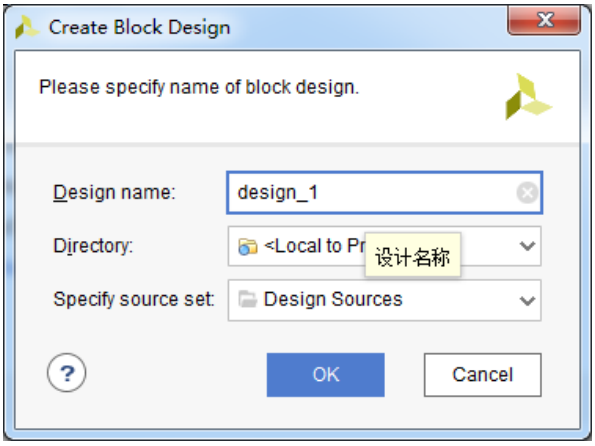
下面介绍 FPGA 工程师负责内容。

5.2 Vivado 工程建立

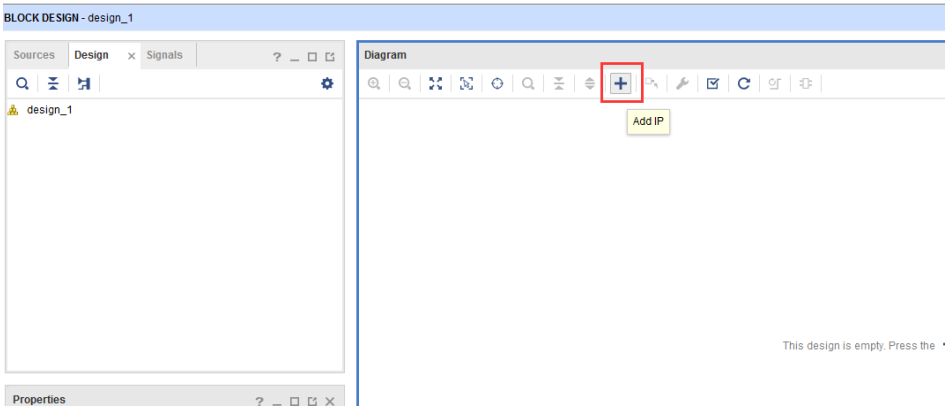
1) 创建一个名为“ps_hello”的工程，建立过程不再赘述，参考“PL 的”Hello World”LED 实验”。
2) 点击“Create Block Design”，创建一个 Block 设计，也就是图形化设计



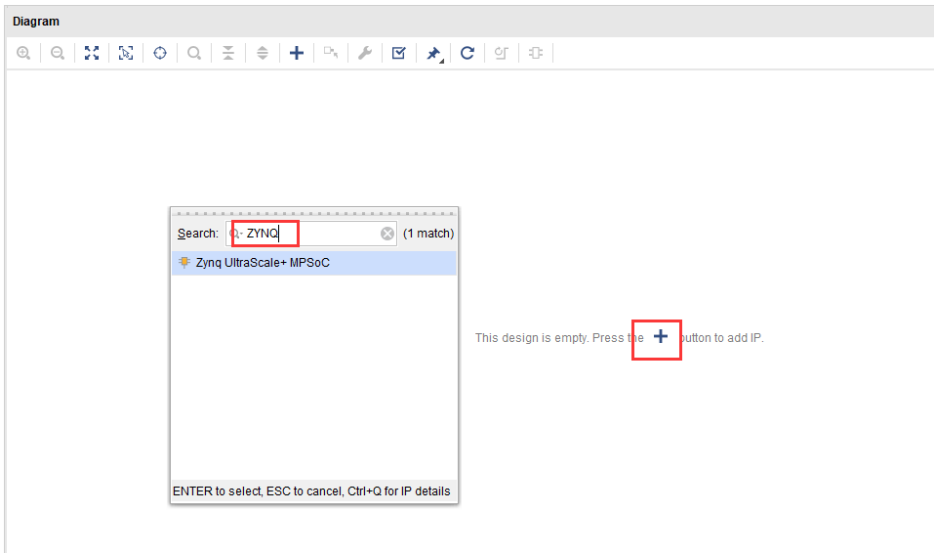
3) “Design name”这里不做修改，保持默认“design_1”，这里可以根据需要修改，不过名字要尽量简短，否则在 Windows 下编译会有问题。



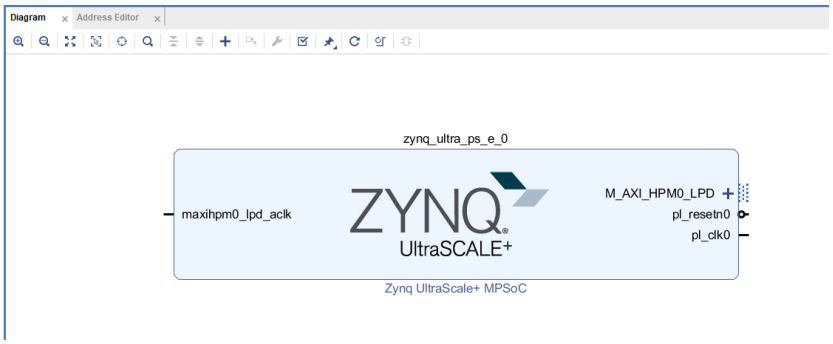
4) 点击“Add IP”快捷图标



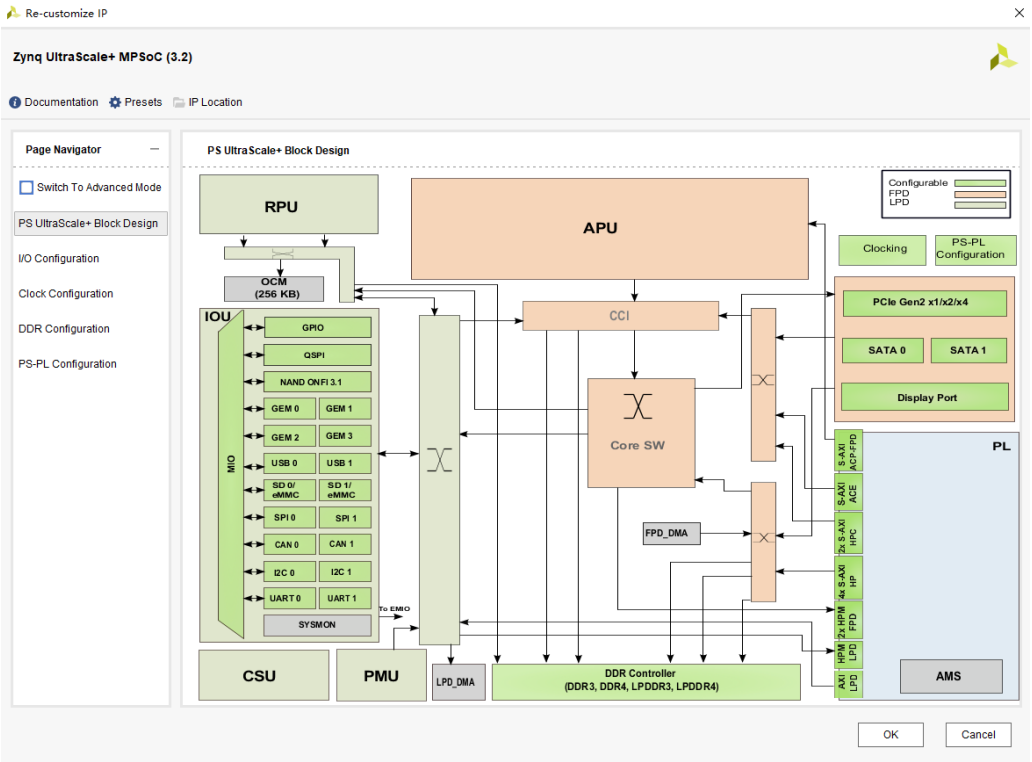
5) 搜索“zynq”，在搜索结果列表中双击”Zynq UltraScale+ MPSoC”



6) 双击 Block 图中的 ZYNQ 核，配置相关参数

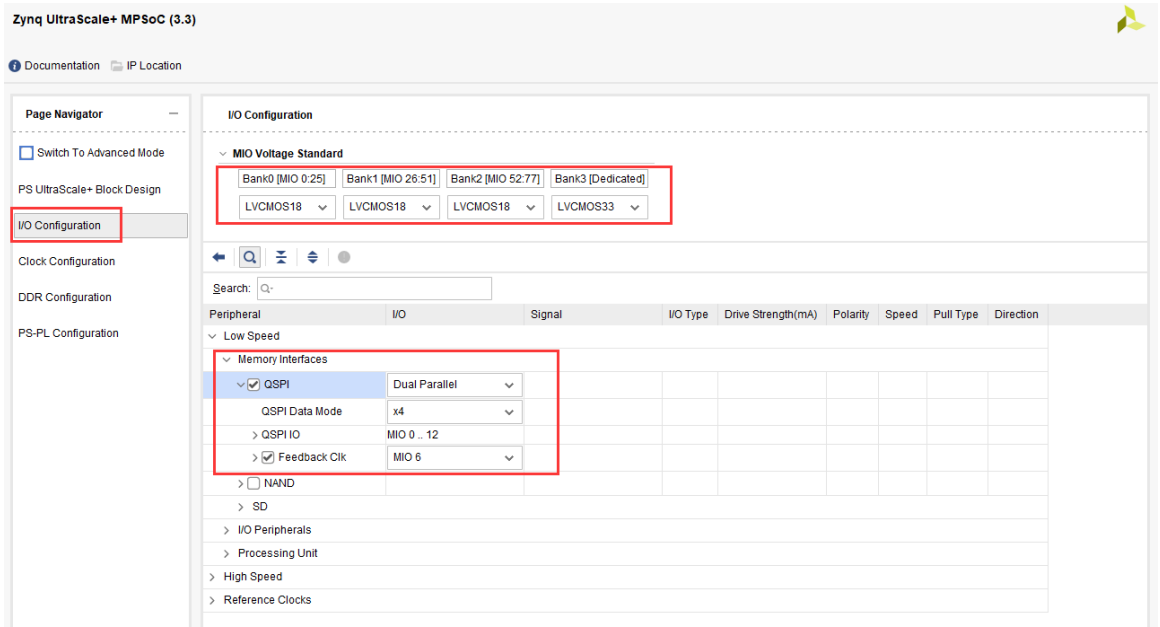


7) 首先出现的界面是 ZYNQ 硬核的架构图，可以很清楚看到它的结构，可以参考 ug1085 文档，里面有对 ZYNQ 的详细介绍。图中绿色部分是可配置模块，可以点击进入相应的编辑界面，当然也可以在左侧的窗口进入编辑。下面对各个窗口的功能一一介绍。

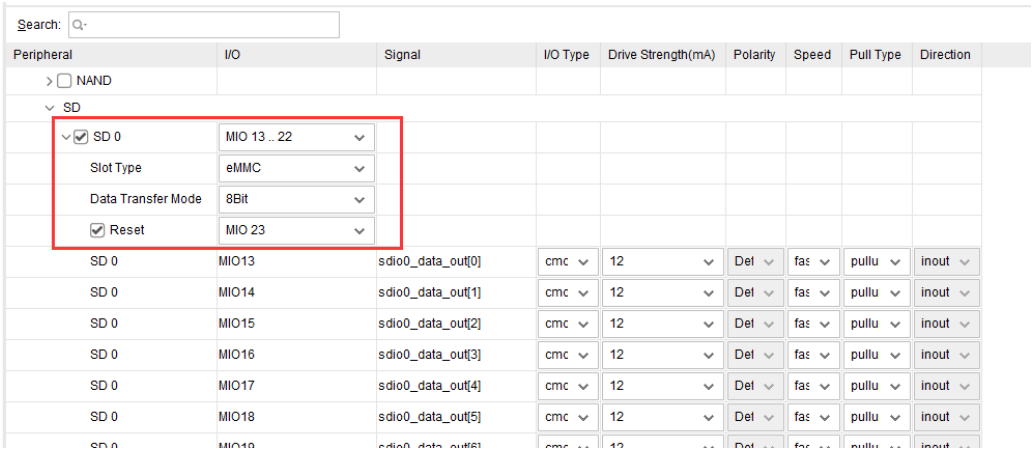


1.1.1 Low Speed 配置

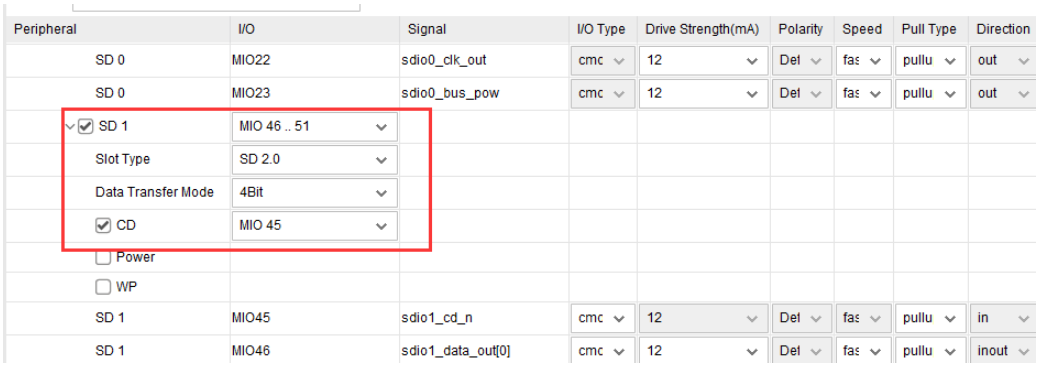
1） 在 I/O Configuration 窗口，配置 BANK0~BANK2 电压为 LVCMOS18，BANK3 电压为 LVCMOS33。首先配置 Low Speed 管脚，勾选 QSPI，并设置为”Dual Parallel”模式，Data Mode 为”x4“，勾选 Feedback Clk



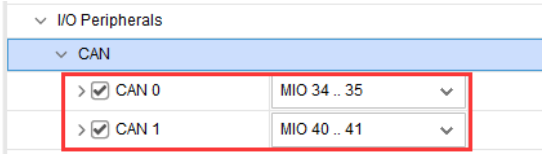
2） 勾选 SD 0，配置 eMMC。选择 MIO13..22，Slot Type 选择 eMMC，Data Transfer Mode 为 8Bit，勾选 Reset，并选择 MIO23。



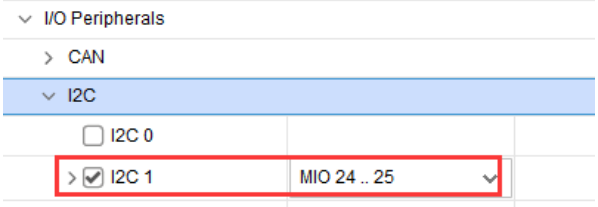
3） 勾选 SD 1，配置 SD 卡。选择 MIO 46..51，Slot Type 选择 SD 2.0，Data Transfer Mode 选择 4Bit，勾选 CD，用于检测 SD 卡插入，选择 MIO45



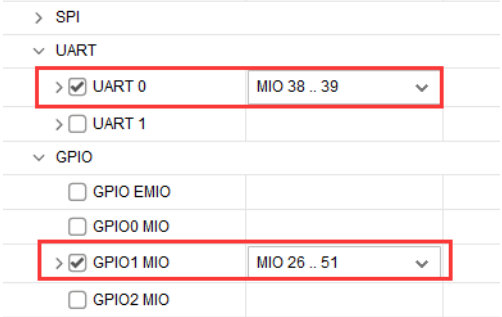
4） 勾选 CAN 0，选择 MIO 34..35，勾选 CAN 1，选择 MIO 40..41



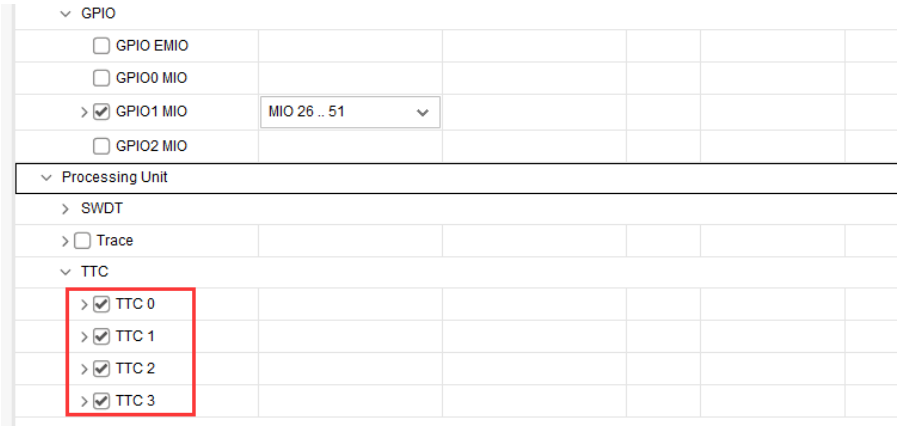
5） 勾选 I2C 1，用于 EEPROM 等的 I2C，选择 MIO24..25



6） 勾选串口 UART 0，选择 MIO 38..39，勾选 GPIO1 MIO

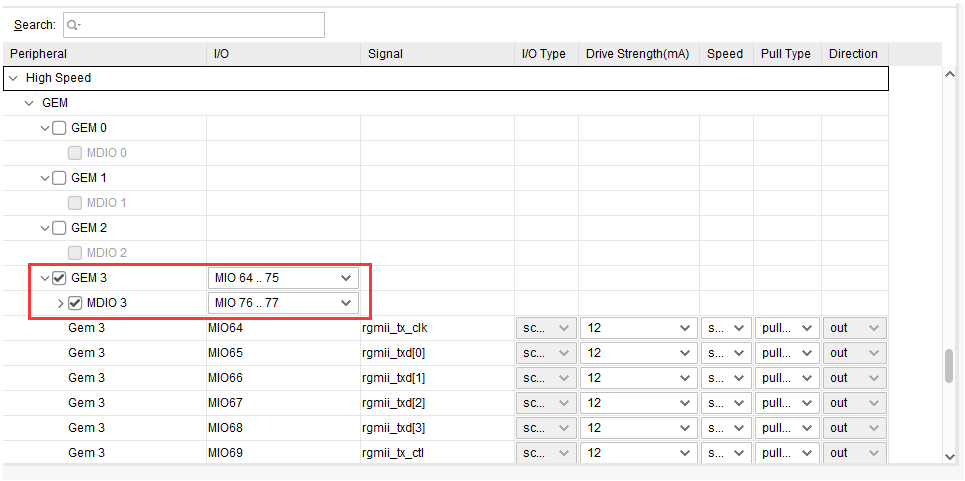


7） 勾选 TTC 0~TTC 3

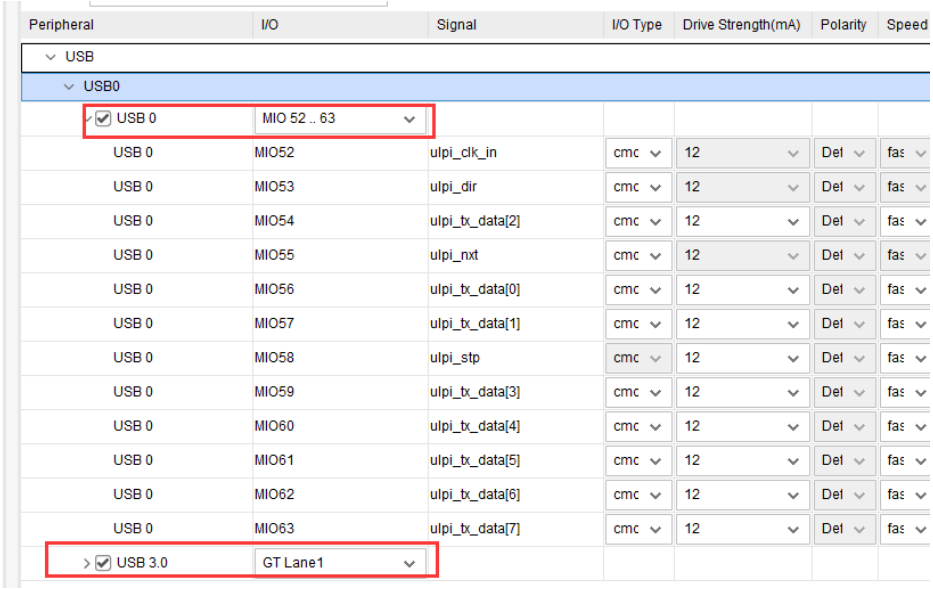


1.1.2 High Speed 配置

1） High Speed 部分首先配置 PS 端以太网，勾选 GEM 3，选择 MIO 64..75，勾选 MDIO 3，选择 MIO 76..77



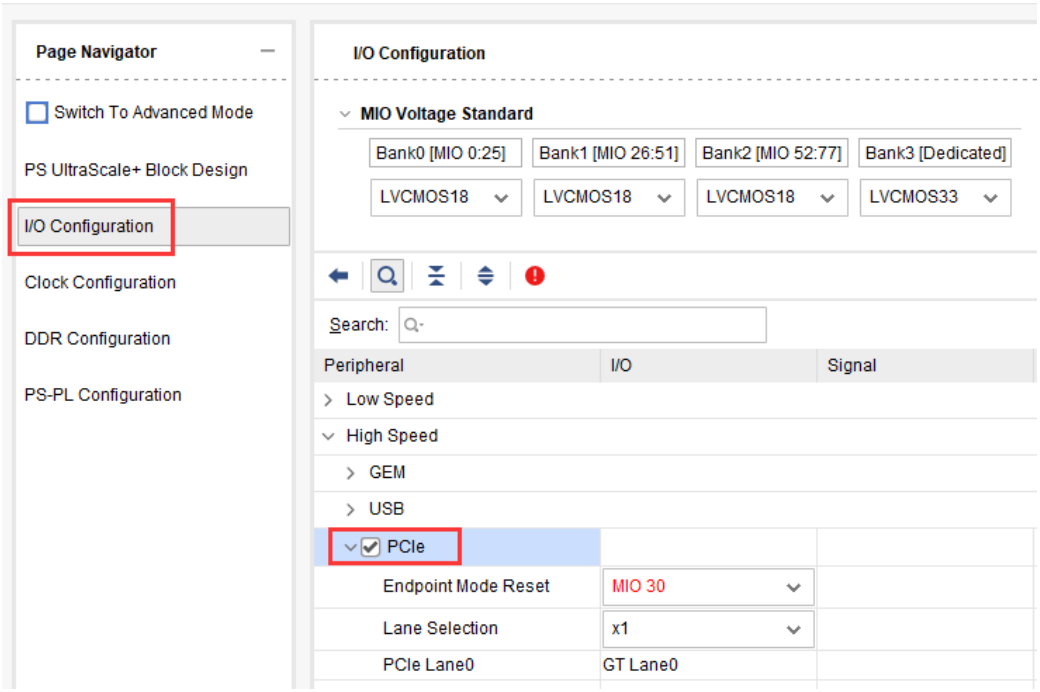
2） 勾选 USB 0，选择 MIO 52..63，勾选 USB 3.0，选择 GT Lane1



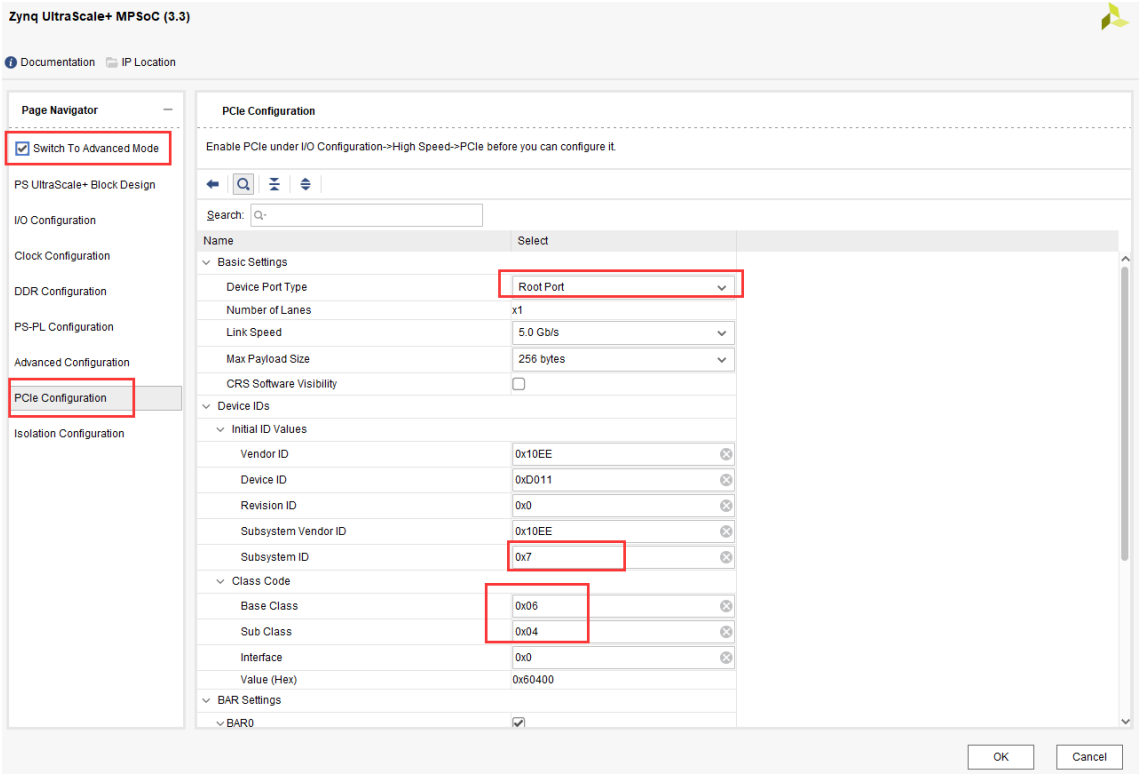
USB 复位选择 MIO 44



3） 勾选 PCIe

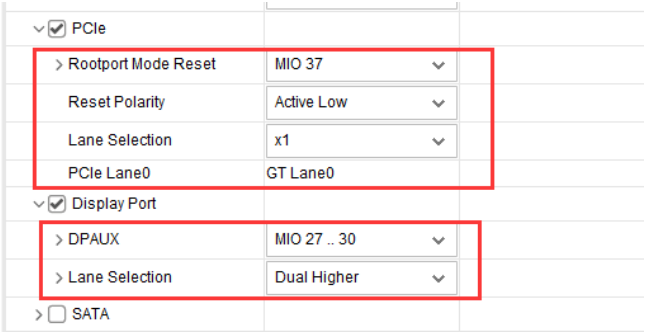


4） 点开 Switch To Advanced Mode，选择 PCIe Configuration，修改以下几个参数，配置为 ROOT 模式



5） 回到 I/O Configuration，复位选择 MIO 37；勾选 Display Port，选择 MIO 27..30，Lane

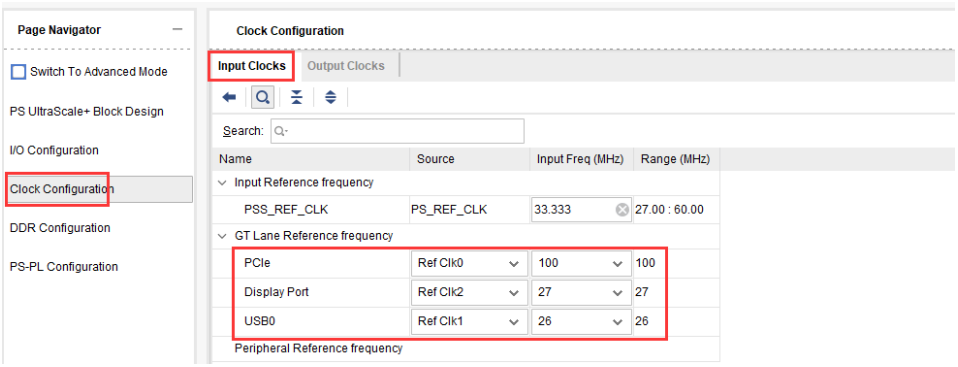
Selection 选择 Dual Higher



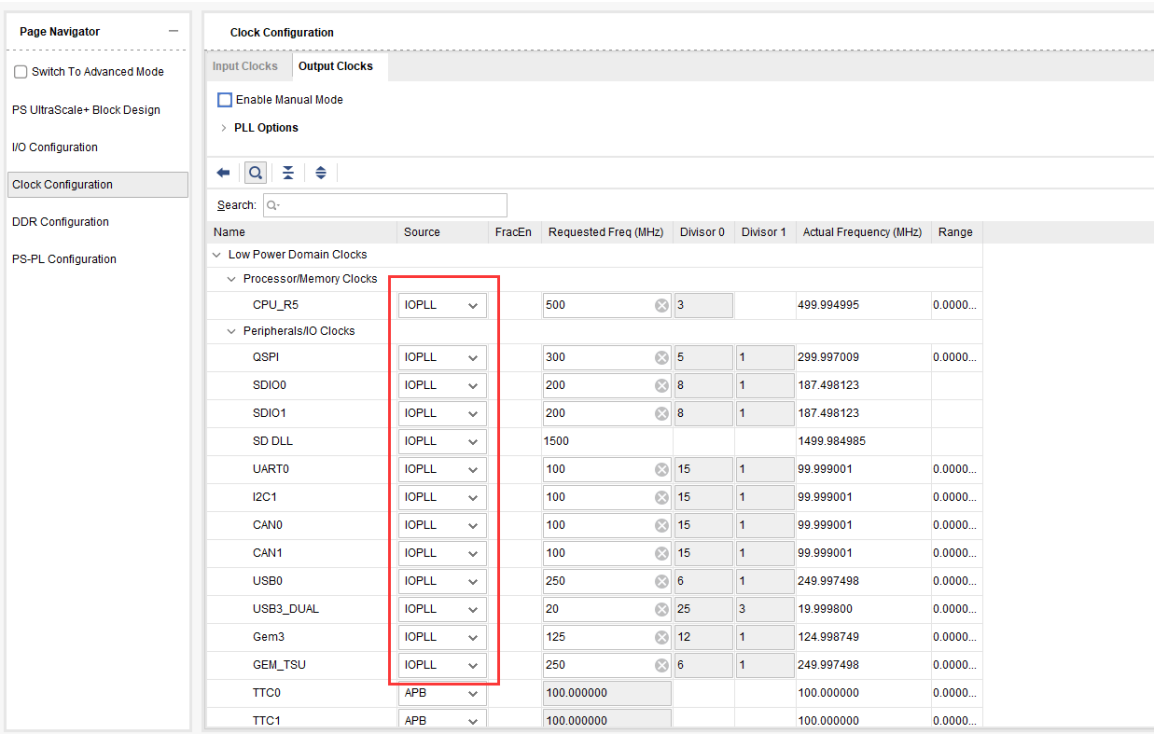
至此，I/O 部分配置完毕

1.1.3 时钟配置

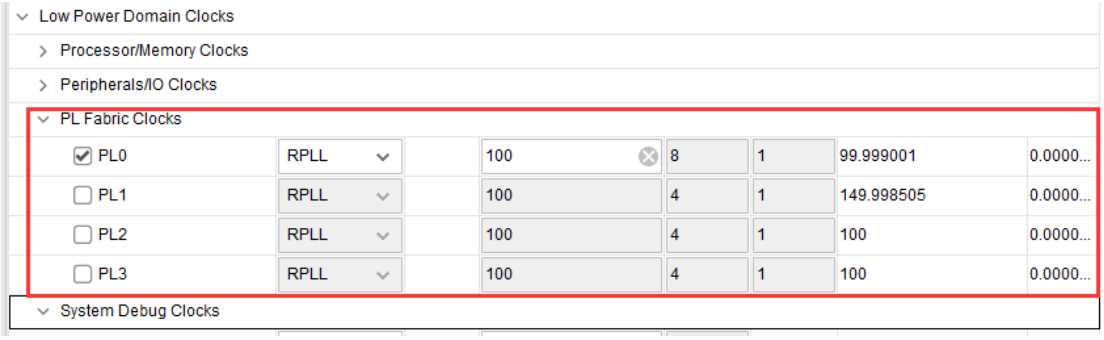
1) 在 Clock Configuration 界面，Input Clocks 窗口配置参考时钟，其中 PSS_REF_CLOCK 为 ARM 的参考时钟默认为 33.333MHz；PCIe 选择 Ref Clk0，100MHz；Display Port 选择 Ref Clk2，27MHz；USB0 选择 Ref Clk1，26MHz。



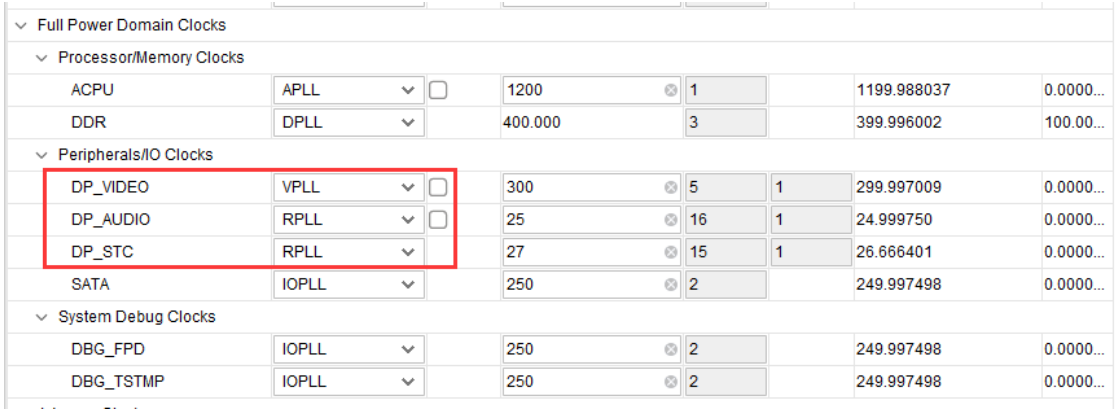
2) 在 Output Clocks 窗口，如果不是 IOPLL，改成 IOPLL，保持一致，用同样的 PLL



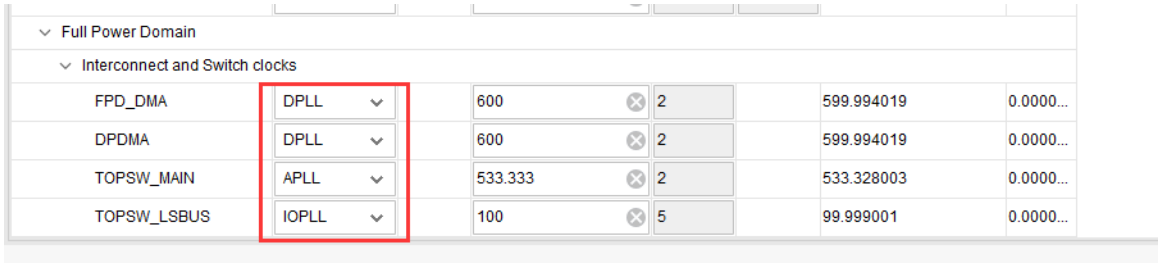
3) PL 的时钟保持默认，这是给 PL 端逻辑提供的时钟。



4) Full Power 部分，其他保持默认，将 DP_VIDEO 改为 VPLL，DP_AUDIO 和 DP_STC 改为 RPLL。



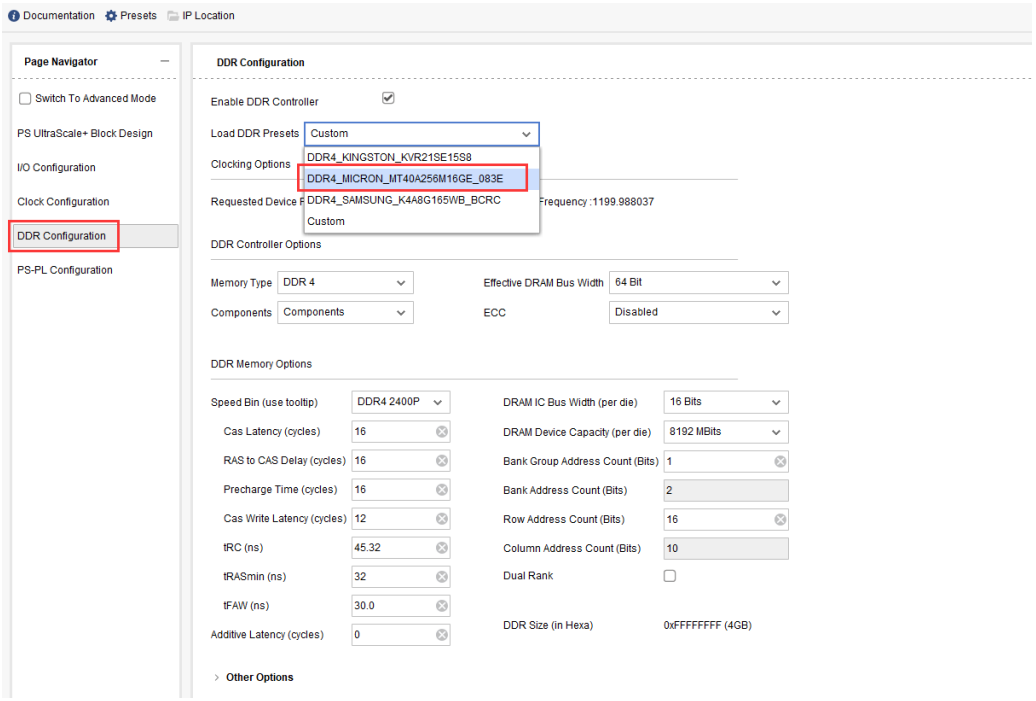
最下面的 Interconnect 修改如下



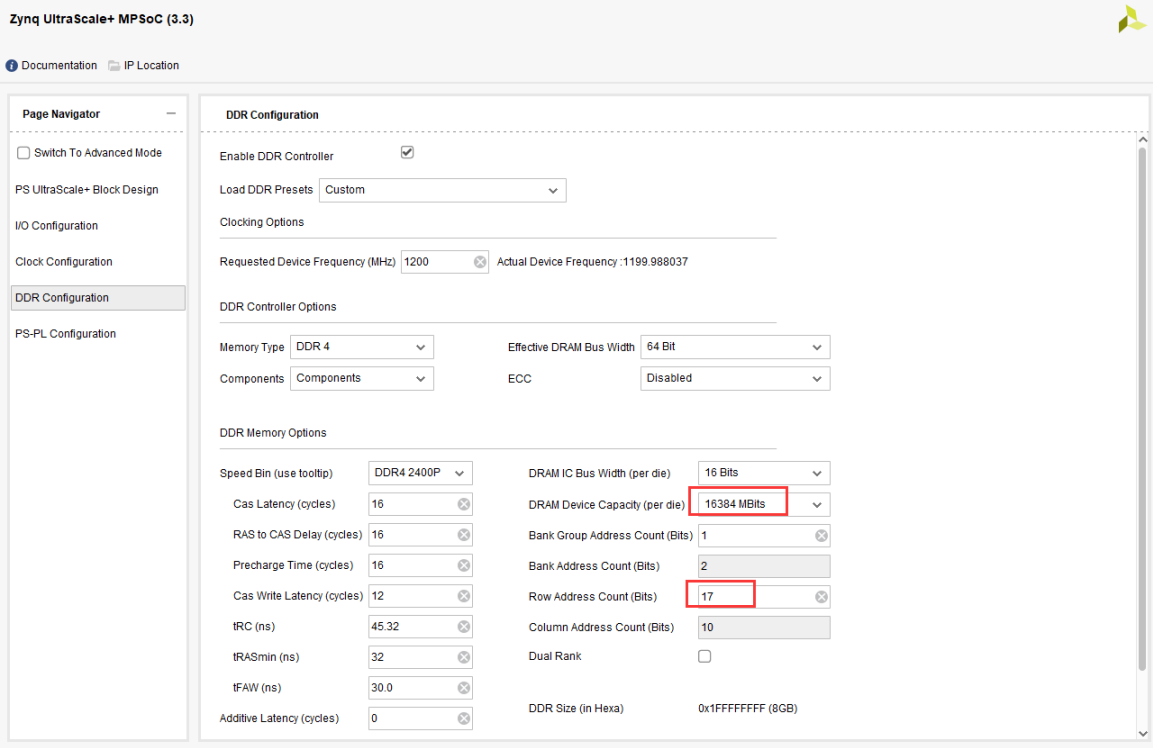
其他部分保持默认，至此，时钟部分配置完成。

5.2.1 DDR 配置

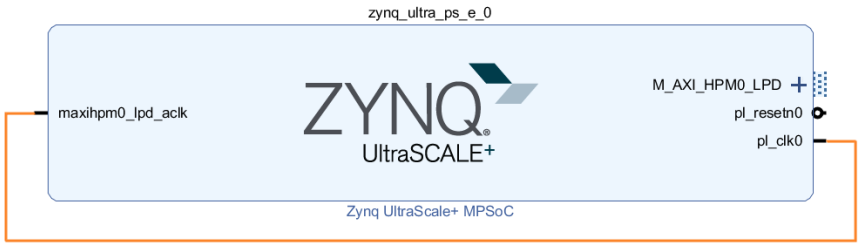
在 DDR Configuration 窗口中，Load DDR Presets 选择”DDR4_MICRON_MT40A256M16GE_083E”



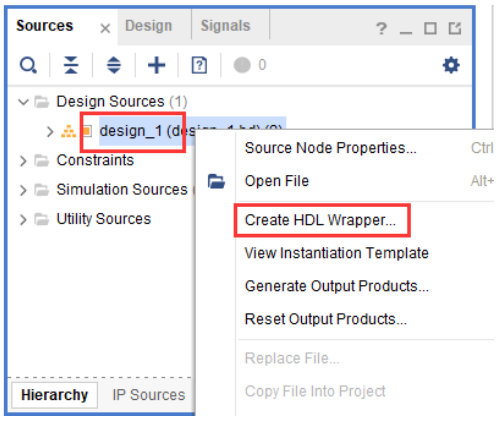
参数做些修改如下



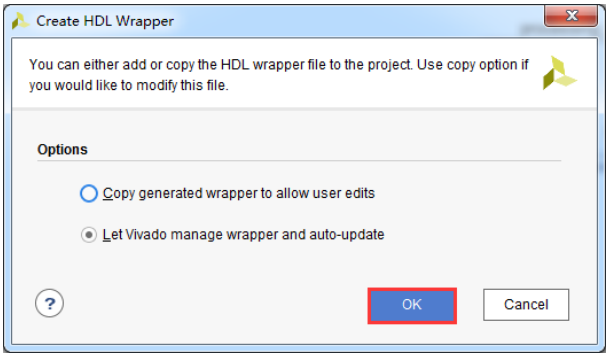
其它保持默认，点击 OK，配置完成，并连接时钟如下：



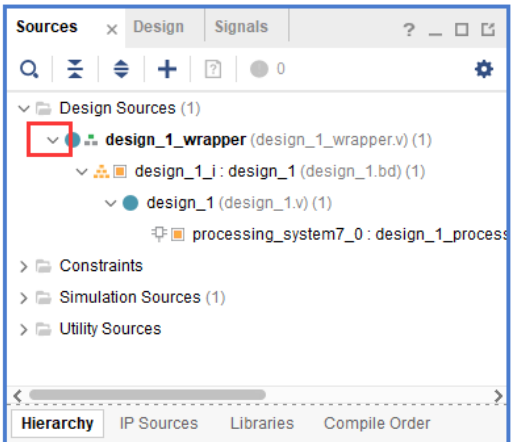
8) 选择 Block 设计，右键“Create HDL Wrapper...”,创建一个 Verilog 或 VHDL 文件，为 block design 生成 HDL 顶层文件。



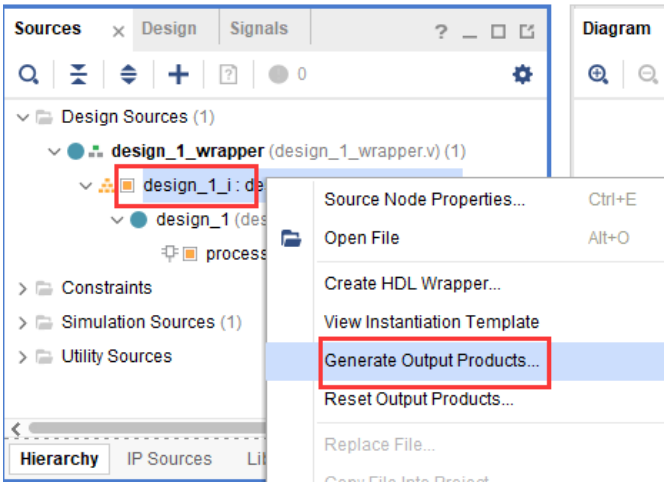
9) 保持默认选项，点击“OK”



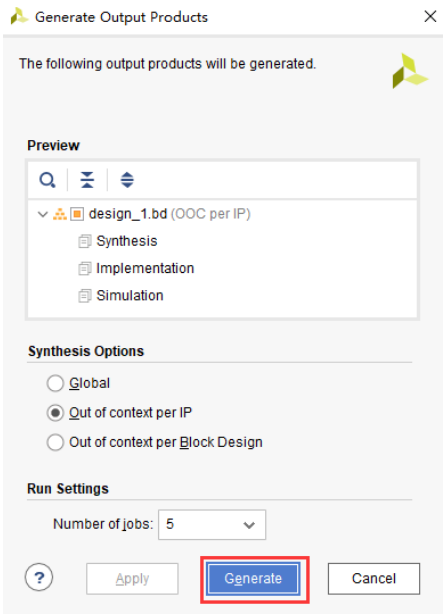
10) 展开设计可以看到 PS 被当成一个普通 IP 来使用。



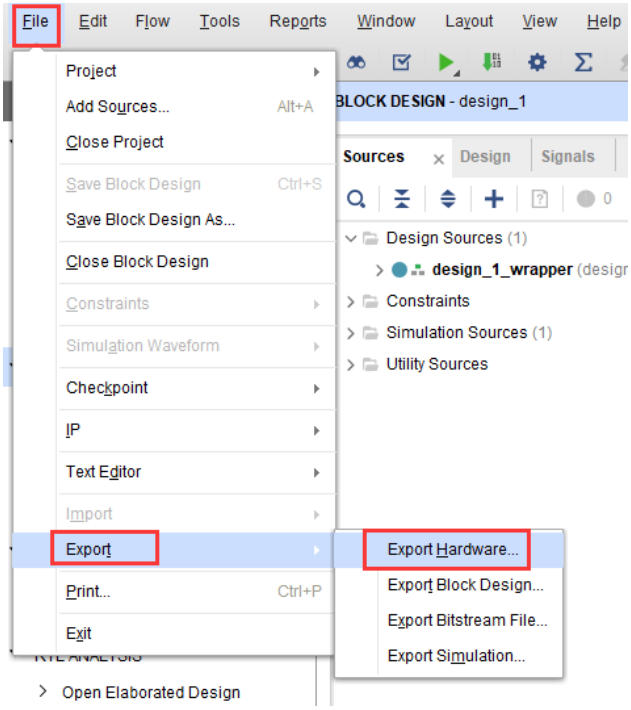
11) 选择 block 设计，右键“Generate Output Products”，此步骤会生成 block 的输出文件，包括 IP，例化模板，RTL 源文件，XDC 约束，第三方综合源文件等等。供后续操作使用。



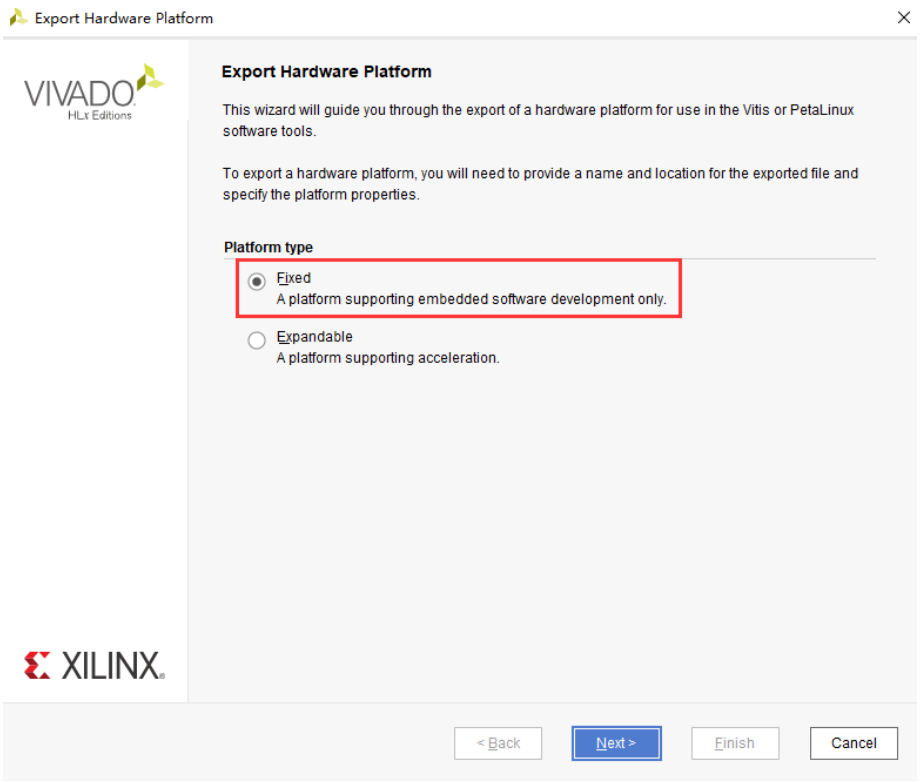
12) 点击“Generate”



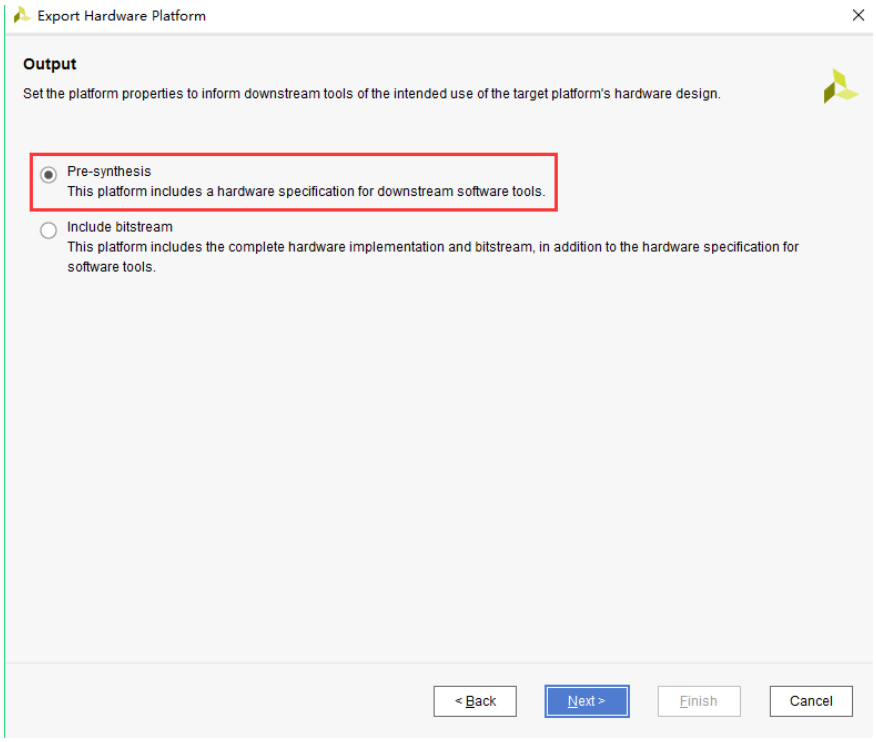
13) 在菜单栏“File -> Export -> Export Hardware...”导出硬件信息，这里就包含了 PS 端的配置信息。



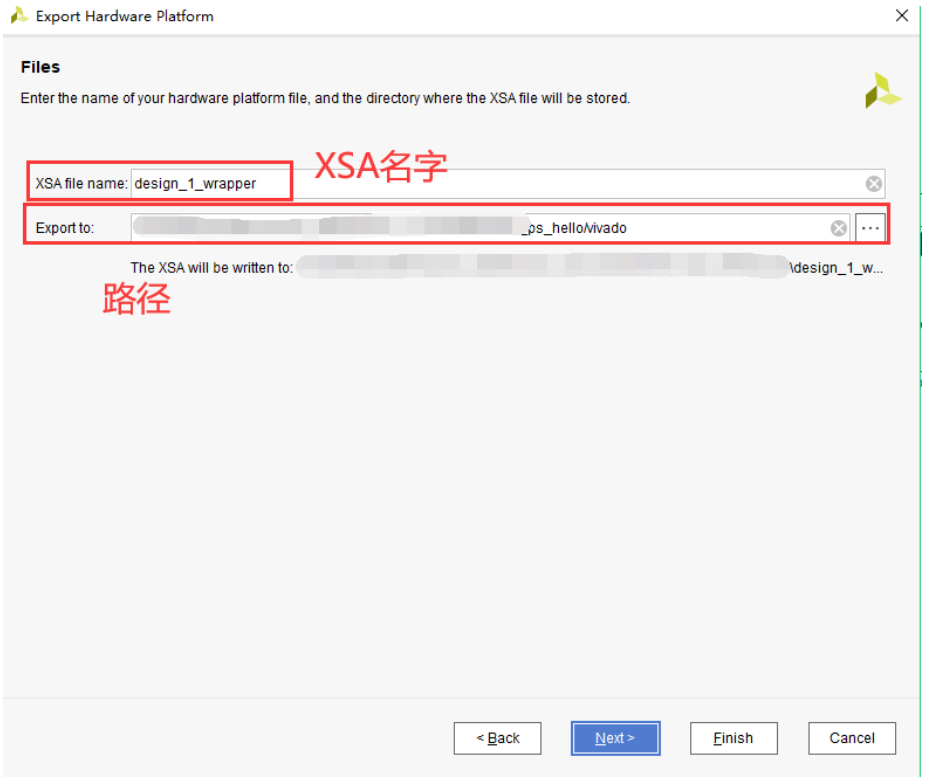
14) 在弹出的窗口中选择 Fixed，点击 Next



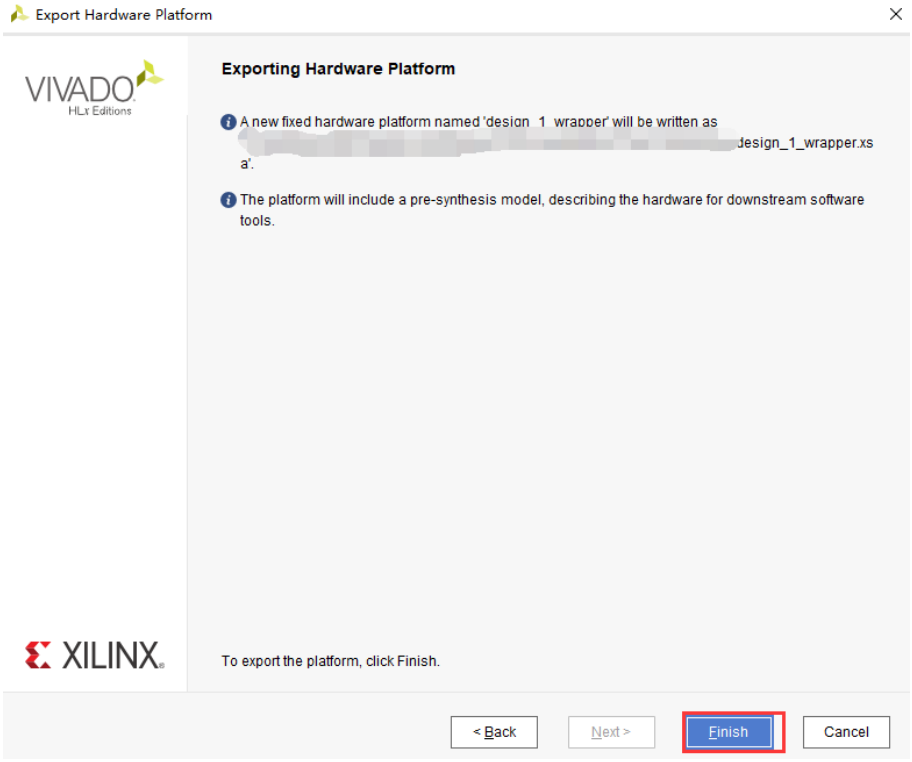
15) 在弹出的对话框中点击“OK”，因为实验仅仅是使用了 PS 的串口，不需要 PL 参与，这里就没有使能不选择“Include bitstream”，点击 Next

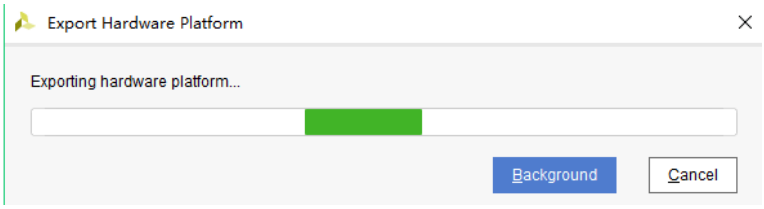


16) 可修改导出名字以及导出路径，默认是在 vivado 工程目录下的，这个文件可以根据自己的需要在合适的位置，不一定要放在 vivado 工程下面，vivado 和 vitis 软件是独立的。在这里我们选择默认不做更改。点击 Next

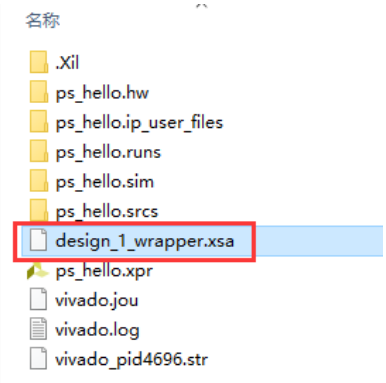

Export Hardware Platform
Files
Enter the name of your hardware platform file, and the directory where the XSA file will be stored.
XSA file name: design_1_wrapper
XSA名字
Export to:
_ps_hello/vivado
The XSA will be written to:
\design_1_w...
路径
< Back
Next >
Finish
Cancel

点击 Finish


Export Hardware Platform
VIVADO
HLx Editions
Exporting Hardware Platform
A new fixed hardware platform named 'design_1_wrapper' will be written as
design_1_wrapper.xsa'.
The platform will include a pre-synthesis model, describing the hardware for downstream software tools.
XILINX
To export the platform, click Finish.
< Back
Next >
Finish
Cancel


Export Hardware Platform
Exporting hardware platform...
Background
Cancel

此时在工程目录下可以看到 xsa 文件，这个文件就是这个文件就包含了 Vivado 硬件设计的信息，可交由软件开发人员使用。



到此为止，FPGA 工程师工作告一段落。

软件工程师工作内容

Vitis 工程目录为“ps_hello/vitis”

以下为软件工程师负责内容。

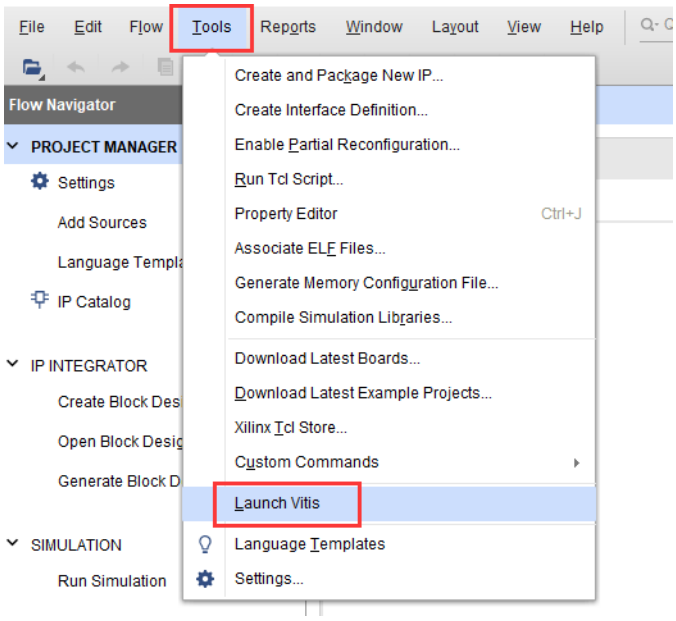
5.3 Vitis 调试

5.3.1 创建 Application 工程

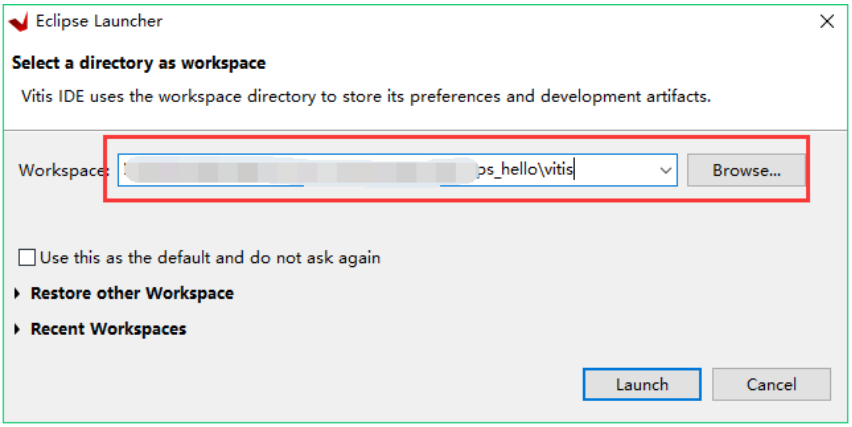
1) 新建一个文件夹，将 vivado 导出的 xx.xsa 文件拷贝进来。
2) Vitis 是独立的软件，可以双击 Vitis 软件打开



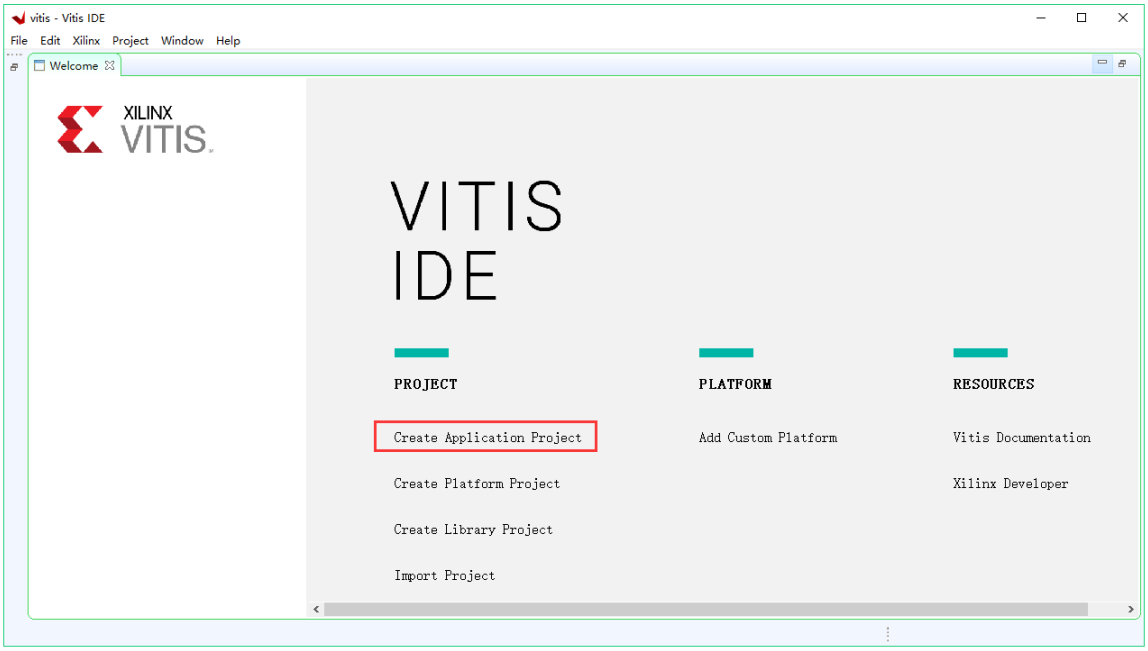
也可以通过在 Vivado 软件中选择 Tools→Launch Vitis 打开 Vitis 软件



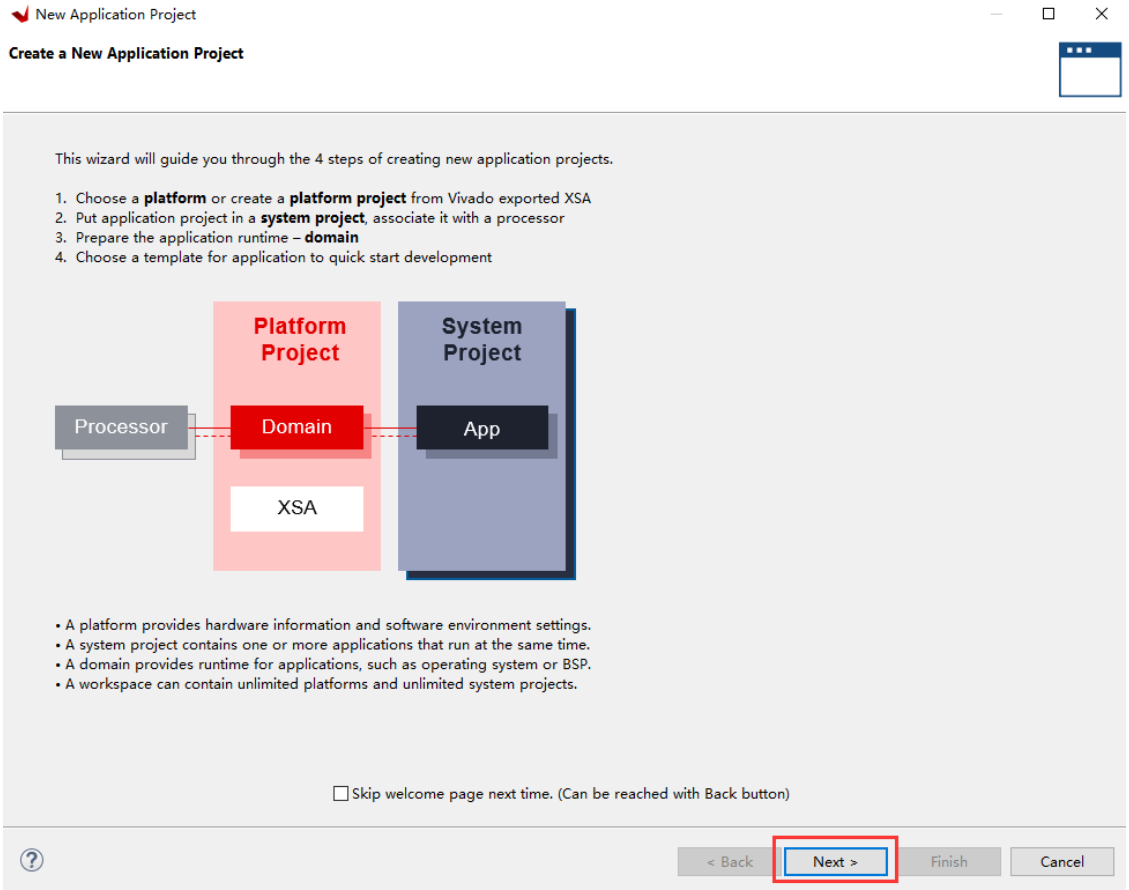
选择之前新建的文件夹，点击”Launch”



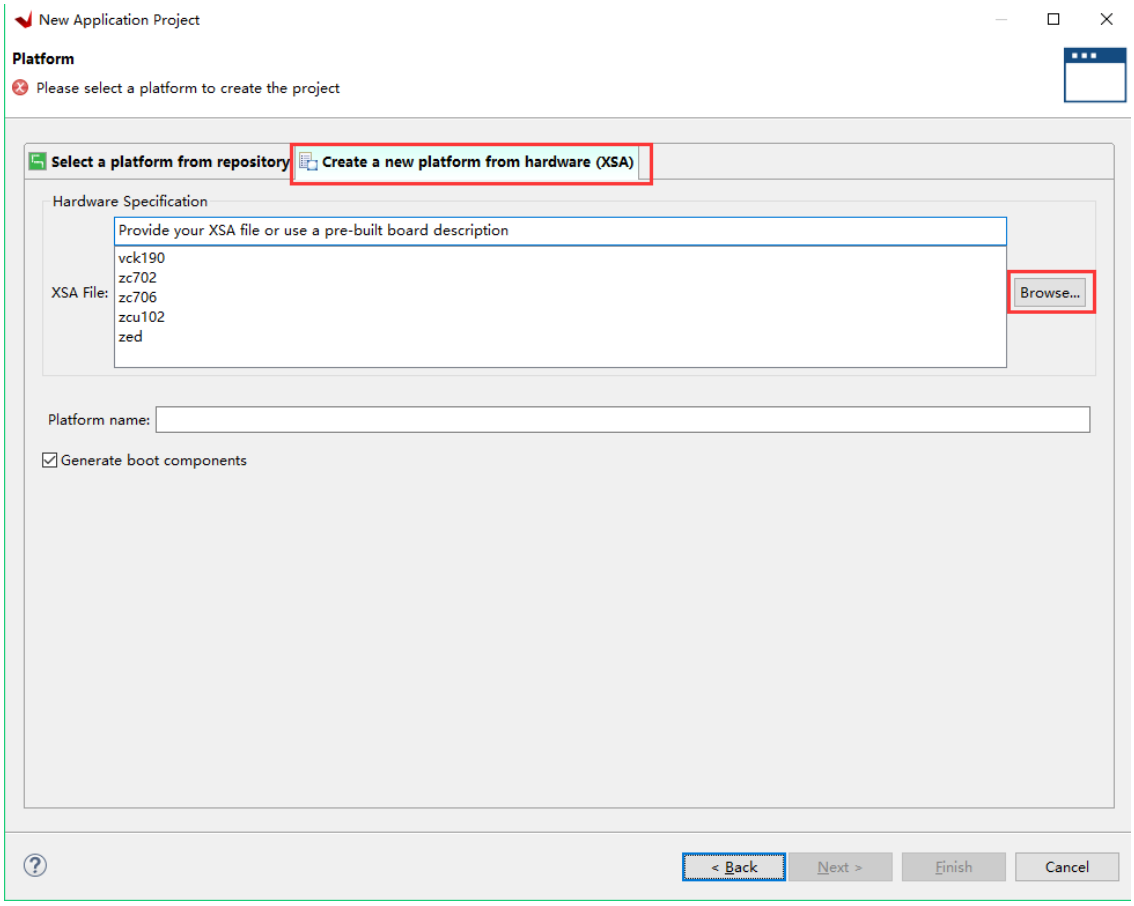
3) 启动 Vitis 之后界面如下，点击“Create Application Project”，这个选项会生成 APP 工程以及 Platfrom 工程，Platform 工程类似于以前版本的 hardware platform，包含了硬件支持的相关文件以及 BSP。



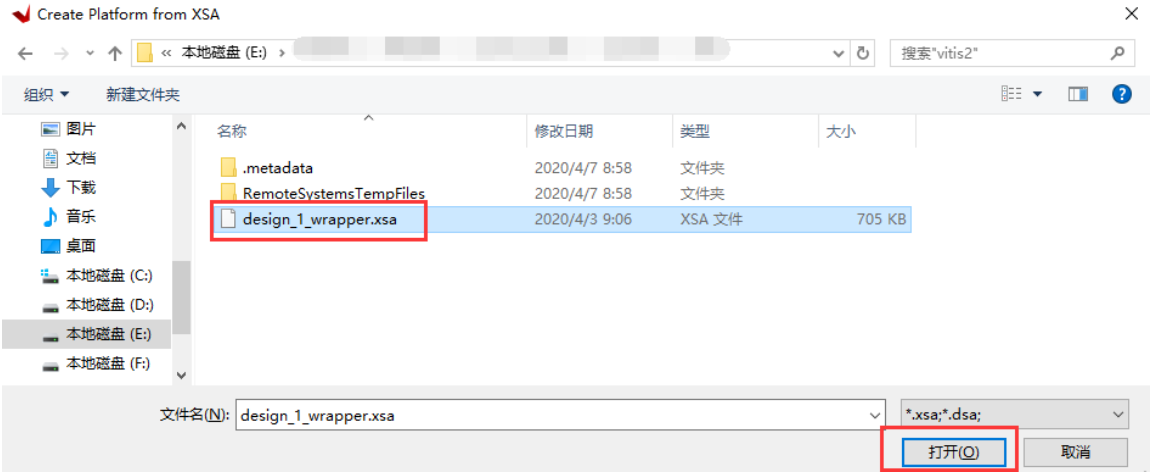
4) 第一页为介绍页，直接跳过，点击 Next



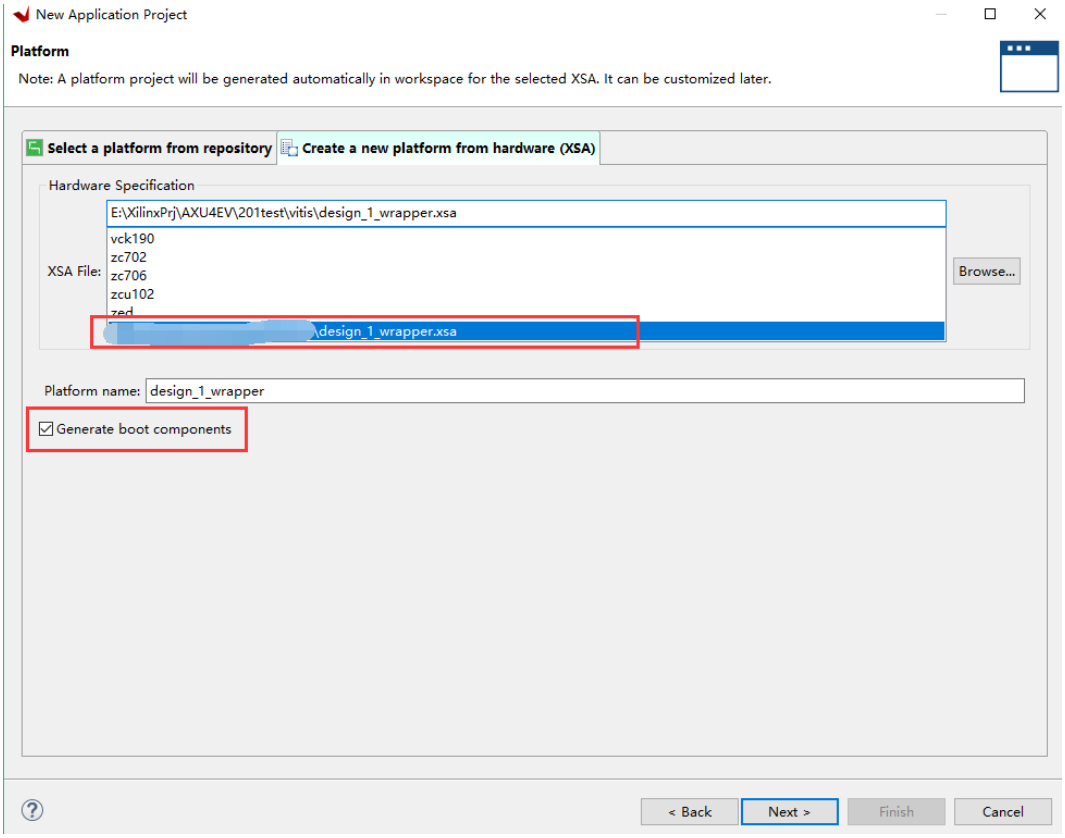
5) 选择“Create a new platform from hardware(XSA)”，选择“Browse”



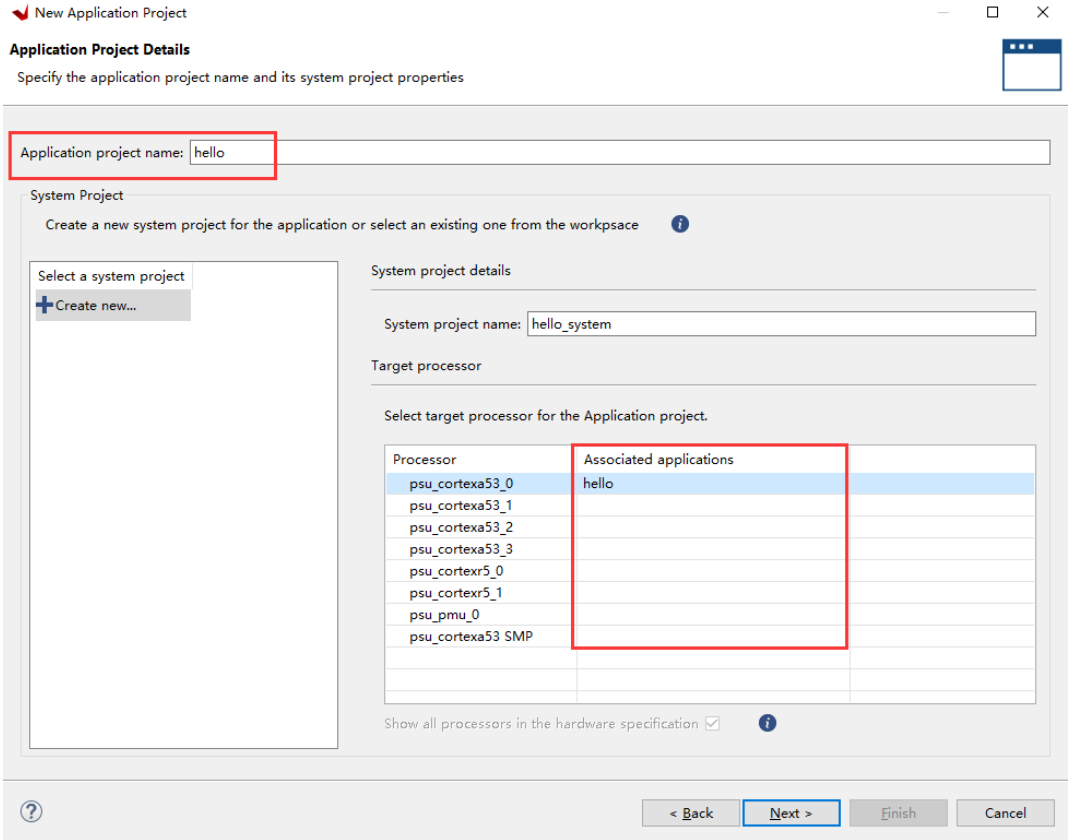
选择之前生成的 xsa，点击打开



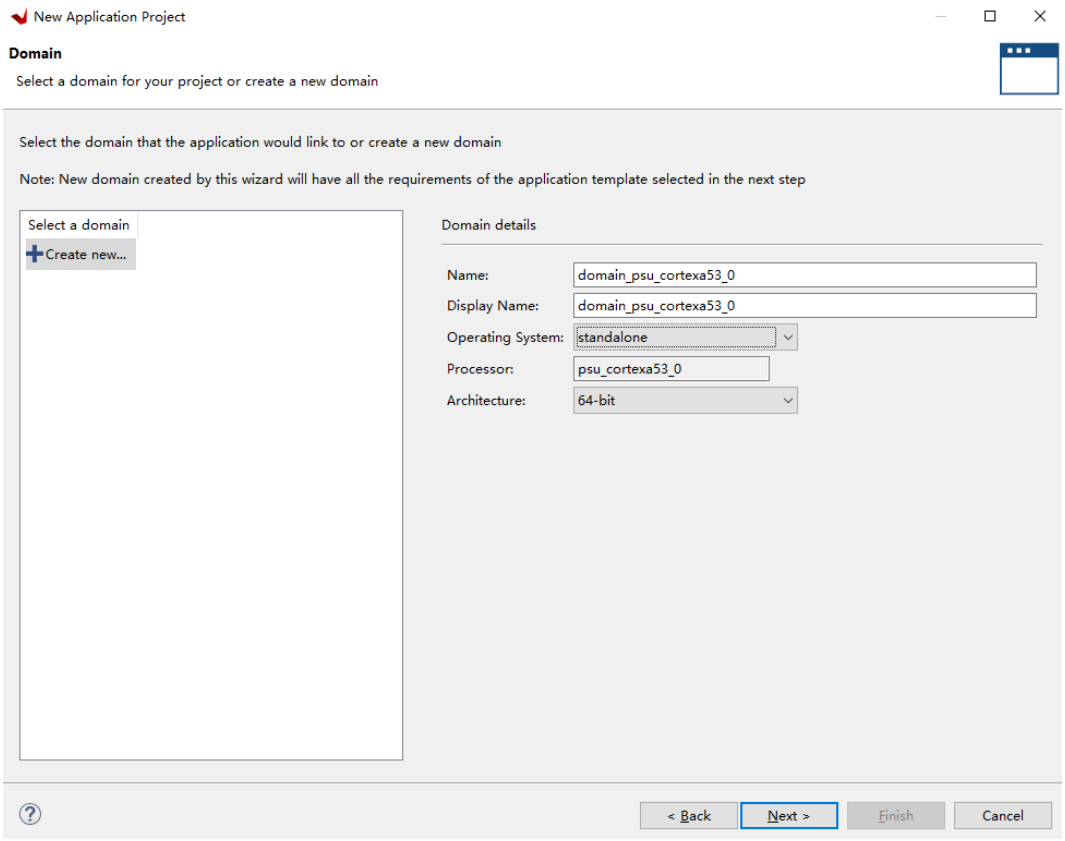
6) 最下面的 Generate boot components 选项，如果勾选上，软件会自动生成 fsbl 工程，我们一般选择默认勾选上。



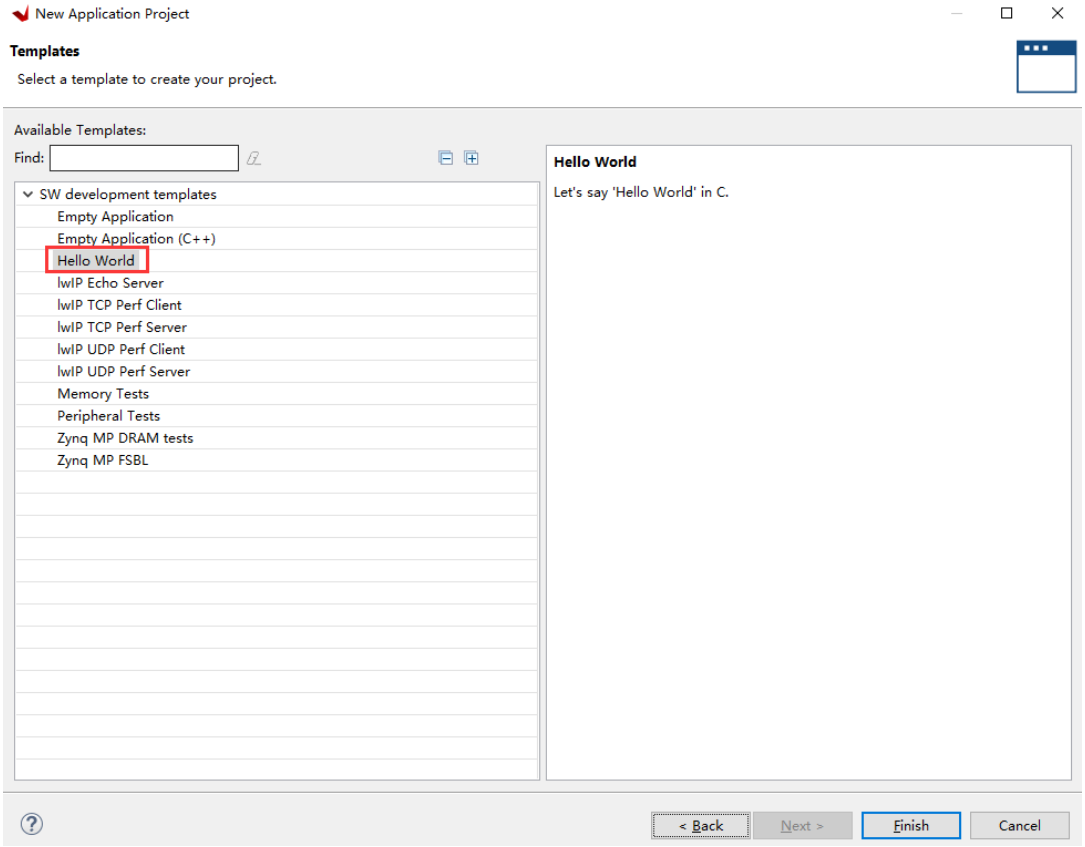
7) 填入 APP 工程名称，在方框处点击可以选择对应的处理器，我们这里保持默认



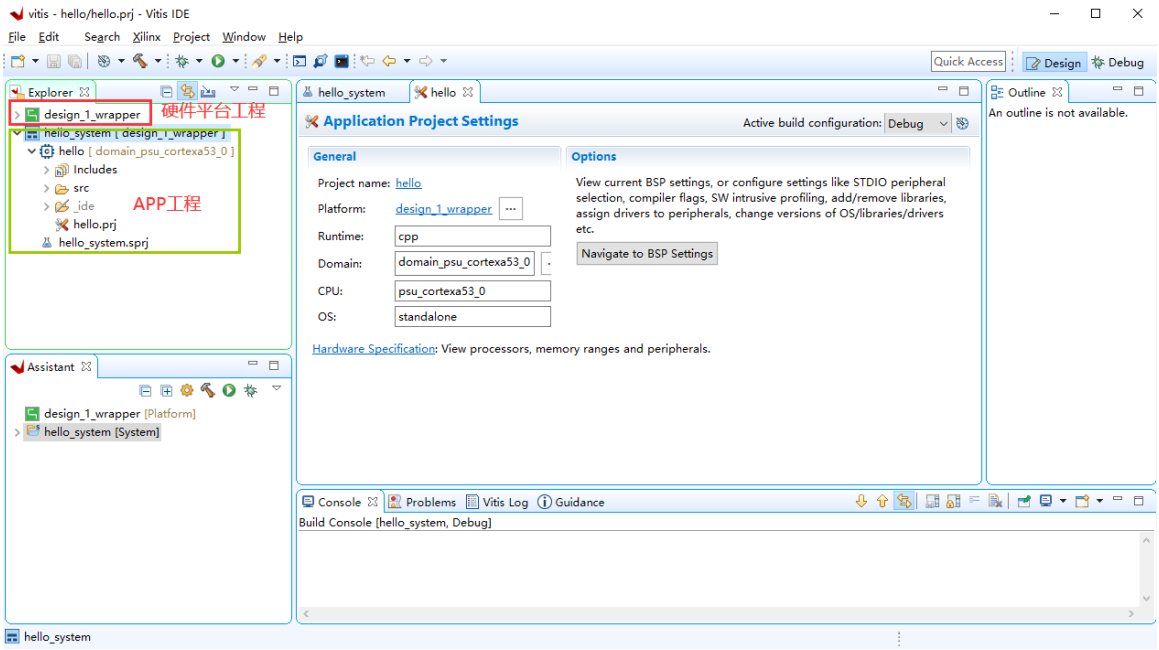
8) 在这个界面可以修改 Domain 名称，选择操作系统，ARM 架构等，这里保持默认，操作系统选择 standalone，也就是裸机。



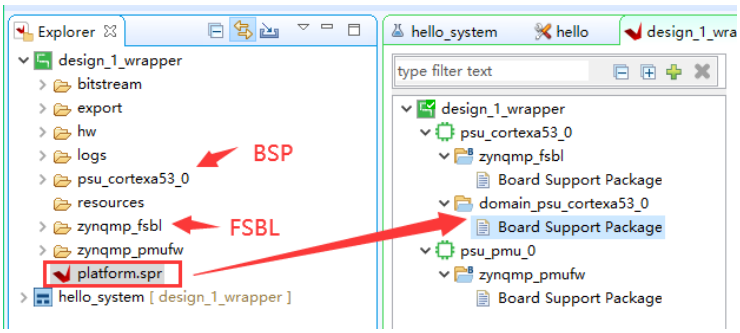
9) 选择”Hellow World”模板，点击“Finish”完成



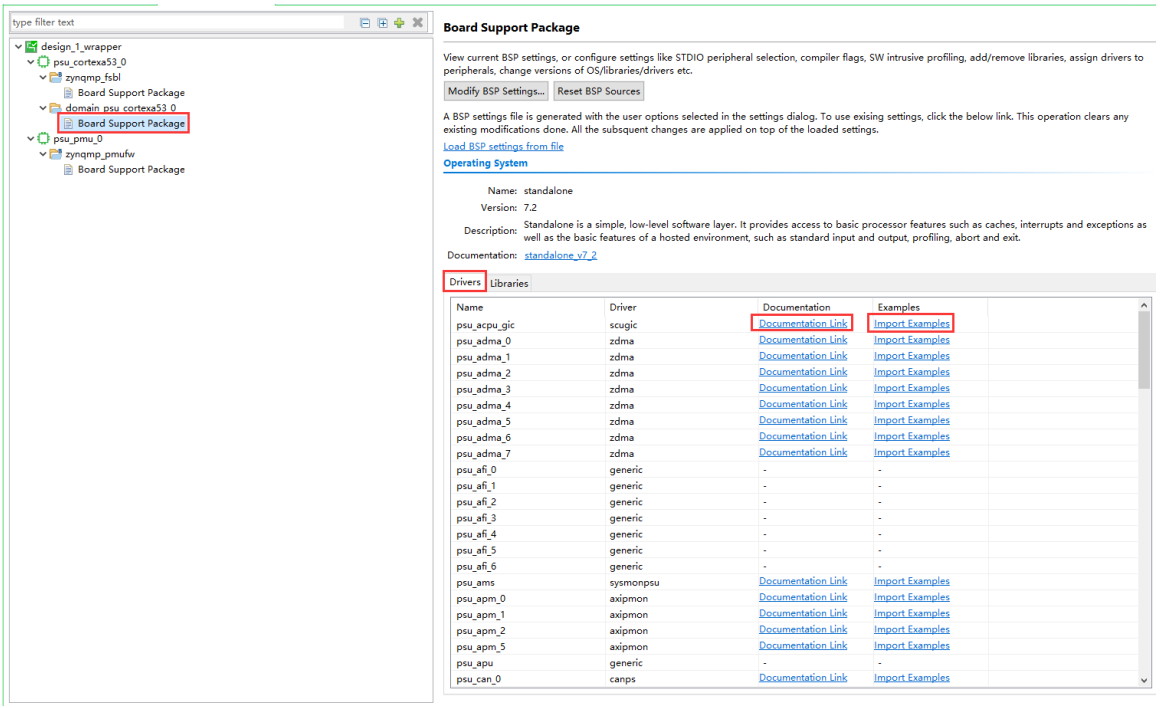
10) 完成之后可以看到生成了两个工程，一个是硬件平台工程，即之前所说的 Platfrom 工程，一个是 APP 工程



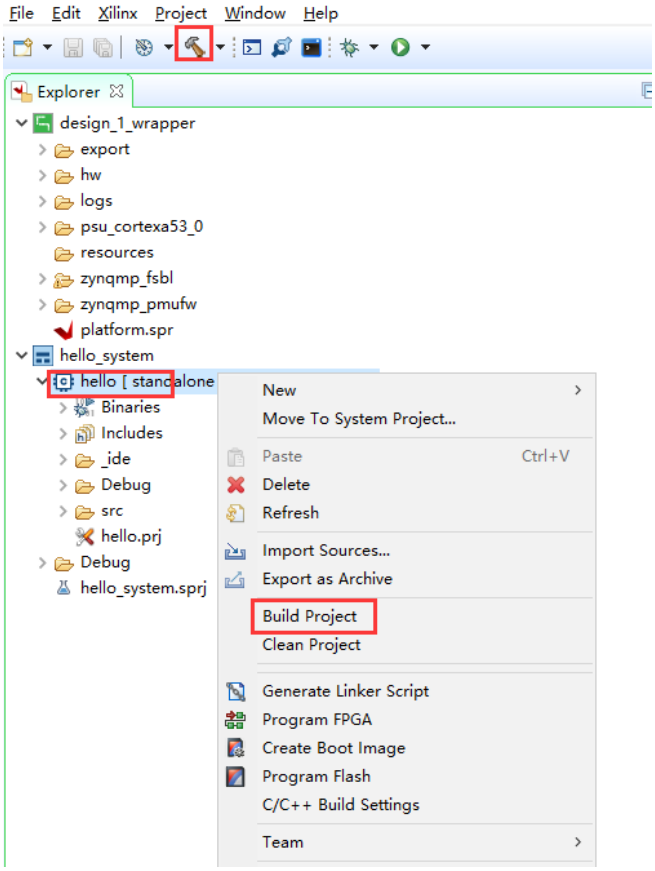
11) 展开 Platform 工程后可以看到里面包含有 BSP 工程，以及 zynq_fsbl 工程（此工程即选择 Generate boot components 之后的结果）,双击 platform.spr 即可看到 Platform 对应生成的 BSP 工程，可以在这里对 BSP 进行配置。软件开发人员比较清楚，BSP 也就是 Board Support Package 板级支持包的意思，里面包含了开发所需要的驱动文件，用于应用程序开发。可以看到 Platform 下有多个 BSP，这是跟以往的 SDK 软件不一样的，其中 zynqmp_fsbl 即是 fsbl 的 BSP，domain_psu_cortexa53_0 即是 APP 工程的 BSP。也可以在 Platform 里添加 BSP，在以后的例程中再讲。



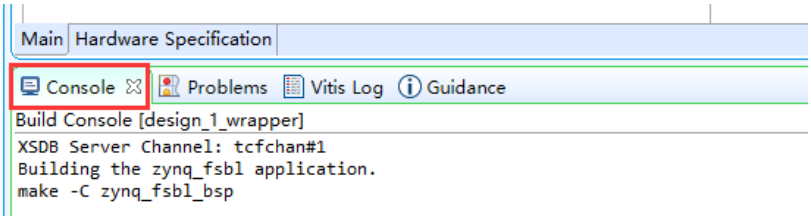
12) 点开 BSP，即可看到工程带有的外设驱动，其中 Documentation 是 xilinx 提供的驱动的说明文档，Import Examples 是 xilinx 提供的 example 工程，加快学习。



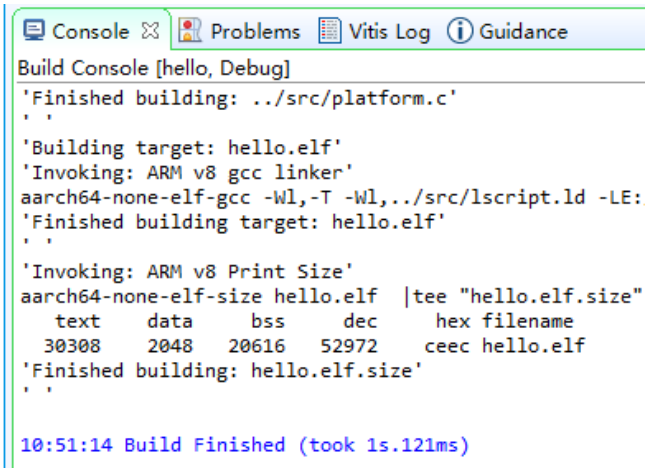
13) 选中 APP 工程，右键 Build Project，或者点击菜单栏的“锤子”按键，进行工程编译



14) 可以在 Console 看到编译过程

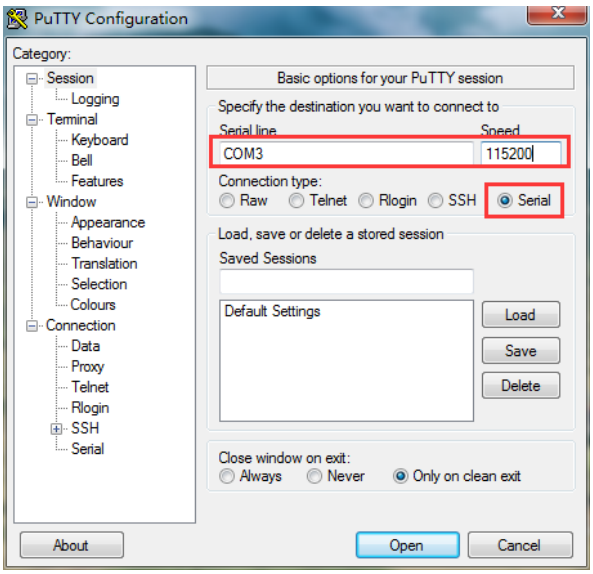


编译结束，生成 elf 文件

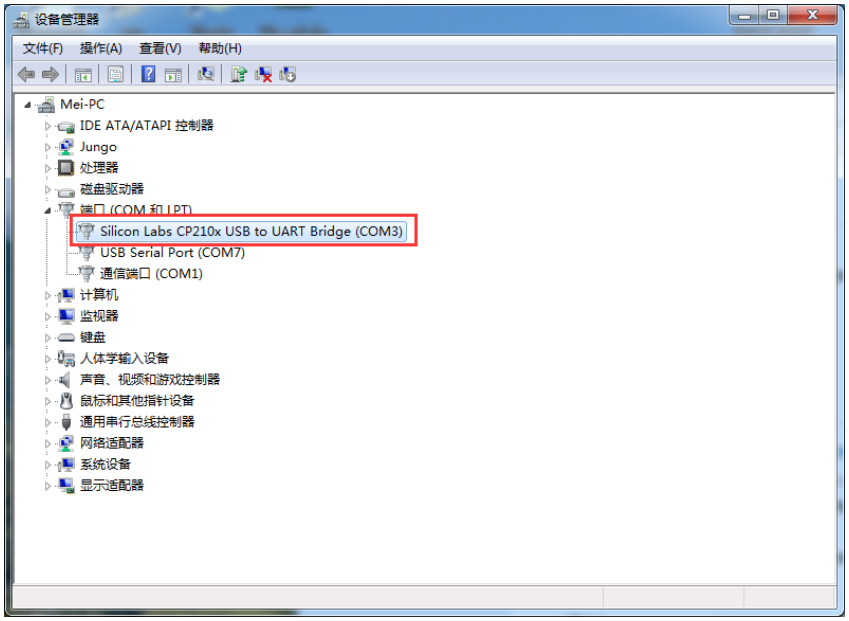


15) 连接 JTAG 线到开发板、UART 的 USB 线到 PC

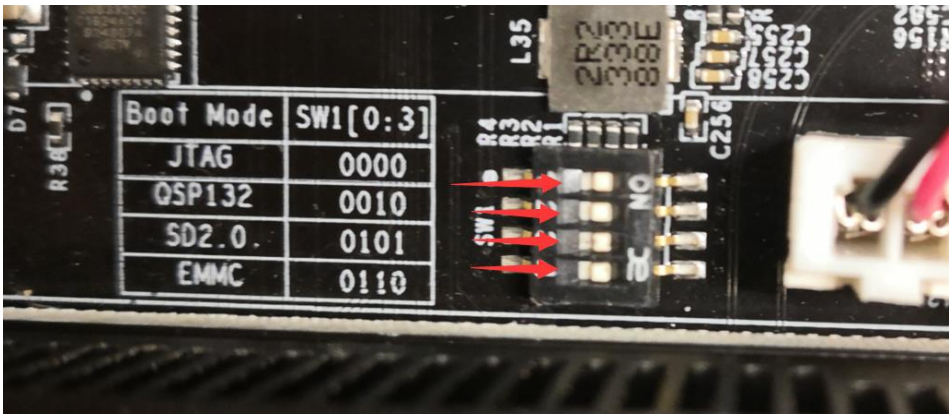
16) 使用 PuTTY 软件做为串口终端调试工具，PuTTY 是一个免安装的小软件



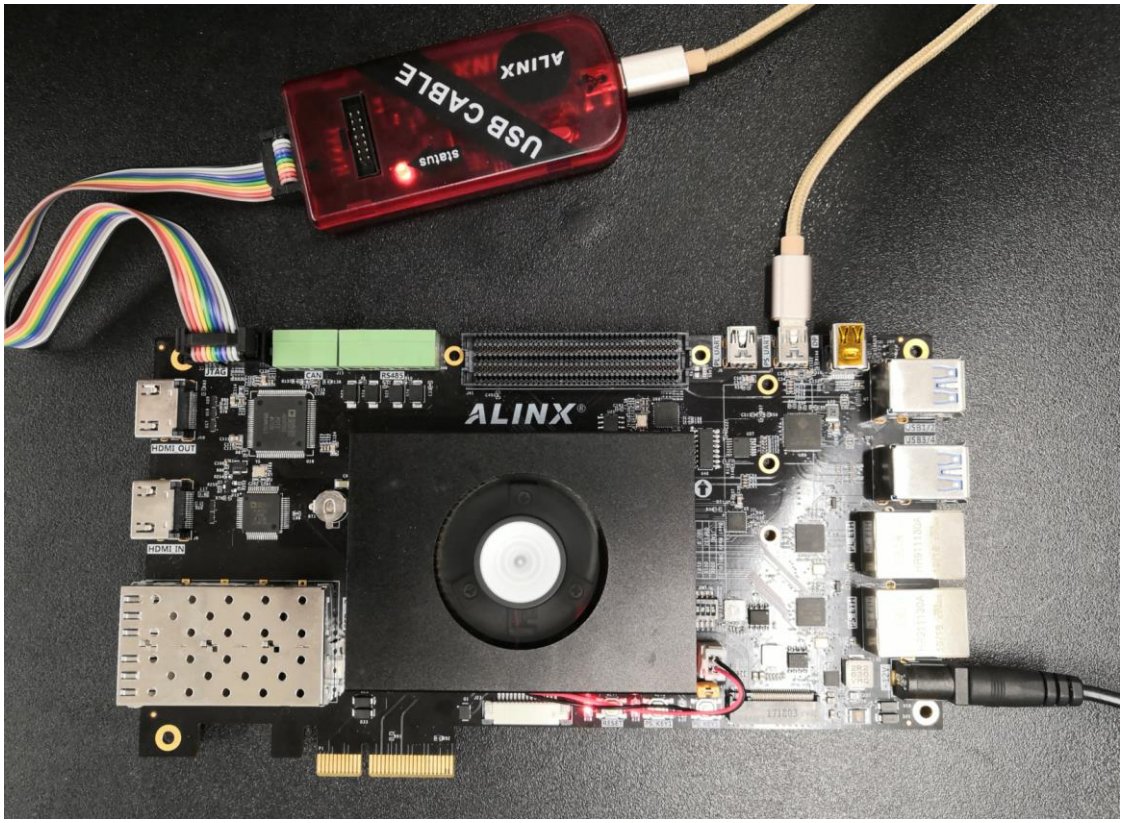
17) 选择 Serial，Serial line 填写 COM3，Speed 填写 115200，COM3 串口号根据设备管理器里显示的填写，点击“Open”

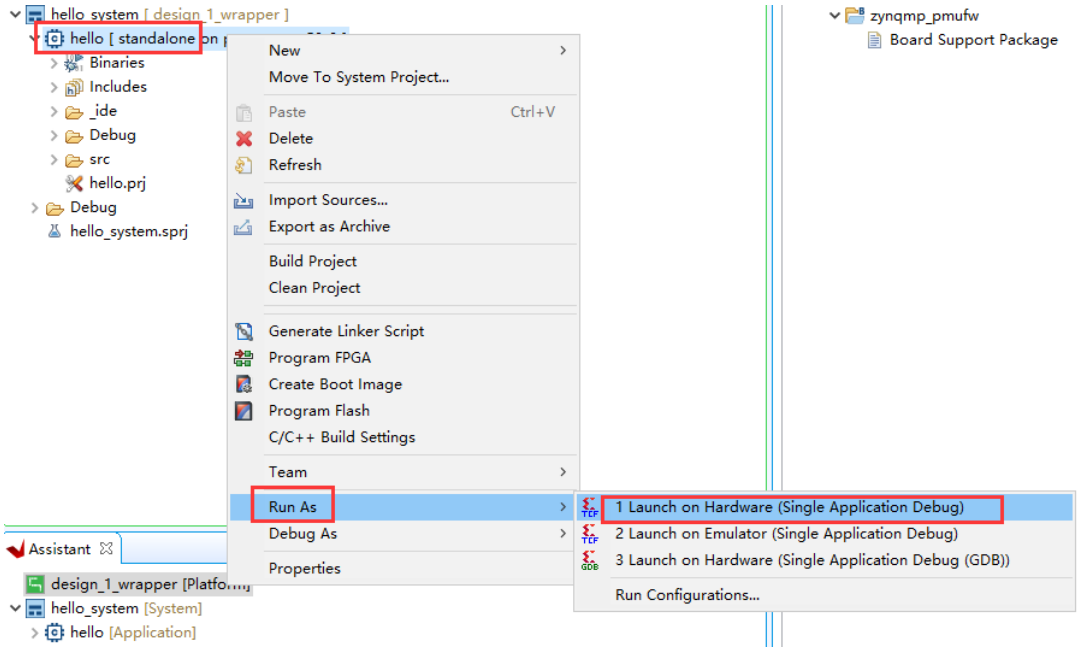


18) 在上电之前最好将开发板的启动模式设置到 JTAG 模式，拔到”ON”的位置

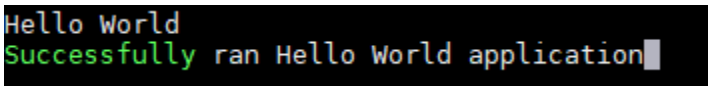


19) 给开发板上电，准备运行程序，开发板出厂时带有程序，这里可以把运行模式选择 JTAG 模式，然后重新上电。选择“hello”，右键，可以看到很多选项，本实验要用到这里的“Run as”，就是把程序运行起来，“Run as”里又有很对选项，选择第一个“Launch on Hardware(Single Application Debug)”，使用系统调试，直接运行程序。

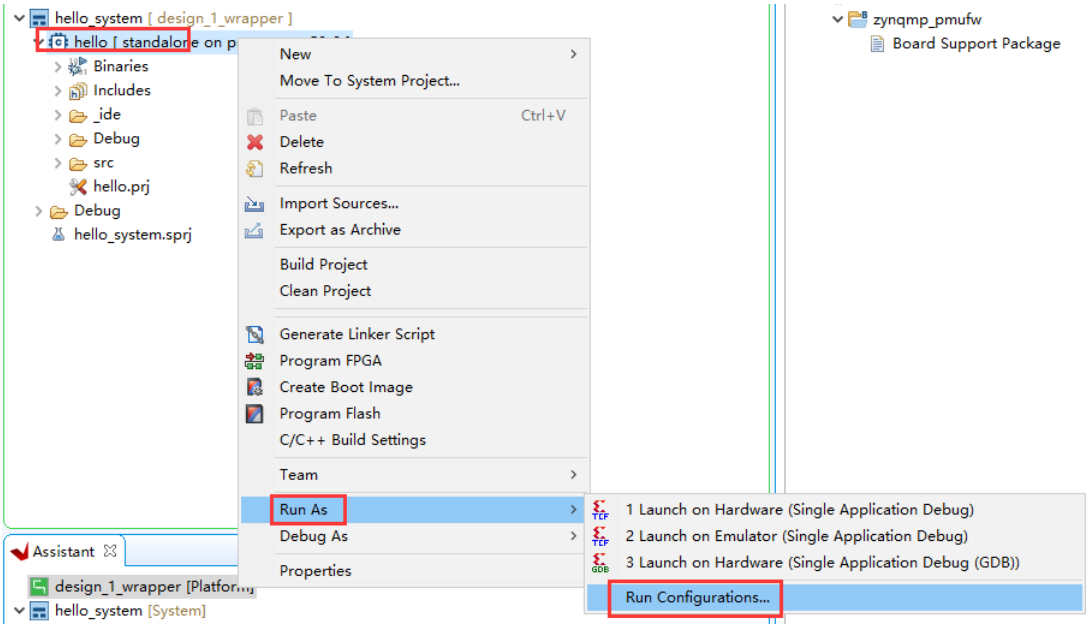




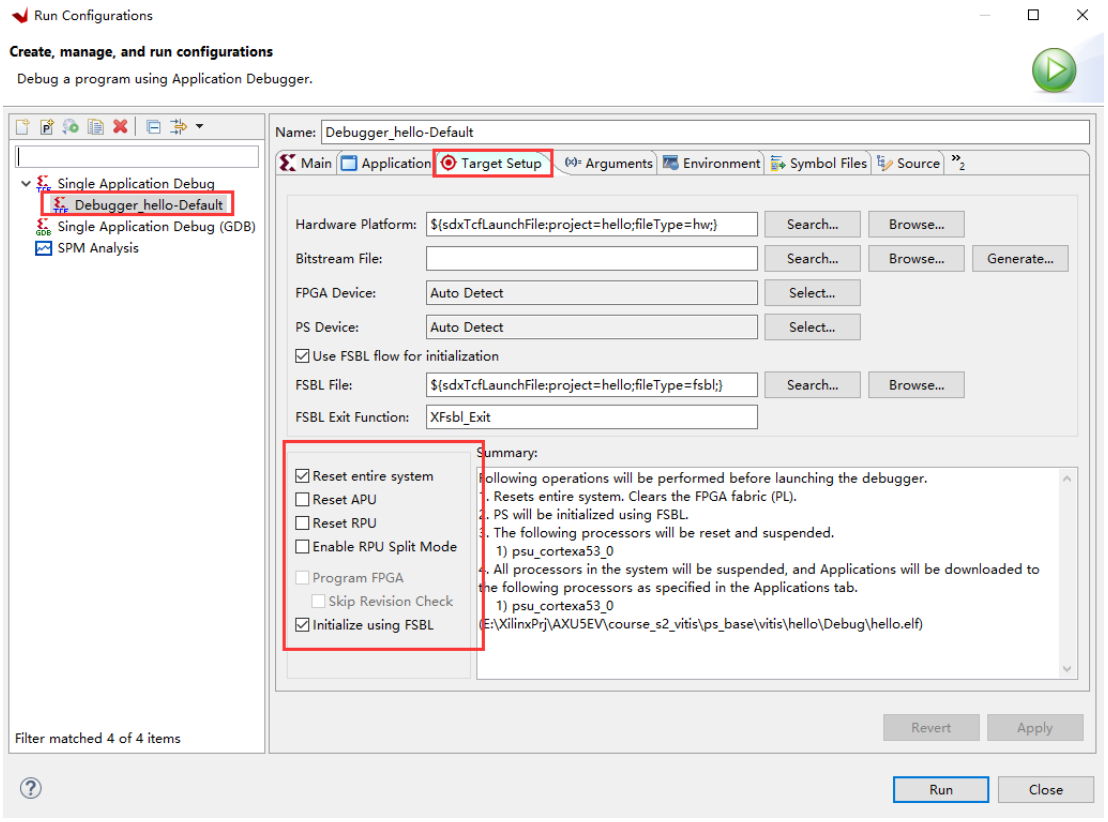
20) 这个时候观察串口软件，即可以看到输出”Hello World”



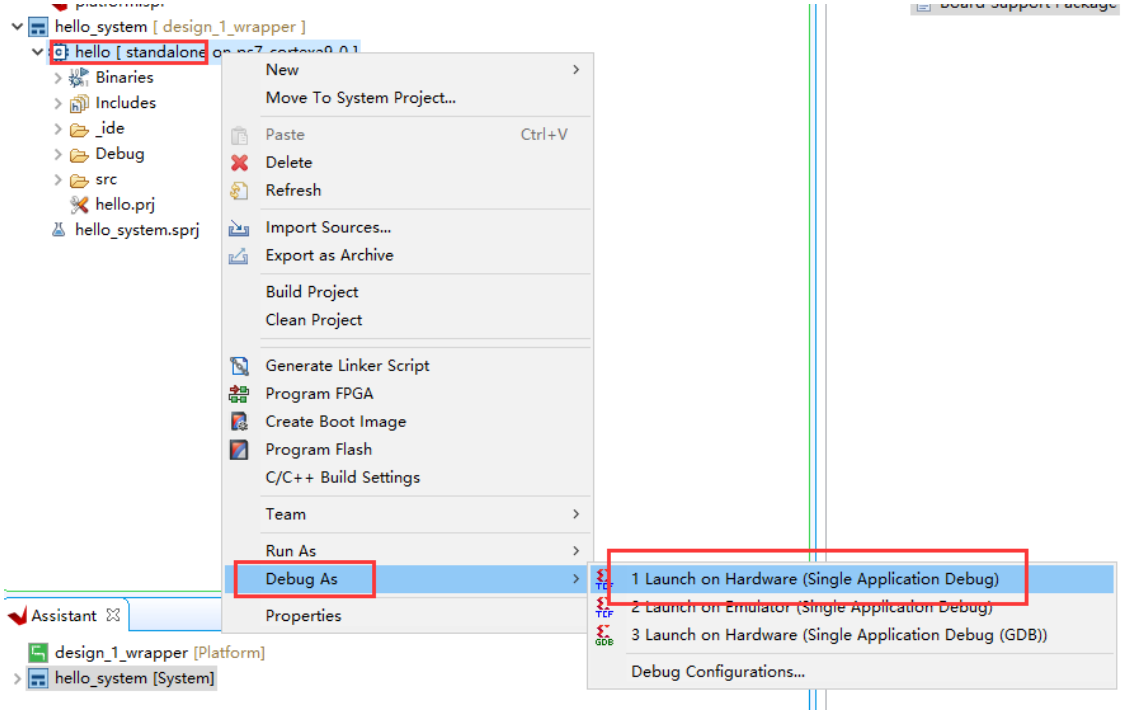
21) 为了保证系统的可靠调试，最好是右键 “Run As -> Run Configuration...”



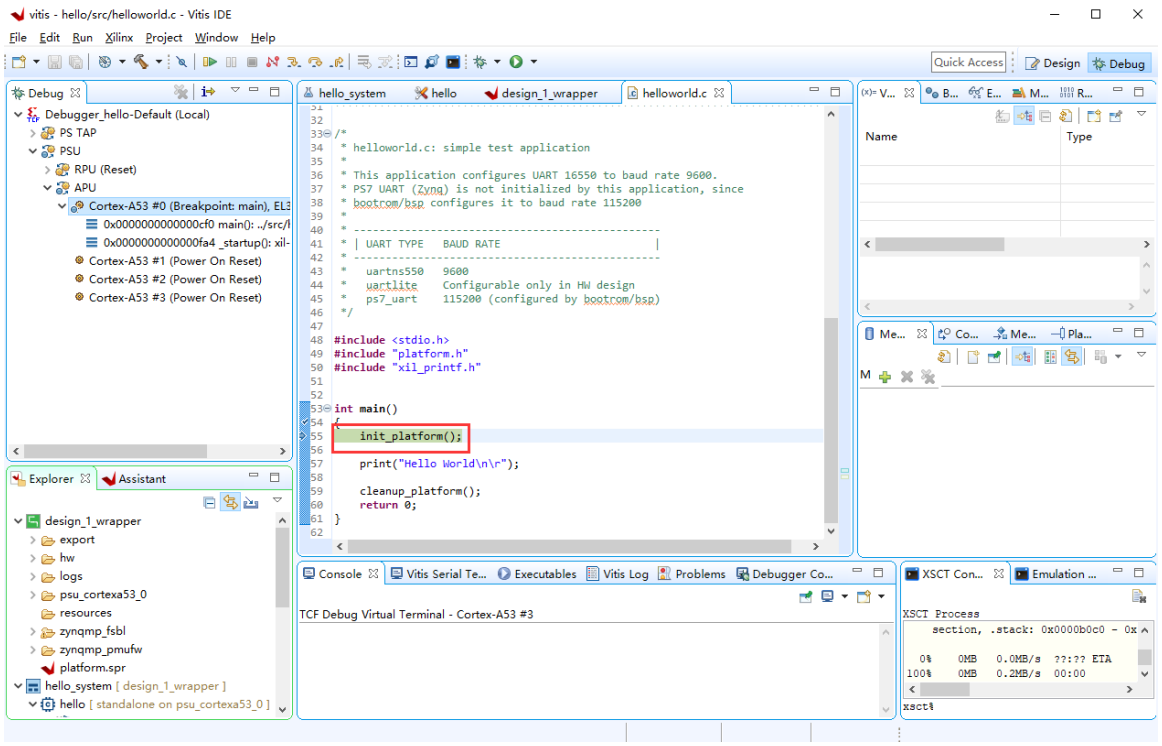
22) 我们可以看一下里面的配置，其中 Reset entire system 是默认选中的，这是跟以前的 SDK 软件不同的。如果系统中还有 PL 设计，还必须选择“Program FPGA”。



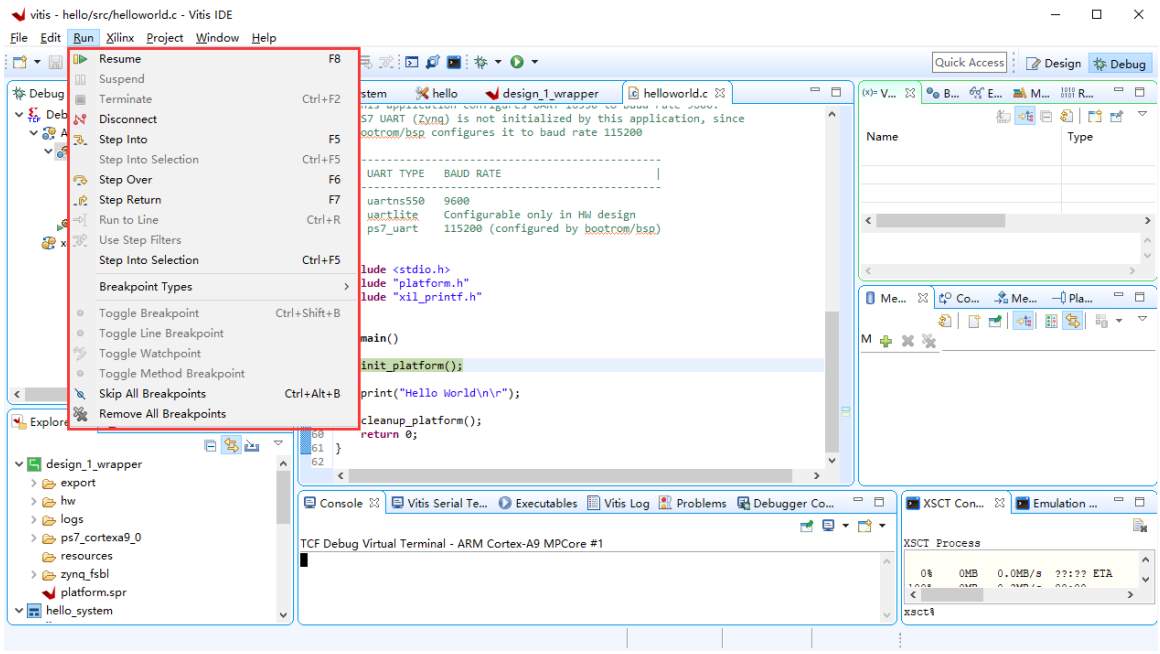
23) 除了“Run As”，还可以“Debug As”，这样可以设置断点，单步运行



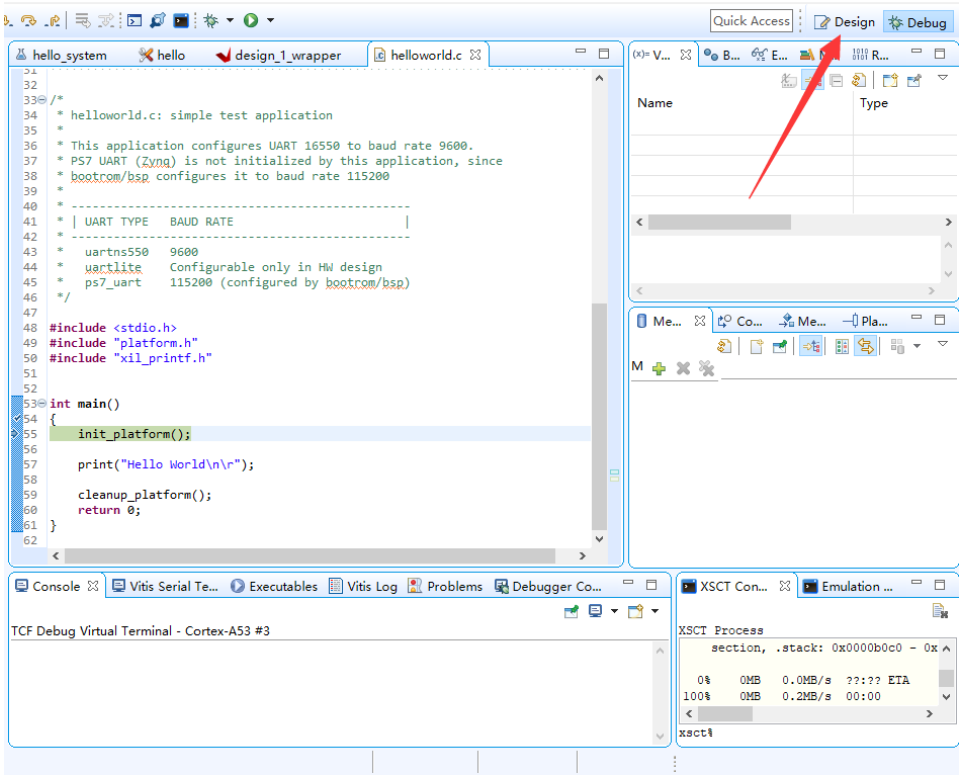
24) 进入 Debug 模式



25) 和其他 C 语言开发 IDE 一样，可以逐步运行、设置断点等



26) 右上角可以切换 IDE 模式



5.4 固化程序

普通的 FPGA 一般是可以从 flash 启动，或者被动加载，ZYNQ 的启动是由 ARM 主导的，包括 FPGA 程序的加载，ZYNQ MPSoC 启动一般为三个步骤，在 UG1085 中也有介绍：

Pre-configuration satge：预加载阶段由 PMU 控制，执行 PMU ROM 中的代码设置系统。PMU 处

理所有的复位和唤醒过程。

Configuration stage：接下来进入最重要的一步，当 BootRom（CSU ROM 代码的一部分）搬运 FSBL 到 OCM 后，处理器开始执行 FSBL 代码，FSBL 主要有以下几个作用：

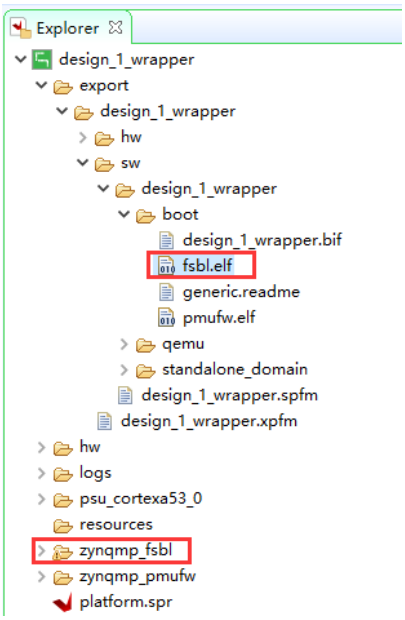
- 初始化 PS 端配置，MIO，PLL，DDR，QSPI，SD 等
- 如果有 PL 端程序，加载 PL 端 bitstream
- 搬运用户程序到 DDR，并跳转执行

Post-configuration stage：FSBL 开始执行后，CSU ROM 代码进入 post-configuration 阶段，负责起系统干预响应，CSU 为验证文件正确性、通过 PCAP 加载 PL、存储管理安全密钥、解密等提供持续的硬件支持。

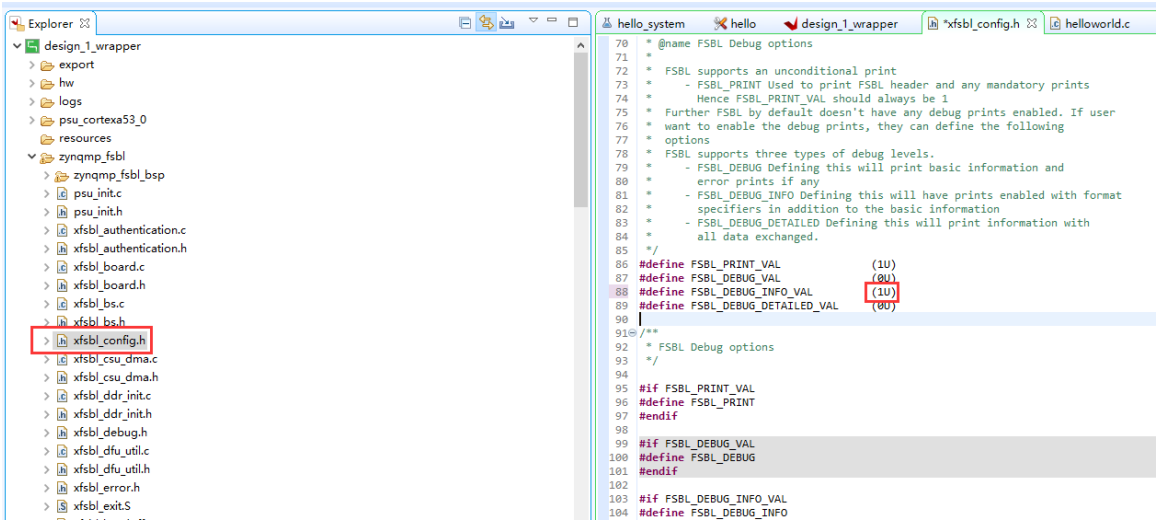
5.4.1 生成 FSBL

FSBL 是一个二级引导程序，完成 MIO 的分配、时钟、PLL、DDR 控制器初始化、SD、QSPI 控制器初始化，通过启动模式查找 bitstream 配置 FPGA，然后搜索用户程序加载到 DDR，最后交接给应用程序执行。

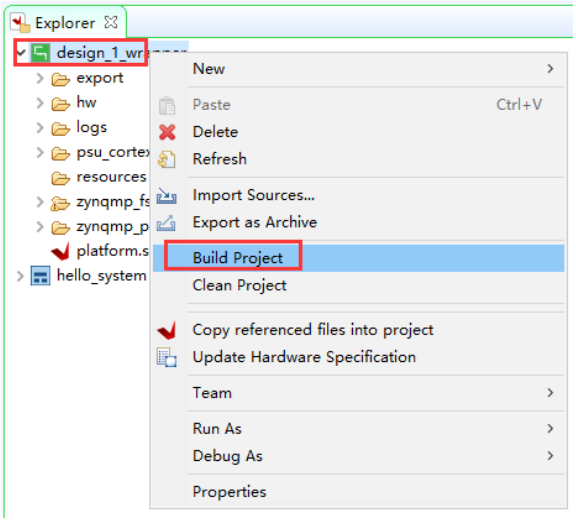
1) 由于在新建时选择了 Generate boot components 选项，所以 Platform 已经导入了 fsbl 的工程，并生成了相应的 elf 文件。



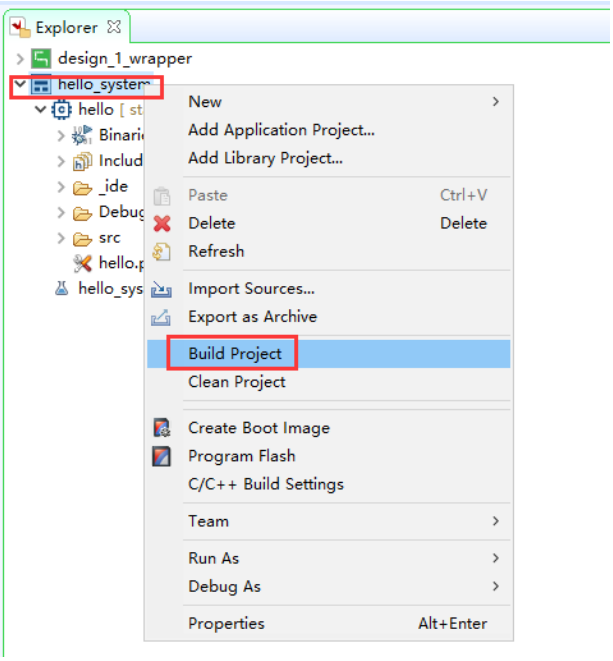
2) 修改调试宏定义 FSBL_DEBUG_INFO_VAL，可以在启动输出 FSBL 的一些状态信息，有利于调试，但是会导致启动时间变长。保存文件。可以看一下 fsbl 里包含了很多外设的文件，包括 psu_init.c，qspi，sd 等，大家可以再仔细读读代码。当然这个 fsbl 模板也是可以修改的，至于怎么修改根据自己的需求来做。



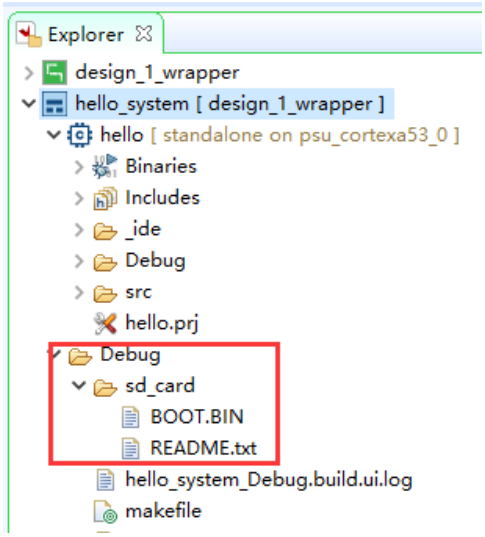
3) 重新 Build Project



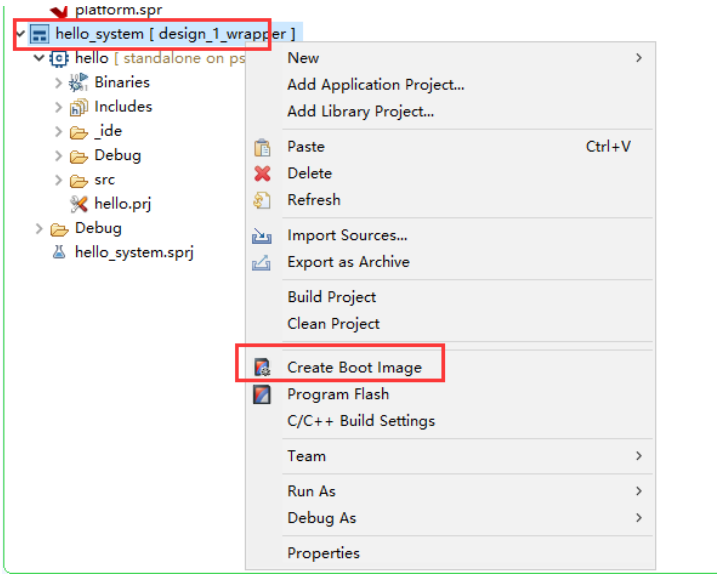
4) 接下来我们可以点击 APP 工程的 system，右键选择 Build project

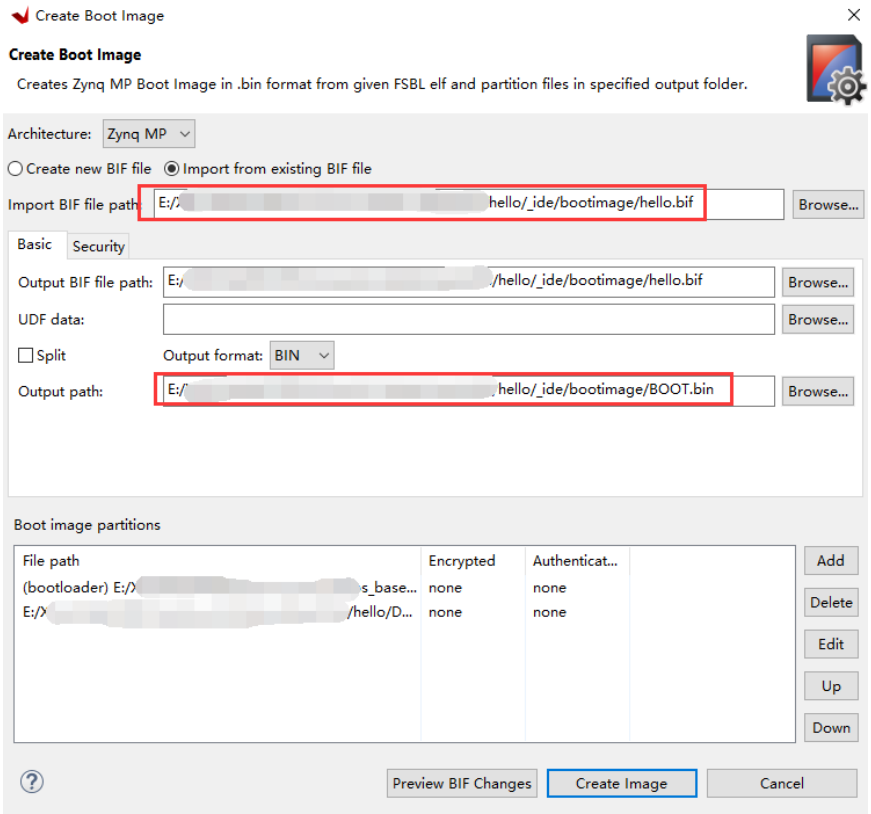


5) 这个时候就会多出一个 Debug 文件夹，生成了对应的 BOOT.BIN

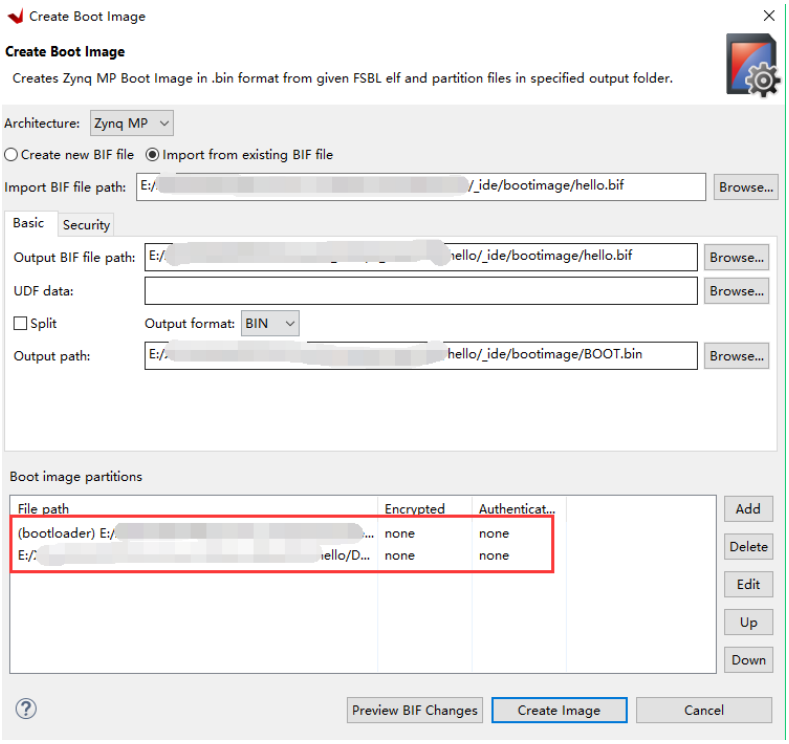


6) 还有一种方法就是，点击 APP 工程的 system 右键选择 Creat Boot Image，弹出的窗口中可以看到生成的 BIF 文件路径，BIF 文件是生成 BOOT 文件的配置文件，还有生成的 BOOT.bin 文件路径，BOOT.bin 文件是我们需要的启动文件，可以放到 SD 卡启动，也可以烧写到 QSPI Flash。





7) 在 Boot image partitions 列表中有要合成的文件，第一个文件一定是 bootloader 文件，就是上面生成的 fsbl.elf 文件，第二个文件是 FPGA 配置文件 bitstream，在本实验中由于没有 FPGA 的 bitstream，不需要添加，第三个是应用程序，在本实验中为 hello.elf，由于没有 bitstream，在本实验中只添加 bootloader 和应用程序。点击 Create Image 生成。

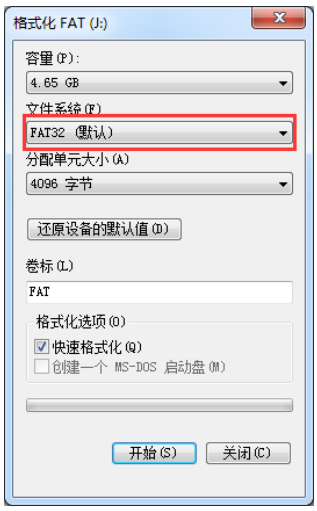


8) 在生成的目录下可以找到 BOOT.bin 文件

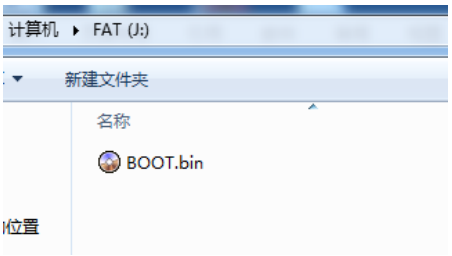


5.4.2 SD 卡启动测试

1) 格式化 SD 卡，只能格式化为 FAT32 格式，其他格式无法启动

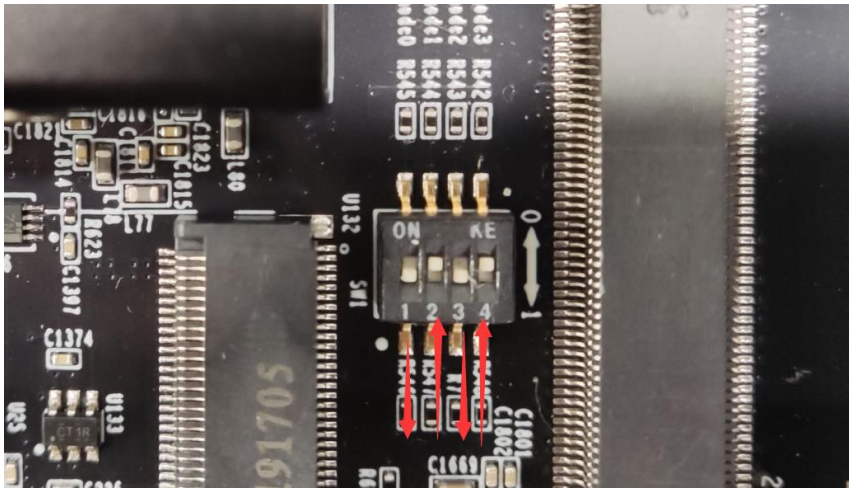


2) 放入 BOOT.bin 文件，放在根目录

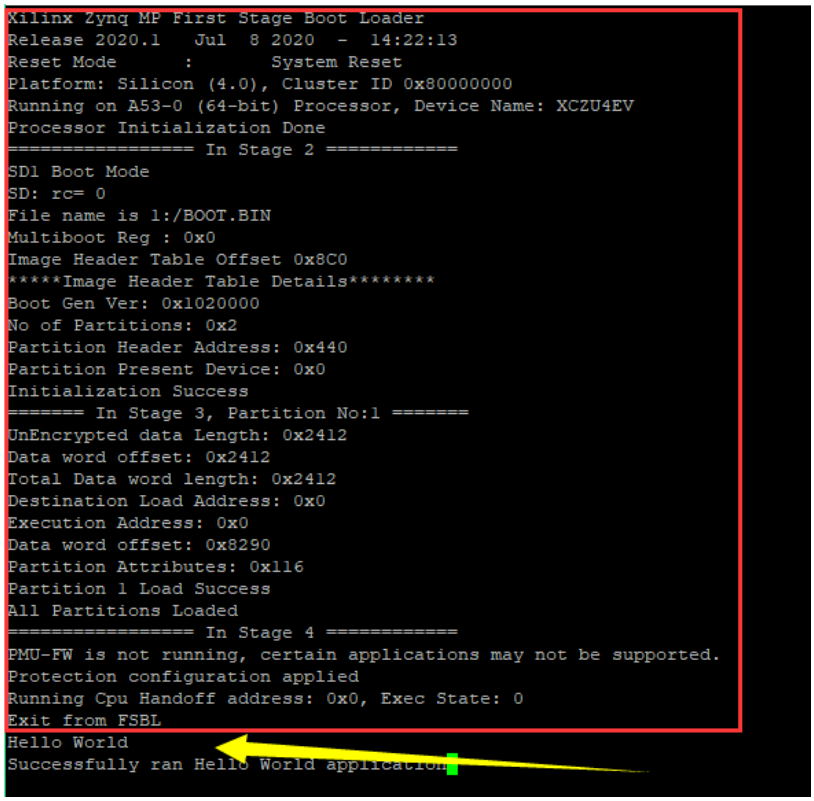


3) SD 卡插入开发板的 SD 卡插槽

4) 启动模式调整为 SD 卡启动

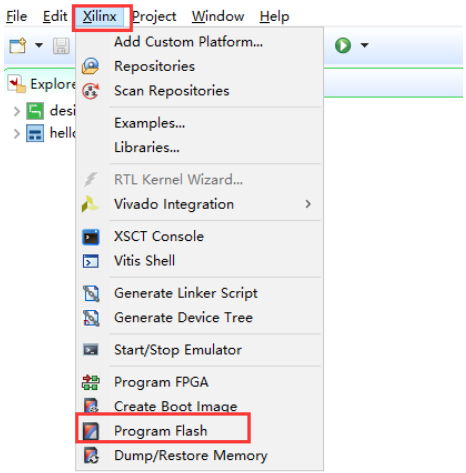


5) 打开串口软件，上电启动，即可看到打印信息，红色框为 FSBL 启动信息，黄色箭头部分为执行的应用程序 helloworld

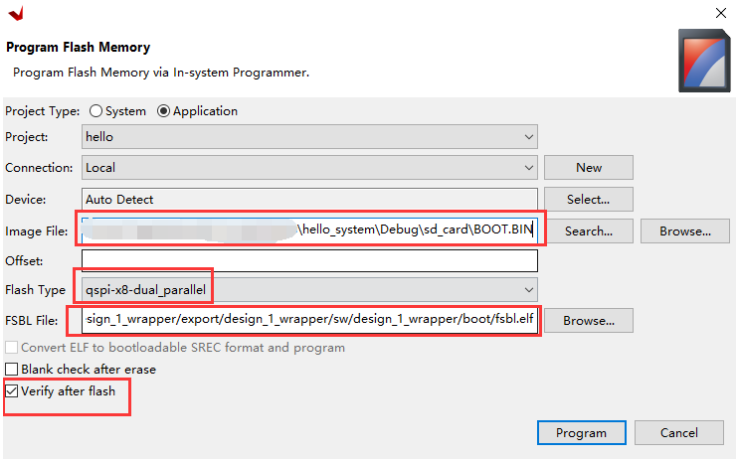


5.4.3 QSPI 启动测试

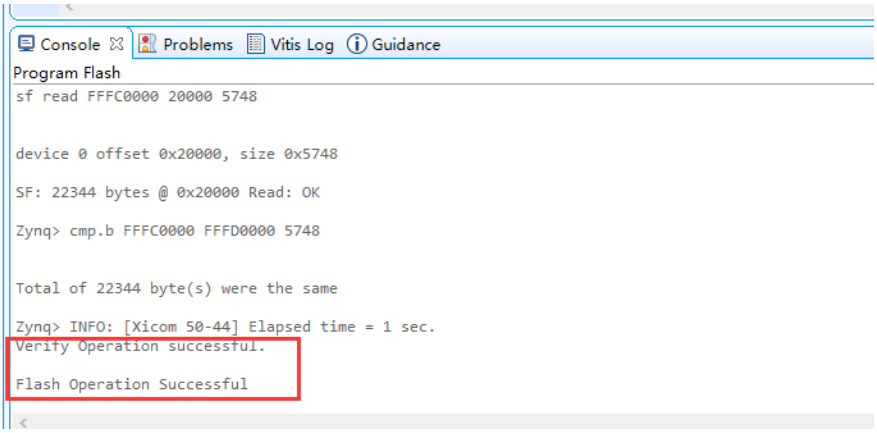
1) 在 Vitis 菜单 Xilinx -> Program Flash



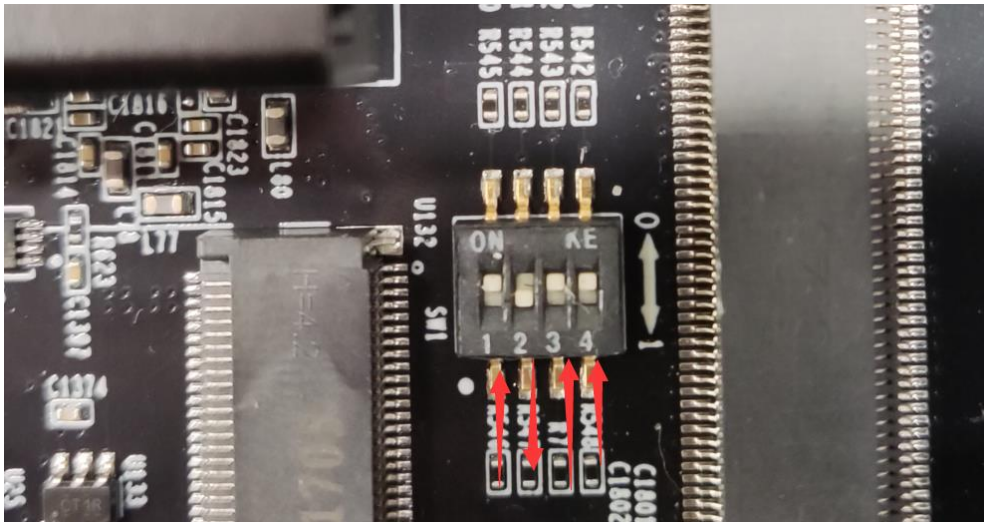
2) Hardware Platform 选择最新的，Image FIle 文件选择要烧写的 BOOT.bin，FSBL file 选择 fsbl.elf。选择 Verify after flash，在烧写完成后校验 flash。



3) 点击 Program 等待烧写完成



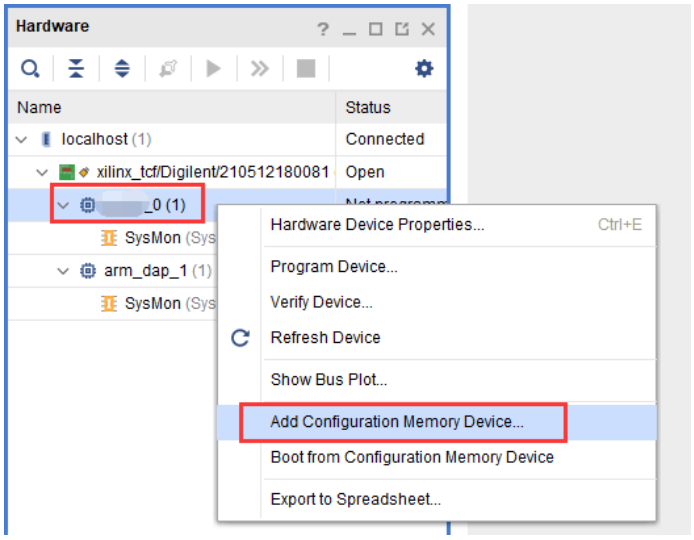
4) 设置启动模式为 QSPI，再次启动，可以在串口软件里看到与 SD 同样的启动效果。



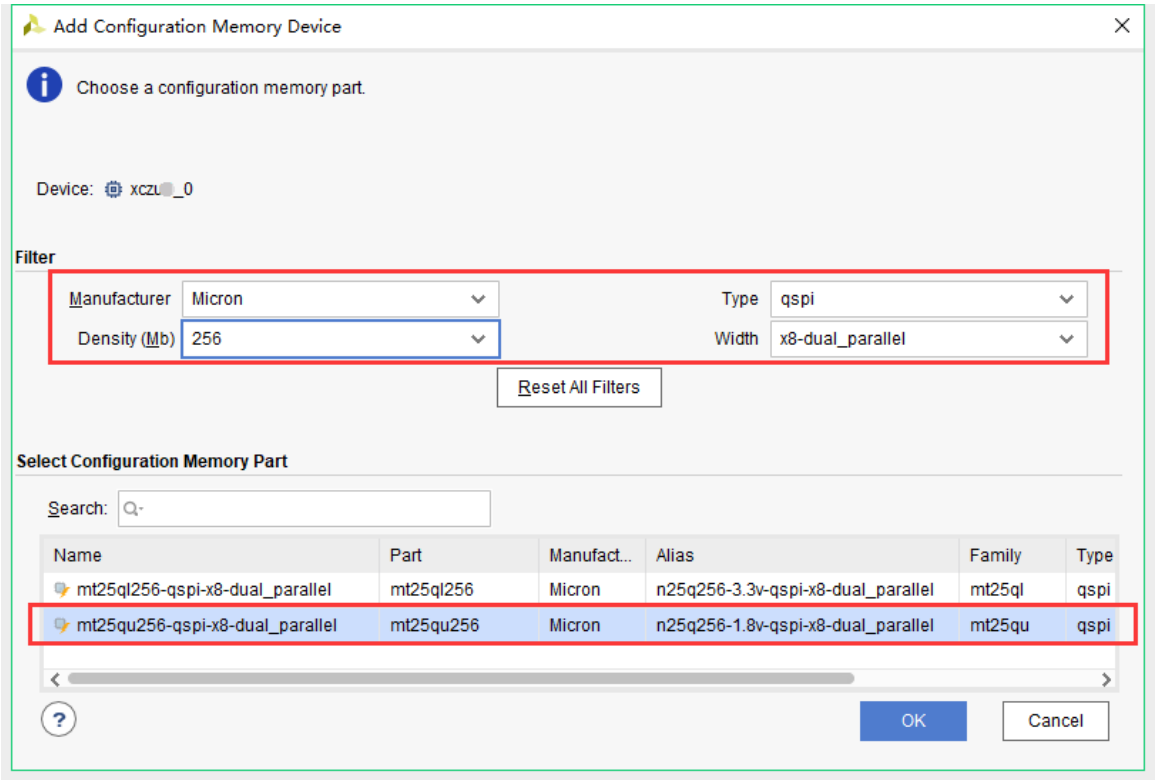
```
Xilinx Zynq MP First Stage Boot Loader
Release 2020.1   Sep 22 2020  -  14:28:07
Reset Mode:System Reset
Platform: Silicon (4.0), Cluster ID 0x80000000
Running on A53-0 (64-bit) Processor, Device Name: XCZU9EG
Processor Initialization Done
================== In Stage 2 ============
QSPI 32 bit Boot Mode
QSPI is in Dual Parallel connection
QSPI is using 4 bit bus
FlashID=0x20 0xBB 0x19
MICRON 256M Bits
Multiboot Reg : 0x0
QSPI Reading Src 0x0, Dest FFFF0040, Length EC0
.Image Header Table Offset 0x8C0
QSPI Reading Src 0x8C0, Dest FFFDD0C8, Length 40
.*****Image Header Table Details********
Boot Gen Ver: 0x1020000
No of Partitions: 0x2
Partition Header Address: 0x440
Partition Present Device: 0x0
QSPI Reading Src 0x1100, Dest FFFDD108, Length 40
.QSPI Reading Src 0x1140, Dest FFFDD148, Length 40
.Initialization Success
======= In Stage 3, Partition No:1 =======
UnEncrypted data Length: 0x2412
Data word offset: 0x2412
Total Data word length: 0x2412
Destination Load Address: 0x0
Execution Address: 0x0
Data word offset: 0x8290
Partition Attributes: 0x116
QSPI Reading Src 0x20A40, Dest 0, Length 9048
.Partition 1 Load Success
All Partitions Loaded
================== In Stage 4 ============
PMU-FW is not running, certain applications may not be supported.
Protection configuration applied
Running Cpu Handoff address: 0x0, Exec State: 0
Exit from FSBL
Hello World
Successfully ran Hello World application
```

5.4.4 Vivado 下烧写 QSPI

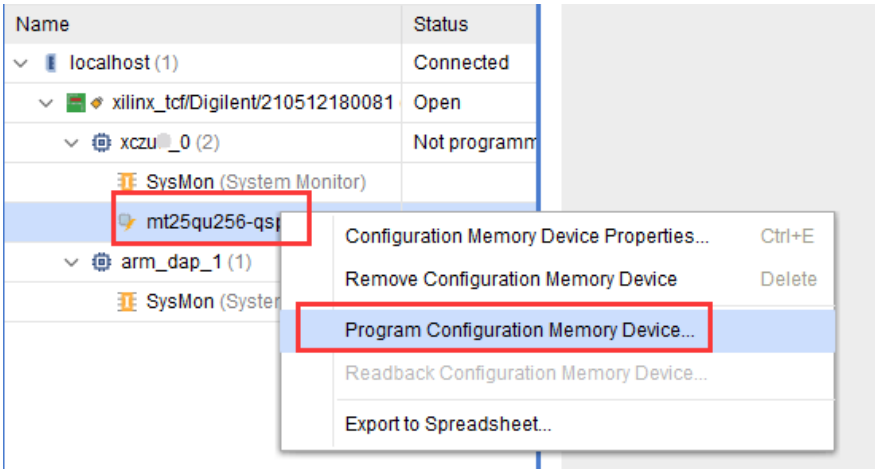
1) 在 HARDWARE MANGER 下选择器件，右键 Add Configuration Memory Device



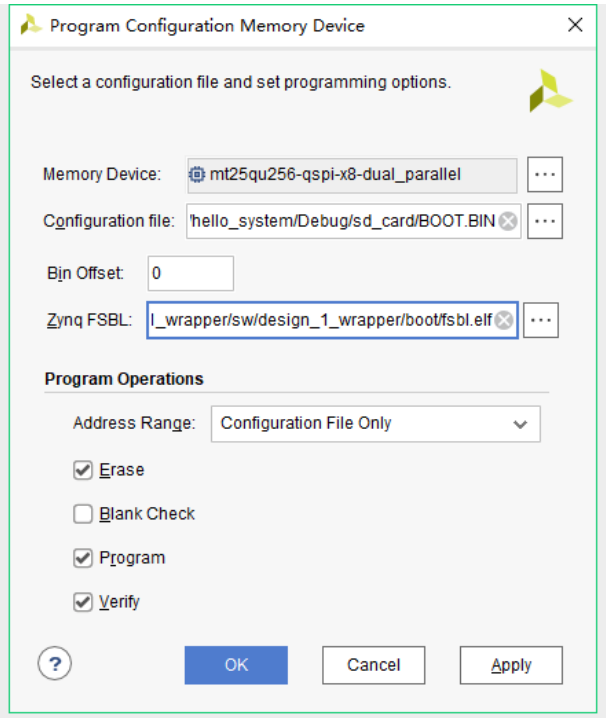
2) 选择尝试 Micron，类型选择 qspi，宽度选择 x8-dual_parallel，Density 选择 256，这时候出现 wt25qu256，选择红框型号。



3) 右键选择编程文件



4) 选择要烧写的文件和 fsbl 文件，就可以烧写了，如果烧写时不是 JTAG 启动模式，软件会给出一个警告，所以建议烧写 QSPI 的时候设置到 JTAG 启动模式

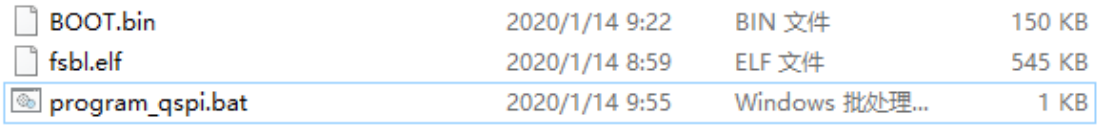


5.4.5 使用批处理文件快速烧写 QSPI

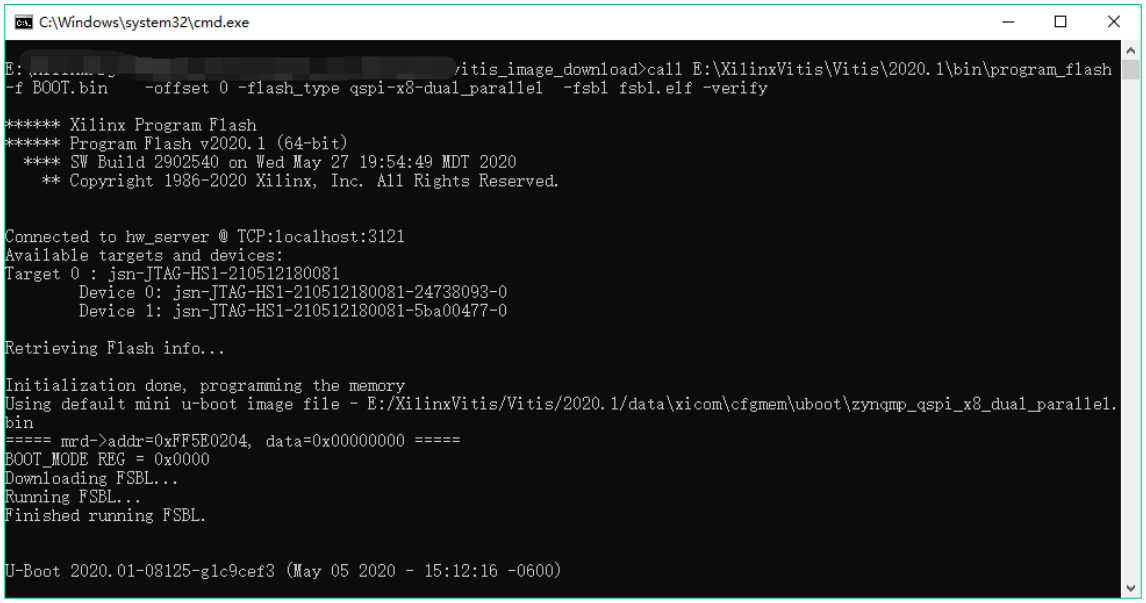
1) 新建一个 program_qspi.txt 文本文件，扩展名改为 bat,内容填写如下，E:\XilinxVitis\Vitis\2020.1\bin\program_flash 为我们工具路径，按照安装路径适当修改，-f 为要烧写的文件，-fsbl 为要烧写使用的 fsbl 文件，-verify 为校验选项。

```
call E:\XilinxVitis\Vitis\2020.1\bin\program_flash -f BOOT.bin
-offset 0 -flash_type qspi-x8-dual_parallel  -fsbl fsbl.elf -
verify
pause
```

2) 把要烧录的 BOOT.bin、fsbl、bat 文件放在一起



3) 插上 JTAG 线后上电，双击 bat 文件即可烧写 flash。

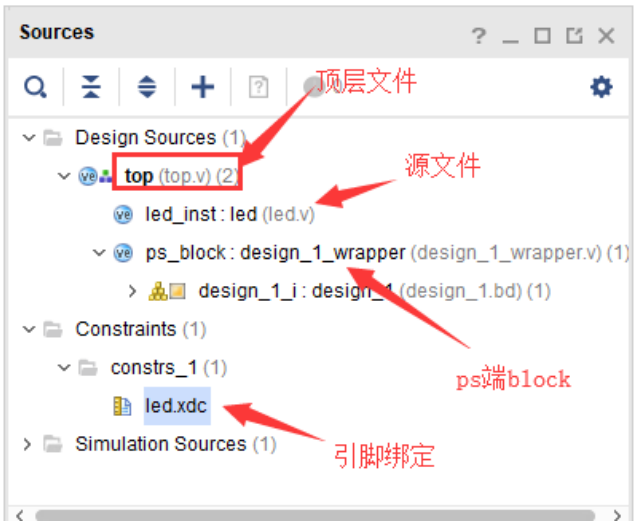


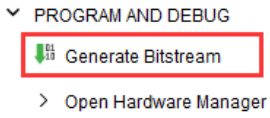
5.5 常见问题

5.5.1 仅有 PL 端逻辑的固化

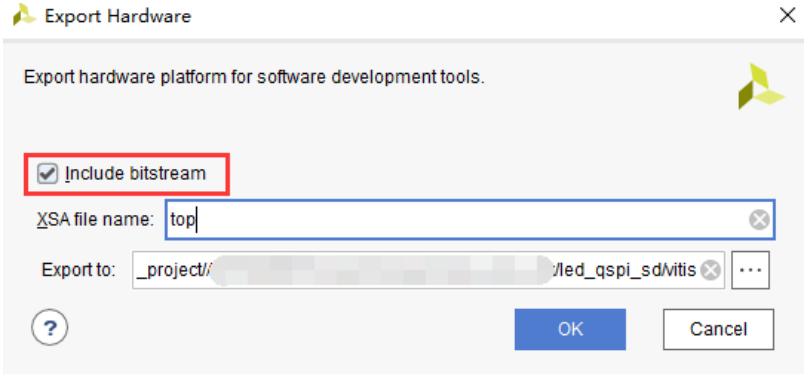
有很多人会问，如果只有 PL 端的逻辑，不需要 PS 端该怎么固化程序呢？不带 ARM 的 FPGA 固化是没问题的，但是对于 ZYNQ 来说，必须要有 PS 端的配合才能固化程序。那么对于前面的”PL 的 “Hello World”LED 实验”该怎么固化程序呢？

1. 根据本章的 PS 端添加 ZYNQ 核并配置，最简单的方法就是在本章工程的基础上添加 LED 实验的 verilog 源文件，并进行例化，组成一个系统，并需要生成 bitstream。

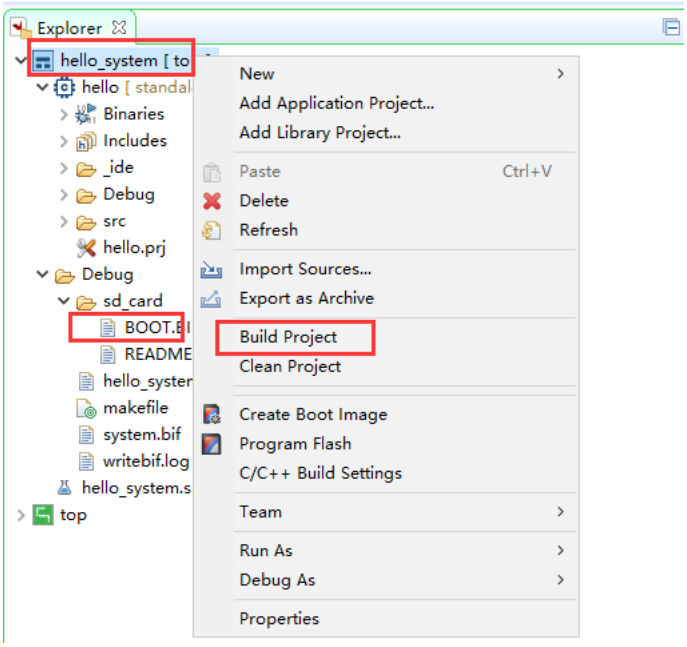




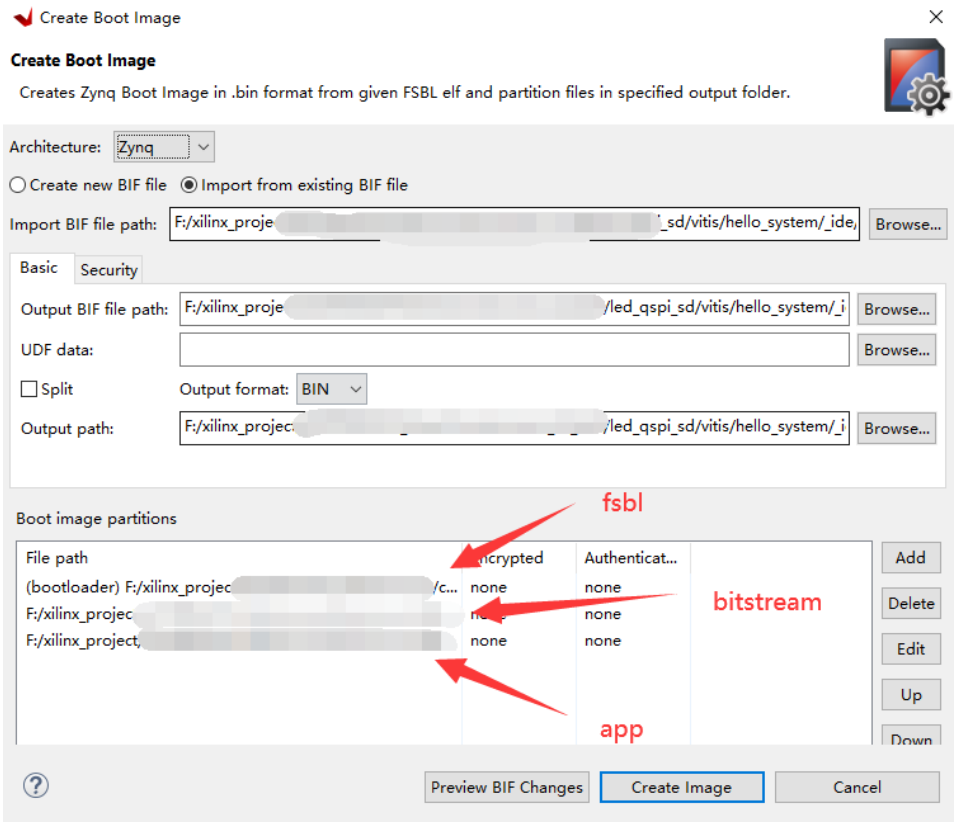
2. 生成 bitstream 之后，导出硬件，选择 include bitstream



3. 在生成 BOOT.BIN 时，还是需要一个 app 工程 hello，仅仅是为了生成 BOOT.BIN，默认情况下在 system 右键 Build Project，即可生成包含 bitstream 的 BOOT.BIN。



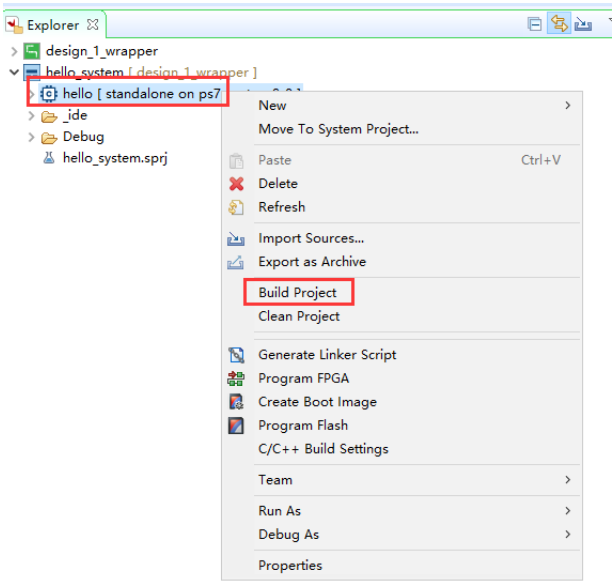
打开 Create Boot Image 界面可以看到，Boot Image Partitions 的文件顺序是 fsbl、bitstream、app，注意顺序不要颠倒，利用这样生成的 BOOT.BIN 就可以按照前面的启动方式测试启动了



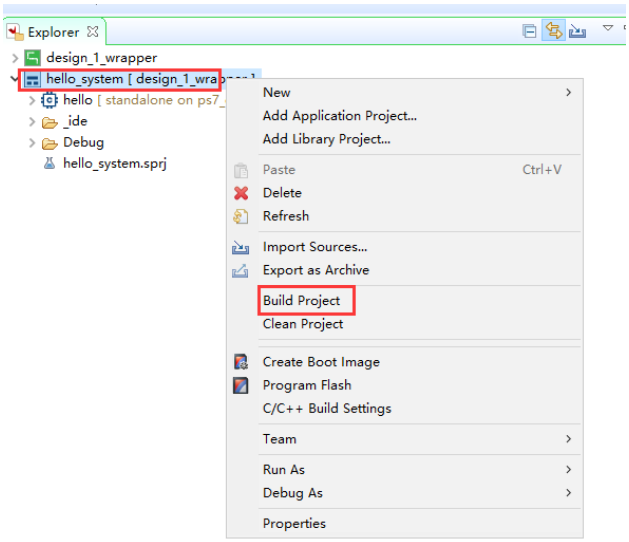
在 course_s2 文件夹，我们提供了一个名为 led_qspi_sd 的工程，大家可以参考。

5.6 使用技巧分享

在频繁的修改源文件，并进行编译的时候，最好选择 APP 工程进行 Build Project，这种情况下只会生成 elf 文件。



如果想生成 BOOT.BIN 文件，可以选择 system 进行编译，这种情况既会生成 elf 也会生成 BOOT.BIN，笔者最开始用的时候就吃过亏，每次编译都是选择 system，结果每次都要等待生成 BOOT.BIN，浪费时间，大家可以注意一下。



5.7 本章小结

本章从 FPGA 工程师和软件工程师两者角度出发，介绍了 ZYNQ 开发的经典流程，FPGA 工程师的主要工作是搭建好硬件平台，提供硬件描述文件 hdf 给软件工程师，软件工程师在此基础上开发应用程序。本章是一个简单的例子介绍了 FPGA 和软件工程师协同工作，后续还会牵涉到 PS 与 PL 之间的联合调试，较为复杂，也是 ZYNQ 开发的核心部分。

同时也介绍了 FSBL，启动文件的制作，SD 卡启动方式，QSPI 下载及启动方式，Vivado 下载 BOOT.BIN 方式，本章没有 FPGA 加载文件，后面的应用中会再介绍添加 FPGA 加载文件制作 BOOT.BIN。

后续的工程都会以本章节的配置为准，后面不再介绍 ZYNQ 的基本配置。

千里之行，始于足下，相信经过本章的学习，大家对 ZYNQ 开发有了基本概念，高楼稳不稳，要看地基打的牢不牢，虽然本章较为简单，但也有很多知识点待诸位慢慢消化。加油!!!

第六章 PS 端 RTC 中断实验

vivado 工程目录为“ps_hello/vivado”

vitis 工程目录为“ps_rtc/vitis”

6.1 RTC 介绍

实时时钟（rtc）单元为系统和应用软件提供精确的时间基准。为了满足高精度的需要，实时时钟还包括校准电路，以补偿温度和电压波动。RTC 由 VCC-PSAUX 或 VCC-PSBATT 电源供电。当辅助电源可用时，RTC 使用它保持计数器活动。当辅助电源不可用时，RTC 自动切换到 VCC PSBATT 电源。RTC 功能如下：

1） 系统掉电时，该单元自动切换至电池供电，实现时钟的不间断运行

2） 支持报警设置和周期中断设置

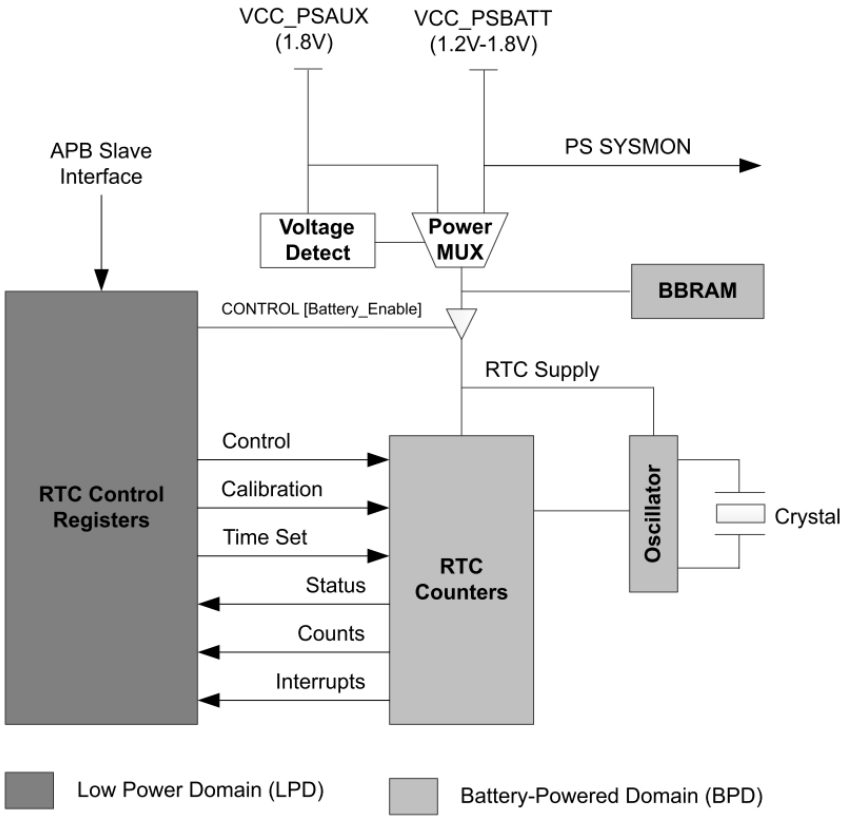
3） 校准电路以确保时间精准

4） 三个计数器

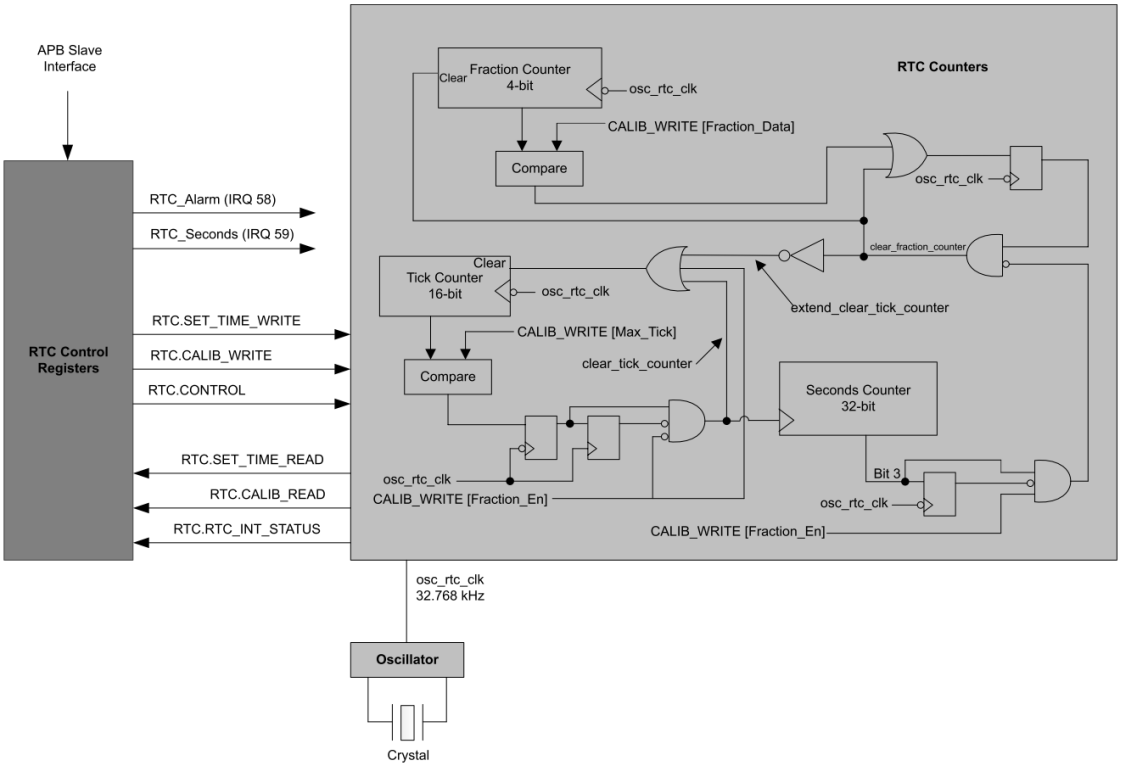
时间秒计数器，32 位，可以计数约 136 年

32 KHz 参考时钟计数器，表示 1 秒的计数

用于校准的 4 位分数计数器



RTC 控制器结构图



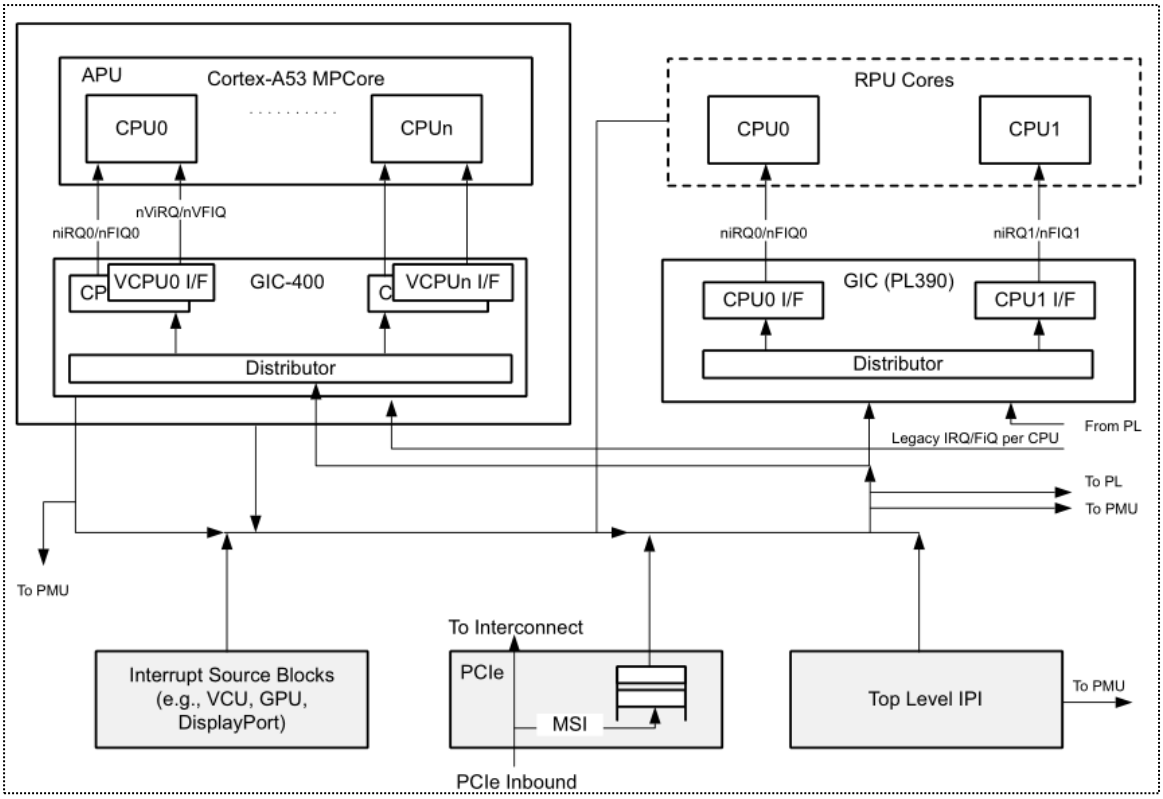
RTC 功能图

6.2 中断介绍

1）ARM cortex-A 系列处理器，提供了 4 个管脚给 soc，实现外界中断的传递。分别是：nIRQ、nFIQ、nVIRQ、nVFIQ。arm 系统中，会有多个外设，均有可能产生中断发送给 core，因此就需要有一个中断控制器来作为中间的桥接，收集 soc 的所有中断信号，然后仲裁选择合适（高优先级）的中断，再发送给 CPU，等待 CPU 处理。

2）这里中间的桥接，就是 arm 公司推出的大名鼎鼎的 gic（general interrupt controller）。gic 其实是一个架构，版本历经了 gicv1，gicv2，gicv3，gicv4。

3）Ultrascale+中断框图如下

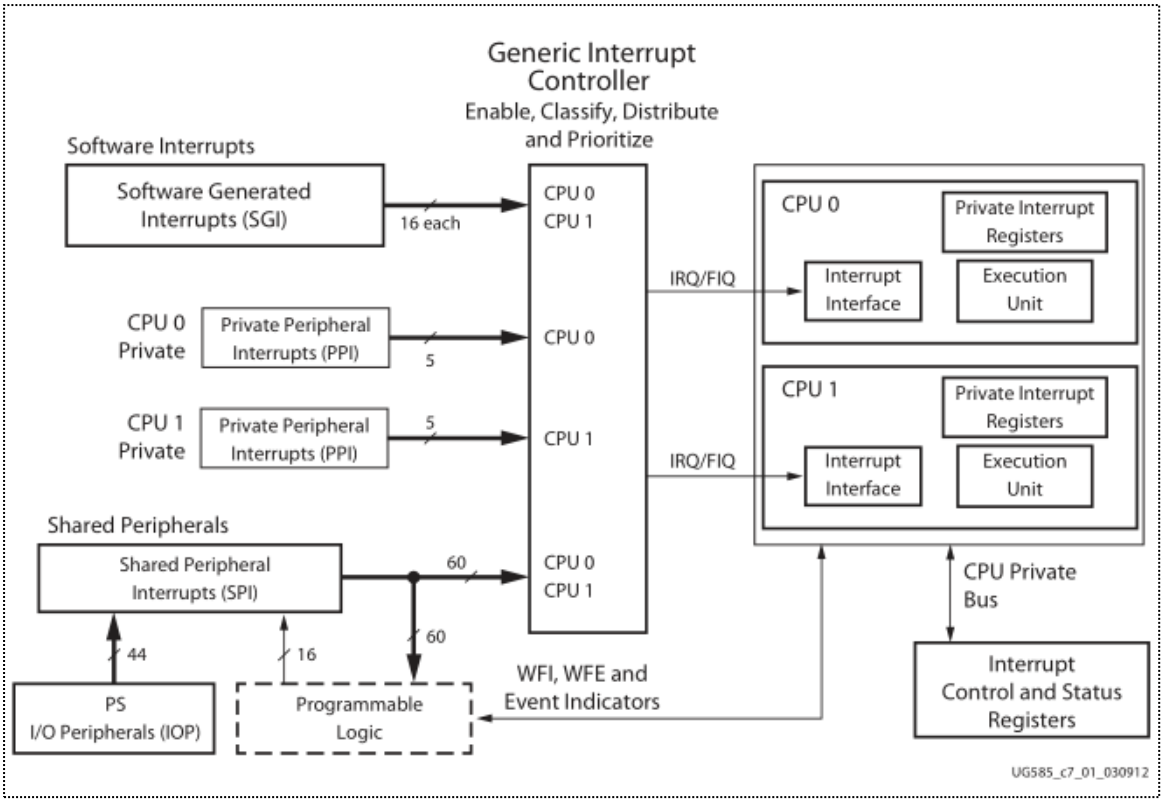


图中包含两个 GIC：

RPU GIC：PL390(arm 公司设计的对应 GICv1 IP)

APU GIC：GIC-400(arm 公司设计的对应 GICv2 IP)

4） RPU GIC，其系统功能框图如下：



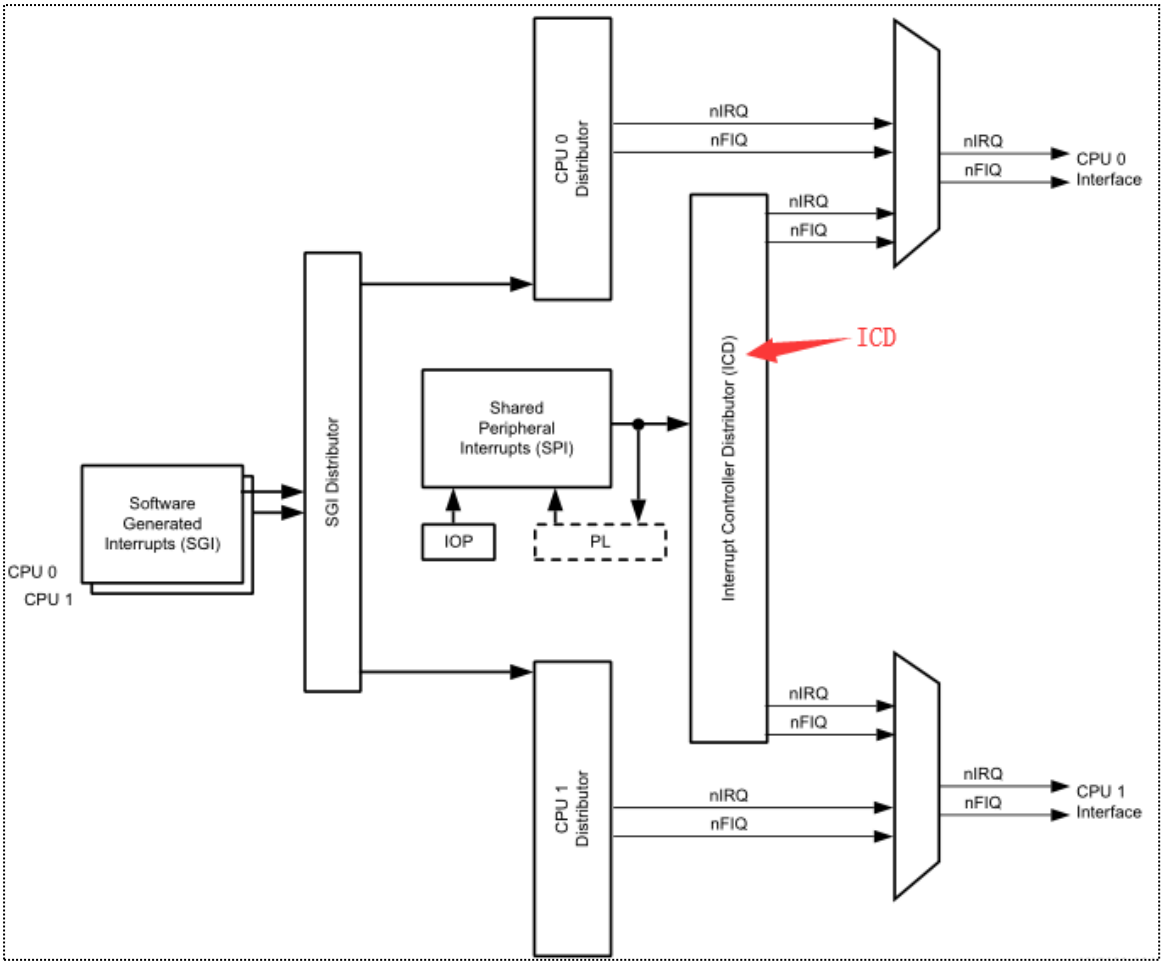
从图中可以看到，中断源主要有三种：

PPI：private peripheral interrupt，私有外设中断，该中断来源于外设，只对固定的 core 有效。

SPI：shared peripheral interrupt，共享外设中断，该中断来源于外设，可以对所有的 core 有效。

SGI：software-generated interrupt，软中断，软件产生的中断，用于给指定的 core 发送中断信号

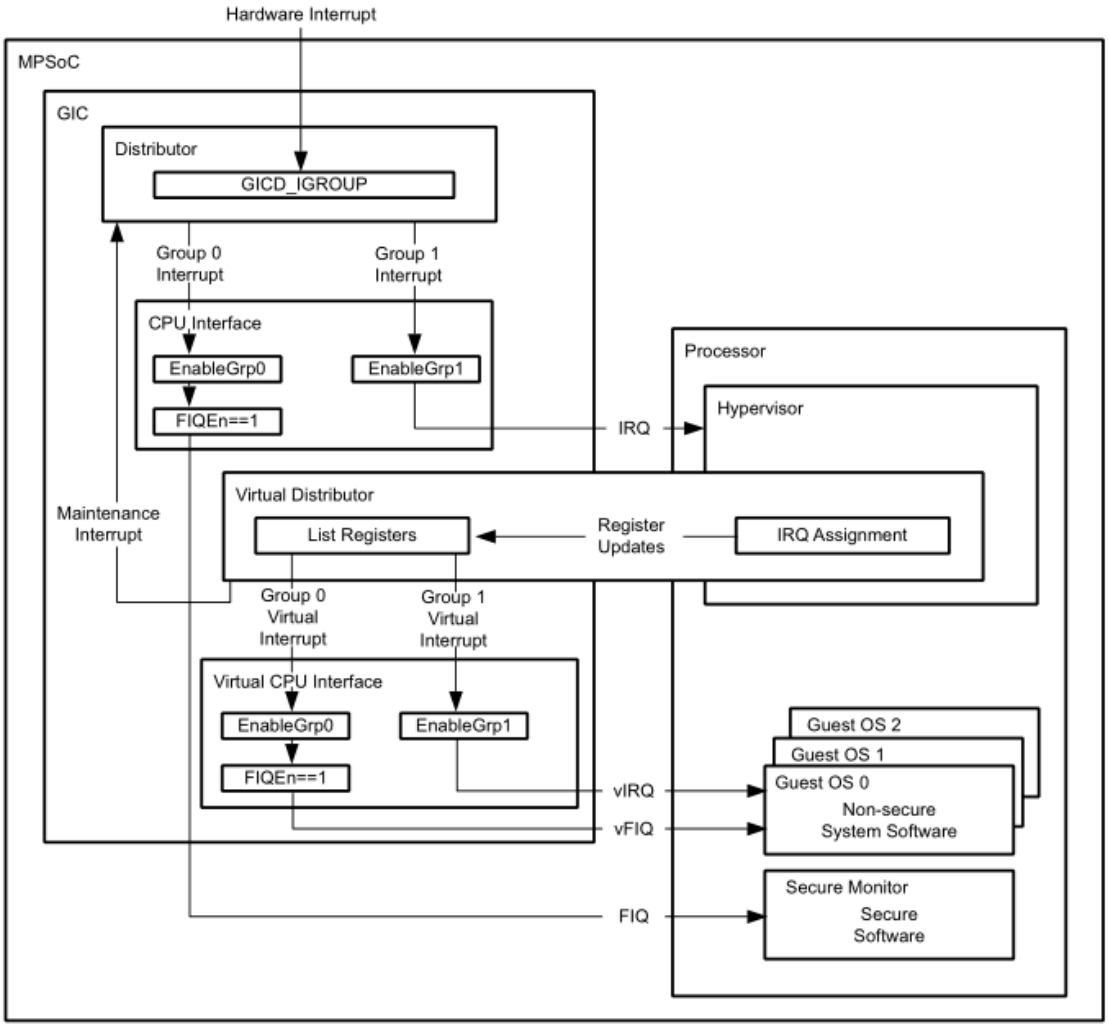
控制器功能框图如下：



上图中， ICD 的控制寄存器主要如下：

Starting Address	Register Set	Count	Description
RPU - Private CPU Bus for RPU MPCore			
0xF900_0000	PL390.enable	1	Interrupt control register (ICDICR).
0xF900_0080	PL390.sgi_security_if_n	1	SGI interrupt security register (ICDISR).
0xF900_0084	PL390.spi_security	5	SPI interrupt security register (ICDISR).
0xF900_0104	PL390.spi_enable_set	5	SPI enable set register (ICDISER).
0xF900_0184	PL390.spi_enable_clr	5	SPI interrupt clear-enable registers (ICDICER).
0xF900_0200	PL390.sgi_pending_set_if_n	1	SGI interrupt set-pending registers (ICDISPR).
0xF900_0204	PL390.spi_pending_set	5	SPI interrupt set-pending registers (ICDISPR).
0xF900_0280	PL390.sgi_pending_clr_if_n	1	SGI pending clear register (ICDICPR).
0xF900_0284	PL390.spi_pending_clr	5	SPI pending clear register (ICDICPR).
0xF900_0300	PL390.sgi_active_if_n	1	SGI active bit registers (ICDABR).
0xF900_0304	PL390.spi_active	5	SPI active bit registers (ICDABR).
0xF900_0400	PL390.priority_sgi_if_n	16	SGI interrupt priority registers (ICDIPR).
0xF900_0420	PL390.priority_spi	160	SPI interrupt priority registers (ICDIPR).
0xF900_0820	PL390.targets_spi	160	SPI target register interrupt (ICDIPTR).
0xF900_0C08	PL390.spi_config	5	SPI interrupt configuration register Interrupt (ICDICR).

4) APU GIC，功能框图如下



GICv2，将中断分成了 group0 和 group1。使用寄存器 GICD_IGROUPRn 来对每个中断，设置组。其中 group0：安全中断，由 nFIQ 驱动，group1：非安全中断，由 nIRQ 驱动。支持最大 1020 个中断。其中断号分配如下：

中断号	分配	中断来源	寄存器
ID0-ID7	非安全软中断	软件	GICD_SGIR
ID8-ID15	安全软中断	软件	GICD_SGIR
ID16-ID31	私有中断	外设	no
ID32-ID1019	共享中断	外设	no

GICv2，主要由两部分组成：distributor 及 cpu interface。

distributor，用来收集所有的中断来源，并且为每个中断源设置中断优先级，中断分组，中断目的 core。当有中断产生时，将当前最高优先级中断，发送给对应的 cpu interface。其功

能有：全局中断使能、每个中断的使能、中断的优先级、中断的分组、中断的目的 core、中断触发方式、对于 SGI 中断，传输中断到指定的 core、每个中断的状态管理、提供软件，可以修改中断的 pending 状态

cpu interface，将 GICD 发送的中断信息，通过 IRQ，FIQ 管脚，发送给连接到该 cpu 接口的 core。其功能有：将中断请求发送给 cpu、对中断进行认可（acknowledging an interrupt）、中断完成识别(indicating completion of an interrupt)、设置中断优先级屏蔽、定义中断抢占策略、决定当前处于 pending 状态最高优先级中断

gicv2，定义了自己的一些寄存器，这些寄存器，都是使用 memory-mapped 的方式去访问的，也就是在 soc 中，会留有一片空间，给 gic。cpu 通过访问这部分空间，来对 gic 进行操作。主要寄存器如下：

APU - AXI Interconnect with Access Restricted to APU MPCore			
0xF901_0000	GIC400.GICD	180	Display controller.
0xF902_0000	GIC400.GICC	15	CPU interface.
0xF904_0000	GIC400.GICH	99	Hypervisor.
0xF906_0000	GIC400.GICV	14	Virtual machine.

这里中断只是作简单介绍，需要详细了解的，请参照 xilinx 提供的文档：ug1085-zynq-ultrascale-trm.pdf。

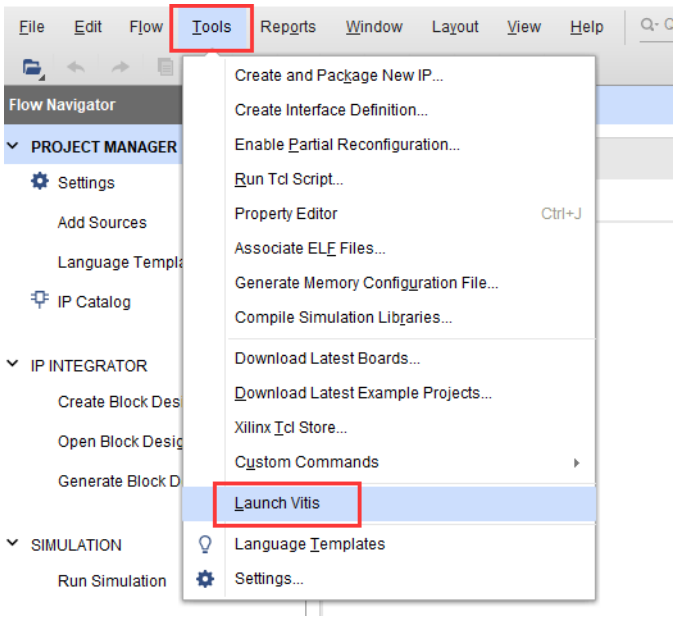
软件工程师工作内容

以下为软件工程师负责内容。

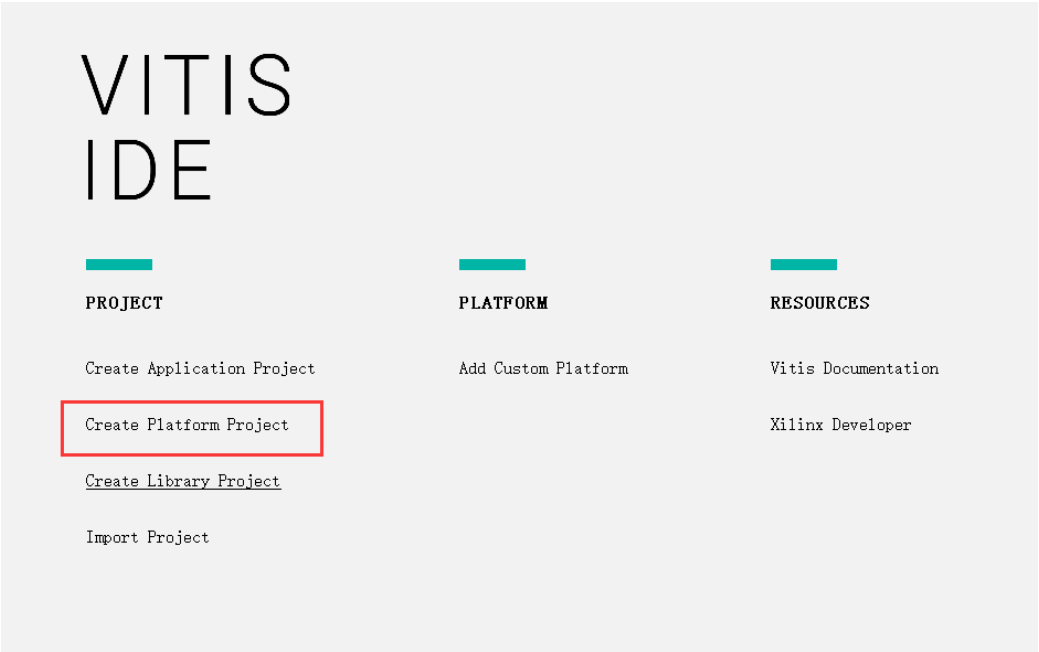
6.3 Vitis 程序编写

6.3.1 创建 Platform 工程

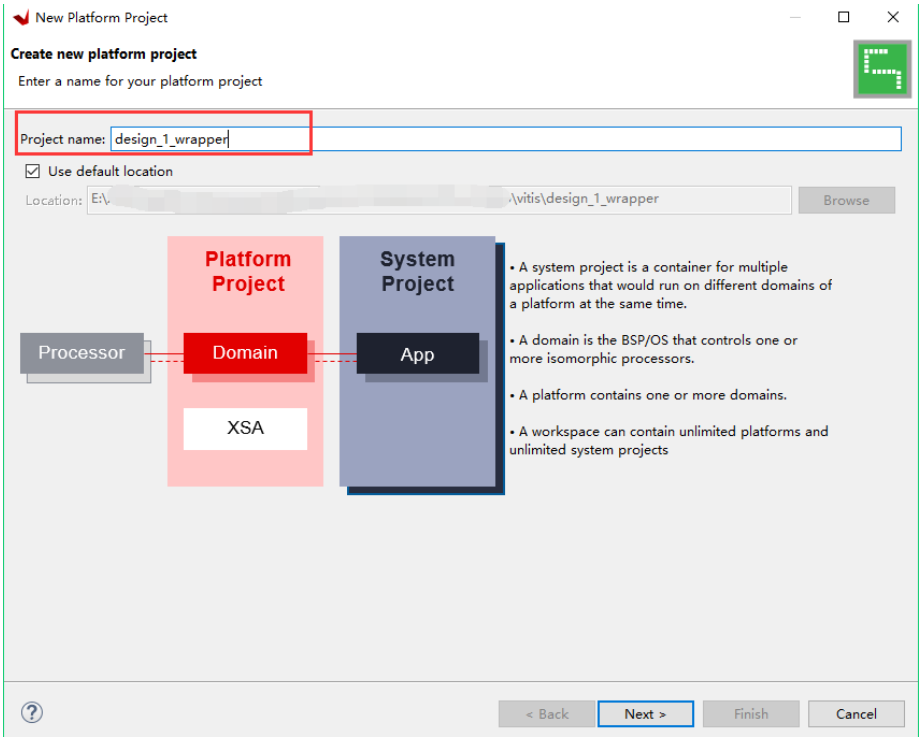
1) 点击 Tools→Launch Vitis



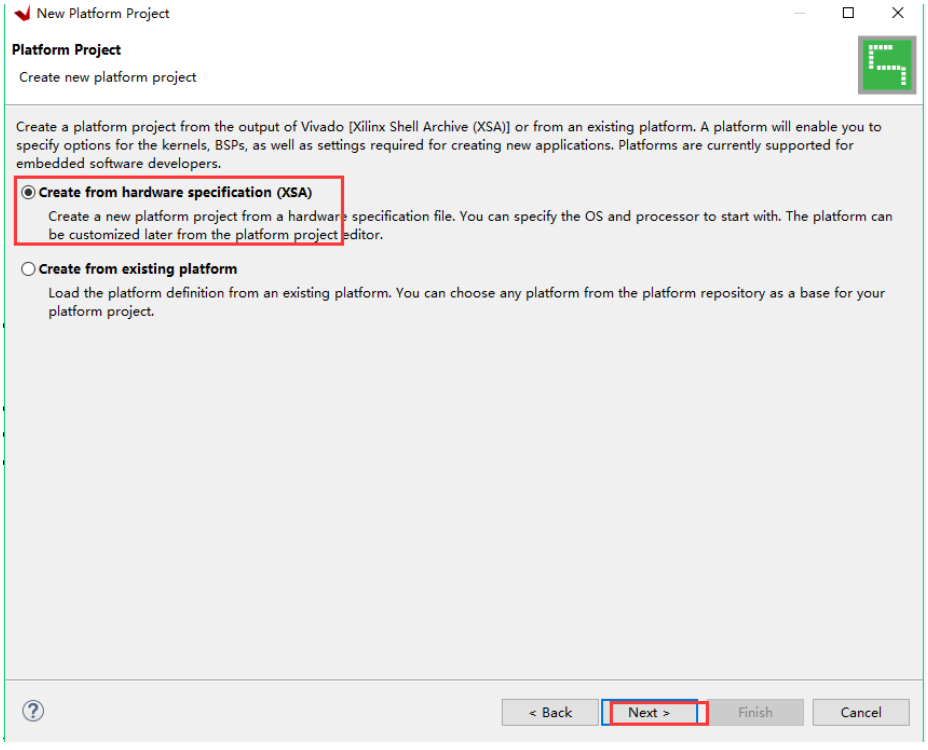
2) 与前面的 Hello World 实验不同，我们只建立 Platform 工程



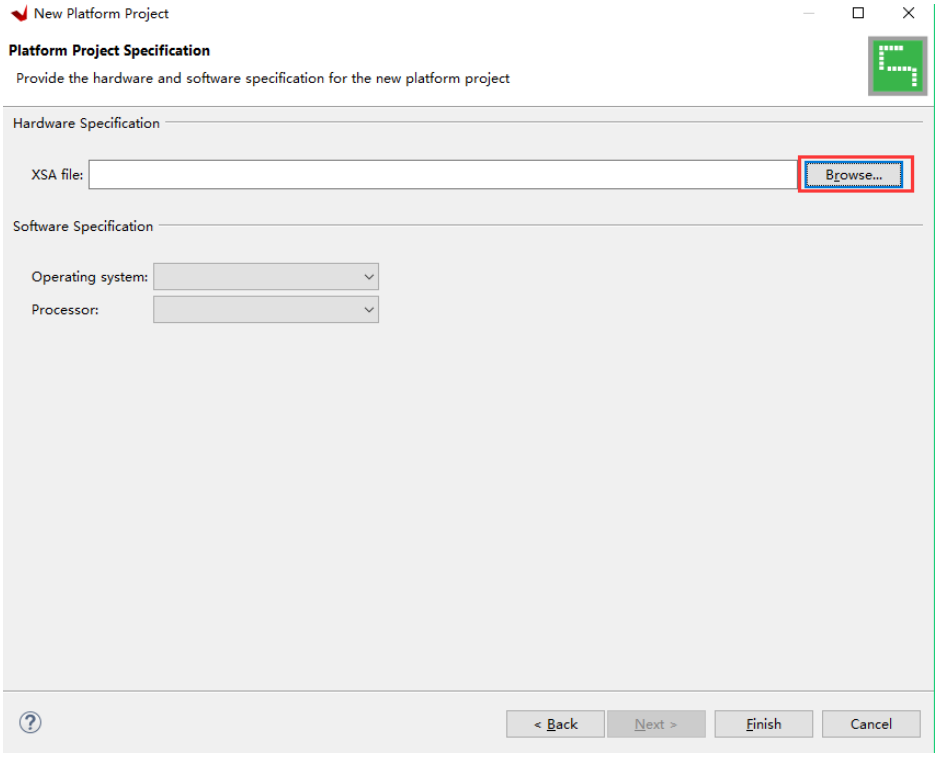
3) 填入工程名字，要与 XSA 文件的名字一样，点击 Next



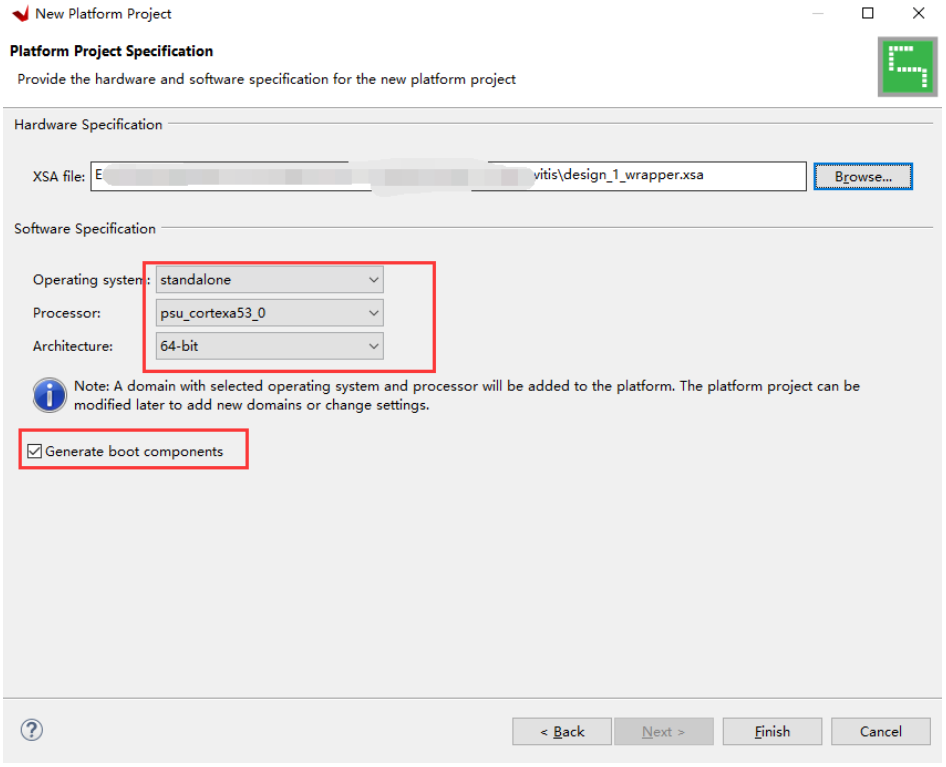
4) 点击 Next



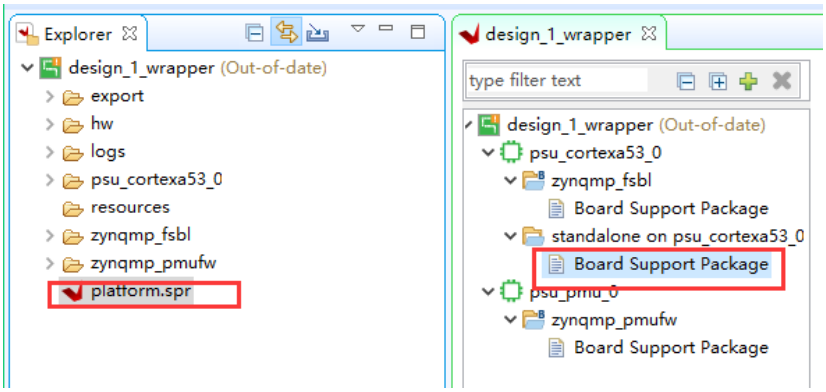
5) 选择 XSA 文件



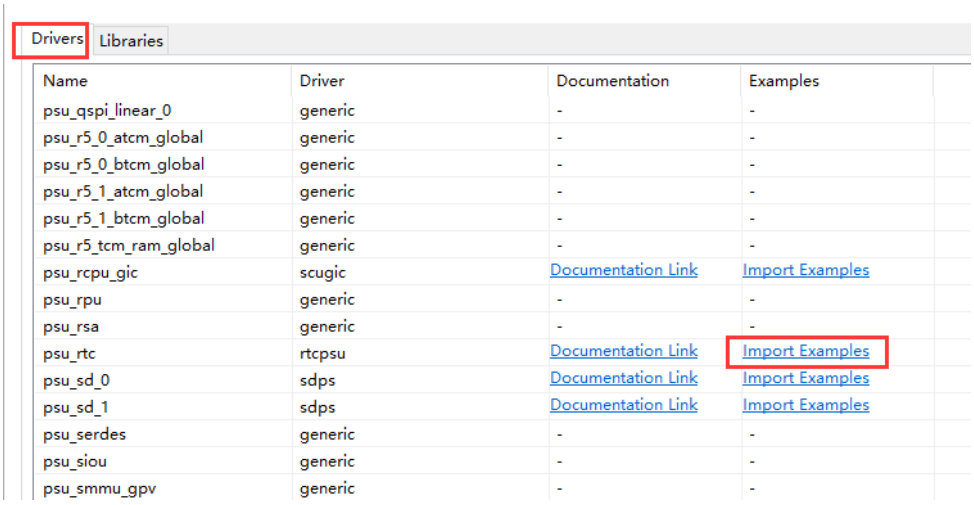
保持默认，点击 Finish



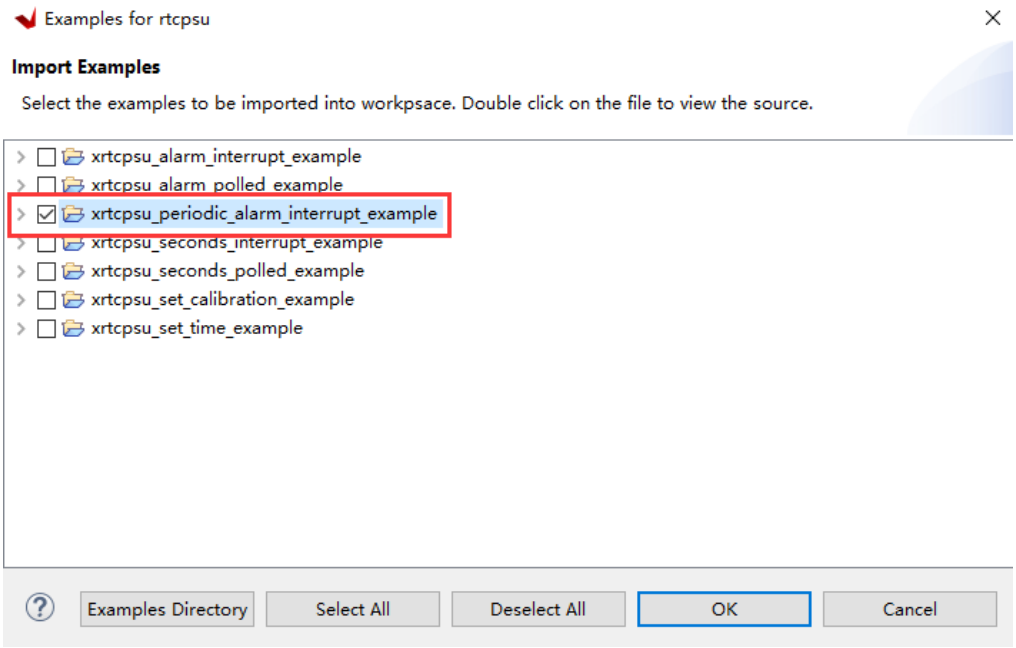
6) 点开 platform.spr，并点开 BSP



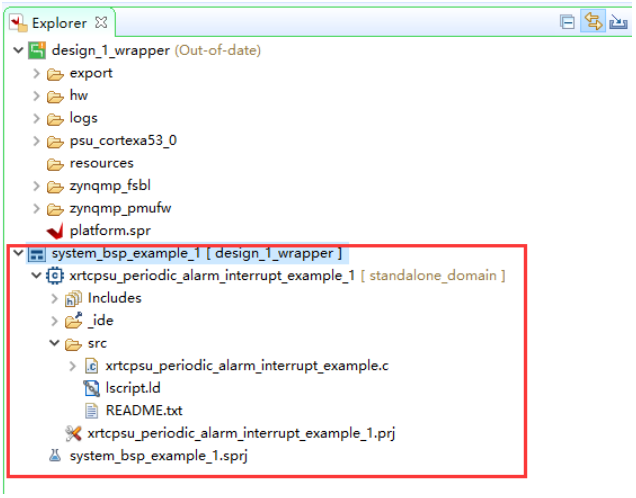
7) 找到 RTC 驱动，并点击 Import Examples



8) 非常幸运，有中断的例子，怎么就知道这个例子就是中断的例子呢？是通过“intr”猜测的，所以，基本功很重要，不然你连找例程都不会。



9) 在这里就导入了 example 工程



下面就是阅读代码，然后修改代码了，当然，可能一下不能完全理解这些代码，只能在以后的应用中去反复练习

10) 通过函数 XRtcPsu_GetCurrentTime 来获取系统秒计数器值，用函数 XRtcPsu_SecToDateTime 将计数值转换为我们能看明白的年月日时分秒。

```
xil_printf("\n\rDay Convention : 0-Fri, 1-Sat, 2-Sun, 3-Mon, 4-Tue, 5-Wed, 6-Thur\n\r");
xil_printf("Current RTC time is..\n\r");
CurrentTime = XRtcPsu_GetCurrentTime(RtcInstPtr);
XRtcPsu_SecToDateTime(CurrentTime,&dt0);
xil_printf("YEAR:MM:DD HR:MM:SS \t %04d:%02d:%02d %02d:%02d:%02d\t Day = %d\n\r",
        dt0.Year,dt0.Month,dt0.Day,dt0.Hour,dt0.Min,dt0.Sec,dt0.WeekDay);
```

11) 设置中断时间，中断时间 PERIODIC_ALARM_PERIOD 宏定义为 2，也就是 2 秒中断一次

```
/*
 * Setup the handlers for the RTC that will be called from the
 * interrupt context when alarm and seconds interrupts are raised,
 * specify a pointer to the RTC driver instance as the callback reference
 * so the handlers are able to access the instance data
 */
XRtcPsu_SetHandler(RtcInstPtr, (XRtcPsu_Handler)Handler, RtcInstPtr);

/*
 * Enable the interrupt of the RTC device so interrupts will occur.
 */
XRtcPsu_SetInterruptMask(RtcInstPtr, XRTC_INT_EN_ALRM_MASK );

CurrentTime = XRtcPsu_GetCurrentTime(RtcInstPtr);
Alarm = CurrentTime + PERIODIC_ALARM_PERIOD;
XRtcPsu_SetAlarm(RtcInstPtr,Alarm,1U);

while( PeriodicAlarms != REPETATIONS);

/*
 * Disable the interrupt of the RTC device so interrupts will not occur.
 */
XRtcPsu_ClearInterruptMask(RtcInstPtr,XRTC_INT_DIS_ALRM_MASK);
XRtcPsu_ResetAlarm(RtcInstPtr);
```

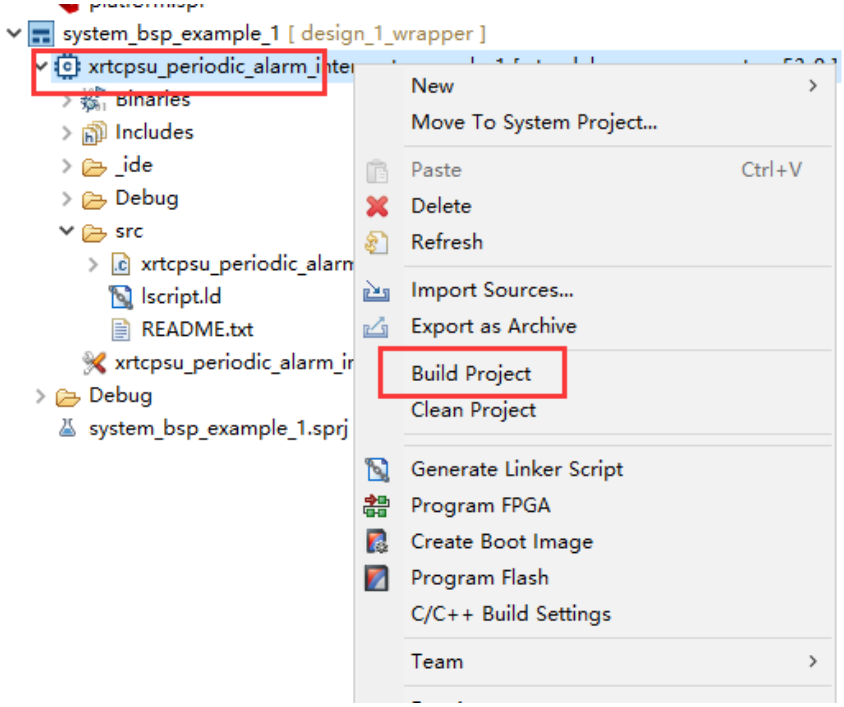
中断处理函数

设置中断时间

连续中断

等待10次中断

12) Build Project 编译



13) 了解一下中断控制器的使用，主要分为几个步骤，初始化中断控制器 *GIC→初始化中断异常→中断服务函数注册→在中断控制器中使能中断→使能中断异常*。有两步需要注意，*在中断控制器中使能中断* 是要根据中断号使能相应的中断，比如本章介绍的 RTC，是在中断控制器 GIC 中的使能中断，而后面的*使能外设中断* 是指在外设中打开它的中断，正常情况下是不打开的，打开之后就可以产生中断传递到中断控制器 GIC。在以后的实验中可以借鉴这种写法。

```
static int SetupInterruptSystem(XScuGic *IntcInstancePtr,
                XRtcPsu *RtcInstancePtr,
                u16 RtcIntrId)
{
    int Status;

#ifndef TESTAPP_GEN
    XScuGic_Config *IntcConfig; /* Config for interrupt controller */

    /* Initialize the interrupt controller driver */
    IntcConfig = XScuGic_LookupConfig(INTC_DEVICE_ID);
    if (NULL == IntcConfig) {
        return XST_FAILURE;
    }

    Status = XScuGic_CfgInitialize(IntcInstancePtr, IntcConfig,
                    IntcConfig->CpuBaseAddress);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }

    /*
     * Connect the interrupt controller interrupt handler to the
     * hardware interrupt handling logic in the processor.
     */
    Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_INT,
                (Xil_ExceptionHandler) XScuGic_InterruptHandler,
                IntcInstancePtr);
#endif

    /*
     * Connect a device driver handler that will be called when an
     * interrupt for the device occurs, the device driver handler
     * performs the specific interrupt processing for the device
     */
    Status = XScuGic_Connect(IntcInstancePtr, RtcIntrId,
                (Xil_ExceptionHandler) XRtcPsu_InterruptHandler,
                (void *) RtcInstancePtr);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }

    /* Enable the interrupt for the device */
    XScuGic_Enable(IntcInstancePtr, RtcIntrId);

#ifndef TESTAPP_GEN
    /* Enable interrupts */
     Xil_ExceptionEnable();
#endif

    return XST_SUCCESS;
}
```

中断初始化

中断异常服务函数注册

中断服务函数注册

使能RTC中断

使能中断异常

6.4 下载调试

1) 打开串口终端
2) 下载调试程序的方法前面教程已经讲解，不再复述
3) 和我们预期一样，串口每两秒会看断一次

```
Day Convention : 0-Fri, 1-Sat, 2-Sun, 3-Mon, 4-Tue, 5-Wed, 6-Thur
Current RTC time is..
YEAR:MM:DD HR:MM:SS      2000:01:03 00:18:09     Day = 3
2Sec Periodic Alarm generated.
2Sec Periodic Alarm generated.
2Sec Periodic Alarm generated.
2Sec Periodic Alarm generated.
2Sec Periodic Alarm generated.
2Sec Periodic Alarm generated.
2Sec Periodic Alarm generated.
2Sec Periodic Alarm generated.
2Sec Periodic Alarm generated.
2Sec Periodic Alarm generated.
Successfully ran RTC Periodic Alarm Interrupt Example Test
```

6.5 实验总结

实验中通过简单的修改 Vitis 的例程，就完成了 RTC，中断的应用，看似简单的操作，但蕴含了丰富的知识，我们需要非常了解 RTC 的原理、中断的原理，这些基本知识是学习好 ZYNQ 的必要条件。

第七章 PS 端 MIO 的使用

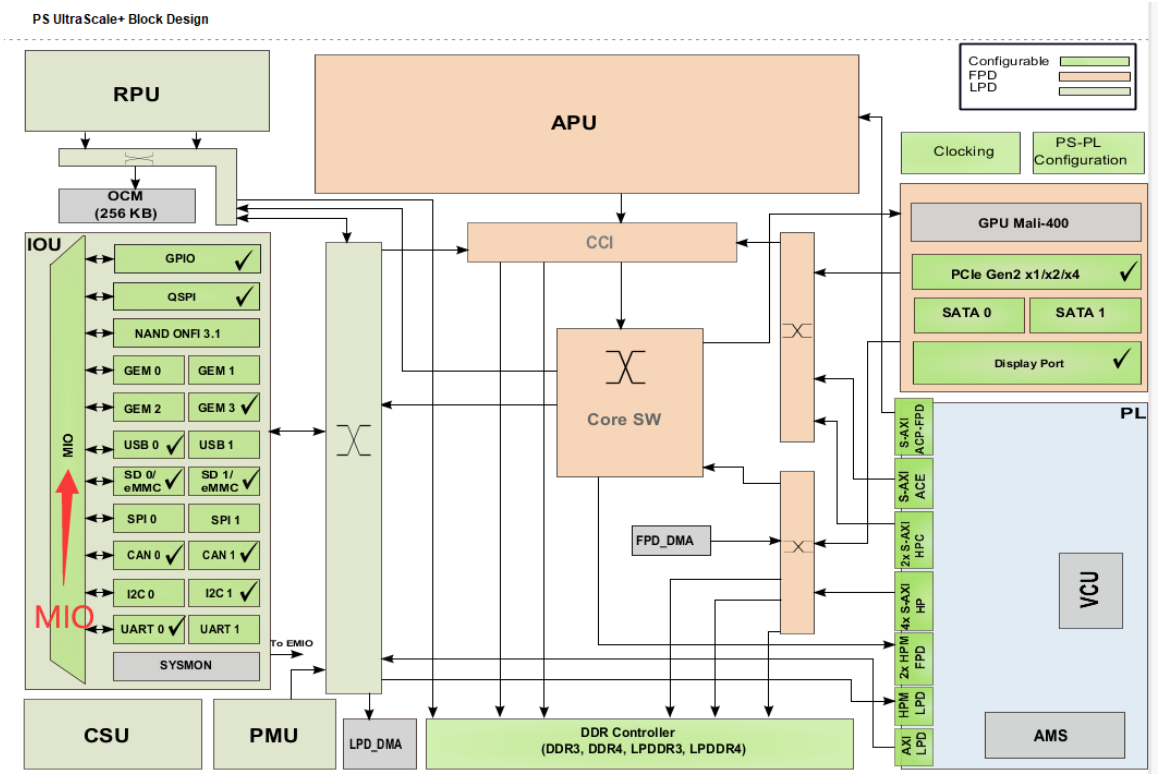
vivado 工程目录为“ps_hello/vivado”

vitis 工程目录为“ps_mio/vitis”

软件工程师工作内容

以下为软件工程师负责内容。

本章介绍 PS 端 MIO 的操作，MIO 是基础的外设 IO，可以连接诸如 SPI，I2C，UART，GPIO 等，通过 VIVADO 软件设置，软件可以将信号通过 MIO 导出，同样也可以将信号通过 EMIO 连接到 PL 端的引脚上。



本实验通过实现 PS 端 LED 灯的闪烁演示 MIO 的操作。

7.1 原理介绍

先来了解 GPIO 的 BANK 分布，在 UG1085 文档 GPIO 一章中可以看到 GPIO 是有 6 个 BANK。

BANK0 有 26 个信号，控制 MIO[0:25]，BANK1 有 26 个信号，控制 MIO[26:51]，BANK2 有 26 个信号，控制 MIO[52:77]，MIO 总共有 78 个引脚，也就是诸如 SPI,I2C,USB,SD 等 PS 端外设接口；

BANK3~BANK5 总共能控制 96 个 PL 端引脚，注意每一组都有三个信号，输入 EMIOGPIOI，输出 EMIOGPIOO，输出使能 EMIOGPIOTN，类似于三态门，共 288 个信号。可以连接到 PL 端引脚，通过 PS 端控制。

本章主要讲 MIO 的控制。

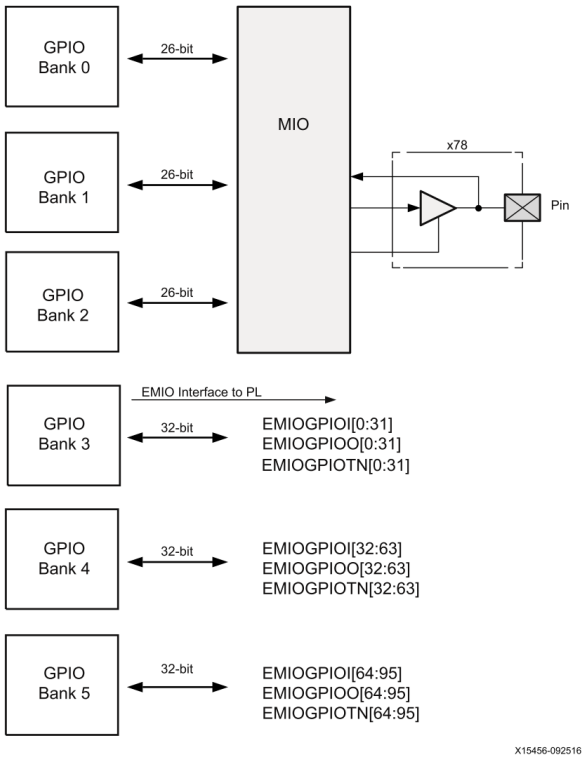
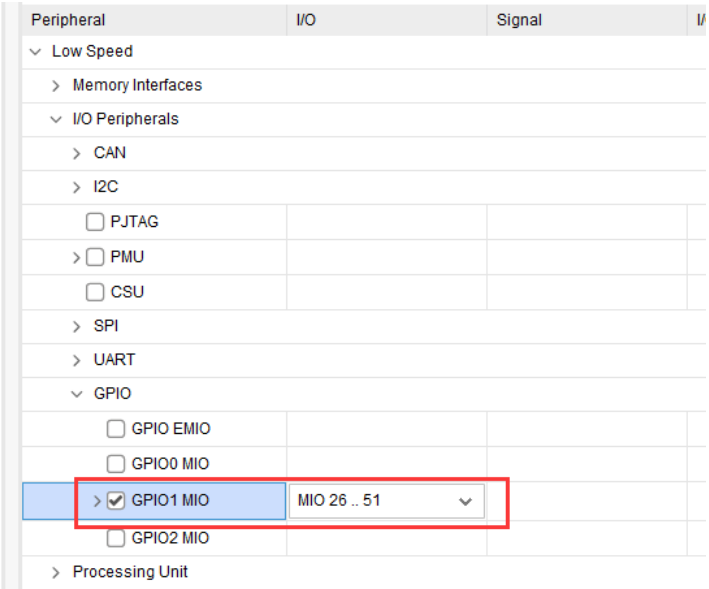


Figure 27-1: **GPIO Block Diagram**

7.2 Vivado 工程

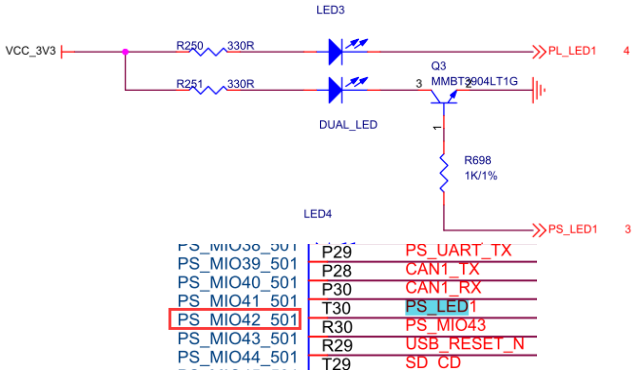
本实验仍是基于“ps_hello”工程。通过原理图我们得知 PS 端的 LED 和按键是在 GPIO1，也就是 MIO BANK1 上的，因此需要打开 GPIO1 MIO，在“ps_hello”工程中已经配置过。



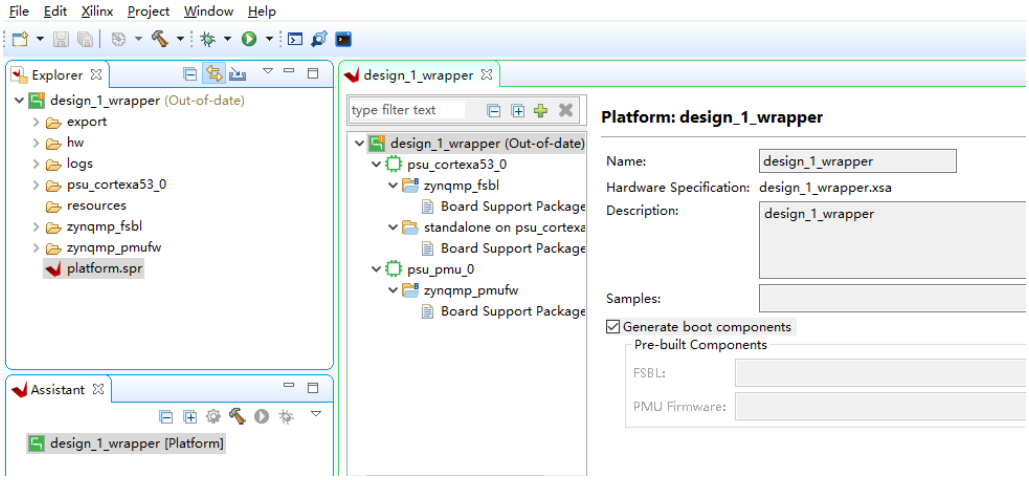
7.3 Vitis 程序开发

7.3.1 MIO 点亮 PS 端 LED 灯

根据原理图得知， LED 灯连接到了 PS 端的 MIO42，可以根据相应的开发板 MIO 的位置控制 LED 灯。



1) 新建 platform 工程过程不再赘述，参考“PS 端 RTC 中断实验”一章



2) 下图为 GPIO 的控制框图，实验中会用到输出部分的寄存器，数据寄存器 DATA，数据掩码寄存器 MASK_DATA_LSW，MASK_DATA_MSW，方向控制寄存器 DIRM，输出使能控制器 OEN。

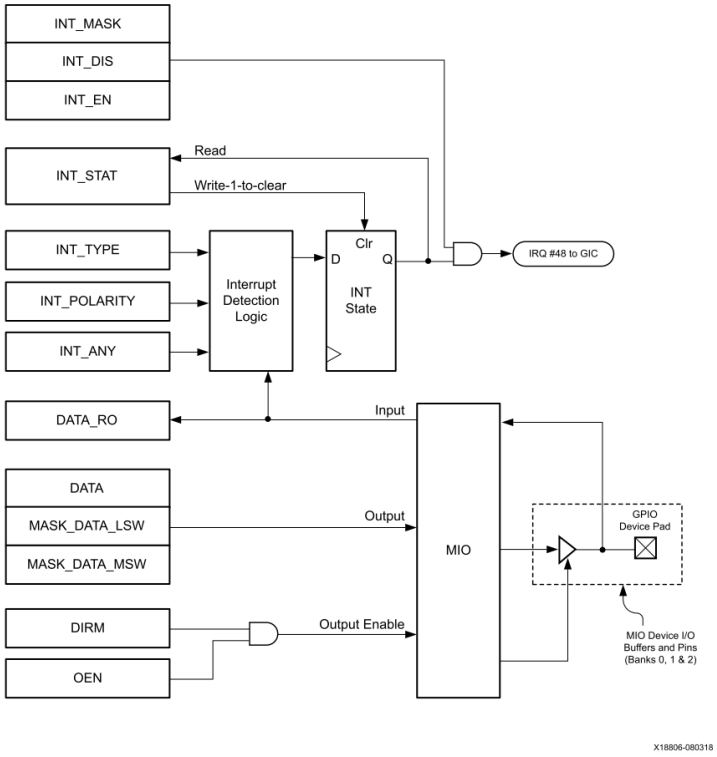
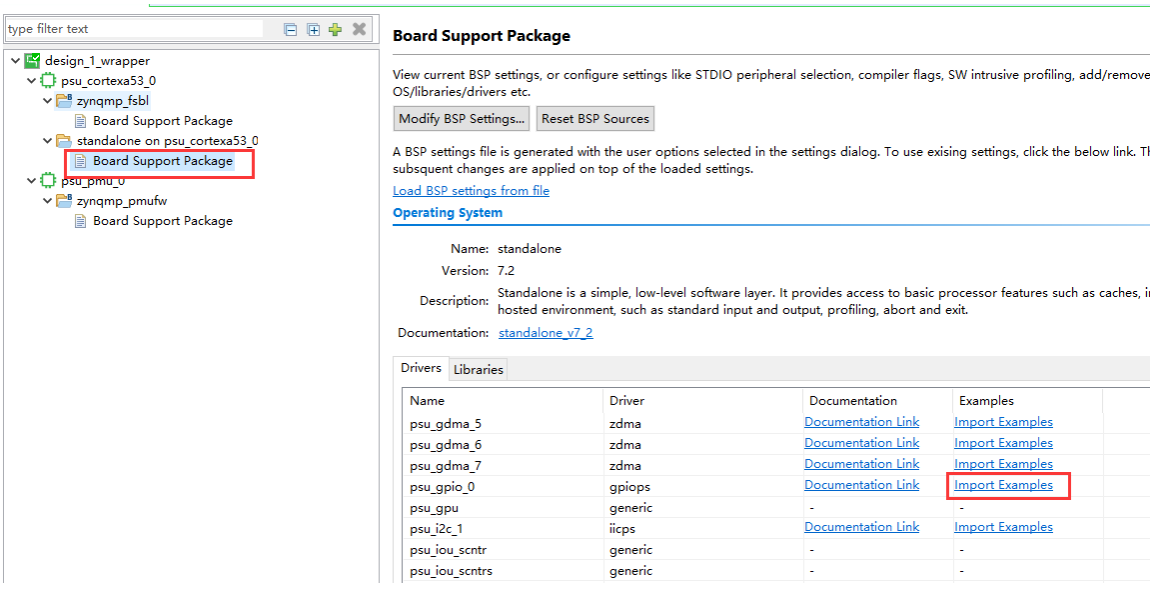
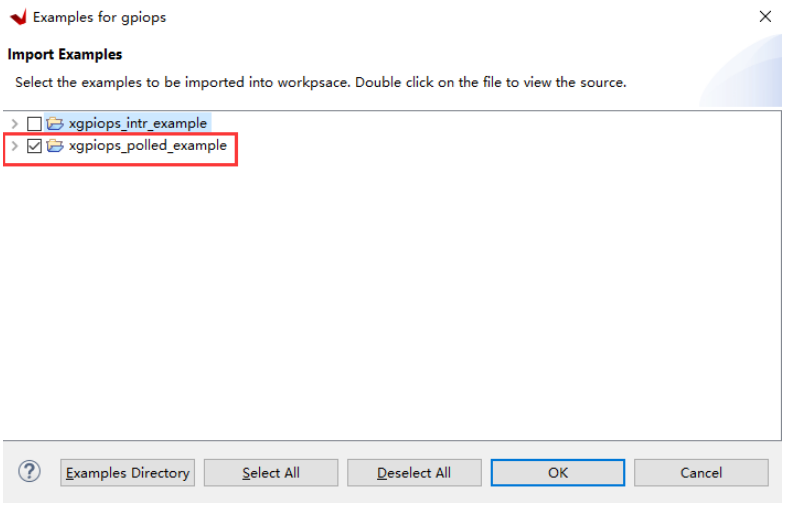


Figure 27-2: **GPIO Channel**

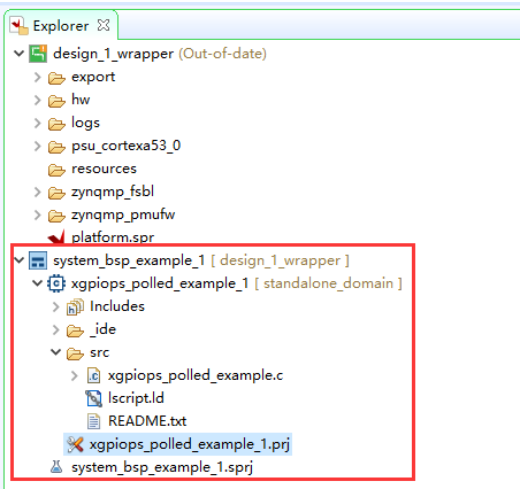
3) 一开始编写代码可能会无从下手，我们可以导入 Xilinx 提供的 example 工程，在 BSP 里找到 psu_gpio_0，点击 Import Examples



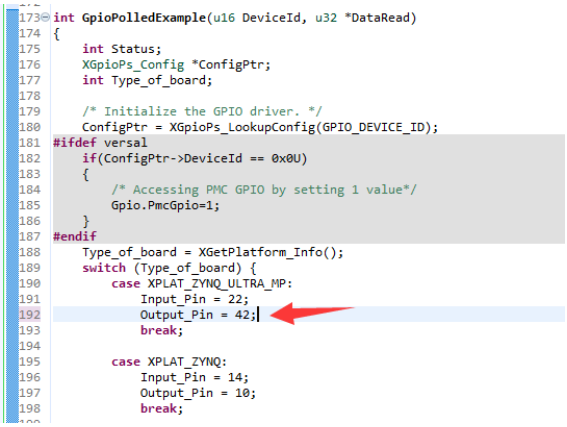
在弹出窗口选择“xgpiops_polled_example”，点击 OK



会出现一个新的 APP 工程



1. 这个 example 工程是测试 PS 端 MIO 的输入输出的，由于开发板 PS 端的 LED 是 MIO42，需要在文件中修改 Output_pin 为 42，测试 MIO42 的 LED 灯。



```
int GpioPolledExample(u16 DeviceId, u32 *DataRead)
{
    int Status;
    XGpioPs_Config *ConfigPtr;
    int Type_of_board;

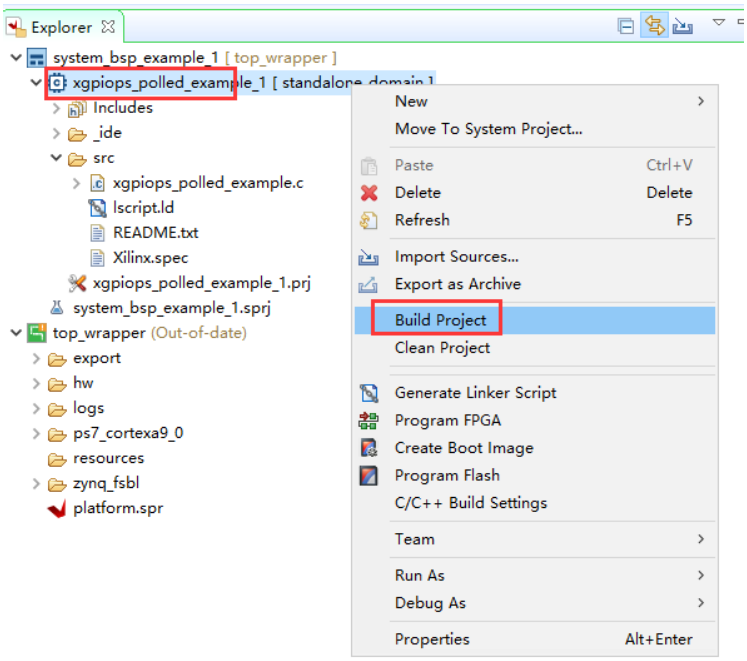
    /* Initialize the GPIO driver. */
    ConfigPtr = XGpioPs_LookupConfig(GPIO_DEVICE_ID);
#ifdef versal
    if(ConfigPtr->DeviceId == 0x0U)
    {
        /* Accessing PMC GPIO by setting 1 value*/
        Gpio.PmcGpio=1;
    }
#endif
    Type_of_board = XGetPlatform_Info();
    switch (Type_of_board) {
        case XPLAT_ZYNQ_ULTRA_MP:
            Input_Pin = 22;
            Output_Pin = 42;
            break;

        case XPLAT_ZYNQ:
            Input_Pin = 14;
            Output_Pin = 10;
            break;
```

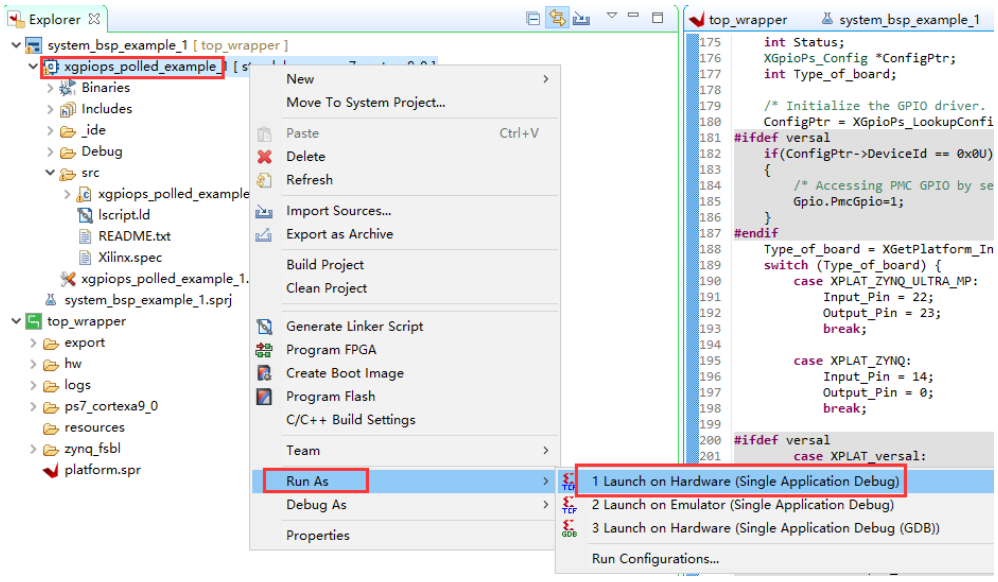
由于只测试 LED 灯，也就是输出，我们把输入功能注释掉。保存文件。

```
214
215     Status = XGpioPs_CfgInitialize(&Gpio, ConfigPtr,
216                     ConfigPtr->BaseAddr);
217     if (Status != XST_SUCCESS) {
218         return XST_FAILURE;
219     }
220     /* Run the Output Example. */
221     Status = GpioOutputExample();
222     if (Status != XST_SUCCESS) {
223         return XST_FAILURE;
224     }
225
226 //  /* Run the Input Example. */
227 //  Status = GpioInputExample(DataRead);
228 //  if (Status != XST_SUCCESS) {
229 //      return XST_FAILURE;
230 //  }
231
232     return XST_SUCCESS;
233 }
```

2. 编译工程

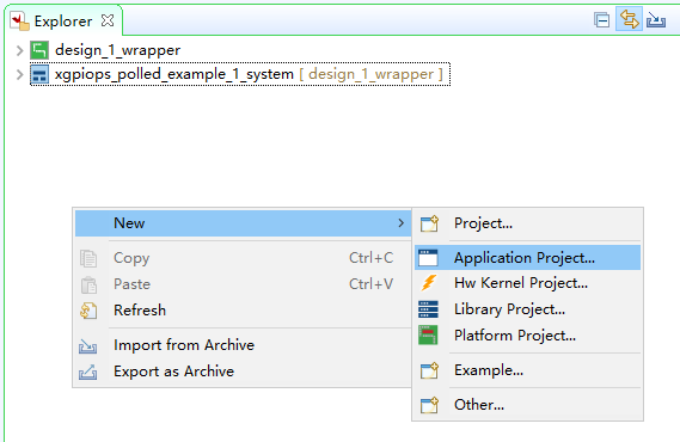


3. Run As→Launch on Hardware(Single Application Debug)，下载结束后，即可看到 PS_LED 快速闪烁 16 次

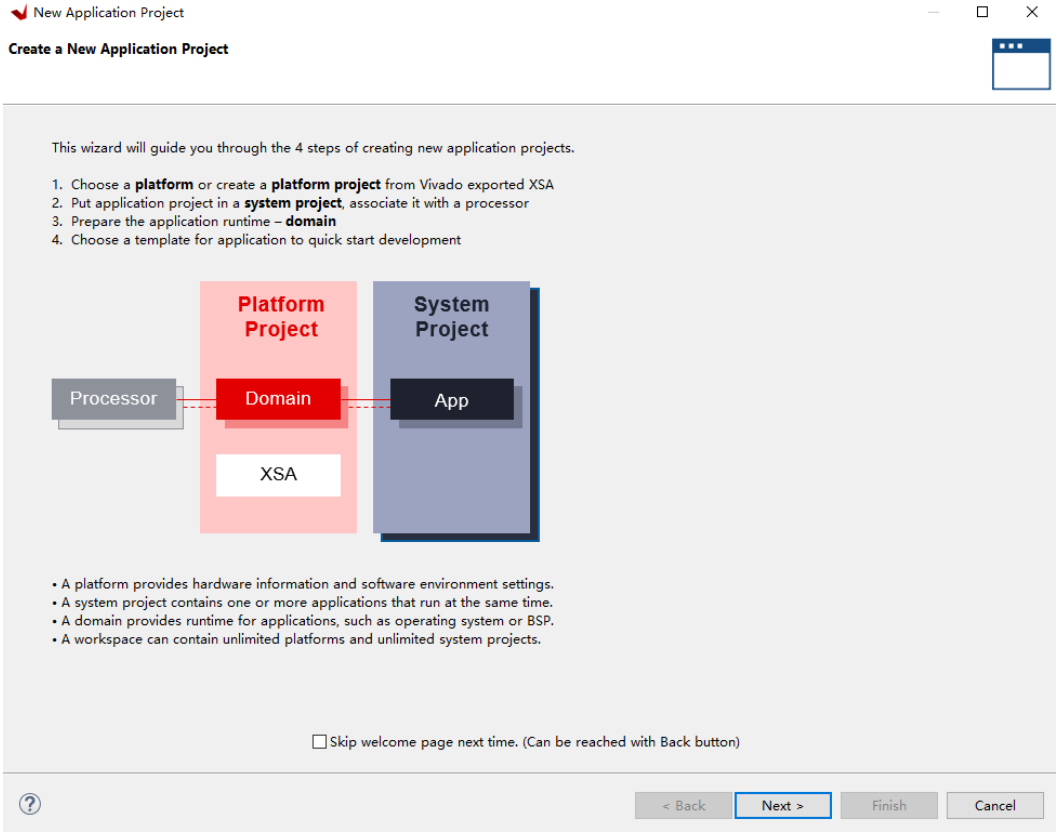


4. 虽然用官方的例子比较方便，但是它的代码看起来比较臃肿，我们可以通过学习它的

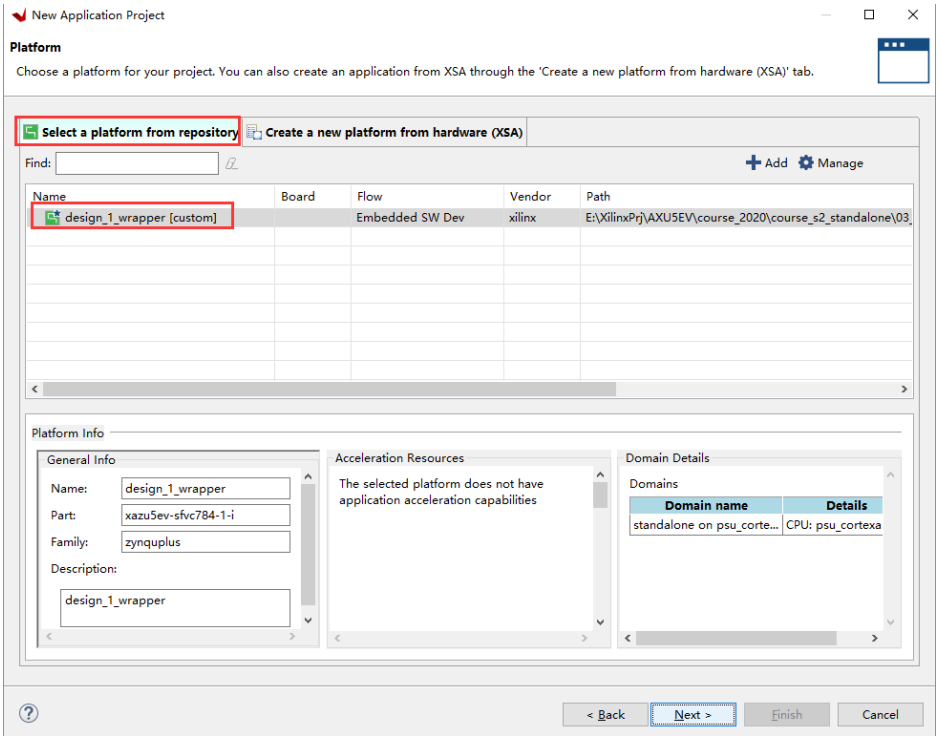
方法，自己简化写一遍。我们新建一个 APP 工程。可以在空白处右键 New→Application Project。在 ps_led_test 的 helloworld.c 中修改。其实程序步骤很简单，初始化 GPIO→设置方向→输出使能→控制 GPIO 输出值。



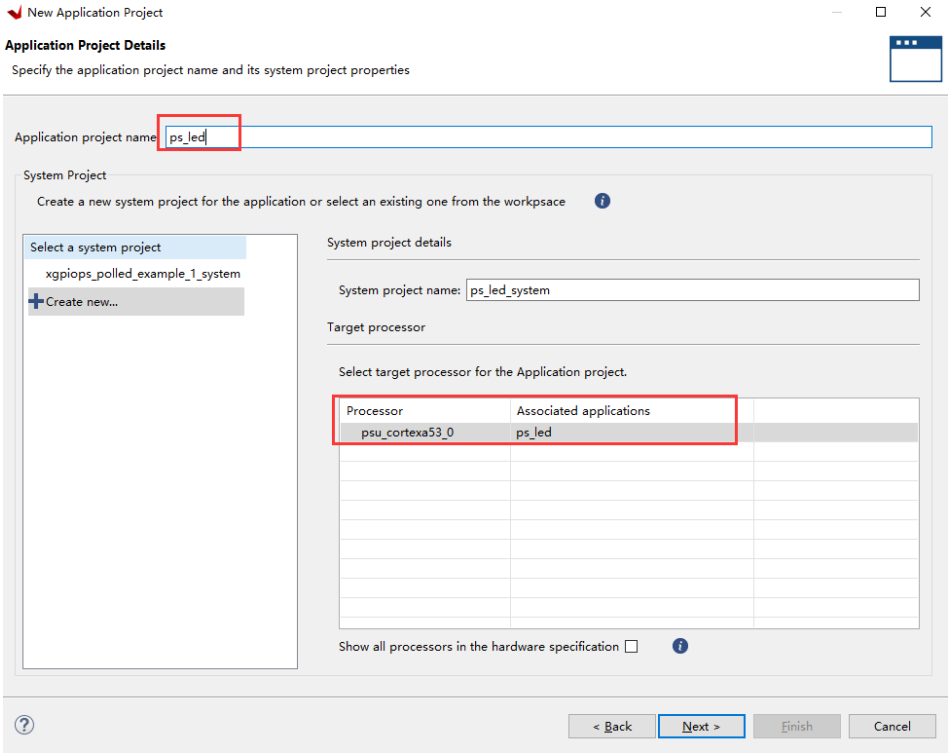
第一页跳过



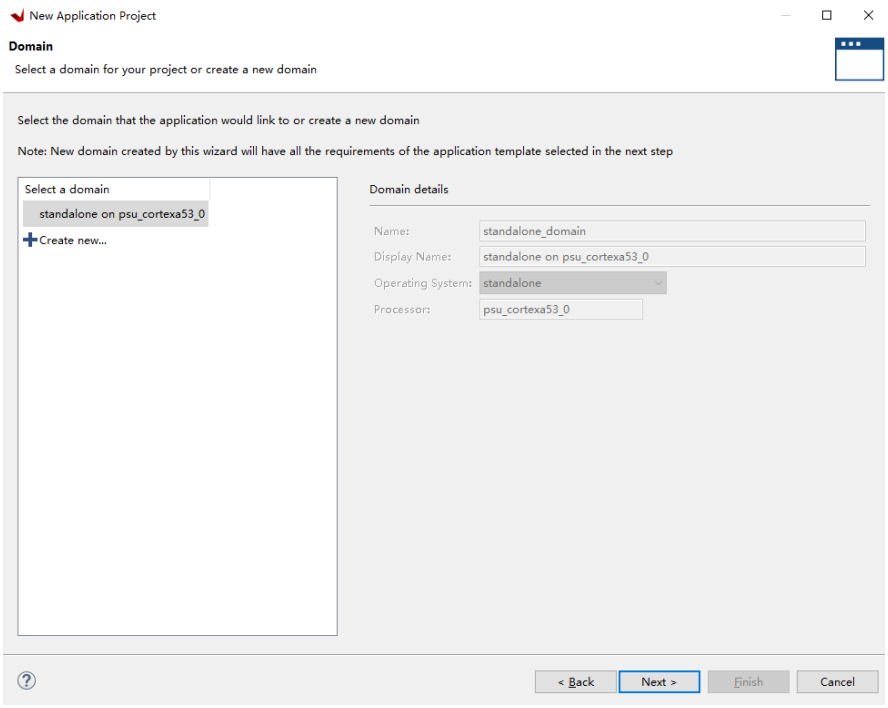
选择硬件描述工程



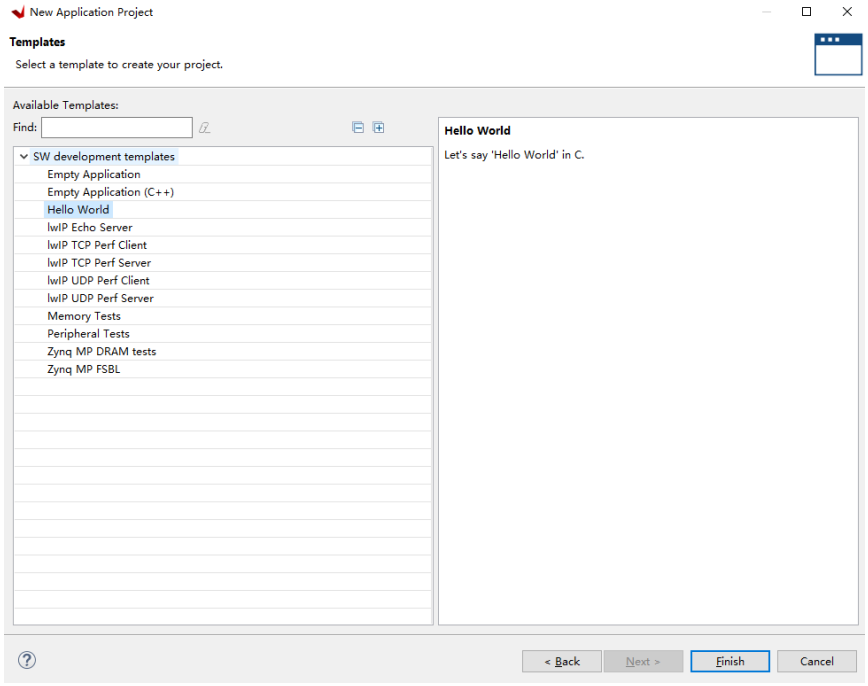
填入工程名称，并选择相应的 CPU



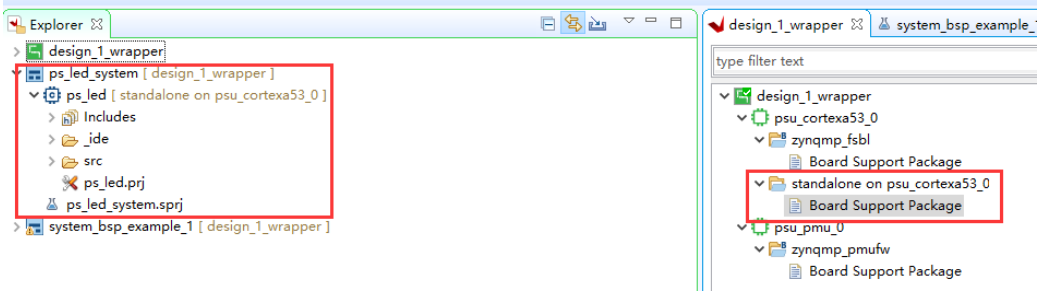
下一步



模板选择 Hellow World 即可



5. 可以看到多了一个 APP 工程，仍然是基于名为 standalone on psu_cortexa53_0 的 BSP，也就是一个 Domain，与前面的 example 工程共用一个 BSP



6. 可以将例程的代码复制到 helloworld.c 中，保存并 Build Project

```
int main()
{
    init_platform();

    int Status;
    XGpioPs_Config *ConfigPtr;

    print("Hello World\n\r");
    /* Initialize the GPIO driver. */
    ConfigPtr = XGpioPs_LookupConfig(GPIO_DEVICE_ID);

    Status = XGpioPs_CfgInitialize(&Gpio, ConfigPtr,
            ConfigPtr->BaseAddr);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }

    /*
     * Set the direction for the pin to be output and
     * Enable the Output enable for the LED Pin.
     */
    XGpioPs_SetDirectionPin(&Gpio, 42, 1);
    XGpioPs_SetOutputEnablePin(&Gpio, 42, 1);

    while(1){
        /* Set the GPIO output to be low. */
        XGpioPs_WritePin(&Gpio, 42, 0x0);
        sleep(1) ;
        /* Set the GPIO output to be high. */
        XGpioPs_WritePin(&Gpio, 42, 0x1);
        sleep(1) ;
    }

    cleanup_platform();
    return 0;
}
```

初始化 GPIO

设置方向为输出，并使能输出

控制输出值

7. 下载方法与前面一样，就可以看到 PS 端的 LED 灯开始闪烁。

7.3.2 MIO 按键中断

前面介绍了 MIO 作为输出控制 LED 灯，这里讲一下利用 MIO 作为按键输入控制 LED 灯。

1. 通过 ug1085 文档看下 GPIO 的结构图，中断的寄存器：

INT_MASK：中断掩码

INT_DIS：中断关闭

INT_EN：中断使能

INT_TYPE：中断类型，设置电平敏感还是边沿敏感

INT_POLARITY：中断极性，设置低电平或下降沿还是高电平或上升沿

INT_ANY：边沿触发方式，需要 INT_TYPE 设置为边沿敏感才能使用

设置中断产生方式时需要 INT_TYPE、INT_POLARITY、INT_ANY 配合使用。具体寄存器含义请参考 ug1085 部分。

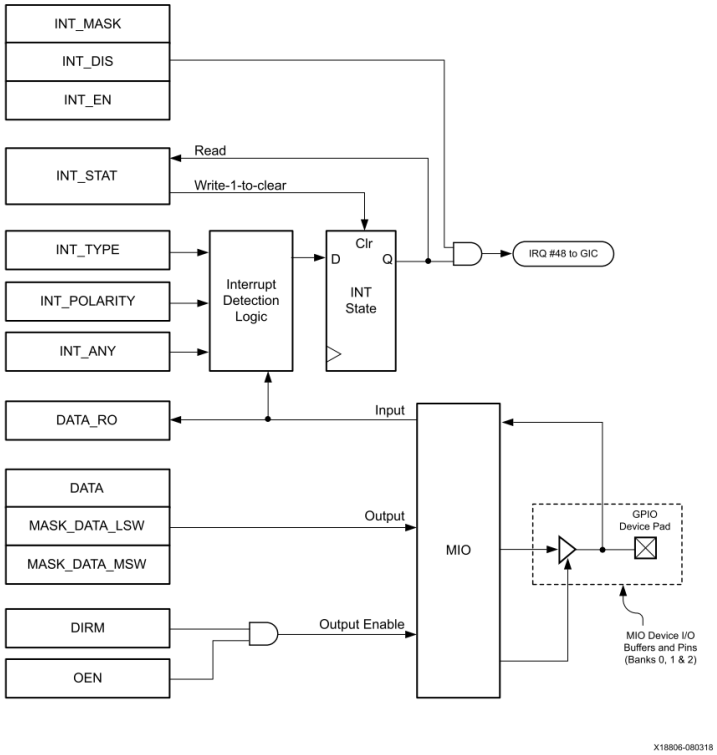
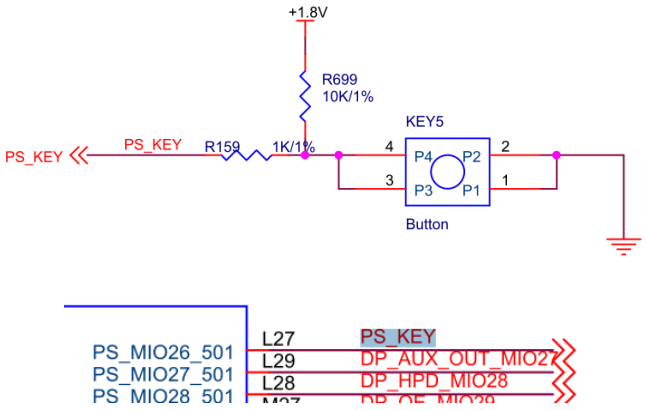


Figure 27-2: **GPIO Channel**

在原理图中可以看出 PS 端的按键接在了 MIO26，按键按下为低电平，不按则为高电平。



2. 本实验设计为按下按键 LED 灯亮，再按下 LED 灭。

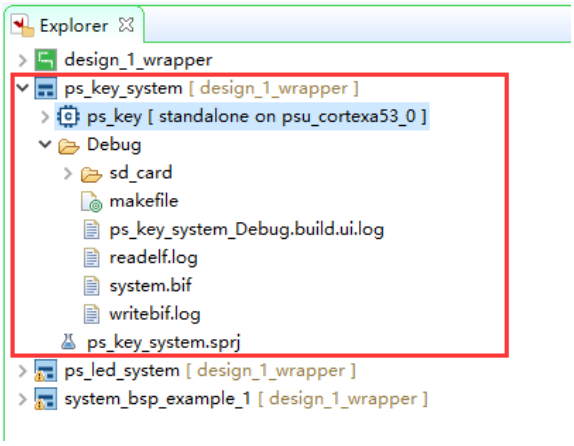
主程序设计流程如下：

GPIO 初始化→设置按键和 LED 方向→设置产生中断方式→设置中断→打开中断控制器→打开中断异常→打开 GPIO 中断→判断 KEY_FLAG 值，是 1，写 LED

中断处理流程：

查询中断状态寄存器→判断状态→清除中断→设置 KEY_FLAG 值

3. 新建 Vitis 工程



4. 定义 PS 按键编号为 26，PS LED 为 42

```
/* GPIO paramter */
#define MIO_ID          XPAR_XGPIOPS_0_DEVICE_ID
#define INTC_DEVICE_ID  XPAR_SCUGIC_SINGLE_DEVICE_ID
#define KEY_INTR_ID     XPAR_XGPIOPS_0_INTR
#define PS_KEY_MIO      26
#define PS_LED_MIO      42

#define GPIO_INPUT      0
#define GPIO_OUTPUT     1
```

5. 在 main 函数中，设置 LED 和按键，将按键中断类型设置为上升沿产生中断。在本实验中，即按键信号的上升沿产生中断。

```
/*
 * Set the direction for the pin to be output and
 * Enable the Output enable for the LED Pin.
 */
XGpioPs_SetDirectionPin(&GPIO_PTR, PS_LED_MIO, GPIO_OUTPUT) ;
XGpioPs_SetOutputEnablePin(&GPIO_PTR, PS_LED_MIO, GPIO_OUTPUT) ;
/*
 * Set the direction for the pin to be input.
 * Set interrupt type as rising edge and enable gpio interrupt
 */
XGpioPs_SetDirectionPin(&GPIO_PTR, PS_KEY_MIO, GPIO_INPUT) ;
XGpioPs_SetIntrTypePin(&GPIO_PTR, PS_KEY_MIO, XGPIOPS_IRQ_TYPE_EDGE_RISING) ;
XGpioPs_IntrEnablePin(&GPIO_PTR, PS_KEY_MIO) ;
```

6. 中断控制器设置函数 IntrInitFuntions 是参考 PS 定时器中断实验所做，而下面的语句是设置中断优先级和触发方式。即操作 ICDIPR 和 ICDICFR 寄存器。

```
XScuGic_SetPriorityTriggerType(InstancePtr, KEY_INTR_ID,
                               0xA0, 0x3);
```

7. 在中断服务程序 GpioHandler 中，判断中断状态寄存器，清除中断，并将按键标志置 1。

```
void GpioHandler(void *CallbackRef)
{
    XGpioPs *GpioInstancePtr = (XGpioPs *)CallbackRef ;
    int Int_val ;

    Int_val = XGpioPs_IntrGetStatusPin(GpioInstancePtr, PS_KEY_MIO) ;
    /*
     * Clear interrupt.
     */
    XGpioPs_IntrClearPin(GpioInstancePtr, PS_KEY_MIO) ;
    if (Int_val)
        key_flag = 1 ;
}
```

8. 在 main 函数中，判断按键标志 key_flag，向 LED 写入数据。

```
while(1)
{
    if (key_flag)
    {
        XGpioPs_WritePin(&GPIO_PTR, 9, key_val) ;
        key_val = ~key_val ;
        key_flag = 0 ;
    }

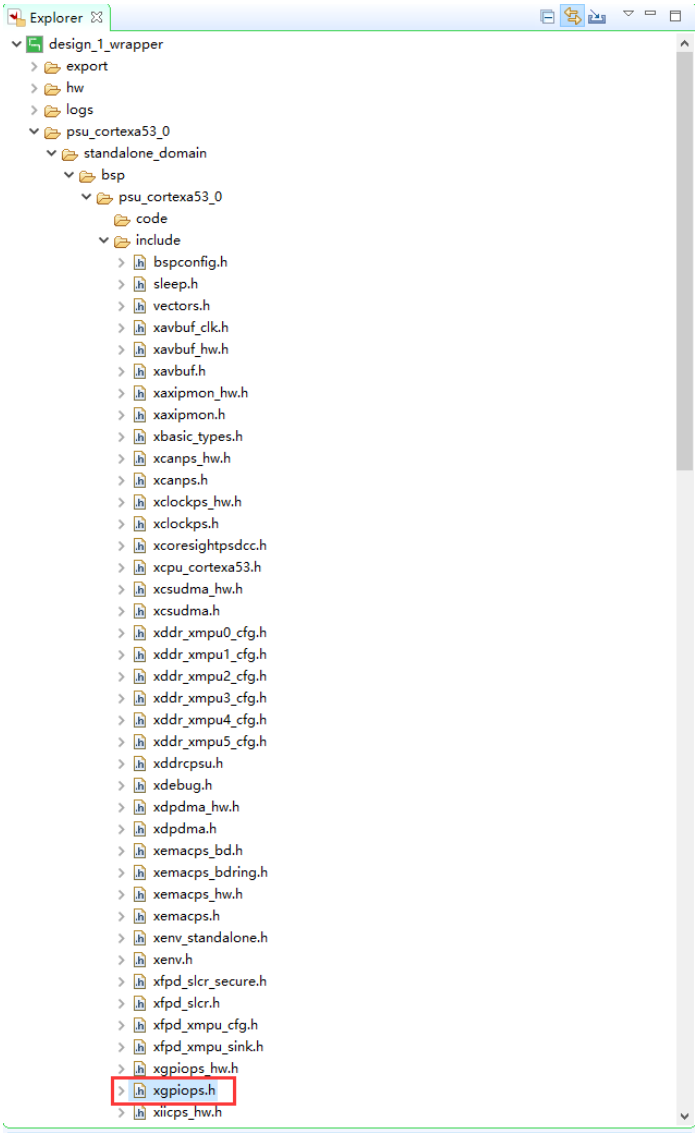
}
```

9. 编译工程并下载程序

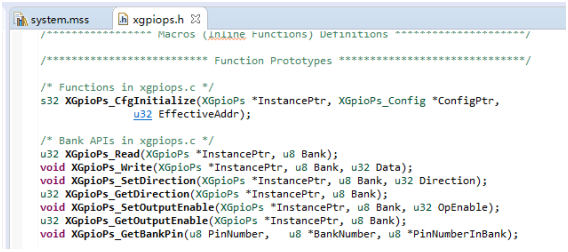
10. 观察实验现象，按下按键 PS_KEY，就可以控制 PS_LED 的亮灭。

7.4 知识点分享

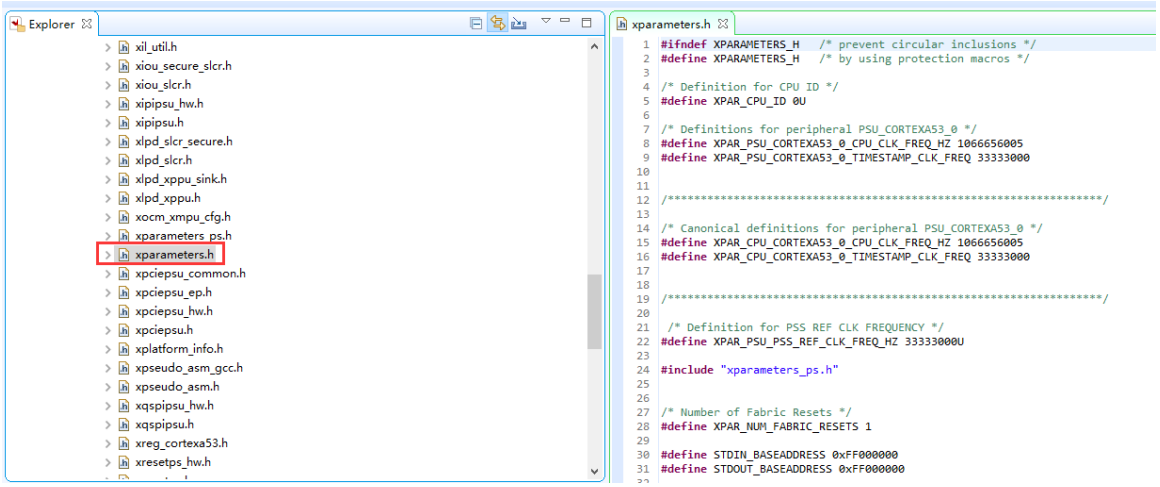
1. 在 platform 中 bsp 的 include 文件夹下包含了 xilinx 的各种头文件，如本章用到的 GPIO，用到了 xgpiops.h，在此文件中可以看到各种宏定义，在调用 GPIO 函数时可以使用这些宏定义，提高可读性。



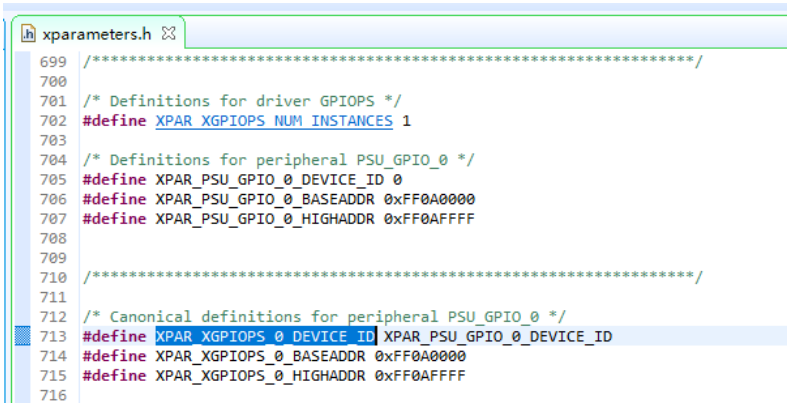
同时也包含外设自带的函数声明



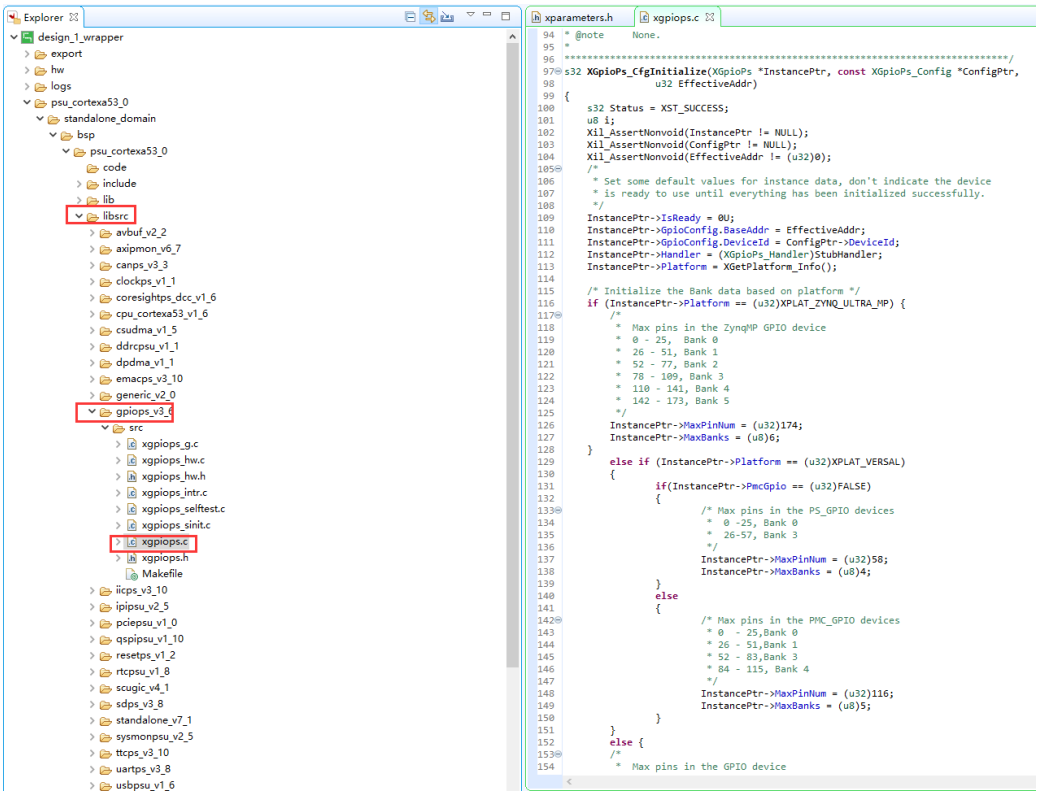
2. 在 xparameters.h 头文件中定义了各个外设的基地址，器件 ID，中断等



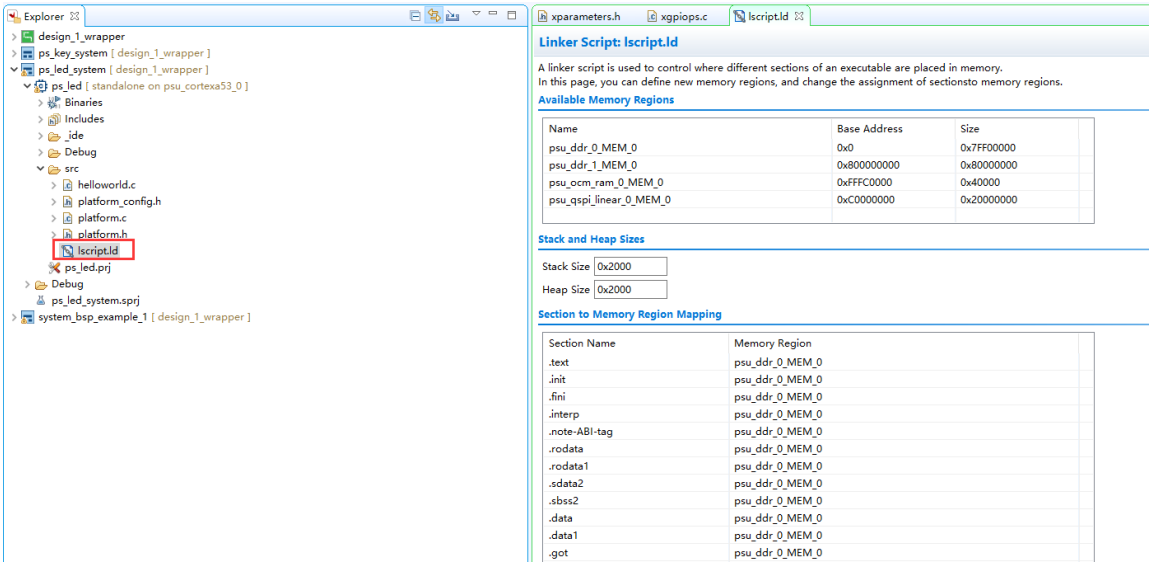
比如程序中的 DEVICE_ID 宏定义就是在这个文件里找到的。



3. 在 libsrc 文件夹中，包含外设函数的定义，使用说明

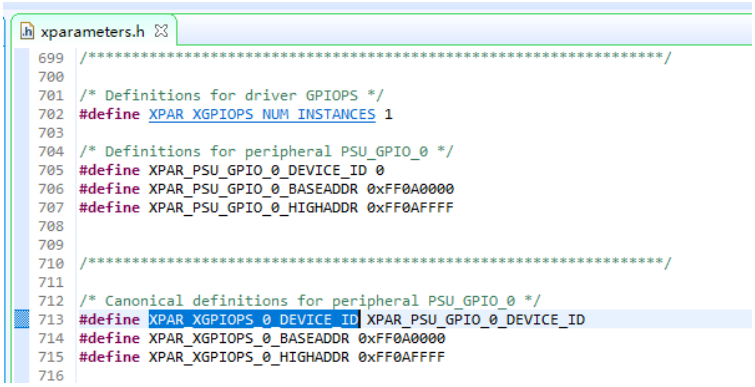


4. 在 src 文件夹下的 lscript.ld 中，定义了可用 memory 空间，栈和堆空间大小等，可根据需要修改。



5. 把鼠标光标放到宏定义或函数上，按下 F3 即可看到在哪里定义的，也可以按 Ctrl+鼠标左键进入。比如下面的 DEVICE_ID 即可进入 xparameter.h 中

```
48 #include <stdio.h>
49 #include "platform.h"
50 #include "xil_printf.h"
51 #include "xgpiops.h"
52 #include "sleep.h"
53
54 #define GPIO_DEVICE_ID      XPAR_XGPIOPS_0_DEVICE_ID
55
```



7.5 本章小结

本章介绍了 MIO 的输入输出控制，以及 GPIO 的使用，相信大家也有了一定的认识。在学习过程中，一定要多看文档，结合模块结构以及寄存器含义加深理解。参考文档 ug1085。

第八章 PS 端 UART 读写控制

vivado 工程目录为“ps_hello/vivado”

vitis 工程目录为“ps_uart/vitis”

软件工程师工作内容

以下为软件工程师负责内容。

在前面的实验中，大家或多或少会发现有打印信息的情况，主要是调用”xil_printf”或”printf”，但是通过什么打印信息呢？我们还记得打印信息之前设置了串口，是的，确实是串口，但这些函数是如何调用串口呢？其实我们可以在”xil_printf”函数定义中看到，注意 outbyte 函数就是调用 UART 打印的。

```
void xil_printf( const char8 *ctrl1, ...)
{
    s32 Check;
#if defined (__aarch64__) || defined (__arch64__)
    s32 long_flag;
#endif
    s32 dot_flag;

    params_t par;

    char8 ch;
    va_list argp;
    char8 *ctrl = (char8 *)ctrl1;

    va_start( argp, ctrl1);

    while ((ctrl != NULL) && (*ctrl != (char8)0)) {

        /* move format string chars to buffer until a  */
        /* format control is found.                    */
        if (*ctrl != '%') {
#ifdef STDOUT_BASEADDRESS
            outbyte(*ctrl);
#endif
            ctrl += 1;
            continue;
        }
```

再进入 outbye 的函数，即可看到调用了 PS 端 UART 的函数，得以在串口中显示。

```
void outbyte(char c) {
    XUartPs_SendByte(STDOUT_BASEADDRESS, c);
}
```

除了打印信息之外，如果我们想用 UART 进行数据传输呢？本章便来介绍 PS 端 UART 的读写控制，实验中，每隔 1S 向外发送一串字符，如果收到数据，产生中断，并将收到的数据再发送出去。

8.1 UART 模块介绍

以下是 UART 模块的结构图，TxFIFO 和 RxFIFO 都为 64 字节。

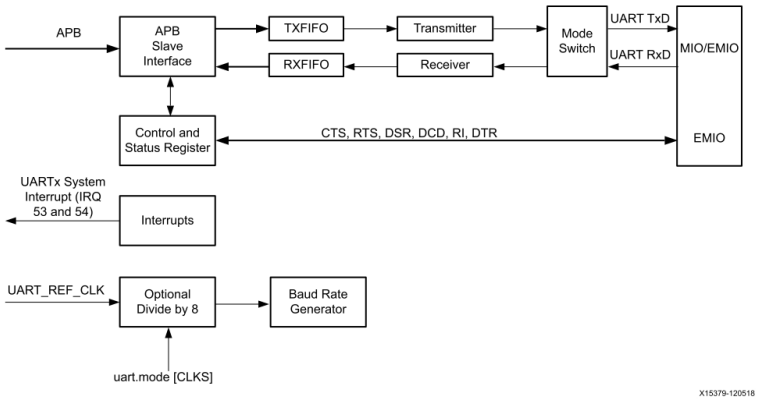


Figure 21-1: **UART Controller**

下图为 UART 的四种模式

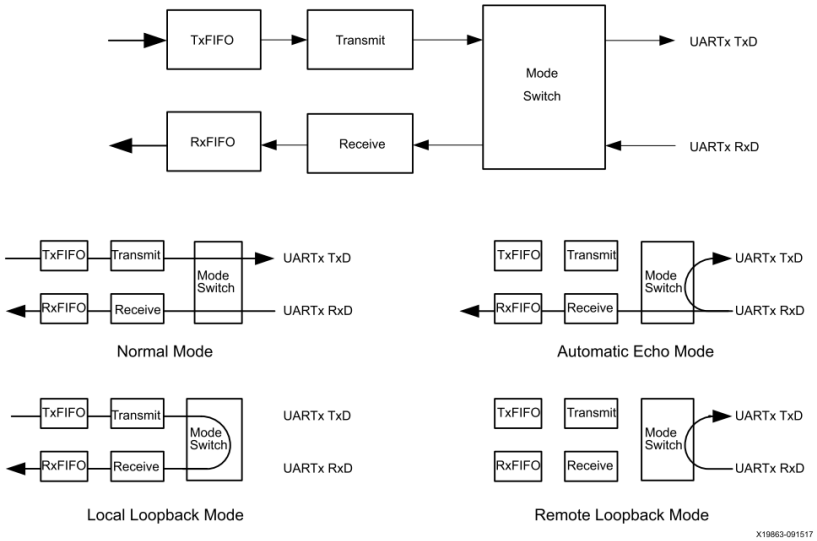


Figure 21-6: **UART Mode Switch for TxD and RxD**

可以用 remote loopback mode 测试物理电路是否正常，使用 API 函数 XUartPs_SetOperMode

```
XUartPs_SetOperMode(&Uart_PS, XUARTPS_OPER_MODE_REMOTE_LOOP);
```

8.2 Vitis 程序开发

1. 本实验流程如下：

主程序流程：

UART 初始化→设置 UART 模式→设置数据格式→设置中断→发送 UART 数据→检查是否收到数据→如果收到，发送收到的数据，如果没有，等待 1 秒钟，继续发数据

中断程序流程：

中断初始化→设置接收 FIFO trigger 中断寄存器，设置为 1，即收到一个数据就中断→打开接收 trigger 中断 REMPTY 及接收 FIFO 空中断 RTRIG

中断服务程序：

判断状态寄存器是 trigger 还是 empty→清除相应中断→trigger 状态读取 RxFIFO 数据，empty 状态将接收标志 ReceivedFlag 置 1

2. 在 main 函数中进行模式的设置，可以直接调用函数，设置为正常模式，数据格式设置为波特率 115200，数据 8bit，无校验位，1bit 停止位。UartFormat 定义在 uart_parameter.h 中。

```
/* Use Normal mode. */
XUartPs_SetOperMode(&Uart_PS, XUARTPS_OPER_MODE_NORMAL);
/* Set uart mode Baud Rate 115200, 8bits, no parity, 1 stop bit */
XUartPs_SetDataFormat(&Uart_PS, &UartFormat) ;
```

```
XUartPsFormat UartFormat =
{
        115200,
        XUARTPS_FORMAT_8_BITS,
        XUARTPS_FORMAT_NO_PARITY,
        XUARTPS_FORMAT_1_STOP_BIT
};
```

3. 中断控制器程序初始化可参考按键中断方式，用法类似。
4. 在 main 函数中将 trigger level 设置为 1，打开 trigger 和 empty 中断。

```
/*Set receiver FIFO interrupt trigger level, here set to 1*/
XUartPs_SetFifoThreshold(UartInstancePtr,1) ;
/* Enable the receive FIFO trigger level interrupt and empty interrupt for the device */
XUartPs_SetInterruptMask(UartInstancePtr,XUARTPS_IXR_RXOVR|XUARTPS_IXR_RXEMPTY);
```

5. 数据的发送和接收函数参考了 UARTPS 的 XUartPs_Send 和 XUartPs_Rev 函数，但它们会打开某些中断，不符合预期，因此做了修改。

```
int UartPsSend(XUartPs *InstancePtr, u8 *BufferPtr, u32 NumBytes) ;
int UartPsRev (XUartPs *InstancePtr, u8 *BufferPtr, u32 NumBytes) ;
```

在接收缓存中设置了最大 2000 字节的缓冲，可以根据需要修改。

```
#define MAX_LEN    2000
```

```
/* UART receiver buffer */
u8 ReceivedBuffer[MAX_LEN] ;
```

6. 在中断服务程序中，将 ReceivedBufferPtr 指针地址和 ReceivedByteNum 加上接收到的个数，如果 FIFO 空了，将 ReceivedFlag 置为 1。同时向中断状态寄存器写数据，清除中断。

```
void Handler(void *CallBackRef)
{
    XUartPs *UartInstancePtr = (XUartPs *) CallBackRef ;
    u32 ReceivedCount = 0 ;
    u32 UartSrValue ;

    UartSrValue = XUartPs_ReadReg(UartInstancePtr->Config.BaseAddress, XUARTPS_SR_OFFSET) & (XUARTPS_IXR_RXOVR|XUARTPS_IXR_RXEMPTY);
    ReceivedFlag = 0 ;

    if (UartSrValue & XUARTPS_IXR_RXOVR)   /* check if receiver FIFO trigger */
    {
        ReceivedCount = UartPsRev(&Uart_PS, ReceivedBufferPtr, MAX_LEN) ;
        ReceivedByteNum += ReceivedCount ;
        ReceivedBufferPtr += ReceivedCount ;
        /* clear trigger interrupt */
        XUartPs_WriteReg(UartInstancePtr->Config.BaseAddress, XUARTPS_ISR_OFFSET, XUARTPS_IXR_RXOVR) ;
    }
    else if (UartSrValue & XUARTPS_IXR_RXEMPTY)       /*check if receiver FIFO empty */
    {
        /* clear empty interrupt */
        XUartPs_WriteReg(UartInstancePtr->Config.BaseAddress, XUARTPS_ISR_OFFSET, XUARTPS_IXR_RXEMPTY) ;
        ReceivedFlag = 1 ;
    }

}
```

Interrupt and Status Registers

There are two status registers that can be read by software. Both show raw status. The Chnl_int_sts register can be read for status and generate an interrupt. The Channel_sts register can only be read for status.

The Chnl_int_sts register is sticky; once a bit is set, the bit stays set until software clears it. Write a 1 to clear a bit. This register is bit-wise AND'ed with the Intrpt_mask mask register. If any of the bit-wise AND functions have a result = 1, then the UART interrupt is asserted to the PS interrupt controller.

Ug1085 UART 清除中断

7. 在 main 函数中，将 ReceivedFlag 和 ReceivedByteNum 清零，ReceivedBufferPtr 指针复位。

```
case UART_RXCHECK :    /* Check receiver flag, send received data */
{
    if (ReceivedFlag)
    {
        /* Reset receiver pointer, flag, byte number */
        ReceivedBufferPtr = ReceivedBuffer ;
        SendBufferPtr = ReceivedBuffer ;
        SendByteNum = ReceivedByteNum ;
        ReceivedFlag = 0 ;
        ReceivedByteNum = 0 ;
        UartPsSend(&Uart_PS, SendBufferPtr, SendByteNum);
    }
```

8. Uart 发送函数中，判断 TxFIFO 是否满，否则继续发送，直到计数达到 NumBytes

```
int UartPsSend(XUartPs *InstancePtr, u8 *BufferPtr, u32 NumBytes)
{

    u32 SentCount = 0U;

    /* Setup the buffer parameters */
    InstancePtr->SendBuffer.RequestedBytes = NumBytes;
    InstancePtr->SendBuffer.RemainingBytes = NumBytes;
    InstancePtr->SendBuffer.NextBytePtr = BufferPtr;

    while (InstancePtr->SendBuffer.RemainingBytes > SentCount)
    {
        /* Fill the FIFO from the buffer */
        if (!XUartPs_IsTransmitFull(InstancePtr->Config.BaseAddress))
        {
            XUartPs_WriteReg(InstancePtr->Config.BaseAddress,
                    XUARTPS_FIFO_OFFSET,
                    ((u32)InstancePtr->SendBuffer.
                            NextBytePtr[SentCount]));

            /* Increment the send count. */
            SentCount++;
        }
    }

    /* Update the buffer to reflect the bytes that were sent from it */
    InstancePtr->SendBuffer.NextBytePtr += SentCount;
    InstancePtr->SendBuffer.RemainingBytes -= SentCount;


    return SentCount;
}
```

9. Uart 接收函数中，判断接收 RxFIFO 是否为空，否则继续读数据，NumBytes 为需要读取的数据个数，但如果接收的 FIFO 空了，计数没有达到这个值，也会结束此函数。

```
int UartPsRev(XUartPs *InstancePtr, u8 *BufferPtr, u32 NumBytes)
{
    u32 ReceivedCount = 0;
    u32 CsrRegister;

    /* Setup the buffer parameters */
    InstancePtr->ReceiveBuffer.RequestedBytes = NumBytes;
    InstancePtr->ReceiveBuffer.RemainingBytes = NumBytes;
    InstancePtr->ReceiveBuffer.NextBytePtr = BufferPtr;

    /*
     * Read the Channel Status Register to determine if there is any data in
     * the RX FIFO
     */
    CsrRegister = XUartPs_ReadReg(InstancePtr->Config.BaseAddress,
            XUARTPS_SR_OFFSET);

    /*
     * Loop until there is no more data in RX FIFO or the specified
     * number of bytes has been received
     */
    while((ReceivedCount < InstancePtr->ReceiveBuffer.RemainingBytes)&&
            (((CsrRegister & XUARTPS_SR_RXEMPTY) == (u32)0)))
    {
        InstancePtr->ReceiveBuffer.NextBytePtr[ReceivedCount] =
                XUartPs_ReadReg(InstancePtr->Config.BaseAddress,XUARTPS_FIFO_OFFSET);

        ReceivedCount++;

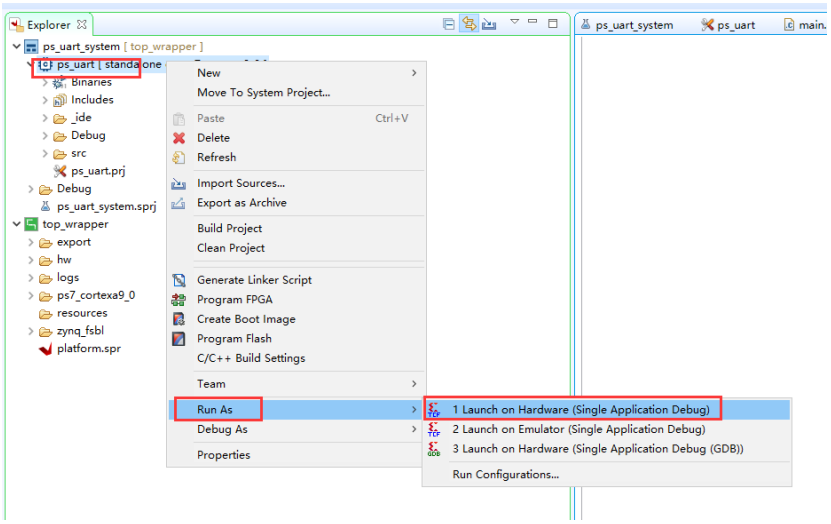
        CsrRegister = XUartPs_ReadReg(InstancePtr->Config.BaseAddress,
                XUARTPS_SR_OFFSET);
    }
```

10. 除了自己写程序外，还可以从 system.mss 中导入模块的例子，参考 Xilinx 提供的程序，方便学习。

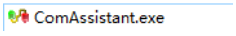
psu_sd_1	sdps	Documentation Link	Import Examples
psu_serdes	generic	-	-
psu_siou	generic	-	-
psu_smmu_gpv	generic	-	-
psu_smmu_reg	generic	-	-
psu_ttc_0	ttcps	Documentation Link	Import Examples
psu_ttc_1	ttcps	Documentation Link	Import Examples
psu_ttc_2	ttcps	Documentation Link	Import Examples
psu_ttc_3	ttcps	Documentation Link	Import Examples
psu_uart_0	uartps	Documentation Link	Import Examples
psu_usb_0	generic	-	-
psu_usb_xhci_0	usbpsu	Documentation Link	Import Examples

8.3 板上验证

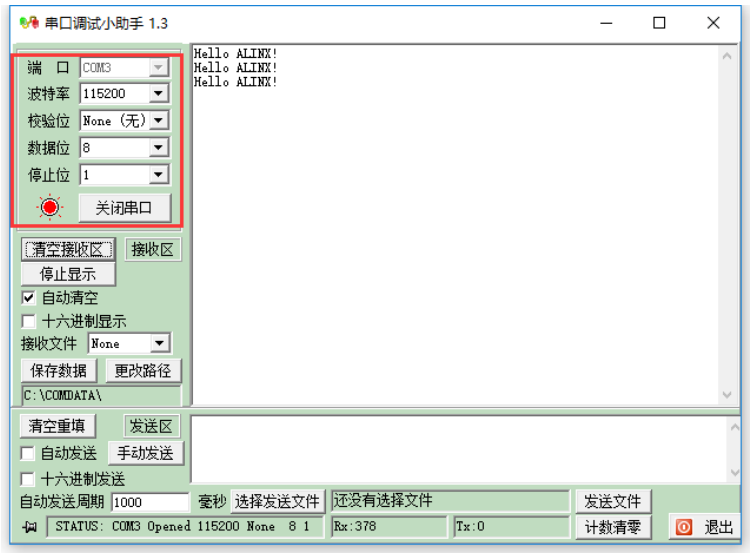
1. 接下来下载程序



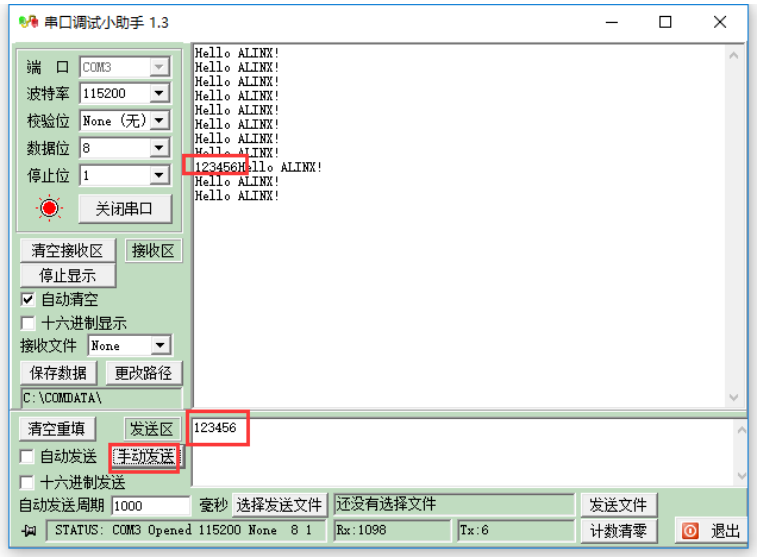
2. 打开工程目录下的串口调试工具



3. 设置好参数如下，打开串口，即可看到打印信息。



4. 在发送区填入数据，点击手动发送，即可看到接收区的数据。



8.4 总结

本章学习了 UART 的发送与接收，以及中断的使用，希望大家能养成良好的习惯，多看文档，理解原理，能对系统的认识有很大提高。

第九章 PS 端 CAN 的使用

vivado 工程目录为“ps_hello/vivado”
vitis 工程目录为“ps_can/vitis”

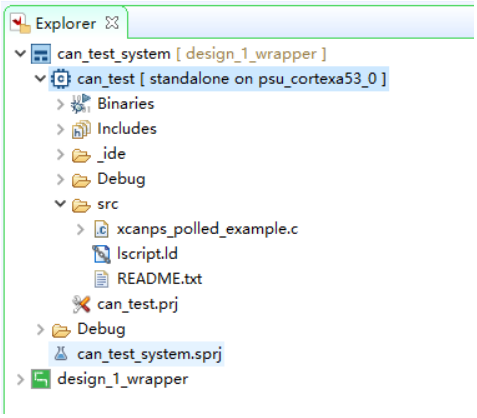
本章介绍利用板子上了两路 CAN 接口进行回环测试。

软件工程师工作内容

以下为软件工程师负责内容。

9.1 Vitis 程序开发

在 VITIS 软件中新建工程 can_test，本实验是在此 example 基础上进行的修改，将两个 CAN 接口都设置为 Normal Mode，并在外部连接起来，进行 loopback 测试。



1. 功能其实也比较简单，主要就是两路 CAN 的初始化，并设置为 Normal Mode，然后是 loopback。

```
int main()
{
    int Status;

    xil_printf("CAN Polled Mode Example Test \r\n");

    /*
     * Run the Can Polled example, specify the Device ID that is generated
     * in xparameters.h .
     */
    Status = CanInitial(XPAR_XCANPS_0_DEVICE_ID, &Can0);
    if (Status != XST_SUCCESS) {
        xil_printf("CAN Initial Failed\r\n");
        return XST_FAILURE;
    }

    Status = CanInitial(XPAR_XCANPS_1_DEVICE_ID, &Can1);
    if (Status != XST_SUCCESS) {
        xil_printf("CAN Initial Failed\r\n");
        return XST_FAILURE;
    }

    Status = CanLoopback(&Can0, &Can1);
    if (Status != XST_SUCCESS) {
        xil_printf("CAN Loopback Failed\r\n");
        return XST_FAILURE;
    }

    xil_printf("Successfully ran CAN Polled Mode Example Test\r\n");
    return XST_SUCCESS;
}
```

初始化CAN0

初始化CAN1

CAN0和CAN1 loopback

2. CanLoopback 函数也比较简单，先是 CAN0 发数据，CAN1 接收数据并进行数据对比，然后清除 RxBuffer 的数据，再是 CAN1 发送数据，CAN0 接收数据并进行数据对比，测试结束。

```
int CanLoopback(XCanPs *CanInstPtr0, XCanPs *CanInstPtr1)
{
    int Status;
    /*
     * Send a frame, receive the frame via the loop back and verify its
     * contents.
     */
    Status = SendFrame(CanInstPtr0);
    if (Status != XST_SUCCESS) {
        return Status;
    }

    Status = RecvFrame(CanInstPtr1);

    /*
     * Clear RxFrame buffer
     */
    memset(RxFrame, 0, XCANPS_MAX_FRAME_SIZE) ;

    /*
     * Send a frame, receive the frame via the loop back and verify its
     * contents.
     */
    Status = SendFrame(CanInstPtr1);
    if (Status != XST_SUCCESS) {
        return Status;
    }

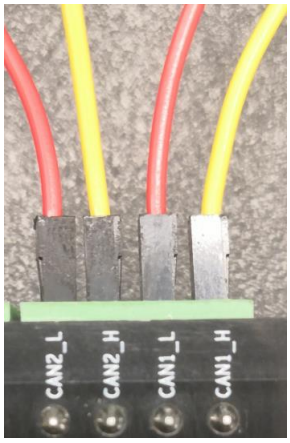
    Status = RecvFrame(CanInstPtr0);

    return Status;

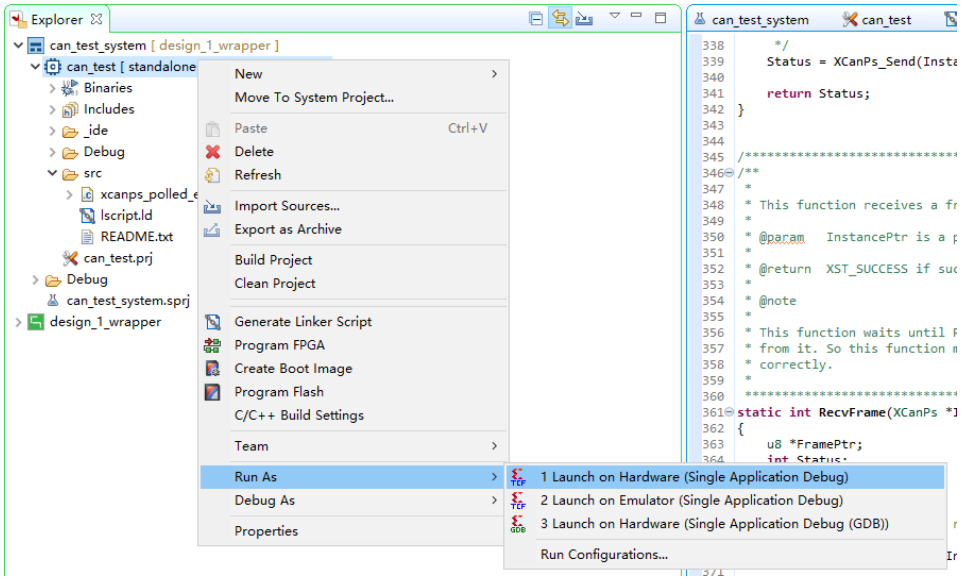
}
```

9.2 下载测试

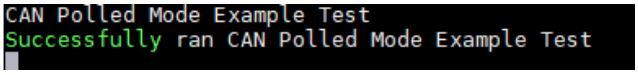
1. 利用杜绑线将 CAN1 的 H 和 L 分别与 CAN2 的 H 和 L 进行连接



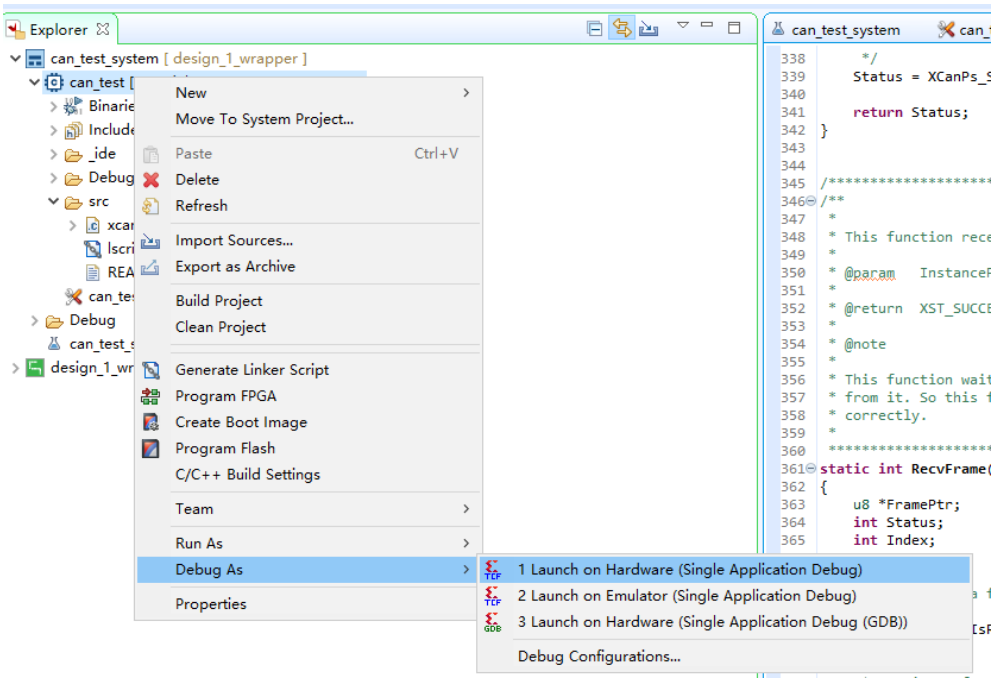
2. 下载程序



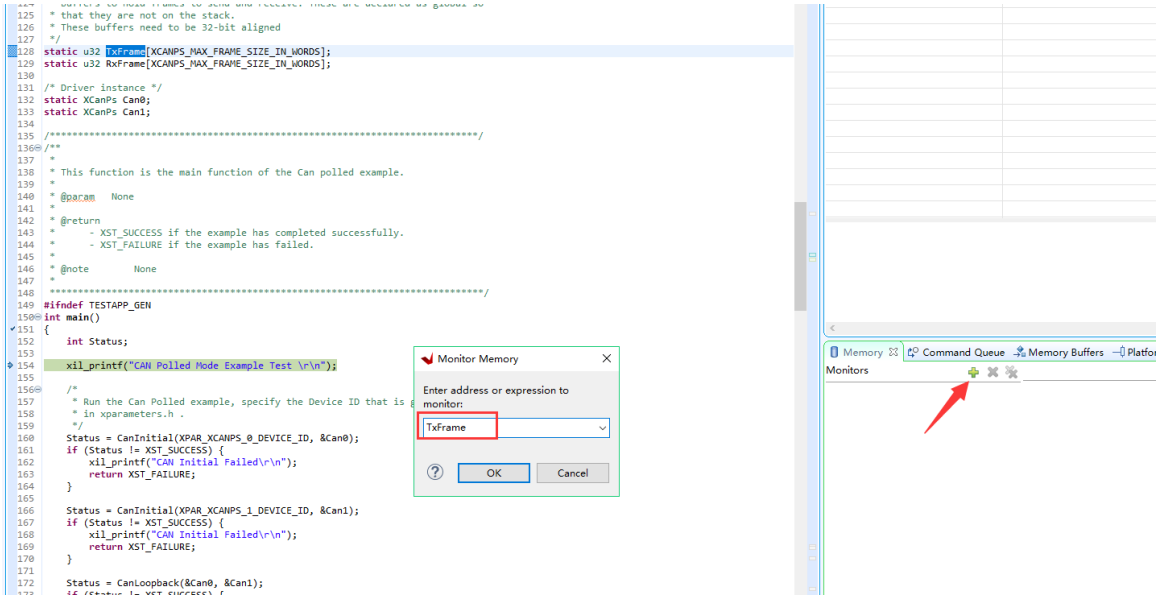
3. 在串口看到打印信息



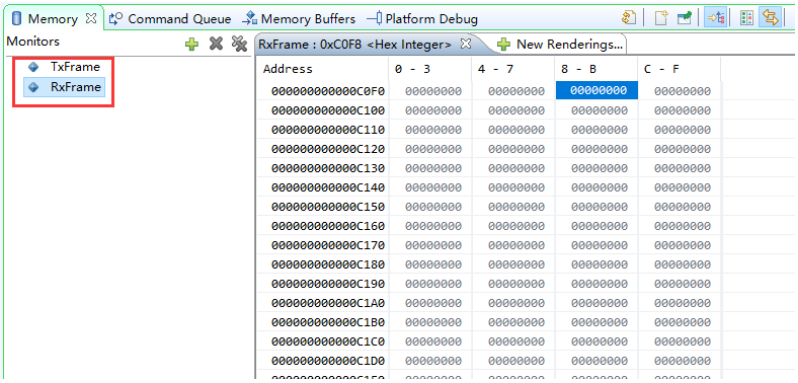
4. 我们也可以利用 Debug 看下程序的运行情况，选择 Debug As 进入 Debug 模式



5. 在 Memory 中添加发送的 buffer



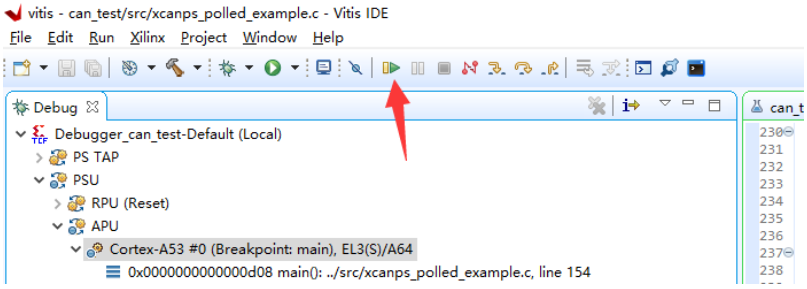
同样的方法，添加 RxFrame，观察两个缓存区的数据



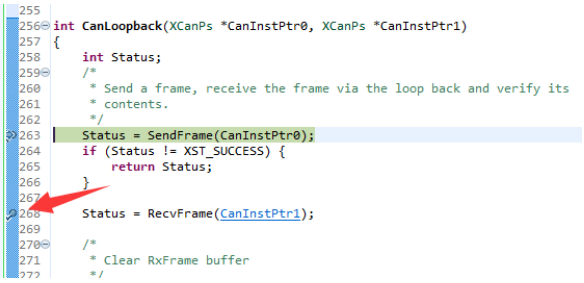
6. 在 CanLoopback 函数中，SendFrame 处添加断点

```
255
256 int CanLoopback(XCanPs *CanInstPtr0, XCanPs *CanInstPtr1)
257 {
258     int Status;
259     /*
260      * Send a frame, receive the frame via the loop back and verify its
261      * contents.
262      */
263     Status = SendFrame(CanInstPtr0);
264     if (Status != XST_SUCCESS) {
265         return Status;
266     }
267
268     Status = RecvFrame(CanInstPtr1);
269
270     /*
271      * Clear RxFrame buffer
272      */
273     memset(RxFrame, 0, XCANPS_MAX_FRAME_SIZE) ;
274
275     /*
276      * Send a frame, receive the frame via the loop back and verify its
277      * contents.
278      */
279     Status = SendFrame(CanInstPtr1);
280     if (Status != XST_SUCCESS) {
281         return Status;
282     }
283
284     Status = RecvFrame(CanInstPtr0);
285
286     return Status;
287
288 }
```

7. 点击运行

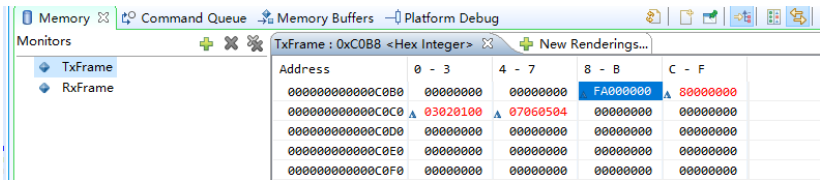


8. 程序运行到此处，然后在 RecvFrame 处再设置一个断点

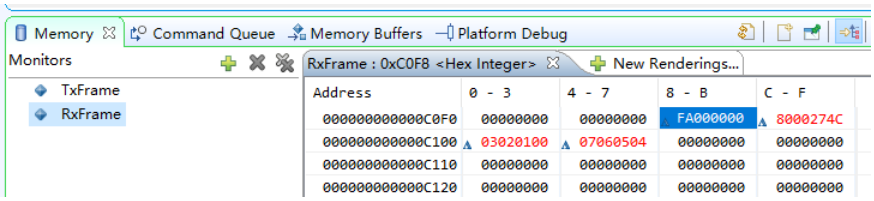


```
255
256 int CanLoopback(XCanPs *CanInstPtr0, XCanPs *CanInstPtr1)
257 {
258     int Status;
259     /*
260      * Send a frame, receive the frame via the loop back and verify its
261      * contents.
262      */
263     Status = SendFrame(CanInstPtr0);
264     if (Status != XST_SUCCESS) {
265         return Status;
266     }
267
268     Status = RecvFrame(CanInstPtr1);
269
270     /*
271      * Clear RxFrame buffer
272      */
```

9. 再次点击运行，可以看到 TxFrame 产生的数据



10. 按 F6 单步运行，即可以看到 RxFrame 接收到的数据，这样就完成了 CAN0 到 CAN1 的数据发送与接收。



11. 同样，大家可以试一下 CAN1 到 CAN0 的数据发送

```
256 int CanLoopback(XCanPs *CanInstPtr0, XCanPs *CanInstPtr1)
257 {
258     int Status;
259     /*
260      * Send a frame, receive the frame via the loop back and verify its
261      * contents.
262      */
263     Status = SendFrame(CanInstPtr0);
264     if (Status != XST_SUCCESS) {
265         return Status;
266     }
267
268     Status = RecvFrame(CanInstPtr1);
269
270     /*
271      * Clear RxFrame buffer
272      */
273     memset(RxFrame, 0, XCANPS_MAX_FRAME_SIZE) ;
274
275     /*
276      * Send a frame, receive the frame via the loop back and verify its
277      * contents.
278      */
279     Status = SendFrame(CanInstPtr1);
280     if (Status != XST_SUCCESS) {
281         return Status;
282     }
283
284     Status = RecvFrame(CanInstPtr0);
285
286     return Status;
287
288 }
```

第十章 PS 端 I2C 的使用

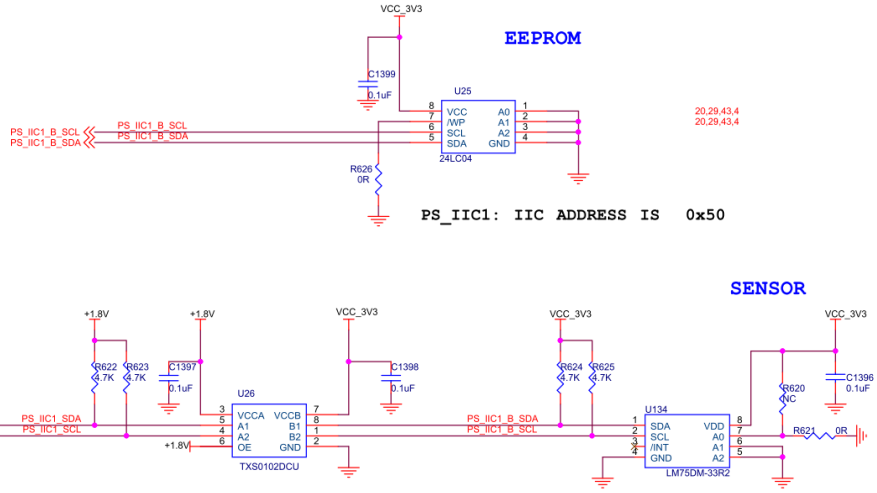
vivado 工程目录为“ps_hello/vivado”

vitis 工程目录为“ps_i2c/vitis”

软件工程师工作内容

以下为软件工程师负责内容。

从原理图中可以看出，PS 端的 I2C1 接了两个外设，分别是 EEPROM，温度传感器，本章介绍如何利用 I2C 读写外设。



10.1 Vitis 程序开发

10.1.1 温度传感器测试

温度传感器用的是 LM75，以下是它的寄存器说明，只要读取 Temperature 寄存器的值就可以了，因此它的地址为 0x48+r/w

Table 1. Slave Address

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
1	0	0	1	A2	A1	A0	R/$\overline{W}$

Table 2. Register Functions

REGISTER NAME	ADDRESS (hex)	POR STATE (hex)	POR STATE (binary)	POR STATE (°C)	READ/ WRITE
Temperature	00	000X	0000 0000 0XXX XXXX	—	Read only
Configuration	01	00	0000 0000	—	R/W
T_{HYST}	02	4B0X	0100 1011 0XXX XXXX	75	R/W
T_{OS}	03	500X	0101 0000 0XXX XXXX	80	R/W

X = Don't care.

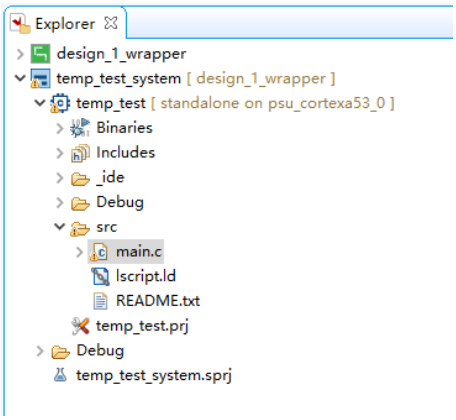
而它的数据是有两个字节，第一个字节为整数位，第二个字节为小数位，最高位为 1，即 0.5 摄氏度，如果为 0，小数位则为 0

Table 4. Temperature Data Output Format

TEMPERATURE (°C)	DIGITAL OUTPUT	
	BINARY	HEX
+125	0111 1101 0XXX XXXX	7D0X
+25	0001 1001 0XXX XXXX	190X
+0.5	0000 0000 1XXX XXXX	008X
0	0000 0000 0XXX XXXX	000X
-0.5	1111 1111 1XXX XXXX	FF8X
-25	1110 0111 0XXX XXXX	E70X
-55	1100 1001 0XXX XXXX	C90X

X = Don't care.

1. 新建 Vitis 工程



2. 在 main.c 中，i2c_init 函数用于 i2c 的初始化及速率设定

```
int i2c_init(XIicPs *Iic,short DeviceID ,u32 IIC_SCLK_RATE)
{
    XIicPs_Config *Config;
    int Status;
    /*
     * Initialize the Can device.
     */
    Config = XIicPs_LookupConfig(DeviceID);
    if (NULL == Config) {
        xil_printf("XIicPs_LookupConfig failure\r\n");
        return XST_FAILURE;
    }

    Status = XIicPs_CfgInitialize(Iic, Config, Config->BaseAddress);
    if (Status != XST_SUCCESS) {
        xil_printf("XIicPs_CfgInitialize failure\r\n");
        return XST_FAILURE;
    }
    /*
     * Set i2c clock rate
     */
    XIicPs_SetSClk(Iic, IIC_SCLK_RATE);
    while (XIicPs_BusIsBusy(Iic));  // Wait
    return XST_SUCCESS;
}
```

3. temp_read 函数设置了读取的数据为 2 字节，用于读取 i2c 的数据

```
void temp_read(XIicPs *InstancePtr, u8 *rd_data, char IIC_ADDR)
{
    /*
     * i2c receiver
     */
    XIicPs_MasterRecvPolled(InstancePtr, rd_data, 2, IIC_ADDR);
    while (XIicPs_BusIsBusy(InstancePtr));
}
```

4. 程序很简单，根据收到的数据值，每秒打印出来显示。要注意的是 ZYNQ 的设备地址为 7bit，例如 LM75 的地址为 0x48+r/w，也就是取前面 7 位 0x48。

```
int main()
{
    //temperature point
    float point ;
    //temperature
    float temp ;

    //Initial i2c
    i2c_init(&rtc_i2c, XPAR_XIICPS_0_DEVICE_ID ,100000) ;//100KHz

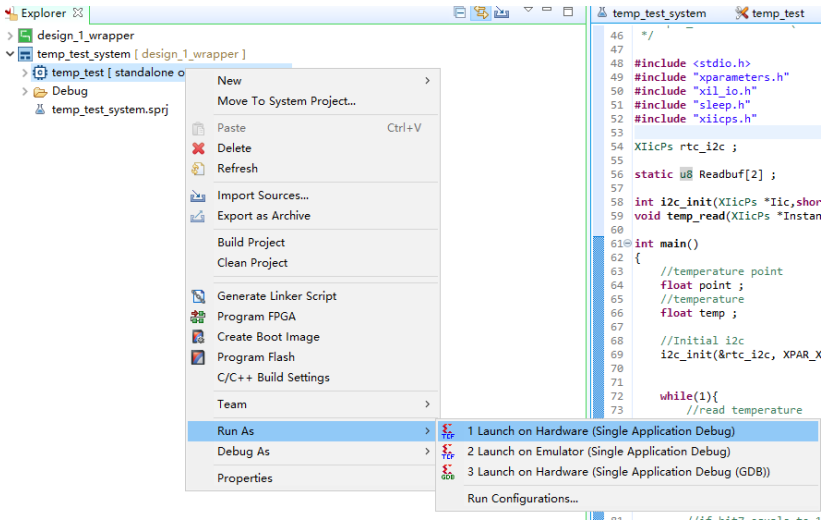

    while(1){
        //read temperature
        temp_read(&rtc_i2c, Readbuf, 0x48) ;
        //if bit 7 equals to 1, point is 0.5C, or 0
        if (Readbuf[1] & 0x80)
            point = 0.5 ;
        else
            point = 0 ;

        //if bit7 equals to 1, then it is negative value
        if (Readbuf[0] & 0x80){
            temp = (float)(Readbuf[0]-256) + point ;
        }
        else{
            temp = (float)(Readbuf[0]) + point ;
        }

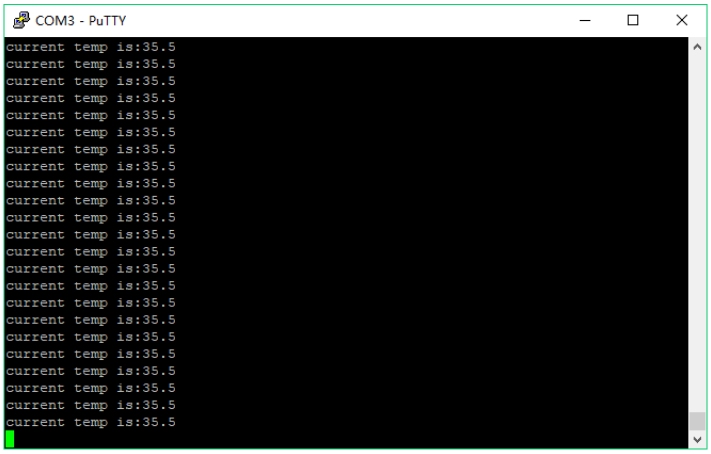
        printf("current temp is:%.1f\t\n",temp);

        sleep(1) ;
    }
    return 0;
}
```

5. 下载程序

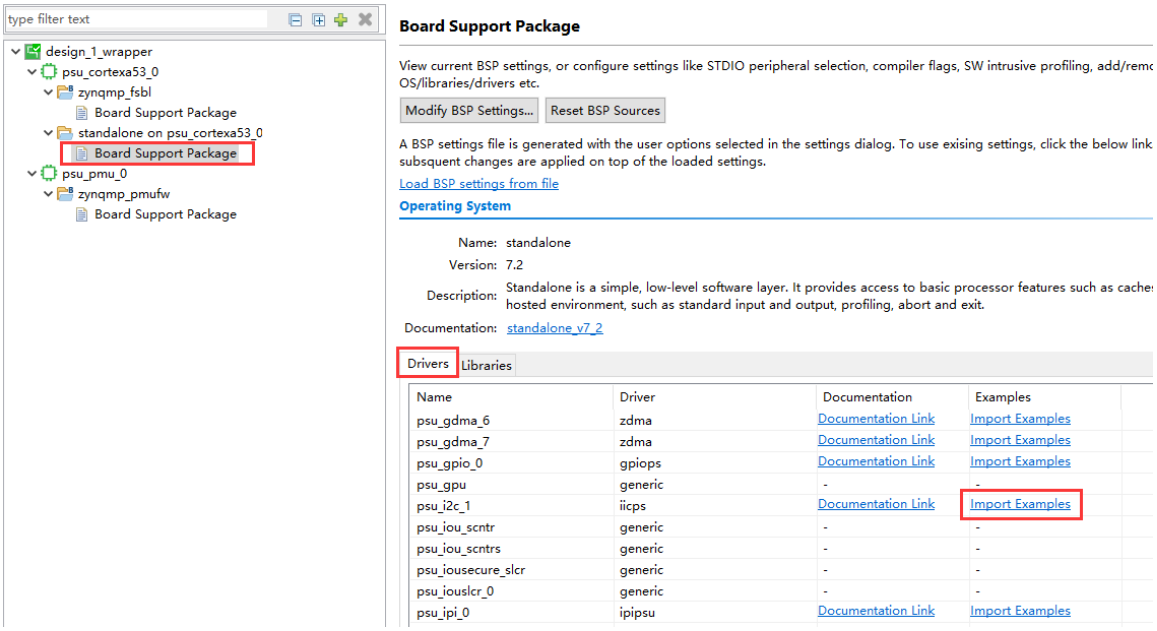


6. 串口打印信息

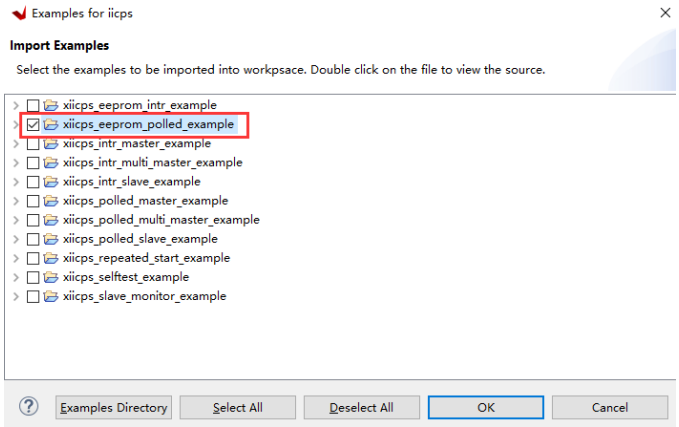


10.1.2 EEPROM 读写

1. 导入 emample 工程



2. 导入 xiicps_eeprom_polled_example 工程



EEPROM 的程序比较简单，具体代码大家可以自己去看，这里不详细说了。下面只对程序的功能和一些要点做一下介绍：

1. EEPROM 的设备地址在程序的前面做了定义，这个地址就是系统访问外部 IIC 外设的设备地址，这里 EEPROM 地址修改为 0x50， 相当于 8bit 的 0xA0。

```
/**Searching for the required EEPROM Address and user can also add
 * their own EEPROM Address in the below array list**/
u16 EepromAddr[] = {0x50,0x55,0};
```

EEPROM 的设备地址可以在 24LC04 的芯片手册上找的到，高 4 位是 A，后面 3 位是 Block 地址，因为 24LC04 只有 2 个 Block， 所以高 2 位 Block Address 是无效的。

Operation	Control Code	Block Select	R/$\overline{W}$
Read	1010	Block Address	1
Write	1010	Block Address	0

2. 由于 EEPROM 的地址为 1 个字节，在程序中修改如下，并保存编译

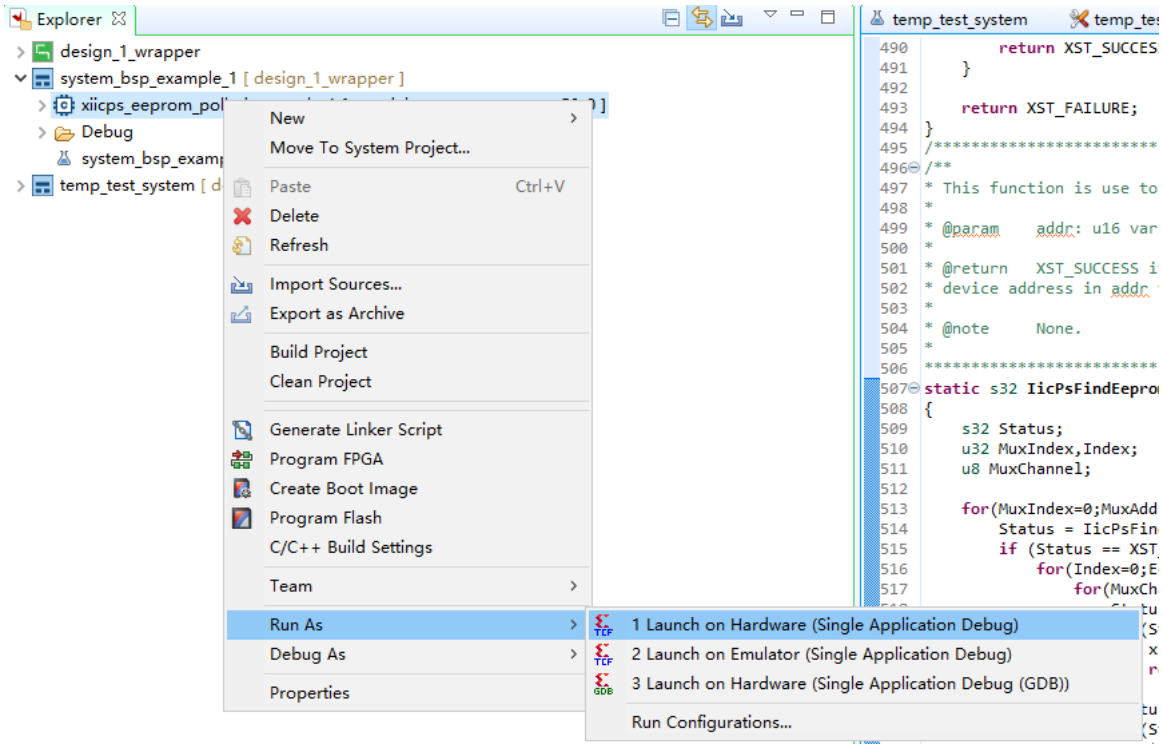
```
static s32 IicPsFindEeprom(u16 *Eeprom_Addr,u32 *PageSize)
{
    s32 Status;
    u32 MuxIndex,Index;
    u8 MuxChannel;

    for(MuxIndex=0;MuxAddr[MuxIndex] != 0;MuxIndex++){
        Status = IicPsFindDevice(MuxAddr[MuxIndex]);
        if (Status == XST_SUCCESS) {
            for(Index=0;EepromAddr[Index] != 0;Index++) {
                for(MuxChannel = MAX_CHANNELS; MuxChannel > 0x0; MuxChannel = MuxChannel >> 1) {
                    Status = MuxInitChannel(MuxAddr[MuxIndex], MuxChannel);
                    if (Status != XST_SUCCESS) {
                        xil_printf("Failed to enable the MUX channel\r\n");
                        return XST_FAILURE;
                    }
                    Status = FindEepromDevice(EepromAddr[Index]);
                    if (Status == XST_SUCCESS) {
                        *Eeprom_Addr = EepromAddr[Index];
                        *PageSize = PAGE_SIZE_16;
                        return XST_SUCCESS;
                    }
                }
            }
        }
    }
    for(Index=0;EepromAddr[Index] != 0;Index++) {
        Status = IicPsFindDevice(EepromAddr[Index]);
        if (Status == XST_SUCCESS) {
            *Eeprom_Addr = EepromAddr[Index];
            *PageSize = PAGE_SIZE_16;
            return XST_SUCCESS;
        }
    }
    return XST_FAILURE;
}
```

3. 程序流程如下：

ReadBuffer 清 0，WriteBuffer 赋 FF→写 16 个字节到 EEPROM→读 EEPROM 的 16 个字节到 ReadBuffer→检验是否正确→Readbuffer 清 0，WriteBuffer 赋值 10~25→写 16 字节到 EEPROM→读 16 字节到 ReadBuffer→检验是否正确→返回

4. 下载程序



5. 串口结果

```
IIC EEPROM Polled Mode Example Test
Successfully ran IIC EEPROM Polled Mode Example Test
```

6. 大家也可以按照”PS 端 CAN 的使用”一章的 Debug 方法调试

第十一章 PS 端 DiplayPort 的使用

vivado 工程目录为“ps_hello/vivado”

vitis 工程目录为“ps_dp/vitis”

本章介绍 PS 端 DisplayPort 的使用。Vivado 工程仍然基于“ps_hello”

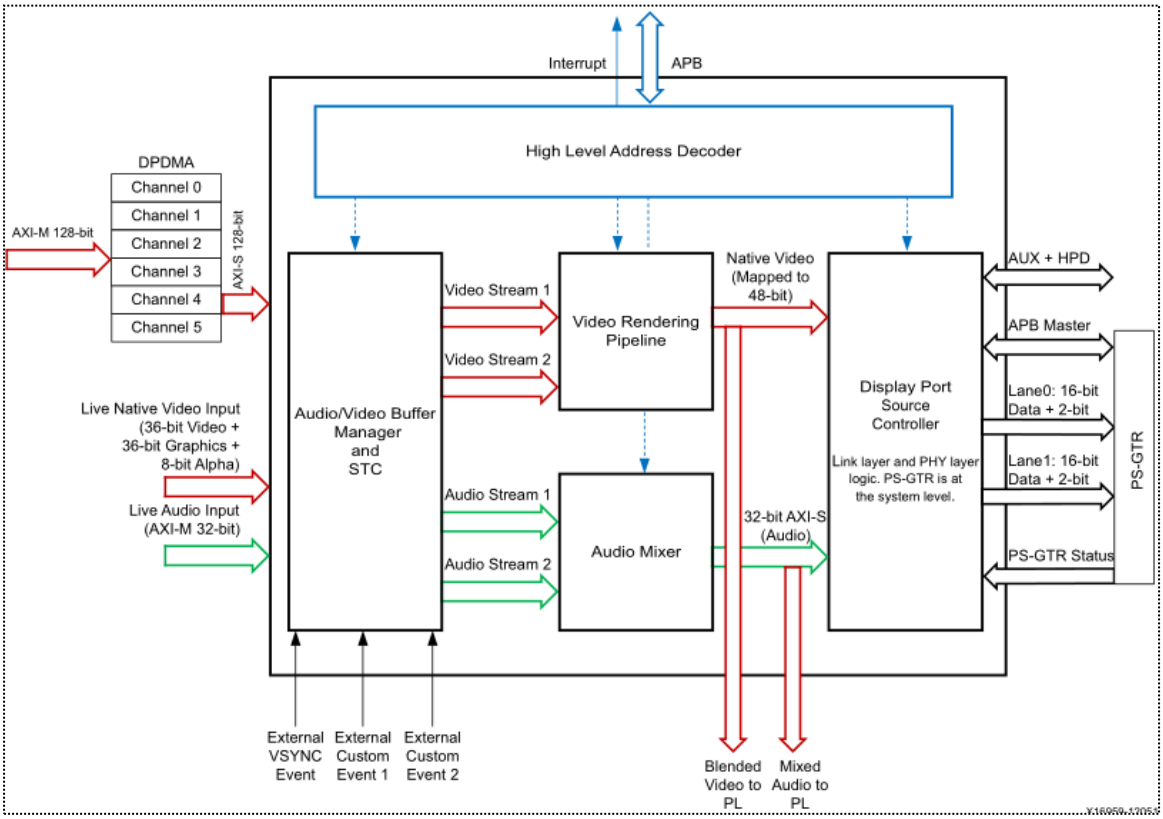
软件工程师工作内容

以下为软件工程师负责内容。

11.1 接口介绍

DisplayPort v1.2 协议，支持 4 个 5.4G 的 lane，但本控制器只支持两个 lane，分辨率最大支持 4096*2160@30。

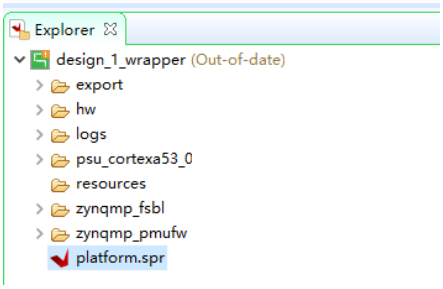
控制器数据接口如下图：



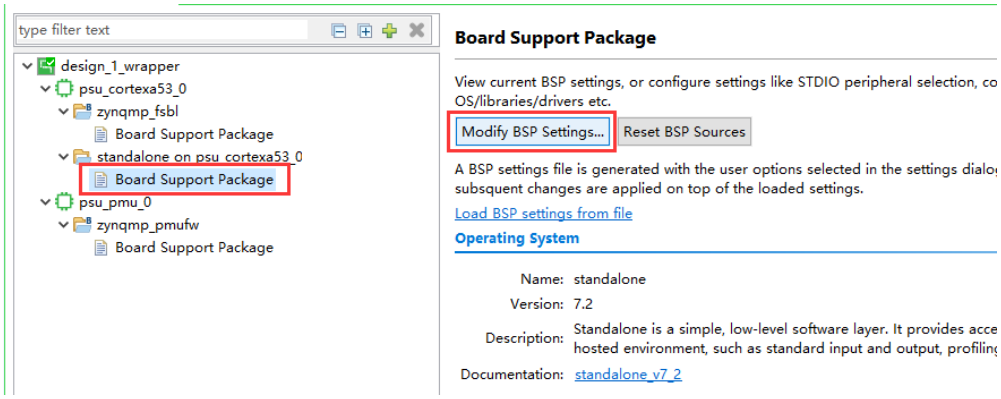
图中，AXI-M 用于读取内存中的视频和音频数据，这里叫非实时音视频，DPDMA 有六个通道，其中 3 路用于视频，1 路用于图形，2 路用于音频。

11.2 Example 工程介绍

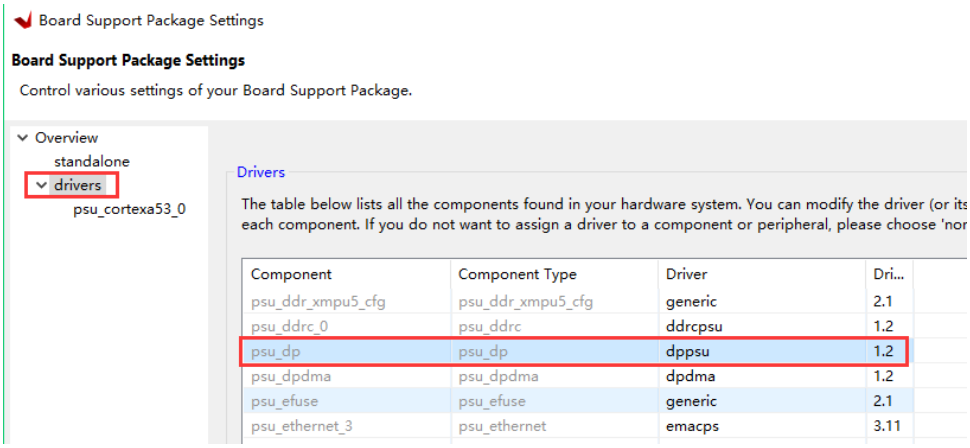
1） 新建 platform，过程不再介绍，在”PS 端 RTC 中断实验”中已经介绍过。



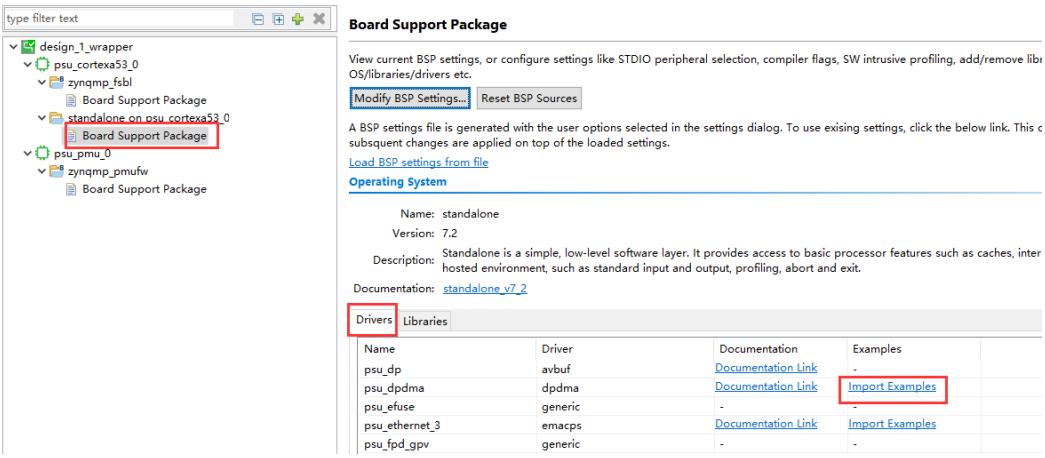
2） 配置 BSP

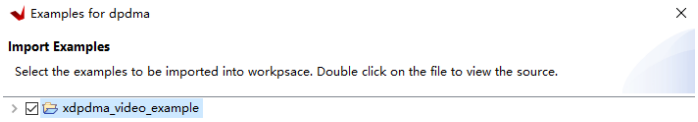


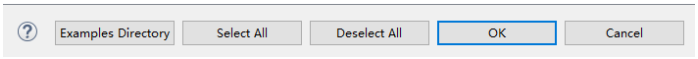
并将 psu_dp 驱动改为 dppsu，然后点击 OK



3） 导入 example 工程





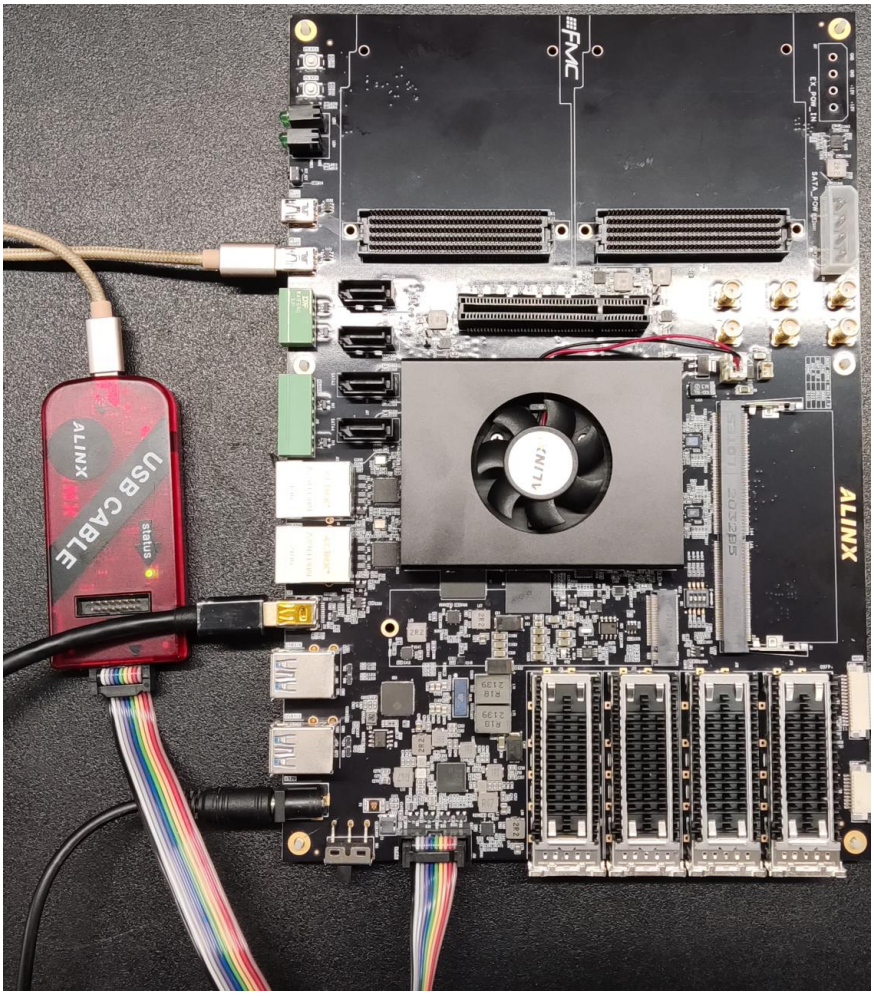


4）例子默认是 1080P，RGBA 显示的，可以将 RGBA 的 Alpha 值都改成 FF，使显示效果更好，保存，并编译工程。

```
*
*****************************************************************************/
u8 *GraphicsOverlay(u8* Frame, Run_Config *RunCfgPtr)
{
    u64 Index;
    u32 *RGBA;
    RGBA = (u32 *) Frame;
    /*
         * Red at the top half
         * Alpha = 0x0F
         * */
    for(Index = 0; Index < (BUFFERSIZE/4) /2; Index ++) {
        RGBA[Index] = 0xFF0000FF;
    }
    for(; Index < BUFFERSIZE/4; Index ++) {
        /*
         * Green at the bottom half
         * Alpha = 0xF0
         * */
        RGBA[Index] = 0xFF00FF00;
    }
    return Frame;
}
```

11.3 板上验证

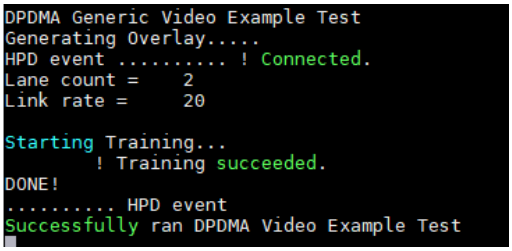
连接板上的 MINI DP 接口



下载后，显示效果如下



在串口工具中可以看到 DP 口进行了训练并成功运行。


DPDMA Generic Video Example Test
Generating Overlay.....
HPD event .... ! Connected.
Lane count =   2
Link rate =   20
Starting Training...
        ! Training succeeded.
DONE!
.... HPD event
Successfully ran DPDMA Video Example Test

第十二章 PS 端 SD 卡读写

vivado 工程目录为“ps_hello/vivado”

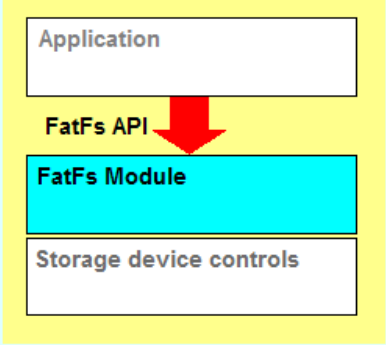
vitis 工程目录为“ps_sd/vitis”

本章介绍使用 FatFs 文件系统模块读取 SD 卡的 BMP 图片，并通过 DP 显示。

12.1 FatFs 简介

FatFs 是一个通用的文件系统模块，用于在小型嵌入式系统中实现 FAT 文件系统。FatFs 的编写遵循 ANSI C，因此不依赖于硬件平台。它可以嵌入到便宜的微控制器中，如 8051, PIC, AVR, SH, Z80, H8, ARM 等等，不需要做任何修改。

应用程序通过 API 函数来调用 FatFs 系统模块，从而来控制 SD 卡这些存储设备。



FatFs 系统提供了很多 API 函数，我们在下面列举了以下我们例程中会用的的 API 函数。

f_mount - 注册/注销一个工作区域（Work Area）

f_open - 打开/创建一个文件

f_close - 关闭一个文件

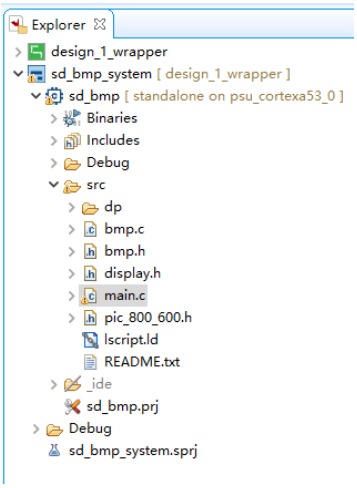
f_read - 读文件

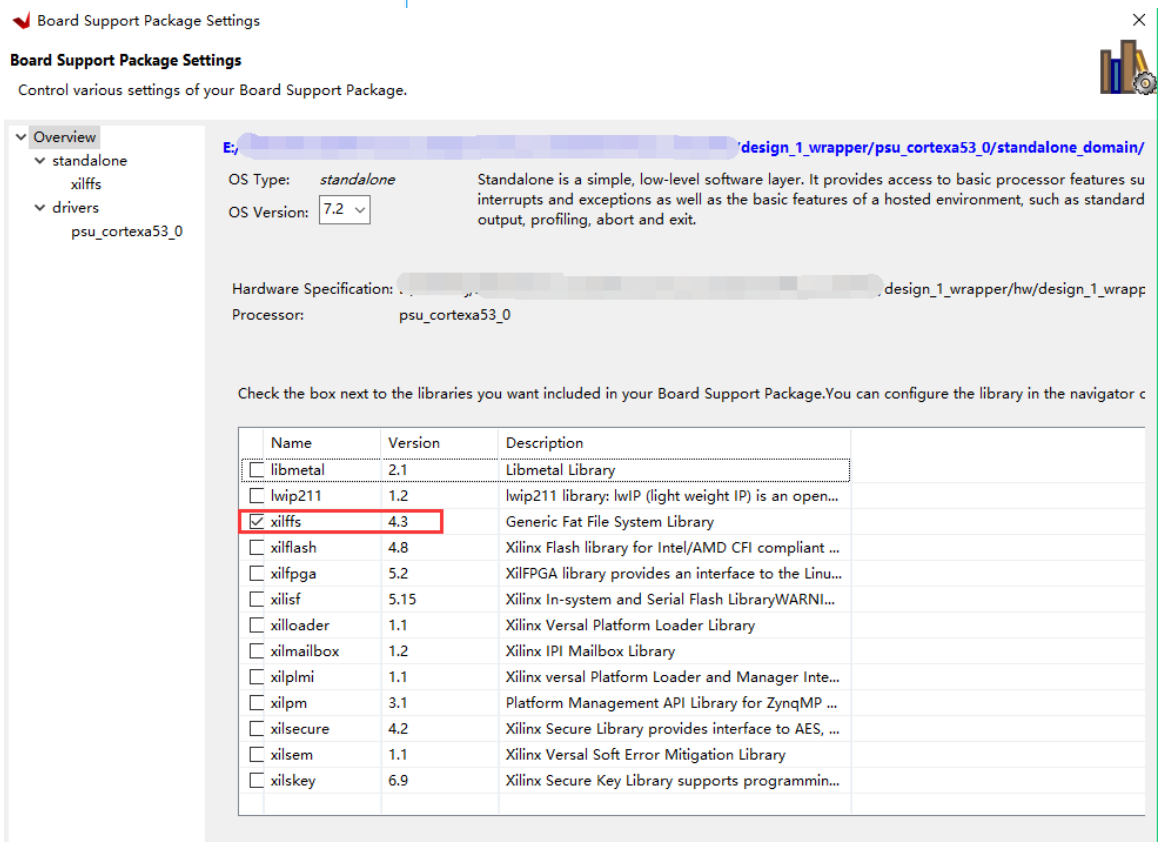
f_write - 写文件

关于 API 函数的介绍和说明，大家可以参考以下的网站进行更深一步的了解，这个网站上
给出来了每个 API 函数的使用说明和例子。
http://elm-chan.org/fsw/ff/00index_e.html

12.2 Vitis 程序开发

1. 打开 Vitis 软件，我们已经为大家生成了一个 sd_bmp 的工程。这里需要配置 BSP 支持包的属性，在 Board Support Package Settings 里选择 xilffs 项，使能项目支持 xilffs 文件系统。





关于 xilffs 库是 Xilinx 提供的 FAT 文件系统支持包，用户可以调用库里的 API 函数实现对 SD/eMMC 等设备的操作。xilffs 库里主要包含 FAT 的文件系统(File System Files)和驱动层文件(Glue Layer Files)。

2. 关于 xilffs 库的介绍和应用，大家可以参考以下 Xilinx 官网链接：
   http://www.wiki.xilinx.com/xilffs

3. 接下来我们来看 sd_bmp 的工程代码。在工程代码里，我们需要把 SD 卡里存储的 bmp

格式的图像数据读出来，去掉图像头后放到 DP 的显示缓冲区中，然后实现图像在 DP 显示器的显示。

4. 在 bmp.c 文件里，我们添加了一个 bmp_read 的函数，在这个函数里首先用 f_open 函数打开一个 SD 卡里的 bmp 的图片文件。然后读取这个文件的前面 54 个字节，因为 BMP 图像文件的前面 54 个字节为图像头文件，里面包含了图像的像素大小信息。再一行一行的读取图像数据存到 DP 的 frame 显示缓冲区中。

   由于 BMP 的存储是上下颠倒的，因此在 bmp_read 函数中调整了顺序，存入 frame 缓存区。

main.c bmp.c

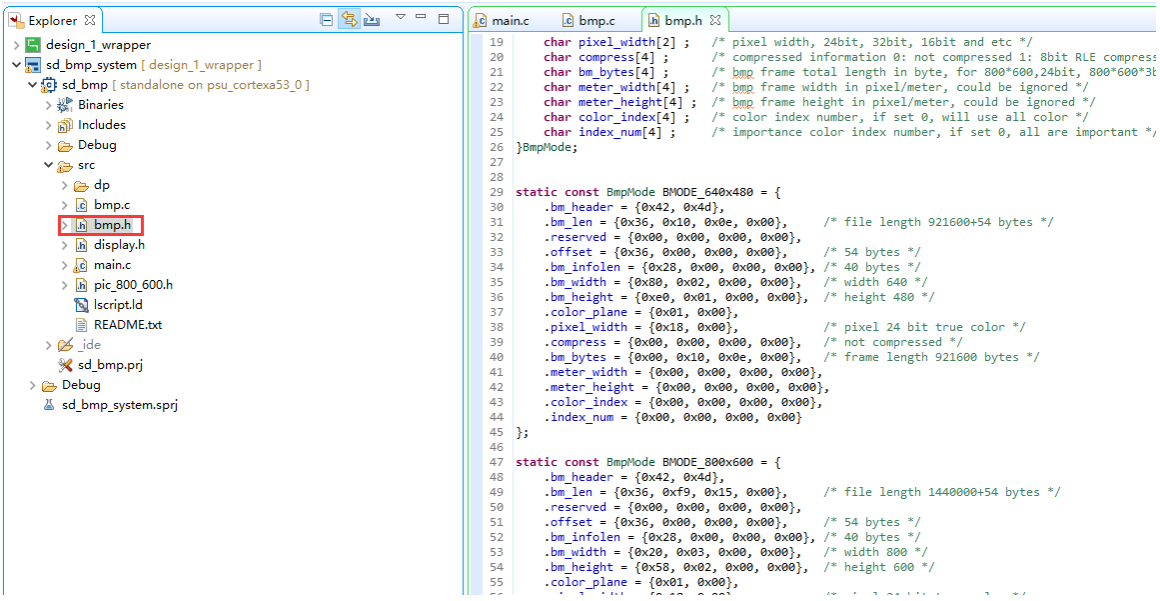
```
#include "xil_types.h"
#include "ff.h"
#include "stdio.h"
#include "string.h"

unsigned char read_line_buf[1920 * 3];
unsigned char Write_line_buf[1920 * 3];
void bmp_read(char * bmp,u8 *frame,u32 stride, FIL *fil)
{
    short y,x;
    short Ximage;
    short Yimage;
    u32 iPixelAddr = 0;
    FRESULT res;
    unsigned char TMPBUF[64];
    unsigned int br;

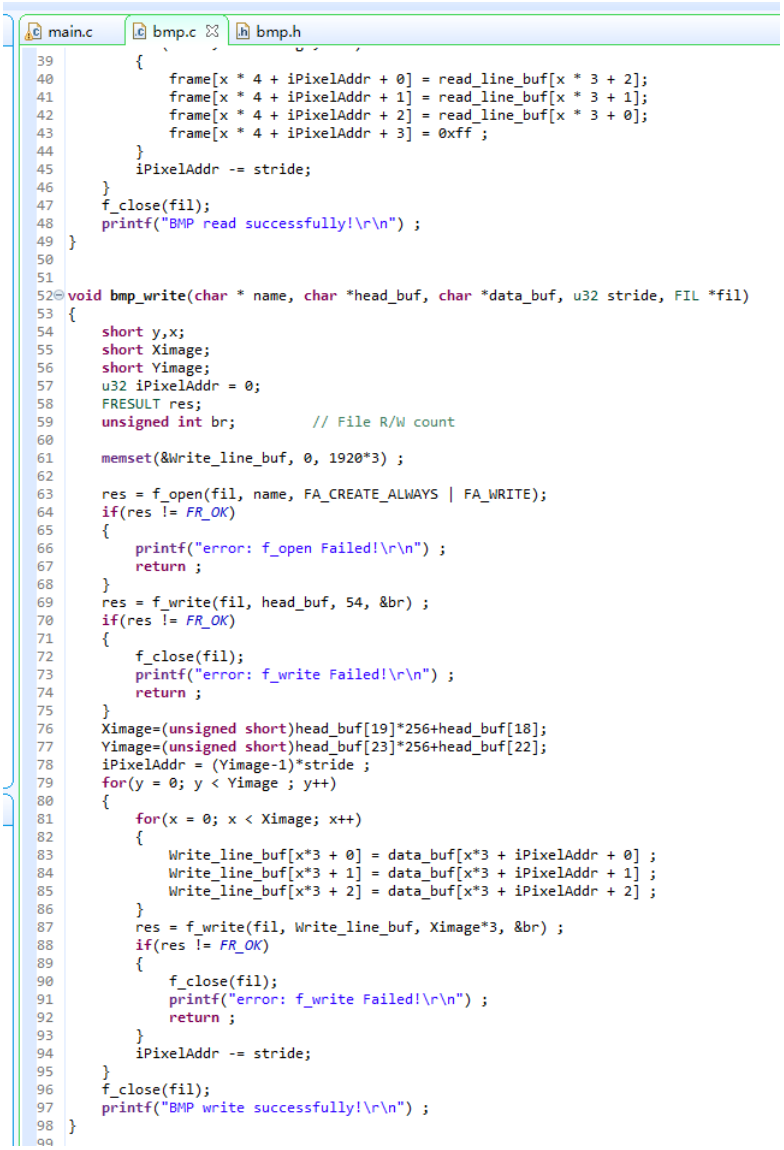
    res = f_open(fil, bmp, FA_OPEN_EXISTING | FA_READ);
    if(res != FR_OK)
    {
        printf("error: f_open Failed!\r\n") ;
        return ;
    }
    res = f_read(fil, TMPBUF, 54, &br);
    if(res != FR_OK)
    {
        f_close(fil);
        printf("Failed to Read!\r\n") ;
        return ;
    }
    Ximage=(unsigned short int)TMPBUF[19]*256+TMPBUF[18];
    Yimage=(unsigned short int)TMPBUF[23]*256+TMPBUF[22];
    iPixelAddr = (Yimage-1)*stride ;

    for(y = 0; y < Yimage ; y++)
    {
        f_read(fil, read_line_buf, Ximage * 3, &br);
        for(x = 0; x < Ximage; x++)
        {
            frame[x * 4 + iPixelAddr + 0] = read_line_buf[x * 3 + 2];
            frame[x * 4 + iPixelAddr + 1] = read_line_buf[x * 3 + 1];
            frame[x * 4 + iPixelAddr + 2] = read_line_buf[x * 3 + 0];
            frame[x * 4 + iPixelAddr + 3] = 0xff ;
        }
        iPixelAddr -= stride;
    }
    f_close(fil);
    printf("BMP read successfully!\r\n") ;
}
```

5. 同时我们也准备了 BMP 文件头结构体，以及一些常用分辨率的图像头设置，放在 bmp.h 文件中。



6. 结合之前小猫图片的显示，将小猫图片保存成 bmp 格式，保存到 SD 卡里，在 bmp_write 函数中，结合 bmp 头和 bmp 数据，保存到 SD 卡。

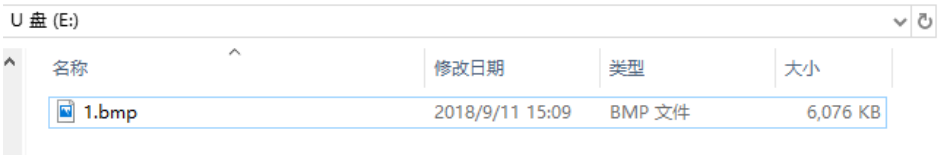


7. 在 main 函数里，调用 bmp_read 函数实现一副图像从 SD 卡读取到 DP 显示缓冲的存储，这里的 BMP 图像的文件名 1.bmp 需要和存储在 SD 卡里的文件名一样。用 bmp_write，将小猫图片写入 SD 卡。

```
main.c  bmp.c  bmp.h
94      if (rc != FR_OK)
95      {
96          return 0 ;
97      }
98
99      rc = f_stat("1:/cat.bmp", &lInfo);
100
101     bmp_write("1:/cat.bmp", (char *)&BMODE_800x600, (char *)&gImage_pic_800_600, 800*3, &fil) ;
102
103     rc = f_stat("1:/1.bmp", &lInfo);
104
105     bmp_read("1:/1.bmp",Frame, DEMO_STRIDE, &fil);
106     Xil_DCacheFlushRange((unsigned int) Frame, DEMO_MAX_FRAME);
107
108     f_unmount("1:/");
109
110
111     return 0;
112 }
```

12.3 板上验证

1. 首先需要先存一副 1920*1080 像素，24bit 的 BMP 文件到 SD 卡里，文件名为 1.bmp（文件在工程目录下），开发板断电情况下，把 SD 卡插入卡座里。



2. 开发板连接 DP 显示器，然后上电，下载程序运行之后我们可以在 DP 显示器上显示 SD 卡里存储的 1.bmp 文件的图像。



3. 之后可将开发板断电，将 SD 卡插到电脑上，可以看到多了 CAT.BMP



1.bmp



CAT.BMP

第十三章 PS 端以太网使用之 lwip

vivado 工程目录为“ps_hello/vivado”

vitis 工程目录为“ps_net/vitis”

软件工程师工作内容

以下为软件工程师负责内容。

开发板有两路千兆以太网，通过 RGMII 接口连接，本实验演示如何使用 Vitis 自带的 LWIP 模板进行 PS 端千兆以太网 TCP 通信。

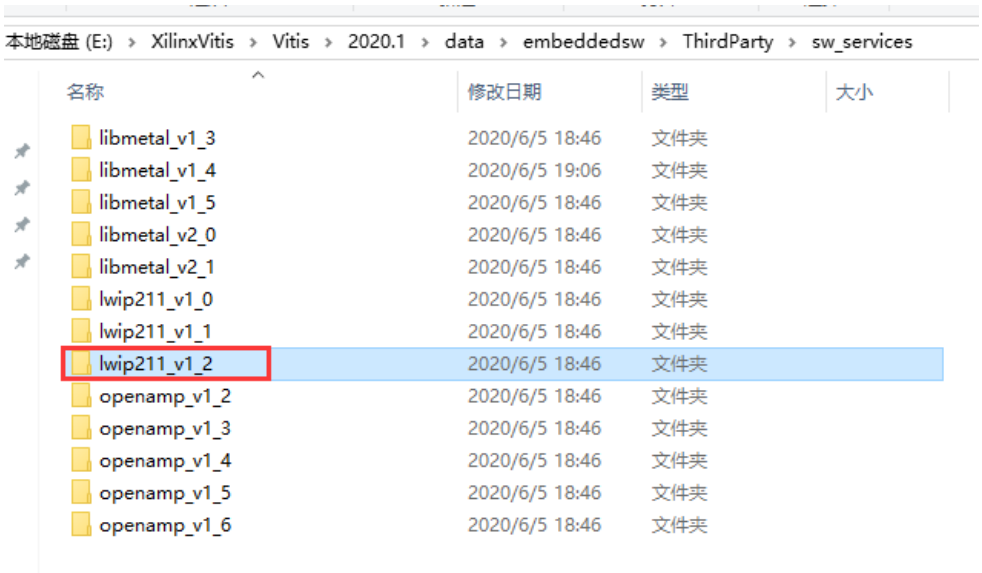
LWIP 虽然是轻量级协议栈，但如果从来没有使用过，使用起来会有一定的困难，建议先熟悉 LWIP 的相关知识。

13.1 Vitis 程序开发

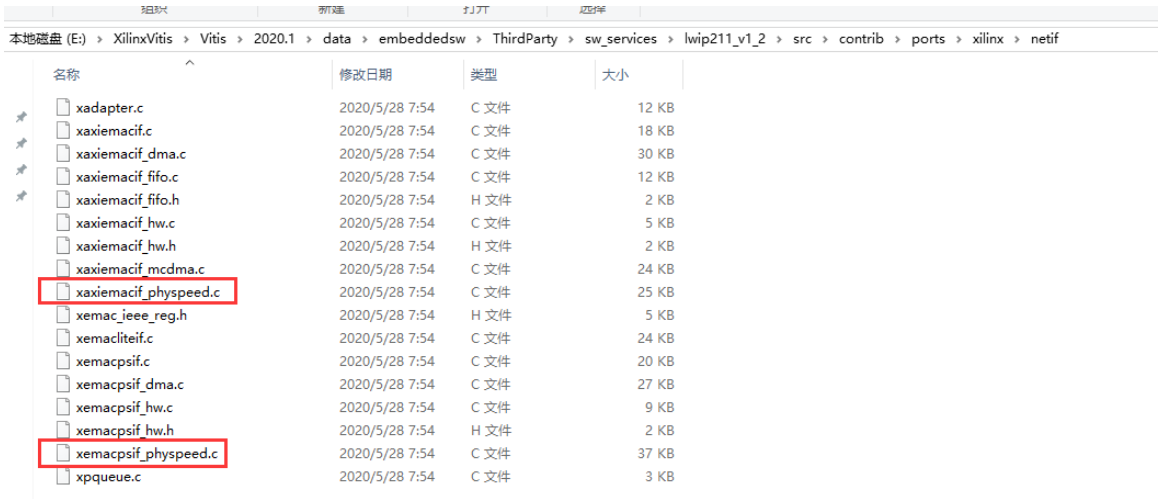
13.1.1 LWIP 库修改

由于自带的 LWIP 库只能识别部分 phy 芯片，如果开发板所用的 phy 芯片不在默认支持范围内，要修改库文件。也可以直接使用修改过的库替换原有的库。

1) 找到库文件目录“X:\xxx\Vitis\2020.1\data\embeddedsw\ThirdParty\sw_services”



2) 找到要修改的文件目录“lwip211_v1_2\src\contrib\ports\xilinx\netif”中文件“xaxiemacif_physpeed.c”和“xemacpsif_physpeed.c”要修改。



3) 修改 PL 端的“xaxiemacif_physpeed.c”文件，添加相关宏定义

```
#define IEEE_MMD_ACCESS_CTRL_DEVAD_MASK          0x1F
#define IEEE_MMD_ACCESS_CTRL_PIDEVAD_MASK        0x801F
#define IEEE_MMD_ACCESS_CTRL_NOPIDEVAD_MASK      0x401F

#define PHY_R0_ISOLATE                           0x0400
#define PHY_DETECT_REG                           1
#define PHY_IDENTIFIER_1_REG                     2
#define PHY_IDENTIFIER_2_REG                     3
#define PHY_DETECT_MASK                          0x1808
#define PHY_MARVELL_IDENTIFIER                   0x0141
#define PHY_TI_IDENTIFIER                        0x2000

/* Marvel PHY flags */
#define MARVEL_PHY_IDENTIFIER                    0x141
#define MARVEL_PHY_MODEL_NUM_MASK                0x3F0
#define MARVEL_PHY_88E1111_MODEL                 0xC0
#define MARVEL_PHY_88E1116R_MODEL                0x240
#define PHY_88E1111_RGMII_RX_CLOCK_DELAYED_MASK  0x0080

/* TI PHY Flags */
#define TI_PHY_DETECT_MASK                       0x796D
#define TI_PHY_IDENTIFIER                        0x2000
#define TI_PHY_DP83867_MODEL                     0xA231
#define DP83867_RGMII_CLOCK_DELAY_CTRL_MASK      0x0003
#define DP83867_RGMII_TX_CLOCK_DELAY_MASK        0x0030
#define DP83867_RGMII_RX_CLOCK_DELAY_MASK        0x0003

/* TI DP83867 PHY Registers */
#define DP83867_R32_RGMIICTL1                    0x32
#define DP83867_R86_RGMIIDCTL                    0x86

#define MICREL_PHY_IDENTIFIER                    0x22
#define MICREL_PHY_KSZ9031_MODEL                 0x220

#define TI_PHY_REGCR            0xD
#define TI_PHY_ADDDR            0xE
```

4) 添加 phy 速度获取函数

```
unsigned int get_phy_speed_ksz9031(XAxiEthernet *xaxiemacp, u32 phy_addr)
{
    u16 control;
    u16 status;
    u16 partner_capabilities;
    xil_printf("Start PHY autonegotiation \r\n");
```

```
	XAxiEthernet_PhyWrite(xaxiemacp,phy_addr, IEEE_PAGE_ADDRESS_REGISTER, 2);
	XAxiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_CONTROL_REG_MAC, &control);
	//control |= IEEE_RGMII_TXRX_CLOCK_DELAYED_MASK;
	control &= ~(0x10);
	XAxiEthernet_PhyWrite(xaxiemacp, phy_addr, IEEE_CONTROL_REG_MAC, control);

	XAxiEthernet_PhyWrite(xaxiemacp, phy_addr, IEEE_PAGE_ADDRESS_REGISTER, 0);

	XAxiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_AUTONEGO_ADVERTISE_REG, &control);
	control |= IEEE_ASYMMETRIC_PAUSE_MASK;
	control |= IEEE_PAUSE_MASK;
	control |= ADVERTISE_100;
	control |= ADVERTISE_10;
	XAxiEthernet_PhyWrite(xaxiemacp, phy_addr, IEEE_AUTONEGO_ADVERTISE_REG, control);

	XAxiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_1000_ADVERTISE_REG_OFFSET,
                                                                    &control);
	control |= ADVERTISE_1000;
	XAxiEthernet_PhyWrite(xaxiemacp, phy_addr, IEEE_1000_ADVERTISE_REG_OFFSET,
                                                                    control);

	XAxiEthernet_PhyWrite(xaxiemacp, phy_addr, IEEE_PAGE_ADDRESS_REGISTER, 0);
	XAxiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_COPPER_SPECIFIC_CONTROL_REG,
                                                                    &control);
	control |= (7 << 12);  /* max number of gigabit attempts */
	control |= (1 << 11);  /* enable downshift */
	XAxiEthernet_PhyWrite(xaxiemacp, phy_addr, IEEE_COPPER_SPECIFIC_CONTROL_REG,
                                                                    control);
	XAxiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_CONTROL_REG_OFFSET, &control);
	control |= IEEE_CTRL_AUTONEGOTIATE_ENABLE;
	control |= IEEE_STAT_AUTONEGOTIATE_RESTART;

	XAxiEthernet_PhyWrite(xaxiemacp, phy_addr, IEEE_CONTROL_REG_OFFSET, control);

	XAxiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_CONTROL_REG_OFFSET, &control);
	control |= IEEE_CTRL_RESET_MASK;
	XAxiEthernet_PhyWrite(xaxiemacp, phy_addr, IEEE_CONTROL_REG_OFFSET, control);

	while (1) {
		XAxiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_CONTROL_REG_OFFSET, &control);
		if (control & IEEE_CTRL_RESET_MASK)
			continue;
		else
			break;
	}
	xil_printf("Waiting for PHY to complete autonegotiation.\r\n");

	XAxiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_STATUS_REG_OFFSET, &status);
	while ( !(status & IEEE_STAT_AUTONEGOTIATE_COMPLETE) ) {
		sleep(1);
		XAxiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_STATUS_REG_OFFSET,
                                                                    &status);
		}

	xil_printf("autonegotiation complete \r\n");

	XAxiEthernet_PhyRead(xaxiemacp, phy_addr, 0x1f, &partner_capabilities);

	if ( (partner_capabilities & 0x40) == 0x40)/* 1000Mbps */
		return 1000;
	else if ( (partner_capabilities & 0x20) == 0x20)/* 100Mbps */
		return 100;
	else if ( (partner_capabilities & 0x10) == 0x10)/* 10Mbps */
		return 10;
	else
		return 0;
}
```

5) 修改函数“get_IEEE_phy_speed”，添加对 KSZ9031 的支持。

```
unsigned get_IEEE_phy_speed(XAxiEthernet *xaxiemacp)
{
	u16 phy_identifier;
	u16 phy_model;
	u8 phytype;

#ifdef XPAR_AXIETHERNET_0_BASEADDR
	u32 phy_addr = detect_phy(xaxiemacp);
```

```
    /* Get the PHY Identifier and Model number */
    XAxiEthernet_PhyRead(xaxiemacp, phy_addr, PHY_IDENTIFIER_1_REG, &phy_identifier);
    XAxiEthernet_PhyRead(xaxiemacp, phy_addr, PHY_IDENTIFIER_2_REG, &phy_model);

/* Depending upon what manufacturer PHY is connected, a different mask is
 * needed to determine the specific model number of the PHY. */
    if (phy_identifier == MARVEL_PHY_IDENTIFIER) {
        phy_model = phy_model & MARVEL_PHY_MODEL_NUM_MASK;

        if (phy_model == MARVEL_PHY_88E1116R_MODEL) {
            return get_phy_speed_88E1116R(xaxiemacp, phy_addr);
        } else if (phy_model == MARVEL_PHY_88E1111_MODEL) {
            return get_phy_speed_88E1111(xaxiemacp, phy_addr);
        }
    } else if (phy_identifier == TI_PHY_IDENTIFIER) {
        phy_model = phy_model & TI_PHY_DP83867_MODEL;
        phytype = XAxiEthernet_GetPhysicalInterface(xaxiemacp);

        if (phy_model == TI_PHY_DP83867_MODEL && phytype == XAE_PHY_TYPE_SGMII) {
            return get_phy_speed_TI_DP83867_SGMII(xaxiemacp, phy_addr);
        }

        if (phy_model == TI_PHY_DP83867_MODEL) {
            return get_phy_speed_TI_DP83867(xaxiemacp, phy_addr);
        }
    }
    else if(phy_identifier == MICREL_PHY_IDENTIFIER)
    {
        xil_printf("Phy %d is KSZ9031\n\r", phy_addr);
        get_phy_speed_ksz9031(xaxiemacp, phy_addr);
    }
    else {
       LWIP_DEBUGF(NETIF_DEBUG, ("XAxiEthernet get_IEEE_phy_speed: Detected PHY with
unknown identifier/model.\r\n"));
    }
#endif
#ifdef PCM_PMA_CORE_PRESENT
    return get_phy_negotiated_speed(xaxiemacp, phy_addr);
#endif
}
```

6) 修改 PS 端“xemacpsif_physpeed.c”文件添加宏定义

```
#define PHY_DETECT_REG                          1
#define PHY_IDENTIFIER_1_REG                    2
#define PHY_IDENTIFIER_2_REG                    3
#define PHY_DETECT_MASK                     0x1808
#define PHY_MARVELL_IDENTIFIER              0x0141
#define PHY_TI_IDENTIFIER                   0x2000
#define PHY_REALTEK_IDENTIFIER              0x001c
#define PHY_XILINX_PCS_PMA_ID1          0x0174
#define PHY_XILINX_PCS_PMA_ID2          0x0C00

#define XEMACPS_GMII2RGMII_SPEED1000_FD     0x140
#define XEMACPS_GMII2RGMII_SPEED100_FD      0x2100
#define XEMACPS_GMII2RGMII_SPEED10_FD       0x100
#define XEMACPS_GMII2RGMII_REG_NUM          0x10

#define PHY_REGCR       0x0D
#define PHY_ADDAR       0x0E
#define PHY_RGMIIDCTL   0x86
#define PHY_RGMIICTL    0x32
#define PHY_STS         0x11
#define PHY_TI_CR       0x10
#define PHY_TI_CFG4     0x31

#define MICREL_PHY_IDENTIFIER                   0x22
#define MICREL_PHY_KSZ9031_MODEL                0x220

#define PHY_REGCR_ADDR  0x001F
#define PHY_REGCR_DATA  0x401F
#define PHY_TI_CRVAL    0x5048
#define PHY_TI_CFG4RESVDBIT7    0x80
```

7) 添加 phy 速度获取函数

```
static u32_t get_phy_speed_ksz9031(XEmacPs *xemacpsp, u32_t phy_addr)
{
    u16_t temp;
    u16_t control;
    u16_t status;
    u16_t status_speed;
    u32_t timeout_counter = 0;
    u32_t temp_speed;
    u32_t phyregtemp;

    xil_printf("Start PHY autonegotiation \r\n");

    XEmacPs_PhyWrite(xemacpsp,phy_addr, IEEE_PAGE_ADDRESS_REGISTER, 2);
    XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_CONTROL_REG_MAC, &control);
    control |= IEEE_RGMII_TXRX_CLOCK_DELAYED_MASK;
    XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_CONTROL_REG_MAC, control);

    XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_PAGE_ADDRESS_REGISTER, 0);

    XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_AUTONEGO_ADVERTISE_REG, &control);
    control |= IEEE_ASYMMETRIC_PAUSE_MASK;
    control |= IEEE_PAUSE_MASK;
    control |= ADVERTISE_100;
    control |= ADVERTISE_10;
    XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_AUTONEGO_ADVERTISE_REG, control);

    XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_1000_ADVERTISE_REG_OFFSET,
                    &control);
    control |= ADVERTISE_1000;
    XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_1000_ADVERTISE_REG_OFFSET,
                    control);

    XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_PAGE_ADDRESS_REGISTER, 0);
    XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_COPPER_SPECIFIC_CONTROL_REG,
                                                                &control);
    control |= (7 << 12);   /* max number of gigabit attempts */
    control |= (1 << 11);   /* enable downshift */
    XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_COPPER_SPECIFIC_CONTROL_REG,
                                                                control);
    XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_CONTROL_REG_OFFSET, &control);
    control |= IEEE_CTRL_AUTONEGOTIATE_ENABLE;
    control |= IEEE_STAT_AUTONEGOTIATE_RESTART;
    XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_CONTROL_REG_OFFSET, control);

    XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_CONTROL_REG_OFFSET, &control);
    control |= IEEE_CTRL_RESET_MASK;
    XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_CONTROL_REG_OFFSET, control);

    while (1) {
        XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_CONTROL_REG_OFFSET, &control);
        if (control & IEEE_CTRL_RESET_MASK)
            continue;
        else
            break;
    }

    XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_STATUS_REG_OFFSET, &status);

    xil_printf("Waiting for PHY to complete autonegotiation.\r\n");

    while ( !(status & IEEE_STAT_AUTONEGOTIATE_COMPLETE) ) {
        sleep(1);
        XEmacPs_PhyRead(xemacpsp, phy_addr,
                        IEEE_COPPER_SPECIFIC_STATUS_REG_2,  &temp);
        timeout_counter++;

        if (timeout_counter == 30) {
            xil_printf("Auto negotiation error \r\n");
            return;
        }
        XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_STATUS_REG_OFFSET, &status);
    }
    xil_printf("autonegotiation complete \r\n");

    XEmacPs_PhyRead(xemacpsp, phy_addr,0x1f,
                    &status_speed);

    if ( (status_speed & 0x40) == 0x40)/* 1000Mbps */
```

```
            return 1000;
      else if ( (status_speed & 0x20) == 0x20)/* 100Mbps */
            return 100;
      else if ( (status_speed & 0x10) == 0x10)/* 10Mbps */
            return 10;
      else
            return 0;
      return XST_SUCCESS;
}
```

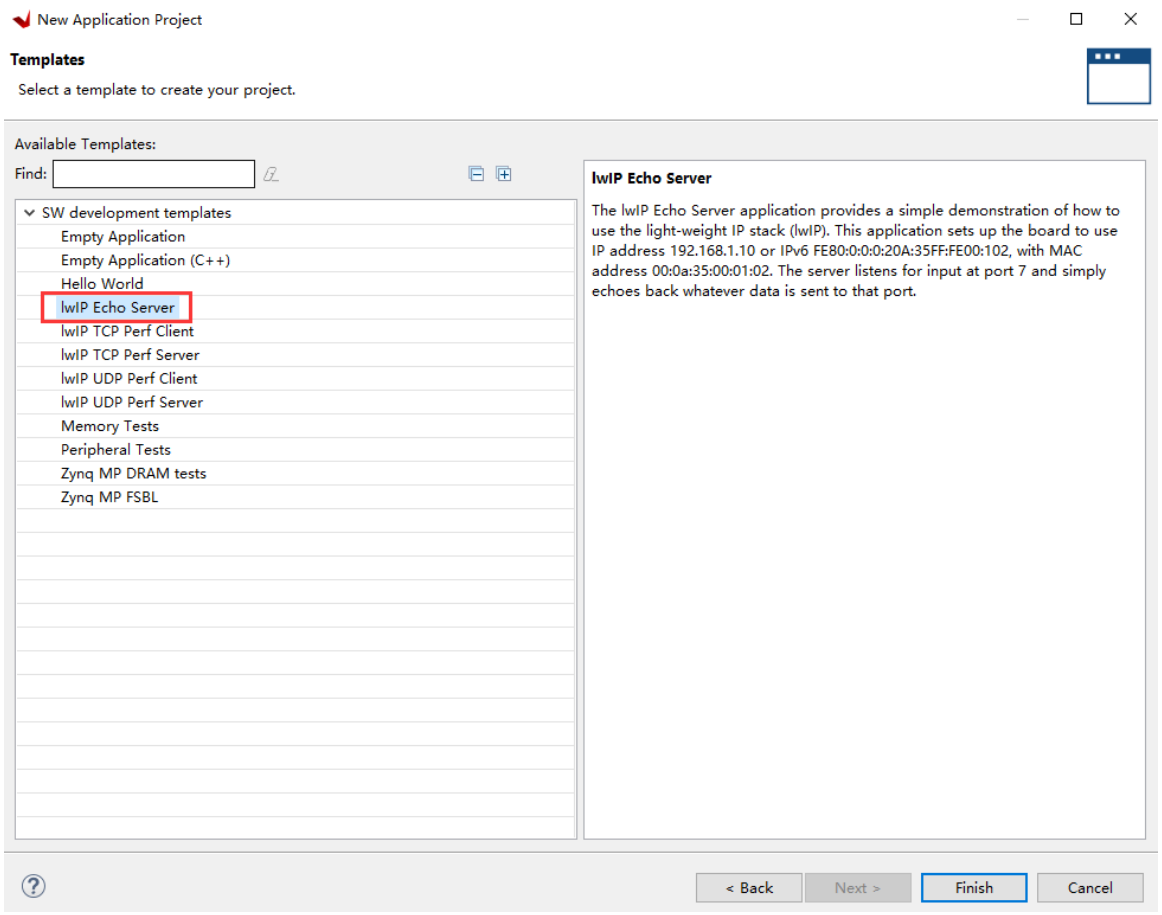
8) 修改函数“get_IEEE_phy_speed”，添加对 KSZ9031 的支持

```
static u32_t get_IEEE_phy_speed(XEmacPs *xemacpsp, u32_t phy_addr)
{
     u16_t phy_identity;
     u32_t RetStatus;

     XEmacPs_PhyRead(xemacpsp, phy_addr, PHY_IDENTIFIER_1_REG,
                     &phy_identity);
     if(phy_identity == MICREL_PHY_IDENTIFIER){
          RetStatus = get_phy_speed_ksz9031(xemacpsp, phy_addr);
     } else if (phy_identity == PHY_TI_IDENTIFIER) {
          RetStatus = get_TI_phy_speed(xemacpsp, phy_addr);
     } else if (phy_identity == PHY_REALTEK_IDENTIFIER) {
          RetStatus = get_Realtek_phy_speed(xemacpsp, phy_addr);
     } else {
          RetStatus = get_Marvell_phy_speed(xemacpsp, phy_addr);
     }

     return RetStatus;
}
```

13.1.2 创建 APP 工程时基于 LWIP 模板

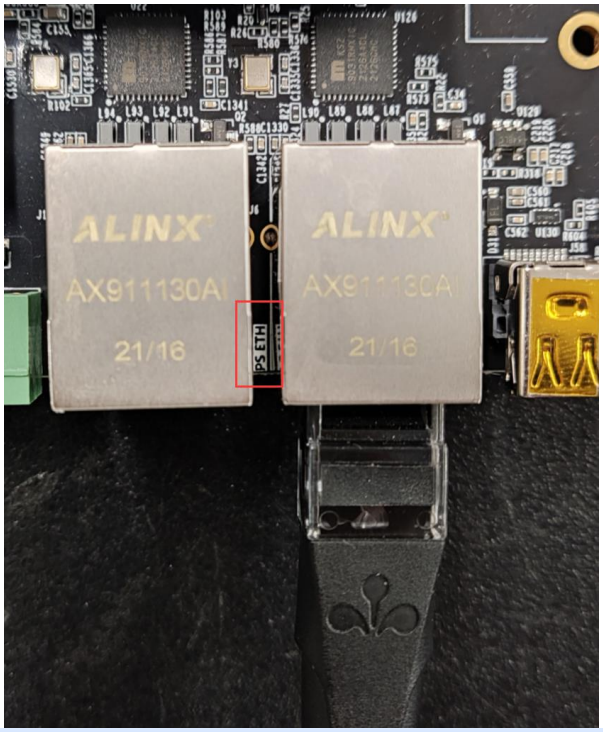


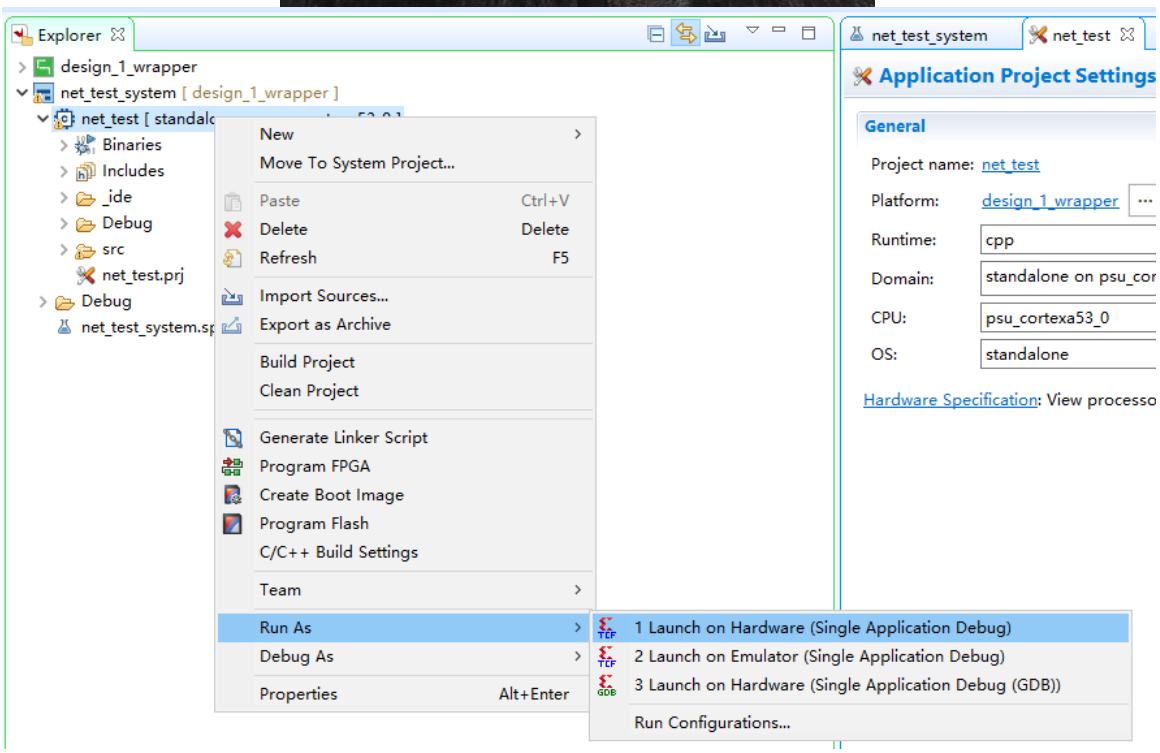
13.2 下载调试

测试环境要求有一台支持 dhcp 的路由器，开发板连接路由器可以自动获取 IP 地址，实验主机和开发板在一个网络，可以相互通信。

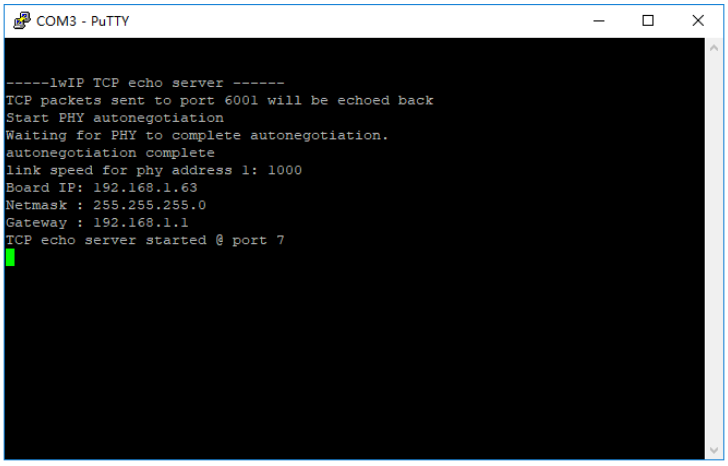
13.2.1 以太网测试

1) 连接串口打开串口调试终端，连接好 PS 端以太网网线到路由器，运行 Vitis 下载程序

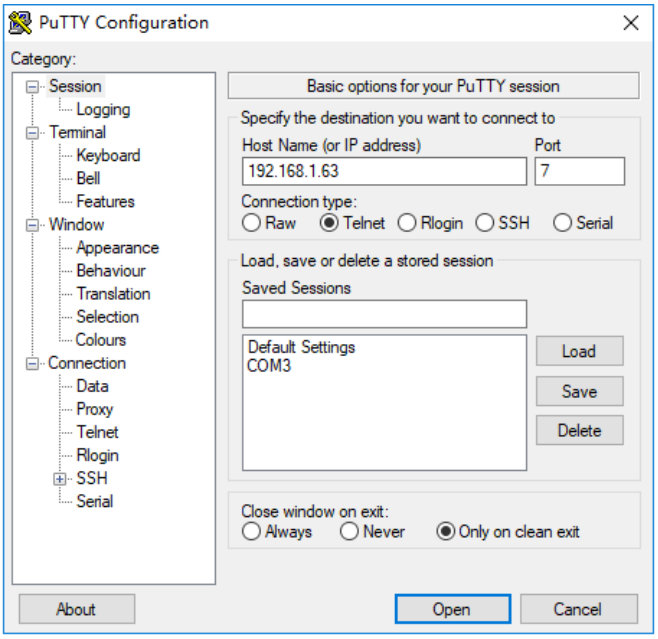




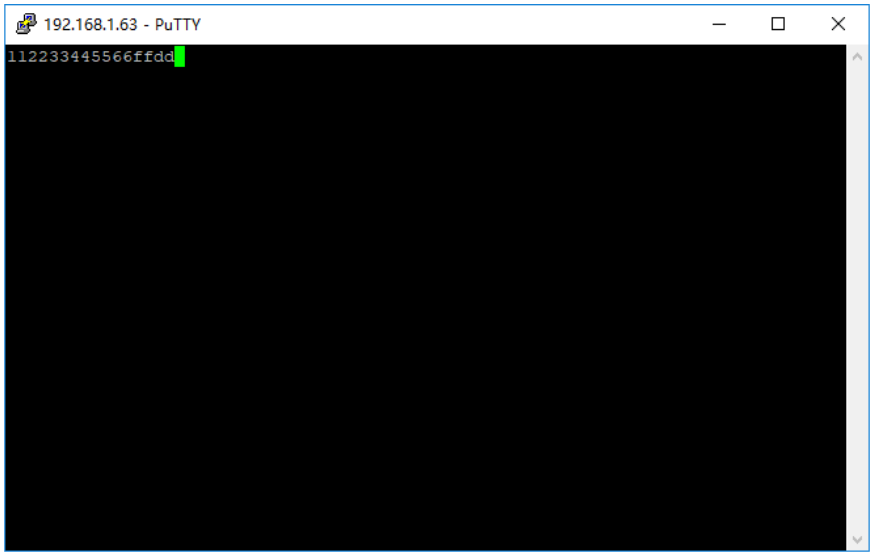
2) 可以看到串口打印出一些信息，可以看到自动获取到地址为“192.168.1.63”，连接速度1000Mbps，tcp 端口为 7



3) 使用 telnet 连接



4) 当输入一个字符时，开发板返回相同字符



13.3 实验总结

通过实验我们更加深刻了解到 Vitis 程序的开发，本实验只是简单的讲解如何创建一个 LWIP 应用，LWIP 可以完成 UDP、TCP 等协议，在后续的教程中我们会提供基于以太网的具体应用，例如 ADC 采集数据通过以太网发送，摄像头数据通过以太网发送上位机显示。

第十四章 PS 端之网络远程更新 QSPI

vivado 工程目录为“ps_hello/vivado”

vitis 工程目录为“ps_remote/vitis”

在实际工作中，会遇到产品升级问题，如果按照程序烧录的方式，可能需要打开产品的外壳，这无疑。本章介绍一种通过网络远程更新 FLASH 程序的方法，包含 UDP 和 TCP 两种方法。

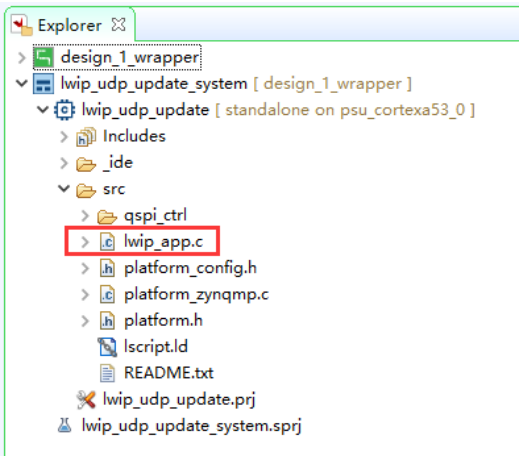
软件工程师工作内容

以下为软件工程师负责内容。

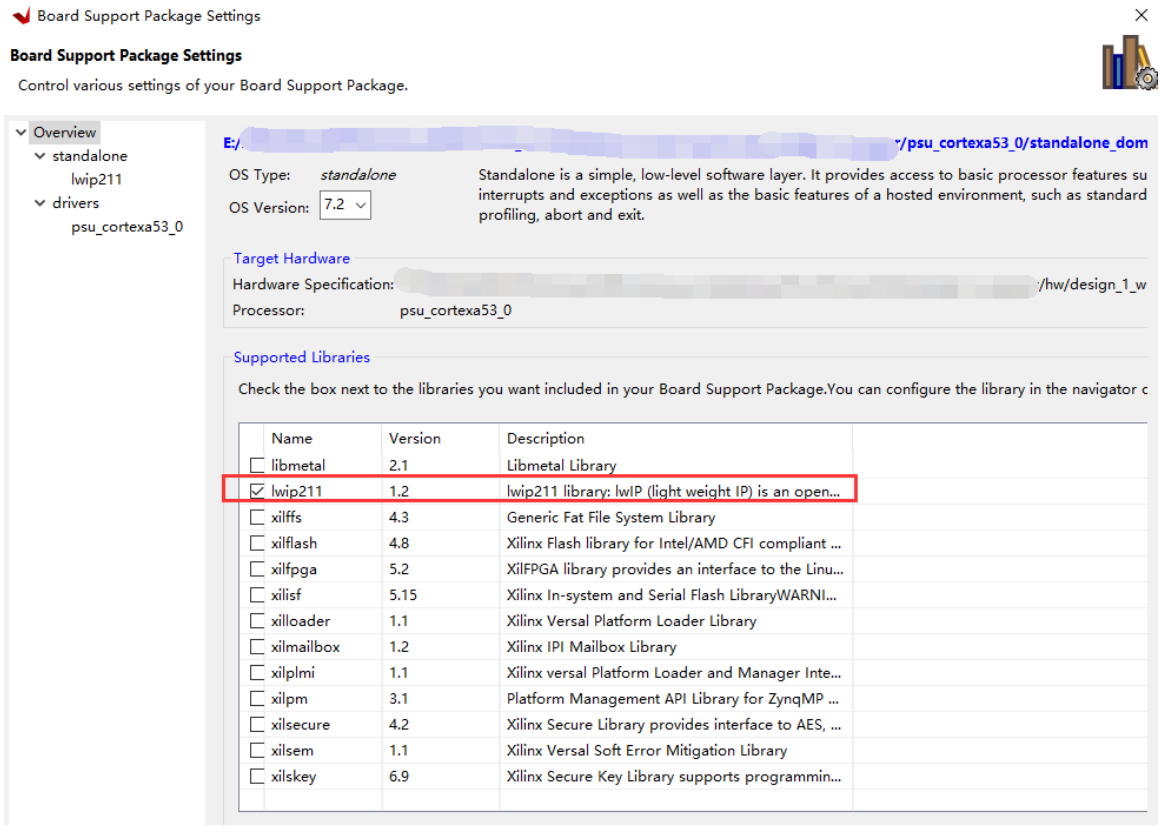
14.1 Vitis 程序开发

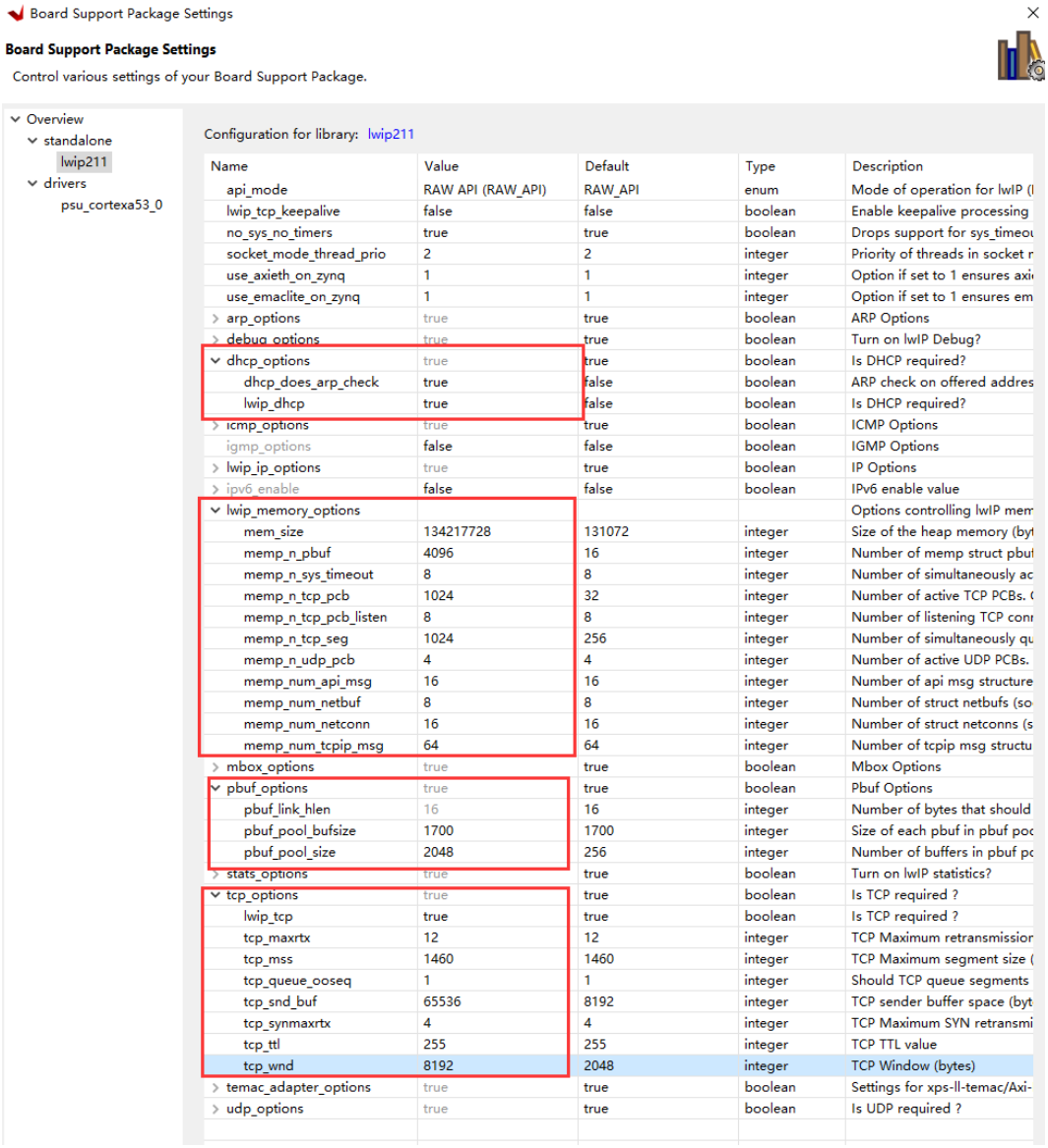
14.1.1 UDP 传输方式

1. LWIP 部分主要处理 BIN 文件的接收，程序为 lwip_app.c



2. 在创建工程后，需要使能 lwip 库，并进行设置，使能 DHCP 功能，将 pbuf 空间尽可能设置大一些，增大缓存空间，提高效率。


Board Support Package Settings
Board Support Package Settings
Control various settings of your Board Support Package.
Overview
standalone
lwip211
drivers
psu_cortexa53_0
E:/
/psu_cortexa53_0/standalone_dom
OS Type: standalone
OS Version: 7.2
Standalone is a simple, low-level software layer. It provides access to basic processor features su
interrupts and exceptions as well as the basic features of a hosted environment, such as standard
profiling, abort and exit.
Target Hardware
Hardware Specification:
/hw/design_1_w
Processor: psu_cortexa53_0
Supported Libraries
Check the box next to the libraries you want included in your Board Support Package.You can configure the library in the navigator c
Name
Version
Description
libmetal 2.1 Libmetal Library
lwip211 1.2 lwip211 library: lwIP (light weight IP) is an open...
xilffs 4.3 Generic Fat File System Library
xilflash 4.8 Xilinx Flash library for Intel/AMD CFI compliant ...
xilfpga 5.2 XilFPGA library provides an interface to the Linu...
xilisf 5.15 Xilinx In-system and Serial Flash LibraryWARNI...
xilloader 1.1 Xilinx Versal Platform Loader Library
xilmailbox 1.2 Xilinx IPI Mailbox Library
xilplmi 1.1 Xilinx versal Platform Loader and Manager Inte...
xilpm 3.1 Platform Management API Library for ZynqMP ...
xilsecure 4.2 Xilinx Secure Library provides interface to AES, ...
xilsem 1.1 Xilinx Versal Soft Error Mitigation Library
xilskey 6.9 Xilinx Secure Key Library supports programmin...



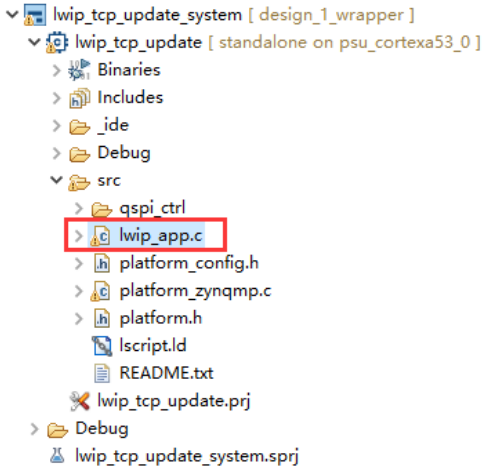
3. udp_receive 函数为设置的接收回调函数，主要功能是接收数据，并将接收到的数据缓存到 FlashRxBuffer 空间，留待更新 Flash 使用，在发送数据后，再发送“update”命令，开始更新 flash，在函数中判断此命令。

4. 在 while 循环语句中，判断 StartUpdate 变量值，更新 Flash。

```
while (1)
{
    xemacif_input(echo_netif);
    if (StartUpdate)
    {
        Status = update_qspi(&QspiInstance, QSPI_DEVICE_ID, ReceivedCount, FlashRxBuffer) ;
        if (Status != XST_SUCCESS)
            xil_printf("Write Flash Error!\r\n") ;
        else
        {
            StartUpdate = 0 ;
            ReceivedCount = 0;
        }
    }
}
```

14.1.2 TCP 传输方式

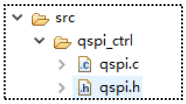
1. TCP 的 LWIP 部分同样也是 lwip_app.c 文件，控制部分参考 lwip echo server 例程，建立一个 TCP Server



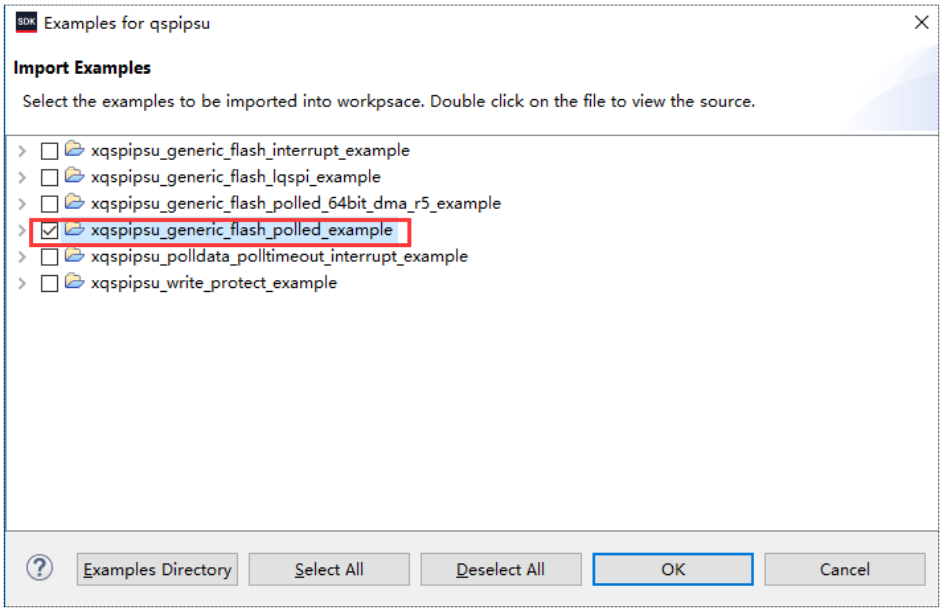
2. 与 UDP 类似，在 recv_callback 接收回调函数中，缓存接收到的 BIN 文件，启动更新命令同样是 update，其他部分也与 UDP 类似。

14.1.3 QSPI Flash 读写控制

UDP 和 TCP 两种方式使用的是同样的 QSPI 读写文件 qspi.c 和 qspi.h



1. qspi.c 文件是根据 xqspipsu_generic_flash_polled_example 做的修改



2. 主要有以下一些函数，写使能及关闭，flash 擦除，flash 写，flash 读，读 Flash ID 等。

```
/************************ Function Prototypes ****************************/


int update_qspi(XQspiPsu *QspiPsuInstancePtr, u16 QspiPsuDeviceId, unsigned int TotoalLen, char *FlashDataToSend) ;

int FlashReadID(XQspiPsu *QspiPsuPtr);
int FlashErase(XQspiPsu *QspiPsuPtr, u32 Address, u32 ByteCount, u8 *WriteBfrPtr);
int FlashWrite(XQspiPsu *QspiPsuPtr, u32 Address, u32 ByteCount, u8 Command,
        u8 *WriteBfrPtr);
int FlashRead(XQspiPsu *QspiPsuPtr, u32 Address, u32 ByteCount, u8 Command,
        u8 *WriteBfrPtr, u8 *ReadBfrPtr);
u32 GetRealAddr(XQspiPsu *QspiPsuPtr, u32 Address);
int BulkErase(XQspiPsu *QspiPsuPtr, u8 *WriteBfrPtr);
int DieErase(XQspiPsu *QspiPsuPtr, u8 *WriteBfrPtr);
int FlashEnterExit4BAddMode(XQspiPsu *QspiPsuPtr,unsigned int Enable);
int FlashEnableQuadMode(XQspiPsu *QspiPsuPtr);


void print_percent(int percent) ;
```

3. 主要的函数为 update_qspi，其中 TotalLen 为要更新的总字节数，FlashDataToSend 为存放更新数据的缓存区域，流程也比较简单，首先是擦除，在这里没选择擦除整个 Flash，而是根据 TotalLen 大小进行 Sector 擦除，因此擦除的空间会比 TotalLen 稍微大一点；然后是写 Flash，利用 FlashWrite 函数进行写入；最后是校验，从 Flash 里读出数据，并与写入的数据进行对比。

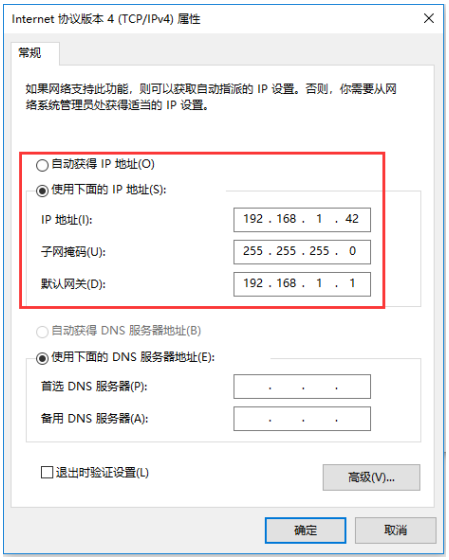
```
int update_qspi(XQspiPsu *QspiPsuInstancePtr, u16 QspiPsuDeviceId, unsigned int TotoalLen, char *FlashDataToSend)
```

14.2 板上验证

我们可以选择其他例程的 BOOT.bin 文件。我们是设定网络环境理想状态下做的实验，在做此实验时，不要打开其他有关以太网传输的上位机软件，由于端口号一样，可能会造成冲突。

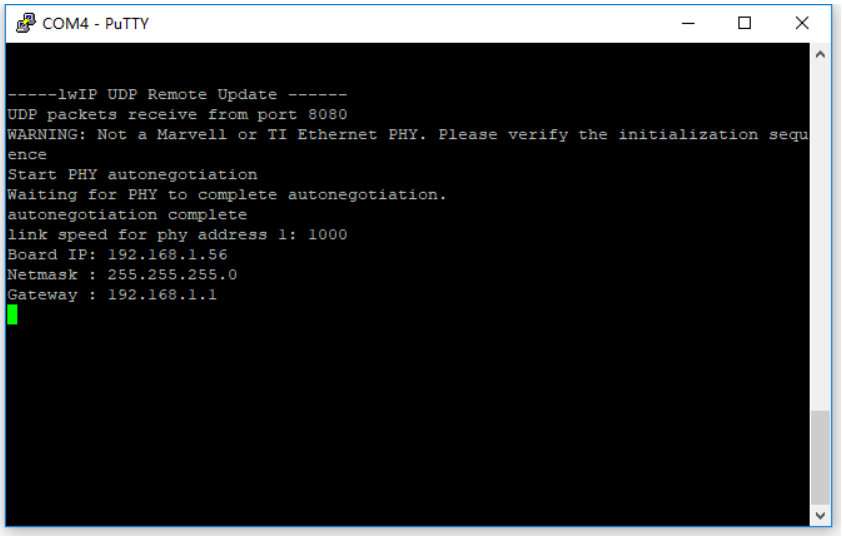
1. 首先连接开发板，将网线连接到 PS 端网口 PS_ETH

2. 如果有 DHCP 服务器，会自动分配 IP 给开发板；如果没有 DHCP 服务器，默认开发板 IP 地址为 192.168.1.10，需要将 PC 的 IP 地址设为同一网段，如下图所示。同时要确保网络里没有 192.168.1.10 的 IP 地址，否则会造成 IP 冲突，导致无法显示。可以在板子未上电前在 CMD 里输入 ping 192.168.1.10 查看是否能 ping 通，如果 ping 通，说明网络中有此 IP 地址，就无法验证。
   没有问题之后打开串口软件。

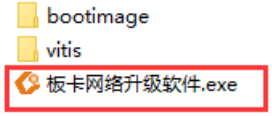


14.2.1 UDP 方式

1. 下载程序，可以看串口里的信息



2. 打开工程目录下的板卡网络升级软件



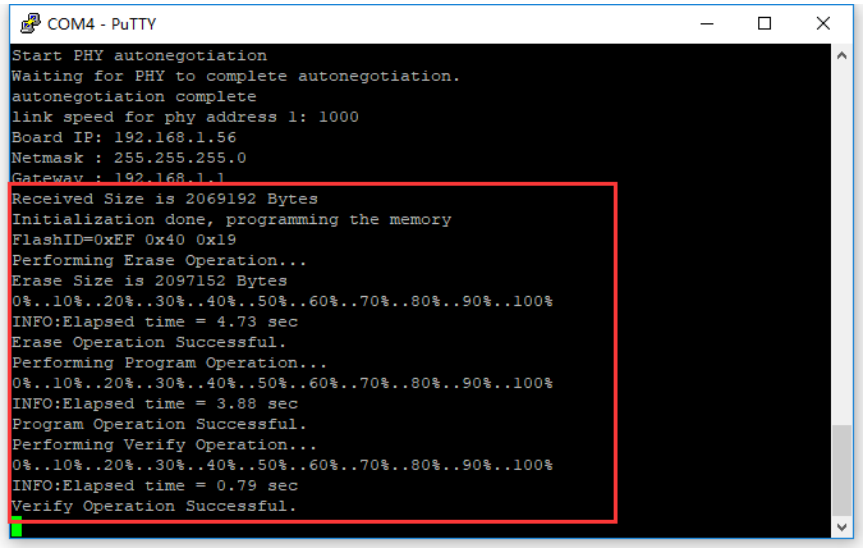
3. 填入板卡的 IP 地址和端口号，选择 UDP 发送方式，选择 BOOT.bin 文件，点击发送



4. 发送完毕后，会显示发送的字节数



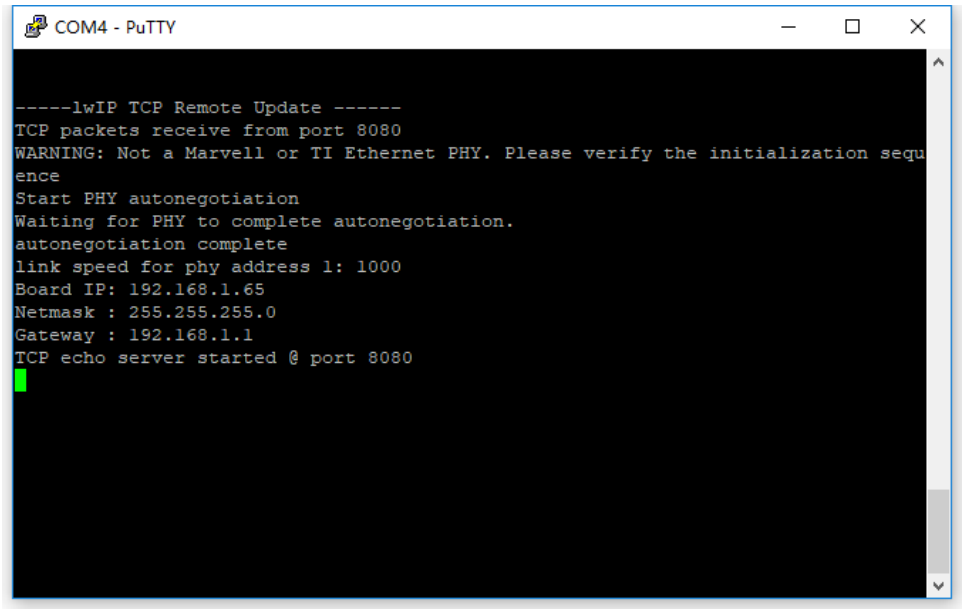
5. 在串口窗口可以看到板卡接收到的字节数，以及擦除，烧写，校验过程。



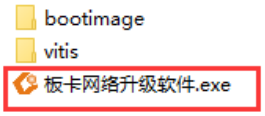
6. 断电通拨码开关选择 QSPI 启动方式，打开电源启动，即可看到程序运行起来。

14.2.2 TCP 方式

1. 下载程序，可以看到 putty 信息



2. 打开工程目录下的板卡网络升级软件



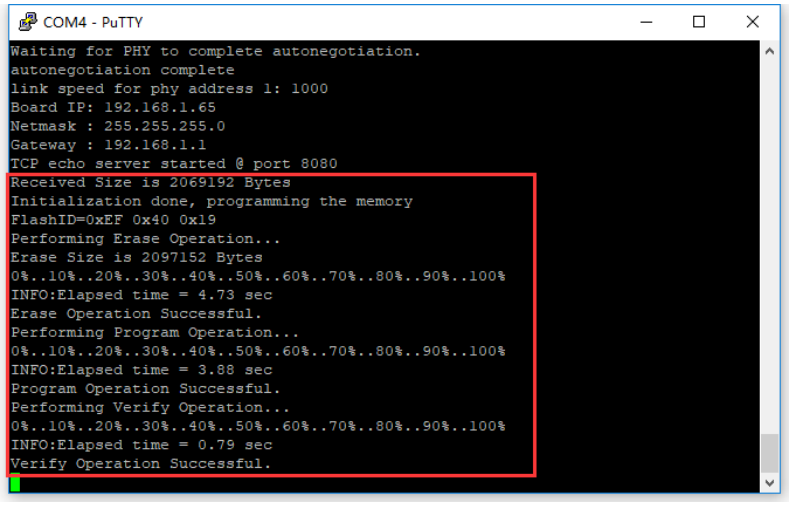
3. 填入 IP 地址和端口号，选择 TCP 发送方式，选择 BOOT.bin 文件，点击发送



4. 与 UDP 一样，也能看到发送的字节数



5. 在串口窗口可以看到板卡接收到的字节数，以及擦除，烧写，校验过程。



6. 断电通过拨码开关选择 QSPI 启动方式，打开电源启动，即可看到程序运行起来。

第十五章 System Monitor 的使用

vivado 工程目录为“ps_hello/vivado”

vitis 工程目录为“ps_sysmon/vitis”

本章介绍 system monitors 的使用，用于监测芯片的电压、温度值等，也可以通过 PL 端的 ADC 引脚作为外部信号的采集。PL 端可以做 17 路 ADC 的采集，但开发板并没有在这些管脚上接设备，因此本章不做讲解。如图所示电压传感器可监测芯片的 VCCINT,VCCAUX,VCCBRAM 等，PL_SYSMON 的 VP_0 和 VN_0 为一对专用的 ADC 模拟输入口。VAUXP[*]和 VAUXN[*]也是 ADC 输入口，但是不用作 ADC 输入口时，可用作普通 IO 使用。本实验主要测量温度以及电压的值。

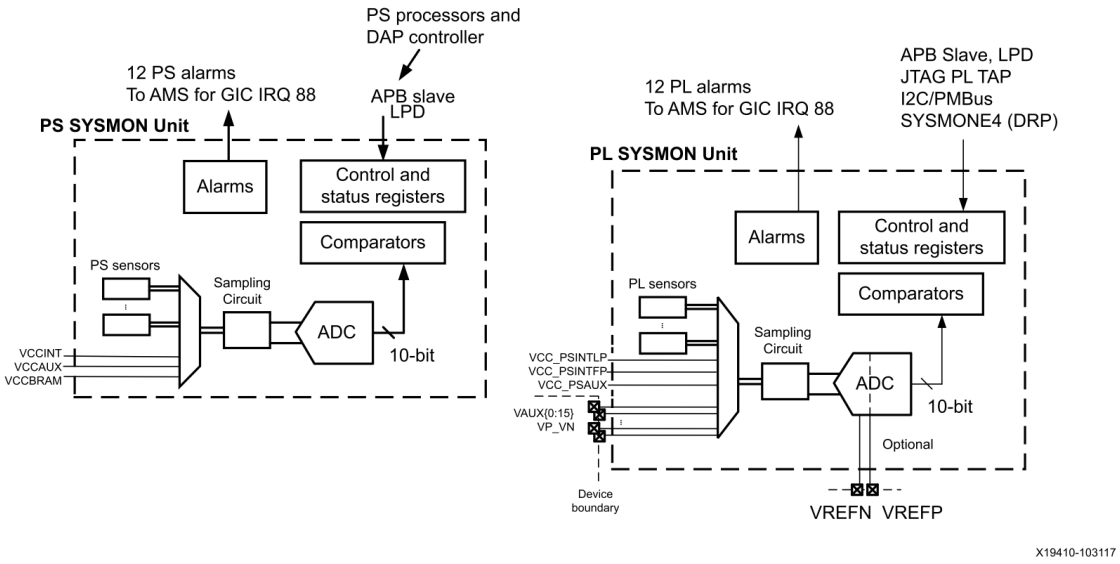


Figure 9-1: **PS SYSMON and PL SYSMON Block Diagram**

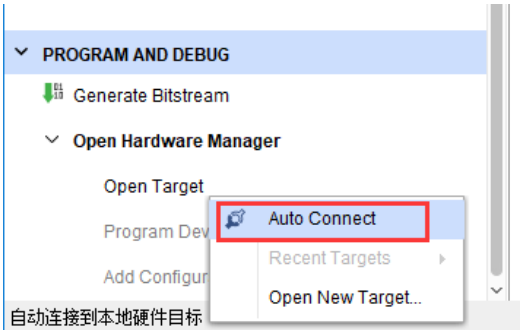
Vivado 工程，同样以“ps_hello”工程为基础。

FPGA 工程师工作内容

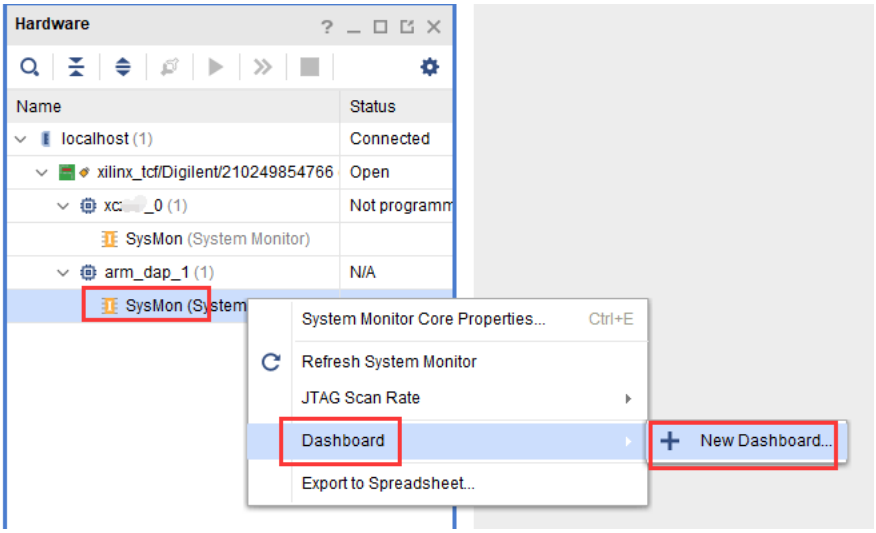
下面介绍 FPGA 工程师负责内容。

15.1 Hardware 读取 System Monitor

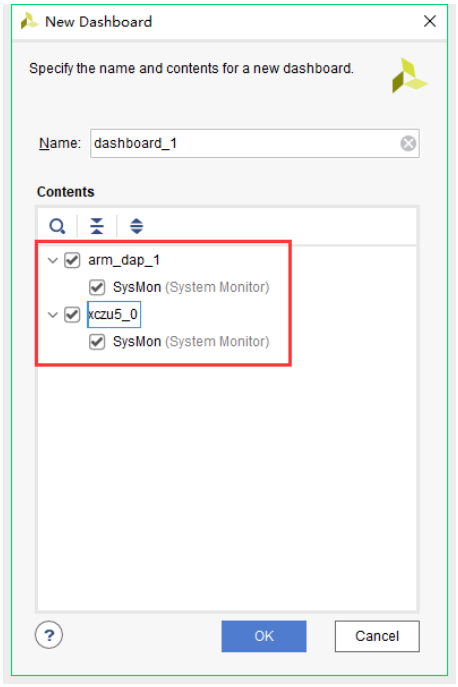
1. 打开工程，连接好开发板电源，JTAG 下载器，并将开发板调整为 JTAG 模式，开发板上电，点击 Open Hardware Manager，再点击 Auto Connect，发现硬件。



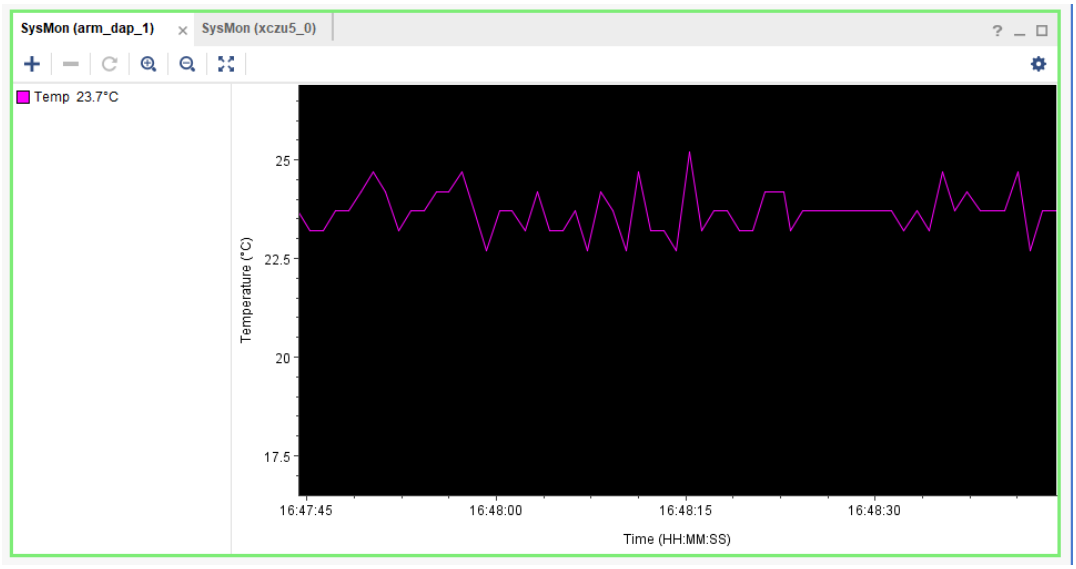
2. 右键选中 SysMon，新建 Dashboard



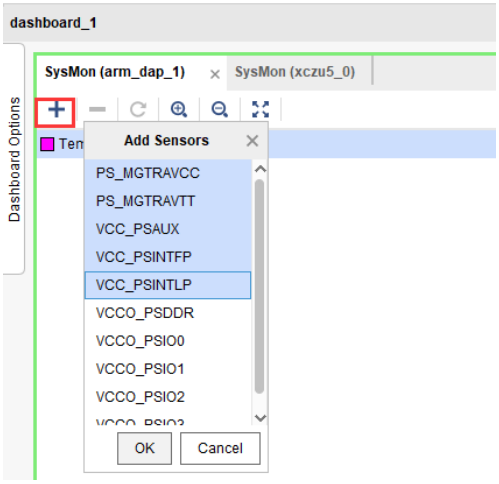
3. PS 端和 PL 端都选择，点击 OK



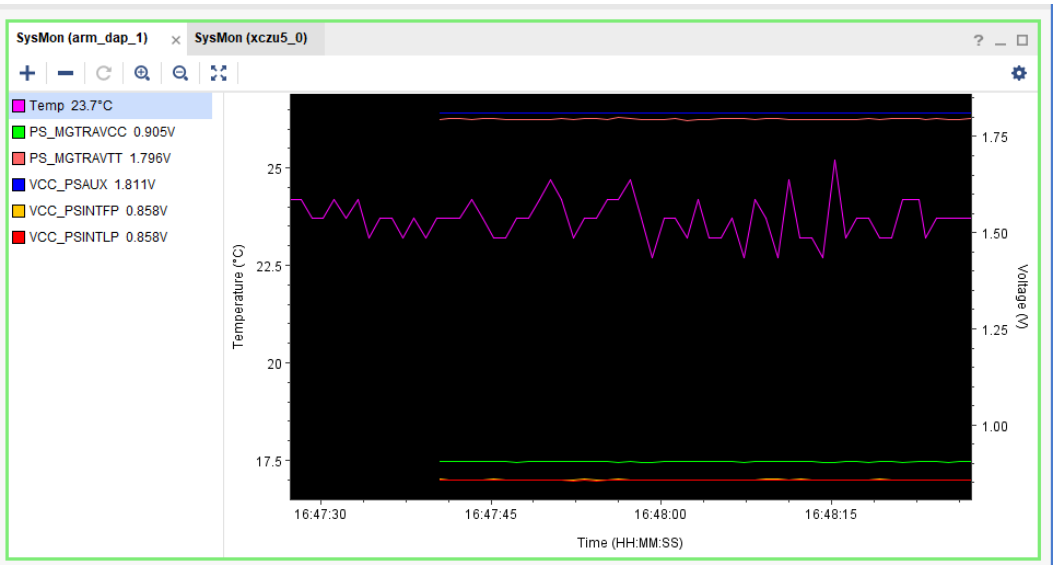
4. 默认会有温度信息



5. 点击+将电压值添加到窗口



6. 显示如下



此方法优点是图形化显示，较为直观，但缺点是无法得到数据值。下面介绍 PS 读取 XADC 信

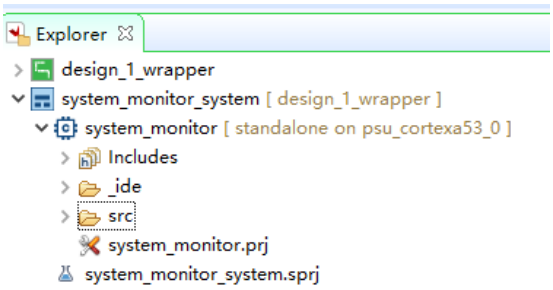
息。

软件工程师工作内容

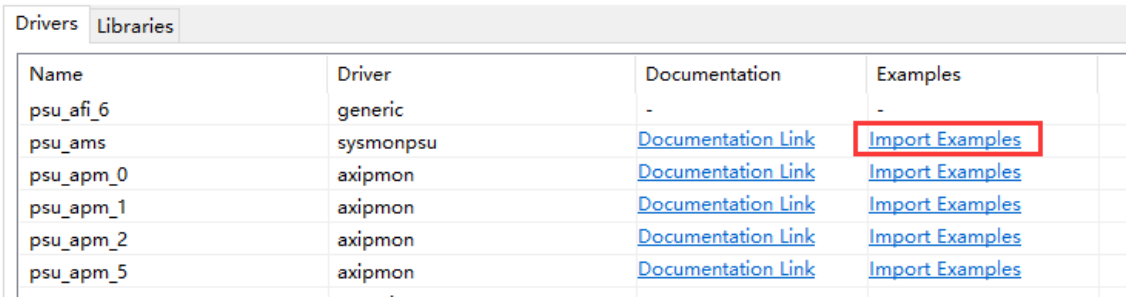
以下为软件工程师负责内容。

15.2 PS 读取 System Monitor 信息

1. 打开 Vitis 软件，新建 Vitis 工程



2. 可以在 BSP 中看到 system monitor，可以导入 Example 学习



3. 此实验现象为读取温度和电压的数据，并每隔 1S 通过串口打印出来。通过 XSysMonPsu_GetAdcData 函数读取原始值，用 XSysMonPsu_RawToTemperature_OnChip 宏将 ADC 值转换为温度值。用 XSysMonPsu_RawToVoltage 转换为电压值。

```
while(1)
{
    /*
     * Read the on-chip Temperature Data (Current/Maximum/Minimum)
     * from the ADC data registers.
     */
    TempRawData = XSysMonPsu_GetAdcData(SysMonInstPtr, XSM_CH_TEMP, XSYSMON_PS);
    TempData = XSysMonPsu_RawToTemperature_OnChip(TempRawData);
    printf("\r\nThe Current Temperature is %0d.%03d Centigrades.\r\n",
            (int)(TempData), SysMonPsuFractionToInt(TempData));

    /*
     * Read the VccInt Votage Data (Current/Maximum/Minimum) from the
     * ADC data registers.
     */
    VccIntRawData = XSysMonPsu_GetAdcData(SysMonInstPtr, XSM_CH_SUPPLY1, XSYSMON_PS);
    VccIntData = XSysMonPsu_RawToVoltage(VccIntRawData);
    printf("\r\nThe Current VCCINT is %0d.%03d Volts. \r\n",
            (int)(VccIntData), SysMonPsuFractionToInt(VccIntData));

    /*
     * Read the VccAux Votage Data (Current/Maximum/Minimum) from the
     * ADC data registers.
     */
    VccAuxRawData = XSysMonPsu_GetAdcData(SysMonInstPtr, XSM_CH_SUPPLY3, XSYSMON_PS);
    VccAuxData = XSysMonPsu_RawToVoltage(VccAuxRawData);
    printf("\r\nThe Current VCCAUX is %0d.%03d Volts. \r\n",
            (int)(VccAuxData), SysMonPsuFractionToInt(VccAuxData));

    sleep(1) ;
}
```

4. 下载后在串口工具中可看到打印信息如下，读到温度，VCCINT，VCCAUX 的值

```
The Current Temperature is 28.811 Centigrades.

The Current VCCINT is 0.849 Volts.

The Current VCCAUX is 1.814 Volts.

The Current Temperature is 29.028 Centigrades.

The Current VCCINT is 0.849 Volts.

The Current VCCAUX is 1.812 Volts.
```

PS 与 PL 互联篇

在 PS 端外设篇中，介绍了 PS 端外设的应用，在此基础上，我们学习 PS 与 PL 之前的互联，这也是 ZYNQ 应用的重点，介绍比如图像显示，摄像头图像采集，ADC 数据采集显示，数据采集及以太网传输等应用。

第十六章 PS 端 EMIO 的使用

实验 Vivado 工程目录为“ps_emio/vivado”。
实验 vitis 工程目录为“ps_emio/vitis”。

前面介绍了 PS 端点亮 LED 灯的实验，但是如果想用 PS 点亮 PL 的 LED 灯，该如何做呢？一是可以通过 EMIO 控制 PL 端 LED 灯，二是通过 AXI GPIO 的 IP 实现控制。本章介绍如何使用 EMIO 控制 PL 端 LED 灯的亮灭。同时也介绍了，利用 EMIO 连接 PL 端按键控制 PL 端 LED 灯。

16.1 原理介绍

前面介绍了 PS 端 MIO 的结构如下，从图中可知 BANK0~BANK2 的 MIO 有 78 个。BANK3~BANK5 的 EMIO 有 96 个，本章就是采用 EMIO 控制 PL 端 LED。

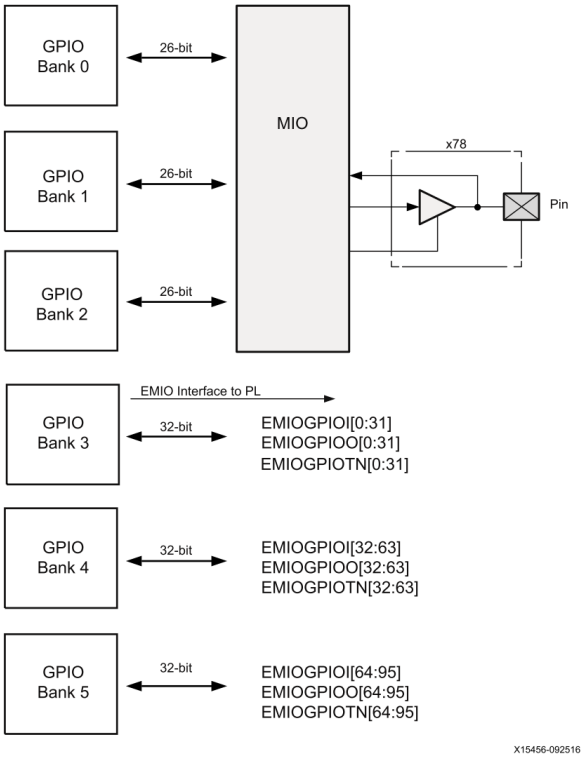


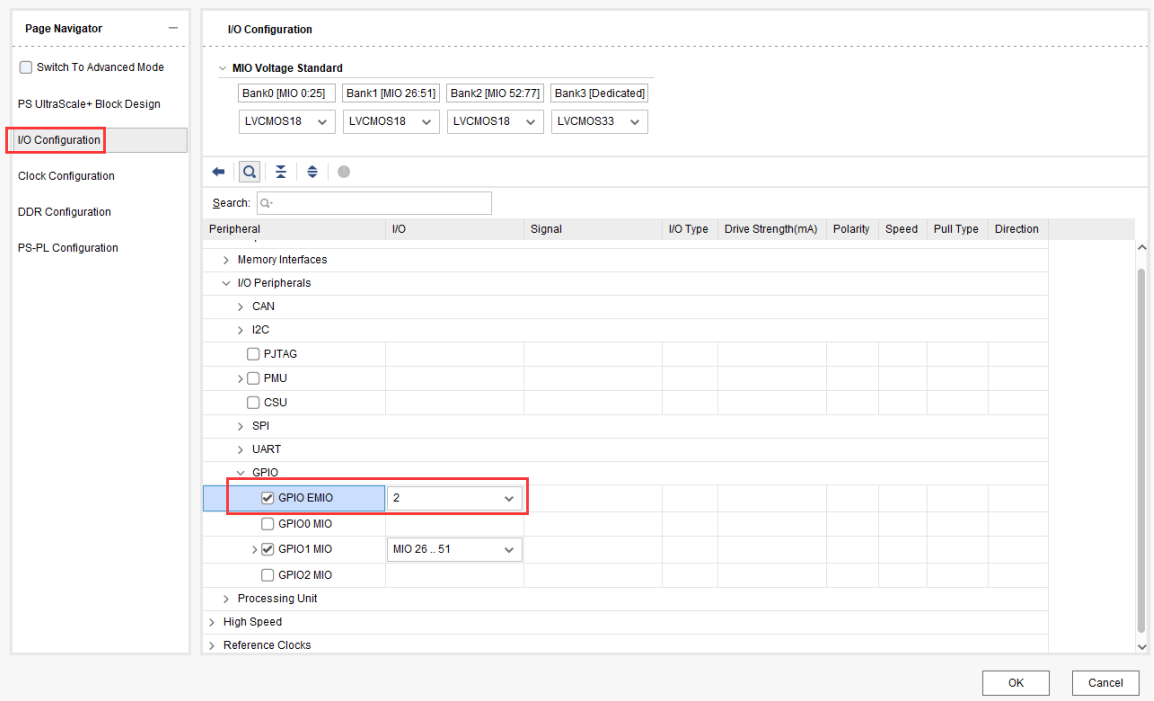
Figure 27-1: **GPIO Block Diagram**

FPGA 工程师工作内容

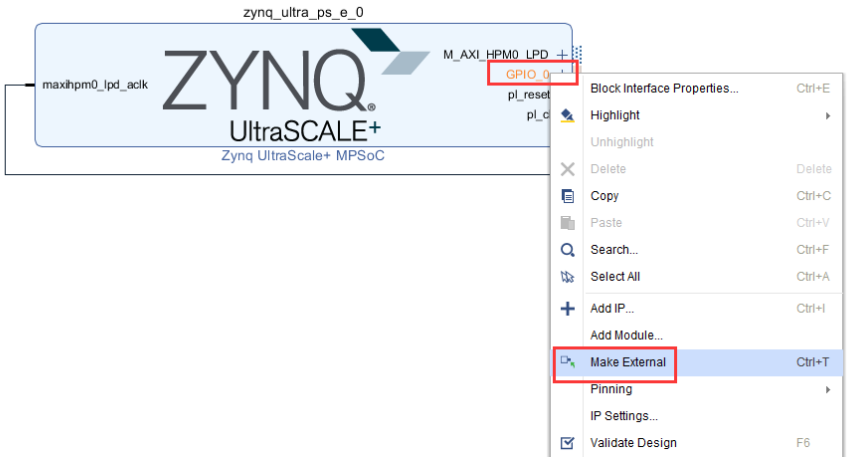
以下为 FPGA 工程师负责内容。

16.2 Vivado 工程建立

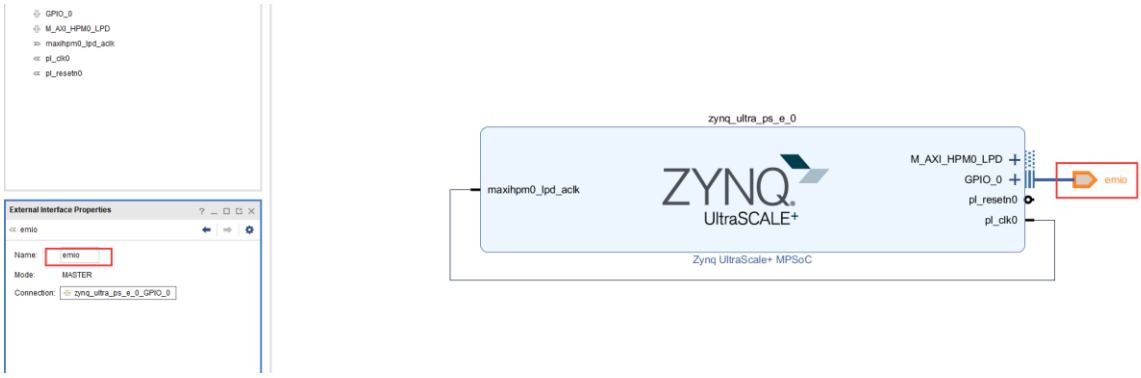
1) 以 ps_hello 工程为基础，另存为一个名为 ps_emio 的工程，打开 ZYNQ 配置，把 GPIO EMIO 勾选上。由于 PL 端的 1 个 LED，1 个按键，因此在 MIO 配置中选择 EMIO 的位宽为 2 位，配置结束，点击 OK。



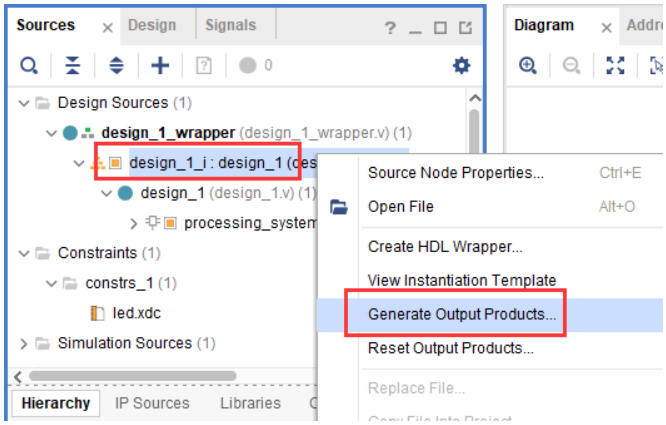
2) 点击多出的 GPIO_0 端口右键选择 Make External，将端口信号导出



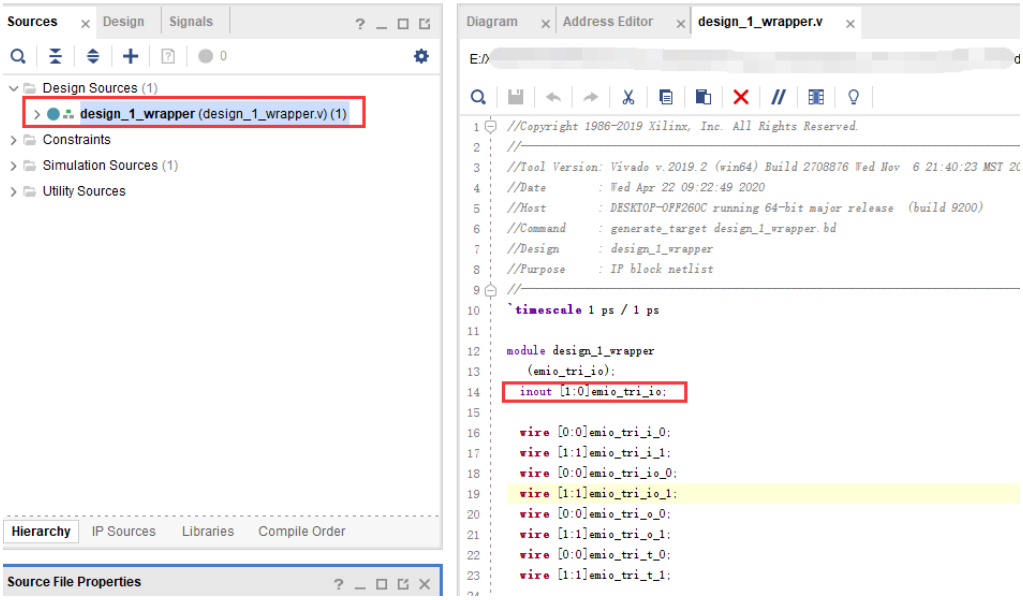
3) 点击引脚并修改引脚名称为 emio，也可以根据自己需求修改名称。保存设计。



4） 点击 xx.bd 右键选择 Generate Output Products，重新生成输出文件

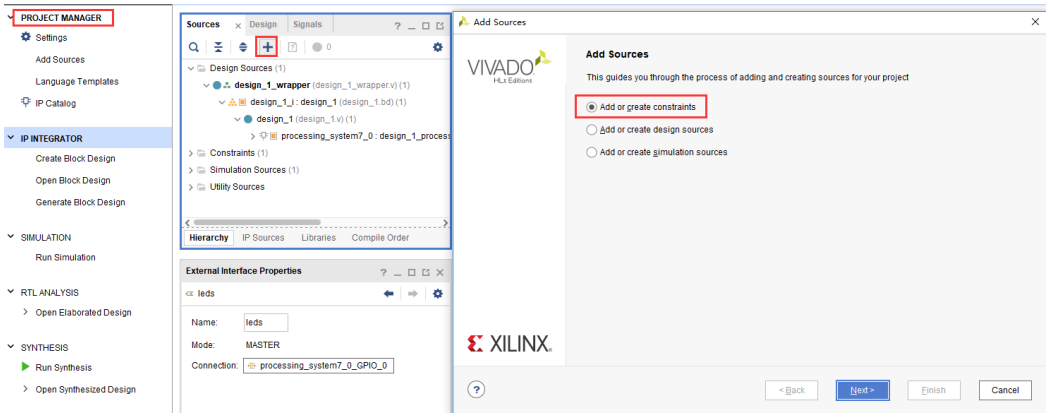


5） 结束后，顶层文件会更新出新的管脚，下面需要对其进行引脚绑定

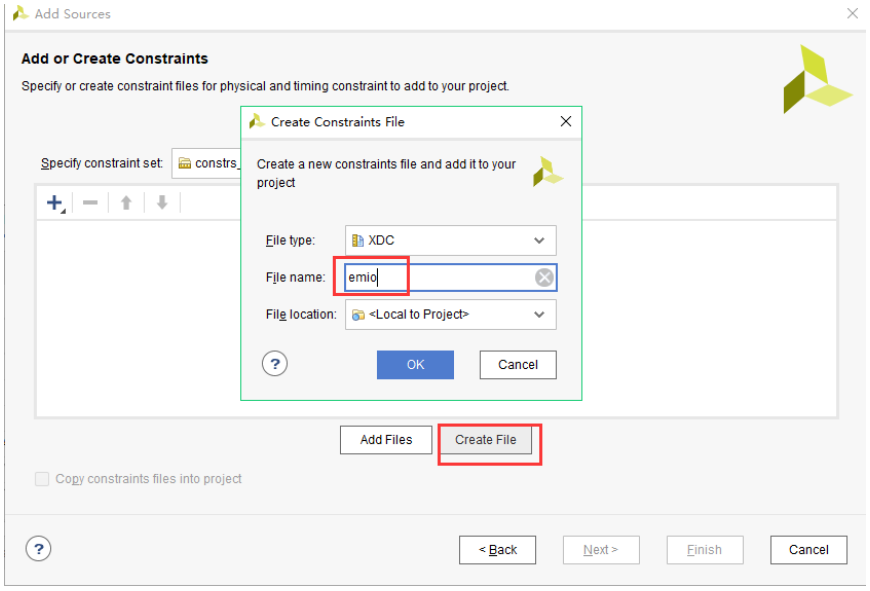


16.3 XDC 文件约束 PL 管脚

6） 新建 XDC 文件，绑定 PL 端引脚



设置文件名称为 emio



7） emio.xdc 添加一下内容，端口名称一定要和顶层文件端口一致

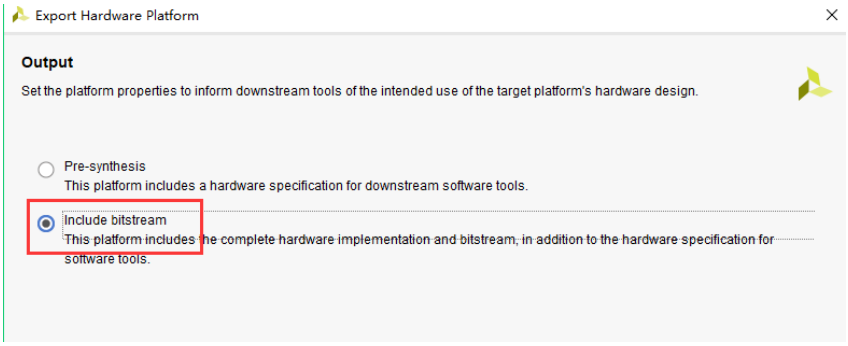
```
####################Compress Bitstream#################################
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]

set_property IOSTANDARD LVCMOS33 [get_ports {emio_tri_io[*]}]
#pl led
set_property PACKAGE_PIN K9 [get_ports {emio_tri_io[0]}]
#pl key
set_property PACKAGE_PIN J9 [get_ports {emio_tri_io[1]}]
```

8） 生成 bit 文件



9） 虽然说没有用到 PL 端的逻辑，但是用到了 PL 端的引脚，因此导出硬件要选择“Include bitstream”



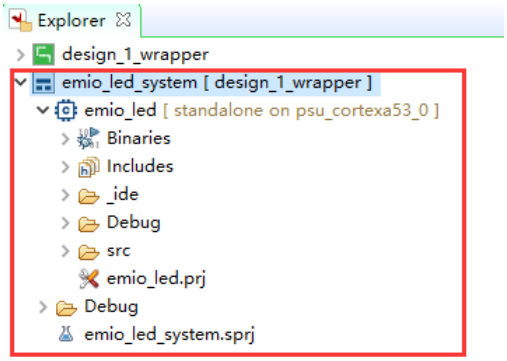
软件工程师工作内容

以下为软件工程师负责内容。

16.4 Vitis 程序编写

16.4.1 EMIO 点亮 PL 端 LED 灯

1) 进入 Vitis 软件，新建名为 emio_led 的工程



2) 代码部分与 PS 端 MIO 操作点亮 LED 类似，由于 MIO 的编号是 0~77，因此 EMIO 的编号是从 78 开始的，只要做以下修改即可

```c
int main()
{
    init_platform();

    int Status;
    XGpioPs_Config *ConfigPtr;

    print("Hello World\n\r");
    /* Initialize the GPIO driver. */
    ConfigPtr = XGpioPs_LookupConfig(GPIO_DEVICE_ID);

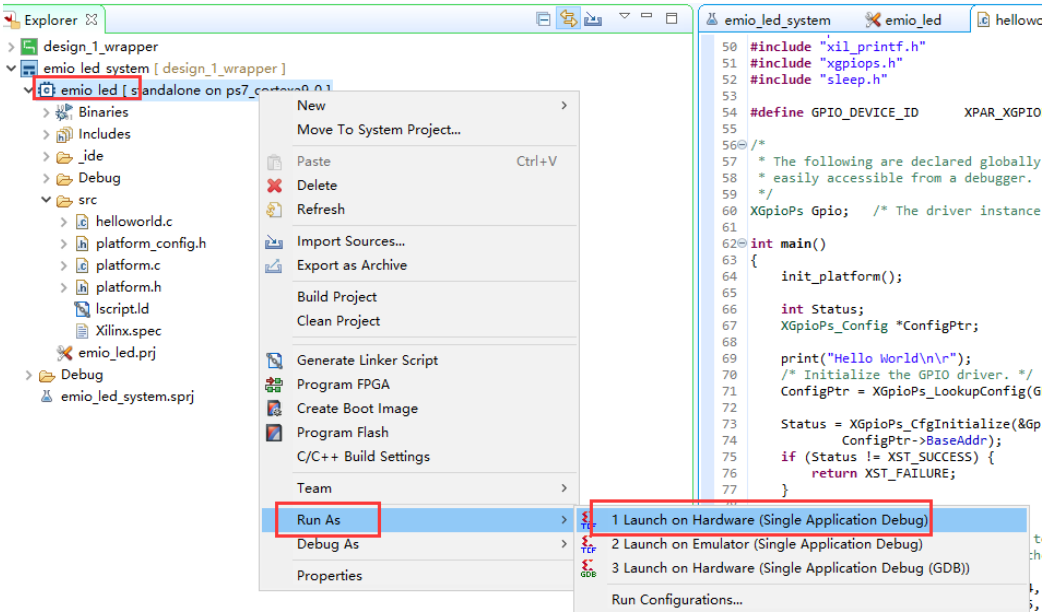
    Status = XGpioPs_CfgInitialize(&Gpio, ConfigPtr,
            ConfigPtr->BaseAddr);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }

    /*
     * Set the direction for the pin to be output and
     * Enable the Output enable for the LED Pin.
     */
    XGpioPs_SetDirectionPin(&Gpio, 78, 1);
    XGpioPs_SetOutputEnablePin(&Gpio, 78, 1);

    while(1){
        /* Set the GPIO output to be low. */
        XGpioPs_WritePin(&Gpio, 78, 0x0);
        sleep(1) ;
        /* Set the GPIO output to be high. */
        XGpioPs_WritePin(&Gpio, 78, 0x1);
        sleep(1) ;
    }

    cleanup_platform();
    return 0;
}
```

3) 编译并下载

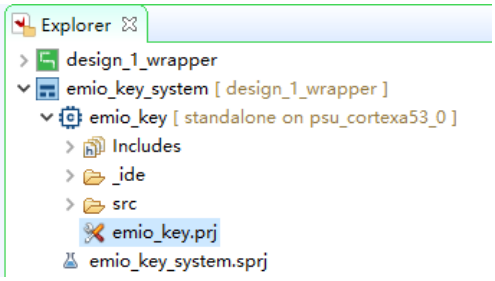


即可看到 PL 端 LED 闪烁。

16.4.2 EMIO 实现 PL 端按键中断

通过 PL 端的按键控制 PL 端 LED 灯的亮灭

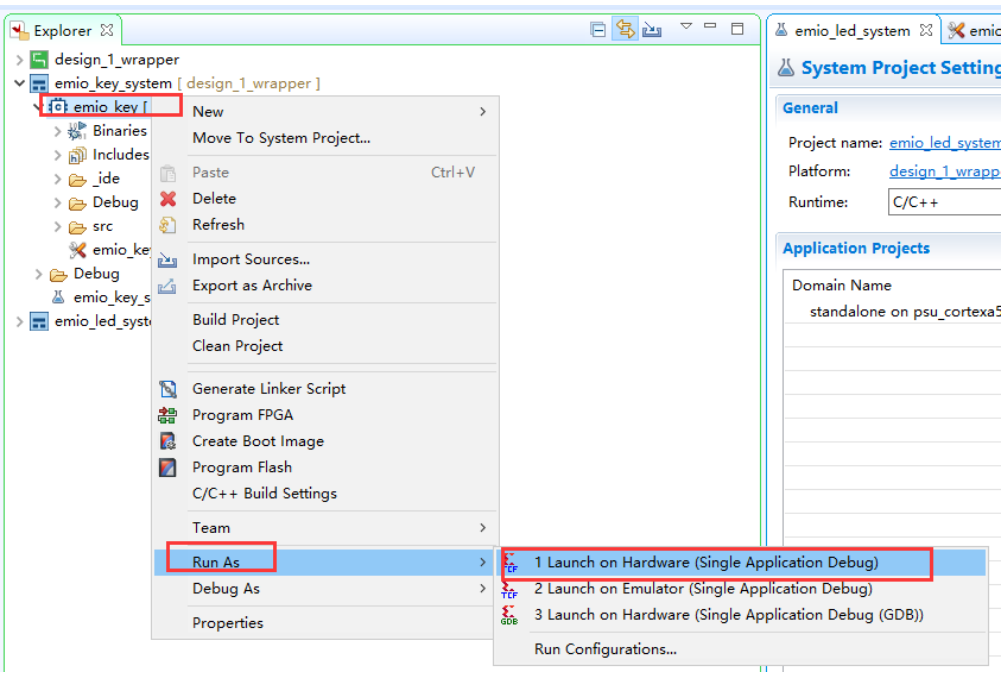
1) 新建名为 emio_key 的工程，模板为 hello world，拷贝例程的程序，保存并编译



1) 由 PS 端 MIO 的使用的 MIO 按键中断程序移植过来，并修改按键的编号为 58，LED 灯编号为 54，保存重新生成 elf。

```
/* GPIO paramter */
#define MIO_ID          XPAR_XGPIOPS_0_DEVICE_ID
#define INTC_DEVICE_ID  XPAR_SCUGIC_SINGLE_DEVICE_ID
#define KEY_INTR_ID     XPAR_XGPIOPS_0_INTR
#define PS_KEY_MIO      79
#define PS_LED_MIO      78
```

2) 下载程序

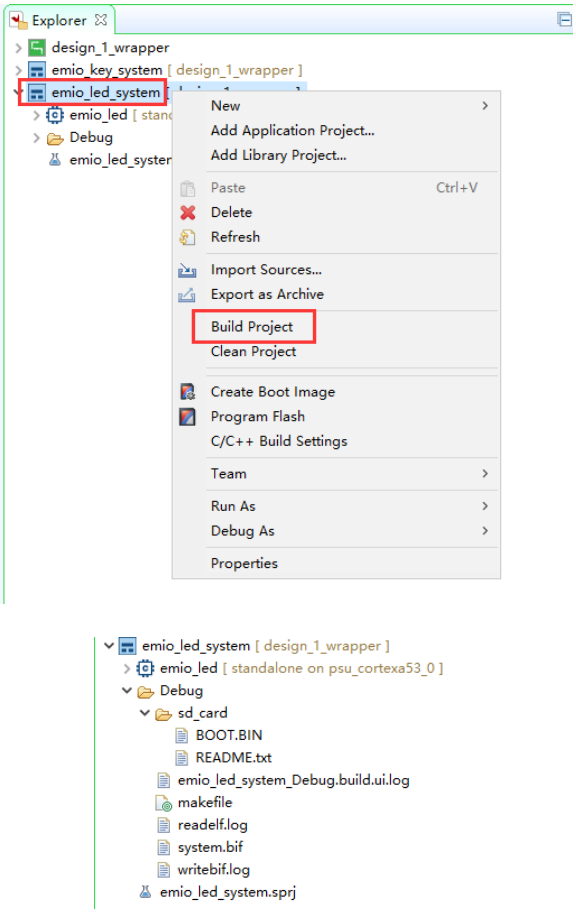


11. 观察实验现象，按下 PL 端按键 PL_KEY，就可以控制 PL 端 LED 灯 PL_LED 的亮灭。

16.5 固化程序

前面介绍过没有 FPGA 加载文件情况下如何生成固化程序（详情参考“体验 ARM，裸机输出”Hello World”一章）。本章内容生成了 FPGA 的加载文件，在这里演示一下如何生成固化程序。

与前面一样，也是点击 system，右键 Build Project 即可

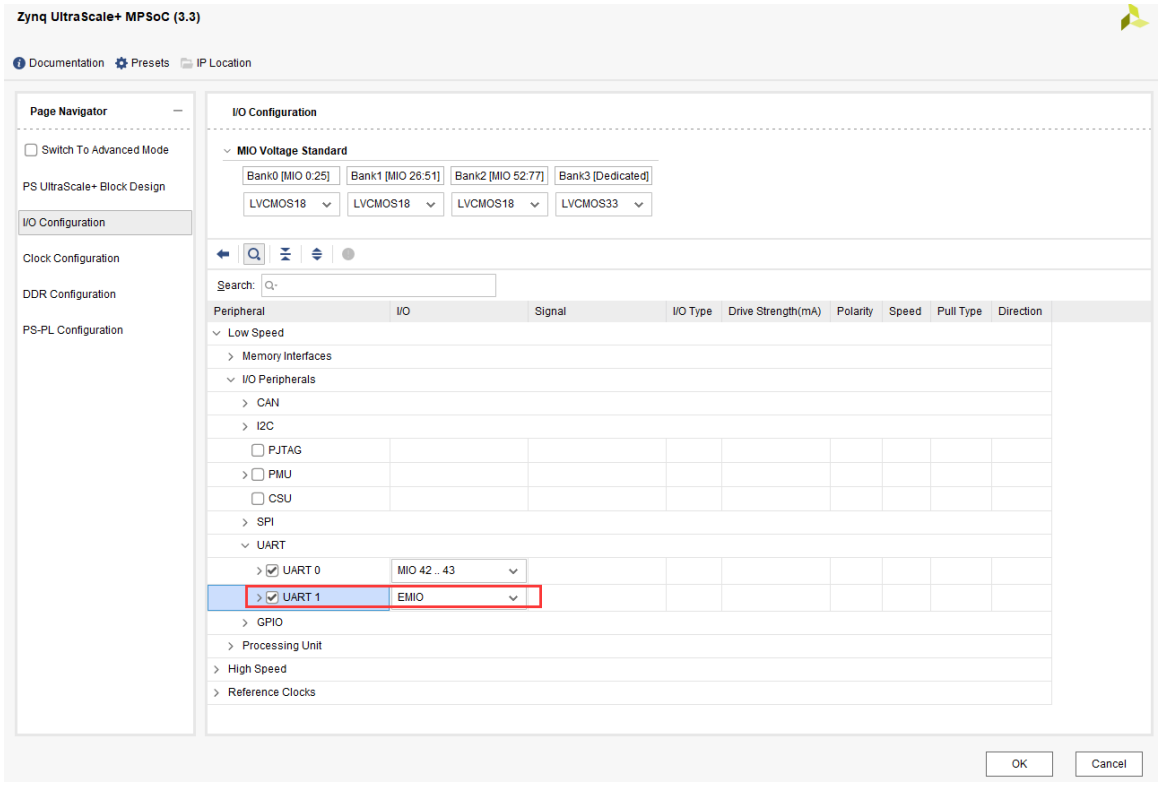


软件会自动添加三个文件，第一个引导程序 fsbl.elf，第二个为 FPGA 的 bitstream，第三个为应用程序 xx.elf，下载方法与前面一样，不再赘述。

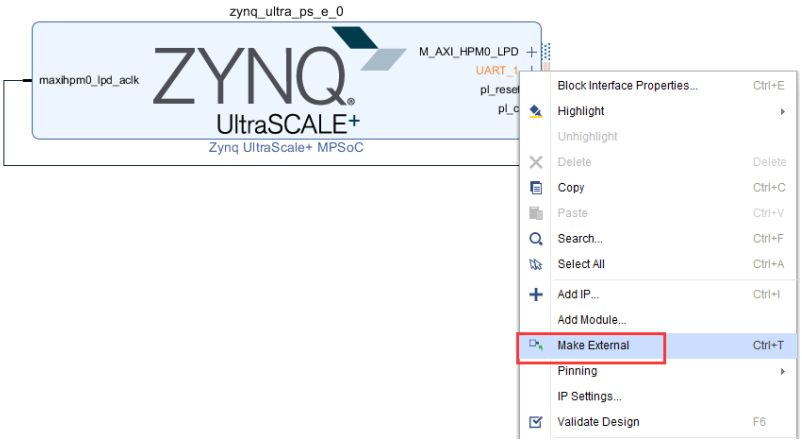
16.6 UART 串口的 EMIO 用法

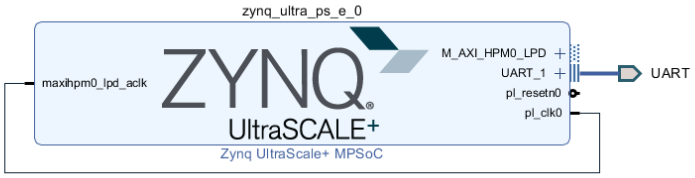
除了 GPIO 可以导出 EMIO，PS 端还有好多 MIO 是可以通过 EMIO 连接到 PL 端的，比如开发板在 PL 端连接了一个 UART，就可以通过这种方式连接。

在配置中，将 UART1 设置为 EMIO



导出引脚



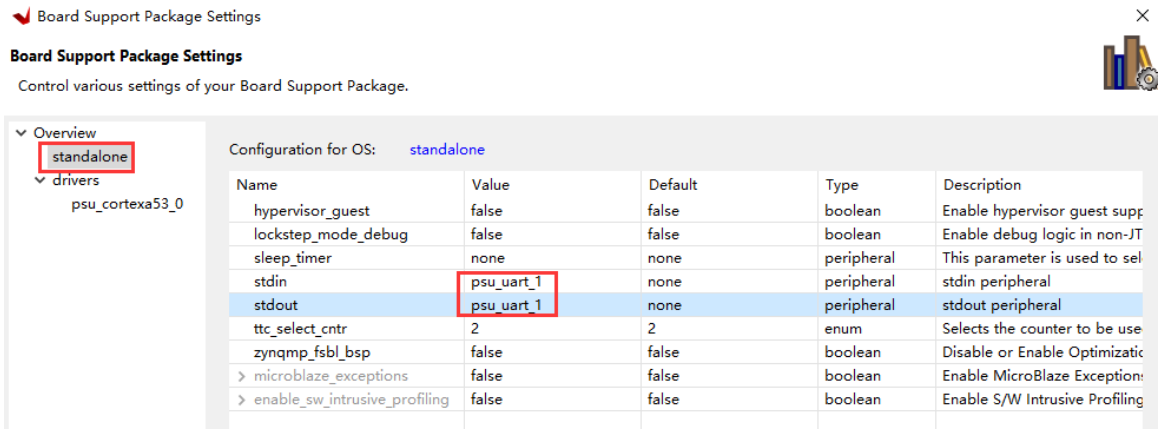


根据原理图绑定引脚，重新编译，即可。

```
set_property PACKAGE_PIN N8 [get_ports {uart_rxd}]
set_property IOSTANDARD LVCMOS33 [get_ports {uart_rxd}]

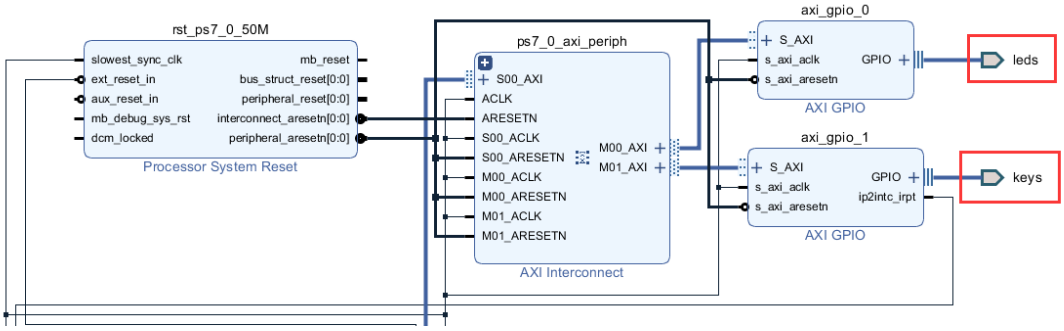
set_property PACKAGE_PIN N9 [get_ports {uart_txd}]
set_property IOSTANDARD LVCMOS33 [get_ports {uart_txd}]
```

用法和 uart0 一样，只不过如果想用 uart1 打印信息，需要在 BSP 中设置成 uart1

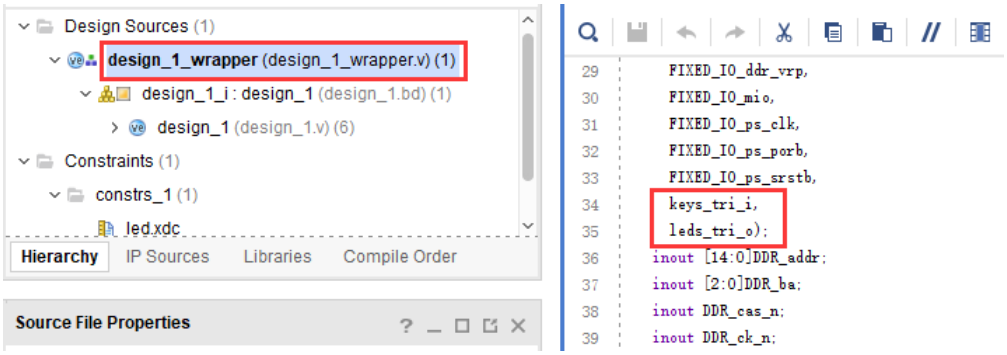


16.7 引脚绑定常见错误

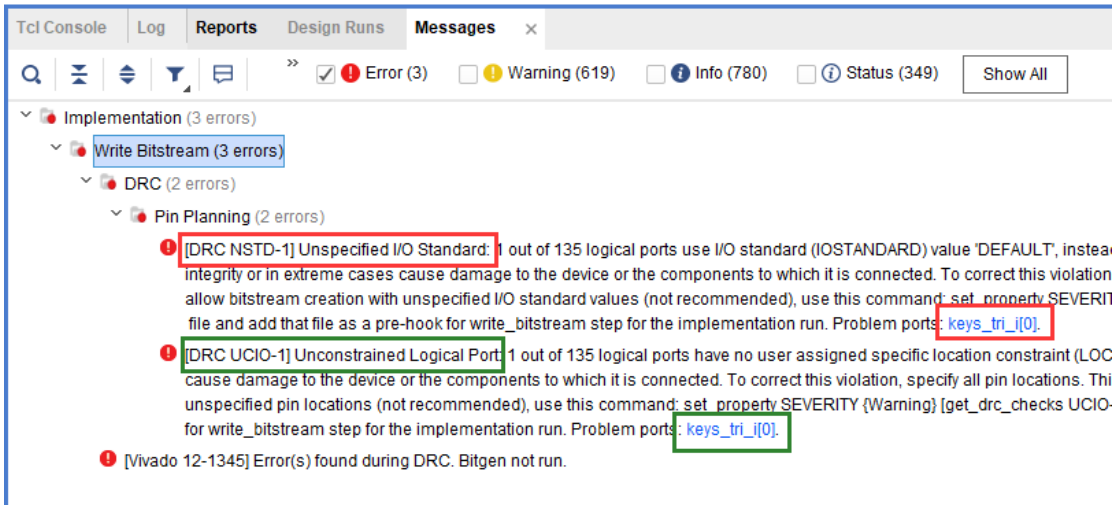
1) 在 block design 设计中，比如下图，GPIO 模块的引脚名设置为了 leds 和 keys，很多人想当然的在 XDC 里按照这样的名称绑定引脚。



如果打开顶层文件就会发现引脚名称是不一样的，一定要仔细检查，以顶层文件里的引脚名称为准。



否则就会出现以下引脚未绑定的错误



2） 如果是手写 XDC 文件，切记注意空格，这也是非常常见的错误

```
1  set_property IOSTANDARD LVCMOS33 [get_ports {leds_tri_o[3]}]
2  set_property IOSTANDARD LVCMOS33 [get_ports {leds_tri_o[2]}]
3  set_property IOSTANDARD LVCMOS33 [get_ports {leds_tri_o[1]}]
4  set_property IOSTANDARD LVCMOS33 [get_ports {leds_tri_o[0]}]
5  set_property PACKAGE_PIN M14 [get_ports {leds_tri_o[0]}]
6  set_property PACKAGE_PIN M15 [get_ports {leds_tri_o[1]}]
7  set_property PACKAGE_PIN K16 [get_ports {leds_tri_o[2]}]
8  set_property PACKAGE_PIN J16 [get_ports {leds_tri_o[3]}]
9
10 set_property IOSTANDARD LVCMOS33 [get_ports {keys_tri_i[0]}]
11 set_property PACKAGE_PIN N15 [get_ports {keys_tri_i[0]}]
```

16.8 本章小节

本章进一步学习了 PS 端的 EMIO 的使用，虽然将 EMIO 连接到了 PL 端的引脚上，但 Vitis 中的用法还是一样的，从这个例子我们也可以看出，一旦与 PL 端发生了联系，就需要生成 bitstream，虽然几乎没有产生逻辑。

第十七章 PL 端 AXI GPIO 的使用

实验 Vivado 工程目录为“ps_axi_gpio /vivado”。
实验 vitis 工程目录为“ps_axi_gpio /vitis”。

可能有些人就会问，怎么又在讲 GPIO，LED 灯，觉得太繁琐，但是 GPIO 是 ZYNQ 的基本操作，本教程力求把各种方法分享给大家，PS 端的 MIO，EMIO，PL 端的 axi gpio，包括输入输出两个方向，以及 PS 与 PL 的基本操作，所以还是希望大家耐心学习。

前面讲过如何用的是 PS 端的 EMIO 点亮 PL 端 LED 灯，但是并没有与 PL 端产生交互。本章介绍另外一种控制方法，在 ZYNQ 当中可以使用 AXI GPIO，通过 AXI 总线控制 PL 端的 LED 灯。同时也介绍了 PL 端按键的使用。

使用 zynq 最大的疑问就是如何把 PS 和 PL 结合起来使用，在其他的 SOC 芯片中一般都会有 GPIO，本实验使用一个 AXI GPIO 的 IP 核，让 PS 端通过 AXI 总线控制 PL 端的 LED 灯，实验虽然简单，不过可以让我们了解 PL 和 PS 是如何结合的。

17.1 原理介绍

一个 AXI GPIO 模块有两个 GPIO，分别是 GPIO 和 GPIO2，也就是 channel1 和 channel2，为双向 IO。

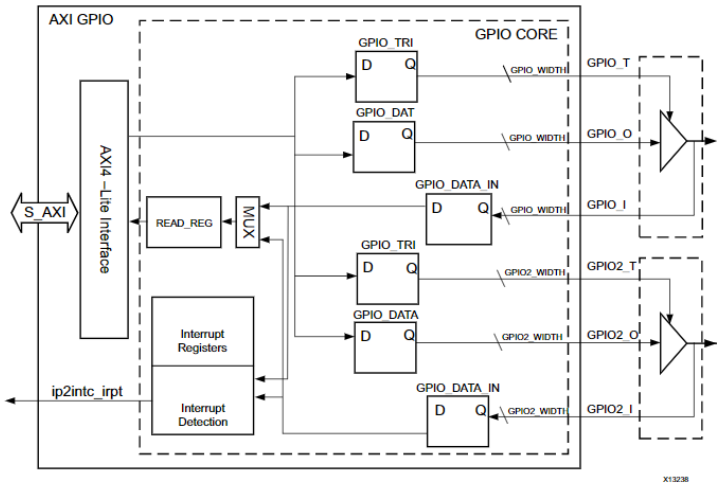


Figure 1-1: **AXI GPIO Block Diagram**

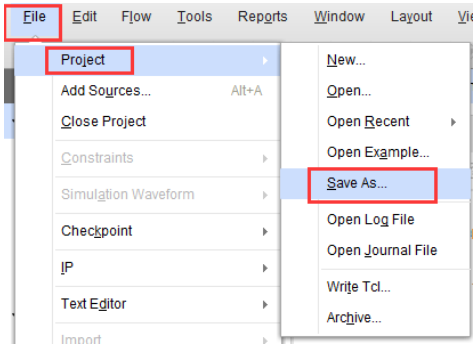
AXI GPIO 结构

FPGA 工程师工作内容

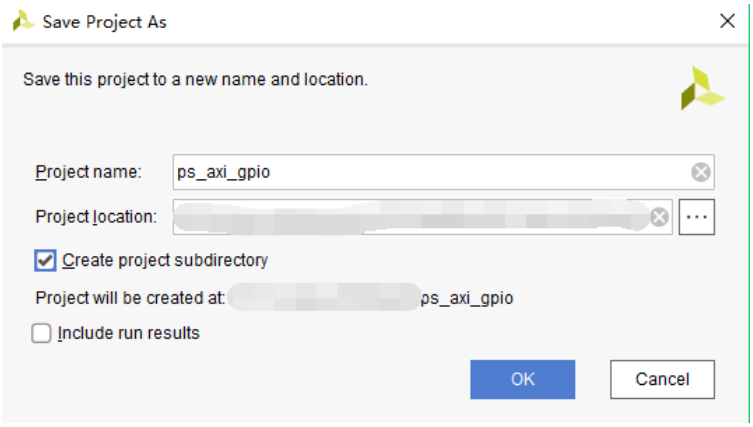
以下为 FPGA 工程师负责内容。

17.2 Vivado 工程建立

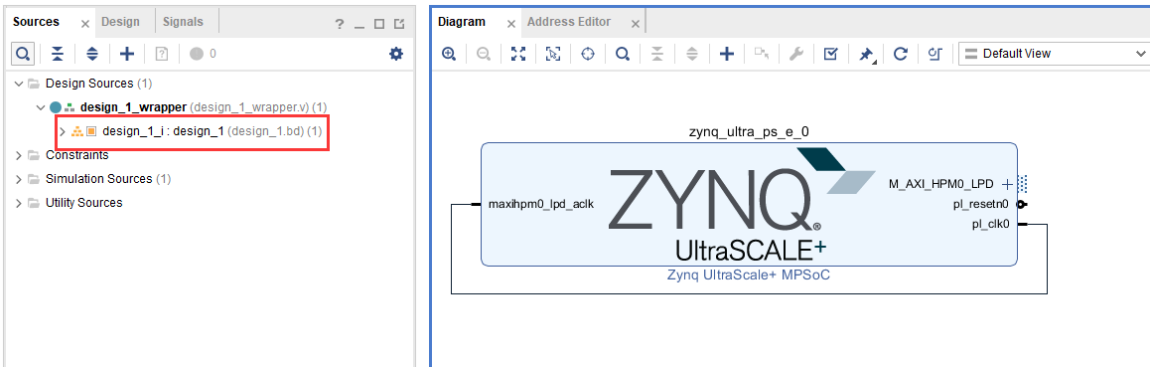
1) 打开“ps_hello”另存为一个名为“ps_axi_gpio”Vivado 工程，表示 PS 通过 AXI 总线控制 gpio



“Create project subdirectory”勾选后会在目录下创建子目录，勾选“Include run results”会包含编译后的结果

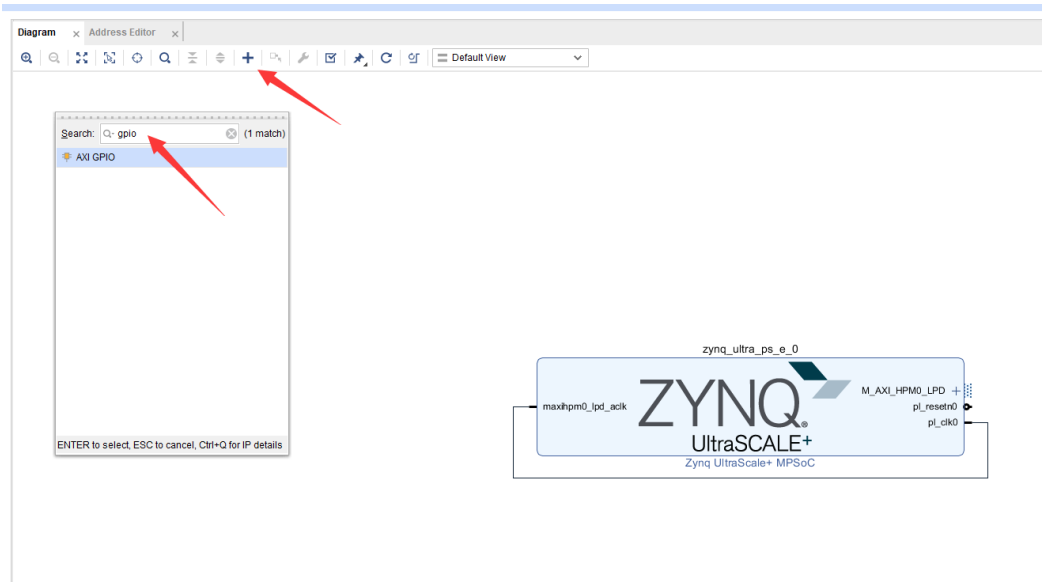


2) 双击 xx.bd 打开 block design

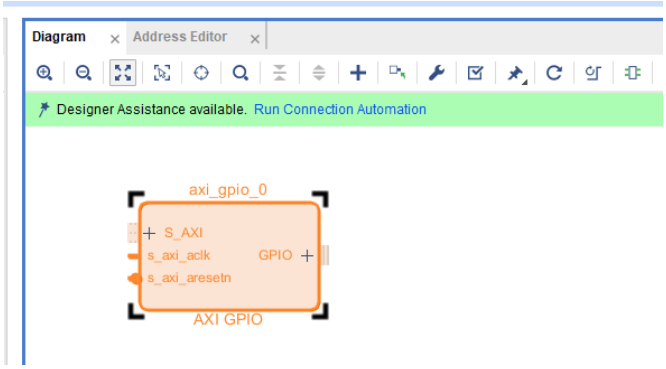


17.2.1 添加 AXI GPIO

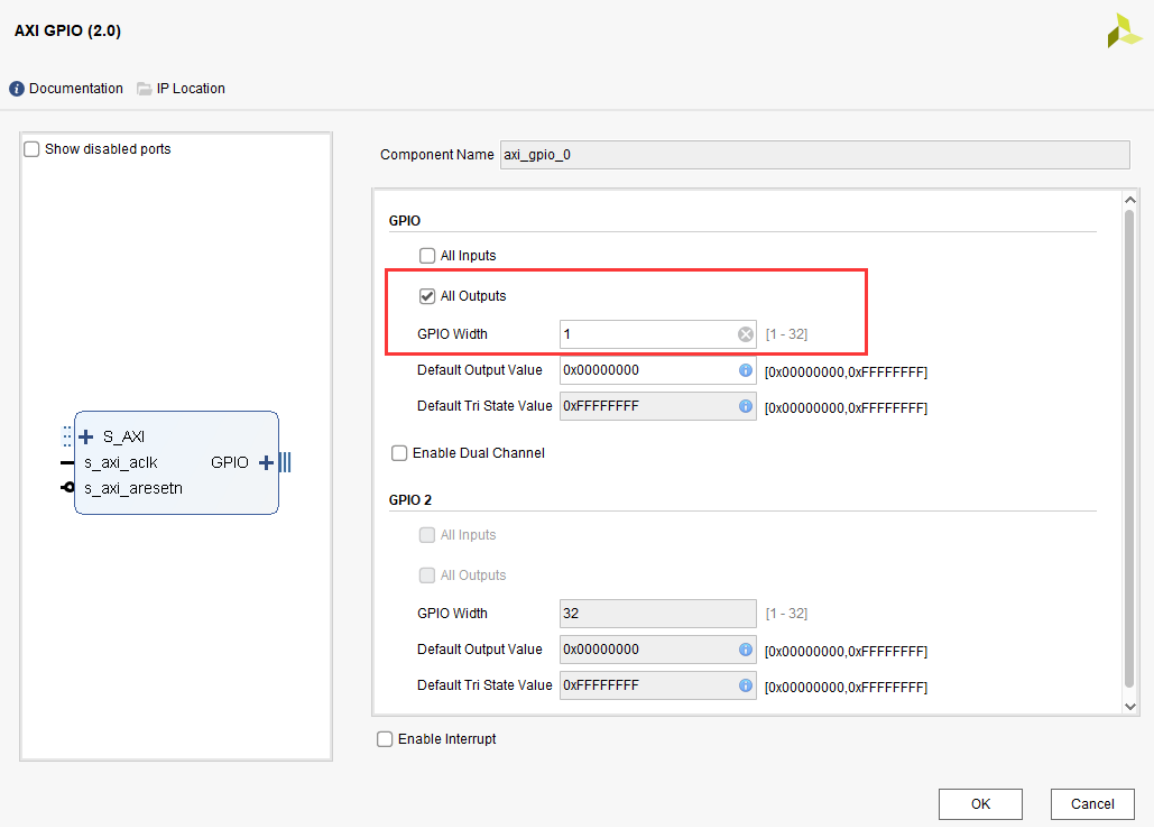
3) 添加一个 AXI GPIO 的 IP 核



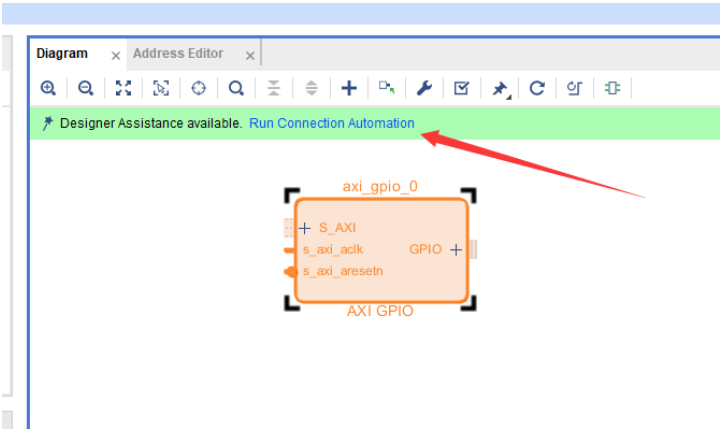
4) 双击刚才添加的“axi_gpio_0”配置参数



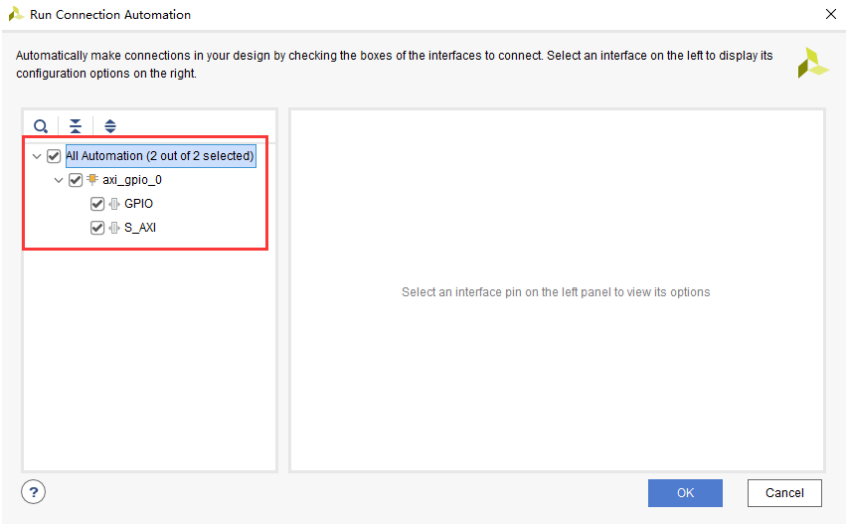
5) 选择“All Outputs”，因为这里控制 LED，只要输出就可以了，“GPIO Width”填 1，控制 1 颗 LED，点击 OK。如果想使用 channel2，需要把”Enable Dual Channel”打开，也就使能了 GPIO2。



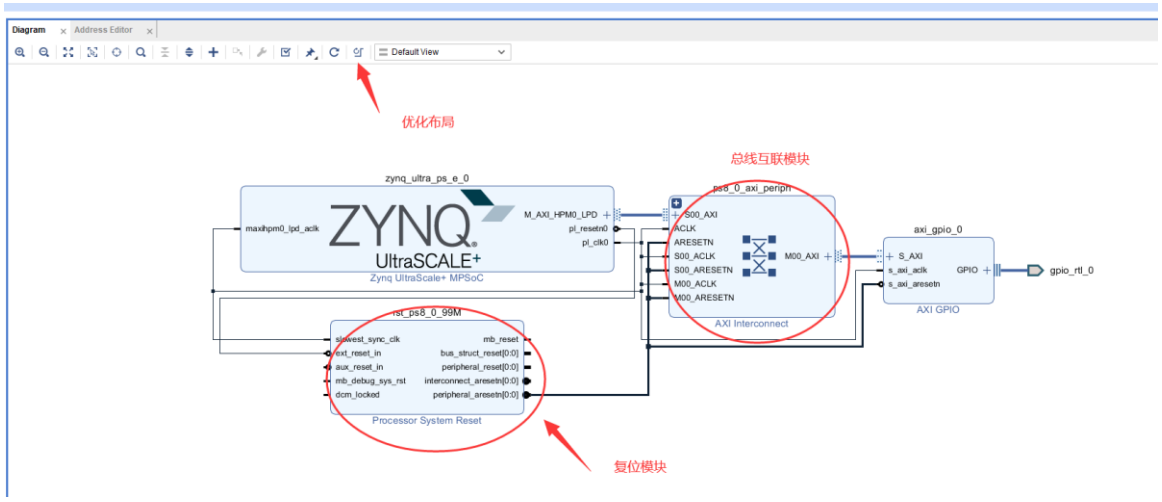
6) 点击“Run Connection Automation”，可以完成部分自动连线



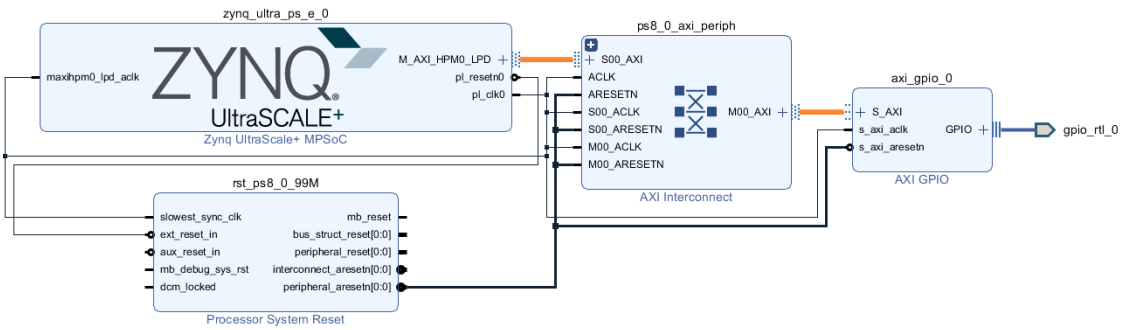
7) 选择要自动连接的端口，这里全选，点击 OK



8) 点击“Optimize Routing”，可以优化布局，同时可以看到多了两个模块，一个是 Processor System Reset 模块，为同步复位模块，提供同一时钟域的复位信号。AXI Interconnect 模块为 AXI 总线互联模块，用于 AXI 模块的交叉互联。

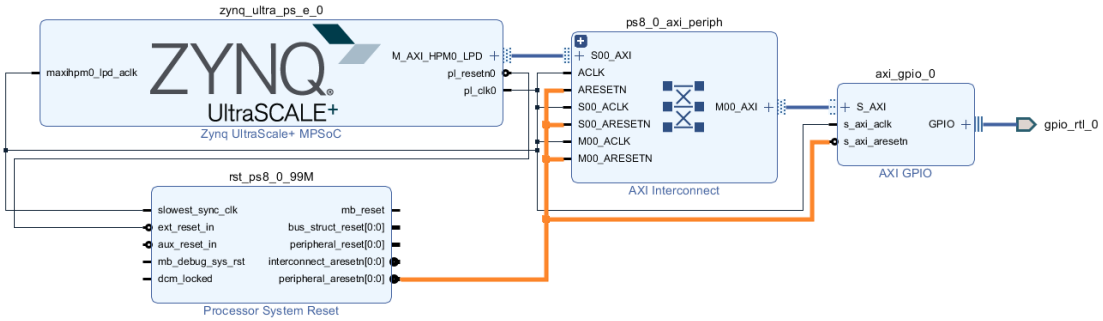


在这个应用中，我们可以看到用到了 ZYNQ 的 HPM0_LPD 口，此接口用于访问 PL 端数据，大部分应用中是为了配置 PL 端模块的寄存器。

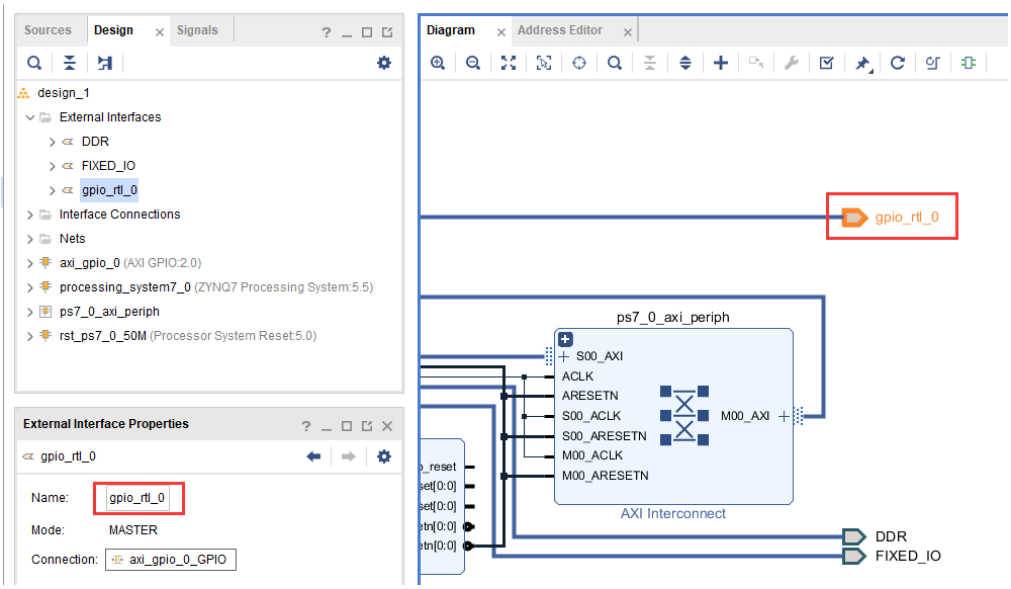


复位信号由 ZYNQ 的复位输出提供，最好是每个时钟域都加一个复位模块，可以根据模块

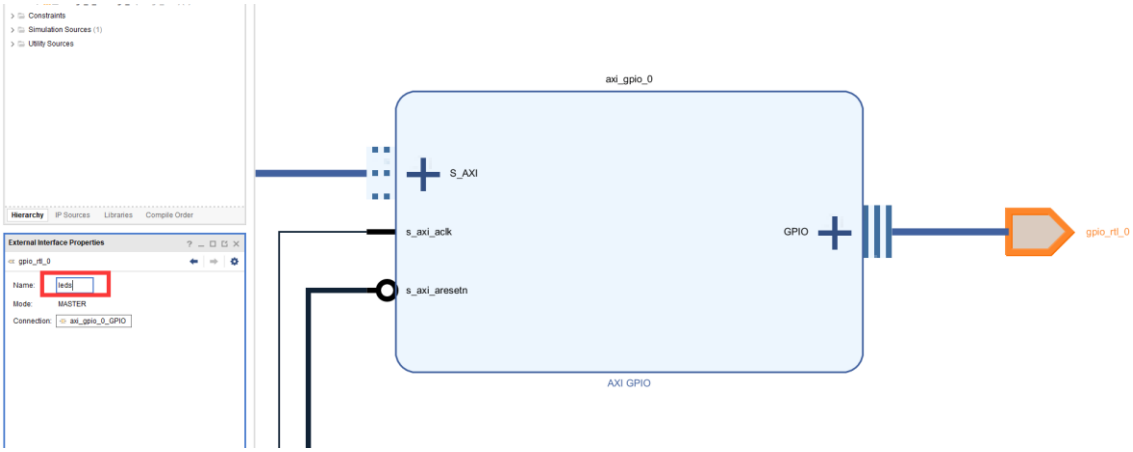
下面的名称搜索添加。



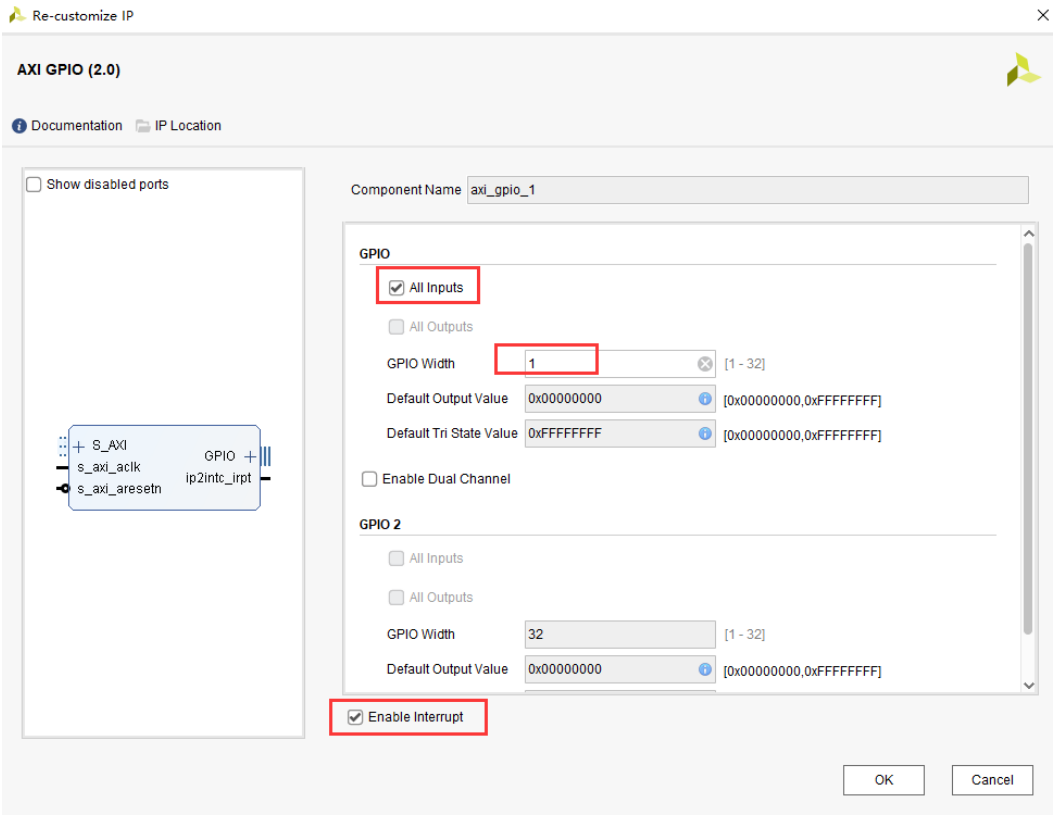
9) 修改 GPIO 端口的名称



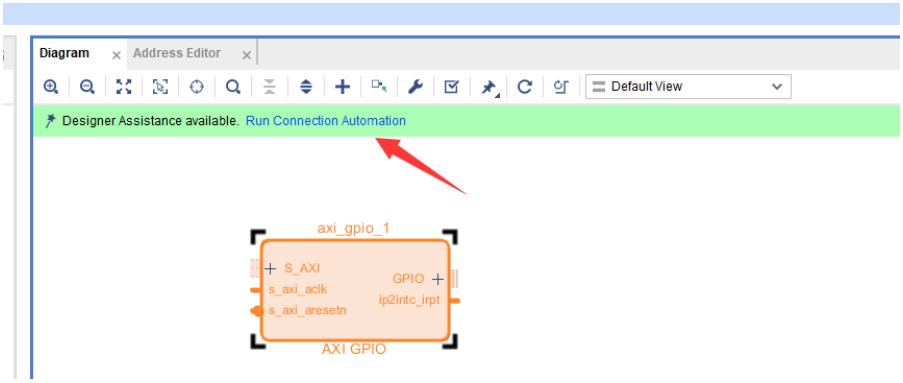
10) 名称修改为 leds



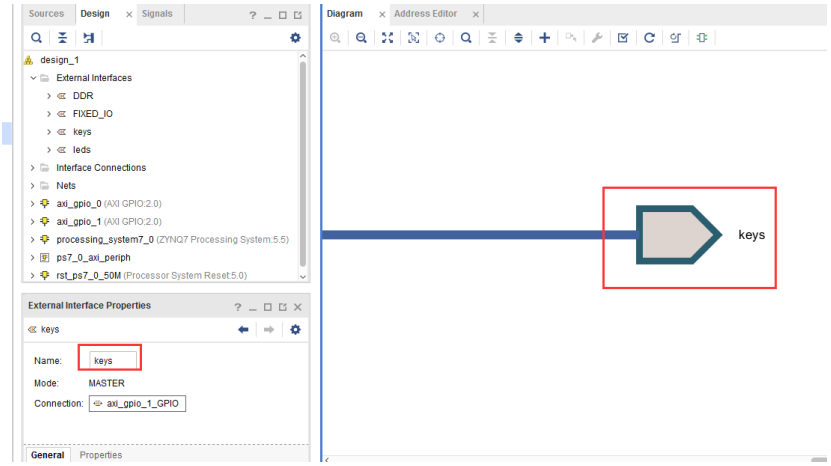
11) 再添加一个 AXI GPIO，连接 PL 端按键，配置 GPIO 参数，都为输入，宽度为 1，使能中断



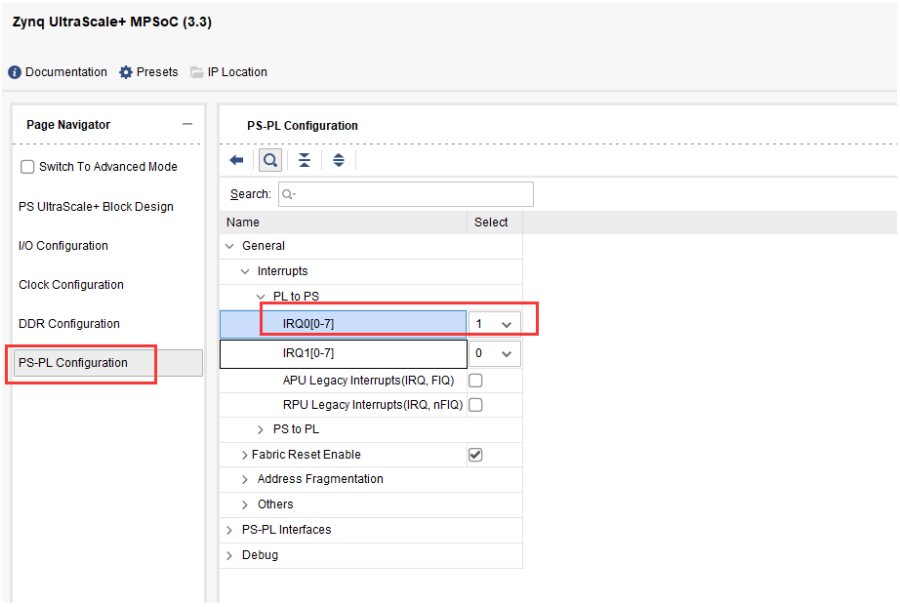
12) 使用自动连接



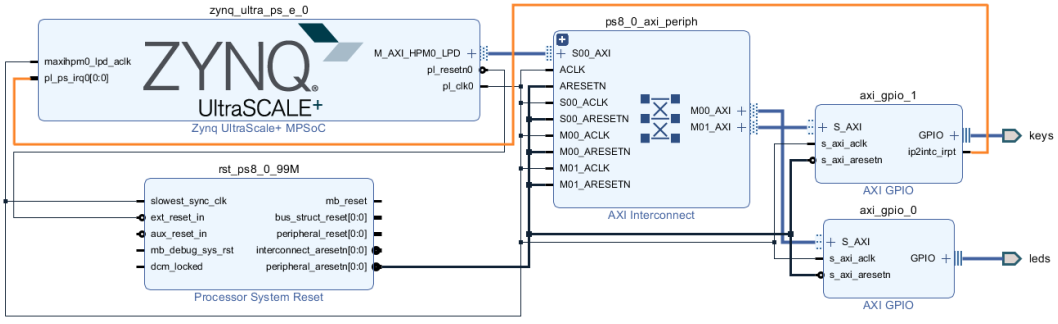
13) 再把端口名称改为 keys



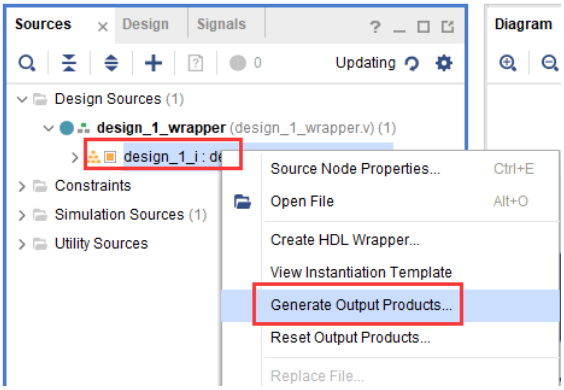
14) 由于是 PL 端过来的中断，在这里需要配置 ZYNQ 处理器的中断，设置 IRQ0[0-7]为 1



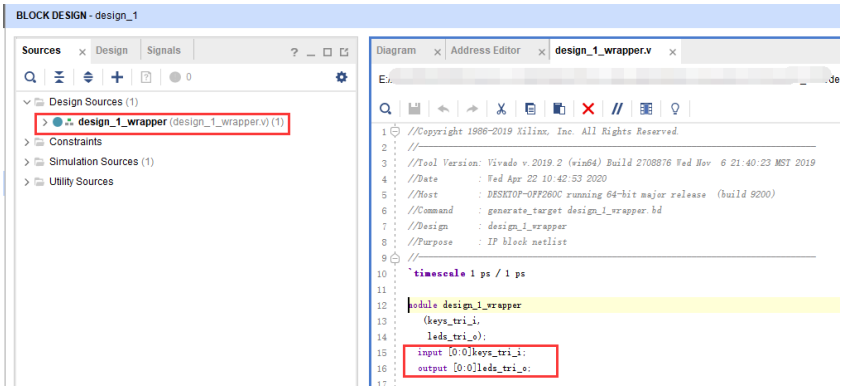
15) 连接 ip2intc_irpt 到 pl_ps_irq



16) 保存设计，点击 xx.bd，右键 Generate Output Products

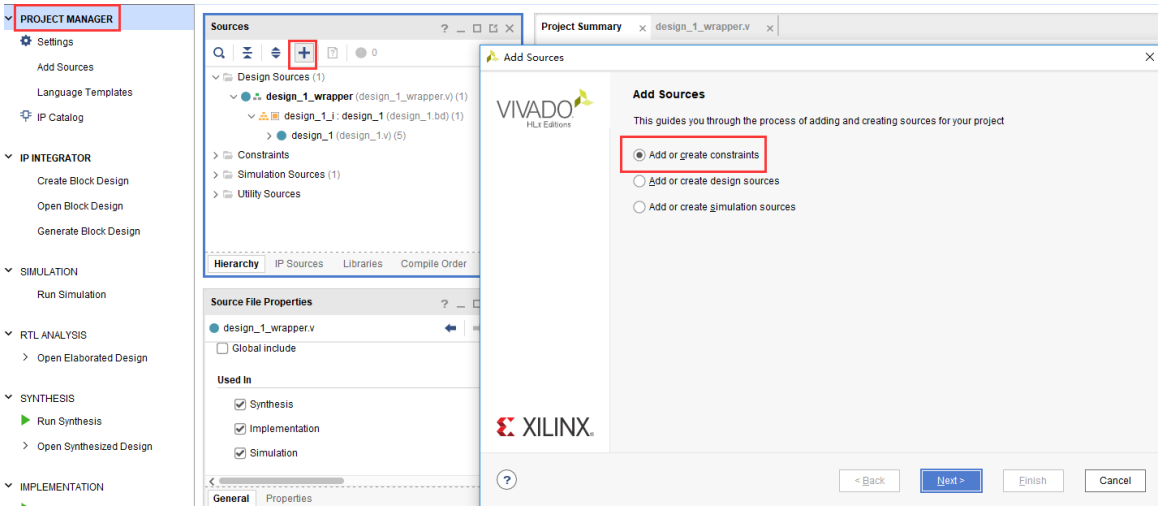


17) 在生成的 Verilog 文件中，可以看到有个 “leds_tri_o” 和”keys_tri_i”的端口，要为他们分配管脚，在绑定引脚时，以这个文件里的引脚名称为准。

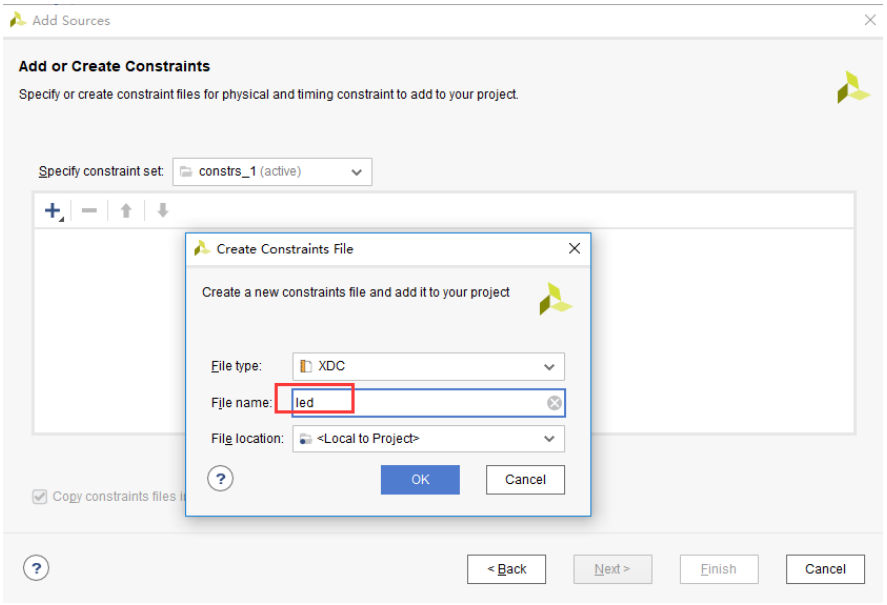


17.3 XDC 文件约束 PL 管脚

4) 创建一个新的 xdc 约束文件



5) 文件名称为 led



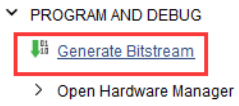
6） led.xdc 添加一下内容，端口名称一定要和顶层文件端口一致

```
##################Compress Bitstream#################################
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]

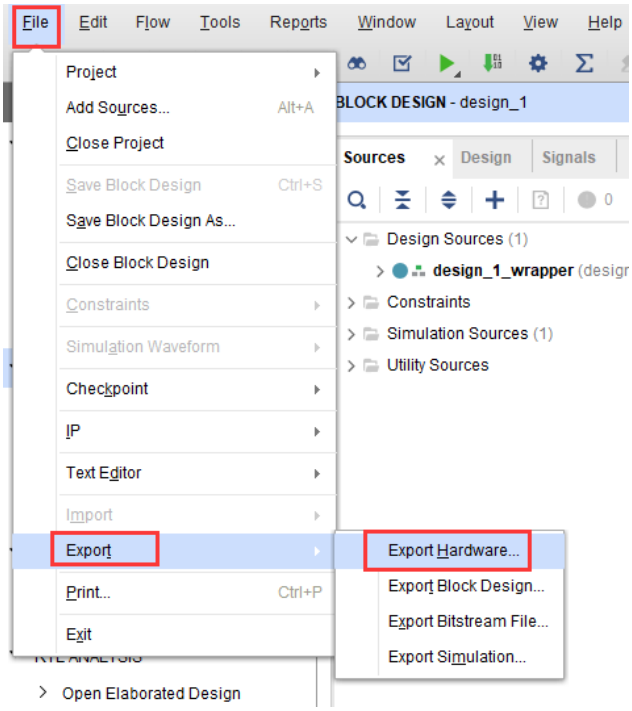
set_property IOSTANDARD LVCMOS33 [get_ports {leds_tri_o[0]}]
set_property PACKAGE_PIN K9 [get_ports {leds_tri_o[0]}]

set_property IOSTANDARD LVCMOS33 [get_ports {keys_tri_i[0]}]
set_property PACKAGE_PIN J9 [get_ports {keys_tri_i[0]}]
```

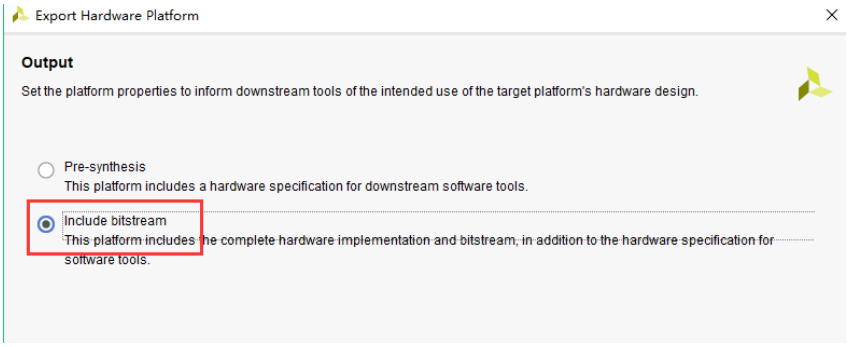
7） 生成 bit 文件



8） 导出硬件 File→Export→Export Hardware



9） 因为要用到 PL，所以选择“Include bitstream”，点击“OK”



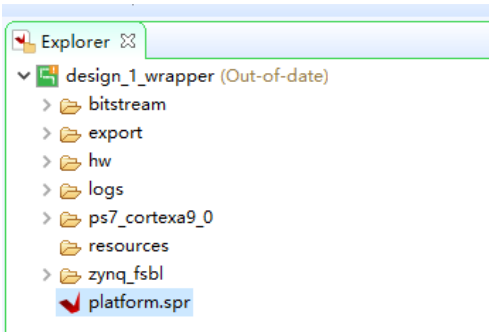
软件工程师工作内容

以下为软件工程师负责内容。

17.4 Vitis 程序编写

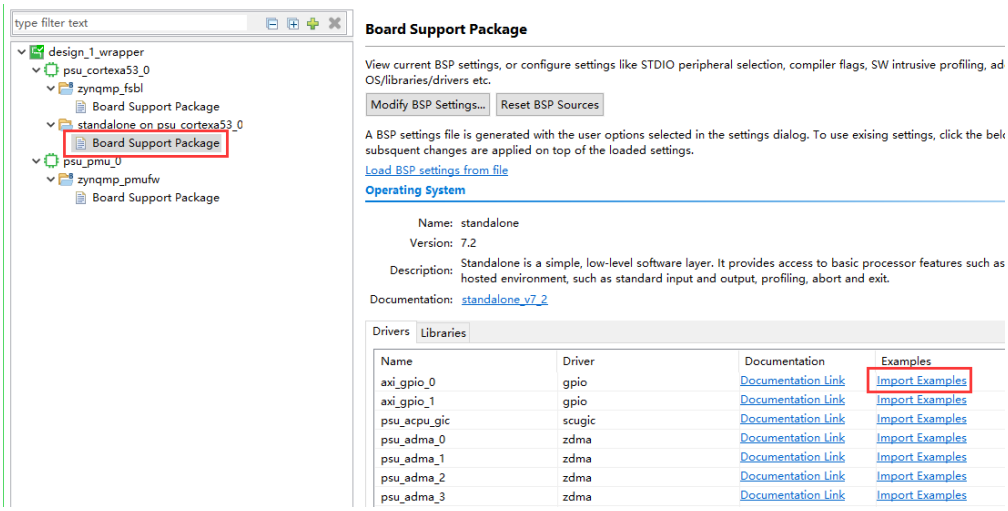
17.4.1 AXI GPIO 点亮 PL 端 LED 灯

1） 创建一个 platform，创建过程参考 “PS 端 RTC 中断实验”一章

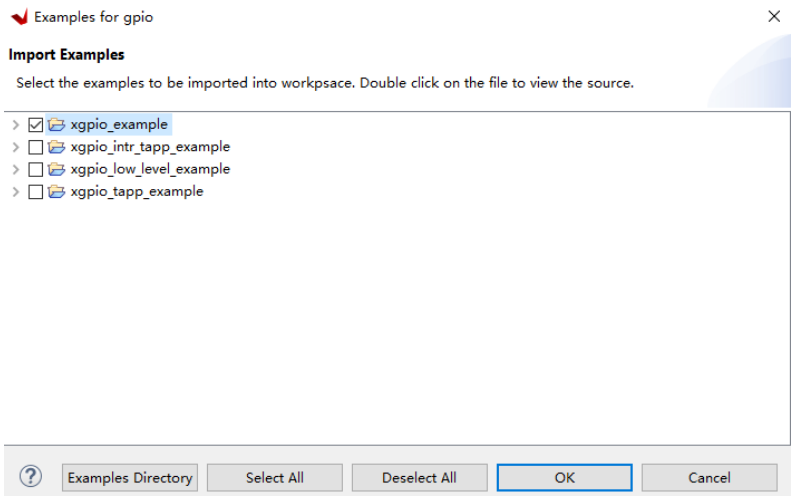


2) 面对一个不熟悉 AXI GPIO，我们如何控制呢？我们可以尝试一下 Vitis 自带的例程

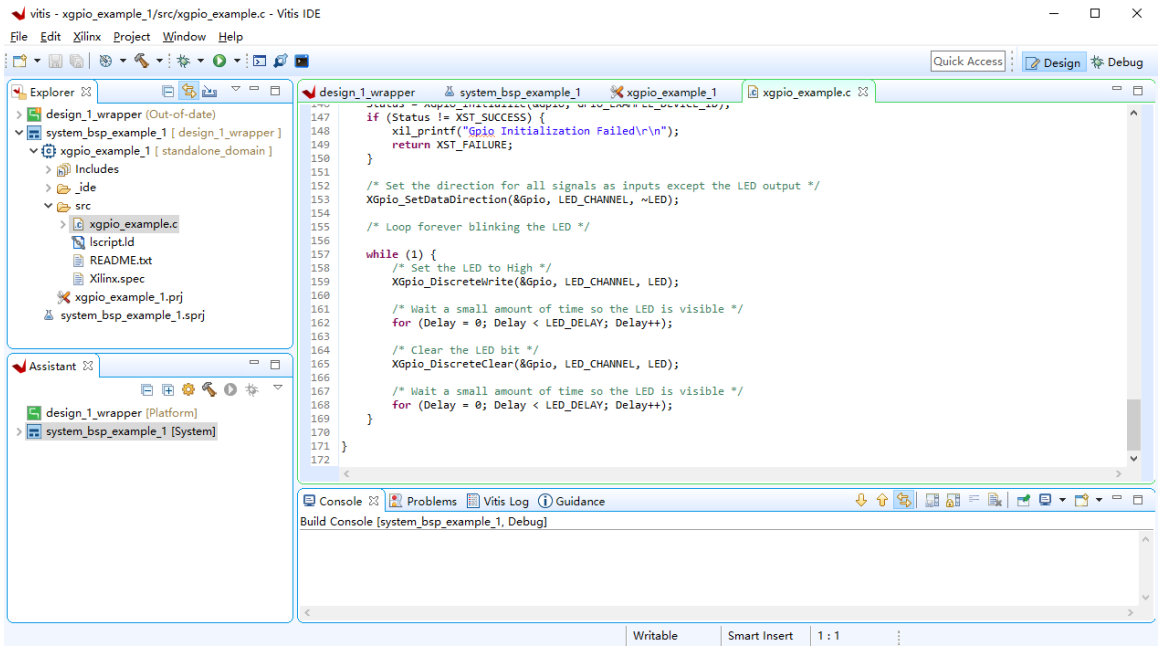
3) 双击“system.mss”,找到“axi_gpio_0”,这里可以点击“Documentation”来看相关文档，这里就不演示，点击“Import Examples”



4) 在弹出的对话框中有多个例程，从名称中可以猜个大概，这里选第一个“xgpio_example”



5) 可以看到例程比较简单，短短几行代码，完成了 AXI GPIO 的操作



里面用到很多 GPIO 相关的 API 函数，通过文档可以了解详细，也可以选中该函数，按“F3”查看具体定义。如果有了这些信息你还不能理解如何使用 AXI GPIO，说明你需要补充 C 语言基础。

其实这些函数都是在操作 GPIO 的寄存器，AXI GPIO 的寄存器也不多，主要是两个 channel 的数据寄存器 GPIO_DATA 和 GPIO2_DATA，两个 channel 的方向控制 GPIO_TRI 和 GPIO2_TRI，以及全局中断使能寄存器 GIER，IP 的中断使能 IP IER 和中断状态寄存器 ISR，具体的功能可以看 AXI GPIO 的文档 pg144。

Table 2-4: **Registers**

Address Space Offset[3]	Register Name	Access Type	Default Value	Description
0x0000	GPIO_DATA	R/W	0x0	Channel 1 AXI GPIO Data Register.
0x0004	GPIO_TRI	R/W	0x0	Channel 1 AXI GPIO 3-state Control Register.
0x0008	GPIO2_DATA	R/W	0x0	Channel 2 AXI GPIO Data Register.
0x000C	GPIO2_TRI	R/W	0x0	Channel 2 AXI GPIO 3-state Control.
0x011C	GIER[1]	R/W	0x0	Global Interrupt Enable Register.
0x0128	IP IER[1]	R/W	0x0	IP Interrupt Enable Register (IP IER).
0x0120	IP ISR[1]	R/TOW[2]	0x0	IP Interrupt Status Register.

比如进入到设置 GPIO 方向的函数中，就可以看到是在向 GPIO 的 GPIO_TRI 寄存器写数据，从而控制方向。

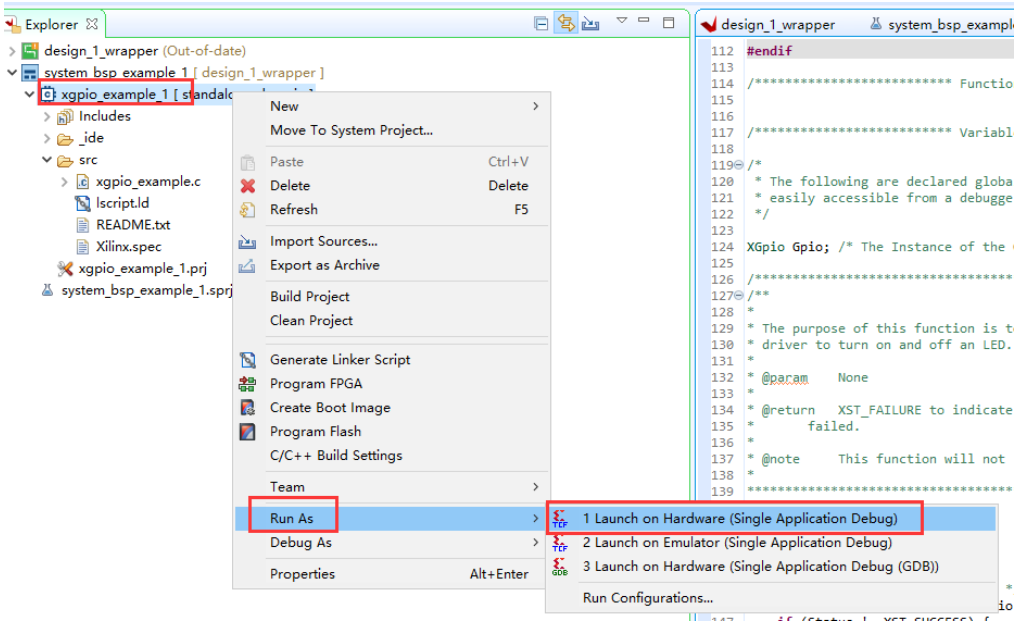
```
void XGpio_SetDataDirection(XGpio *InstancePtr, unsigned Channel,
                u32 DirectionMask)
{
    Xil_AssertVoid(InstancePtr != NULL);
    Xil_AssertVoid(InstancePtr->IsReady == XIL_COMPONENT_IS_READY);
    Xil_AssertVoid((Channel == 1) ||
             ((Channel == 2) && (InstancePtr->IsDual == TRUE)));

    XGpio_WriteReg(InstancePtr->BaseAddress,
            ((Channel - 1) * XGPIO_CHAN_OFFSET) + XGPIO_TRI_OFFSET,
            DirectionMask);
}
```

其他的函数也可以按此法自行研究。

17.4.2 下载调试

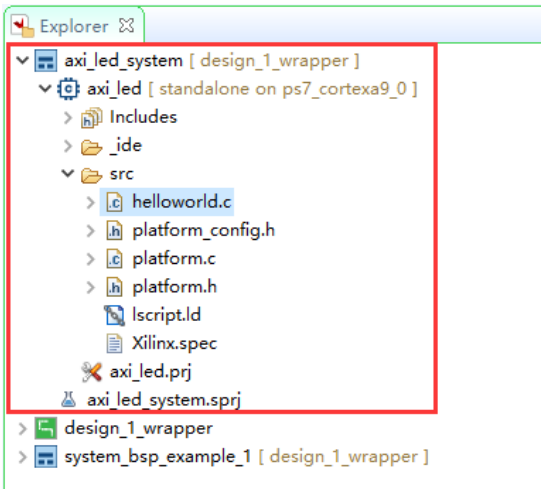
1) 首先编译 APP 工程，编译方法前面的例程已经介绍过了。虽然 Vitis 可以提供一些例程，但有一部分例程是需要自己修改的，这个简单的 LED 例程就不修改了，尝试运行一下，发现不能达到预期效果，甚至提示一些错误。下载后可以看到开发板 PL LED 快速闪烁。



17.4.3 寄存器方式实现

如果觉得 Xilinx 提供的 API 函数比较繁琐，效率低，也可以采取操作寄存器的方式实现 LED 的控制。

比如下面我们新建了 axi_led 的工程，修改 helloworld.c 如下。



```
#define GPIO_BASEADDR 0x80000000
#define DATA_OFFSET   0x0
#define TRI_OFFSET    0x4

#define LED_DELAY     10000000

int main()
{
    volatile int Delay;
    /* Set the direction for all signals as outputs */
    *(int *)(GPIO_BASEADDR + TRI_OFFSET) = 0x0 ;

    while (1) {
        /* Set the LED to High */
        *(int *)(GPIO_BASEADDR + DATA_OFFSET) = 0xF ;

        /* Wait a small amount of time so the LED is visible */
        for (Delay = 0; Delay < LED_DELAY; Delay++);

        /* Set the LED to Low */
        *(int *)(GPIO_BASEADDR + DATA_OFFSET) = 0x0 ;

        /* Wait a small amount of time so the LED is visible */
        for (Delay = 0; Delay < LED_DELAY; Delay++);
    }

    return 0;
}
```

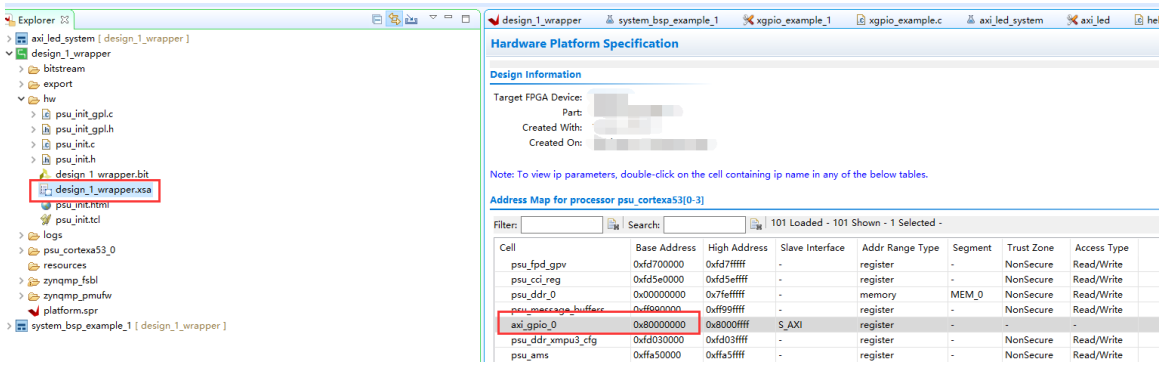
GPIO基地址

偏移地址

设置方向为输出

向数据寄存器写入数据

其中定义的基地址 GPIO_BASEADDR 可以在 xx.xsa 中里找到



由于我们只启用了 channel1，因此定义了下面的寄存器地址

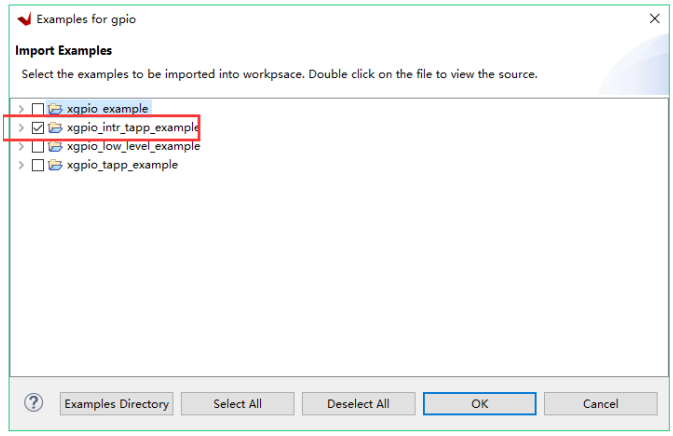
Address Space Offset[3]	Register Name	Access Type	Default Value	Description
0x0000	GPIO_DATA	R/W	0x0	Channel 1 AXI GPIO Data Register.
0x0004	GPIO_TRI	R/W	0x0	Channel 1 AXI GPIO 3-state Control Register.

这样直接操作寄存器的方式效率会比调用 Xilinx API 函数高，而且更直观，对于理解程序如何运行有很大帮助。但是对于大工程来讲，这种方式使用起来就比较复杂，主要依据个人需求选择。

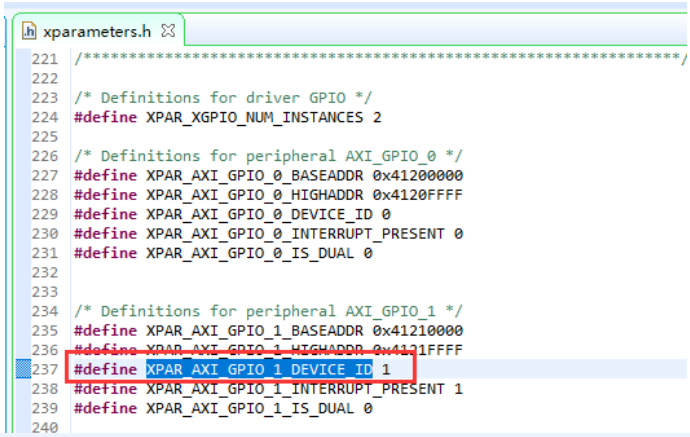
17.4.4 AXI GPIO 之 PL 端按键中断

前面的定时器中断实验的中断属于 PS 内部的中断，本实验中断来自 PL，PS 最大可以接收 16 个来自 PL 的中断信号，都是上升沿或高电平触发。

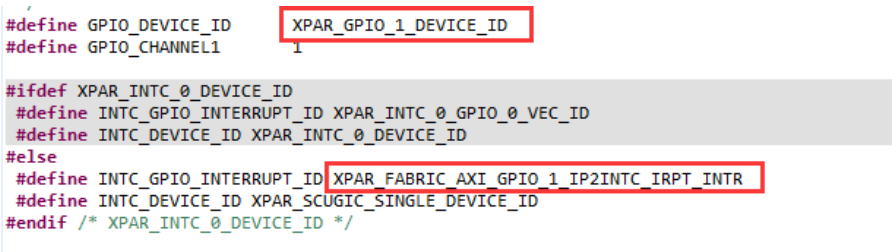
1) 和前面的教程一样，在不熟悉 Vitis 程序编写的情况下，我们尽量使用 Vitis 自带例程来修改，选择“xgpio_intr_tapp_example”



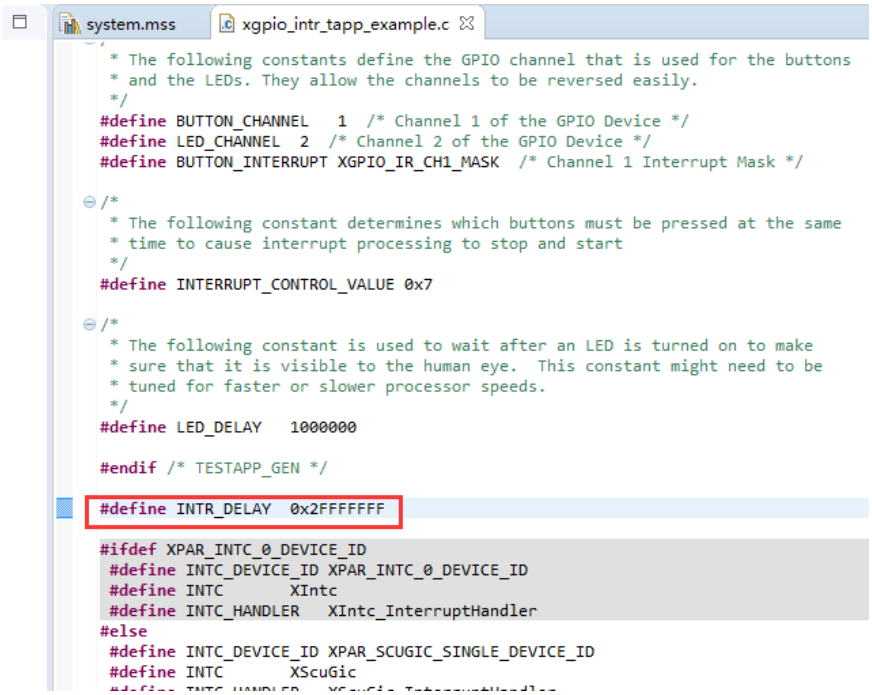
2) 需要修改部分代码，按键的 axi gpio 模块叫做 axi_gpio_1，在 xparameters.h 中找到它的 device id



3) 然后可以修改 GPIO 和中断号的宏定义如下

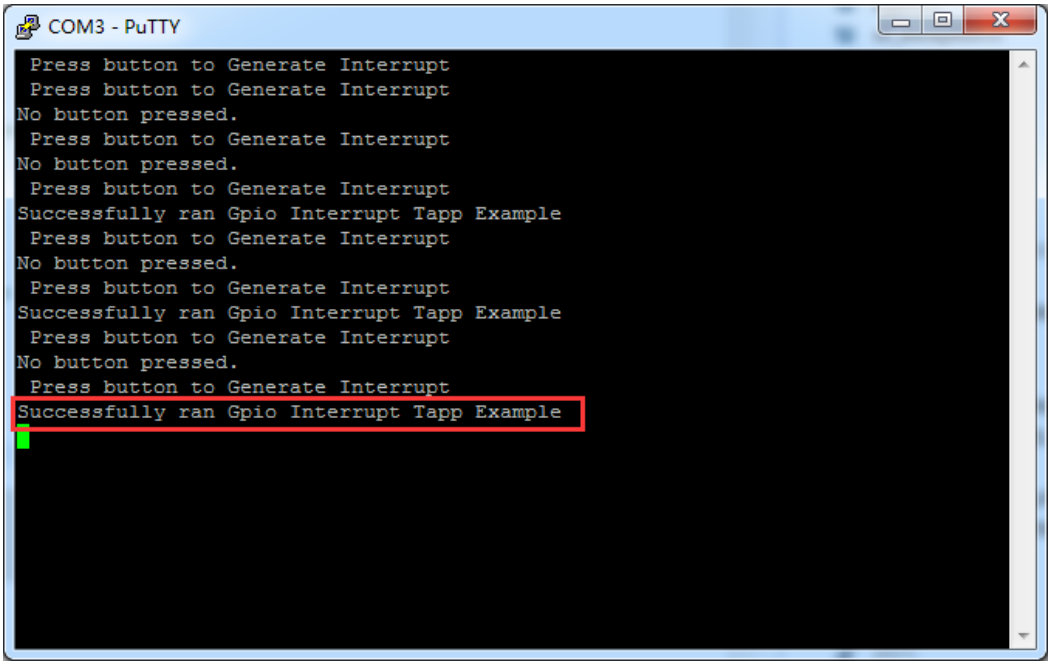


4) 修改测试延时时间，让我们有足够的时间去按按键



17.4.5 下载调试

1) 保存文件，编译工程，打开串口终端，下载程序。如果一直不按按键，串口显示“No button pressed.”，如果按下“PL_KEY”按键显示“Successfully ran Gpio Interrupt Tapp Example”。



17.5 实验总结

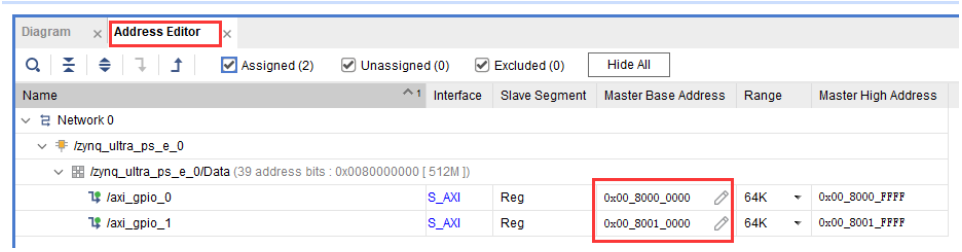
通过实验我们了解到 PS 可以通过 AXI 总线控制 PL，但几乎没有体现出 ZYNQ 的优势，因为对于控制 LED 灯，无论是 ARM 还是 FPGA，都可以轻松完成，但是如果把 LED 换成串口呢，控制 100 路串口通信，8 路以太网等应用，我想还没有哪个 SOC 能完成这种功能，只有 ZYNQ 可以，这就是 ZYNQ 和普通 SOC 的不同之处。

PL 端可以给 PS 发送中断信号，这提高了 PL 和 PS 数据交互的效率，在需要大数量、低延时的应用中需要用到中断处理。

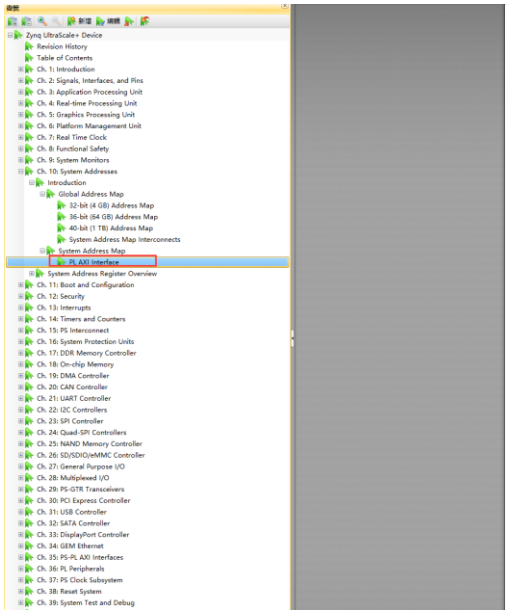
到本章结束已经把 ZYNQ 的 PS 端 MIO、EMIO，PL 端 GPIO 如何使用讲完了，包括输入和输出以及中断处理，这些都是最基础的操作，大家还是要多多思考，理解清楚。

17.6 知识点分享

1) 在设计好后，可以看到 Address Editor 中，已经为 AXI 外设分配好了地址空间，其中偏移地址和空间大小是可以修改的。



但是修改偏移地址是有限制的，详情参考 UG585 文档 System Address 一章， AXI 外设连接到了 M_AXI_GP0 口， 在 8000_0000 到 9FFF_FFFF 地址空间内修改。



PL AXI Interface

The AXI interface from the LPD to PL is assigned a fixed address space of 512 MB in the lower 4 GB address space. It is typically used for LPD to PL communications because it provides a low-latency path from the LPD masters like the RPU and the LPD DMA unit to the PL. The AXI interfaces from the FPD to PL are assigned multiple address ranges.

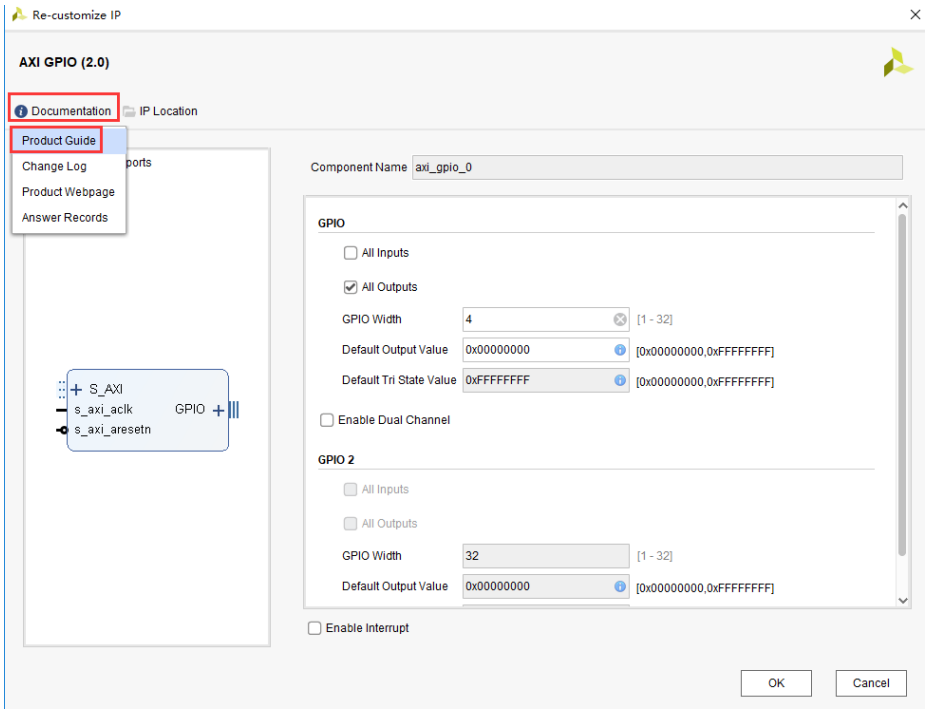
The comprehensive system-level addresses map is shown in Table 10-1.

Table 10-1: **Top-Level System Address Map**

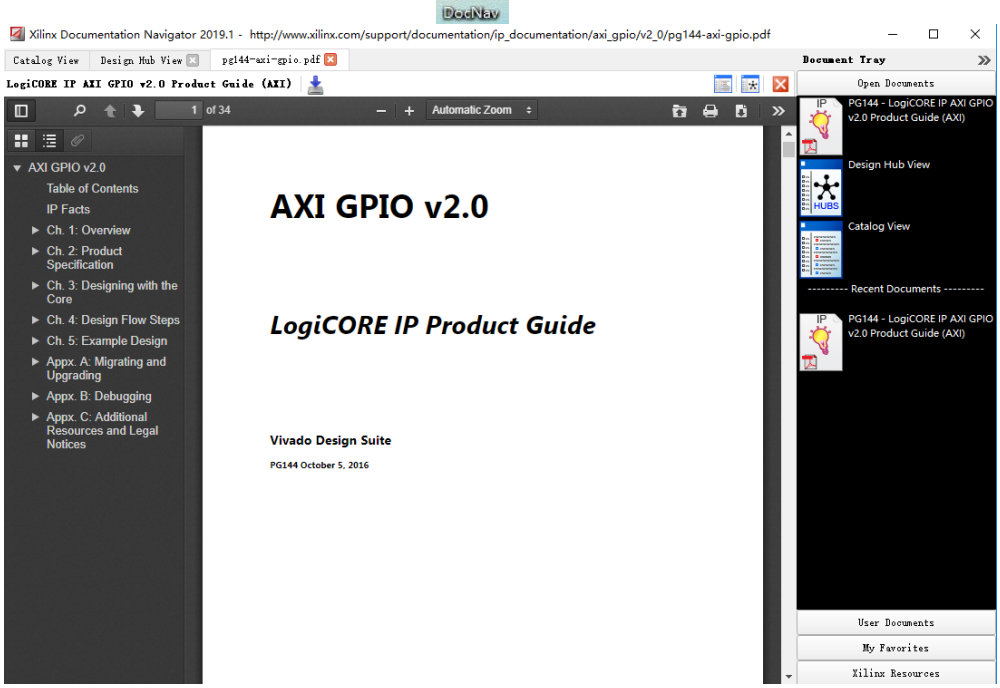
Slave Name	Size	Start Address	End Address
DDR Low	2 GB	0x0000_0000	0x7FFF_FFFF
M_AXI_HPM0_LPD (LPD_PL)	512 MB	0x8000_0000	0x9FFF_FFFF
VCU(1)	64 MB	0xA000_0000	0xA3FF_FFFF
M_AXI_HPM0_FPD (HPM0) interface(1)	192 MB	0xA400_0000	0xAFFF_FFFF
M_AXI_HPM1_FPD (HPM1) interface	256 MB	0xB000_0000	0xBFFF_FFFF
Quad-SPI	512 MB	0xC000_0000	0xDFFF_FFFF
PCIe Low	256 MB	0xE000_0000	0xEFFF_FFFF
Reserved	128 MB	0xF000_0000	0xF7FF_FFFF
STM CoreSight	16 MB	0xF800_0000	0xF8FF_FFFF
APU GIC	1 MB	0xF900_0000	0xF90F_FFFF
Reserved	63 MB	0xF910_0000	0xFCFF_FFFF
FPD slaves	16 MB	0xFD00_0000	0xFDFF_FFFF
Upper LPD slaves	16 MB	0xFE00_0000	0xFEFF_FFFF
Lower LPD slaves	12 MB	0xFF00_0000	0xFFBF_FFFF
CSU, PMU, TCM, OCM	4 MB	0xFFC0_0000	0xFFFF_FFFF
Reserved	12 GB	0x0001_0000_0000	0x0003_FFFF_FFFF
M_AXI_HPM0_FPD (HPM0)	4 GB	0x0004_0000_0000	0x0004_FFFF_FFFF
M_AXI_HPM1_FPD (HPM1)	4 GB	0x0005_0000_0000	0x0005_FFFF_FFFF
PCIe High	8 GB	0x0006_0000_0000	0x0007_FFFF_FFFF
DDR High	32 GB	0x0008_0000_0000	0x000F_FFFF_FFFF
M_AXI_HPM0_FPD (HPM0)	224 GB	0x0010_0000_0000	0x0047_FFFF_FFFF
M_AXI_HPM1_FPD (HPM1)	224 GB	0x0048_0000_0000	0x007F_FFFF_FFFF
PCIe High	256 GB	0x0080_0000_0000	0x00BF_FFFF_FFFF

2) 在使用一个模块时，需要配套的文档辅助开发，但是这些文档该如何去找呢，例如 XILINX 的 IP，打开模块的配置，在左上角点击 Documentation，再点击 Product Guide，如果在安

装 Vivado 的时候安装了 DocNav，就会跳转过去打开文档。

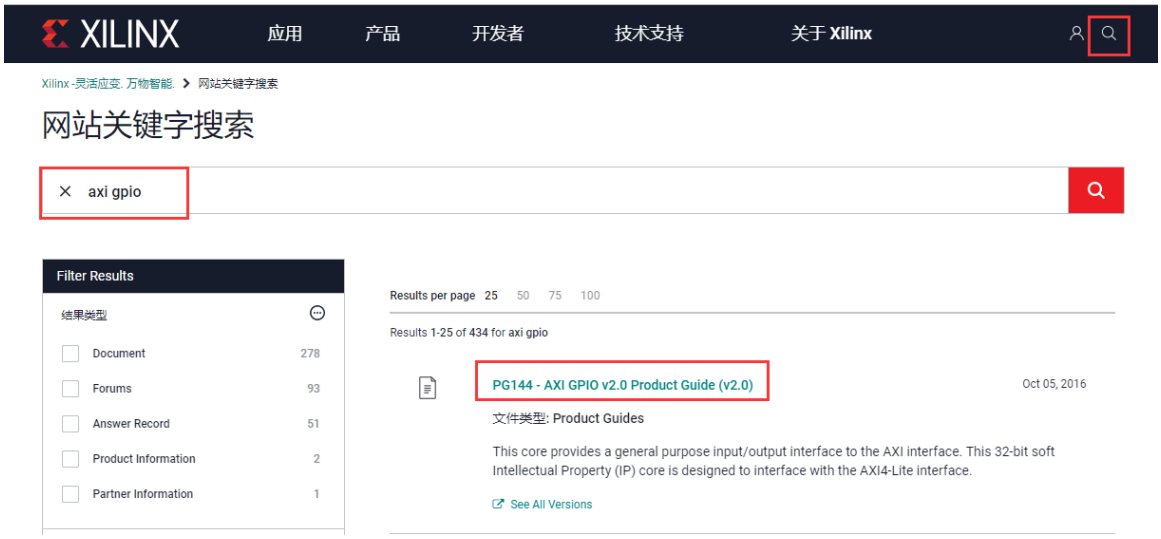






此功能需要电脑联网，DocNav 会从网站加载文档。可以点击下载按钮下载到本地。

还有一种方法是在 Xilinx 官网根据模块的名称搜索资料下载（页面可能会有所变化）


XILINX
应用
产品
开发者
技术支持
关于 Xilinx
Xilinx -灵活应变, 万物智能 > 网站关键字搜索
网站关键字搜索
axi gpio
Filter Results
结果类型
Document 278
Forums 93
Answer Record 51
Product Information 2
Partner Information 1
Results per page 25 50 75 100
Results 1-25 of 434 for axi gpio
PG144 - AXI GPIO v2.0 Product Guide (v2.0)
Oct 05, 2016
文件类型: Product Guides
This core provides a general purpose input/output interface to the AXI interface. This 32-bit soft Intellectual Property (IP) core is designed to interface with the AXI4-Lite interface.
See All Versions

第十八章 PL 端 RS485 测试

实验 Vivado 工程目录为“rs485_test/vivado”。
实验 vitis 工程目录为“rs485_test/vitis”。

本章介绍两路 RS485 接口的读写环回测试实验。

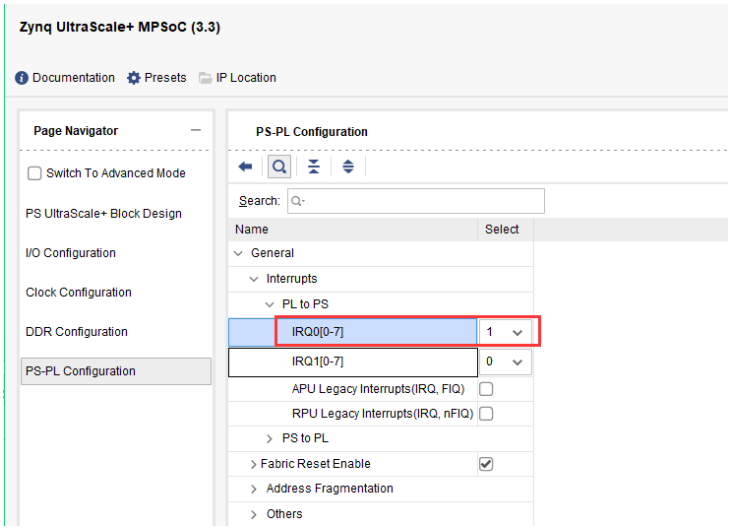
FPGA 工程师工作内容

以下为 FPGA 工程师负责内容。

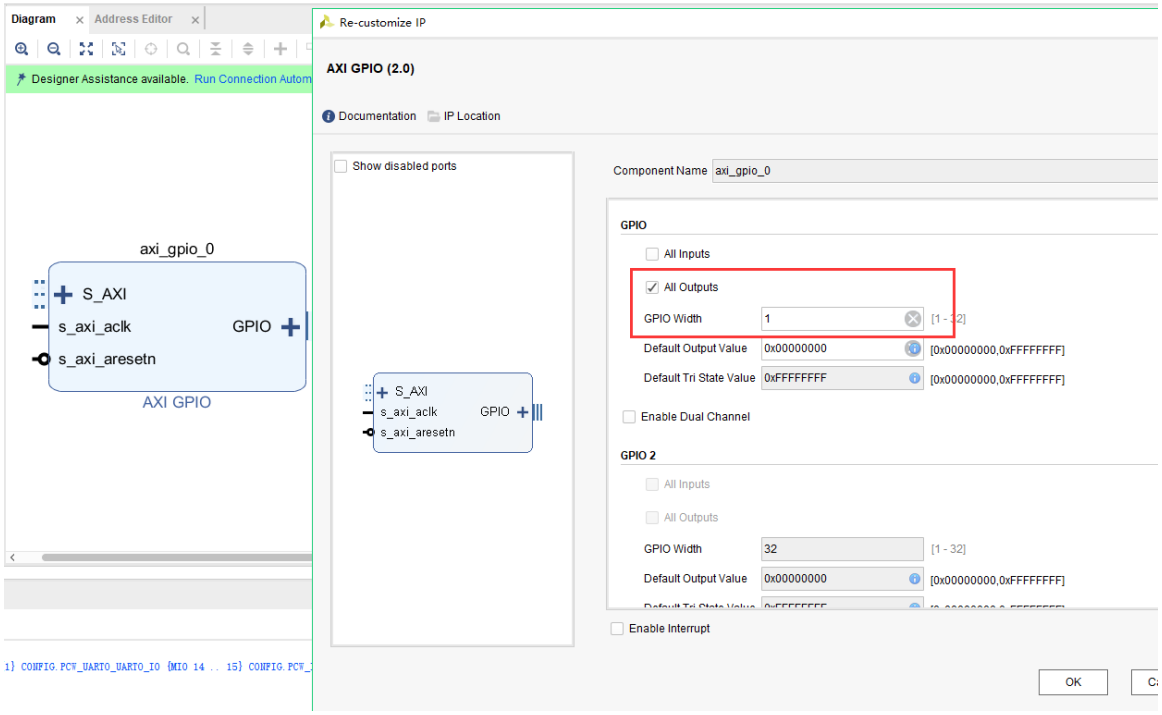
18.1 硬件环境搭建

基于”ps_hello”工程，另存为一份 rs485_test 工程

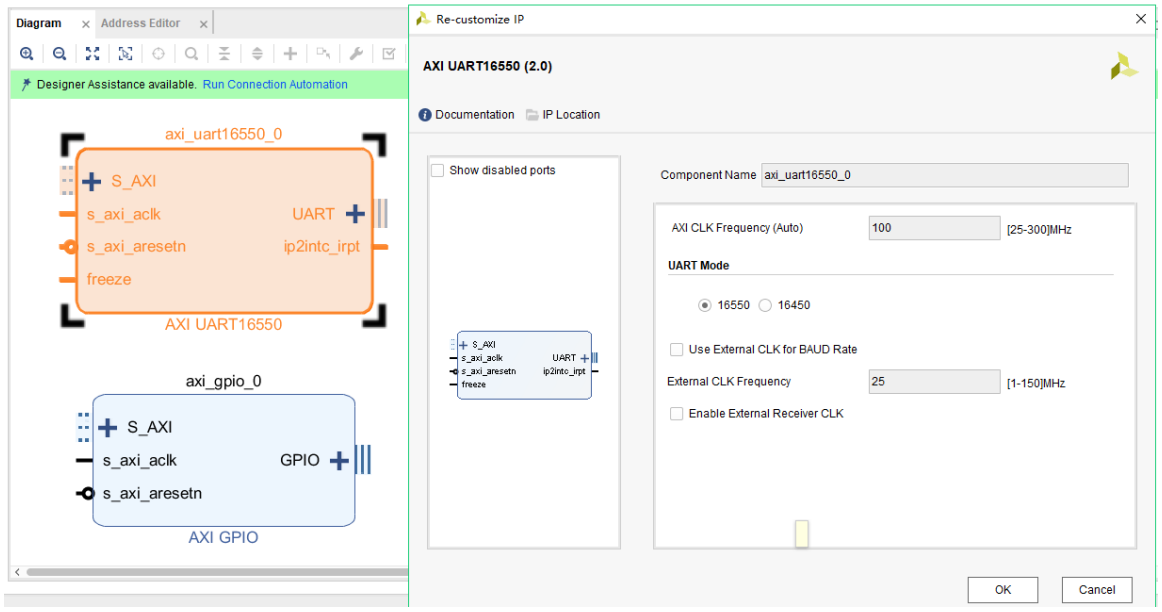
1. 打开 PL 端中断



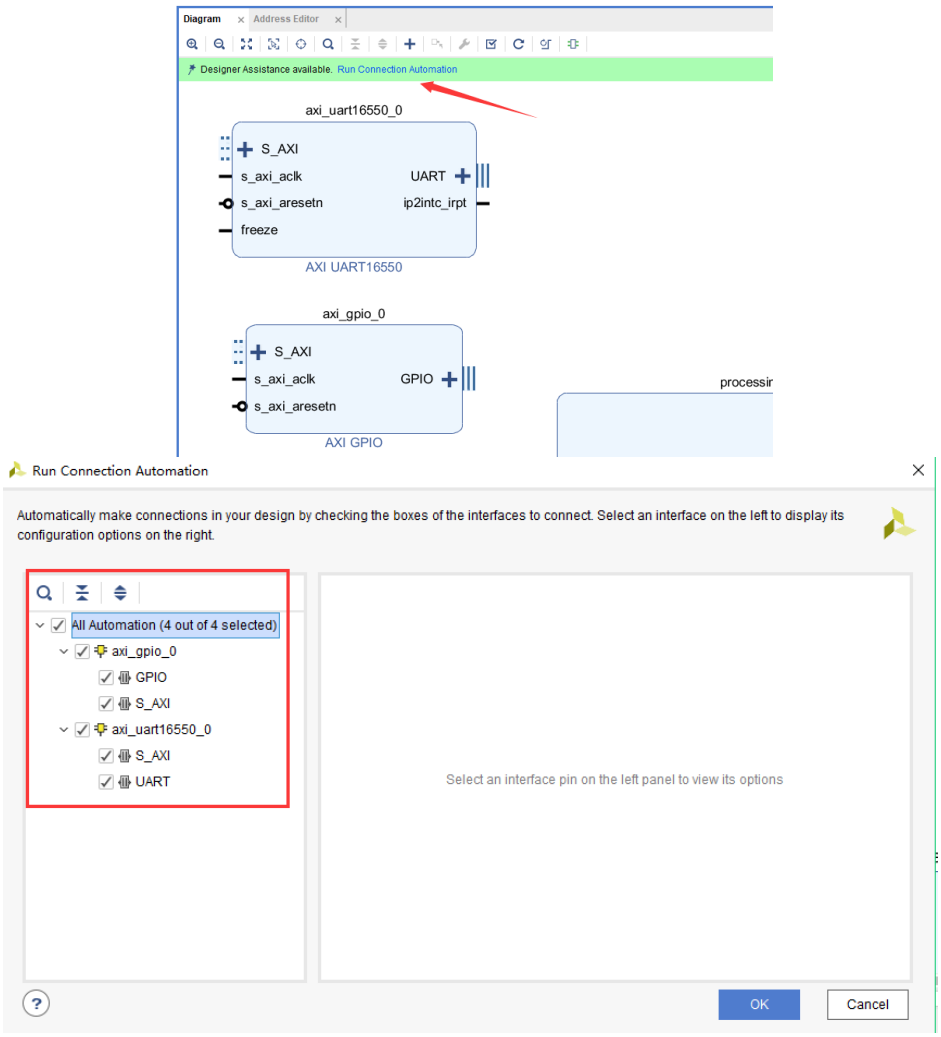
2. 添加 AXI GPIO 模块，并配置为输出，位宽为 1，用于第一路 RS485 的 DE 控制



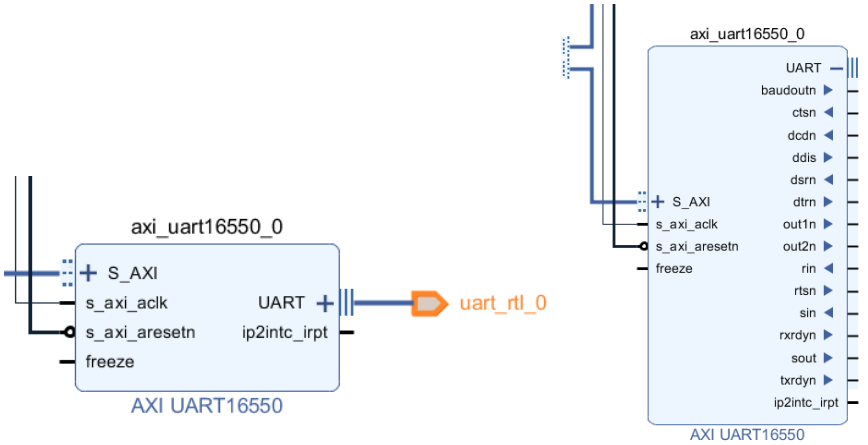
3. 添加 UART16550 模块，用于第二路 RS485 的数据端口



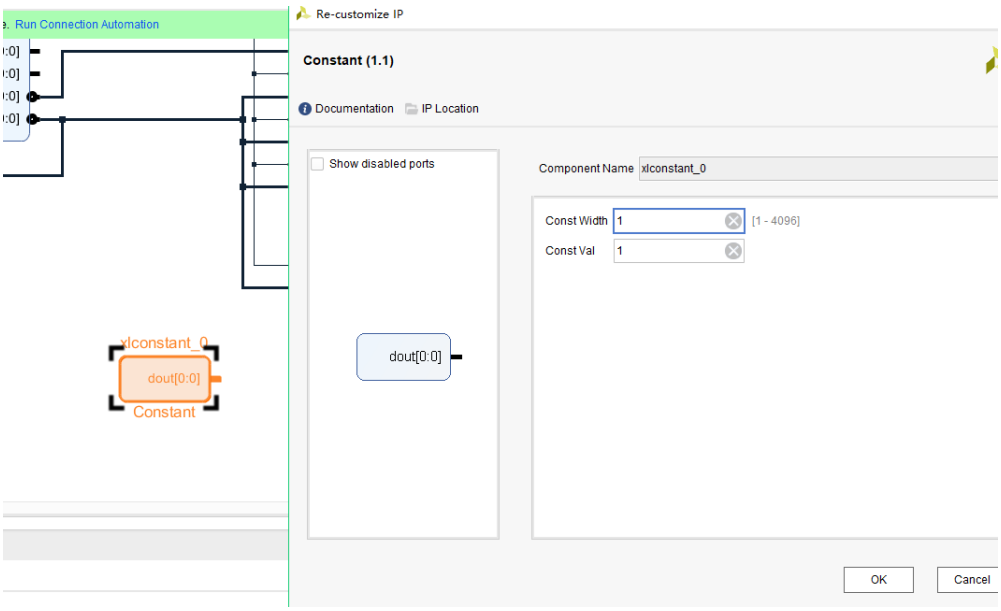
4. 自动连接



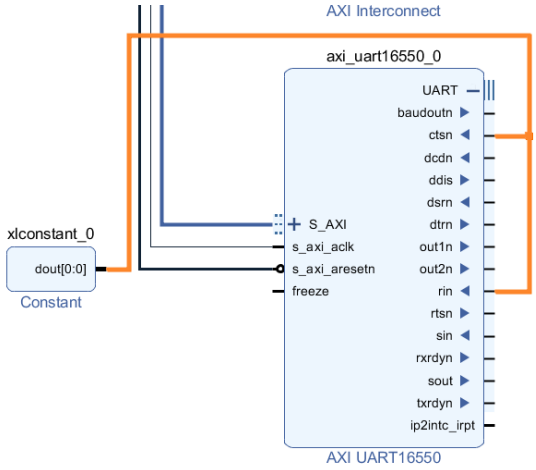
5. 删除 UART 引脚，并展开 UART 接口



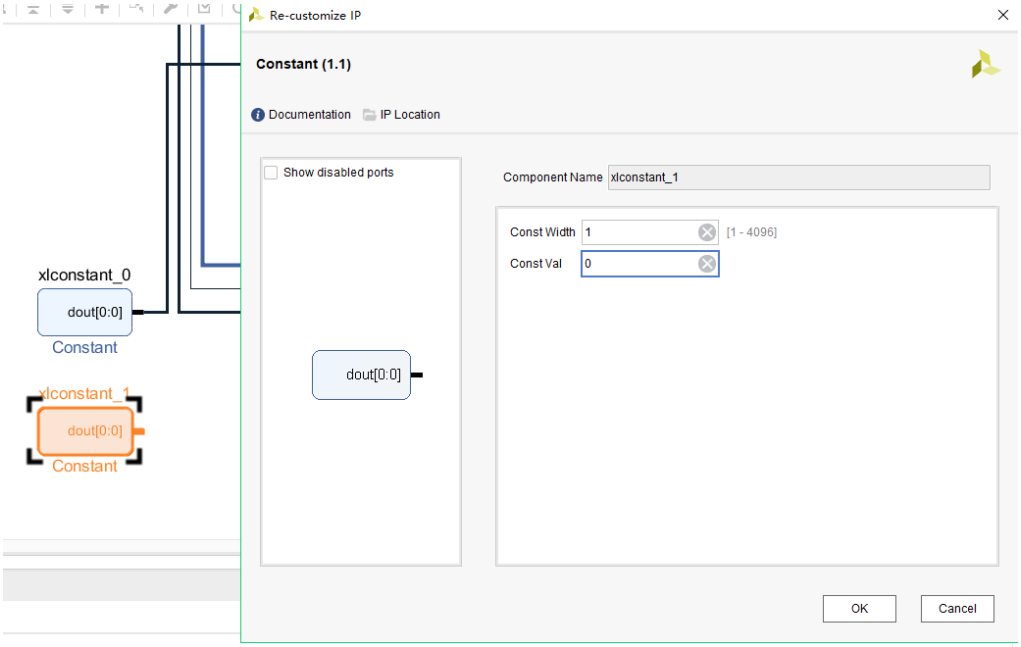
6. 添加 constant 模块，并设置位宽为 1，值为 1，也就是常量



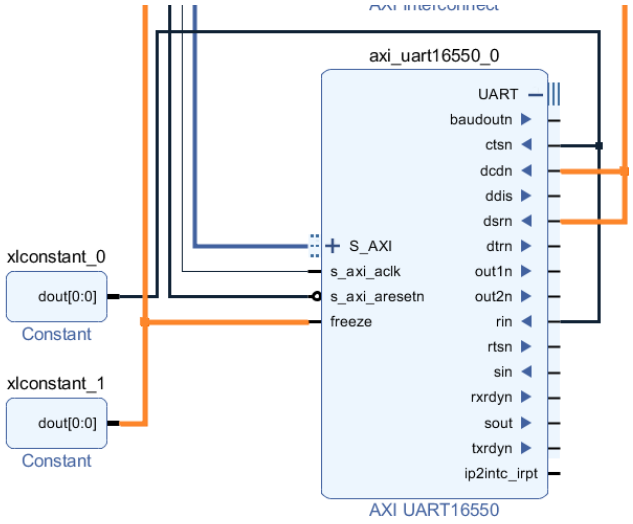
7. 连接 ctsn，rin 信号



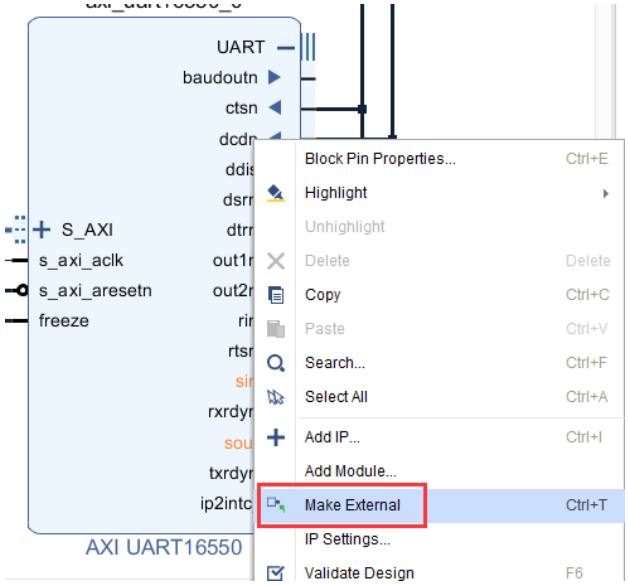
8. 再添加一个 constant 模块，值设置为 0



9. 连接 freeze，dcdn，dsrn 信号

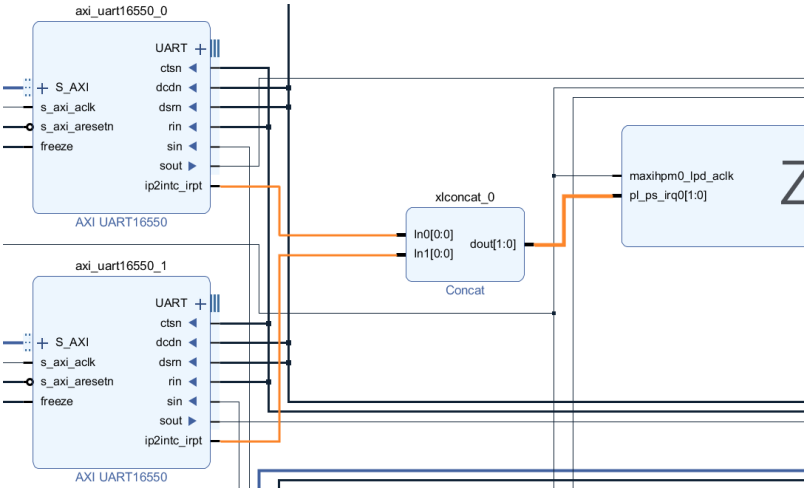


10. 将 sin 和 sout 导出引脚

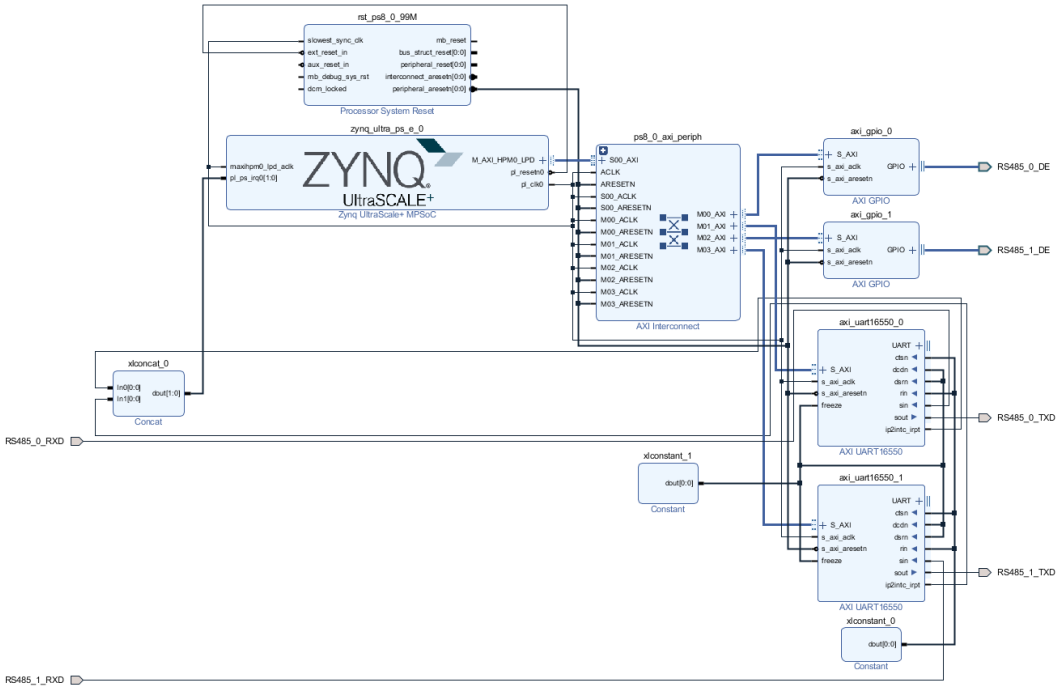


11. 同样的方法配置第二路 485 接口

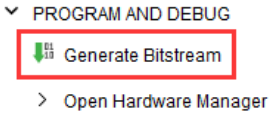
12. 添加 concat 模块，连接两路中断到 pl_ps_irq0



13. 自动连接并修改引脚名称



14. 引脚绑定，并生成 bitstream



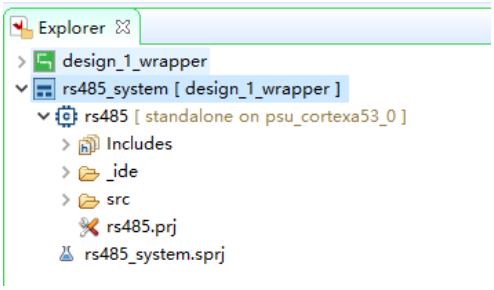
15. 导出硬件平台信息

软件工程师工作内容

以下为软件工程师负责内容。

18.2 Vitis 程序开发

1. 新建 Vitis 工程 rs485，本实验是根据 UART 和 UART16550 模块的 example 工程修改而来，程序流程为，初始化 RS485 DE 和 UART→设置 RS485 0 为发送，设置 RS485 1 为接收，发送 16 个字节数据，接收后进行数据对比→反向，设置 RS485 1 为发送，设置 RS485 0 为接收，发送 16 个字节数据，接收后进行数据对比→程序结束



2. PL 端 GPIO 设置

```
int PLGpioInitial(XGpio *GpioInstPtr, u16 DeviceId)
{
    int Status ;

    /* initial gpio */
    Status = XGpio_Initialize(GpioInstPtr, DeviceId) ;
    if (Status != XST_SUCCESS)
        return XST_FAILURE ;

    /* set gpio as output */
    XGpio_SetDataDirection(GpioInstPtr, 1, 0x0);

    return XST_SUCCESS ;
}
```

3. 在对 UART 初始化中，设置为波特率 115200，8bit，无奇偶校验位，1bit 停止位

```
XUartNs550Format UartNs550Format =
{
        115200,
        XUN_FORMAT_8_BITS,
        XUN_FORMAT_NO_PARITY,
        XUN_FORMAT_1_STOP_BIT
};
```

4. 在回环的函数中，首先是将 RS485_0 设置为输出，RS485_1 设置为输入，设置完后，在发送数据之前等待 1ms 时间，是因为有切换延迟。

```
int UartLoopback(void)
{
    unsigned int SentCount;
    unsigned int ReceivedCount = 0 ;
    u16 Index;
    u32 LoopCount = 0;

    /*
     * Initialize the send buffer bytes with a pattern and zero out
     * the receive buffer.
     */
    for (Index = 0; Index < TEST_BUFFER_SIZE; Index++) {
        SendBuffer[Index] = '0' + Index;
        RecvBuffer[Index] = 0;
    }

    /*
     * Set rs485_0 to tx
     */
    XGpio_DiscreteWrite(&rs485_0_de, 1, 1) ;
    /*
     * Set rs485_1 to rx
     */
    XGpio_DiscreteWrite(&rs485_1_de, 1, 0);

    /* wait 1ms */
    usleep(1000) ;

    memset(RecvBuffer, 0, TEST_BUFFER_SIZE) ;
    /* Block sending the buffer. */
    SentCount = XUartNs550_Send(&UartNs550_0, SendBuffer, TEST_BUFFER_SIZE);
    if (SentCount != TEST_BUFFER_SIZE) {
        return XST_FAILURE;
    }
```

12. 测试后，切换方向 RS485_0 设置为输入，RS485_1 设置为输出，再次进行数据发送

```
xil_printf("RS485_0 to RS485_1 Check Done!\r\n");

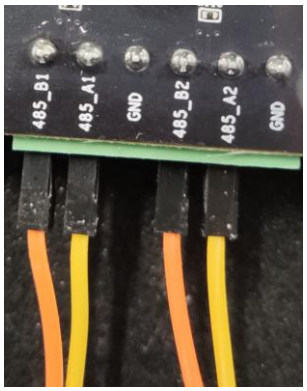
/*
 * Set rs485_0 to rx
 */
XGpio_DiscreteWrite(&rs485_0_de, 1, 0) ;
/*
 * Set rs485_1 to tx
 */
XGpio_DiscreteWrite(&rs485_1_de, 1, 1);

/* wait 1ms */
usleep(1000) ;

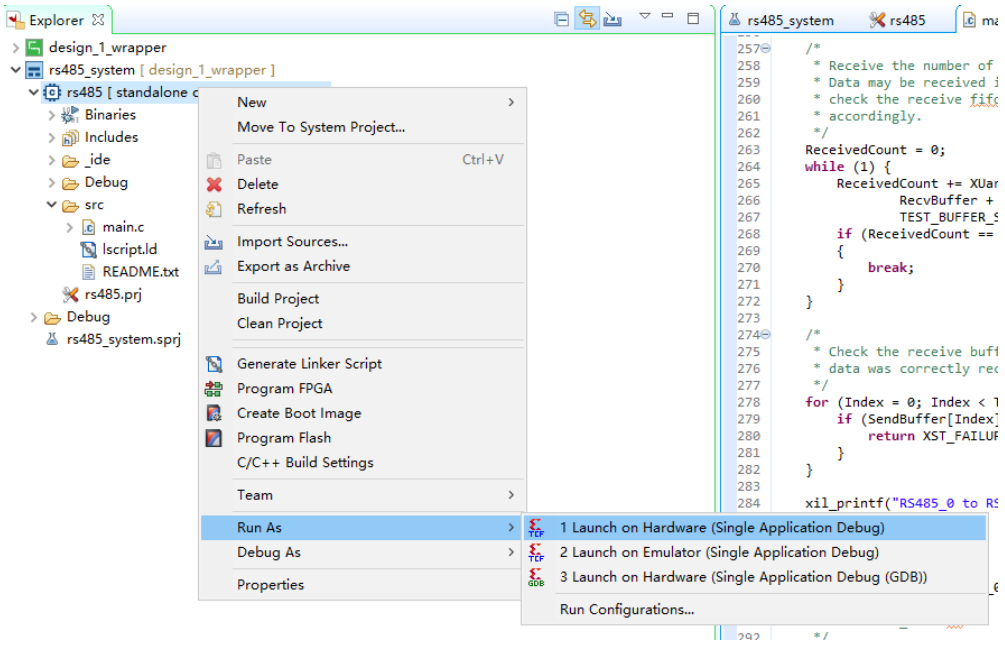
memset(RecvBuffer, 0, TEST_BUFFER_SIZE) ;
/*
 * Send the buffer thru the UART waiting till the data can be
 * sent (block), if the specified number of bytes was not sent
 * successfully, then an error occurred
 */
SentCount = XUartNs550_Send(&UartNs550_1, SendBuffer, TEST_BUFFER_SIZE);
if (SentCount != TEST_BUFFER_SIZE) {
    return XST_FAILURE;
}
```

18.3 板上验证

1. 用杜邦线将 A 与 A 连接，B 与 B 连接如下：



2. 下载



3. 串口打印信息



18.4 本章小结

本章介绍了利用板子上的两路 RS485 进行回环测试，大家可以在此基础上功能扩展进行测试。

第十九章 PL 端以太网实验

实验 Vivado 工程目录为“pl_net/vivado”。

实验 vitis 工程目录为“pl_net/vitis”。

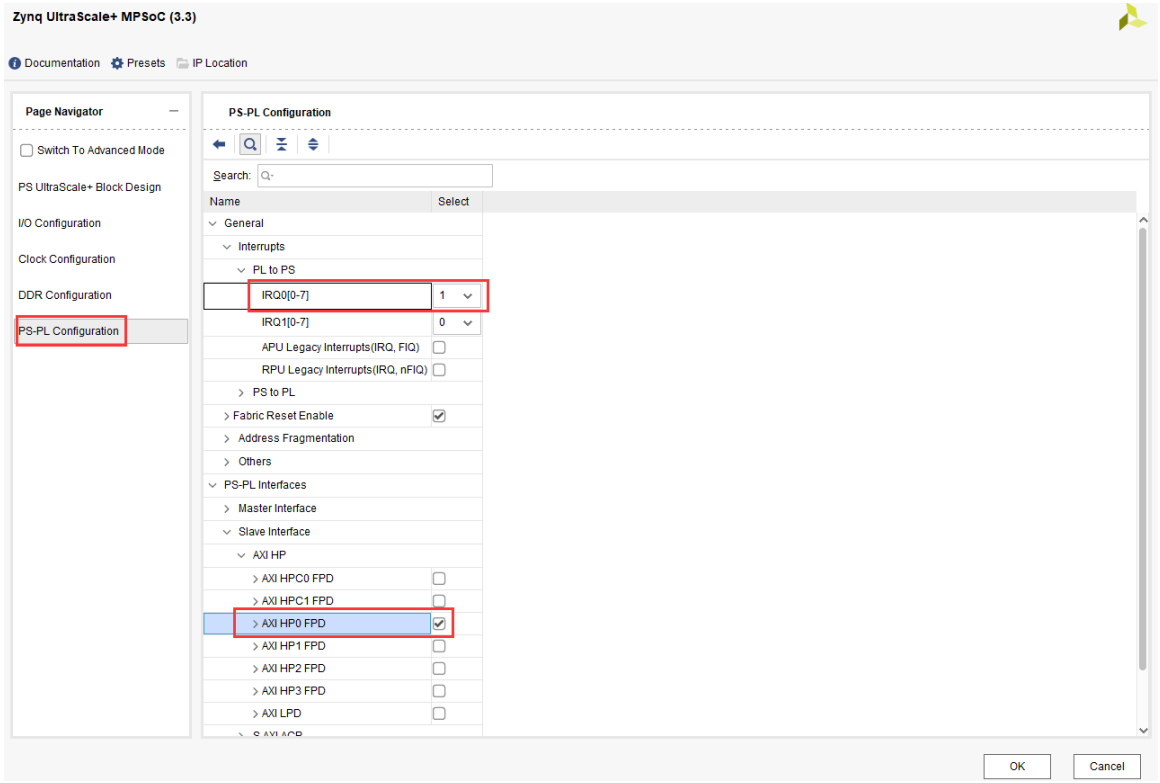
前面介绍了 PS 端的以太网使用方法，本章介绍 PL 端以太网。

FPGA 工程师工作内容

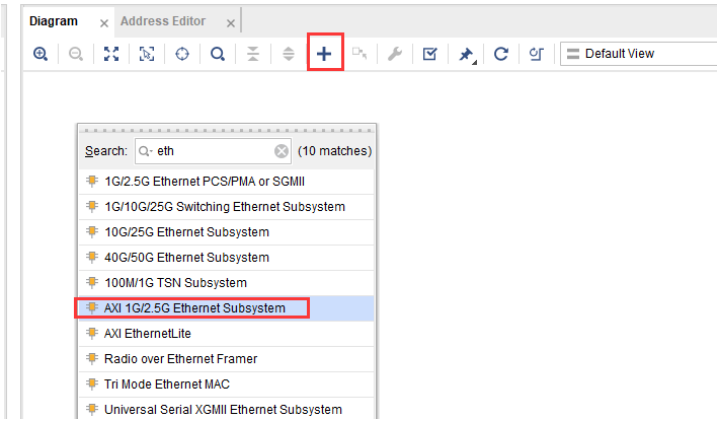
以下为 FPGA 工程师负责内容。

19.1 硬件环境搭建

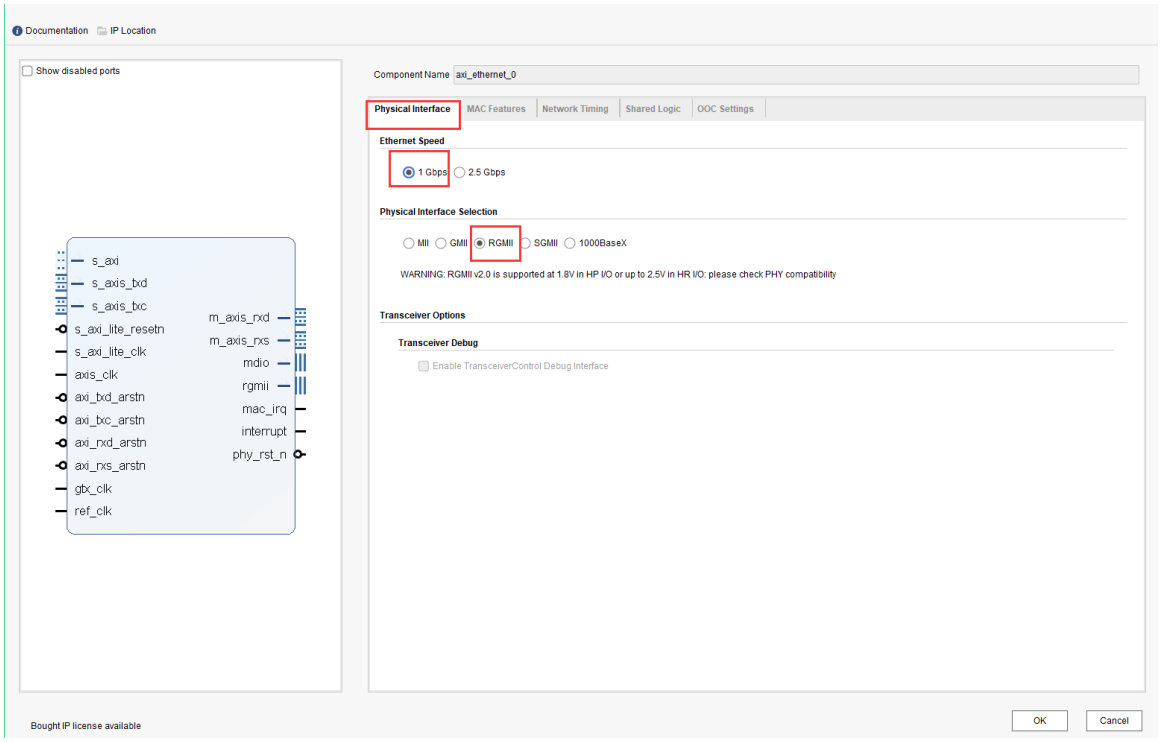
1. 以”ps_hello”为基础，配置中断和 AXI HP0 FPD



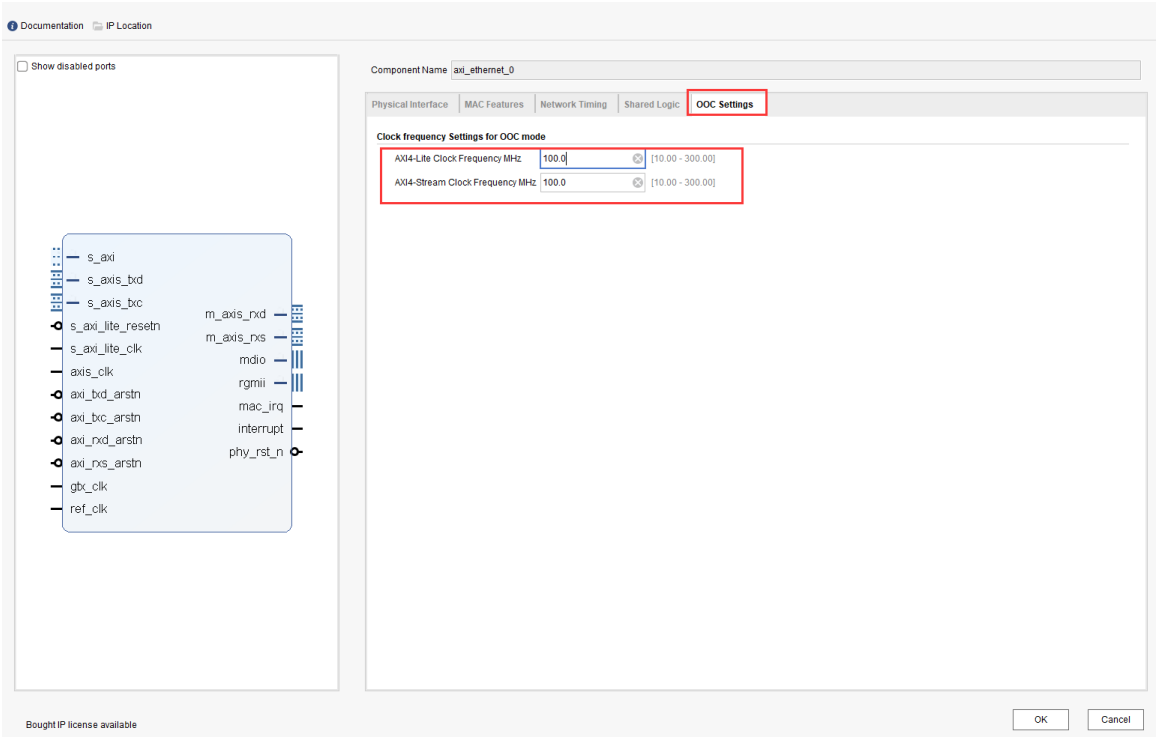
2. 添加 AXI 1G/2.5G Ethernet Subsystem 模块



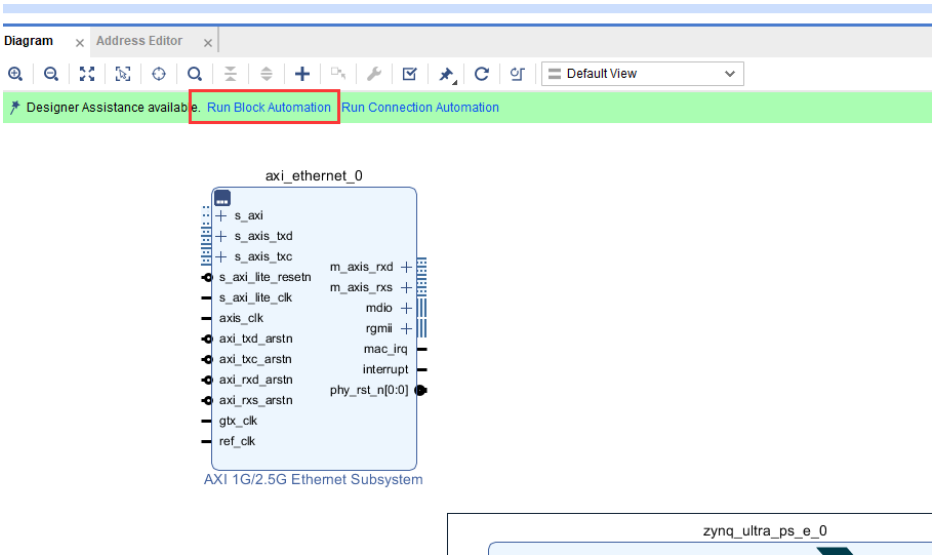
3. 配置速度为 1Gbps，PHY 接口为 RGMII

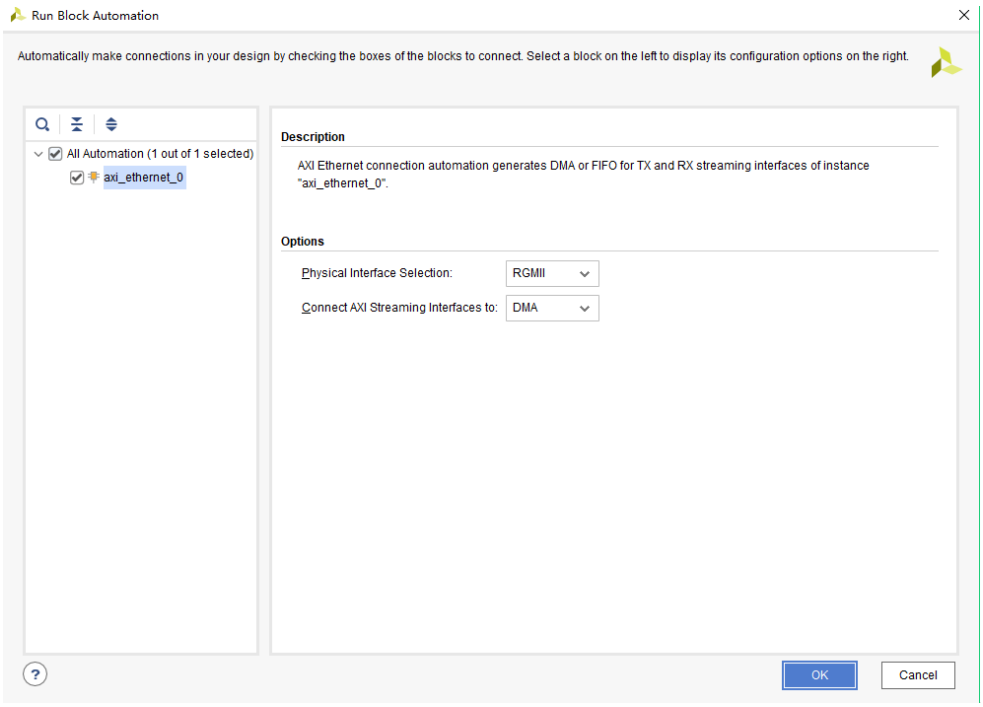


4. 时钟默认为 100MHz，其他配置保持默认，点击 OK

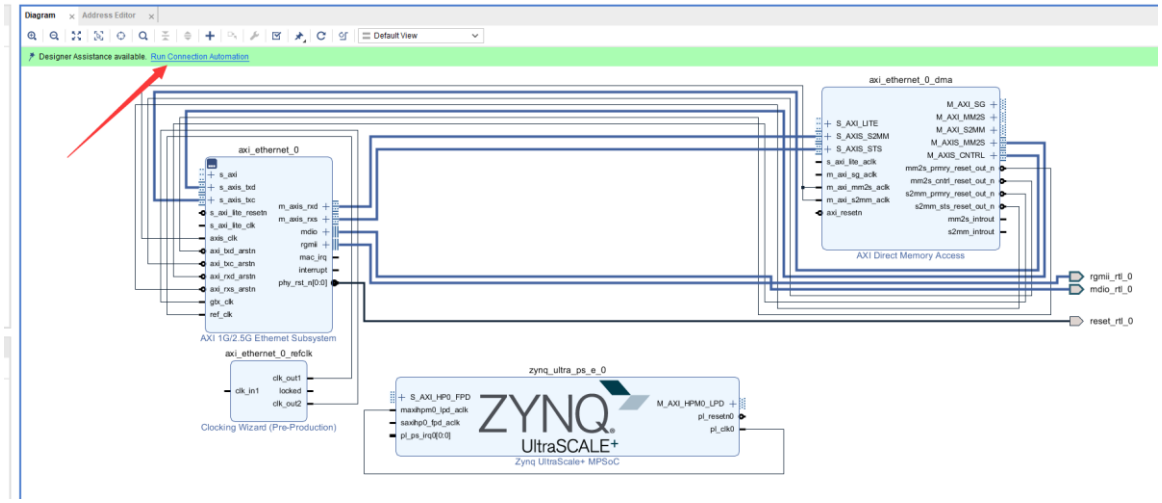


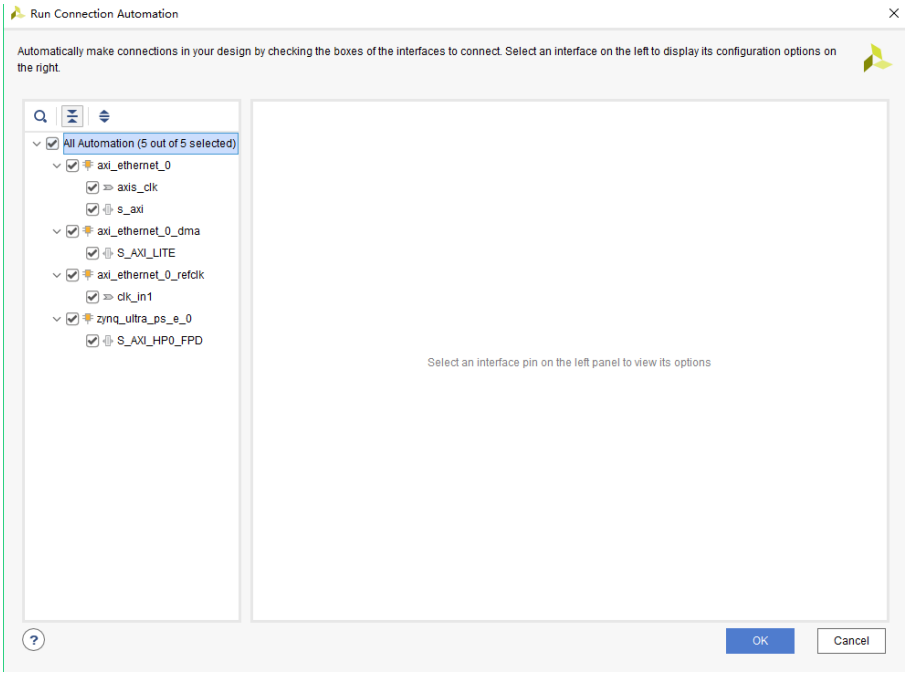
5. 点击 Run Block Automation



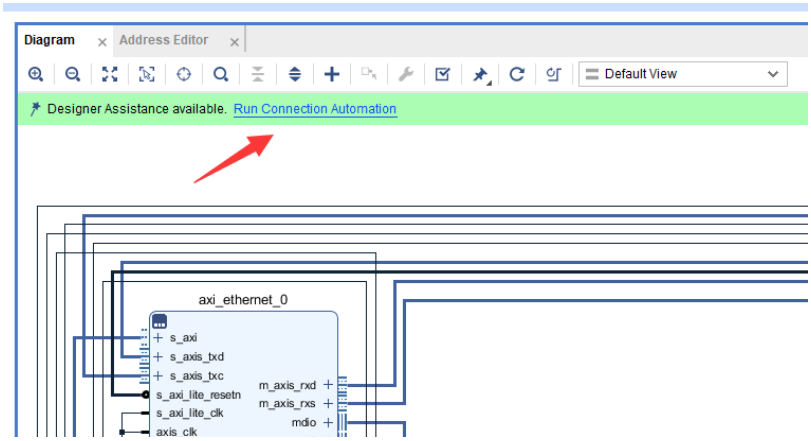


6. 点击 Run Connectino Automation，自动连接

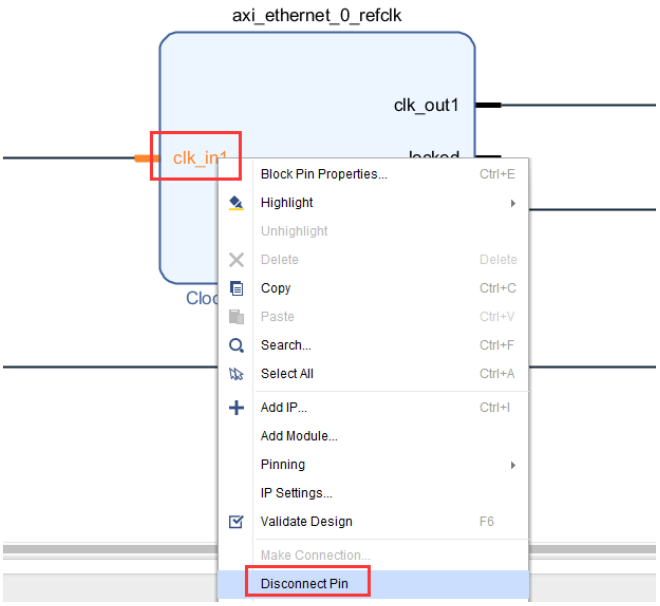




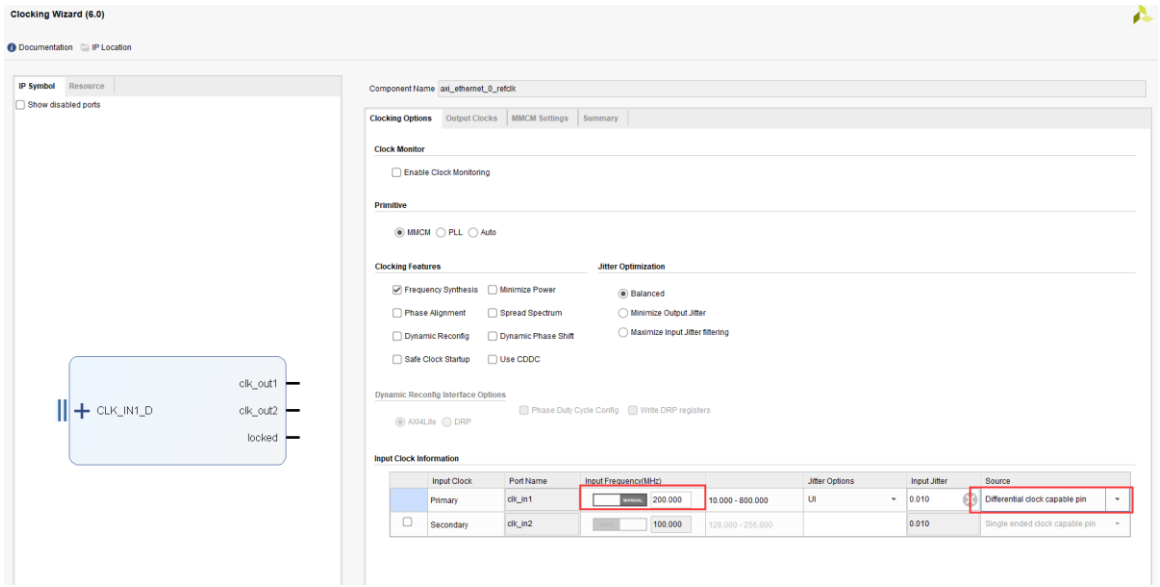
7. 继续自动连接



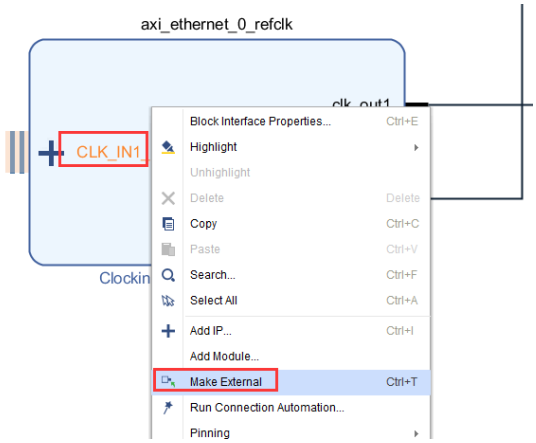
8. 选择 Pll 的输入时钟，Disconnect Pin



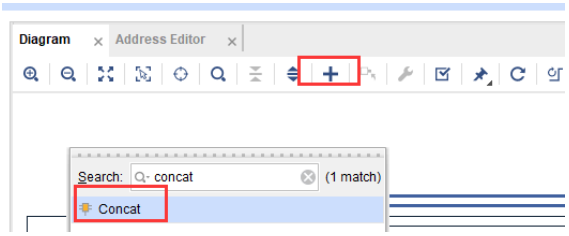
点开 PLL，配置输入时钟为 200MHz，并设置为差分输入



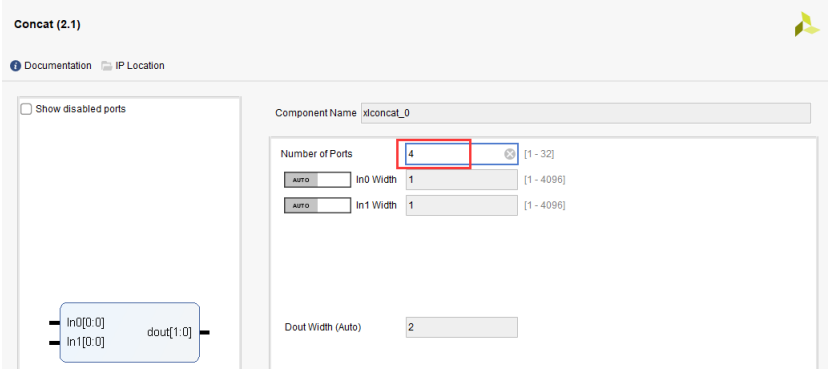
9. 导出引脚



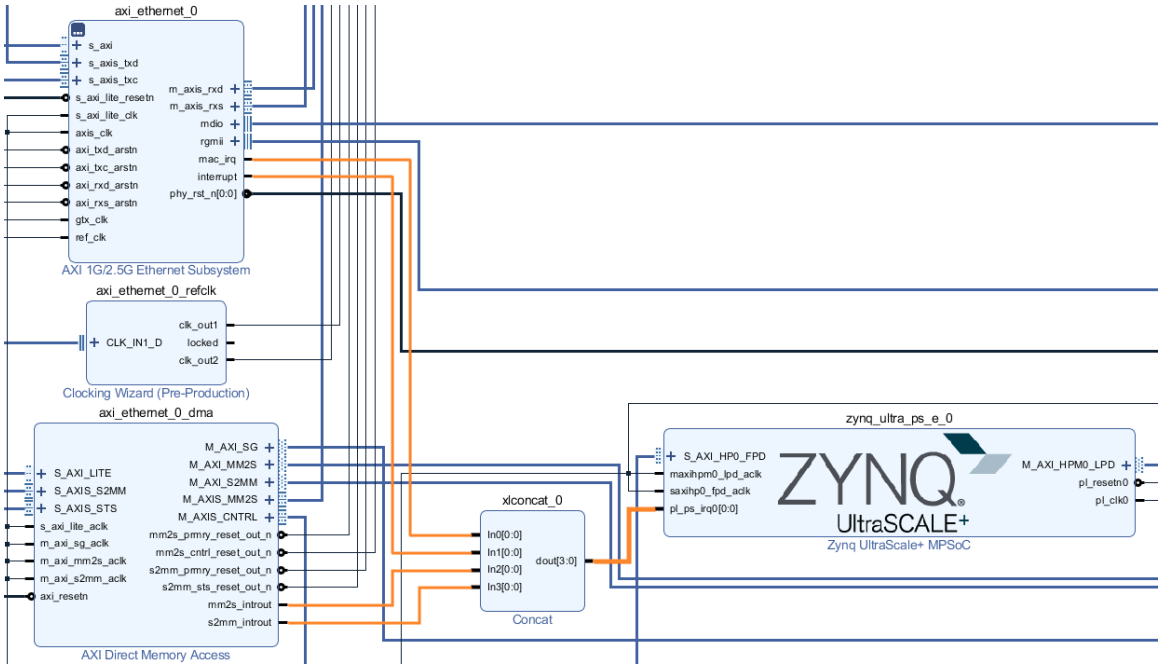
10. 添加 concat 模块



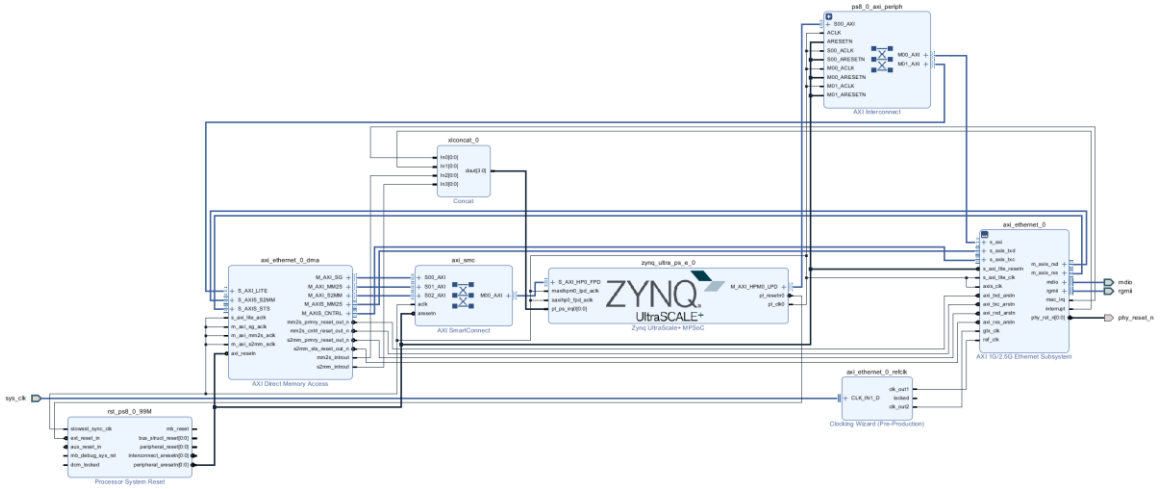
配置为 4



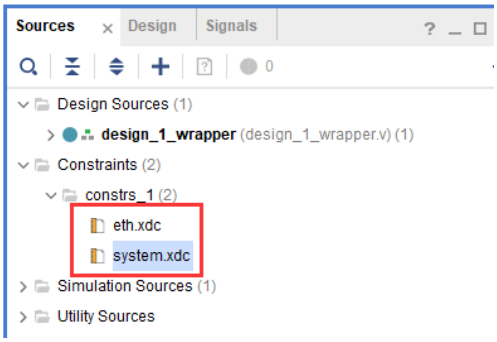
11. 连接中断信号



12. 修改引脚名称



13. 绑定引脚，并生成 bitstream，导出硬件信息



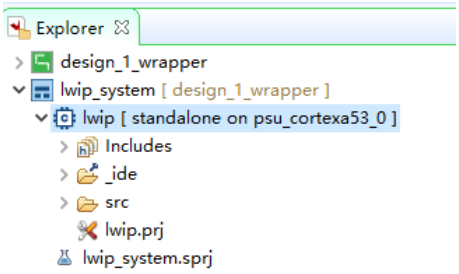
软件工程师工作内容

以下为软件工程师负责内容。

19.2 Vitis 程序开发

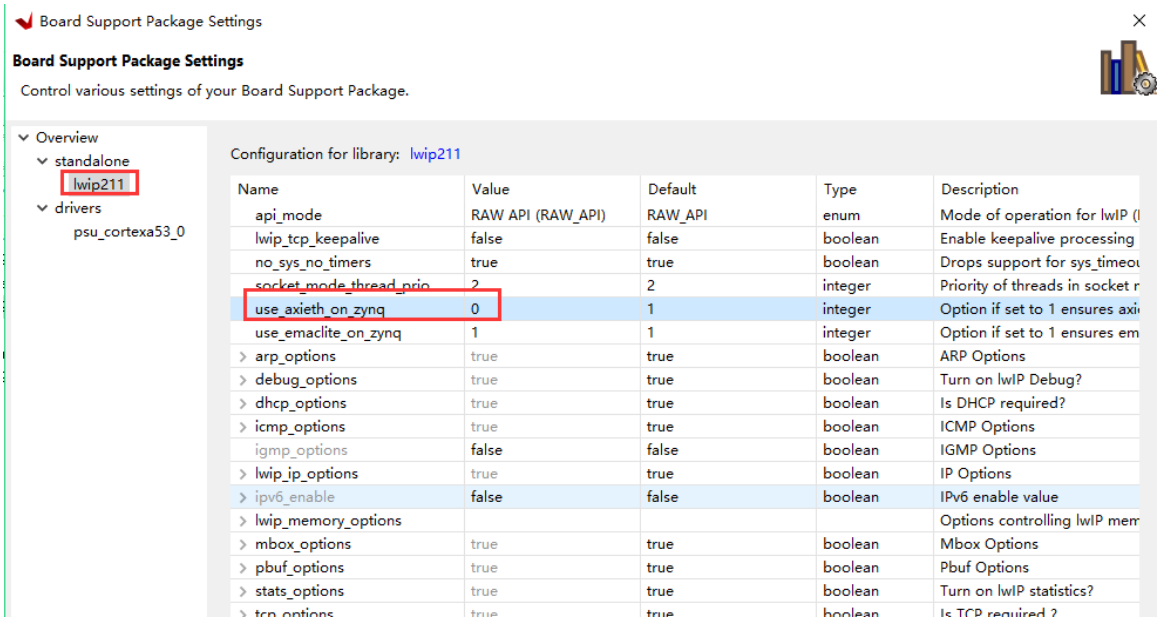
19.2.1 PL 端以太网测试

Vitis 工程建立方式与 PS 端是一样的，过程不再赘述，lwip 库在前面已经修改过。测试方法前面已经介绍过。

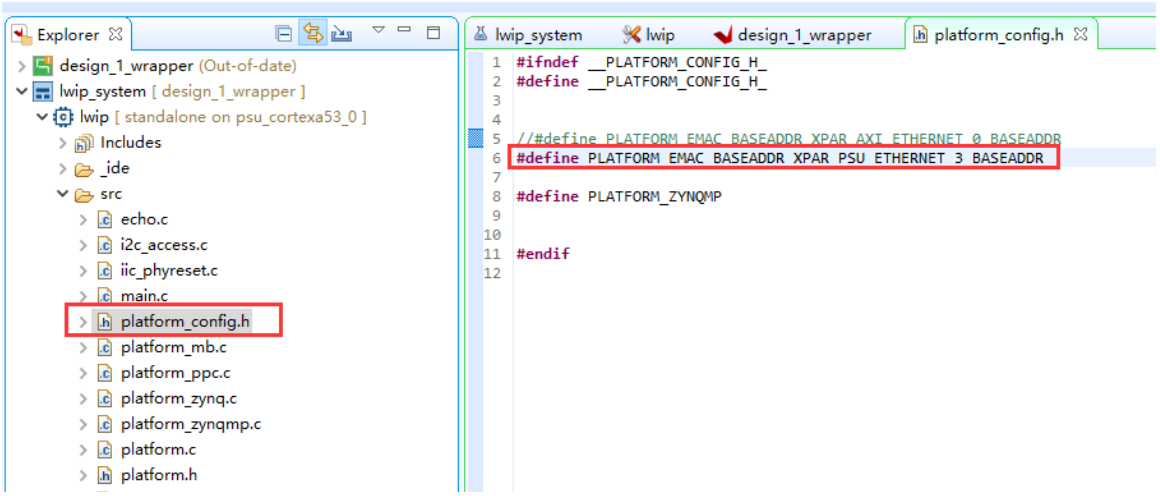


19.2.2 PS 端以太网测试

1. 如果想在这个工程测试 PS 端以太网，需要修改 BSP 设置，use_axieth_onzynq 默认为 1，也就是 PL 端的 AXI 以太网，将值改为 0，点击 OK



2. 修改 platform_config.h，重新编译即可


Explorer
design_1_wrapper (Out-of-date)
lwip_system [design_1_wrapper]
lwip [standalone on psu_cortexa53_0]
Includes
_ide
src
echo.c
i2c_access.c
iic_phyreset.c
main.c
platform_config.h
platform_mb.c
platform_ppc.c
platform_zynq.c
platform_zynqmp.c
platform.c
platform.h
lwip_system
lwip
design_1_wrapper
platform_config.h
#ifndef __PLATFORM_CONFIG_H_
#define __PLATFORM_CONFIG_H_
//#define PLATFORM_EMAC_BASEADDR XPAR_AXI_ETHERNET_0_BASEADDR
#define PLATFORM_EMAC_BASEADDR XPAR_PSU_ETHERNET_3_BASEADDR
#define PLATFORM_ZYNQMP
#endif

第二十章 自定义 IP 实验

实验 Vivado 工程目录为“custom_pwm_ip /vivado”。

实验 vitis 工程目录为“custom_pwm_ip /vitis”。

Xilinx 官方为大家提供了很多 IP 核，在 Vivado 的 IP Catalog 中可以查看这些 IP 核， 用户在构建自己的系统中，不可能只使用 Xilinx 官方的免费 IP 核，很多时候需要创建属于自己的用户 IP 核，创建自己的 IP 核有很多好处，例如系统设计定制化；设计复用，可以在在 IP 核中加入 license，有偿提供给别人使用；简化系统设计和缩短设计时间。用 ZYNQ 系统设计 IP 核，最常用的就是使用 AXI 总线将 PS 同 PL 部分的 IP 核连接起来。本实验将为大家介绍如何在 Vivado 中构建 AXI 总线类型的 IP 核，此 IP 核用来产生一个 PWM，用这个控制开发板上的 LED，做一个呼吸灯的效果。

FPGA 工程师工作内容

以下为 FPGA 工程师负责内容。

20.1 PWM 介绍

我们经常使用 PWM 来控制 LED，蜂鸣器等，通过调节脉冲的占空比来调节 LED 的亮度。在其他开发板中我们使用过的一个 pwm 模块如下：

```
//////////////////////////////////////////////////////////////////////////////////
//                                                                              //
//                                                                              //
//  Author: meisq                                                               //
//          msq@qq.com                                                          //
//          ALINX(shanghai) Technology Co.,Ltd                                  //
//          heijin                                                              //
//     WEB: http://www.alinx.cn/                                                //
//     BBS: http://www.heijin.org/                                              //
//                                                                              //
//////////////////////////////////////////////////////////////////////////////////
//                                                                              //
// Copyright (c) 2017,ALINX(shanghai) Technology Co.,Ltd                        //
//                    All rights reserved                                       //
//                                                                              //
// This source file may be used and distributed without restriction provided    //
// that this copyright statement is not removed from the file and that any      //
// derivative work contains the original copyright notice and the associated    //
// disclaimer.                                                                  //
//                                                                              //
//////////////////////////////////////////////////////////////////////////////////

//================================================================================
//   Description:  pwm model
//   pwm out period = frequency(pwm_out) * (2 ** N) / frequency(clk);
//
//================================================================================
//  Revision History:
//  Date          By          Revision    Change Description
//--------------------------------------------------------------------------------
//  2017/5/3     meisq         1.0         Original
//*******************************************************************************/
`timescale 1ns / 1ps
module ax_pwm
#(
  parameter N = 32 //pwm bit width
)
(
```

```
    input       clk,
    input       rst,
    input[N - 1:0]period,
    input[N - 1:0]duty,
    output      pwm_out
    );

reg[N - 1:0] period_r;
reg[N - 1:0] duty_r;
reg[N - 1:0] period_cnt;
reg pwm_r;
assign pwm_out = pwm_r;
always@(posedge clk or posedge rst)
begin
    if(rst==1)
    begin
        period_r <= { N {1'b0} };
        duty_r <= { N {1'b0} };
    end
    else
    begin
        period_r <= period;
        duty_r   <= duty;
    end
end

always@(posedge clk or posedge rst)
begin
    if(rst==1)
        period_cnt <= { N {1'b0} };
    else
        period_cnt <= period_cnt + period_r;
end

always@(posedge clk or posedge rst)
begin
    if(rst==1)
    begin
        pwm_r <= 1'b0;
    end
    else
    begin
        if(period_cnt >= duty_r)
            pwm_r <= 1'b1;
        else
            pwm_r <= 1'b0;
    end
end

endmodule
```

可以看到这个 PWM 模块需要 2 个参数“period”、“duty”来控制频率和占空比，”period”为步进值，也就是计数器每个周期要加的值。Duty 为占空比的值。我们需要设计一些寄存器来控制这些参数，这里需要使用 AXI 总线，PS 通过 AXI 总线来读写寄存器。

$$\text{PWM 频率} = \frac{period}{2\text{^}N} \times \text{clk 频率} \quad \text{(单位为 Hz)}$$

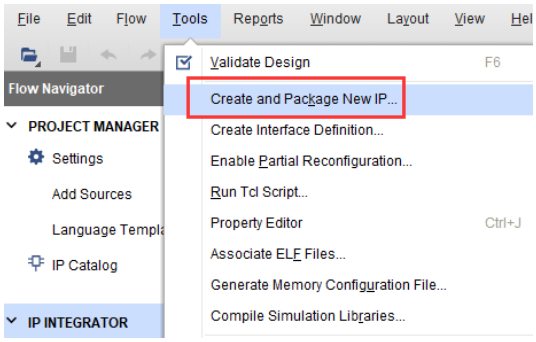
$$\text{PWM 占空比} = 1 - \frac{duty+1}{2\text{^}N}$$

20.2 Vivado 工程建立

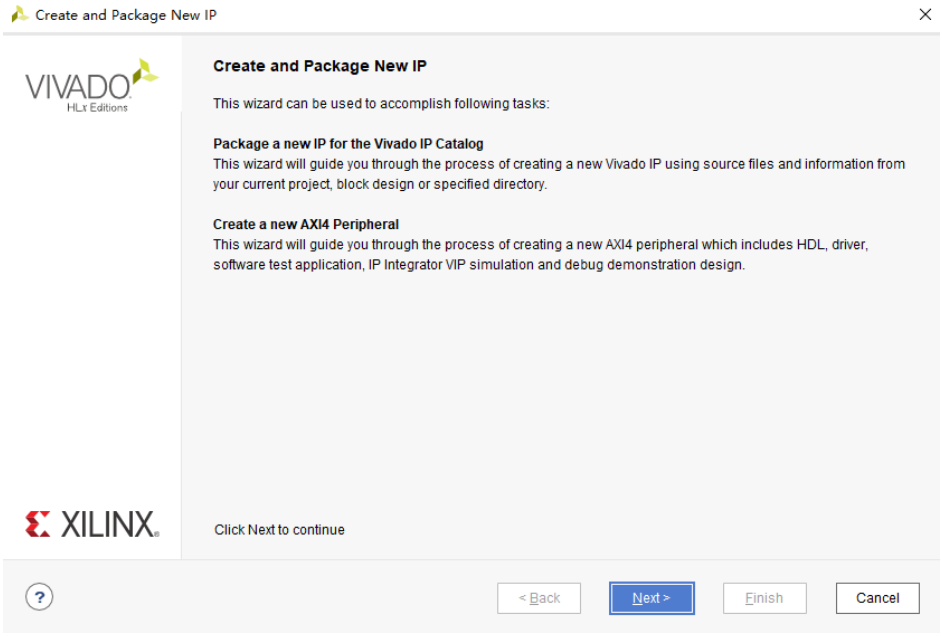
用”ps_hello”工程另存为一个名为“custom_pwm_ip”工程

20.2.1 创建自定义 IP

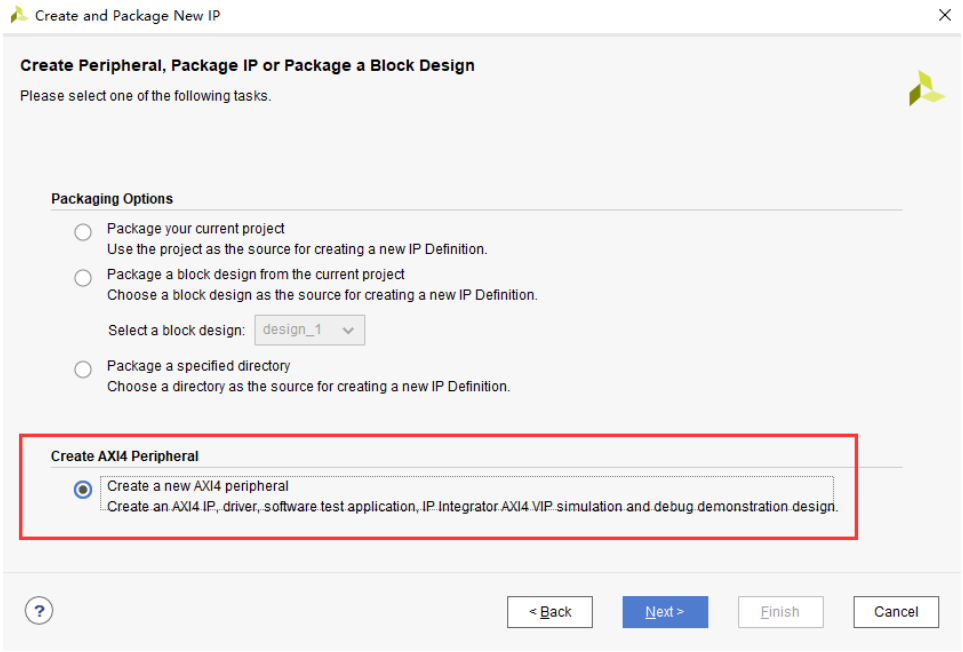
1) 点击菜单“Tools->Create and Package IP...”



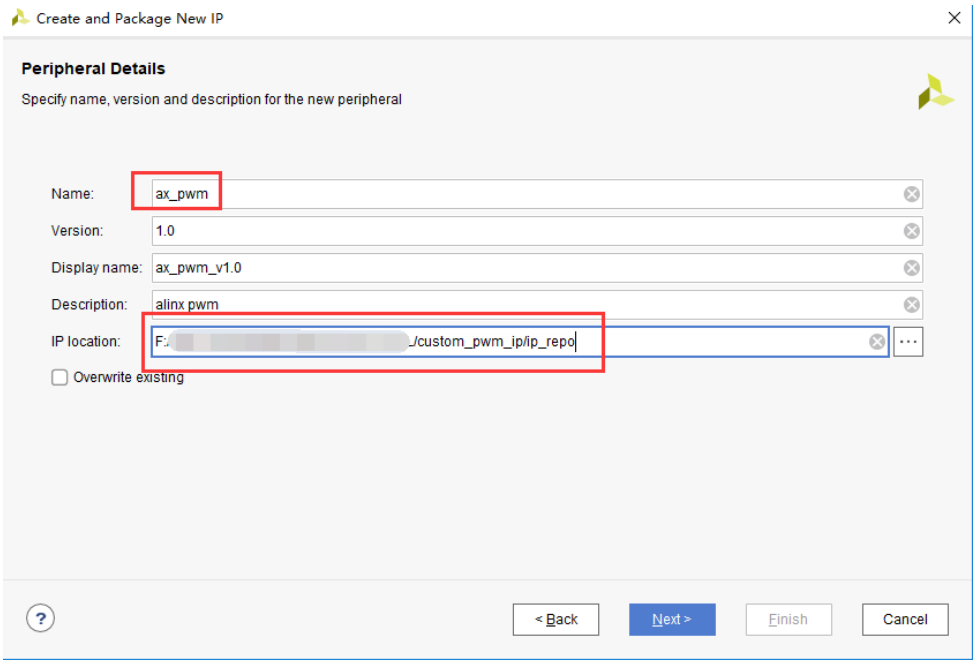
2) 选择“Next”



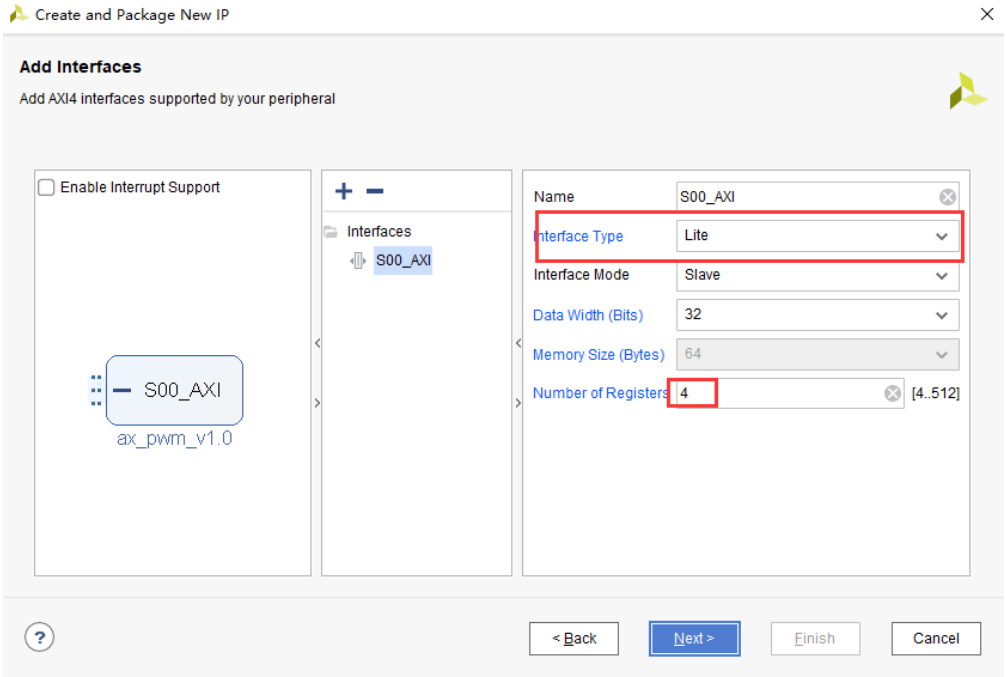
3) 选择创建一个新的 AXI4 设备



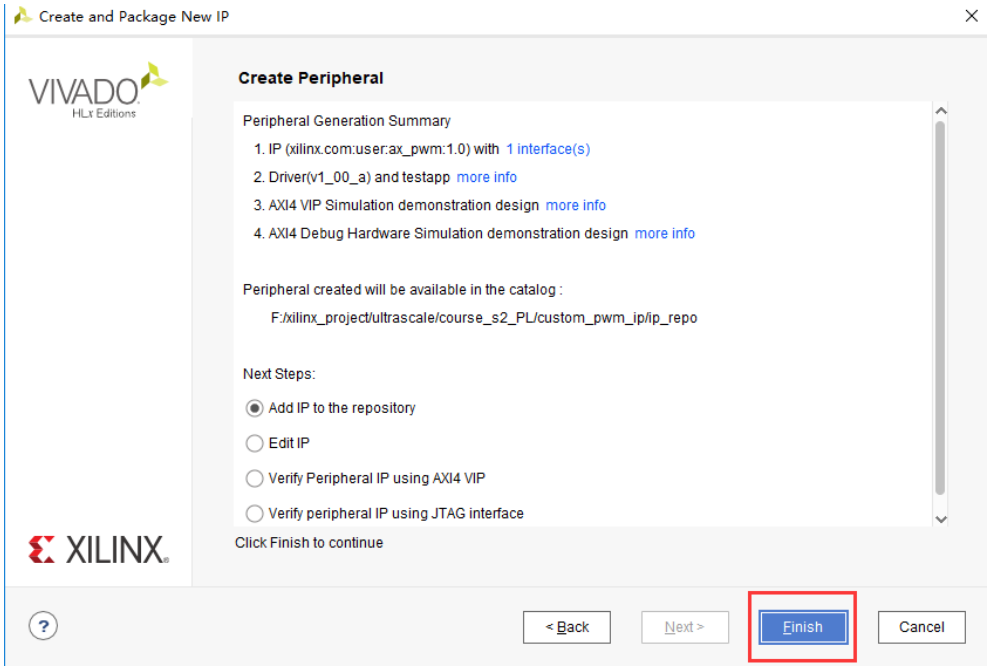
4) 名称填写“ax_pwm”,描述填写“alinx pwm”，然后选择一个合适的位置用来放 IP



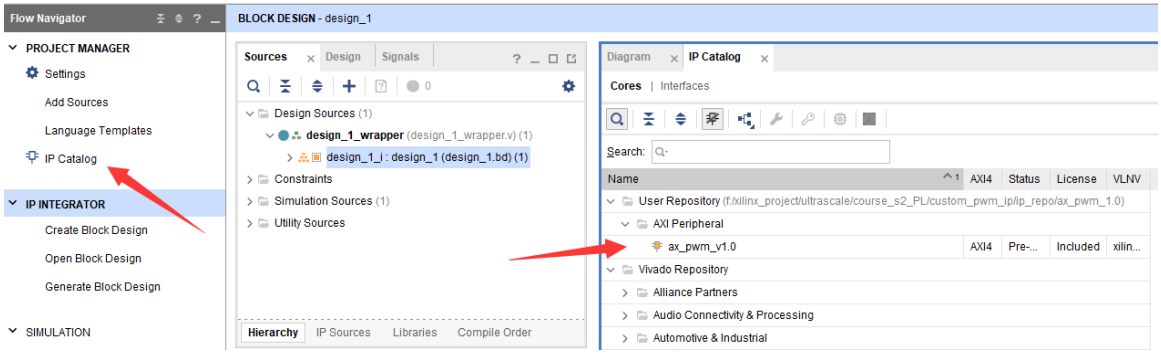
5) 下面参数可以指定接口类型、寄存器数量等，这里不需要修改，使用 AXI Lite Slave 接口，4 个寄存器。



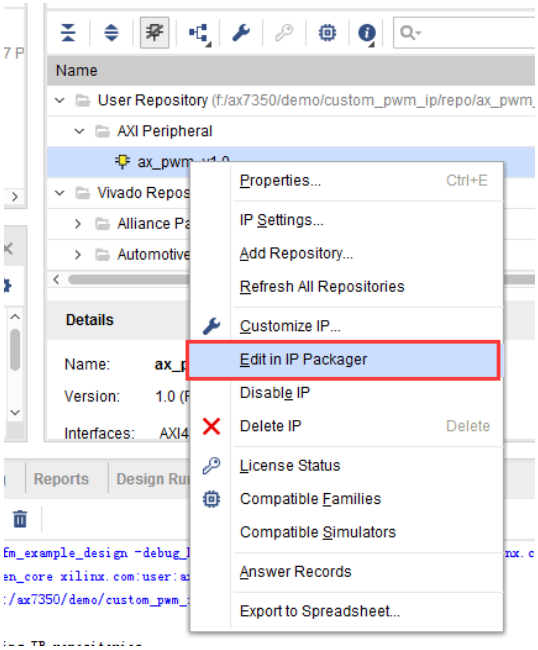
6) 点击“Finish”完成 IP 的创建



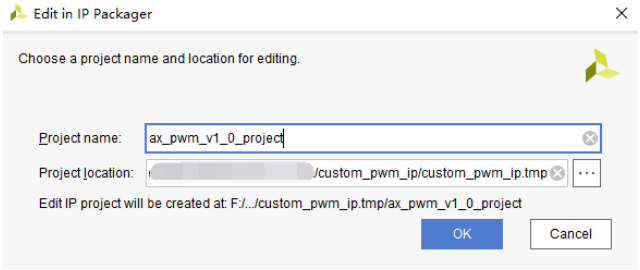
7) 在“IP Catalog”中可以看到刚才创建的 IP



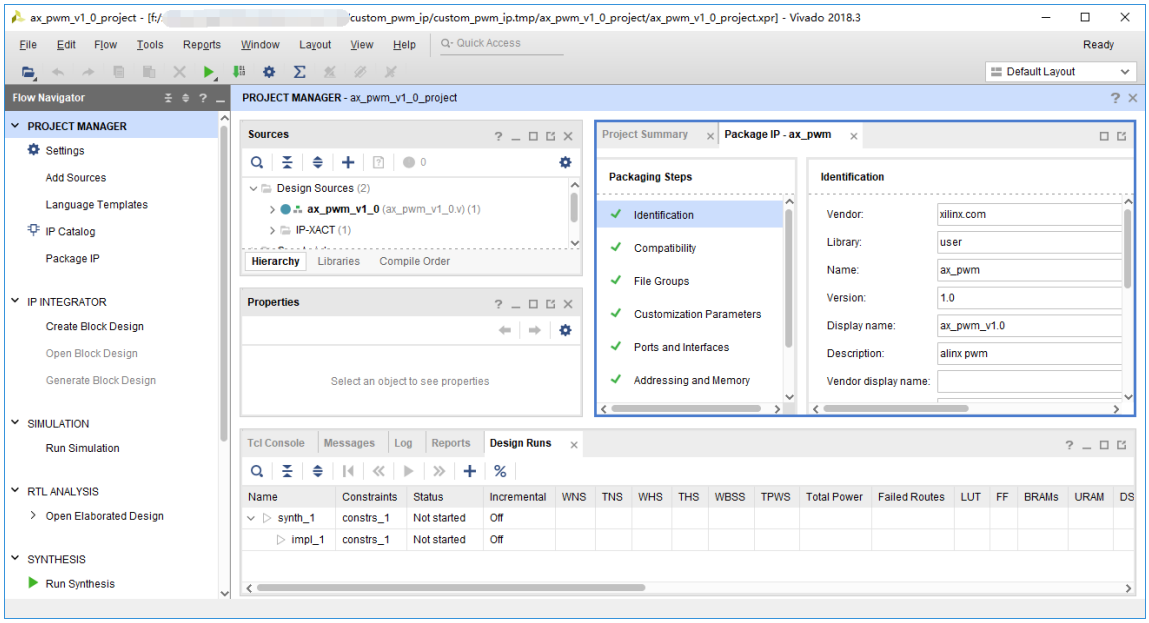
8) 这个时候的 IP 只有简单的寄存器读写功能，我们需要修改 IP，选择 IP，右键“Edit in IP Packager”



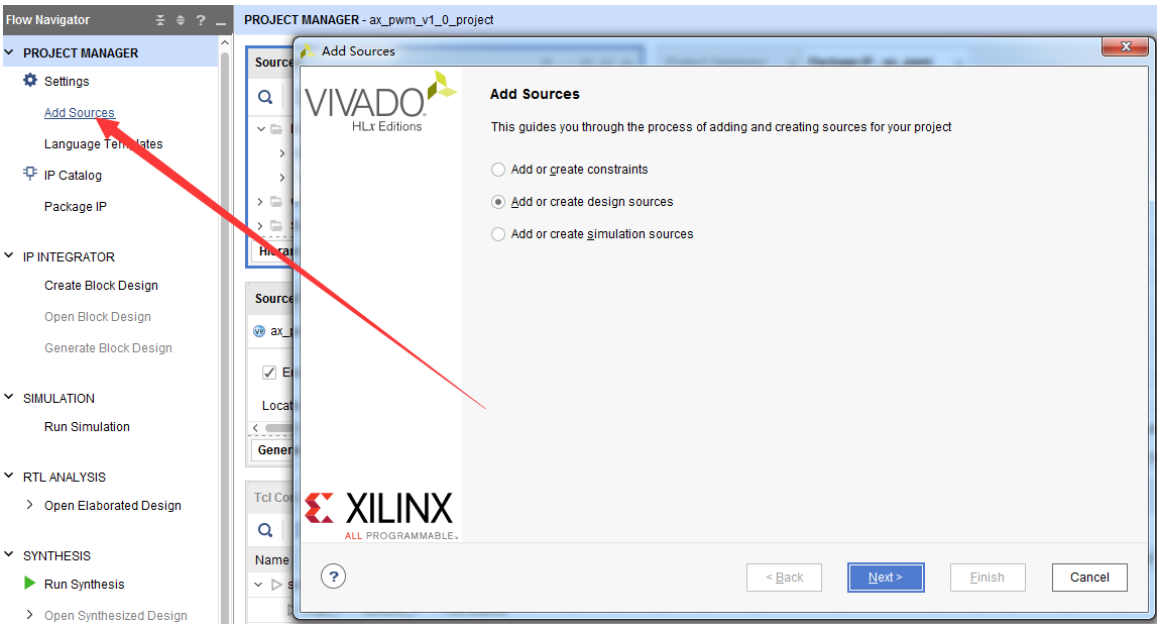
9) 这是弹出一个对话框，可以填写工程名称和路径，这里默认，点击“OK”



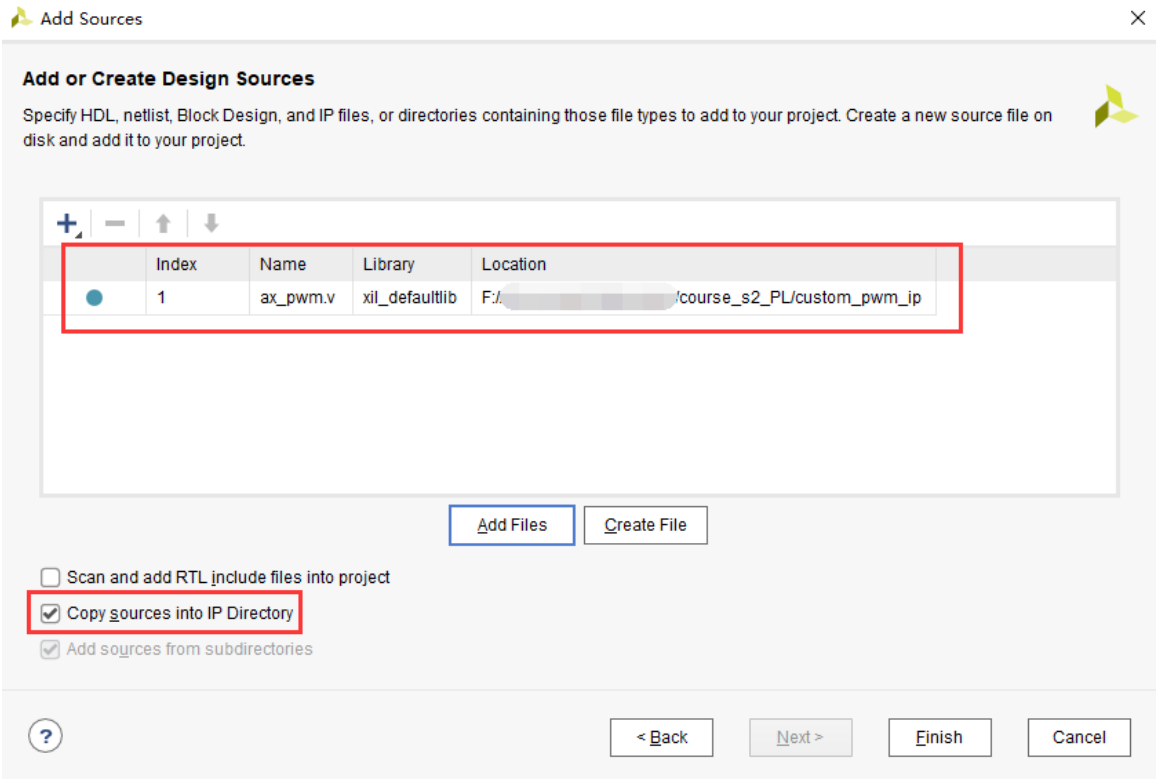
10) Vivado 打开了一个新的工程



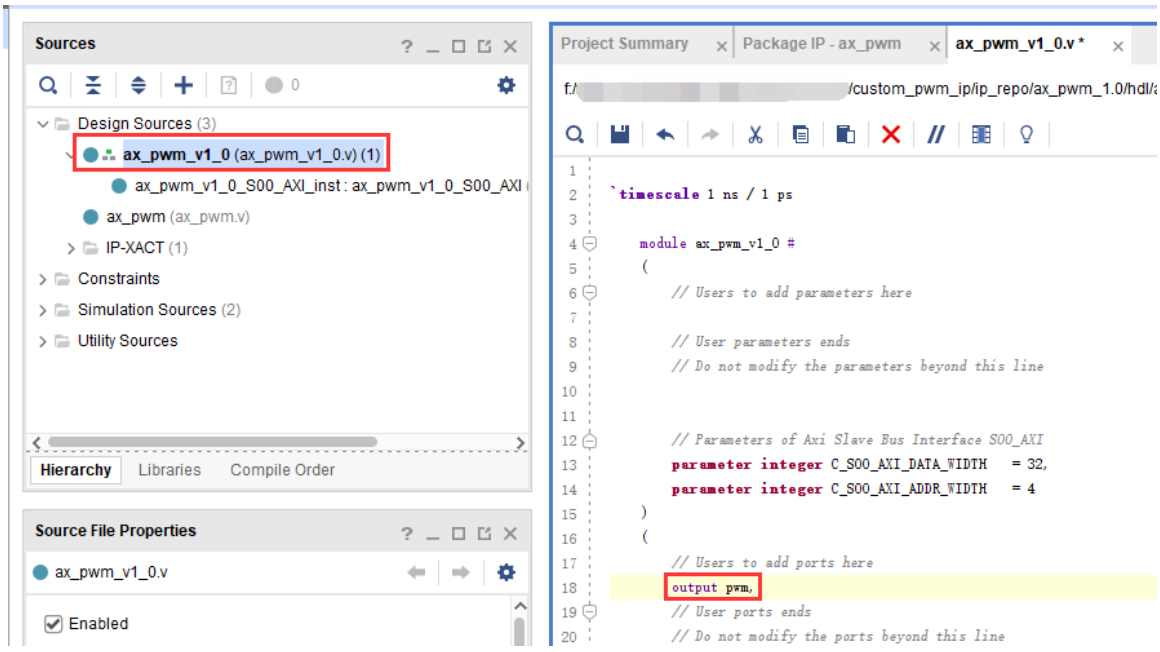
11) 添加 PWM 功能的核心代码



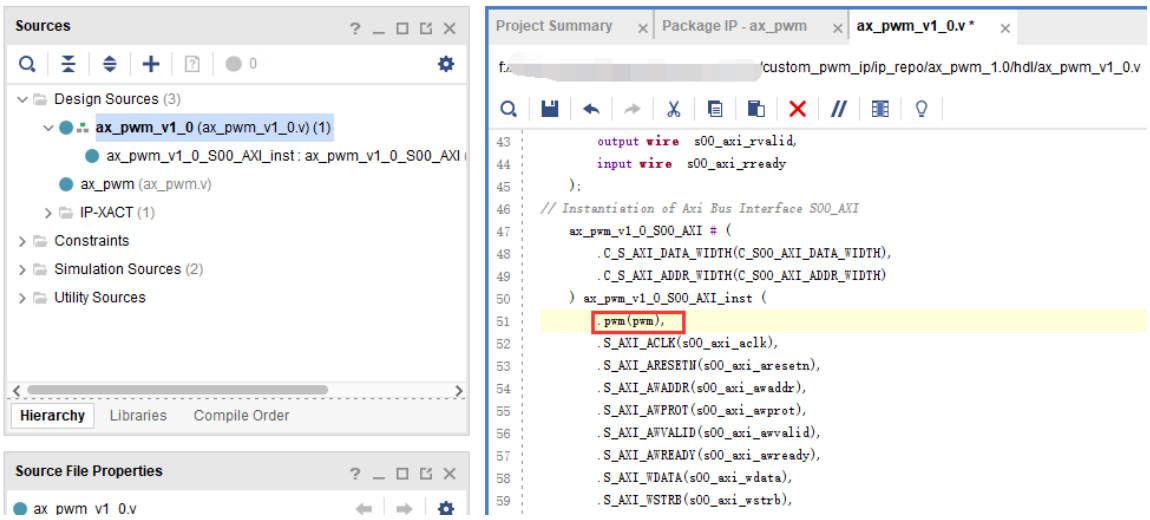
12) 添加代码时选择复制代码到 IP 目录



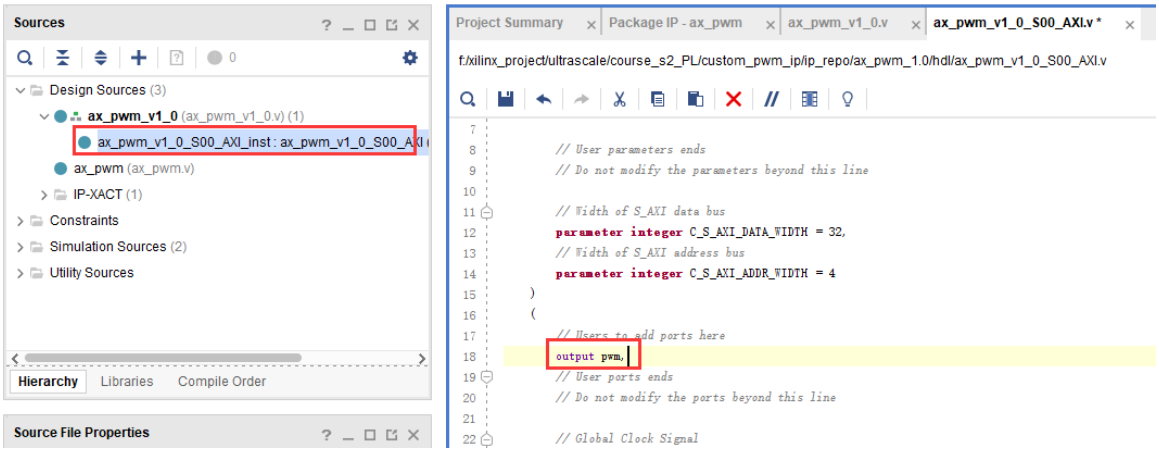
13) 修改“ax_pwm_v1_0.v”，添加一个 pwm 输出端口



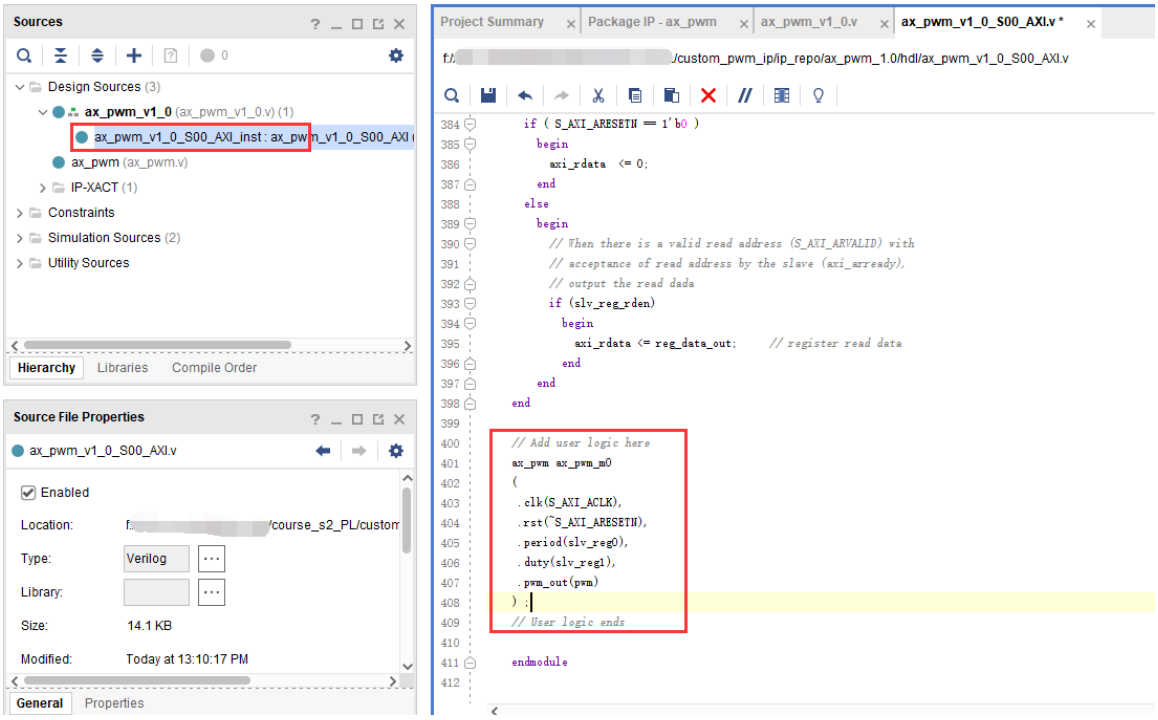
14) 修改“ax_pwm_v1_0.v”，在例化“ax_pwm_V1_0_S00_AXI”,中添加 pwm 端口的例化



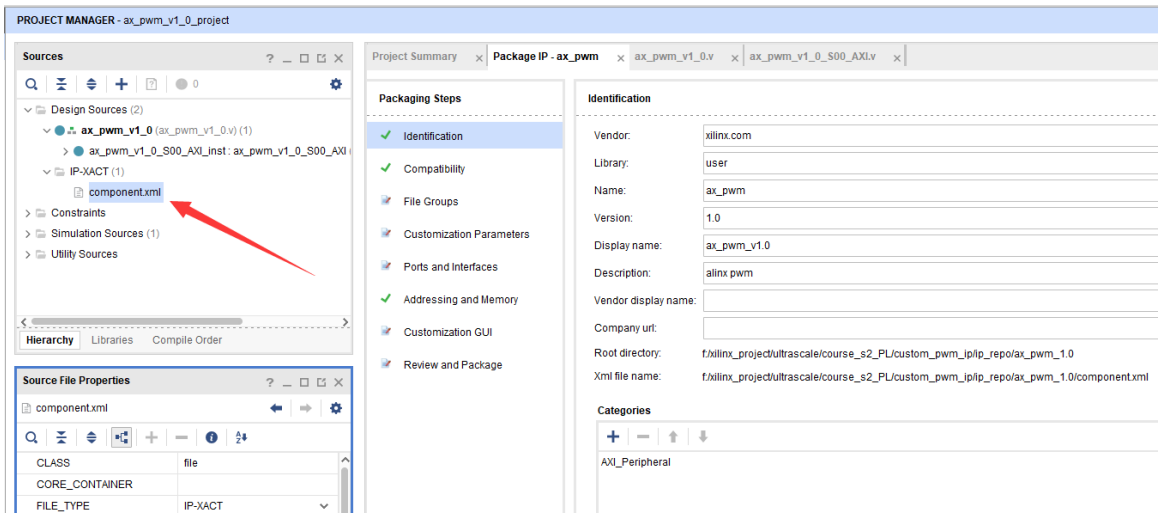
15) 修改“ax_pwm_v1_0_s00_AXI.v”文件，添加 pwm 端口，这个文件是实现 AXI4 Lite Slave 的核心代码



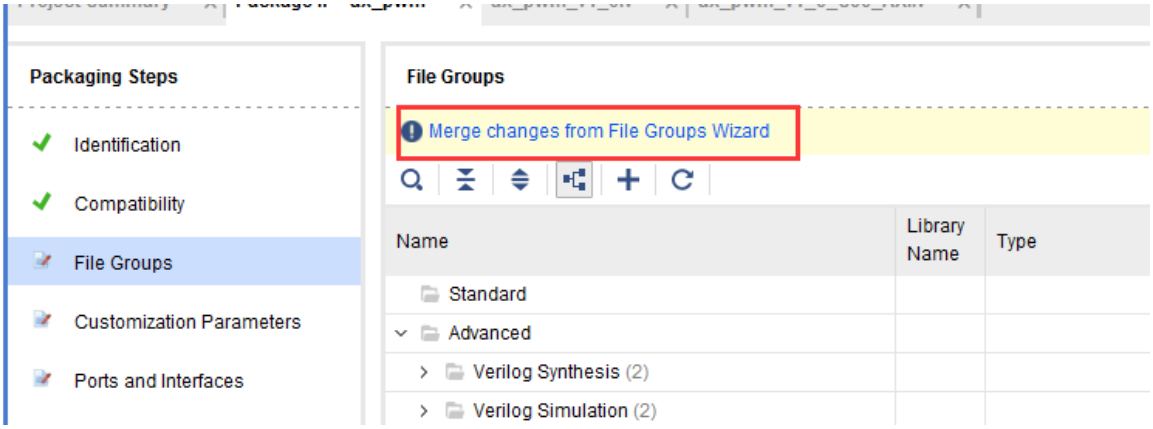
16) 修改“ax_pwm_v1_0_s00_AXI.v”文件，例化 pwm 核心功能代码，将寄存器 slv_reg0 和 slv_reg1 用于 pwm 模块的参数控制。



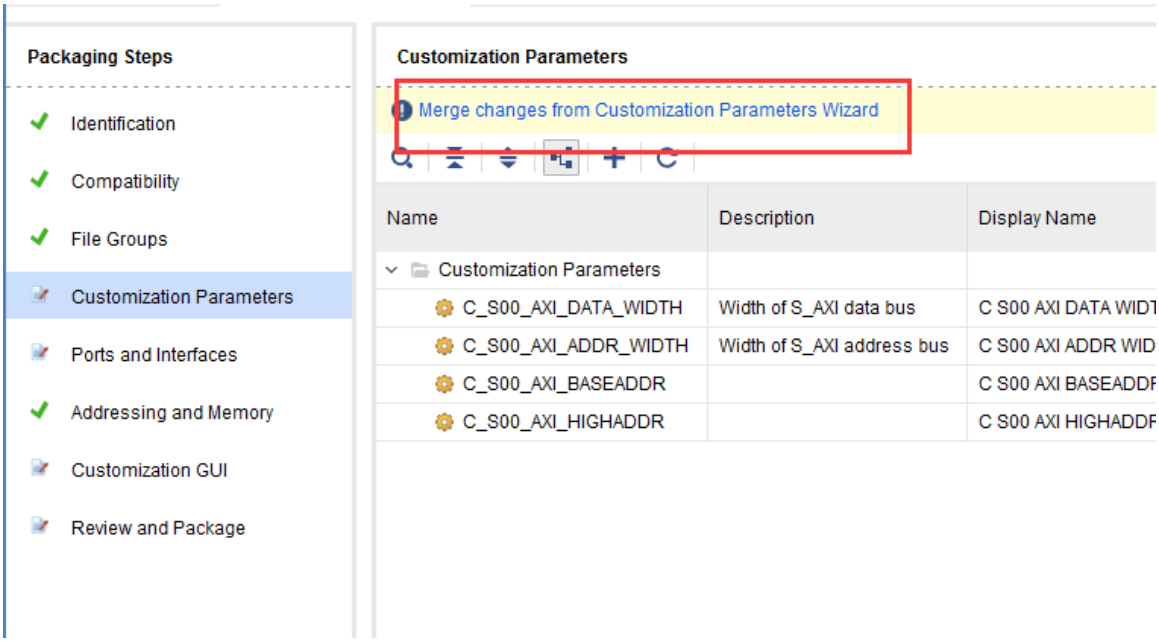
17) 双击“component.xml”文件



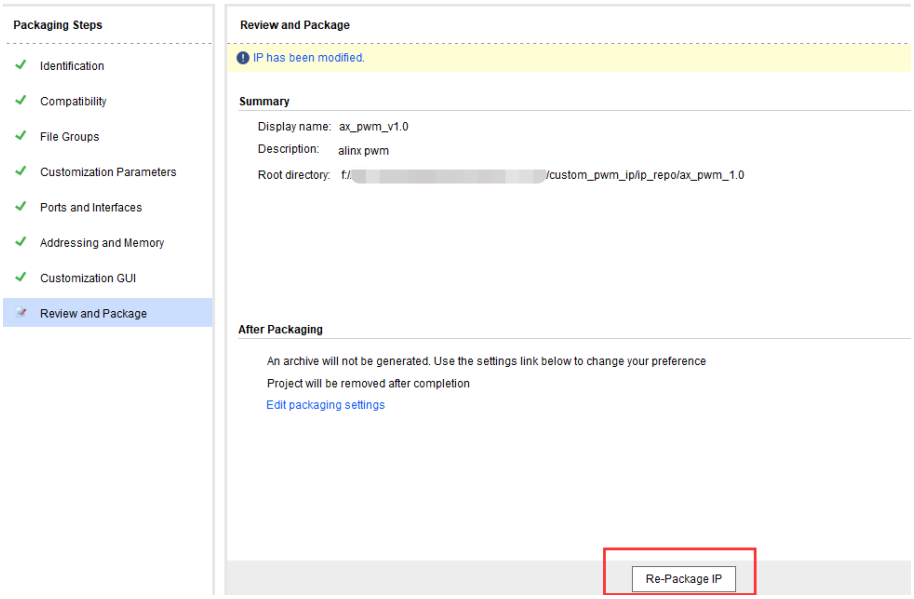
18) 在“File Groups”选项中点击“Merge changers from File Groups Wizard”



19) 在“Customization Parameters”选项中点击“Merge changes form Customization Parameters Wizard”

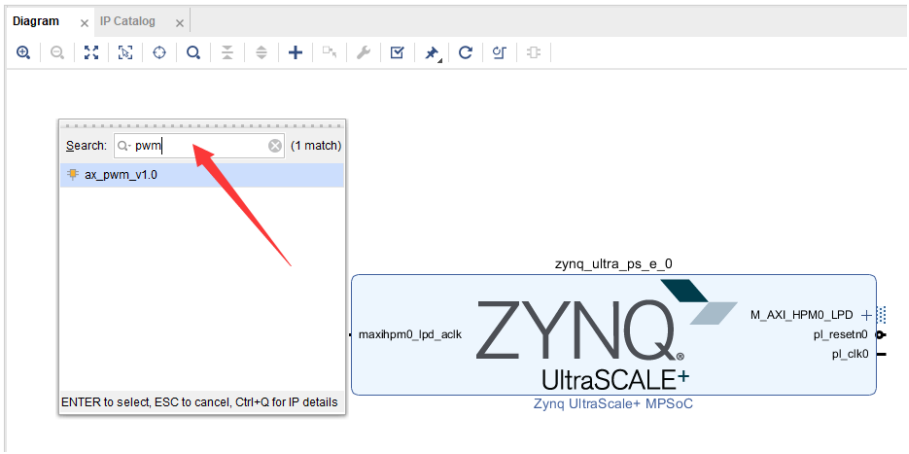


20) 点击“Re-Package IP”完成 IP 的修改

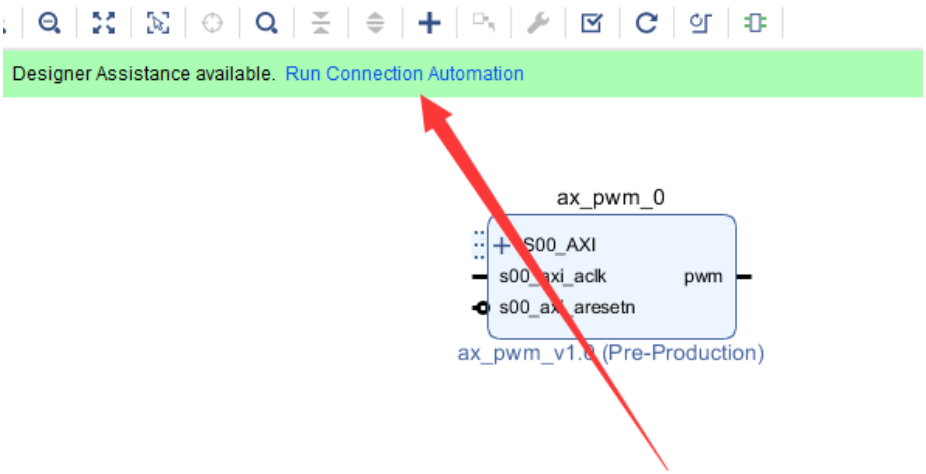


20.2.2 添加自定义 IP 到工程

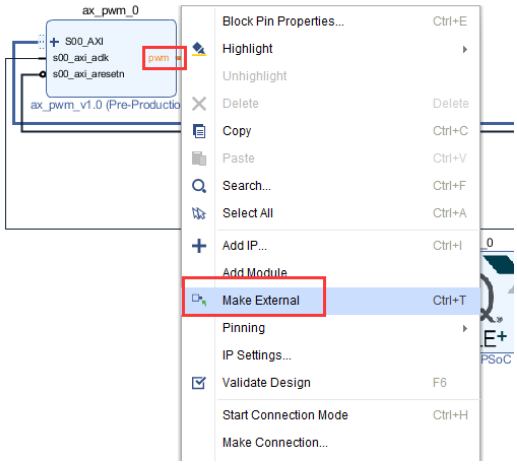
1) 搜索“pwm”，添加“ax_pwm_v1.0”

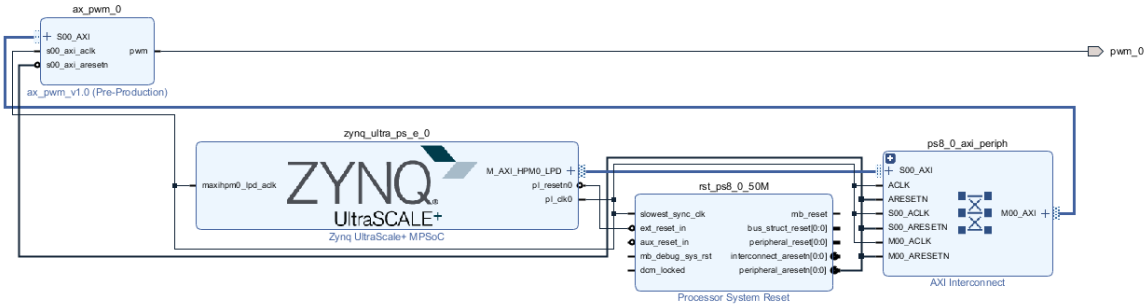


2) 点击“Run Connection Automation”

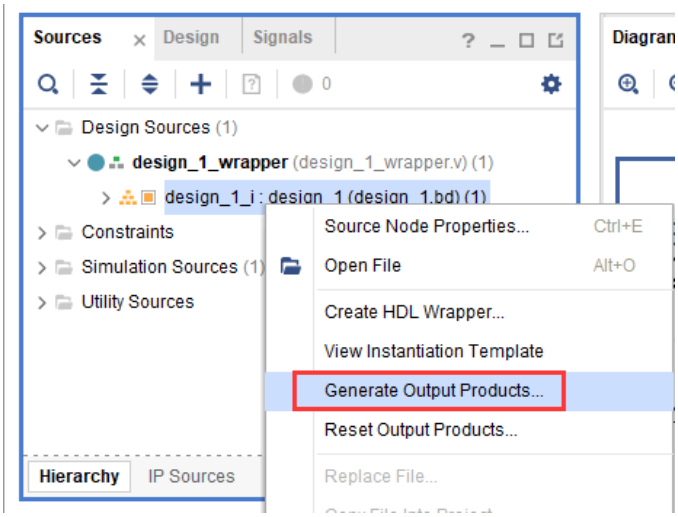


3) 导出 pwm 端口





4) 保存设计，并 Generate Output Products



5) 添加 xdc 文件分配管脚，把 pwm_0 输出端口分配给 PL LED，做一个呼吸灯

```
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]

set_property IOSTANDARD LVCMOS33 [get_ports pwm_0]
set_property PACKAGE_PIN K9 [get_ports pwm_0]
```

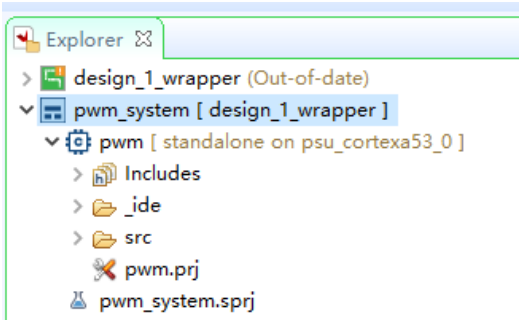
6) 编译生成 bit 文件，导出硬件

软件工程师工作内容

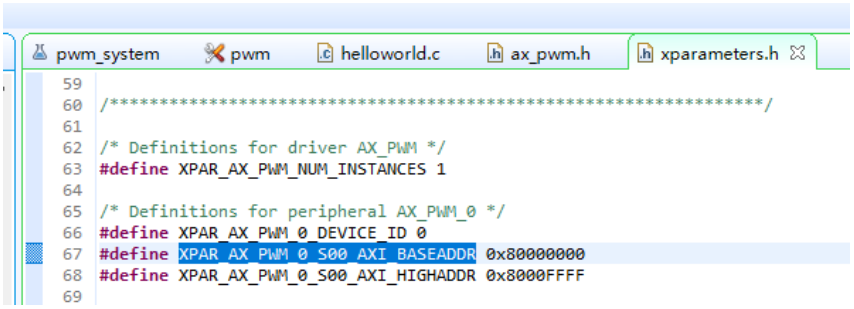
以下为软件工程师负责内容。

20.3 Vitis 软件编写调试

1) 启动 Vitis，新建 APP，模板选择“Hello World”



2) 在 bsp 里找到“xparameters.h”文件，这个非常重要的文件，里面找到了自定 IP 的寄存器基地址，可以找到自定义 IP 的基地址。



3) 有个寄存器读写宏和自定义 IP 的基地址，我们开始编写代码，测试自定义 IP，我们先通过写寄存器 AX_PWM_S00_AXI_SLV_REG0_OFFSET，控制 PWM 输出频率，然后通过写寄存器 AX_PWM_S00_AXI_SLV_REG1_OFFSET 控制 PWM 输出的占空比。

```
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include "ax_pwm.h"
#include "xil_io.h"
#include "xparameters.h"
#include "sleep.h"

unsigned int duty;

int main()
{
    init_platform();

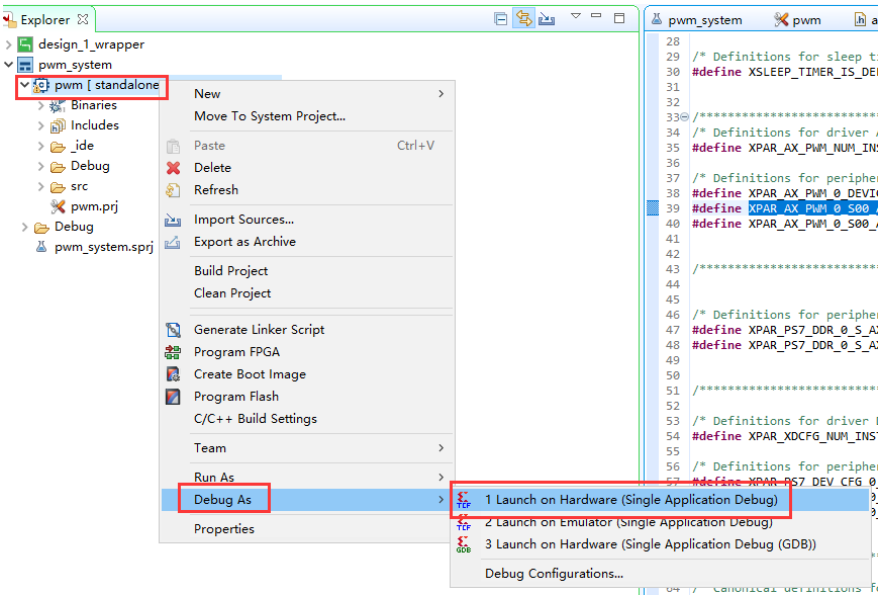
    print("Hello World\n\r");

    //pwm out period = frequency(pwm_out) * (2^N) / frequency(clk);
    AX_PWM_mWriteReg(XPAR_AX_PWM_0_S00_AXI_BASEADDR,
AX_PWM_S00_AXI_SLV_REG0_OFFSET, 17179);//200hz
    //duty = (2^N) * (1 - (duty cycle)) - 1
    while (1) {
        for (duty = 0x8fffffff; duty < 0xffffffff; duty = duty +
100000) {
            AX_PWM_mWriteReg(XPAR_AX_PWM_0_S00_AXI_BASEADDR,
AX_PWM_S00_AXI_SLV_REG1_OFFSET, duty);
            usleep(100);
        }
    }
```

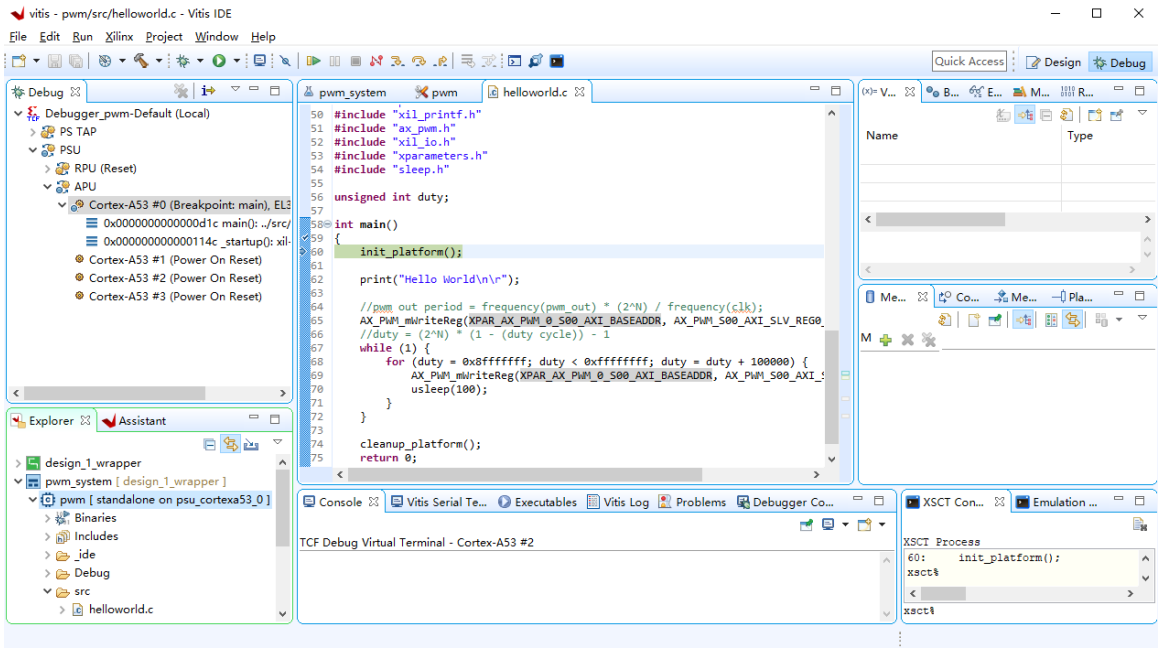
```
    cleanup_platform();
    return 0;
}
```

4) 通过运行代码，我们可以看到 PL LED 呈现出一个呼吸灯的效果。

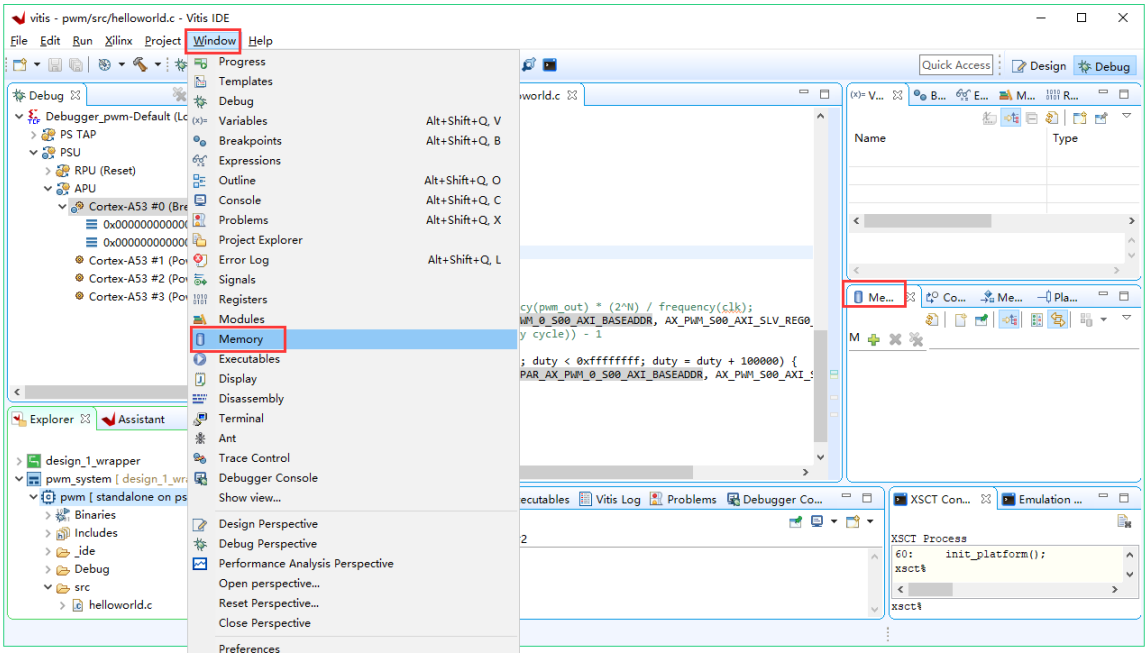
5) 通过 debug，我们来查看一下寄存器



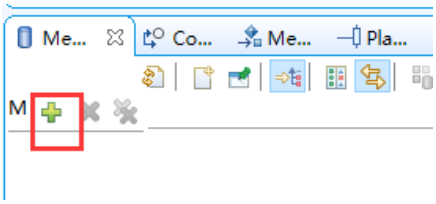
6) 进入 debug 状态，按“F6”可以单步运行。

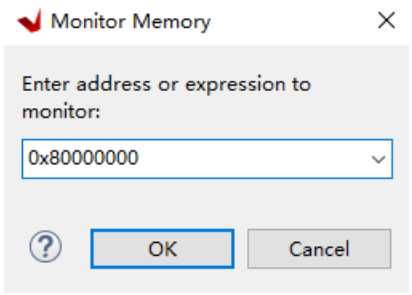


7) 通过菜单可以查看“Memory”窗口

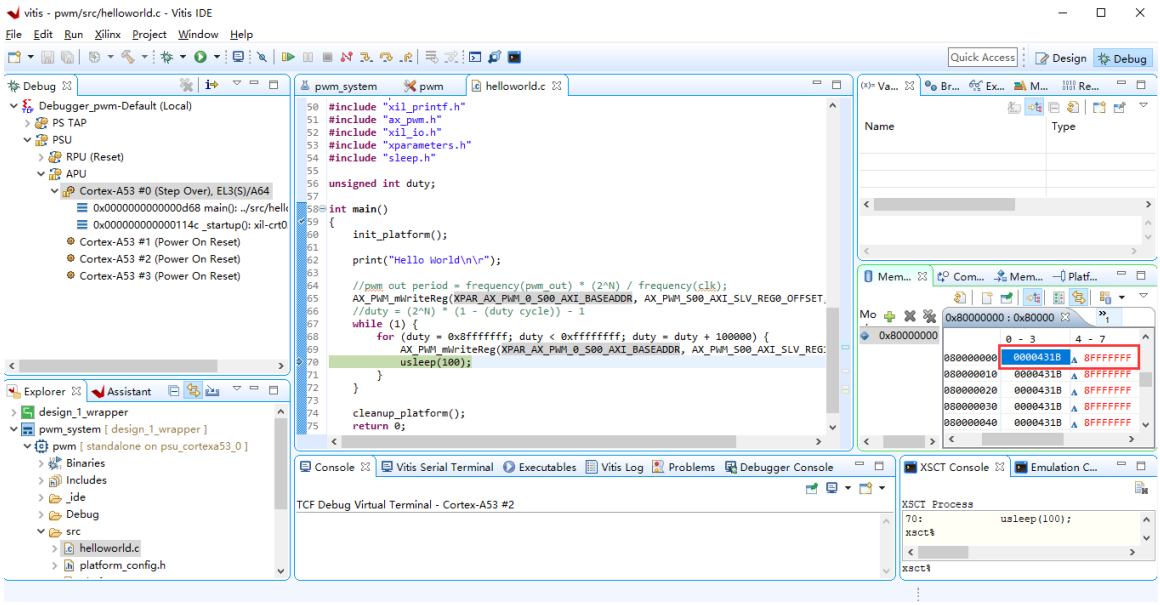


8) 添加一个监视地址“0x80000000”





9) 单步运行，观察变化



20.4 实验总结

通过本实验我们掌握了更多的 Vitis 调试技巧，掌握了 ARM + FPGA 开发的核心内容，就是 ARM 和 FPGA 数据交互。

第二十一章 双核 AMP 的使用

实验 Vivado 工程目录为“ps_axi_gpio /vivado”。
实验 vitis 工程目录为“dualcore_amp /vitis”。

前面的例程都是采用的单核 CPU，在有些情况下，比如多任务处理，并行处理等，需要用到双核 CPU，本章将简单介绍双核的使用方法。并将实现以下功能：

1. CPU0 实现 PS 端按键中断，控制 PS 端 LED 的亮灭，并向 CPU1 发出软件中断，让 CPU1 打印 CPU0 内存空间的一串字符
2. CPU1 实现 PL 端按键中断，控制 PL 端 LED 的亮灭，并向 CPU0 发出软件中断，让 CPU0 打印 CPU1 内存空间的一串字符
3. 内存空间的划分，共享内存空间的使用
4. FSBL 启动 Flash

Vivado 利用“PL 端 AXI GPIO 的使用”的工程。

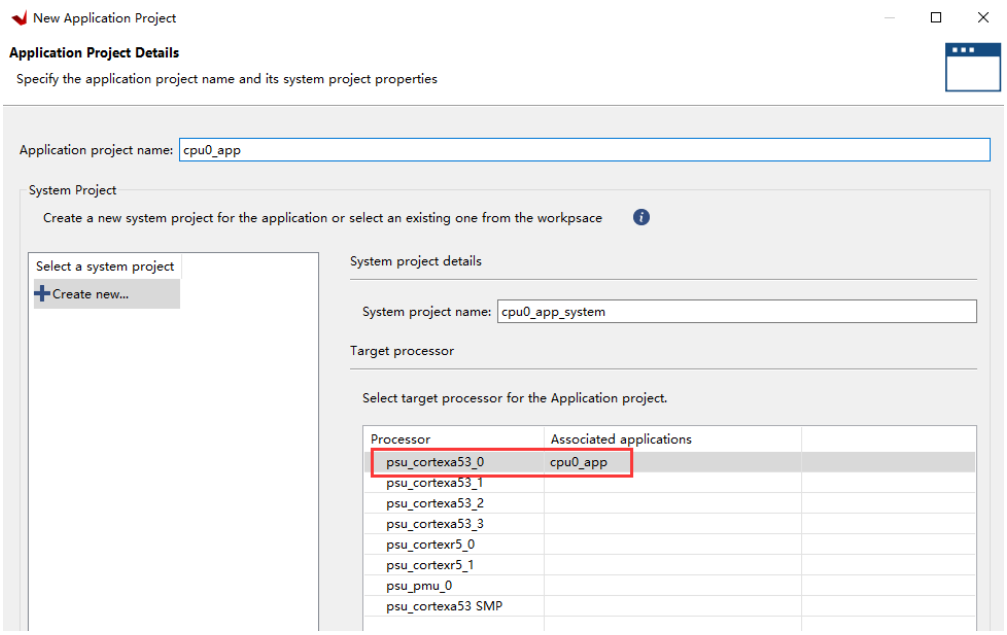
软件工程师工作内容

以下为软件工程师负责内容。

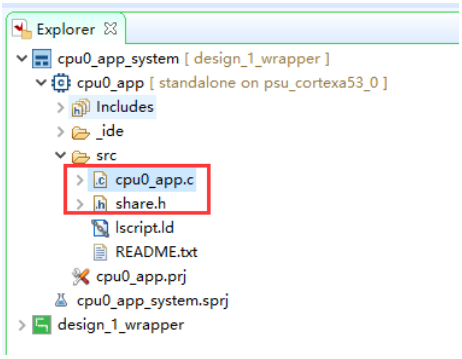
21.1 Vitis 程序开发

21.1.1 CPU0 Vitis 工程建立

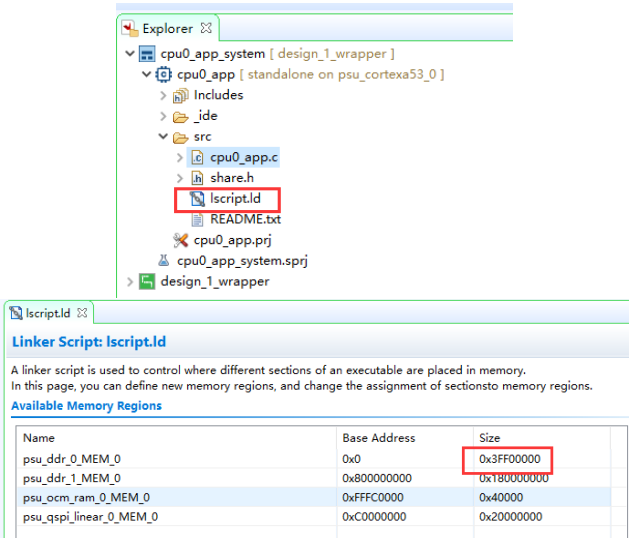
1. 新建工程，注意 CPU 选择 ps7_cortexa53_0，也就是 CPU0



2. 已经为大家准备好代码，cpu0_app.c 和 share.h，share.h 内包含共享内存结构体，后面会讲到。

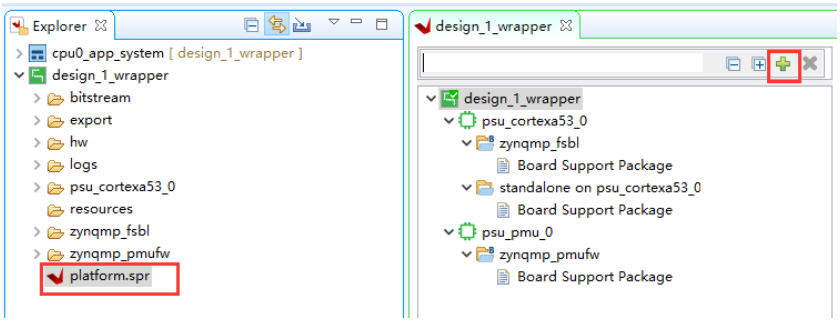


3. 在 lscript.ld 里设置 CPU0 的访问空间，例如 psu_ddr_0_MEM_0 为 2GByte，将 CPU0 空间设置为一半，当然也可以根据需要修改。

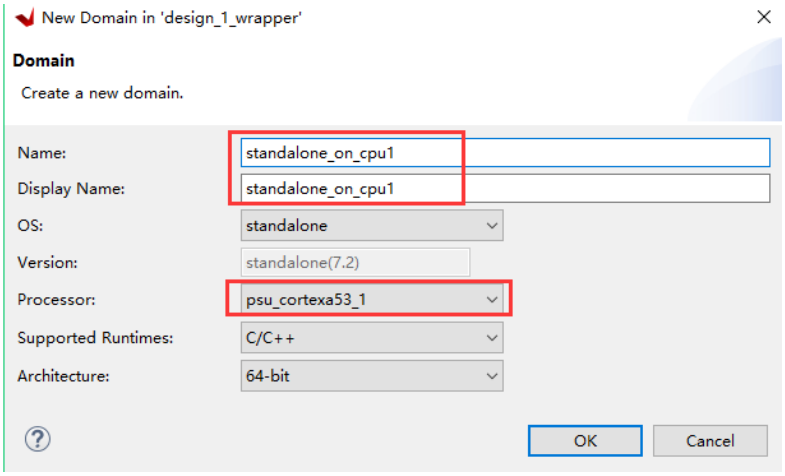


21.1.2 CPU1 Vitis 工程建立

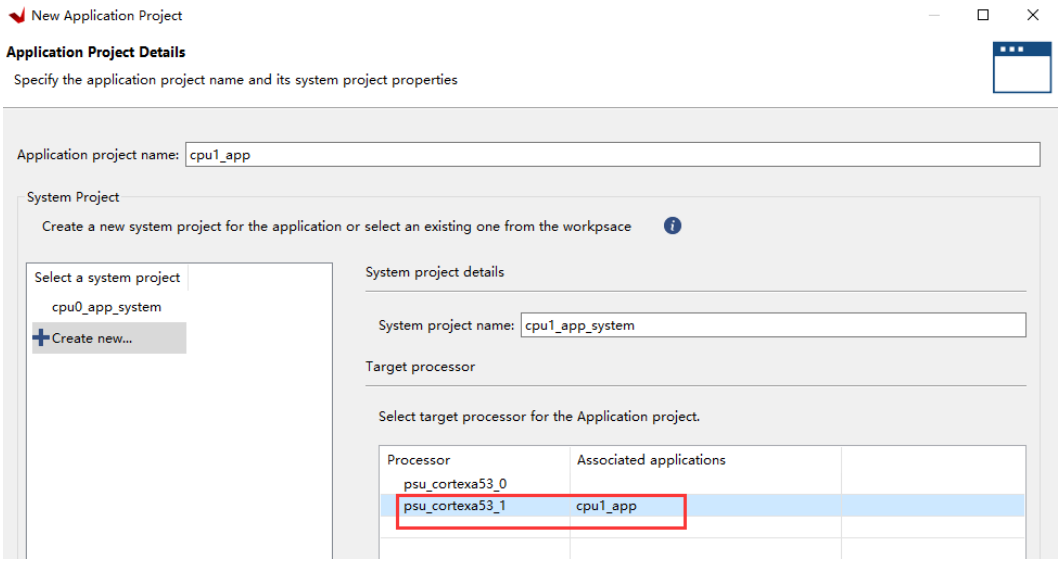
1. 在新建 CPU1 的 APP 工程之前，我们可以先新建一个基于 CPU1 的 Domain，也就是所谓的 BSP，在 platform.spr 中点击“+”



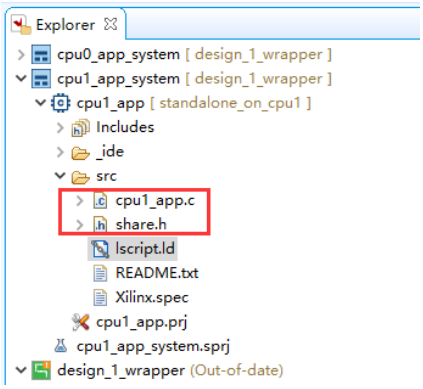
2. 填入名称，并且 Processor 选择 ps7_cortexa9_1，也就是 CPU1，点击 OK



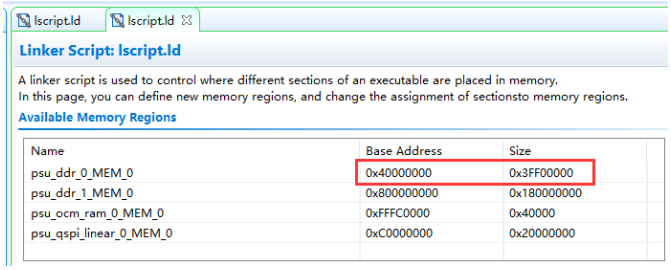
3. 新建 CPU1 工程的时候，选择 CPU1



4. 同样也准备了代码，cpu1_app.c 和 share.h



5. 设置 CPU1 内存空间，注意不要与 CPU0 重合，中间保留了 0x3FF00000~0x3FFFFFFF 的空间，用于共享内存



编译 CPU0 和 CPU1 的 APP 工程

21.1.3 CPU0 程序介绍

1. 软件中断使用 ID 号 1 和 2。

```
u16 SoftIntrIdToCpu0 = 1 ;
u16 SoftIntrIdToCpu1 = 2 ;
```

并连接中断号 1 到软件中断服务函数。

```
InterruptConnnect(&InterrruptInst, SoftIntrIdToCpu0, (void *)SoftHandler, (void *)&InterrruptInst) ;
```

2. 在 while 循环语句中，将字符数组的地址和长度赋给共享结构体，这里要提一下共享内存结构体，在 share.h 中定义了结构体 ShareMem，用于在共享内存中传递信息。

```
while(1)
{
    if (key_flag)
    {
        /*
         * initial shared Struct
         */
        SharePtr->addr = (unsigned int *)&Cpu0_Data ;
        SharePtr->length = sizeof(Cpu0_Data) ;
        /*
         * Write led value
         */
        XGpioPs_WritePin(&GPIO_PTR, 9, LedVal) ;
        LedVal = ~LedVal ;
        /*
         * Software interrupt to CPU1
         */
        XScuGic_SoftwareIntr(&InterrruptInst, SoftIntrIdToCpu1, CPU1) ;
        key_flag = 0 ;
    }
```

```
typedef struct
{
    unsigned int length;
    unsigned int *addr;
}ShareMem;
```

并且双核约定好共享地址，这样就能传递参数。

```
#define SHARE_BASE  0x3FFFFF00
```

3. 在 while 循环中判断有来自 CPU1 的软件中断，打印出来 CPU1 内存空间中的字符串。

```
else if (soft_flag)
{
    /*
     * When Software interrupt, print data in CPU1
     */
    Cpu1Data = (unsigned char *)SharePtr->addr ;
    xil_printf("This is CPU0, Now Start to Print:\r\n") ;
    xil_printf("%s\r\n", Cpu1Data) ;
    soft_flag = 0 ;
}
```

21.1.4 CPU1 程序介绍

1. 在 CPU1 程序中同样有一个字符数组，Cpu0Data 指向 CPU0 内存空间的字符串地址。

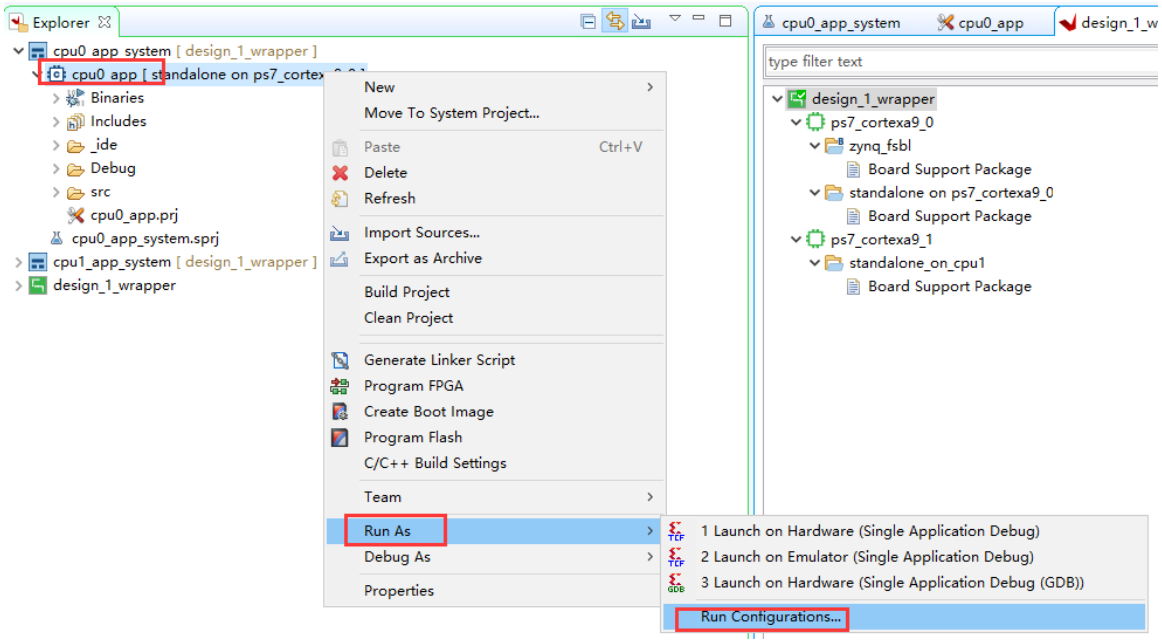
```
unsigned char Cpu1_Data[12] = "Hello Cpu0!" ;
unsigned char *Cpu0Data ;
```

2. 在 PLGpioSetup 函数中需要将按键中断号绑定到 CPU1，其他部分都与 CPU0 类似，不再赘述。

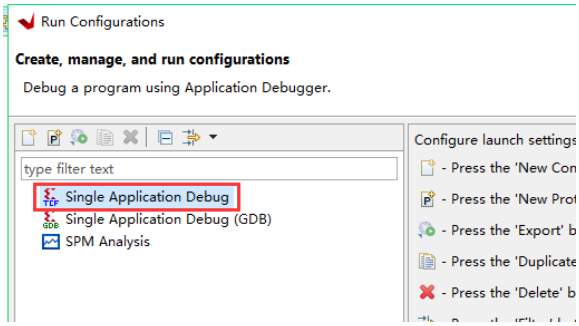
```
/* Connect key interrupt to CPU1 */
XScuGic_InterruptMaptoCpu(InstancePtr, XPAR_CPU_ID, KEY_INTR_ID) ;
```

21.2 板上验证

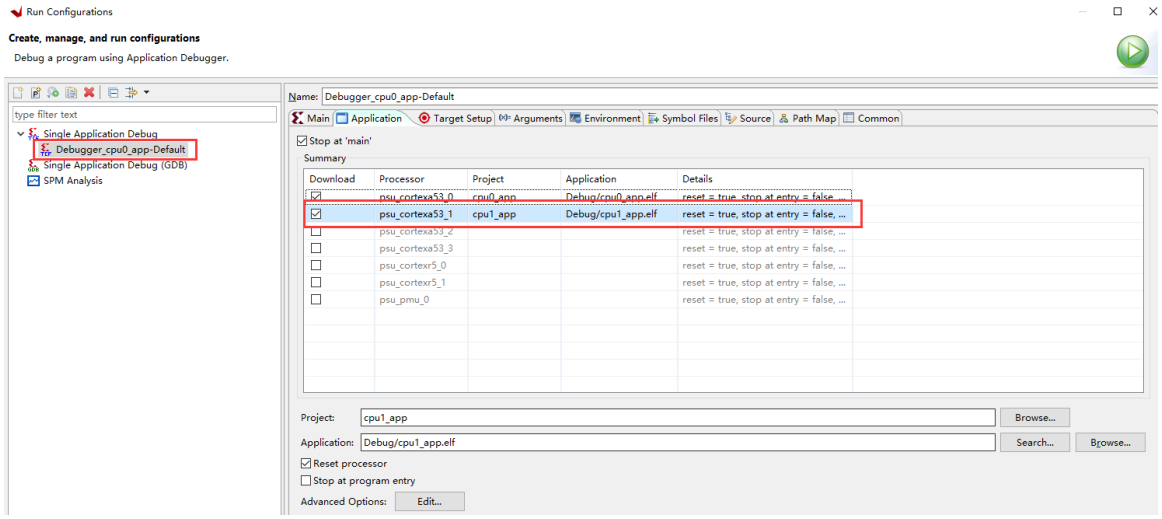
1. 下载时注意进入 Run Configurations 配置



2. 双击 Single Application Debug



3. 勾选 CPU1，其他默认，点击 Run



4. 打开串口软件，测试 CPU0，按下 PS 端按键，控制 PS 端 LED 灯亮，表明 CPU0 在运行，同时 CPU1 接收到 CPU0 设置的软件中断，并打印出信息。

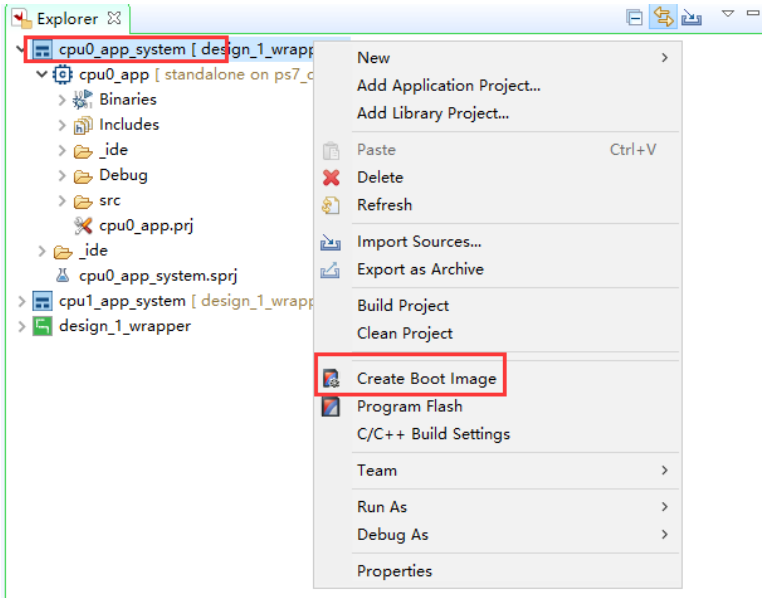


5. 测试 CPU1，按下 PL 端按键，控制 PL 端 LED 灯亮，表明 CPU1 在运行，同时 CPU0 接收到 CPU1 设置的软件中断，并打印出信息。

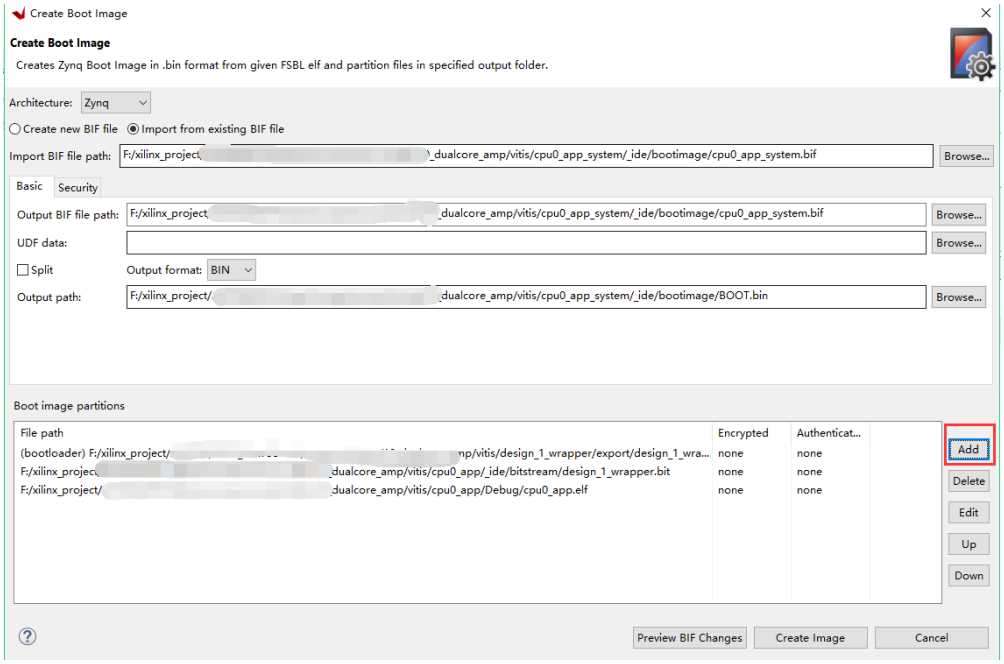


21.3 QSPI Flash 启动

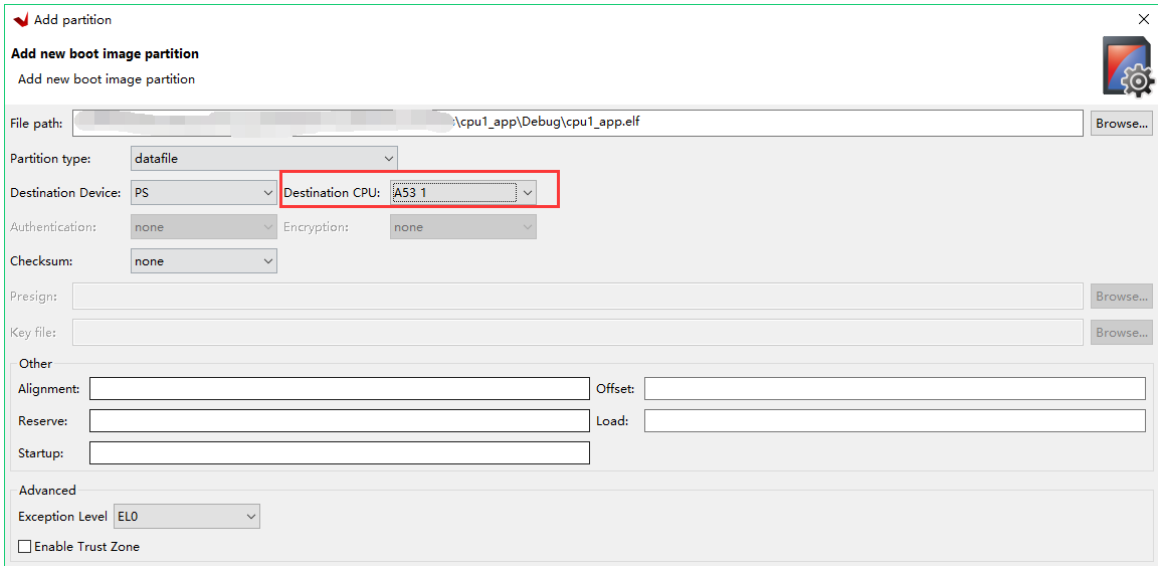
生成 BOOT.BIN 的方式与前面 Build Porject 生成不同，我们需要进行配置。在 CPU0 的 system 右键选择 Create Boot Image



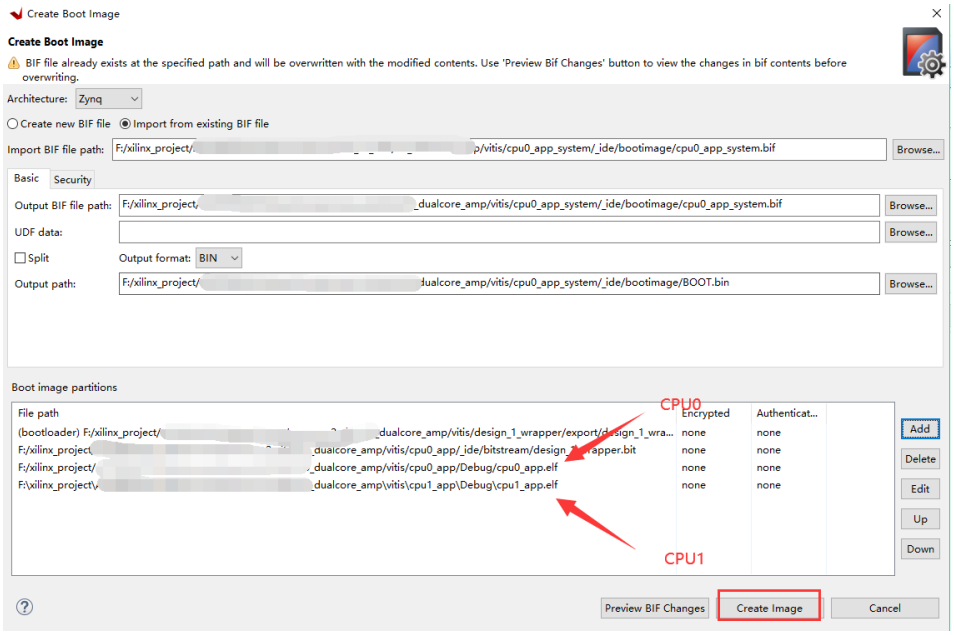
点击 Add，添加 CPU1 的 elf 文件，



选择相应的 CPU1 的 elf，选择目录 CPU 为 A53 1



添加后结果如下，点击 Create Image



21.4 本章小结

本章较为简单的介绍了如何在裸机下使用双核，以及中断使用，双核之间通信。在本实验中并未用到共享内存结构体中的长度成员，大家可以试验根据长度和地址将两个核的数据进行拷贝。

第二十二章 ZYNQ 下使用 Free RTOS

实验 Vivado 工程目录为“ps_axi_gpio /vivado”。
实验 vitis 工程目录为“freertos /vitis”。

学习 ZYNQ，很大一部分都是 FPGA 开发人员，不太擅长使用 Linux，所以我建议大家还是先使用实时操作系统或者裸机运行，这样也有更大的灵活性。本章给大家讲解如何搭建 Free RTOS 实时操作系统运行环境，这里不深入探讨 Free RTOS 的具体使用。本实验以 FreeRTOS Hello World 做举例，并实现两个 LED 灯以不同间隔持续闪烁。

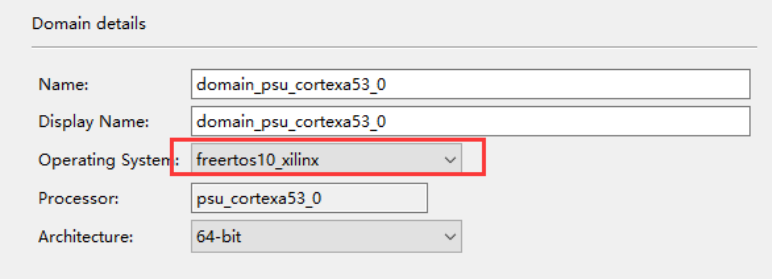
本实验基于“PL 端 AXI GPIO 的使用”工程，硬件环境不需要修改。

软件工程师工作内容

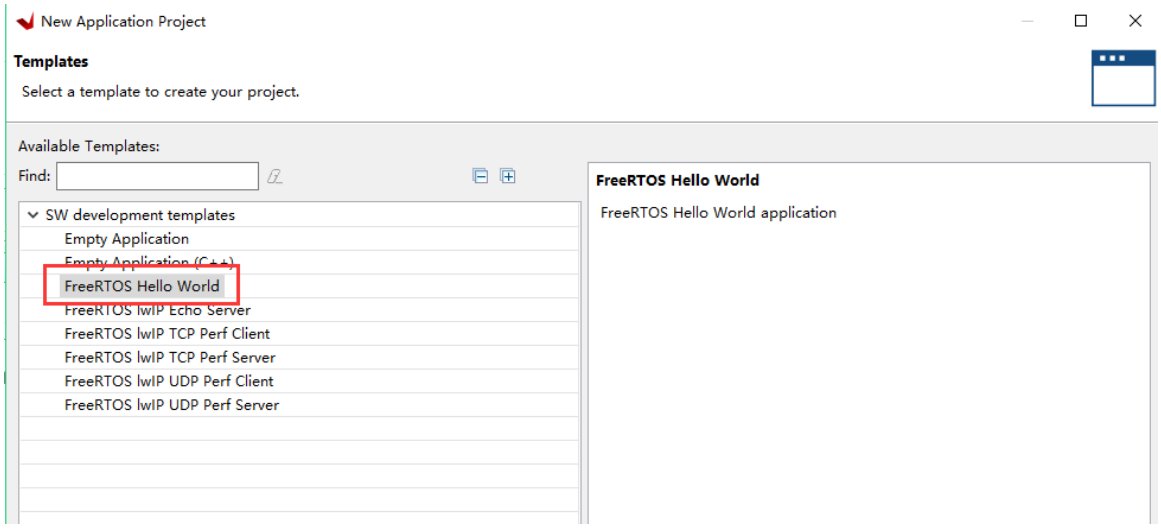
以下为软件工程师负责内容。

22.1 Vitis 程序开发

1. 参考 Hello World 一章，新建工程，OS 选择 freertos10_xilinx



2. 本实验选择 FreeRTOS Hello World 做举例



Hello World 例子中建立了两个任务，发送任务和接收任务，接收任务的优先级高于发送任务。并且创建队列，由发送任务向队列发送数据，接收任务从队列中读取数据并打印。例子中设置了 timer，但在本实验中删除了 timer，让发送和接收任务一直工作。

```
xTaskCreate(    prvTxTask,                  /* The function that implements the task. */
                ( const char * ) "Tx",      /* Text name for the task, provided to assist debugging only. */
                configMINIMAL_STACK_SIZE,   /* The stack allocated to the task. */
                NULL,                       /* The task parameter is not used, so set to NULL. */
                tskIDLE_PRIORITY,           /* The task runs at the idle priority. */
                &xTxTask );

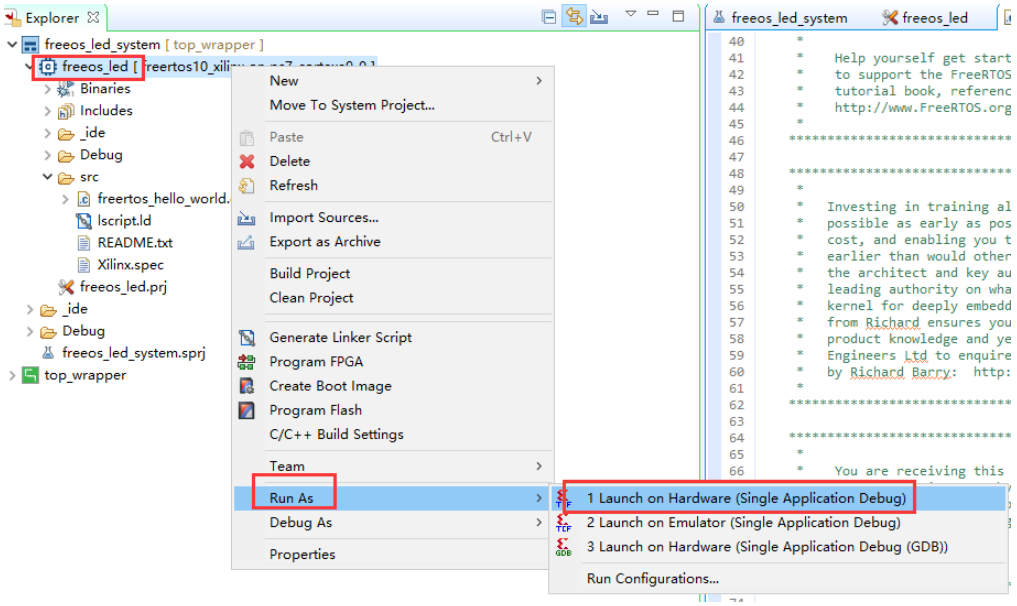
xTaskCreate( prvRxTask,
             ( const char * ) "GB",
             configMINIMAL_STACK_SIZE,
             NULL,
             tskIDLE_PRIORITY + 1,
             &xRxTask );
```

3. 在此基础上增加 PS 和 PL 端 LED 闪烁任务，PS 端闪烁间隔为 100ms，PL 端闪烁间隔为 1S

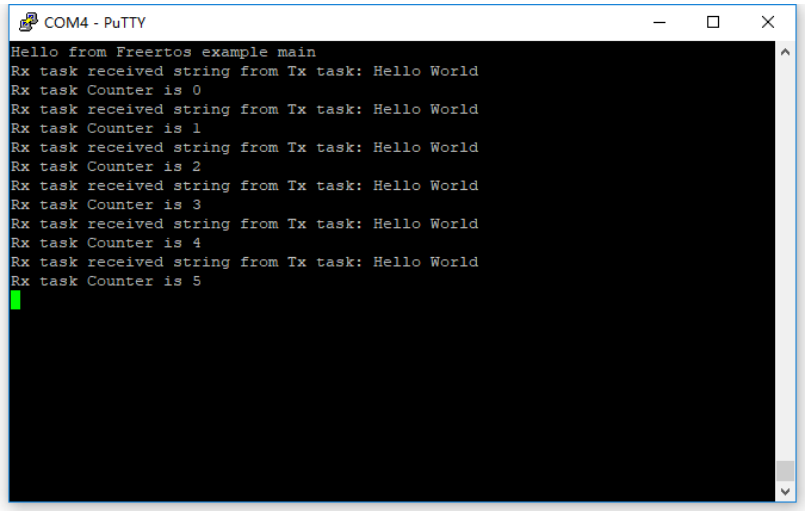
```
xTaskCreate( prvPsLedTask,
             ( const char * ) "Ps Led",
             configMINIMAL_STACK_SIZE,
             NULL,
             tskIDLE_PRIORITY + 1,
             NULL);
xTaskCreate( prvPlLedTask,
             ( const char * ) "PL Led",
             configMINIMAL_STACK_SIZE,
             NULL,
             tskIDLE_PRIORITY + 1,
             NULL);
```

22.2 板上验证

1. 下载界面设置，下载程序



2. 打开串口，正在持续打印数据



3. 同时也能看到开发板上 PS 端和 PL 端 LED 闪烁，直观地体现了多任务并行处理。

22.3 本章小结

相对于复杂的 Linux，FreeRTOS 等实时操作系统给我们带来更灵活更方便的开发，可以更直接的和底层 FPGA 进行交互，但是 FreeRTOS 本身也有一点难度，要想熟练应用，还是要结合具体的项目多加练习。

第二十三章 PL 读写 PS 端 DDR 数据

实验 Vivado 工程目录为“pl_read_write_ps_ddr/vivado”。
实验 vitis 工程目录为“pl_read_write_ps_ddr /vitis”。

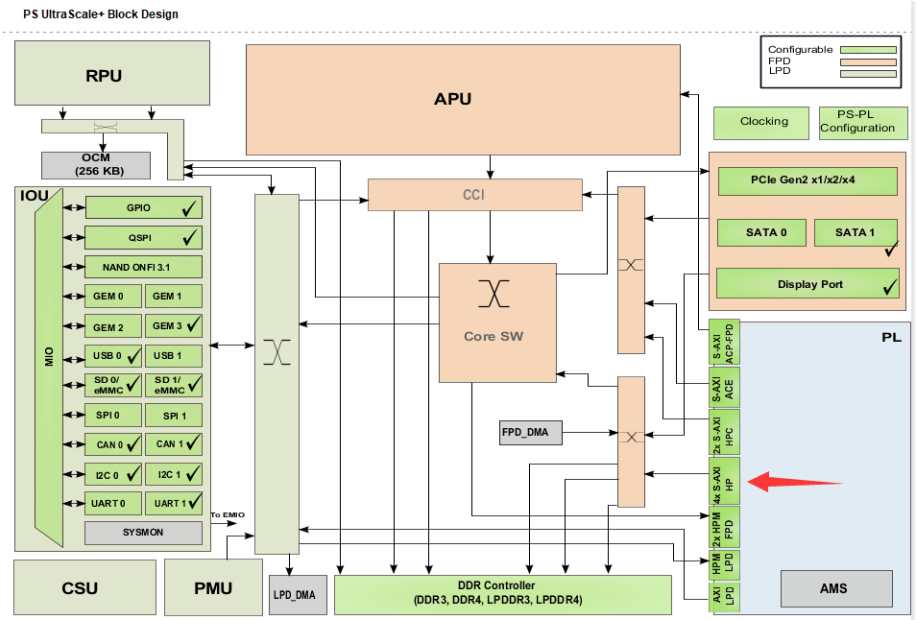
PL 和 PS 的高效交互是 zynq soc 开发的重中之重，我们常常需要将 PL 端的大量数据实时送到 PS 端处理，或者将 PS 端处理结果实时送到 PL 端处理，常规我们会想到使用 DMA 的方式来进行，但是各种协议非常麻烦，灵活性也比较差，本节课程讲解如何直接通过 AXI 总线来读写 PS 端 ddr 的数据，这里面涉及到 AXI4 协议，vivado 的 FPGA 调试等。

FPGA 工程师工作内容

以下为 FPGA 工程师负责内容。

23.1 ZYNQ 的 HP 端口使用

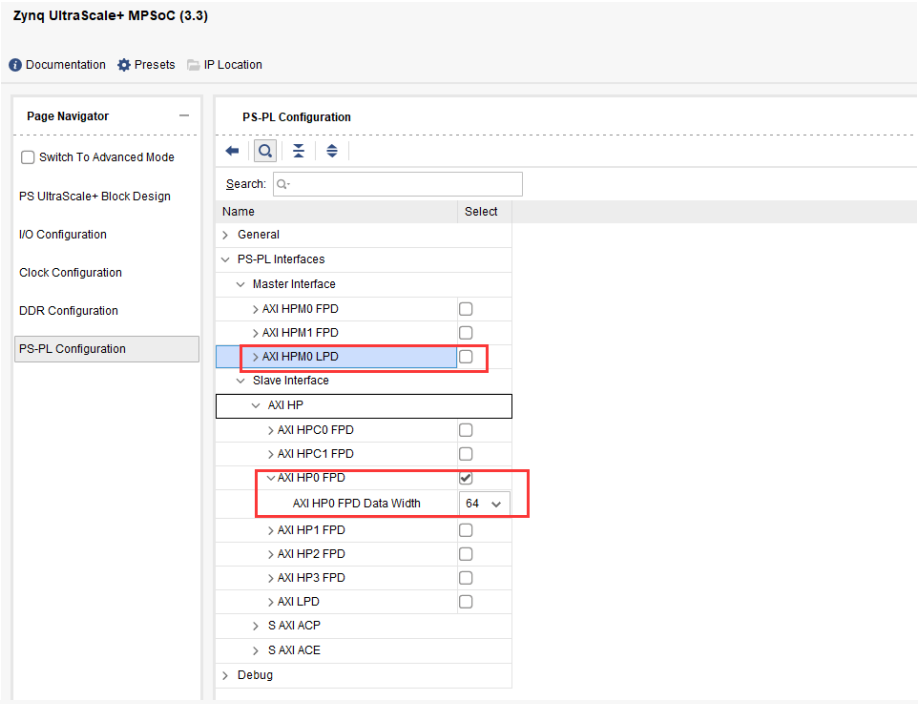
zynq 7000 SOC 的 HP 口是 High-Performance Ports 的缩写，如下图所示，一共有 4 个 HP 口，HP 口是 AXI Slave 设备，我们可以通过这 4 个 HP 接口实现高带宽的数据交互。

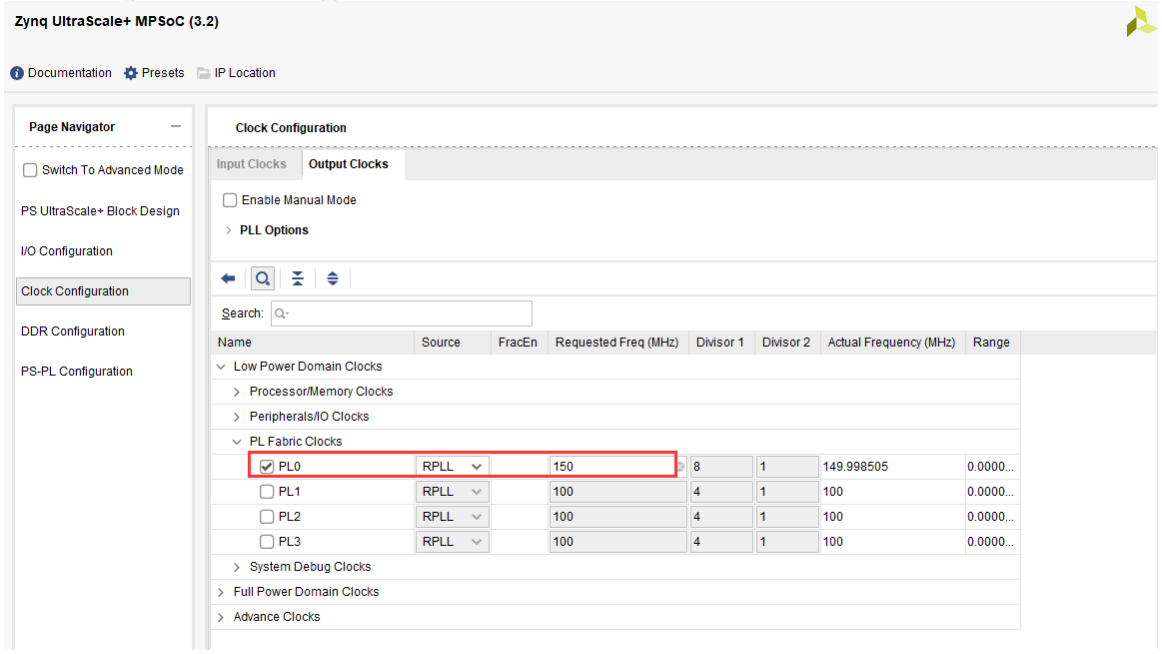


23.2 硬件环境搭建

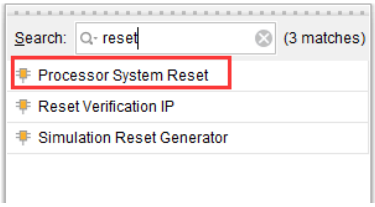
1. 基于“ps_hello”工程，在 vivado 的界面中 HP 的配置如下图(HP0~HP3)，这里面有使能控制，

数据位宽选择，可选择 32bit、64bit 或 128bit 的位宽。我们的实验启用 HP0 配置为 64bit 位宽，使用的时钟是 150Mhz，HP 的带宽是 150Mhz * 64bit，对于视频处理，ADC 数据采集等应用都有足够的带宽。不需要 AXI HPM0 LPD，取消选择。

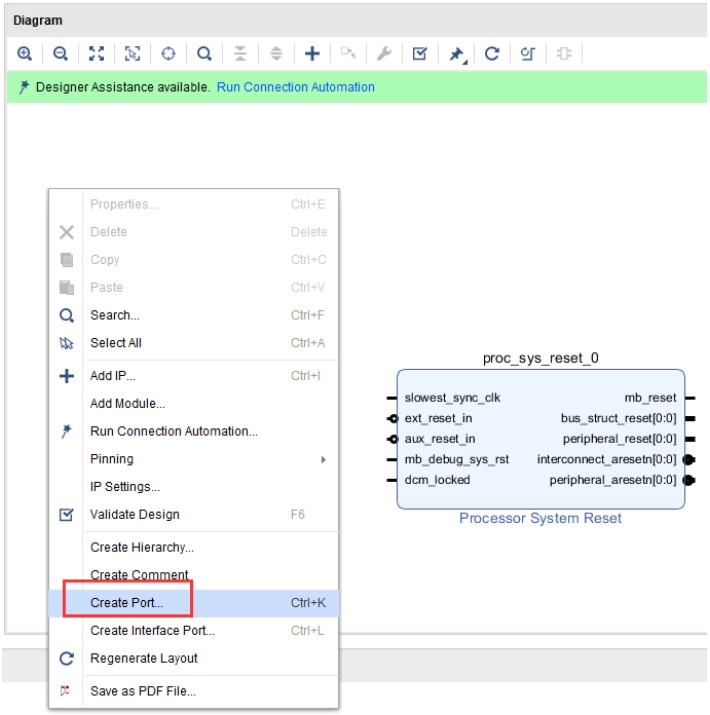




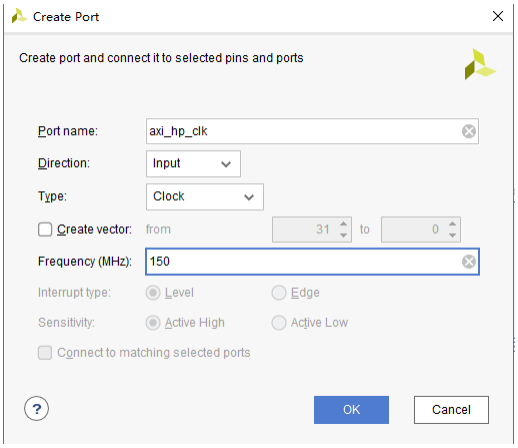
2. 添加复位模块，用于复位



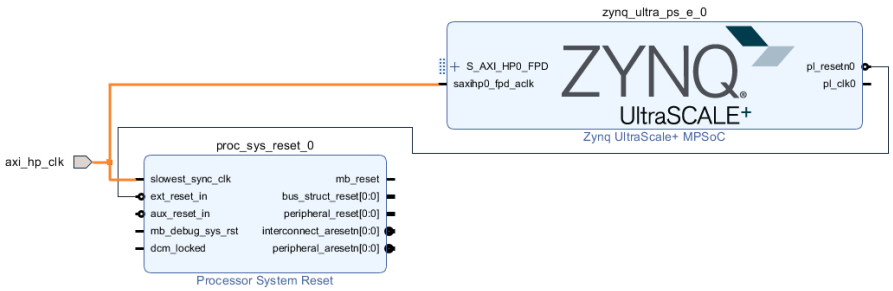
3. 在空白处右键选择”Creat Port”



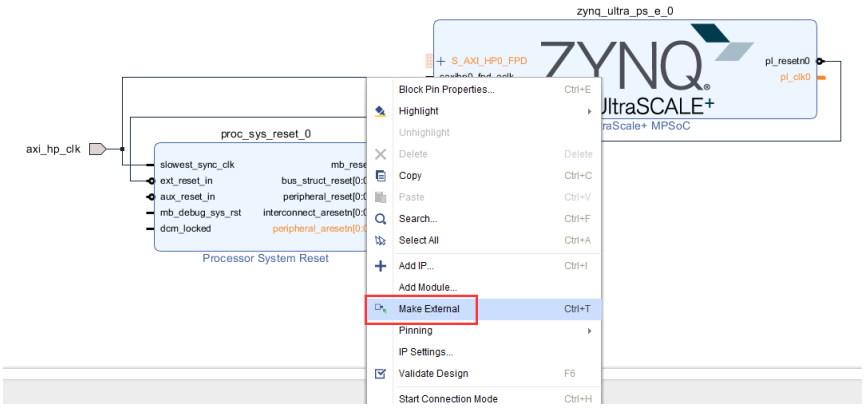
配置如图



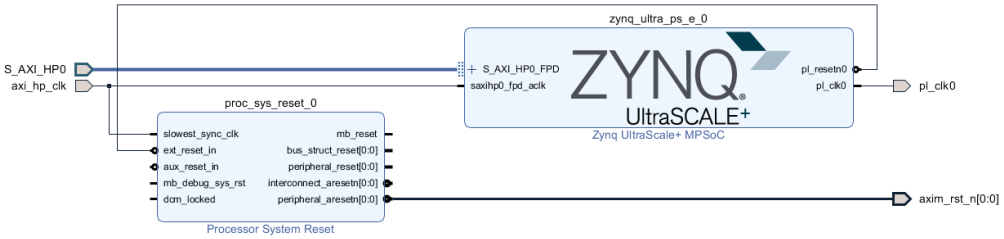
4. 连接时钟和复位



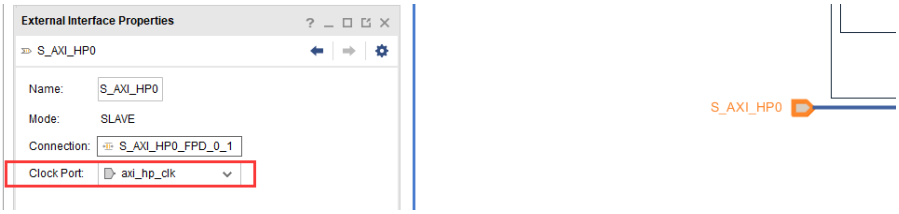
5. 选中引脚，点击 Make External，将信号导出



并修改引脚名称如下图



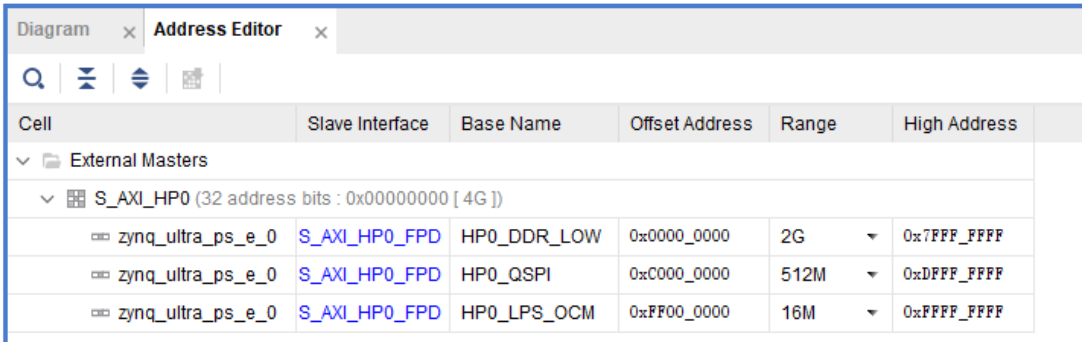
并选择总线同步时钟为 axi_hp_clk



6. 点开 Address Editor，如果发现地址没有分配，点击自动分配地址按钮

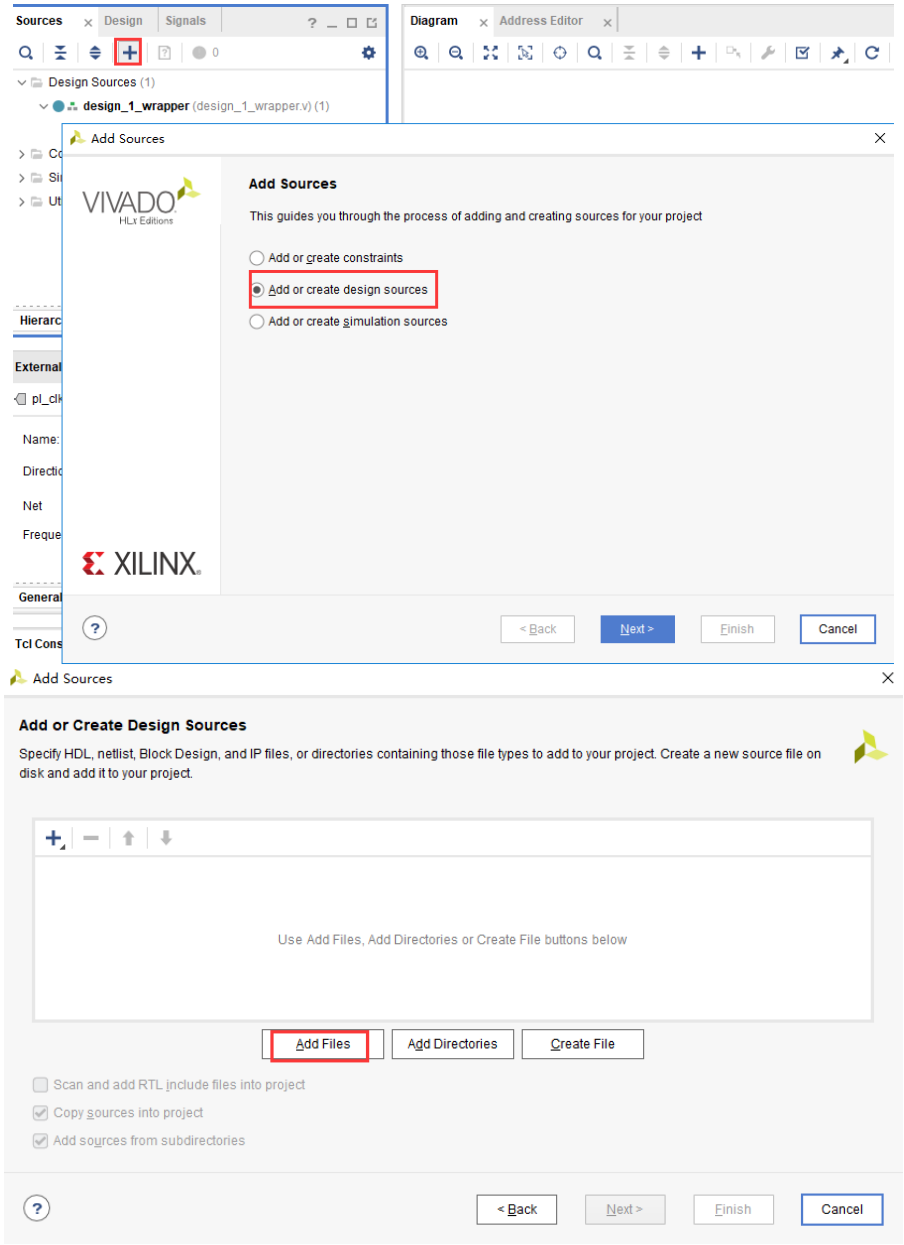


分配后的结果，可以看到访问 DDR, QSPI, OCM 的地址空间

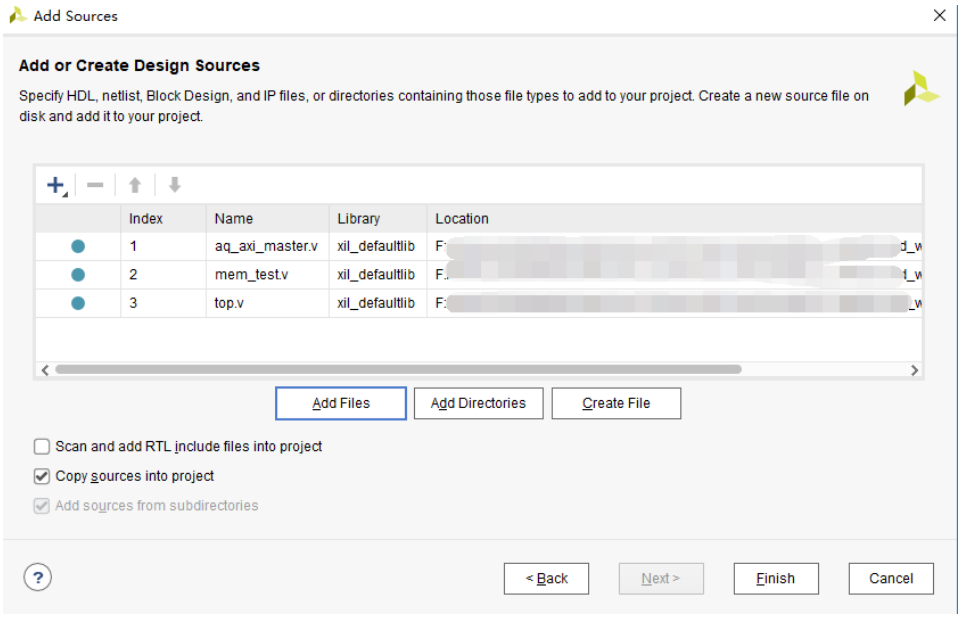


保存设计，重新 Generate Ouput Product

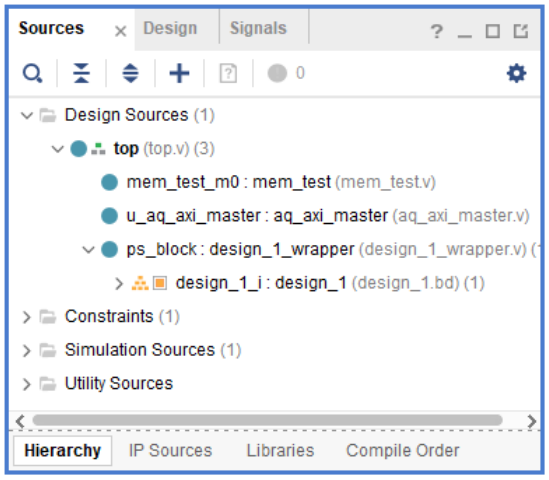
7. 添加 hdl 文件



点击 Finish



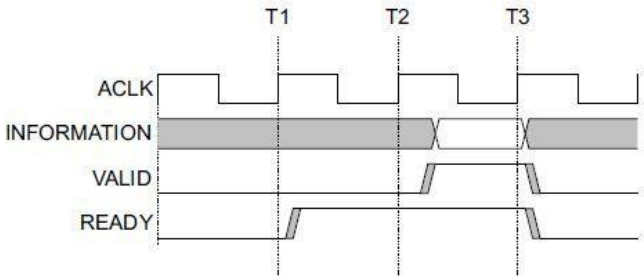
HDL 层级关系更新结果



23.3 PL 端 AXI Master

AXI4 相对复杂，但 SOC 开发者必须掌握，对于 zynq 的开发者，笔者建议能够在一些已有的模板代码基础上修改。AXI 协议的具体内容可参考 Xilinx UG761 AXI Reference Guide。在这里我们简单了解一下。

AXI4 所采用的是一种 READY，VALID 握手通信机制，即主从模块进行数据通信前，先根据操作对各所用到的数据、地址通道进行握手。主要操作包括传输发送者 A 等到传输接受者 B 的 READY 信号后，A 将数据与 VALID 信号同时发送给 B，这是一种典型的握手机制。

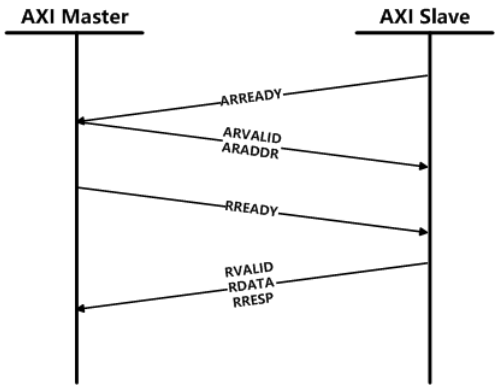


AXI 总线分为五个通道：

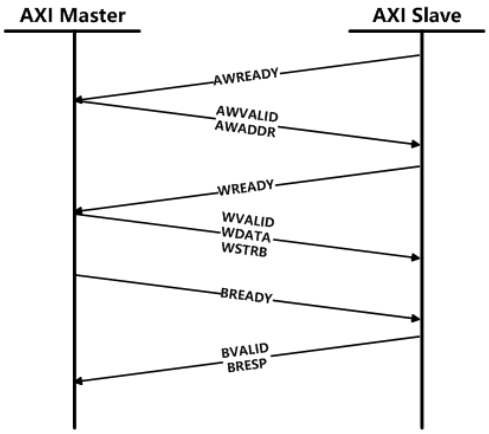
- 读地址通道，包含 ARVALID, ARADDR, ARREADY 信号；
- 写地址通道，包含 AWVALID，AWADDR, AWREADY 信号；
- 读数据通道，包含 RVALID, RDATA, RREADY, RRESP 信号；
- 写数据通道，包含 WVALID, WDATA，WSTRB, WREADY 信号；
- 写应答通道，包含 BVALID, BRESP, BREADY 信号；
- 系统通道，包含：ACLK，ARESETN 信号；

其中 ACLK 为 axi 总线时钟，ARESETN 是 axi 总线复位信号，低电平有效；读写数据与读写地址类信号宽度都为 32bit；READY 与 VALID 是对应的通道握手信号；WSTRB 信号为 1 的 bit 对应 WDATA 有效数据字节，WSTRB 宽度是 32bit/8=4bit；BRESP 与 RRESP 分别为写回应信号，读回应信号，宽度都为 2bit， ‘h0 代表成功，其他为错误。

读操作顺序为主与从进行读地址通道握手并传输地址内容，然后在读数据通道握手并传输所读内容以及读取操作的回应，时钟上升沿有效。如图所示：



写操作顺序为主与从进行写地址通道握手并传输地址内容，然后在写数据通道握手并传输所读内容，最后再写回应通道握手，并传输写回应数据，时钟上升沿有效。如图所示：



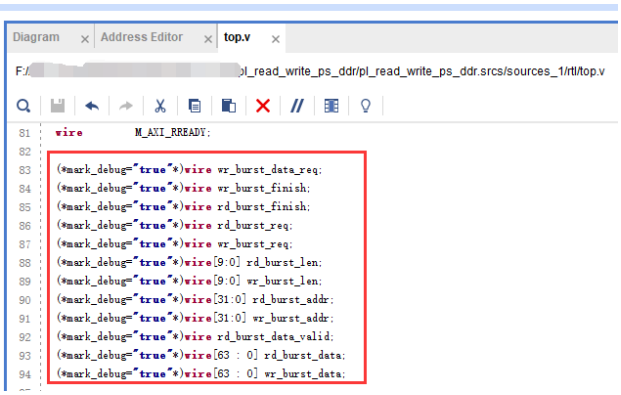
在我们不擅长写 FPGA 的一些代码时我们往往要借鉴别人的代码或者使用 IP core。在这里笔者从 github 上找到一个 AXI master 的代码，地址是 https://github.com/aquaxis/IPCORE/tree/master/aq_axi_vdma。这个工程是一个自己写的 VDMA，里面包含了大量可参考的代码。笔者这里主要使用了 aq_axi_master.v 这个代码用于 AXI master 读写操作。借鉴别人代码有时会节省很多时间，但如果不能理解的去借鉴，出现问题了很难解决。具体可以参考 aq_axi_master.v 代码，有部分修改。

23.4 ddr 读写数据的检验

有了 AXI Master 读写接口以后比较编写了一个简单的验证模块，这个验证模块是用来验证 ddr ip 的，通过写入数据，然后读取出来比较。这里要注意的是 PS 端 DDR 的起始地址和大小，还有地址的单位是 byte 还是 word，AXI 总线的地址单位是 byte，测试模块的地址单位是 word（这里的 word 不一定是 4byte）。文件名 mem_test.v。

23.5 Vivado 软件的调试技巧

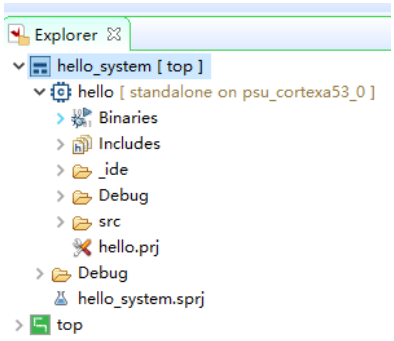
AXI 读写验证模块只有一个 error 信号用于指示错误，如果有数据错误我们希望能更精确的信息，altera 的 quartus II 软件中有 signal tap 工具，xilinx 的 ISE 中有 chipscope 工具，这些都是嵌入式逻辑分析仪，对我们调试有很大帮助，在 vivado 软件中调试更加方便。在插入调试信号时有些信息可能会被优化掉，或者信号名称改变了就不容易识别，这个时候我们可以在程序代码里加入*mark_debug="true"*这样的属性，如下图的信号：



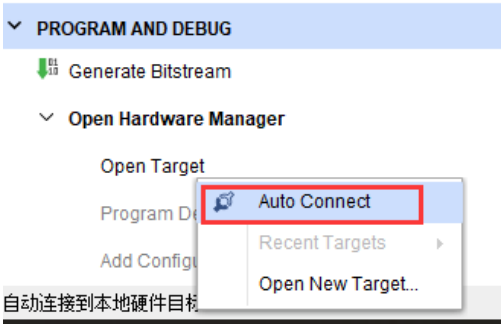
具体的添加方法在”PL 的“Hello World”LED 实验”中已经讲过，可参考。并在 XDC 文件里绑定 error 信号到 PL 端 LED 灯上。

23.6 Vitis 工程开发

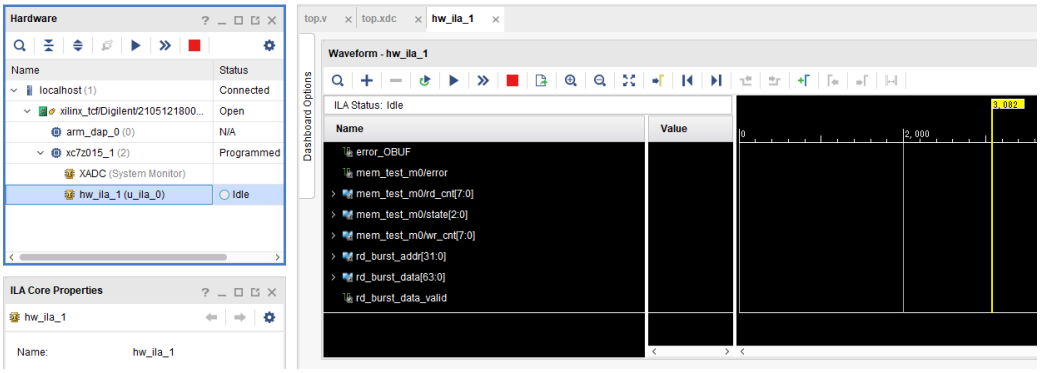
以 hello world 为模板新建 vitis 工程如下



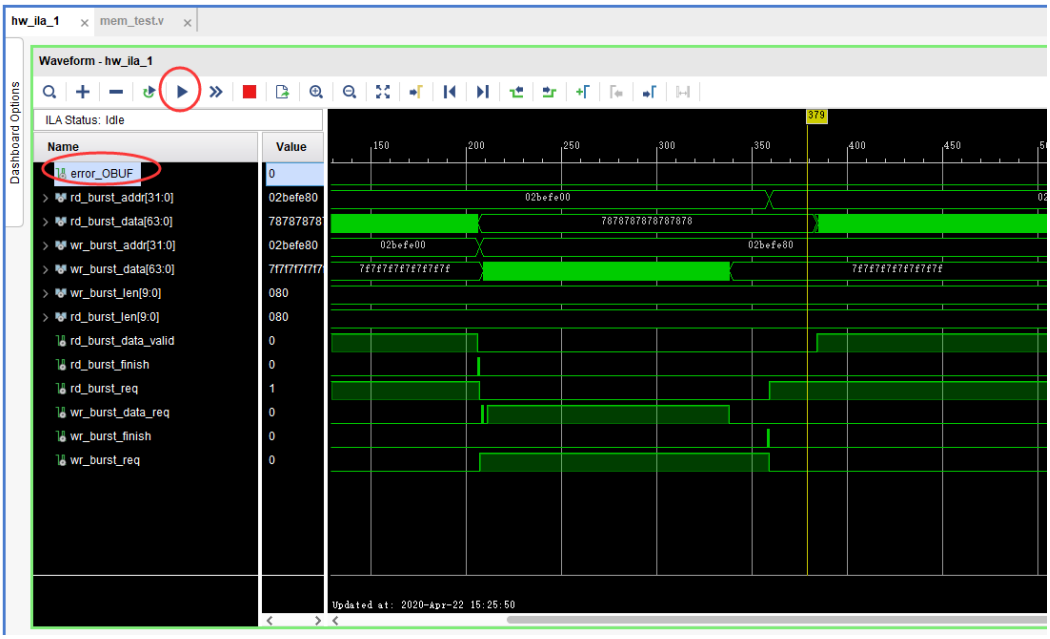
通过 vitis 下载程序后，系统会复位并且下载 FPGA 的 bit 文件。然后回到 vivado 界面点击 Program and Debug 栏自动连接目标如下图所示：



自动连接硬件后可发现 JTAG 连上的设备，其中有一个 hw_ila_1 的设备，这个设备就是我们 debug 设备，选中后可点击上方黄色三角按钮捕捉波形。如果有些信号没有显示完整，可点击波形旁边的“+”按钮添加。



点击捕获波形以后如下图所示，如果 error 一直为低，并且读写状态有变化，说明读写 DDR 数据正常，用户在这里可以自己查看其它的信号来观察写入 DDR 的数据和从 DDR 读出的数据。



23.7 本章小结

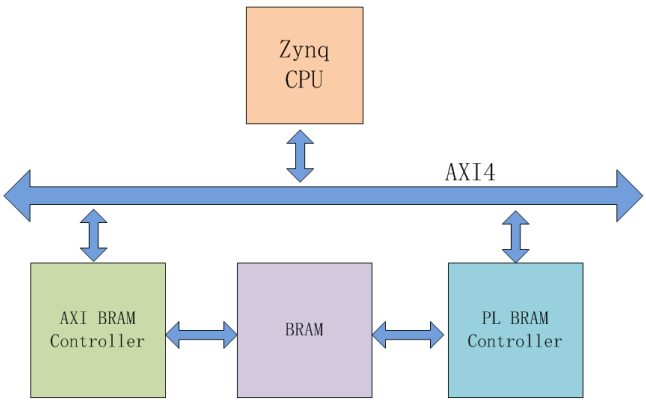
zynq 系统相对于单个 FPGA 或单个 ARM 要复杂很大，对开发者的基础知识要求较高，本章内容涉及到 AXI 协议、zynq 的互联资源、vivado 的和 Vitis 的调试技巧。这些都仅仅是基础知识，笔者在这里也仅仅是抛砖引玉，大家还是要多多练习，在不断练习中掌握技巧。

第二十四章 通过 BRAM 实现 PS 与 PL 数据交互

实验 Vivado 工程目录为“bram_test /vivado”。
实验 vitis 工程目录为“bram_test /vitis”。

有时 CPU 需要与 PL 进行小批量的数据交换，可以通过 BRAM 模块，也就是 Block RAM 实现此要求。本章通过 Zynq 的 GP Master 接口读写 PL 端的 BRAM，实现与 PL 的交互。在本实验中加入了自定义的 FPGA 程序，并利用 AXI4 总线进行配置，通知其何时读写 BRAM。

以下为本实验原理图，CPU 通过 AXI BRAM Controller 读取 BRAM 数据，CPU 仅配置自定义的 PL BRAM Controller 的寄存器，不通过它读写数据。



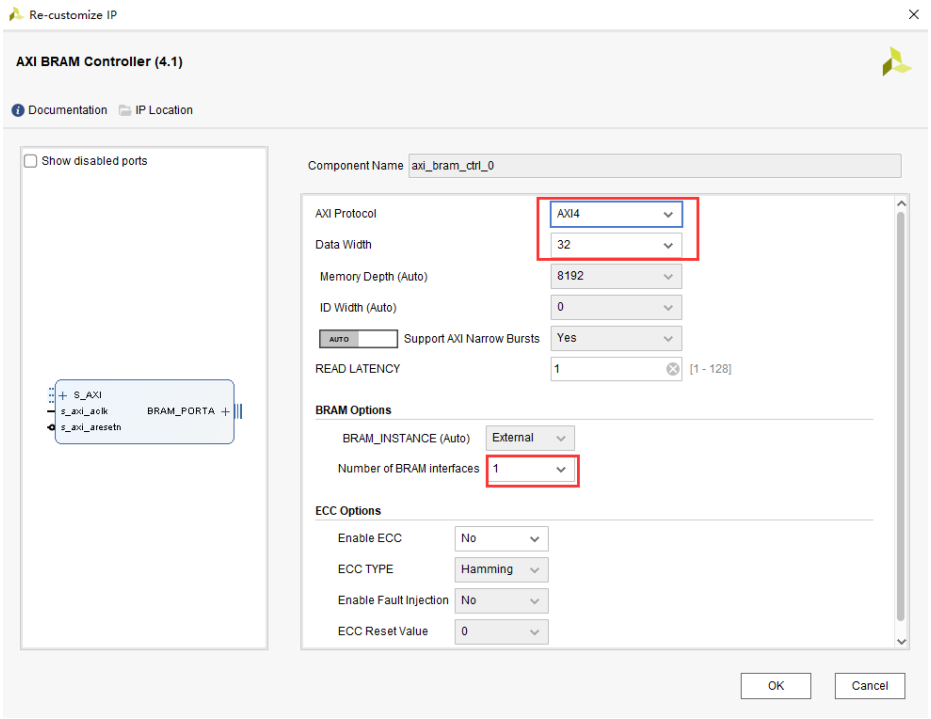
FPGA 工程师工作内容

以下为 FPGA 工程师负责内容。

24.1 硬件环境搭建

以“ps_hello”为基础，另存为一份工程，并配置打开 ZYNQ 的中断

1. 首先添加 AXI BRAM Controller 模块，用于 PS 端控制 BRAM，双击打开配置，连接 AXI 总线，可用于读写 BRAM 模块，AXI 模式设置为 AXI4，数据宽度设置为 32 位，memory depth 不在这里设置，需要在 Address Editor 里设置。BRAM 端口数量设置为 1 个，用于连接双口 RAM 的 PORTA。不使能 ECC 功能。

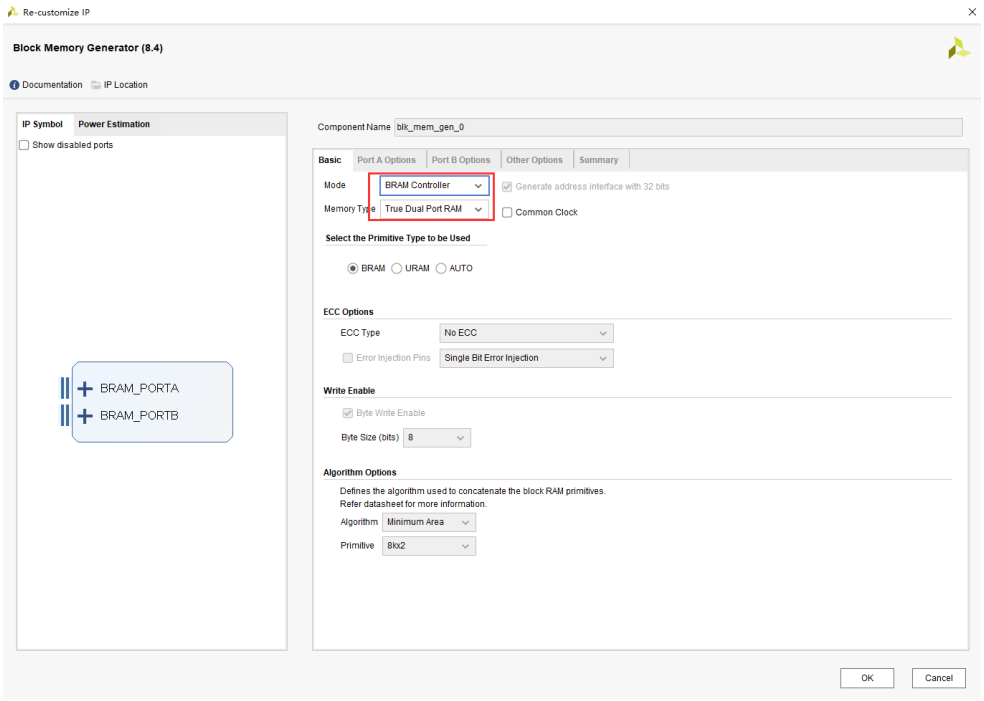


由于 AXI4 总线为字节询址，BRAM 数据宽度设置也是 32 位，同样都是 32 位数据宽度，因此在映射到 BRAM 地址时，需要按 4 字节询址，即去掉最后两位，下图为 BRAM 控制器与 BRAM 的映射关系。

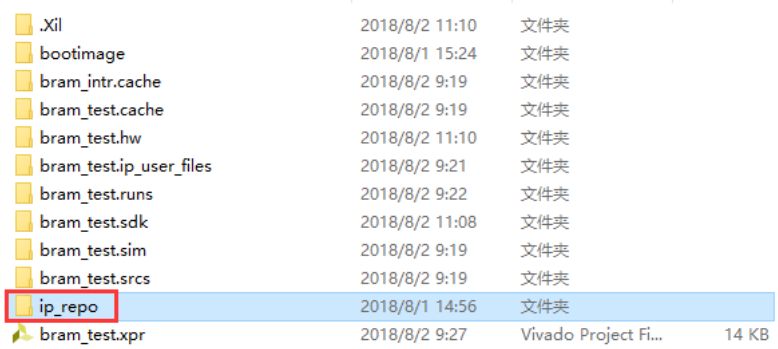
Table 3-1: **BRAM Configuration for UltraScale, 7 Series, or Zynq-7000 Devices**

Supported Memory Sizes / BRAM Memory Configuration	Number of BRAM Primitives (36k/each)	Size of BRAM_Addr (each port)	Typical BRAM_Addr Bit Usage with BRAM Configuration
32-bit Data BRAM Data Width			
4k / (1024 x 32)	1	10	BRAM_Addr [11:2]
8k / (2048 x 32)	2	11	BRAM_Addr [12:2]
16k / (4096 x 32)	4	12	BRAM_Addr [13:2]
32k / (8192 x 32)	8	13	BRAM_Addr [14:2]
64k / (16384 x 32)	16	14	BRAM_Addr [15:2]
128K / (32,768x32)	32	15	BRAM_Addr [16:2]

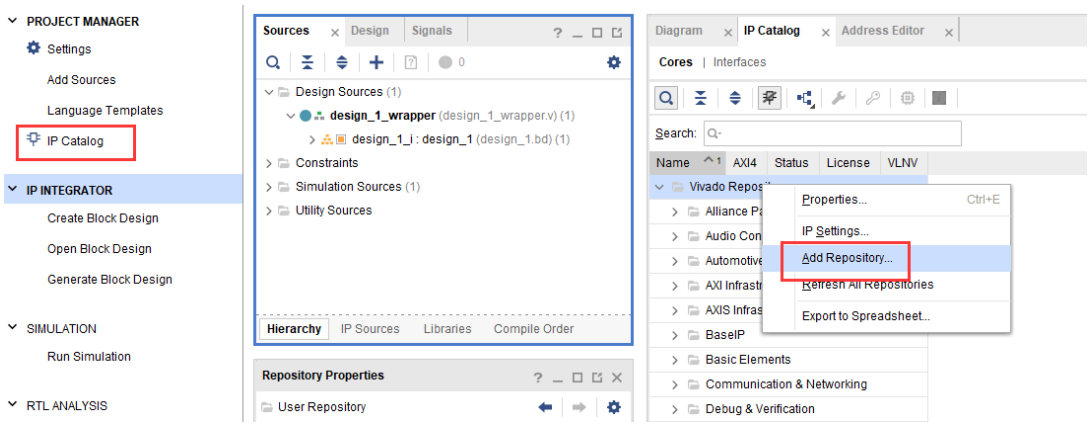
2. 添加 BRAM 模块，BRAM 设置如下，有两种模式选择，standalon 模式，此模式可以自由配置 RAM 的数据宽度和深度。BRAM Controller 模式，此模式下地址线和数据端口默认为 32 位，本实验因为用到了 BRAM 控制器，因此选择 BRAM Controller 模式。Memory 类型选择双口 RAM，一端连 BRAM 控制器，一端连 PL RAM 控制器。



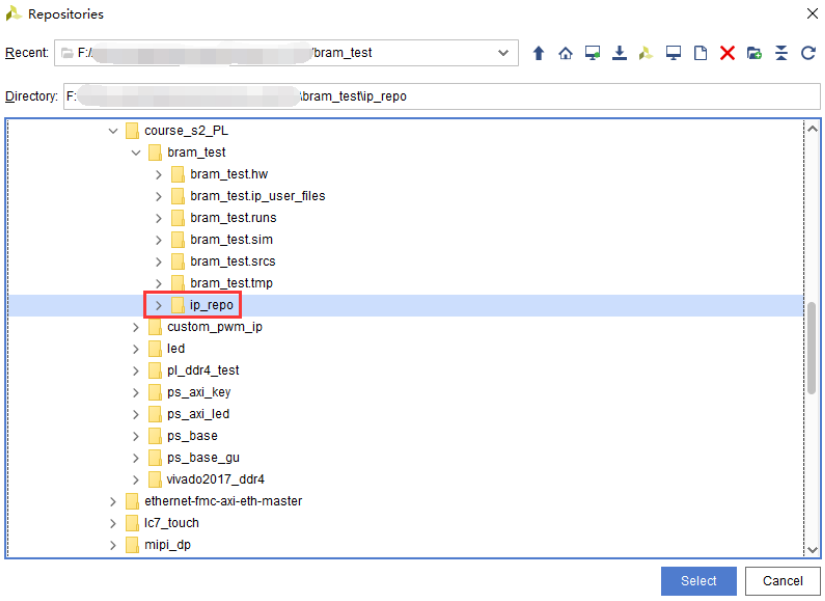
3. 添加自定义的 PL RAM 控制器 pl_ram_ctrl，功能很简单，start 信号有效后开始读取 BRAM 的数据，可通过 ILA 逻辑分析仪观察读取的数据，PL RAM 控制器读 BRAM 结束后，开始向 BRAM 写数据，写完数据使能 intr 信号，即中断信号，CPU 即可读取 BRAM 的数据。将 PL 控制器信号与 BRAM 的 PORTB 连接。自定义 IP 在 ip_repo 文件夹中。



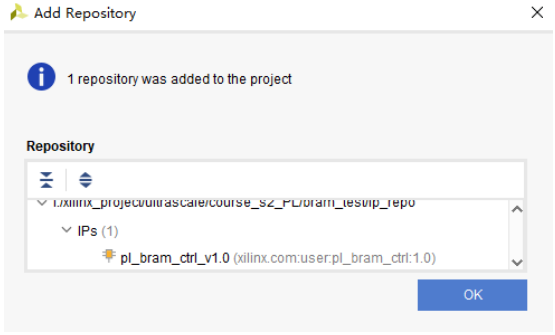
如果想添加自定义 IP 到 IP 库中，点击 IP Catalog，在 Vivado Repository 右键点击 Add Repository



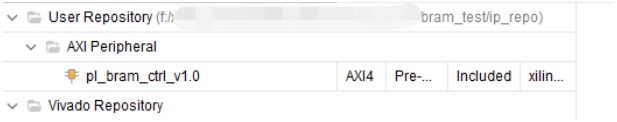
找到自定义 IP 所在文件夹，点击 Select



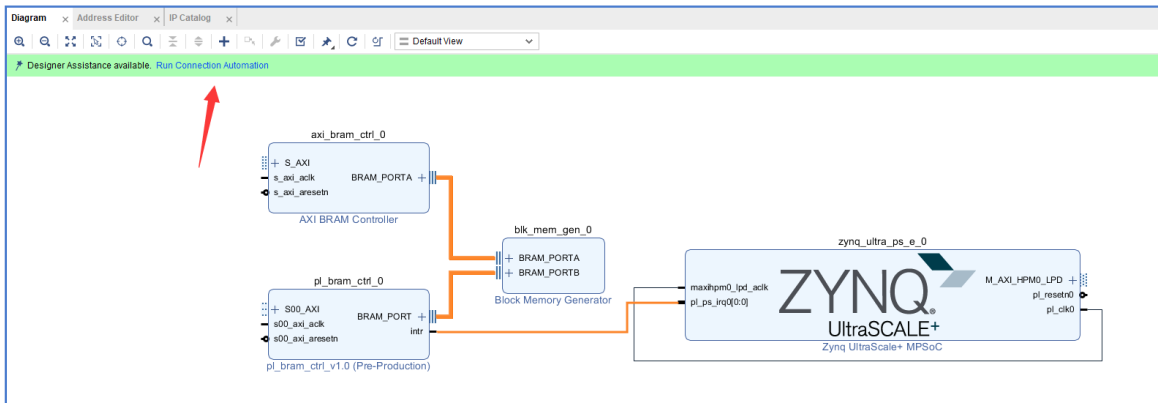
跳出窗口，选择 IP，点击 OK



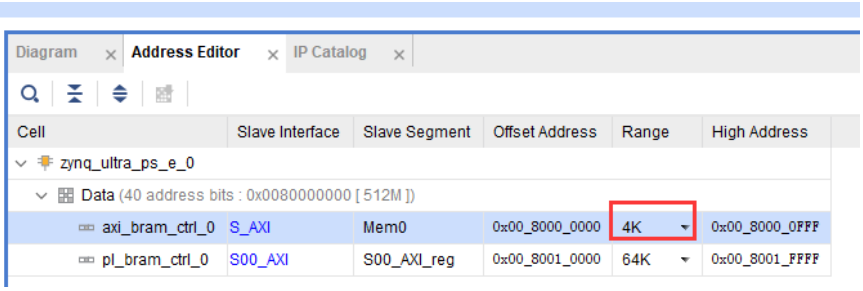
即可看到，出现了刚添加的 IP



4. 连接 AXI BRAM Controller 的 BRAM_PORTA 到 BRAM 的 PORTA，连接 pl_bram_ctrl 的 BRAM_PORT 到 BRAM 的 PORTB。连接 pl_bram_ctrl 模块的中断信号 intr 到 ZYNQ 的中断口。并点击自动连接

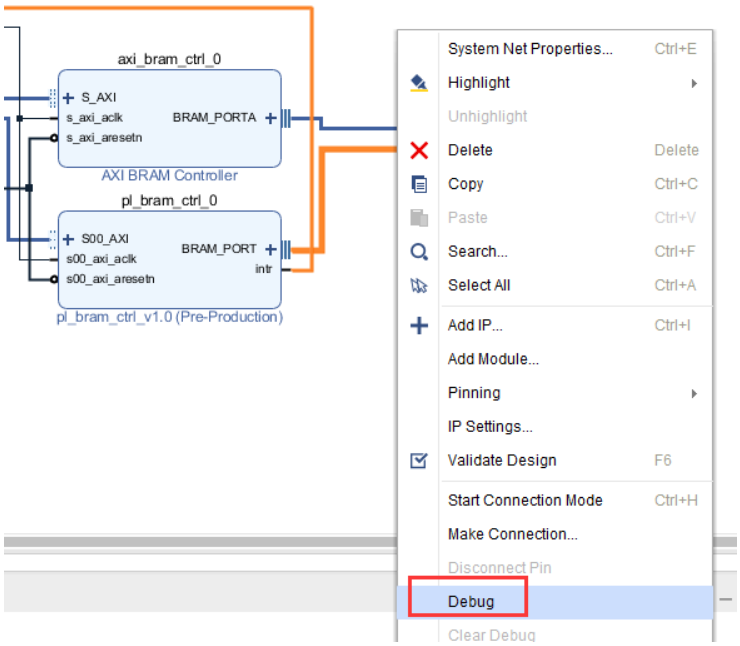


5. 在 Address Editor 里选择 BRAM 询址大小，如设置 4K 空间，即可询址 BRAM 空间为 1K 深度。

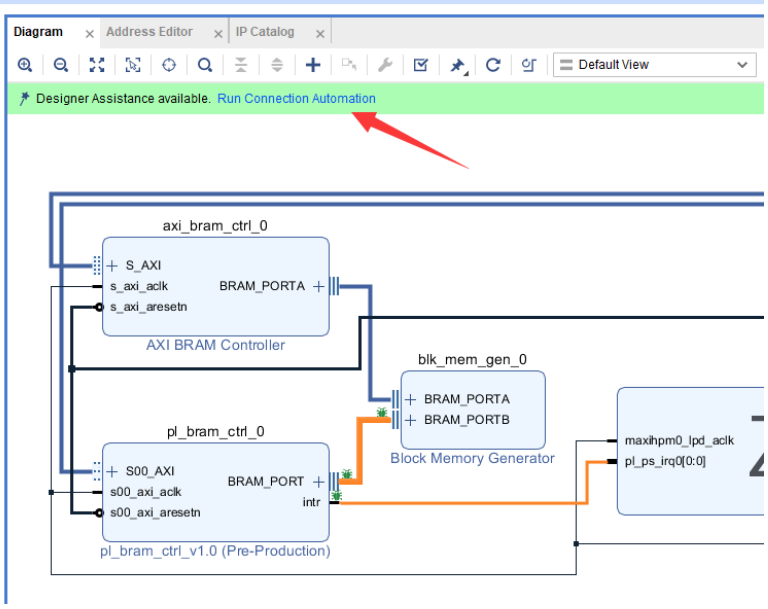


24.1.1 Block Design 添加逻辑分析仪方法

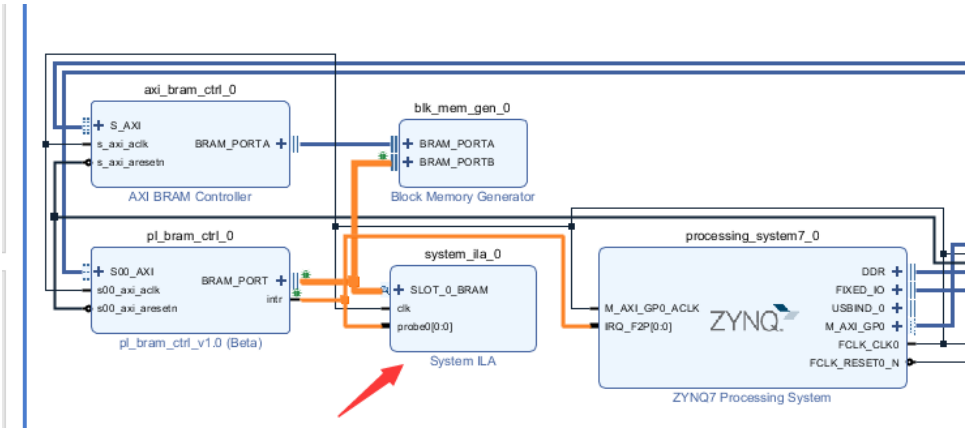
6. 再介绍一种添加逻辑分析仪的方法，选中 BRAM_PORT 总线和 intr 中断，右键选择 Debug



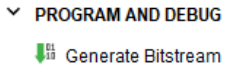
7. 可以看到总线上多了小昆虫，点击 Run Connection Automation，自动连接



自动添加了一个 ILA 模块，并且有一个总线接口，一个信号接口



8. 保存设计，之后点击 Generate Bitsream 生成 bit 文件，并导出 Hardware 信息。



软件工程师工作内容

以下为软件工程师负责内容。

24.2 Vitis 程序开发

1. 程序设计流程为：输入起始地址和长度→CPU 通过 BRAM 控制器写入 BRAM 数据→通知 PL 控制器读取 BRAM 数据→PL 内部读完后向相同位置写入数据，初始数据由 CPU 告知→写完后使能 write_end 信号，触发 GPIO 中断→中断读取 BRAM 数据，打印显示

2. 进入 Vitis 后，在 Vitis 下新建项目，已经准备好程序。程序也比较简单，首先中断设置

```
int main()
{

    int Status;
    Intr_flag = 1 ;

    IntrInitFuntion(INTC_DEVICE_ID) ;

    while(1)
    {
```

3. While 语句中需要输入起始地址和长度，之后调用 bram_write 函数

```
while(1)
{
    if (Intr_flag)
    {
        Intr_flag = 0 ;
        printf("Please provide start address\t\n") ;
        scanf("%d", &Start_Addr) ;
        printf("Start address is %d\t\n", Start_Addr) ;
        printf("Please provide length\t\n") ;
        scanf("%d", &Len) ;
        printf("Length is %d\t\n", Len) ;
        Status = bram_read_write() ;
        if (Status != XST_SUCCESS)
        {
            xil_printf("Bram Test Failed!\r\n") ;
            xil_printf("**********************************************\r\n");
            Intr_flag = 1 ;
        }
    }
}
```

4. 在 bram_read_write();函数里先通过 BRAM 控制器写入数据，数据初值为 TEST_START_VAL，

之后配置 PL RAM 控制器参数，有长度，起始地址，初始数据，以及开始信号。并在函数中判断测试长度是否超出 BRAM 控制器地址范围，如果超出，会报错，需要重新输入地址和长度。

```
int bram_read_write()
{

    u32 Write_Data = TEST_START_VAL ;
    int i ;

    /*
     * if exceed BRAM address range, assert error
     */
    if ((Start_Addr + Len) > (BRAM_CTRL_HIGH - BRAM_CTRL_BASE + 1)/4)
    {
        xil_printf("****************************************\r\n");
        xil_printf("Error! Exceed Bram Control Address Range!\r\n");
        return XST_FAILURE ;
    }
    /*
     * Write data to BRAM
     */
    for(i = BRAM_BYTENUM*Start_Addr ; i < BRAM_BYTENUM*(Start_Addr + Len) ; i += BRAM_BYTENUM)
    {
        XBram_WriteReg(XPAR_BRAM_0_BASEADDR, i , Write_Data) ;
        Write_Data += 1 ;
    }
    //Set ram read and write length
    PL_RAM_CTRL_mWriteReg(PL_RAM_BASE, PL_RAM_LEN , BRAM_BYTENUM*Len) ;
    //Set ram start address
    PL_RAM_CTRL_mWriteReg(PL_RAM_BASE, PL_RAM_ST_ADDR , BRAM_BYTENUM*Start_Addr) ;
    //Set pl initial data
    PL_RAM_CTRL_mWriteReg(PL_RAM_BASE, PL_RAM_INIT_DATA , (Start_Addr+1)) ;
    //Set ram start signal
    PL_RAM_CTRL_mWriteReg(PL_RAM_BASE, PL_RAM_START , 1) ;

    return XST_SUCCESS ;
}
```

5. 中断服务程序中，BRAM 控制器读取 BRAM 的数据，并打印

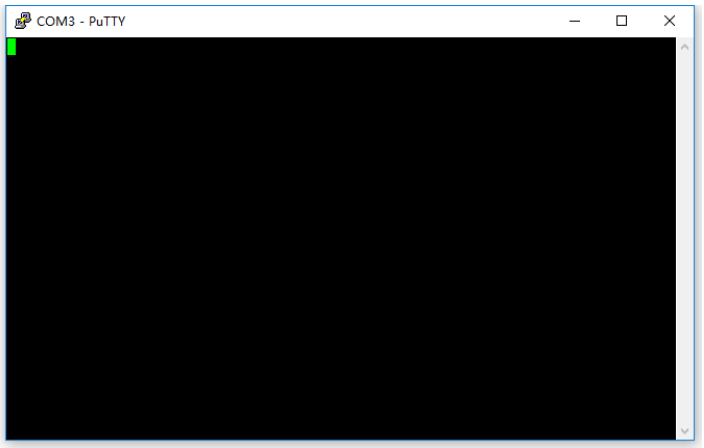
```
void IntrHandler(void *CallbackRef)
{
    int Read_Data ;

    int i ;
    printf("Enter interrupt\t\n");
    //clear interrupt status
    PL_BRAM_CTRL_mWriteReg(PL_RAM_BASE, PL_RAM_CTRL , INTRCLR_MASK) ;

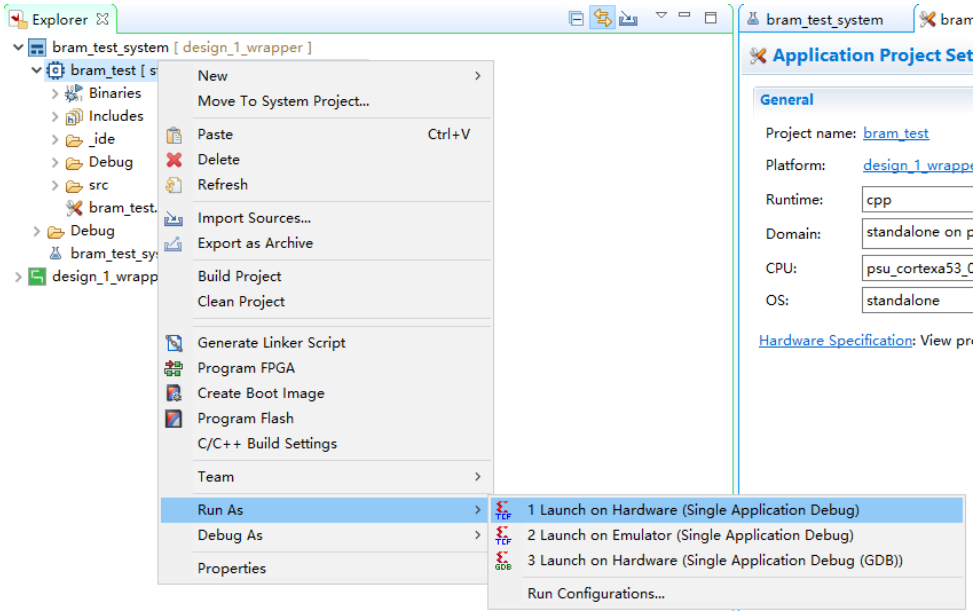
    for(i = BRAM_BYTENUM*Start_Addr ; i < BRAM_BYTENUM*(Start_Addr + Len) ; i += BRAM_BYTENUM)
    {
        Read_Data = XBram_ReadReg(XPAR_BRAM_0_BASEADDR , i) ;
        printf("Address is %d\t Read data is %d\t\n",  i/BRAM_BYTENUM ,Read_Data) ;
    }
    Intr_flag = 1 ;
}
```

24.3 实验现象

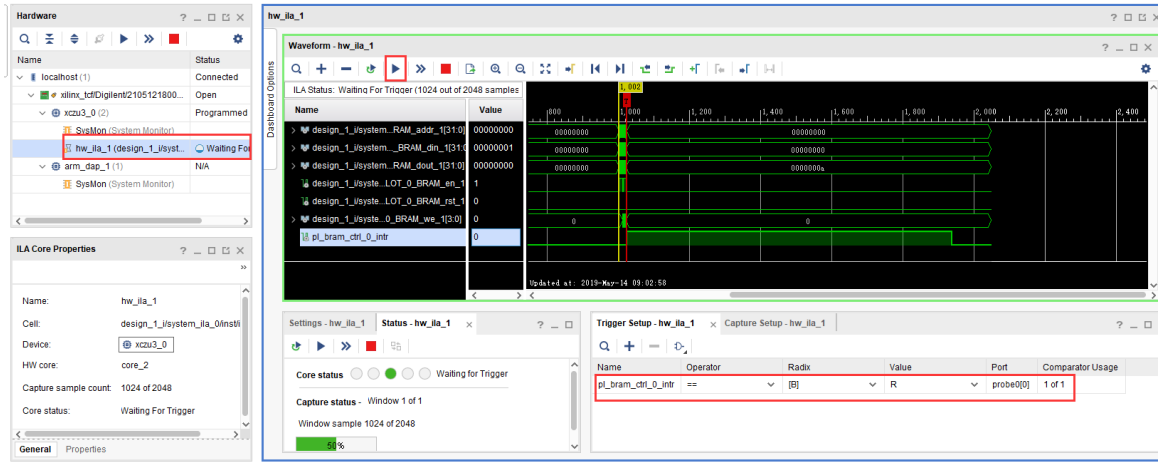
1. 打开串口



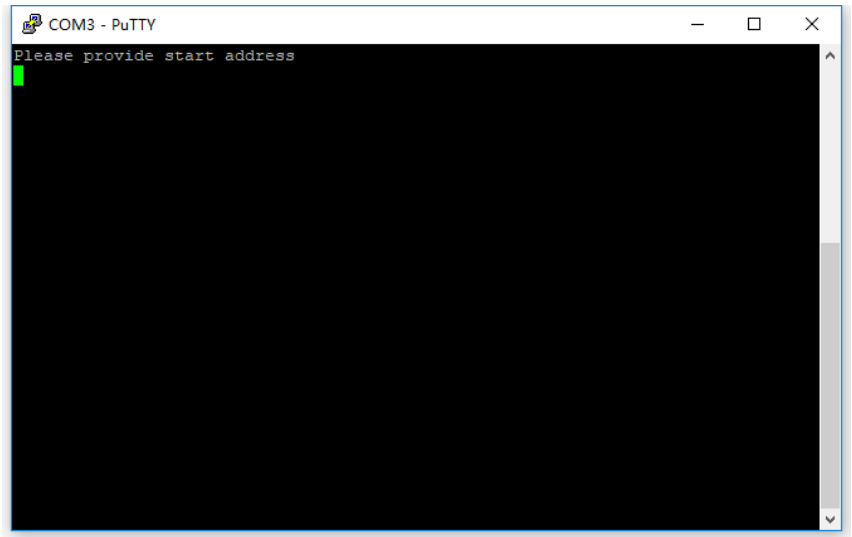
2. 通过 Run Configurations 下载程序



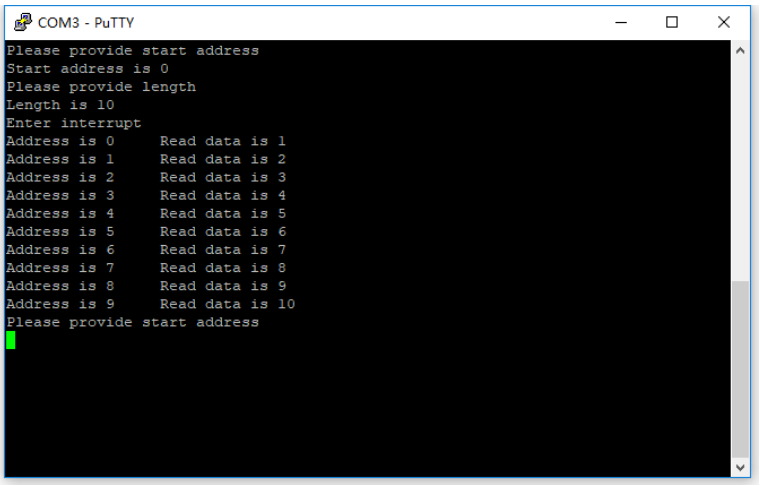
3. 打开 Vivado 的 Hardware Manager（与 PL 端联合调试），设置将 intr 中断信号作为触发信号，选择上升沿触发，点击开始按钮，可以看到 hw_ila_1 变成了 Waiting for trigger 状态



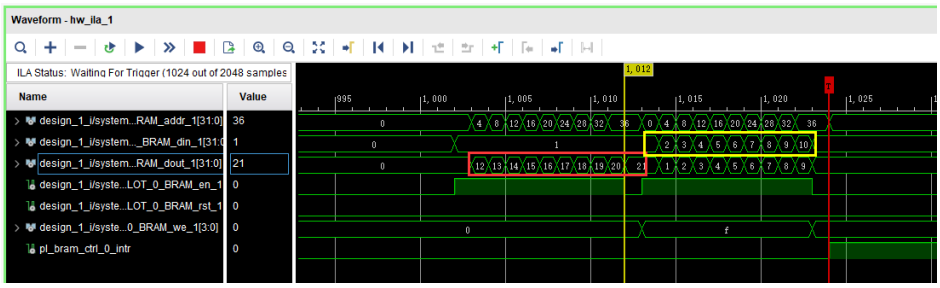
4. 在串口软件中，输入起始地址，由于 BRAM 询址为 1K，那么可以设置为 0~1023，长度设置为 1~1024，注意起始地址+长度不要超过 1024，因为超出了询址空间。



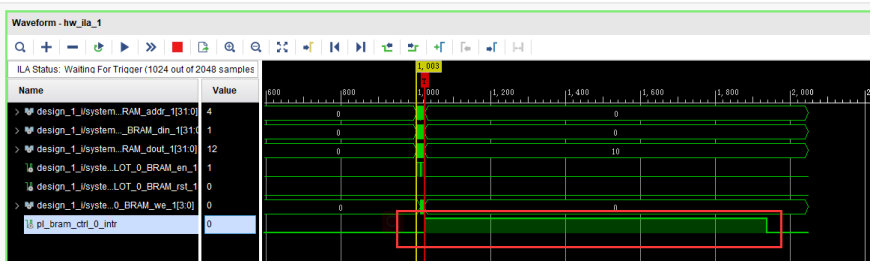
5. 输入的数据为十进制数，输入结束按回车



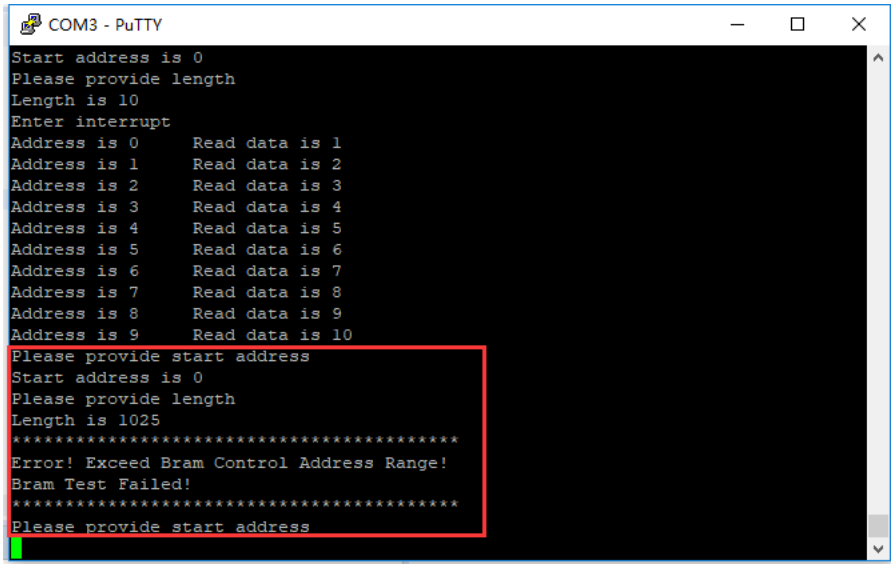
6. 打开 ILA 逻辑分析仪，可以看到已经触发，首先是 PL 控制器从 BRAM 读数据，之后是写数据，可以看到红色为 PL 读出的 BRAM 数据，正是 CPU 写入的数据，从 12 开始，共 10 个数据，PL 写入的数据为黄色部分从 1 开始，共 10 个数据，正与上图 CPU 读 BRAM 的数据相符。



同样也能看到中断信号的状态



7. 如果超出范围，打印错误信息，需要重新输入有效信息



1.1 本章小结

以上就是 PS 与 PL 通过 BRAM 实现低带宽数据交互的实验，两者通过 GP 口进行数据互连，可以实现小批量的数据交互。

知识点为逻辑分析仪的使用，中断的使用，自定义 IP 等。

第二十五章 基于 AN5641 模块的 MIPI 采集 DP 显示

实验 Vivado 工程目录为“an5641_mipi_dp /vivado”。
实验 vitis 工程目录为“an5641_mipi_dp /vitis”。

本章介绍基于 AN5641 模块的 MIPI CSI-2 图像采集。MIPI 协议比较复杂，本章旨在介绍基础的概念，待用户深入学习。

25.1 原理介绍

MIPI 联盟，即移动产业处理接口（Mobile Industry Processor Interface）联盟，MIPI 联盟制定了一整套的接口标准，将移动设备的接口，比如摄像头、显示屏等接口标准化。用于摄像头采集的称为 CSI 接口，用于显示的称为 DSI。由于是摄像头，所以我们要学习的是 CSI 接口。CSI 接口又分为物理层（D-PHY）和协议层（CSI-2）。

25.1.1 MIPI 物理层(D-PHY)

以下为物理层的结构图，Dp/Dn 为差分接口，接口部分又分为 LP（low power），顾名思义，即低功耗模式，可控制接口进入 sleep 状态，或者进行状态的切换，电压摆幅 1.2V；HS(high speed)，即高速接口，主要用于图像的传输，电压摆幅 200mV。而 D-PHY 的主要作用就是进行模式的切换，以及将数据串行转为并行。

具体内容可以参考工程目录下面的 *mipi_D-PHY_specification_v01-00-00.pdf*

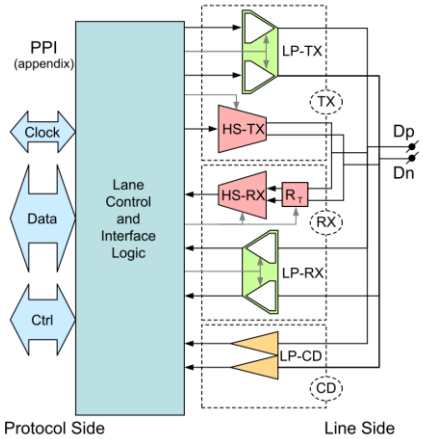


Figure 1 Universal Lane Module Functions

25.1.2 MIPI 协议层(CSI-2)

以下为协议层的结构图，CSI-2 需要将从 D-PHY 过来的并行数据协议解析，包括 bit 位的顺序调整，长包，短包数据解析，解析出图像数据。

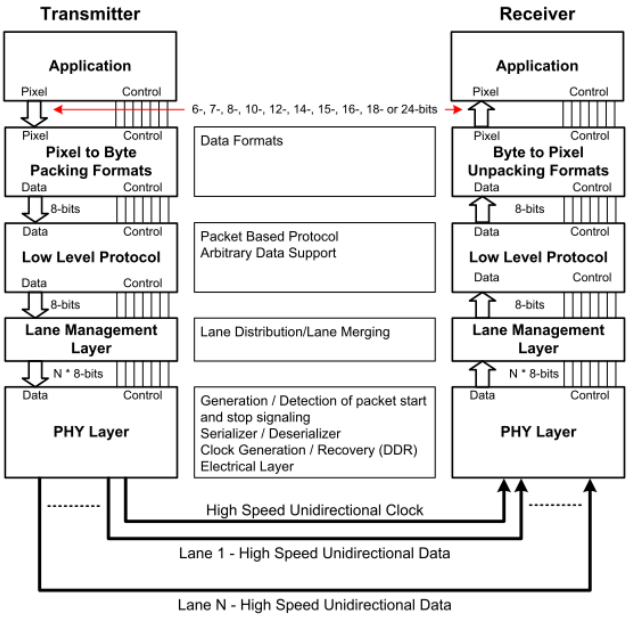


Figure 2 CSI-2 Layer Definitions

在 CSI-2 协议层很重要的就是短包与长包，短包可用于解释图像的帧起始位置，行号等，用于图像的同步。格式如下：

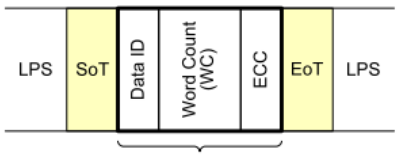


Figure 31 Short Packet Structure

长包主要用于图像数据的传输，并指定图像的格式，如 RGB888/RGB565/RAW10 等，可通过 Data ID 指定。其格式如下：

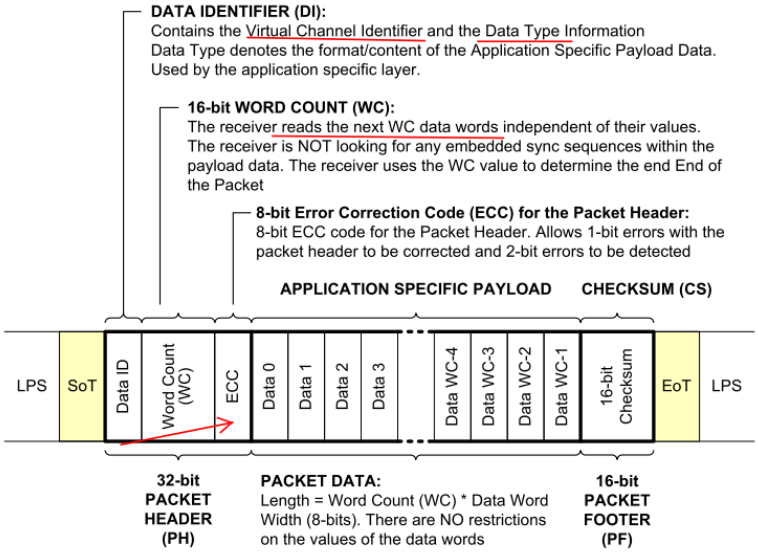


Figure 30 Long Packet Structure

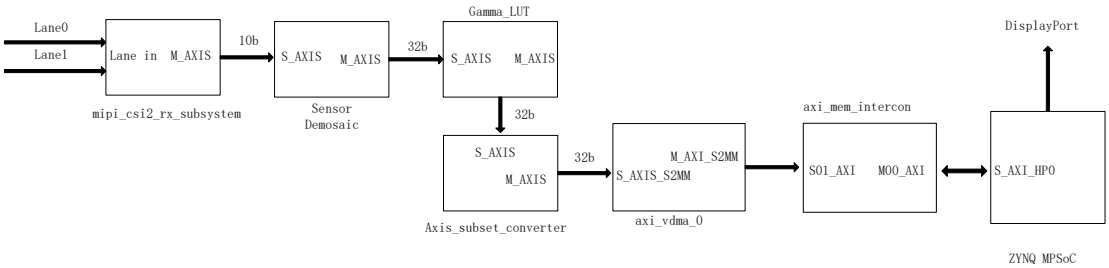
关于 MIPI CSI-2 具体内容，请参考工程目录下的*_MIPI Alliance Specification for Camera Serial Interface 2 (CSI-2).pdf*

FPGA 工程师工作内容

以下为 FPGA 工程师负责内容。

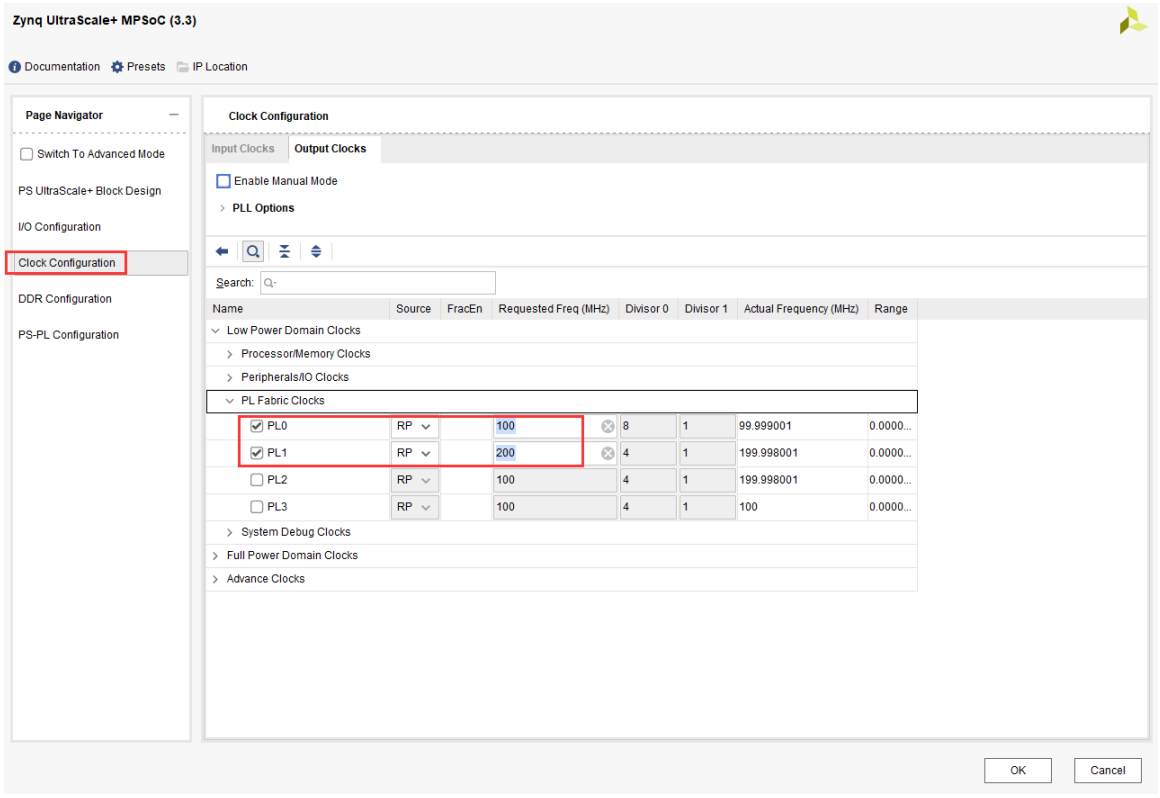
25.2 硬件环境搭建

工程采用两个 lane 的 MIPI 输入，MIPI 摄像头配置为 RAW10 输出。通过 mipi_csi2_rx_subsystem 模块进行协议解析并转换成 AXIS 流数据，并通过 bayertoRGB 模块将 RAW 转换成 RGB 数据，之后经过 Gammer 校正等模块，进入 VDMA，之后进入 HP 口。

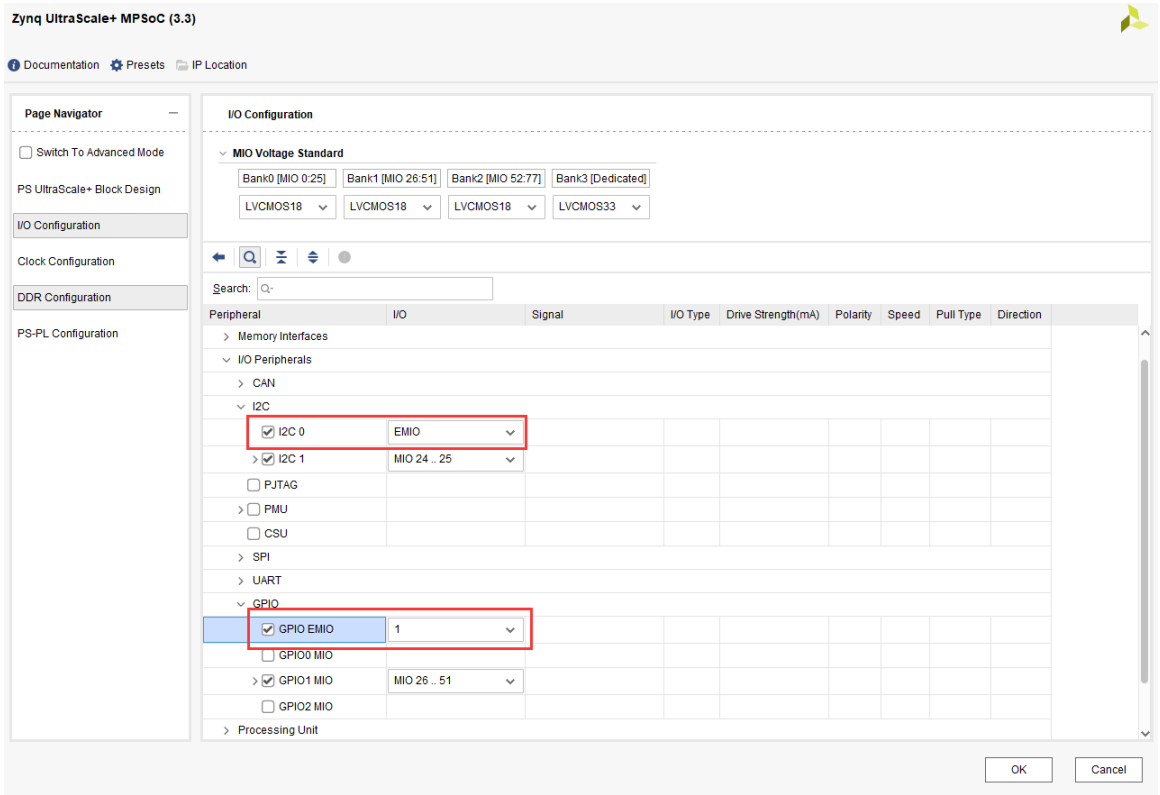


基于“ps_hello”工程，我们添加 MIPI 采集部分的模块。

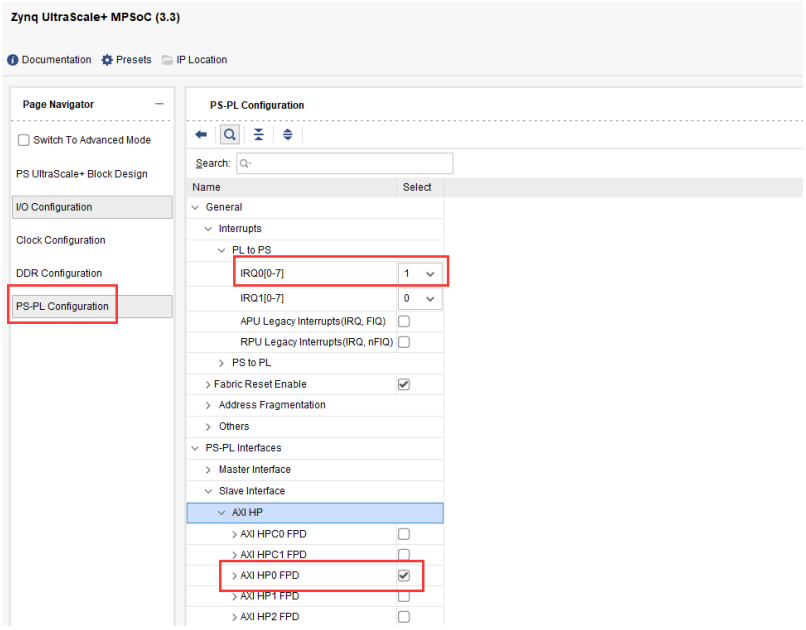
1. 首先配置 ZYNQ 核，100MHz 用于数据传输，200MHz 用于 MIPI 模块参考时钟。



配置 i2c 为 EMIO，用于配置摄像头寄存器，GPIO EMIO 设置为 1，用于配置摄像头使能



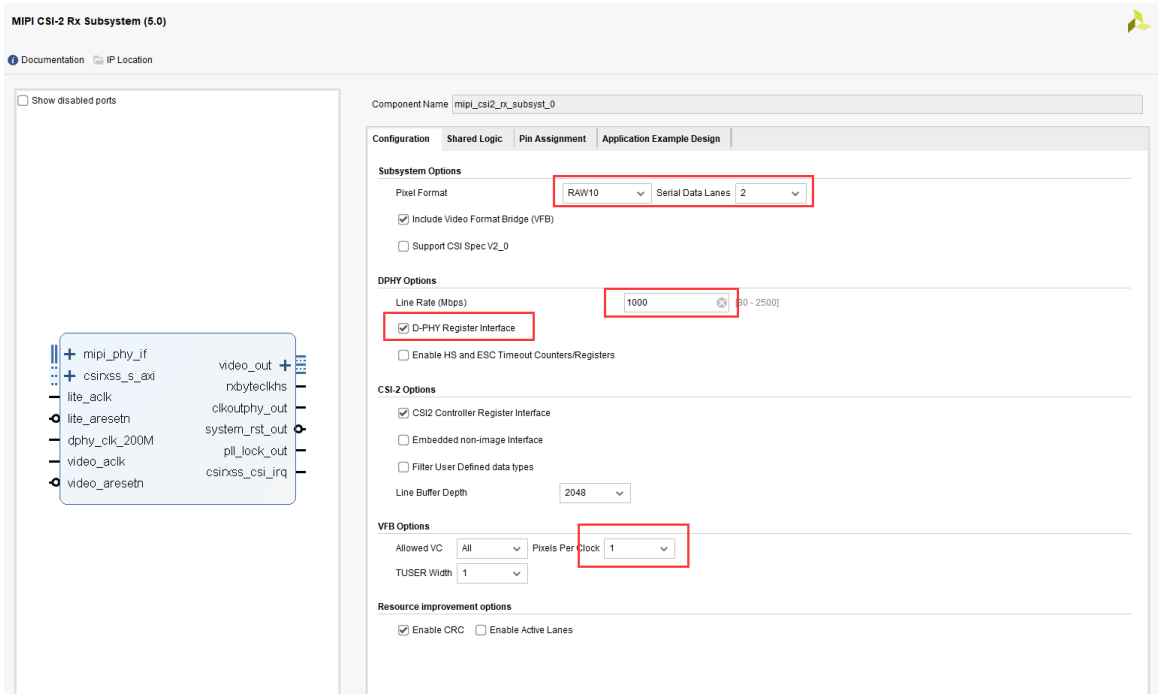
配置中断和 HP 口



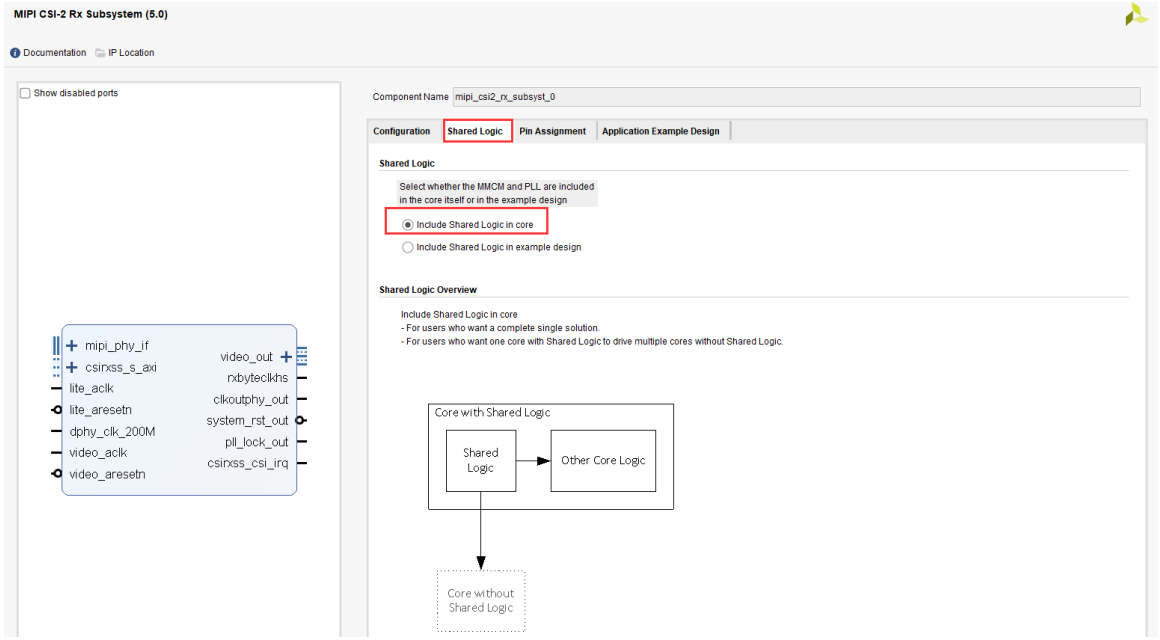
并将时钟连接上



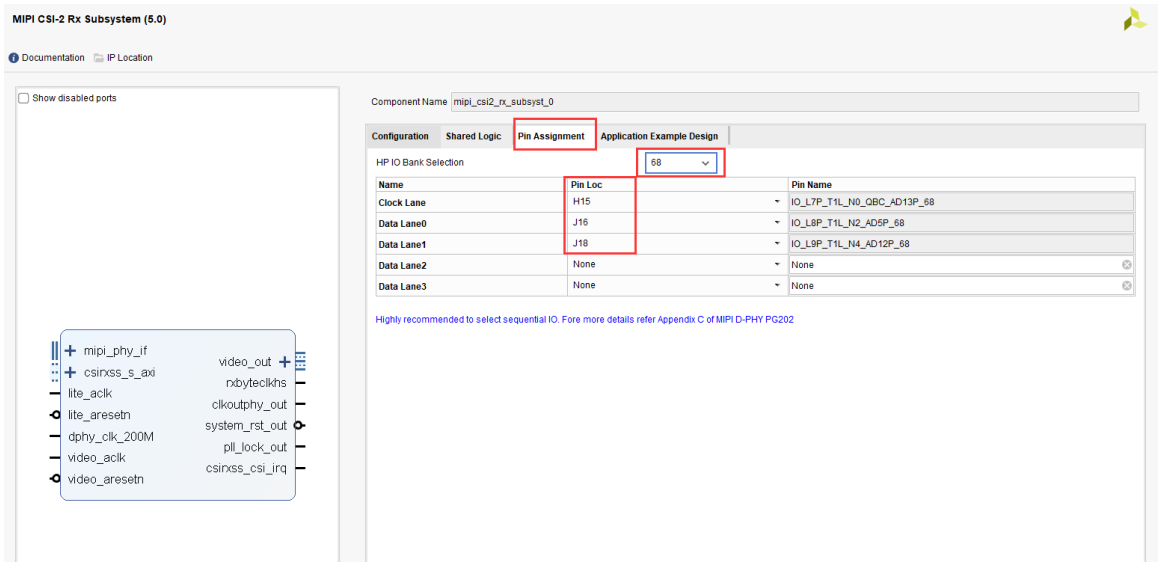
2. 添加 mipi_csi2_rx_subsystem 模块，用于 MIPI 数据的接收与解析，转成 axi-stream 接口。配置如下，数据格式选择 RAW10，选择 2 lane，Line Rate 配置为 1000Mbps，指的是最大支持的速率，也可以根据自己的需求填写，范围为 80-2500；Pixels Per Clock 默认配置为 1，表示 1 个周期为 1 个像素；



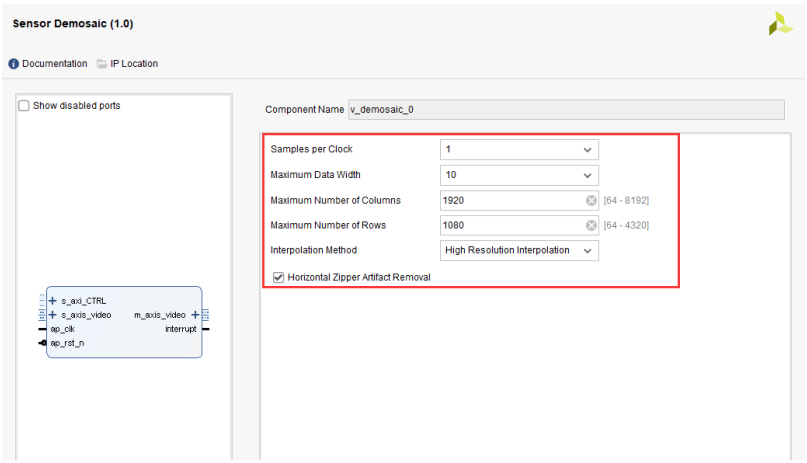
Shared Logic 选择“Include Shared Logic in core”



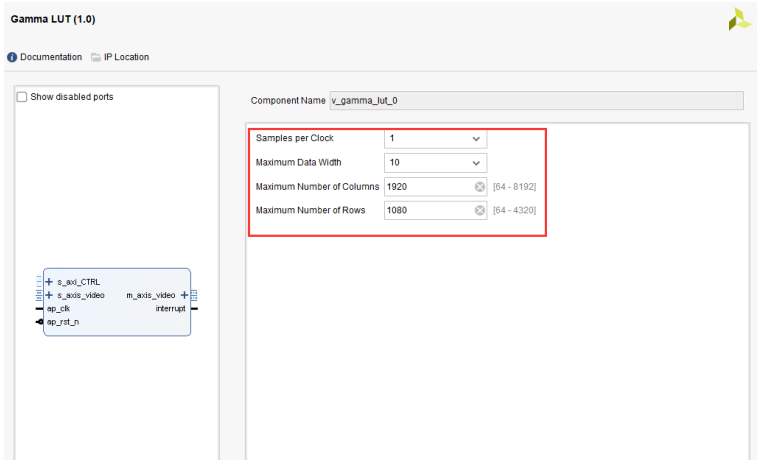
Pin Assignment 根据原理图按照以下配置



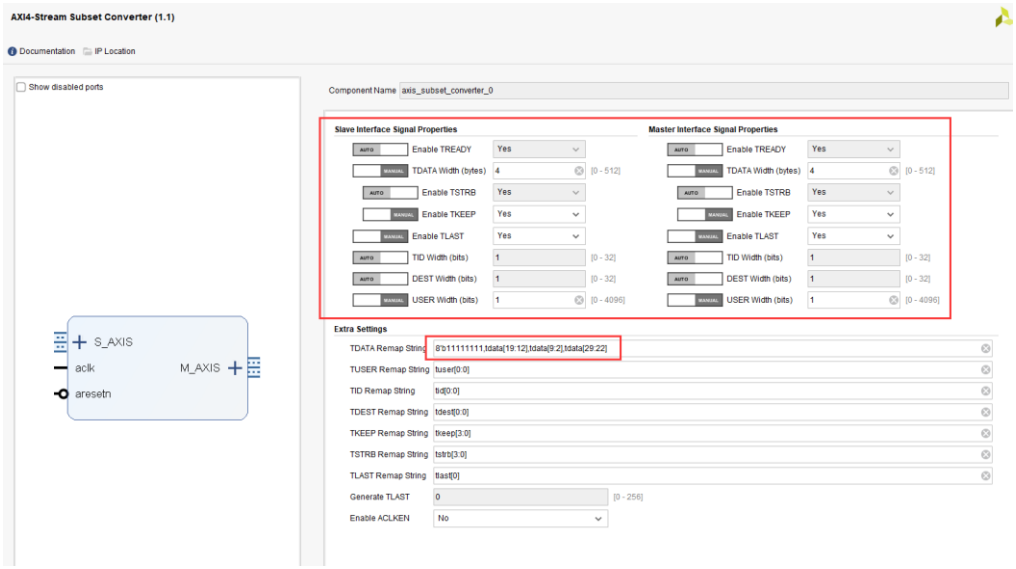
3. 添加 Sensor Demosaic 模块，用于将 RAW10 转成 RGB，配置如下，数据位宽设置为 10，最大分辨率为 1920*1080



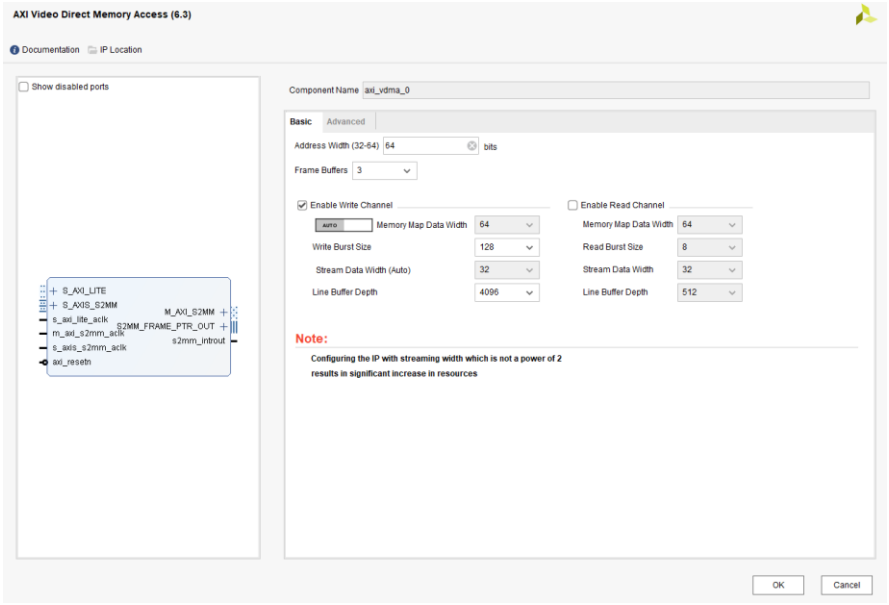
4. 添加 Gamma LUT 模块，用于 Gamma 校正



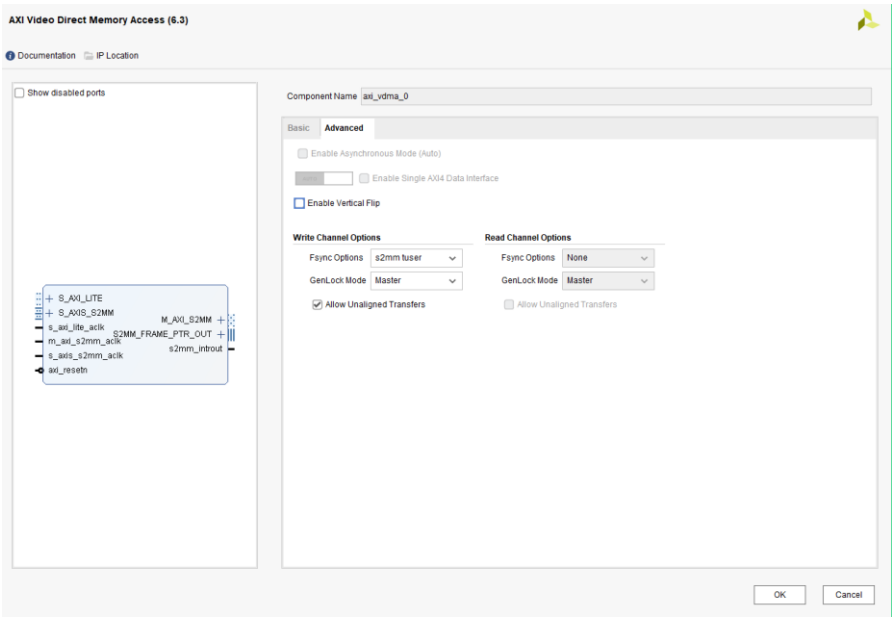
5. 添加 subset 模块，调整图像数据的顺序，因为经过实际操作发现，图像的 RGB 数据顺序需要调整。



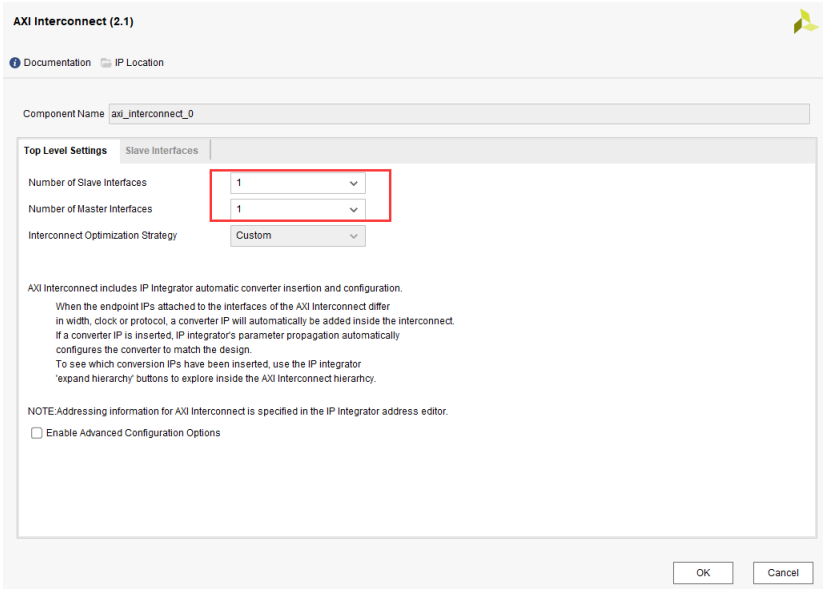
6. 添加 VDMA 配置如下，打开写通道，关闭读通道。这里主要牵涉到两个接口，Memory Map 接口，采用 AXI4 接口，与 ZYNQ HP 口进行数据交互，向 PS 端 DDR 写入图像数据，按照默认 Auto。另外一个接口就是 Stream 接口，也就是 AXI4 stream 流接口，在这里主要是用来接收摄像头采集的图像，这里的 Stream Data Width 是 Auto 的，也就是连接多少位宽就是多少。Frame Buffers 为帧缓存数，可以储存多帧图像，本实验中使能 3 帧图像缓存。Line Buffer Depth 类似于 fifo 缓存，以 Stream Data Width 为单位，设置大些，可以缓存的数据越多。VDMA 也是一种 DMA，本实验的主要作用是将采集的摄像头数据写到 PS 端 DDR 中。



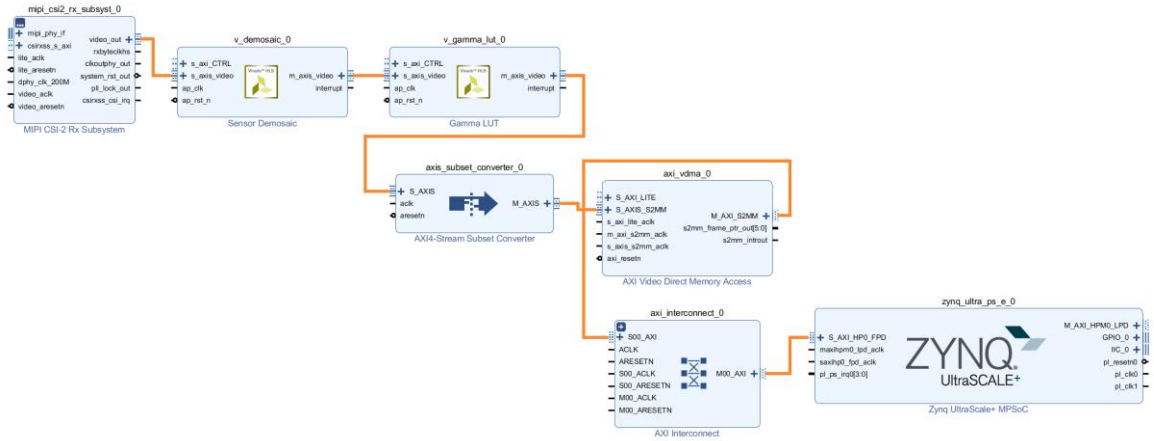
配置 VDMA 高级参数，在这里使能 Allow Unaligned Transfers，如果不使能，在软件中就要对数据按照 Memory Map Data Width 对齐，比如我们设置的是 64 位，也就是要 8 字节对齐。但这里使能了，就可以进行不对齐的数据传输。GenLock 用于避免读和写通道同时访问同一个 frame，而使图像显示不正常。由于我们只有一个写通道，设置它的意义并不是很大，需要与写通道配置才有用处。组合方式比较多，具体可以参考 VDMA 的手册 PG020。



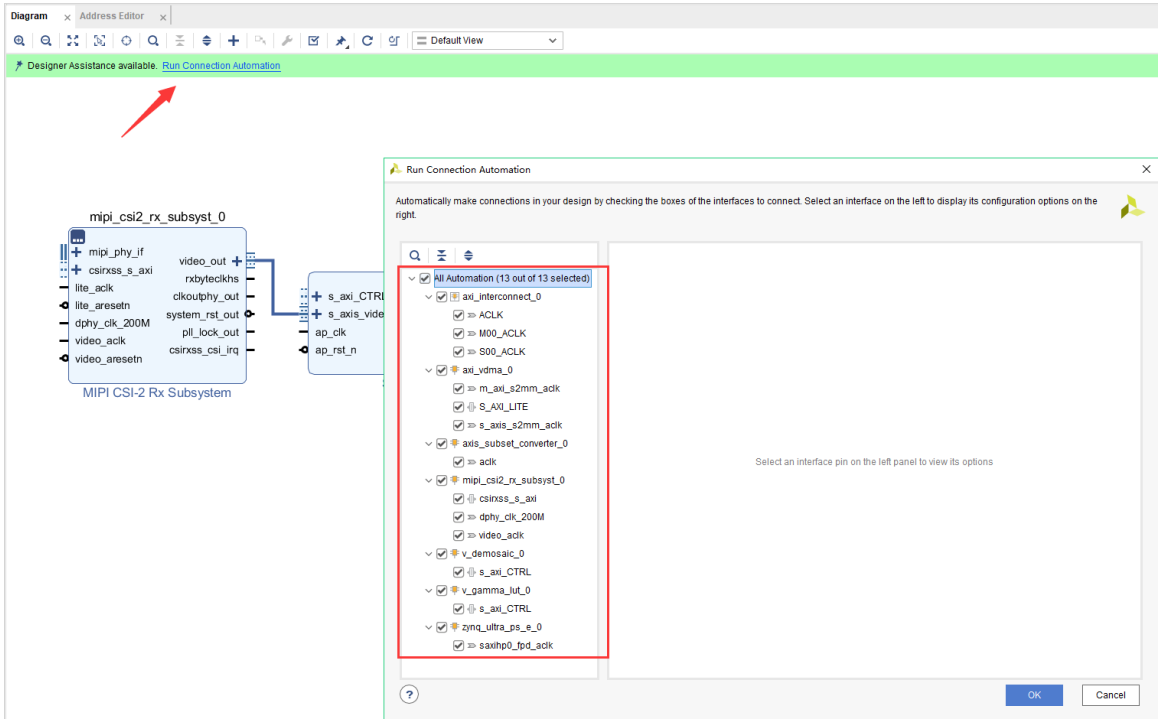
7. 添加一个 AXI Interconnect 模块，并配置成 1 个 master，1 个 slave



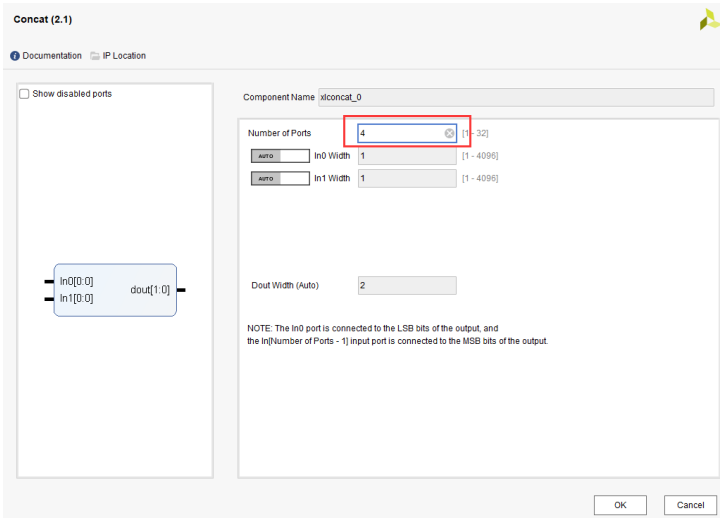
8. 连接关键信号

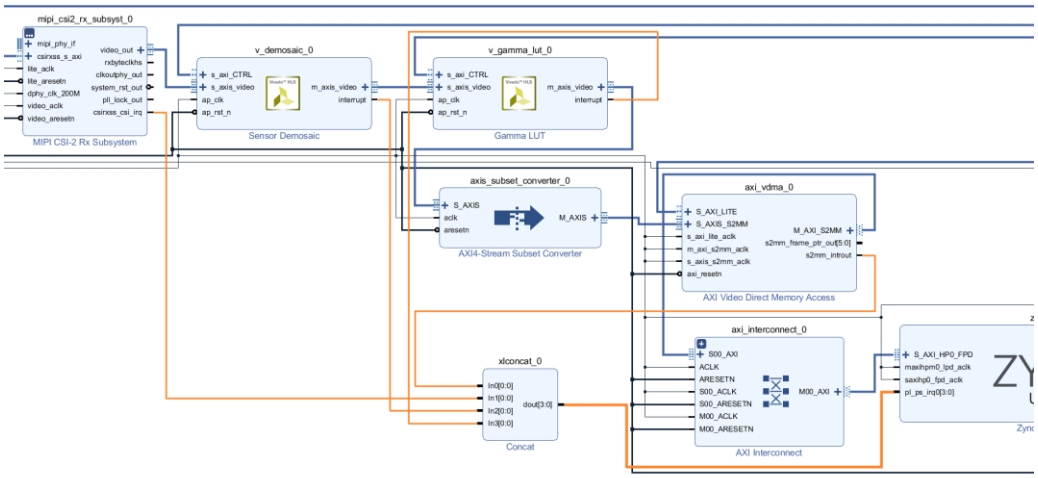


9. 自动连接

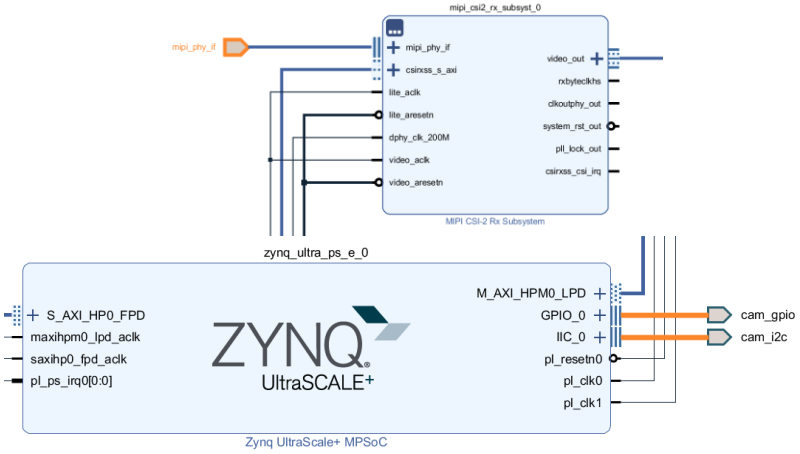


10. 添加 concat 模块连接中断





11. 导出引脚，并修改名称



12. 至此硬件搭建完毕，生成 bitstream，并导出硬件信息。

软件工程师工作内容

以下为软件工程师负责内容。

25.3 Vitis 程序开发

关于 Vitis 程序也较为简单，在 VDMA 的基础上，添加摄像头的初始化，VDMA 的配置，前面要进行摄像头的复位，以及 I2C 的初始化。

```
PsGpioSetup() ;
/*
 * Reset sensor
 */
XGpioPs_WritePin(&Gpio, CAM_EMIO, 0) ;
usleep(1000000);
XGpioPs_WritePin(&Gpio, CAM_EMIO, 1) ;
usleep(1000000);

i2c_init(&ps_i2c0, XPAR_XIICPS_0_DEVICE_ID,100000);
```

在最后，配置 MIPI 摄像头，以及启动摄像头的 VDMA

```
/*
 * DP dma demo
 */
xil_printf("DPDMA Generic Video Example Test \r\n");
Status = DpdmaVideoExample(&RunCfg, pFrames[0]);
if (Status != XST_SUCCESS) {
    xil_printf("DPDMA Video Example Test Failed\r\n");
    return XST_FAILURE;
}

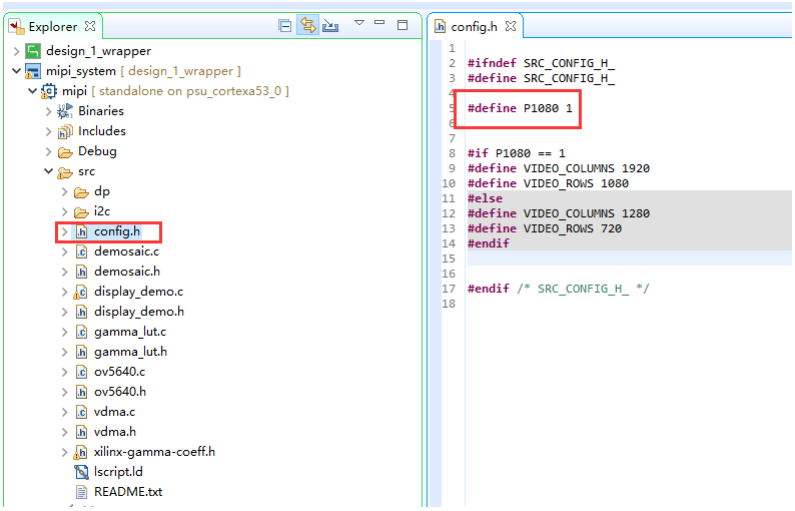
gamma_lut_init();
demosaic_init();

/* Start Sensor Vdma */
vdma_write_init(XPAR_AXIVDMA_0_DEVICE_ID,HORSIZE,VERSIZE,DEMO_STRIDE,(unsigned int)pFrames[0]);

/*
 * Initialize Sensor
 */
sensor_init(&ps_i2c0);

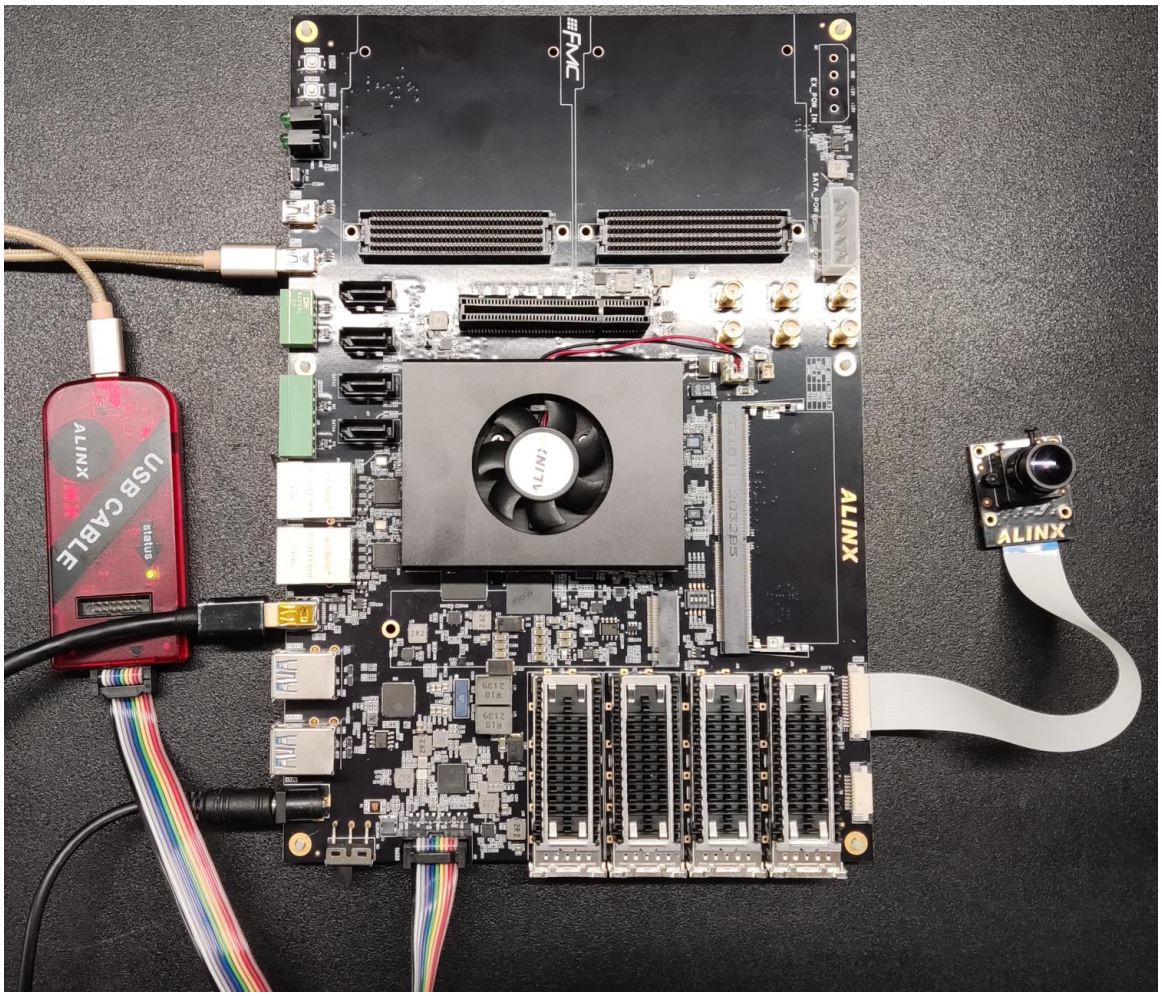
return 0;
```

目前程序支持两种分辨率的配置，720p@60fps 和 1080p@30fps，如果想改成 1080p，需要修改 config.h 中的宏定义，将 P1080 的值改为 1 即可。

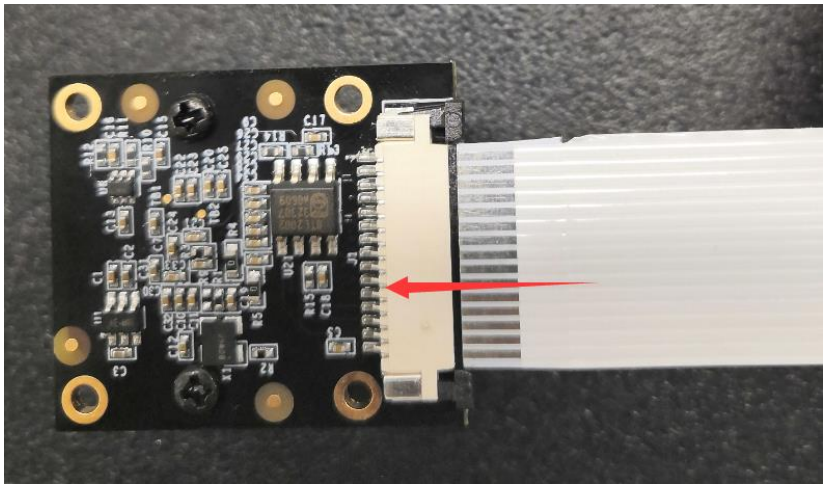


25.4 板上验证

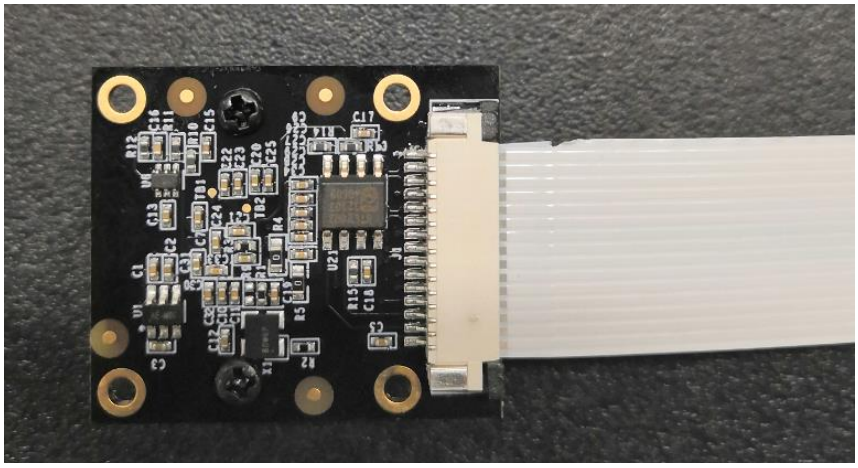
连接 MIPI 摄像头如下图所示



硬件连接图

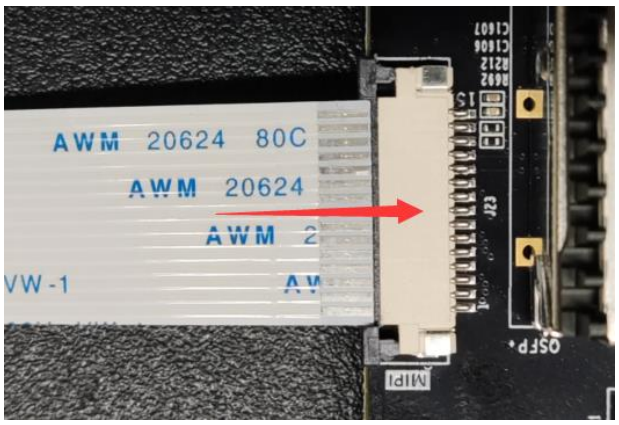


摄像头排线连接



摄像头连接后效果

注意排线的方向一定不要接反!!!



板卡排线连接

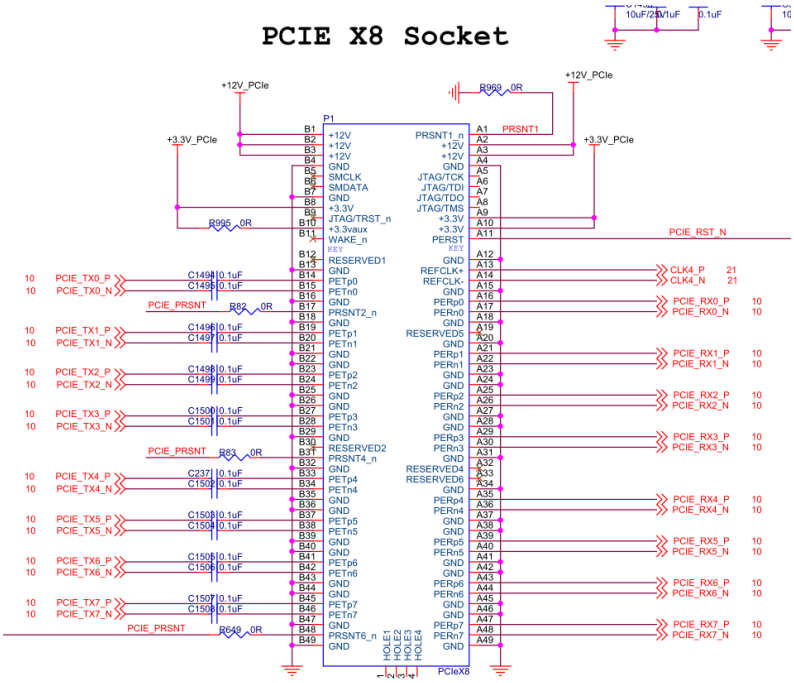


板卡连接后效果

下载程序后，显示器则会显示图像。

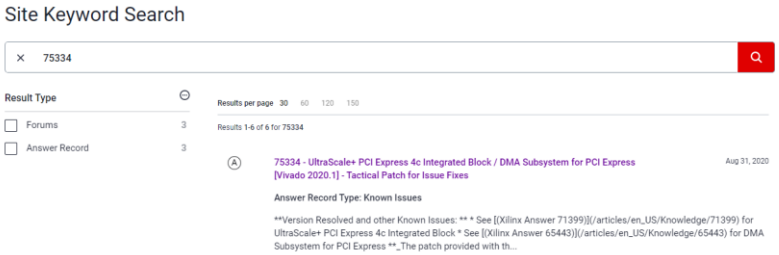
第二十六章 PCIe ROOT

板卡上设计了 PCIE X8 的 ROOT 插槽，本章节介绍 Vivado 工程的创建，应用部分不在本章介绍，请参考

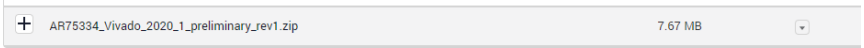


26.1 硬件环境搭建

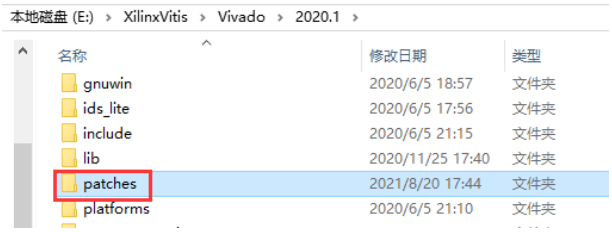
由于 2020.1 的 XDMA 有些 bug，需要添加一个补丁，AR75334，可以在 xilinx 官网搜索，或者在提供的例程下面找到这个压缩包。



下载补丁

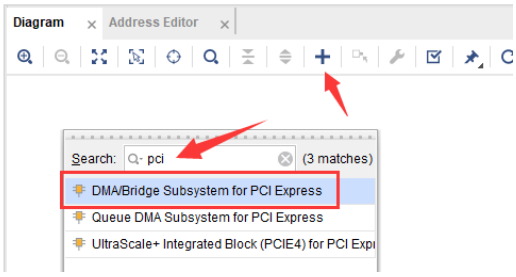


找到 2020.1 安装路径，如果 xx\Vivado\2020.1 目录下没有 patches 文件夹，则新建一个，并将下载的压缩包解压到这个文件夹下。

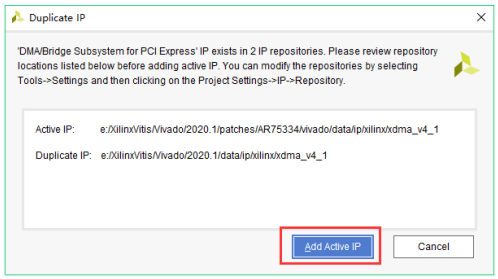


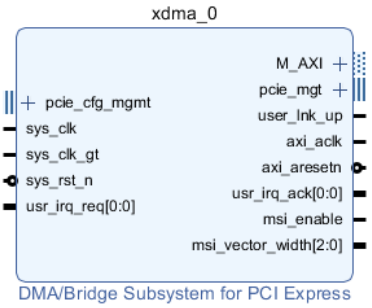
利用 ps_hello 工程另存为 pcie_root 工程。

1. 添加 XDMA 模块

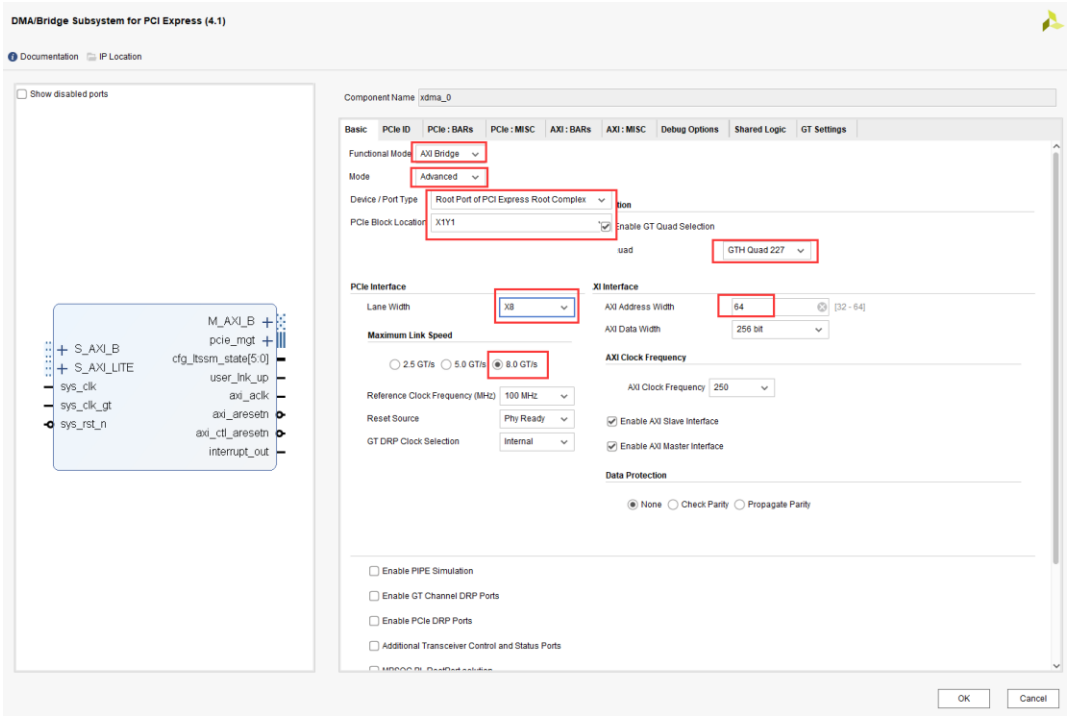


添加当前有效的 IP，也就是 AR75334

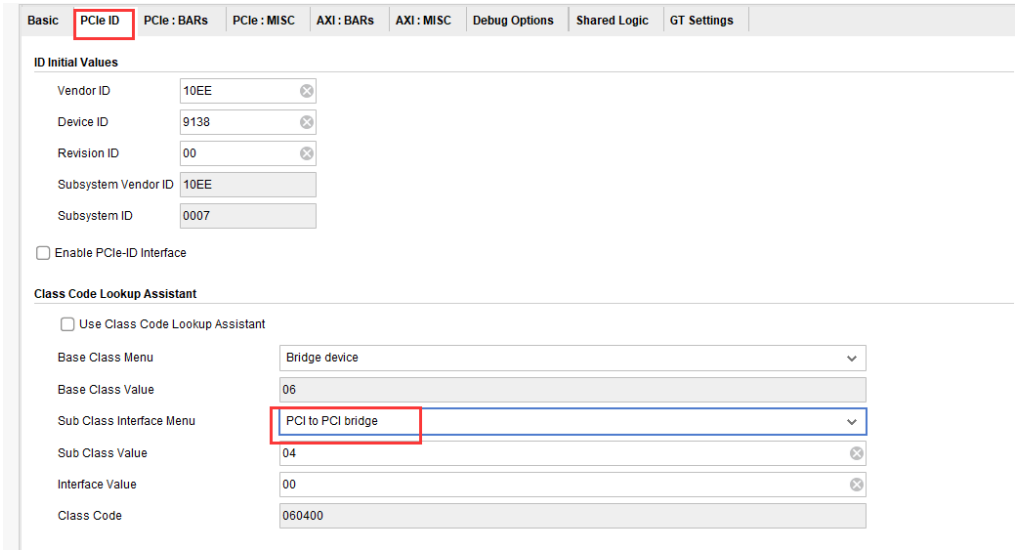




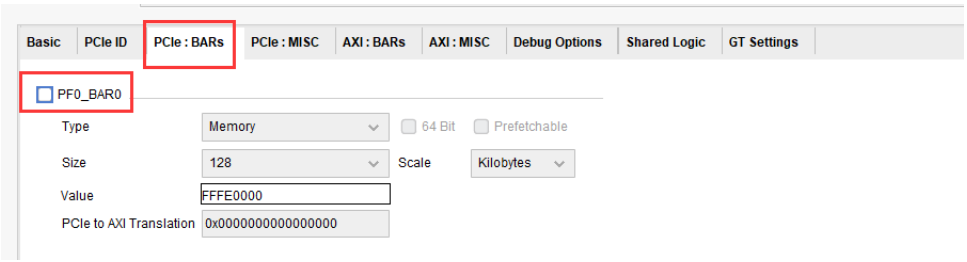
2. 打开 XDMA 进行配置



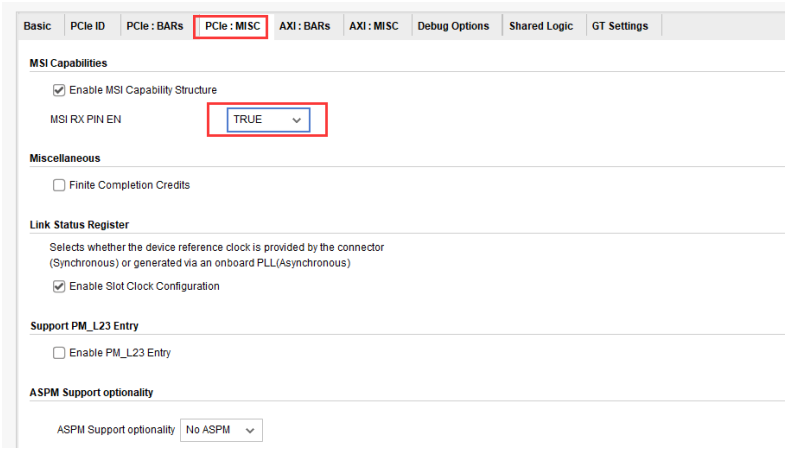
3. PCIe ID 中 Sub Class Interface Menu 选择 PCI to PCI bridge



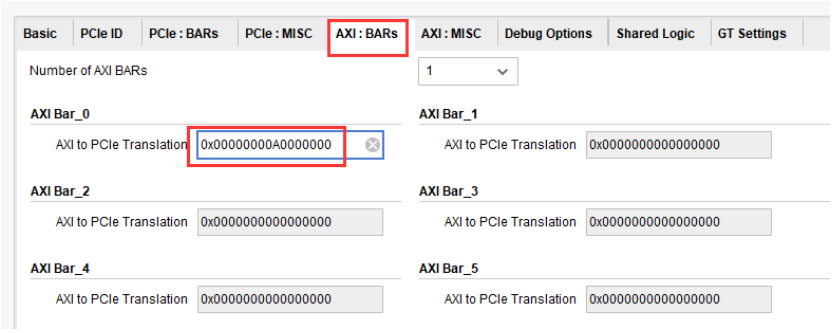
4. 取消勾选 PF0_BAR0



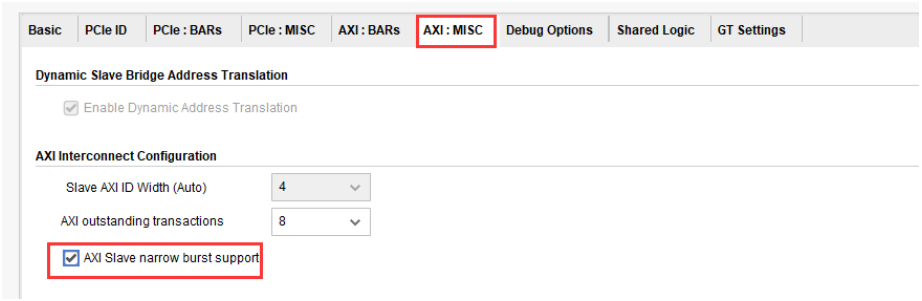
5. 配置 MSI RX PIN EN 为 TRUE



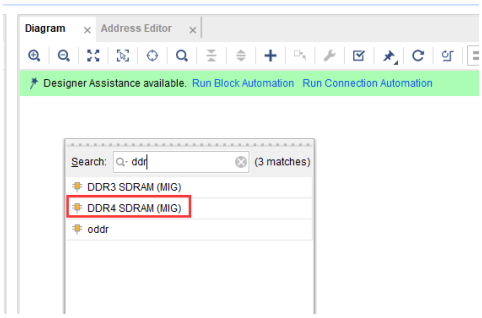
6. 将 BAR 的地址设置为 0x00000000A0000000，这个地址是 AXI BAR 总线的地址，后面的步骤会讲到。



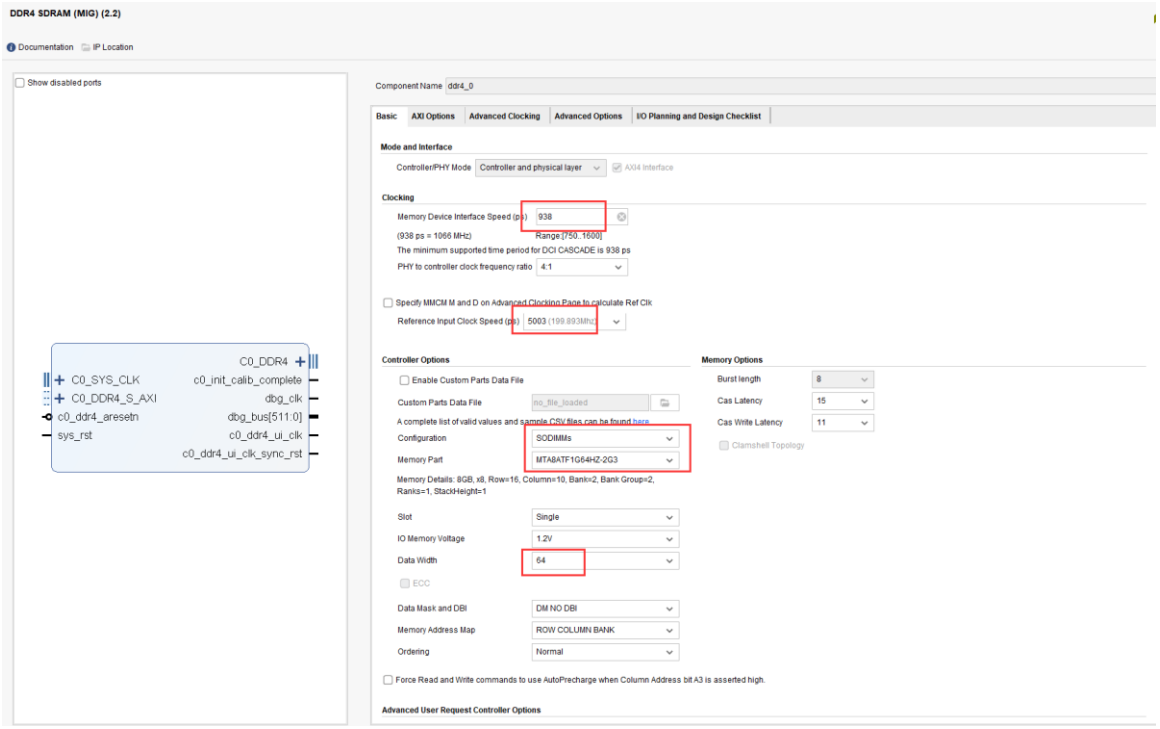
7. 将 AXI 的 narrow burst 打开



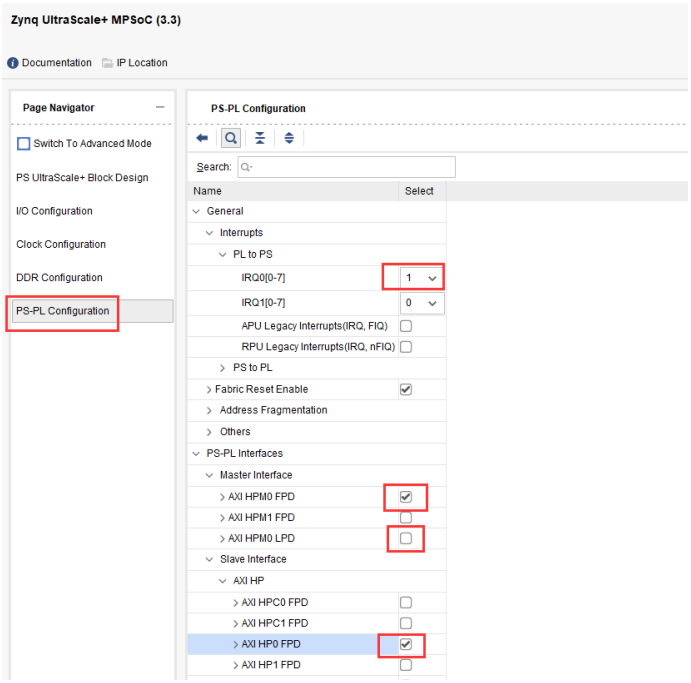
8. 其他保持默认，点击 OK，完成配置
9. 添加 DDR4



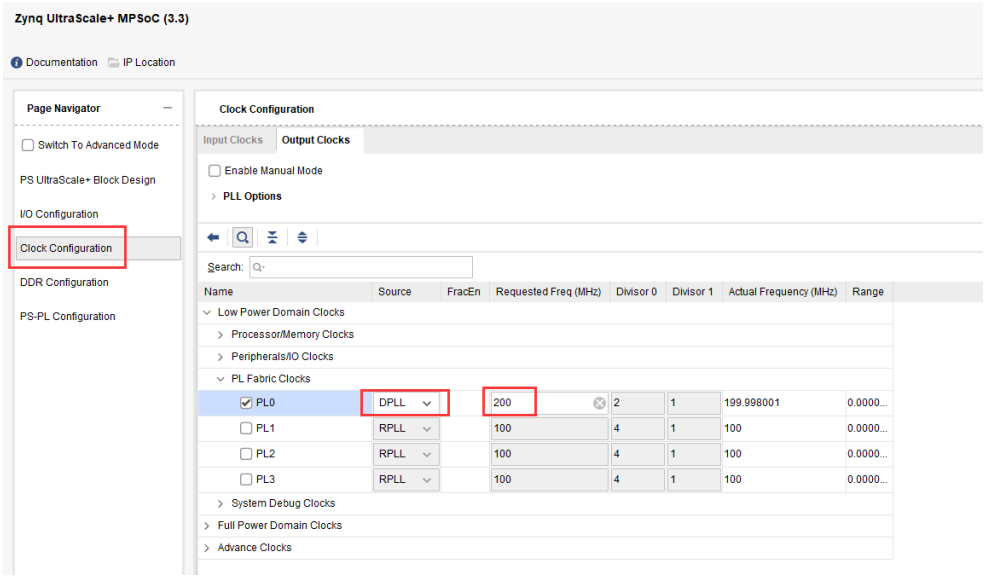
10. 根据自己的需要选择 SODIMM 型号，这里以 MTA8ATF1G64HZ-2G3 举例，其他保持默认。



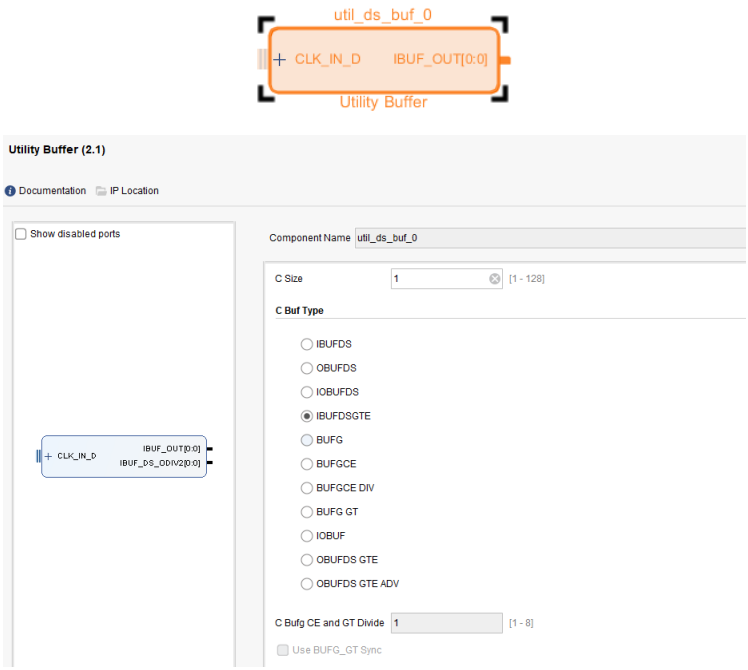
11. 配置 ZYNQ 核，打开中断，打开 AXI HPM0 FPD，关闭 AXI HPM0 LPD，打开 AXI HP0 FPD



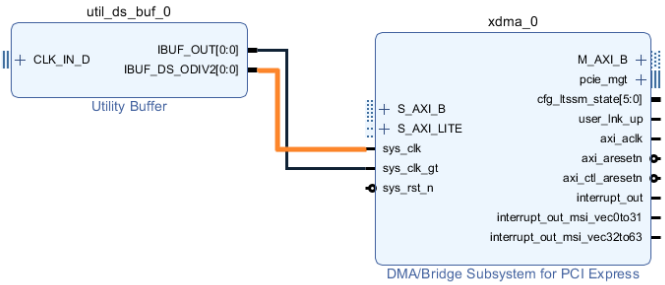
12. 将 PL 端时钟配置为 200MHz，增大带宽，配置完成，点击 OK



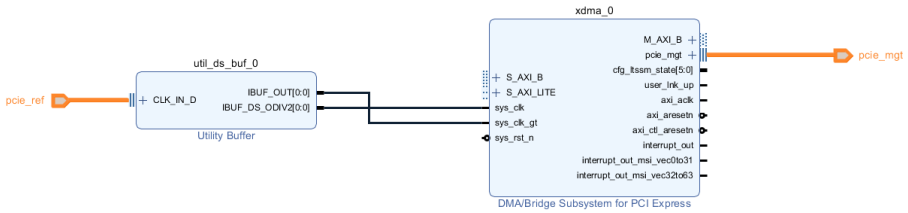
13. 添加 utility buffer，配置成 IBUFDSGTE



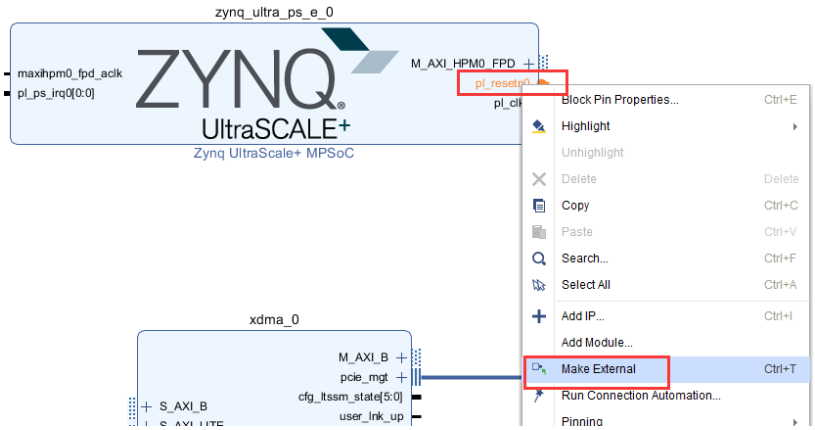
14. 连接时钟如下

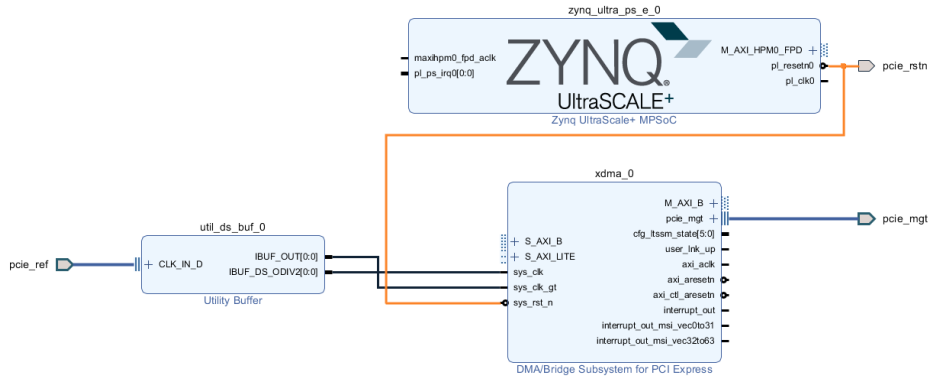


15. 导出参考时钟和收发器，并修改引脚名称

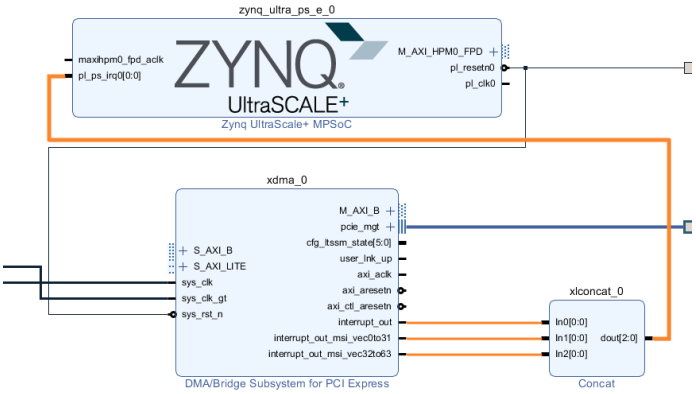


16. 将 pl_resetn0 导出，并修改名称，用于 Endpoint pcie 设备的复位，并连接到 xdma 的 sys_rst_n 引脚。

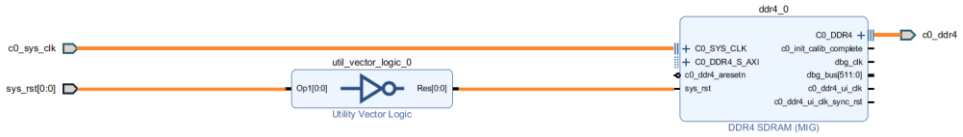




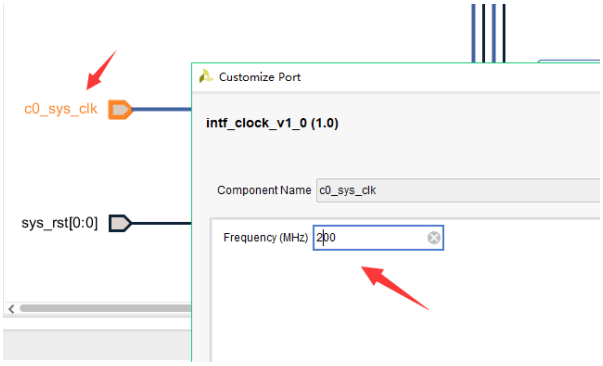
17. 添加 concat 连接中断



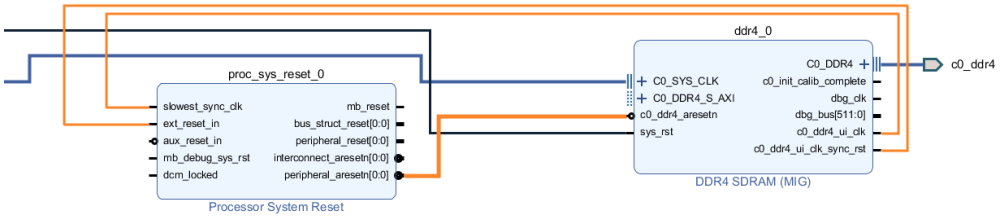
18. 添加 utility vector logic，并配置成 1 位的反相器连接到 ddr4 的 sys_rst，因为 sys_rst 是高电平复位，外部按键是低电平复位，因此添加反相器。并将引脚导出如下：



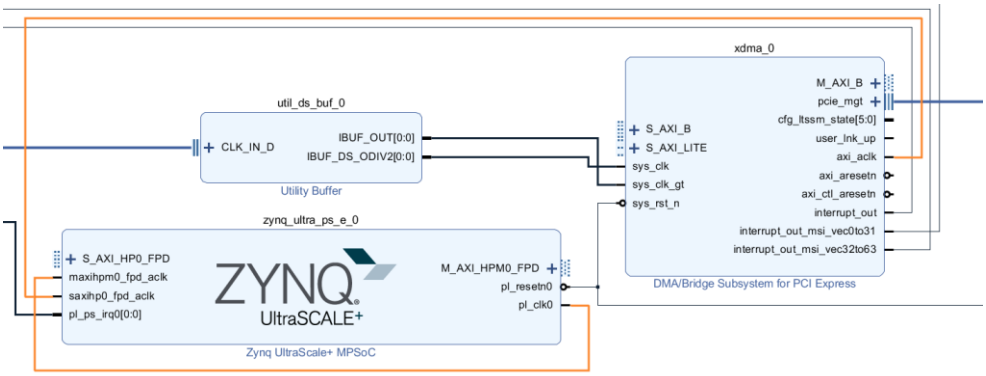
双击参考时钟，并修改为 200



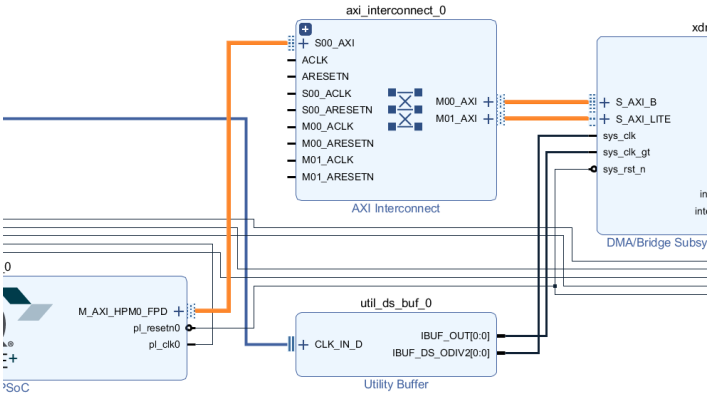
19. 添加复位模块并连接如下



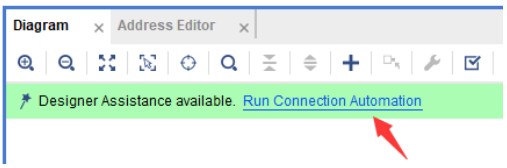
20. 连接时钟如下



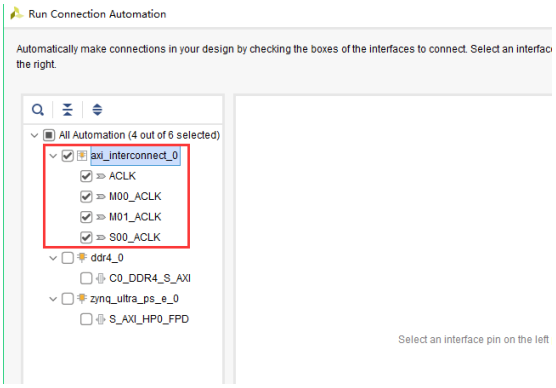
21. 添加一个 AXI Interconnect，连接如下



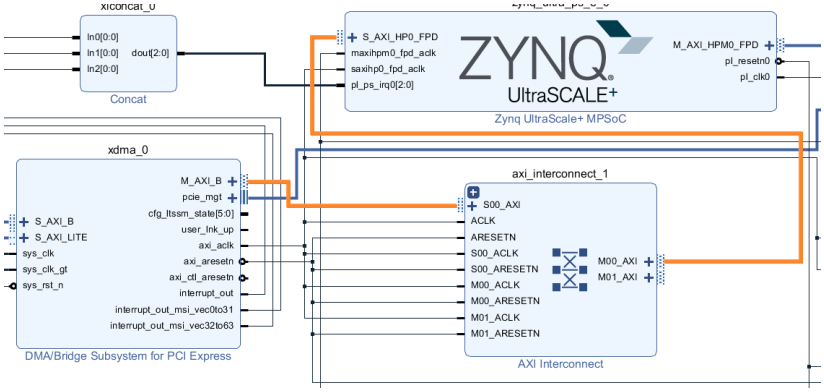
22. Run Connection Automation 自动连接



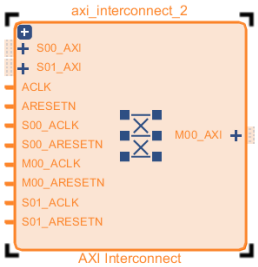
选择以下信号，点击 OK



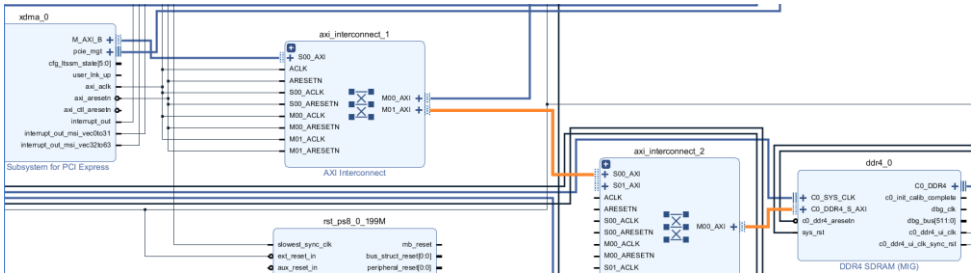
23. 再添加一个 AXI interconnect，一个 AXI master 连接 HP 口，一个连接 SODIMM，并连接时钟和复位，可参考提供的例程。



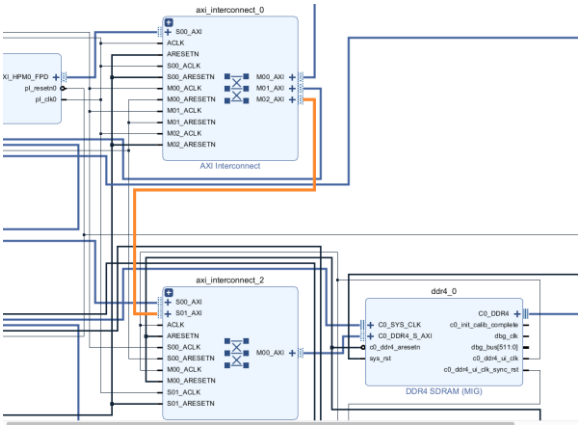
24. 再添加一个 AXI interconnect，并配置成 2 个 slave，1 个 master



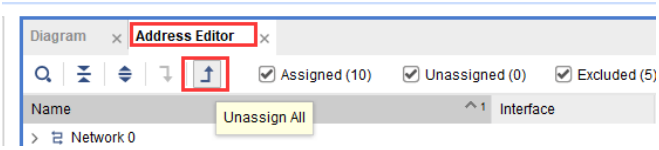
连接 M00_AXI 到 ddr4 的 C0_DDR4_S_AXI，S00_AXI 连接到 axi_interconnect_1 的 M01_AXI，用于 xdma 访问 DDR4。



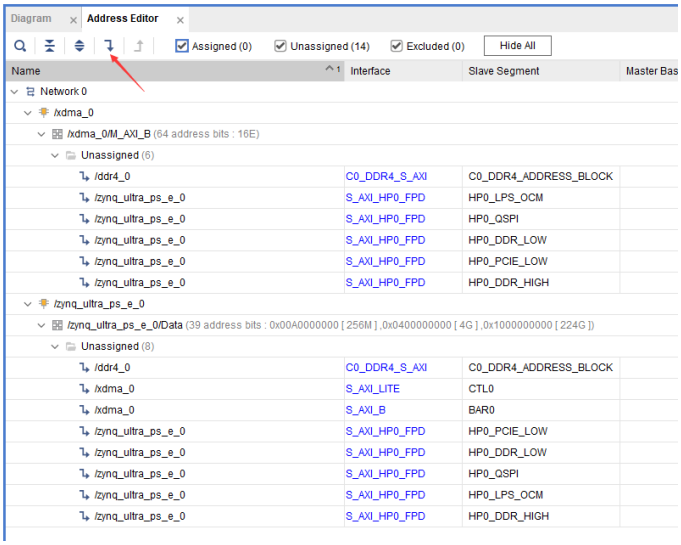
前面生成的 AXI Interconnect 增加一个 Master 接口连接到新建的 axi_interconnect_2 的 S01_AXI。并连接相应的时钟和复位，可参考提供的例程。



25. 在 Address Editor 中 Unassign All，不分配地址



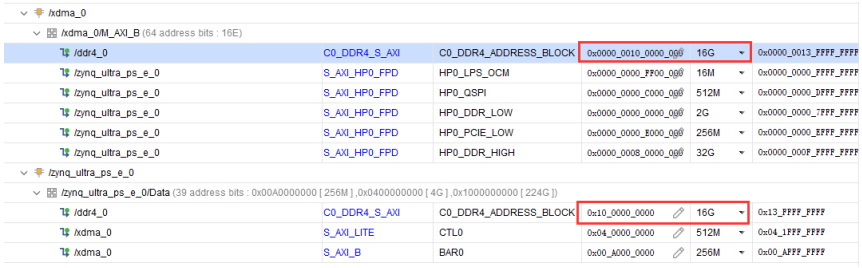
再点击 Assign All



将 BAR0 地址空间设置为 256M，这里的 0x00A0000000 就是在 XDMA IP 里设置的 BAR 地址。



将 XDMA 的 DDR 地址改成下面的一致，空间改为 8G



26. 保存设计，绑定引脚，并生成 bit 文件，之后导出 xsa 文件

